



Algoritmi e Strutture di Dati

Definizioni di base

M. Patrignani

Nota di copyright

- Queste slides sono protette dalle leggi sul copyright
- Il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- Le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- Ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- Gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- Questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Parlano di noi (1)



Francesco Antinucci

L'algoritmo al potere

Vita quotidiana ai tempi di Google

Edizione: 2009

Collana: i Robinson / Letture

ISBN: 9788842090533

Argomenti: **Attualità culturale e di costume**, **Informatica**

Pagine 124

14,00 11,20 Euro

Acquista

[embed](#) | [condividi](#)

[➔ In breve](#)

[📖 Indice](#)

[📝 Recensioni \(4\)](#)

In breve

Rendere accessibile a chiunque tutto ciò che è scritto, parlato, ritratto, filmato. Decretare la validità delle conoscenze con un voto di maggioranza. Scegliere le cure in base alla popolarità. Ritornare con l'intero pianeta alla condizione arcaica del villaggio, dove i comportamenti di ognuno erano sotto gli occhi di tutti. Questo è il futuro che ci annunciano Google, Wikipedia, YouTube, Facebook & Co. Forse.

Parlano di noi (2)



Rai News

CULTURA

Intervista a Michele Mezza, autore di 'Giornalismi nella rete'



Tutto il potere emana dall'algoritmo

Viviamo nel tempo del dominio dell'algoritmo. In circa dieci anni ci siamo quasi 'consegnati' nelle mani dei colossi che, grazie alla loro potenza tecnologica, hanno anche il potere di rendere giornalisti e pubblico 'sudditi' del calcolo matematico. Come non soccombere alle scelte di Google, Facebook - che con i loro suggerimenti, link, like, riconoscimenti facciali e così via si sostituiscono alla nostra iniziativa - è l'oggetto del libro di Michele Mezza 'Giornalismi nella rete'.

Parlano di noi (3)



ABBONATI



Accedi



PRIMO
PIANO

EDITORIALE

COMMENTI

POLITICA

ECONOMIA

CRONACA

ITALIA

MONDO

CULTURA

SPORT

Lo strapotere dell'algoritmo ci cambia la vita

10 ANNI
DE ILFATTOQUOTIDIANO.IT



22 GIUGNO
10 ORE DI LIVE STREAMING

di Giovanni Valentini | 1 DICEMBRE 2018



“Il servizio pubblico televisivo, contrariamente a quanto si possa pensare, è destinato a svolgere un ruolo ancora più importante al cospetto di una platea di fruitori altamente frammentata e polarizzata di cui occorre, almeno in una certa misura, ricostruire la coesione” (da Is competition a click away? di Stefano Mannoni e Guido Stazi - Editoriale

Parlano di noi (4)

2020, 07:24

QELSI

Algocrazia, ovvero: il dominio dell'algoritmo



Il termine “*algocrazia*” è stato coniato per definire il crescente dominio degli impenetrabili algoritmi sulla nostra vita, si tratta di quello che in molti ormai percepiamo essere il controllo sui nostri dati esercitato dai titani del web, controllo che sta garantendo loro poteri sovrastanti a quelli degli Stati nazionali.

Parlano di noi (5)

IL GIORNO

CRONACA **SPORT** **COSA FARE** **EDIZIONI** ▾

CORONAVIRUS

PROTESTE A MILANO

SPARA AL VICINO

DISPERSO NELL'ADDA



HOME › **COMMENTO**

Pubblicato il 5 giugno 2018

Il dominio dell'algoritmo

Ci vuole un nuovo umanesimo digitale che possa garantire in Rete il rispetto delle libertà fondamentali

di RUBEN RAZZANTE

Condividi

Tweet

Invia tramite email

Milano, 5 giugno 2018 - Qualcuno l'ha efficacemente ribattezzata "algocrazia". Trattasi del dominio di un imperscrutabile algoritmo sulle nostre vite, del controllo sui nostri dati esercitato dai giganti del web, che stanno acquisendo poteri superiori a quelli degli Stati nazionali.

POTREBBE INTERESSARTI ANCHE

IL GIORNO

Parlano di noi (6)



Openlabs

Benvenuti a OpenLabs ▼ Eventi ▼ Progetto Scuole

IL LATO OSCURO DEGLI ALGORITMI

IL LATO OSCURO DEGLI ALGORITMI E DEI LORO PADRONI

di Francesco Vitali Gentilini (pubblicato su limes 02/2017)

L'uso spregiudicato di specifici modelli matematici ha cambiato il destino europeo e americano con l'arrivo di Brexit e Trump.

Definizioni di base

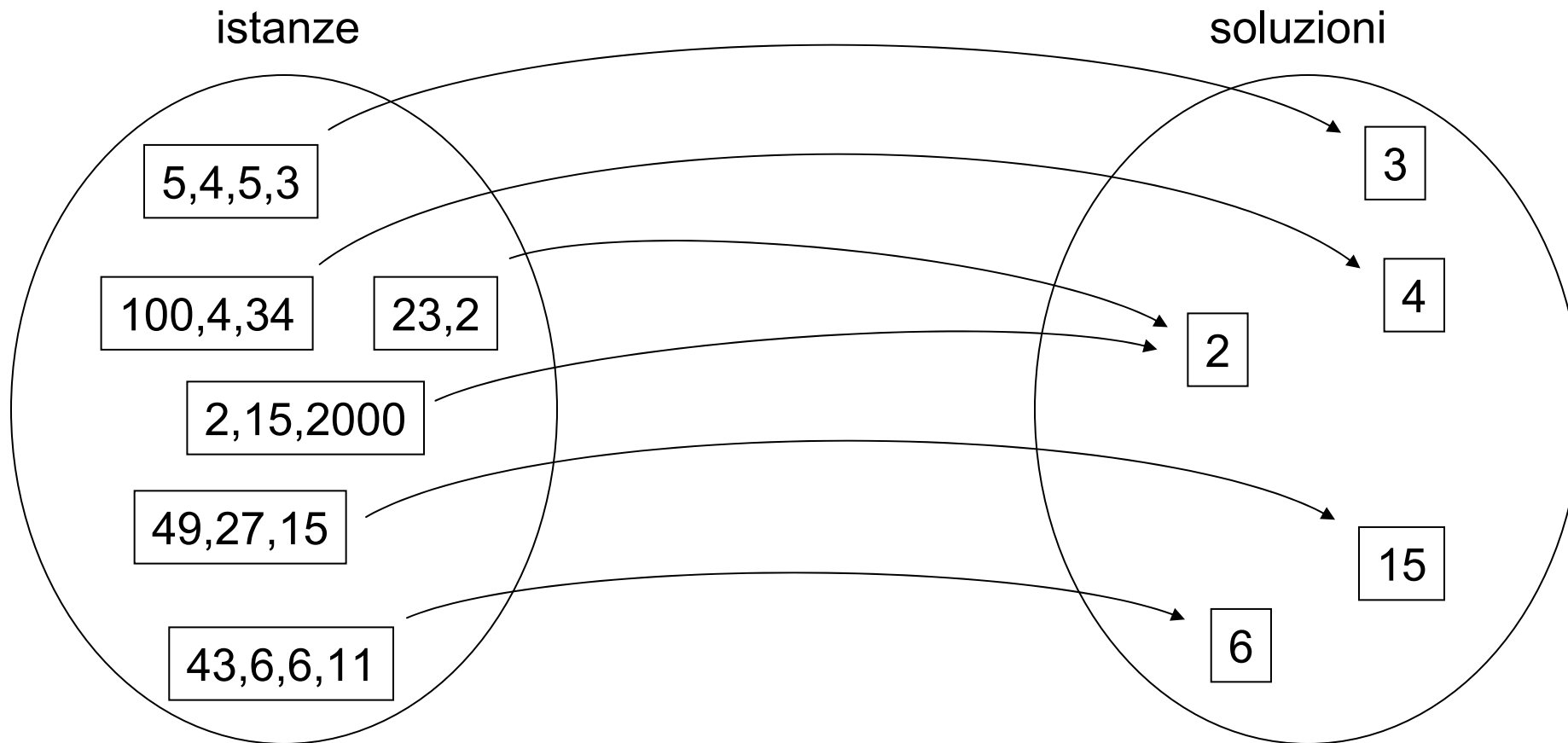
- Problemi computazionali
- Algoritmi
 - Random Access Machine (RAM)
 - pseudocodifica
- Strutture di dati

Problemi

- Un *problema computazionale* esprime la relazione tra due insiemi di valori, detti rispettivamente di *input* e di *output*
 - un insieme legittimo di valori di input è chiamato *istanza*, e i vincoli che rispetta sono descritti in termini assoluti
 - esempio: un insieme di n interi positivi $\{a_1, a_2, \dots, a_n\}$
 - un insieme legittimo di valori di output è chiamato *soluzione* ed è descritto in termini dell'istanza
 - esempio: il valore minore tra $a_1, a_2, \dots, a_n$

Esempio di problema computazionale

minimo di un insieme di interi positivi



Esempi di problemi computazionali

- Ordinamento
 - input: una sequenza di numeri
 - output: la stessa sequenza ordinata
- Ricerca
 - input: una collezione di dati ed una chiave k
 - output: il dato associato alla chiave k
- Percorsi ottimali
 - input: un grafo pesato e due nodi u e v
 - output: il cammino di costo minimo tra u e v
- Prodotto di matrici
 - input: n matrici $A_1, A_2, \dots, A_n$
 - output: il loro prodotto $A_1 A_2 A_3 \dots A_n$
- Involucro convesso
 - input: n punti sul piano
 - output: il più piccolo poligono convesso che li contiene
- ...

Importanza dei problemi computazionali

- I problemi computazionali sono il cuore di molte applicazioni
 - motori di ricerca
 - instradamento del traffico in Internet
 - computer aided design
 - computer audio
 - computer grafica
 - simulatori ed emulatori
 - giochi
 - crittografia
 - bioinformatica (sequenziamento, ecc)
 - compressione di dati

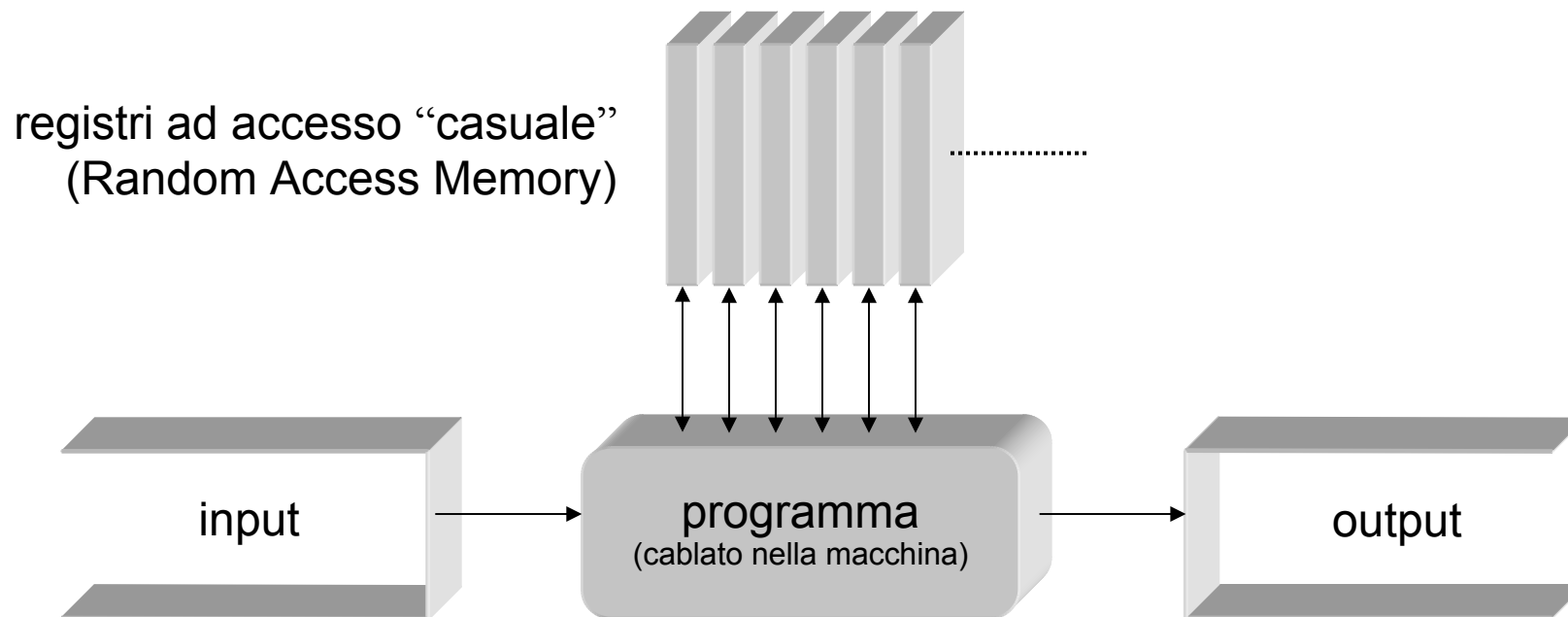
Definizione di algoritmo

- Un *algoritmo* è una procedura di calcolo ben definita che, a partire da un insieme di valori in input, produce valori in output
 - questa definizione sarà più rigorosa quando avremo definito cos'è una “procedura di calcolo ben definita”
- Un algoritmo è detto *corretto* per un problema computazionale p se per ogni istanza x di p
 1. l'algoritmo termina
 2. l'algoritmo produce un output y corretto
 - cioè tra x e y vale la relazione specificata da p

Random Access Machine

Che vuol dire “procedura di calcolo ben definita”?

- Risposta rigorosa
 - computabile da una Random Access Machine
 - astrazione ragionevole di un calcolatore
 - più elementare del modello di von Neumann



RAM

- Caratteristiche della Random Access Machine
 - memoria costituita da un numero arbitrario di registri indirizzabili individualmente (Random Access Memory)
 - input e output sono sequenze di dati (streams o nastri)
 - ogni operazione (trasferimento, somma, sottrazione, moltiplicazione, divisione) ha costo unitario
 - le istruzioni sono eseguite in maniera sequenziale
- Limitazioni
 - i valori in input, output e memoria RAM sono supposti illimitati
 - questa ipotesi è talvolta irragionevole
 - la RAM modella solo macchine mono-processore

Algoritmi e Random Access Machines

- Specificare una RAM che implementi un dato algoritmo è molto oneroso
 - le operazioni di base della RAM costituiscono un linguaggio macchina elementare
- Esempio di RAM (somma di due numeri)

READ R1

→ legge un dato di input e lo copia in R1

READ R2

→ legge un dato di input e lo copia in R2

LOAD R1

→ copia il registro R1 nell'accumulatore

ADD R2

→ somma l'accumulatore con R2

STORE R3

→ copia l'accumulatore in R3

WRITE R3

→ scrive il contenuto di R3 in output

Pseudocodice

che vuol dire “procedura di calcolo ben definita”?

- Risposta pragmatica (ma equivalente alla precedente)
 - descrivibile con uno pseudocodice elementare
 - che dispone di variabili, array, procedure, istruzioni condizionali, istruzioni ripetitive, ecc
 - esempio di pseudocodice

CALCOLO-MINIMO (A)

```
1. min = A[0]
2. for i = 1 to A.length-1
3.     if A[i] < min then
4.         ▷ agguirno il valore temporaneo di min
5.         min = A[i]
6.     return min
```

Strutture di dati

- I dati su cui opera l'algoritmo sono conservati in un insieme di variabili atomiche, oggetti ed array
- Scopo
 - salvataggio temporaneo di risultati parziali
 - preprocessing sull'input per rendere più facile la loro elaborazione successiva
- I dati in memoria possono essere organizzati secondo diverse strategie o discipline
 - esempio: un array può essere ordinato
- Tali discipline sono chiamate *strutture di dati*

Strutture di dati

- Una *struttura di dati* è un contenitore di dati
 - è concepita per rendere il dato disponibile nella forma più utile e veloce per produrre l'output
 - il dato contenuto nella struttura di dati è a tutti gli effetti un semi-lavorato
 - esempio
 - supponiamo che un insieme di numeri sia conservato in un array ordinato
 - l'elemento minimo dell'insieme sarà il primo elemento dell'array



Algoritmi e Strutture di Dati

Pseudocodifica

M. Patrignani

Nota di copyright

- Queste slides sono protette dalle leggi sul copyright
- Il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- Le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- Ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- Gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- Questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Che cos'è la pseudocodifica

- La pseudocodifica è un linguaggio
 - non è né compilato né interpretato, in quanto non deve essere eseguito su nessuna piattaforma
- La pseudocodifica serve ai programmatori per descrivere un algoritmo
 - la sintassi non è rigorosa, in quanto l'unico requisito è che la semantica sia comprensibile
 - si suppone che un algoritmo descritto tramite pseudocodice possa essere implementato in qualsiasi linguaggio di programmazione

Restrizioni della pseudocodifica

- Vista la sua universalità, la pseudocodifica presenta alcune restrizioni
 - il tipo delle variabili è fisso e non è modificabile a run-time
 - non è possibile eseguire casting implicito od esplicito
 - esistono i riferimenti, ma non è supportata l'algebra dei puntatori
 - non esistono gli operatori * e &

Variabili e assegnazioni

- Le variabili dello pseudocodice hanno associato un tipo (intero, reale, booleano, ...) ma non necessitano di dichiarazione
 - la prima volta che vengono usate si inferisce il loro tipo
 - in caso di equivoco si ricorre a dei commenti
- Le assegnazioni sono denotate da un segno di uguaglianza (=)

```
1.      max = -1           // max è un intero
2. max = max + 1          // incremento max
```

```
1. /* definisco un reale che chiamo area */
2. area = base * altezza / 2
```

Semantica delle assegnazioni

- Un'assegnazione è un'operazione

`<left-value> = <right-value>`

individua una zona di memoria che ha un determinato tipo (e quindi una specifica dimensione)

è una generica espressione che produce un valore del tipo coerente con il left-value

- Queste due assegnazioni sono diverse

`a = b     // nella cella di mem di a metto il valore di b`

`b = a     // nella cella di mem di b metto il valore di a`

Esempi di assegnazioni

- `<variabile> = <costante>`

```
1.    verificato = false
```

- `<variabile> = <variabile>`

```
1.    a = b
```

- `<variabile> = <espressione generica>`

```
1.    verificato = verificato and FUNZIONE(a, b, c)
```

- Uso scorretto delle assegnazioni

```
1.  a = b + c
```

```
2.  b = 3
```

```
3.  c = 4
```

```
4.  return a
```


Istruzioni condizionali

- **if-then-else**, con ovvio significato
 - spesso per brevità si tace il “then”
- Un blocco di istruzioni è denotato tramite indentazione

```
1. if i < 0 then  
2.    // prendo il valore assoluto di i  
3.    i = -i
```

```
1. if i < 0 then  
2.    abs = -i  
3.    i_minore_zero = true  
4. else  
5.    abs = i  
6.    i_minore_zero = false
```

Espressioni booleane

- Sono espressioni booleane
 - le variabili booleane e le costanti **true** e **false**
 - il risultato di operatori booleani come: **and**, **or**, **not**
 - possono anche essere scritti **&&**, **||**, **!**
 - il risultato di operatori relazionali come: **==**, **>**, **≥**, **<**, **≤**

```
1. if i < max and trovato == false then  
2.     i = i+1
```

- Attenzione
 - talvolta viene usato il simbolo uguale (=) per denotare l'operatore relazionale e la freccia (←) per l'assegnazione
 - è però indispensabile utilizzare i simboli in modo coerente!

Leggi di de Morgan

- Quando un not si distribuisce su un and quest'ultimo viene trasformato in or
- Le seguenti espressioni sono dunque equivalenti
 - not (condizione-1 and condizione-2)
 - esempio: spero che non piovano e io sia all'aperto
 - (not condizione-1) or (not condizione-2)
 - esempio: spero che non piova oppure che io non sia all'aperto
- Quando un not si distribuisce su un or quest'ultimo viene trasformato in and
- Le seguenti espressioni sono dunque equivalenti
 - not (condizione-1 or condizione-2)
 - esempio: spero che non piovano o nevichi
 - (not condizione-1) and (not condizione-2)
 - esempio: spero che non piova e che non nevichi

Istruzione ripetitiva for

- Esempi

```
1. for i = 0 to A.length-1
2.    /* pongo a zero tutte le celle di A */
3.    A[i] = 0
```

```
1. for i = A.length-1 down to 1
2.    /* traslo in avanti tutti i valori di A */
3.    A[i] = A[i-1]
4. A[0] = new    // inserisco new in testa
```

- Non è possibile porre condizioni aggiuntive ad un for
 - esempio errato

```
1. for i = 0 to A.length-1 and trovato == false
```

Istruzione ripetitiva `while`

- Il blocco viene eseguito finché (=mentre) la condizione è verificata
 - la condizione è una “condizione di permanenza nel ciclo”
 - se la condizione è falsa il blocco non viene eseguito mai
 - spesso per brevità si tace il “do”

```
1. // pongo a zero tutte le posizioni di A
2. i = 0
3. while i < A.length do
4.     A[i] = 0
5.     i = i+1
```

```
1. while i < A.length and not trovato do
2.     if A[i] == quello_che_cerco then
3.         trovato = true
4.     else
5.         i = i+1
```

Istruzione ripetitiva **repeat until**

- Il blocco viene eseguito finché (=mentre) la condizione è falsa
 - la condizione è una “condizione di uscita dal ciclo”
 - il blocco viene eseguito almeno la prima volta
- Esempio

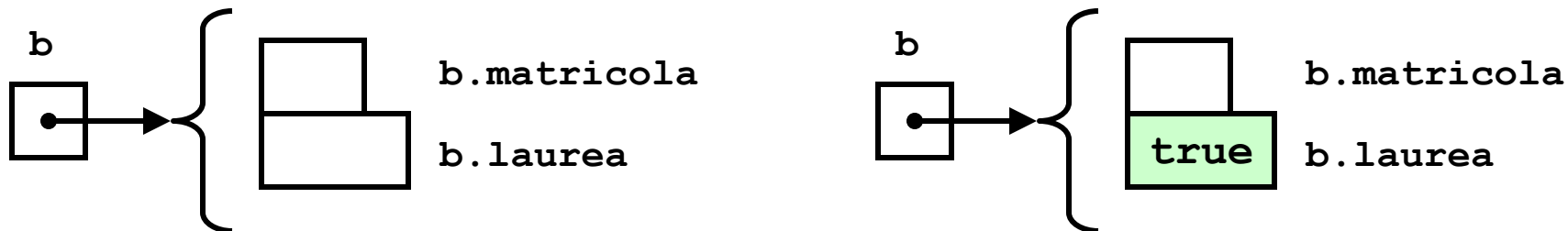
```
1. // pongo a zero tutte le posizioni di A
2. i = 0
3. repeat
4.     A[i] = 0
5.     i = i+1
6. until i == A.length
```

Pseudocodice: oggetti

- E' possibile definire dati compositi (oggetti) strutturati in attributi
- Si accede all'attributo **campo** dell'oggetto **x** tramite **x.campo**

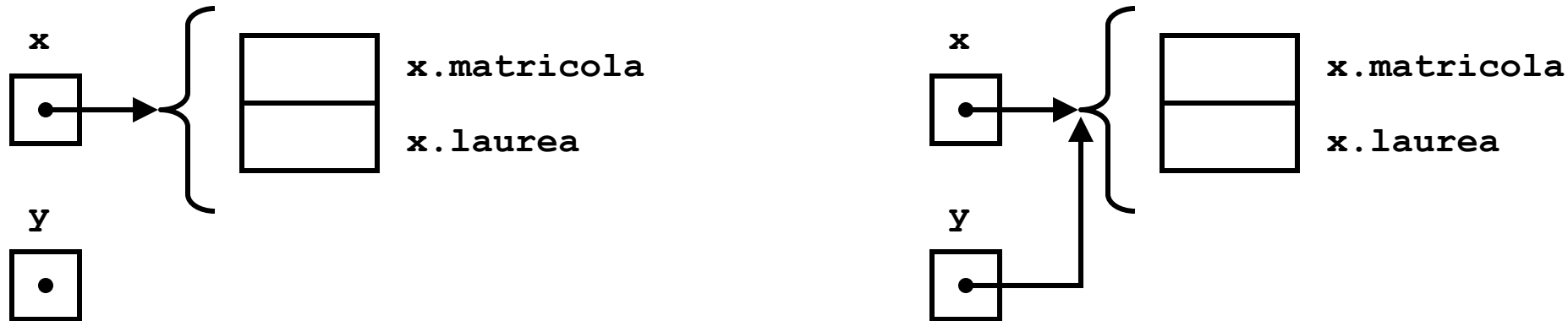
```
1. // b è un oggetto con gli attributi  
2. // matricola (intero) e laurea (booleano)  
3. b.laurea = true           // con mille auguri!
```

- Una variabile che rappresenta un oggetto è trattata come un riferimento (puntatore) all'oggetto



Oggetti e operazioni sui riferimenti

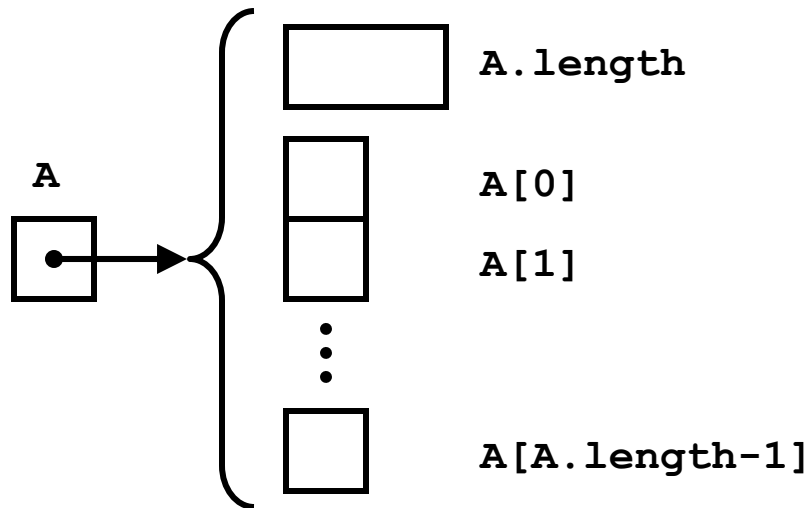
- Se **x** e **y** sono due (riferimenti ad) oggetti
 - dopo l'assegnazione **y = x** si ha **y.campo = x.campo**
 - se si modifica un attributo di **x** (**x.campo = 3**), si modifica anche il corrispondente attributo di **y** (**y.campo = 3**)



- Per porre a zero un riferimento si usa la costante **NULL**
- **ATTENZIONE:**
 - in linguaggio C si fa distinzione tra un oggetto ed il suo riferimento
 - in pseudocodice abbiamo solo il riferimento

Pseudocodice: array

- Si possono definire array di qualunque tipo
- $A[i]$ identifica l'elemento dell'array A in posizione i
- La lunghezza dell'array A è data da $A.length$
 - $A.length$ è un campo, non una funzione
 - **ATTENZIONE:** non c'è equivalente di $A.length$ in linguaggio C
- Le posizioni dell'array A vanno da 0 ad $A.length-1$



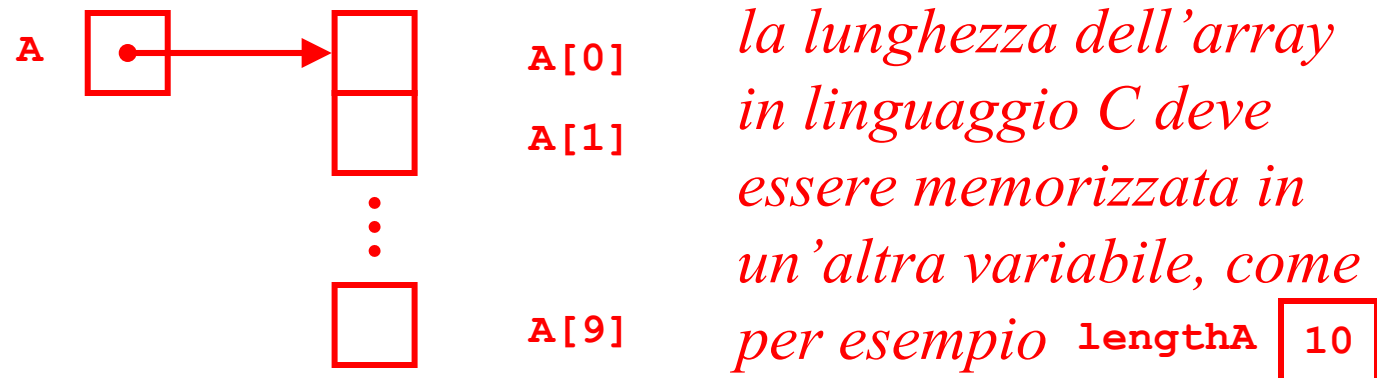
di fatto, un array è visto come un oggetto che ha alcuni campi indicizzati tramite un intero

Esempi di uso di array

```
1. A[0] = 1           // A è un array di 10 interi
2. A[A.length-1] = 0 // l'ultimo elemento è zero
```

```
1. // A è un array contenente 3 array di 10 interi
2. A[2][9] = 0      // l'ultimo elemento di A[2] è 0
3. A[2,9] = 0       // equivalente alla precedente
```

- **ATTENZIONE:** in linguaggio C gli array non hanno il campo length!!



Pseudocodice e tipi delle variabili

- Lo pseudocodice è privo di definizioni di tipo
- Le variabili si suppongono allocate in memoria nel momento in cui vengono menzionate per la prima volta
- I commenti vengono utilizzati per esplicitare il loro tipo, se ciò non è evidente dal contesto (come nelle linee 2 e 5 qui sotto)
 - ciò è evidentemente necessario per gli oggetti

1. $i = p - 1$	<i>(in assenza di commento il tipo di i è lo stesso di p)</i>
2. $x.info = 1$	<i>/* x è un oggetto con i campi: info</i>
3.	<i>(intero) next e prev (riferimenti ad</i>
4.	<i>oggetti dello stesso tipo) */</i>
5. $TEMP[1] = 3$	<i>// TEMP è un array di 20 interi</i>
6. $y = x$	<i>(il tipo di y è lo stesso di x: commento non necessario)</i>
7. $TEMP[2] = 4$	<i>(TEMP già noto: commento non necessario)</i>

Pseudocodice e procedure

- Le variabili utilizzate in una procedura sono sempre locali alla procedura stessa
 - non esistono variabili globali
- Alcune procedure non ritornano alcun valore
 - la sequenza delle loro istruzioni termina, oppure viene eseguita l'istruzione **return**
- Alcune procedure ritornano un valore
 - le loro istruzioni terminano sempre con **return** *<espressione>*
 - dove la valutazione di *<espressione>* genera il valore ritornato
 - non si possono ritornare due o più valori
 - eventualmente si usano degli oggetti costruiti opportunamente

Procedure e parametri

- Il passaggio dei parametri ad una procedura è solo per valore
 - la procedura riceve il valore dei *parametri formali* tramite i *parametri attuali* presenti nella chiamata
 - un parametro attuale è in generale un'espressione, che può anche ridursi ad una singola variabile
- Un parametro è a tutti gli effetti una variabile locale inizializzata con il valore ricevuto al momento della chiamata
- se il valore di un parametro formale viene modificato, la modifica non ha effetto all'esterno sul parametro attuale
 - non c'è "side effect"
- Se però il parametro è un oggetto o un'array, l'oggetto non viene riprodotto, ma viene passato il valore del riferimento (indirizzo, puntatore) all'oggetto/array stesso
 - le assegnazioni `oggetto.campo=valore` e `array[indice]=valore` hanno quindi un side effect sull'oggetto e sull'array passati come parametro

Esercizi sullo pseudocodice

1. Scrivi la procedura `MASSIMO(a,b)` che ritorni il massimo tra due interi a e b
2. Scrivi la procedura `MASSIMO(A)` che riceva come parametro un array di interi A e ritorni il massimo dei valori contenuti
3. Scrivi la procedura `SOMMA(M)` che riceva come parametro una matrice (un array di array) di interi M e ritorni la somma dei valori contenuti
 - puoi usare una funzione `SOMMA(A)` che somma gli elementi di un array per realizzare `SOMMA(M)`

Esercizi sullo pseudocodice

4. Scrivi la procedura `POSITIVO(A)` che riceva come parametro un array di interi `A` e ritorni `true` se `A` contiene solo valori maggiori di zero, `false` altrimenti
5. Scrivi la procedura `MASSIMO_E_MINIMO(A)` che riceva come parametro un array di interi `A` e ritorni il valore del massimo e del minimo contenuto nell'array
 - attenzione: non si possono ritornare due valori!
 - occorre ritornare un oggetto con due campi

Algoritmi e Strutture di Dati

Esercitazioni in linguaggio C

Compilazione in linguaggio C

M. Patrignani

Nota di copyright

- Queste slides sono protette dalle leggi sul copyright
- Il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- Le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- Ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- Gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- Questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Richiami sul linguaggio C

- Compilazione
- Linking
- Suddivisione del codice in più file
- Dichiarazioni e definizioni
- Compilazione secondo lo standard ANSI

Compilazione in linguaggio C

- La compilazione traduce il codice sorgente nel codice oggetto (cioè compilato)
 - usualmente il codice sorgente è in uno o più file con estensione .c
 - il codice oggetto viene messo in file con estensione .o



Compilazione da linea di comando

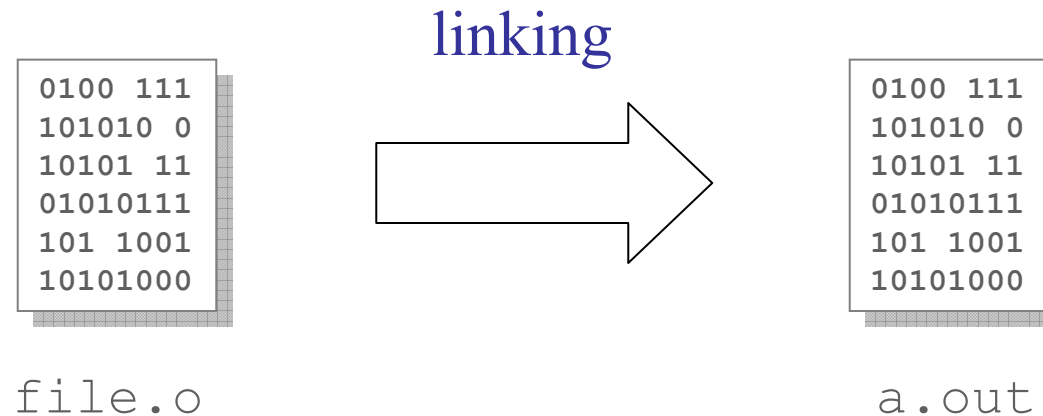
- Avviene chiamando un compilatore (cc, gcc, ecc) specificando che si vuole compilare un file sorgente

```
localhost:~$ ls
file.c
localhost:~$ cc -c file.c
localhost:~$ ls
file.c file.o
```

- l'opzione `-c` significa “voglio solo la compilazione”

Linking in linguaggio C

- La fase di linking trasforma il codice oggetto in un file eseguibile
 - vengono anche utilizzate (linkate, appunto) le librerie standard del linguaggio C



Linking da linea di comando

- Avviene chiamando un compilatore (cc, gcc, ecc) e fornendo come parametro il file oggetto

```
localhost:~$ ls
file.c file.o
localhost:~$ cc file.o -o esegui
localhost:~$ ls
esegui file.c file.o
```

- Se non viene specificato il nome del file di output (`-o esegui`) allora viene generato un file eseguibile di nome `a.out`

Un esempio di errore di linking

- Il file linkato deve contenere la funzione `main`

```
localhost:~$ cc file.o -o esegui  
/usr/lib/gcc/x86_64-linux-  
gnu/4.9/../../../../x86_64-linux-gnu/crt1.o: In  
function `__start':  
(.text+0x20): undefined reference to `main'  
collect2: error: ld returned 1 exit status  
localhost:~$
```

- in realtà il messaggio d'errore dice che la funzione standard `__start` non ha trovato la funzione `main`

Compilazione e linking

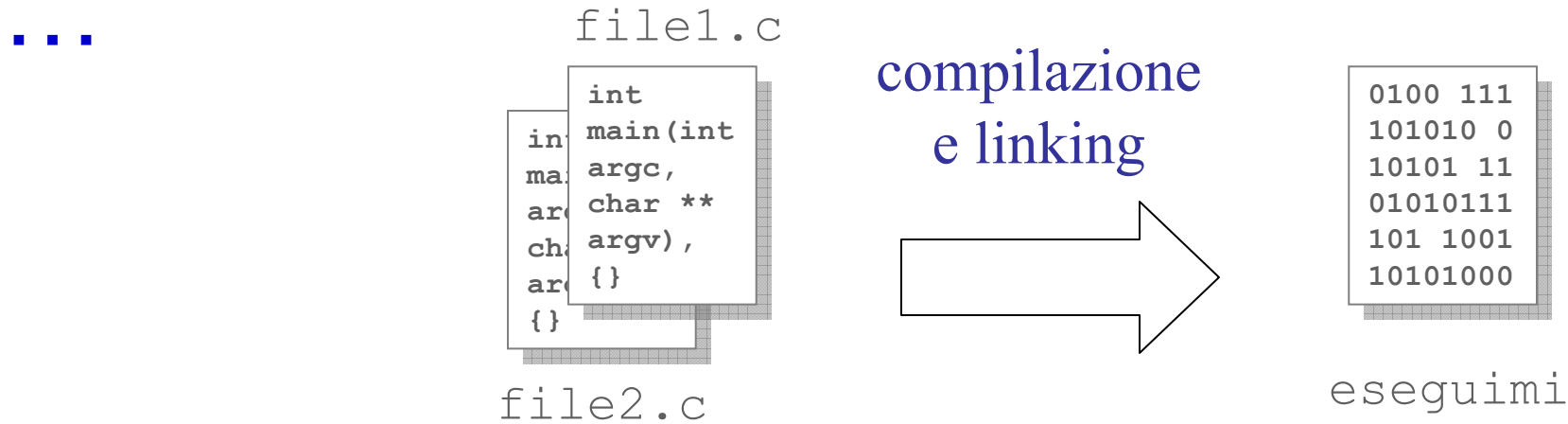
- Si può compilare e linkare con un solo comando

```
localhost:~$ ls
prova.c
localhost:~$ cc prova.c -o prova
localhost:~$ ls
prova prova.c
```

- Il file `prova.o` non viene nemmeno conservato
- In un Integrated Development Environment (IDE) la compilazione e il linking avvengono automaticamente al “build” del progetto

Codice distribuito in file diversi

- Quando il codice viene distribuito in diversi file



... la compilazione e il linking generano un singolo file eseguibile

- uno e uno solo dei file oggetto deve contenere la funzione **main**

Creazione del file eseguibile

- Per generare un file eseguibile è necessario
 - 1.che la compilazione vada a buon fine e generi un file oggetto per ogni singolo file compilato
 - 2.che il linkaggio vada a buon fine
 - ogni funzione che viene utilizzata in un file oggetto deve essere contenuta in un (unico) altro file oggetto o libreria specificata nella fase di linking

Esempio di errore di compilazione

```
int main(int argc, char** argv) {  
    float a = 0.5;  
    float b = sin(a);  
}
```

```
localhost:~$ cc file.c -o esegui  
file.c: In function 'main':  
file.c:3:13: warning: incompatible implicit  
declaration of built-in function 'sin'  
    float b = sin(a);  
                ^  
  
/tmp/cctqIk3Y.o: In function 'main':  
file.c:(.text+0x1e): undefined reference to `sin'  
collect2: error: ld returned 1 exit status  
localhost:~$
```

Esempio di errore di compilazione

```
#include <math.h> /* contiene sin */  
  
int main(int argc, char** argv) {  
    float a = 0.5;  
    float b = sin(a);  
}
```

```
localhost:~$ cc file.c -o esegui  
/tmp/cc6y9GHF.o: In function `main':  
file.c:(.text+0x1e): undefined reference to `sin'  
collect2: error: ld returned 1 exit status  
localhost:~$
```

- Anche includendo il file `math.h` l'eseguibile non viene generato

Esempio di errore di compilazione

```
#include <math.h> /* contiene sin */  
  
int main(int argc, char** argv) {  
    float a = 0.5;  
    float b = sin(a);  
}
```

```
localhost:~$ cc -c file.c  
localhost:~$ ls  
file.c file.o  
localhost:~$ ls
```

- La compilazione, in realtà, va a buon fine

Esempio di errore di compilazione

```
#include <math.h> /* contiene sin */  
  
int main(int argc, char** argv) {  
    float a = 0.5;  
    float b = sin(a);  
}
```

```
localhost:~$ cc file.o -o esegui  
file.o: In function `main':  
file.c:(.text+0x1e): undefined reference to `sin'  
collect2: error: ld returned 1 exit status  
localhost:~$
```

- L'errore è nella fase di linking

Correzione dell'errore

```
#include <math.h> /* contiene sin */  
  
int main(int argc, char** argv) {  
    float a = 0.5;  
    float b = sin(a);  
}
```

```
localhost:~$ cc file.o -lm -o esegui  
localhost:~$ ls  
esegui file.c file.o  
localhost:~$
```

- Il parametro `-lm` richiede di linkare la libreria “m” (si tratta della libreria “`libm.a`”)

Compilazione e linkaggio corretti

```
#include <math.h> /* contiene sin */  
  
int main(int argc, char** argv) {  
    float a = 0.5;  
    float b = sin(a);  
}
```

```
localhost:~$ ls  
file.c  
localhost:~$ cc file.c -lm -o esegui  
localhost:~$ ls  
esegui file.c  
localhost:~$
```


Compilazione ad una sola passata

- La sintassi del linguaggio C prevede che il codice possa essere compilato con una sola passata
- tipi, variabili e funzioni, devono essere dichiarati prima di essere usati
 - attenzione: devono essere **dichiarati**, non necessariamente **definiti**
 - molte volte ciò può essere garantito scegliendo un opportuno ordine per le definizioni

Dichiarazioni e definizioni

- **variabile o funzione**

- la dichiarazione non genera codice nel file .o

- serve solo ad informare il compilatore che il costrutto esiste

- la definizione genera codice nel file .o

- la zona di memoria dove risiede la variabile o la sequenza di istruzioni che realizza la funzione

- **tipo**

- la dichiarazione non genera codice nel file .o

- informa il compilatore che esiste un tipo, ma non dice come è fatto

- la definizione non genera codice nel file .o

- informa il compilatore come è fatto un tipo, anche se ancora non c'è nessuna variabile di quel tipo

Variabili o funzioni

■ Dichiarazione di variabile o funzione

- è un costrutto che annuncia al compilatore che la variabile o la funzione potrà essere utilizzata
- non introduce codice nel file oggetto e può essere iterata
 - purché sia coerente con le precedenti
 - la compilazione di un file con sole dichiarazioni produce un file oggetto vuoto (con la sola intestazione)

■ Definizione di variabile o funzione

- è un costrutto che descrive in dettaglio come è composta la variabile o la funzione che viene definita
- una definizione è anche implicitamente una dichiarazione
- introduce codice nel file oggetto e non può essere iterata

Esempi di dichiarazioni

- Dichiarazione di una funzione

```
extern float funzione(float parametro);
```

analoga a

```
float funzione(float parametro);
```

- Dichiarazione di una variabile

```
extern int variabile_globale;
```

- in questo caso la parola chiave **extern** è necessaria, altrimenti il compilatore equivoca la dichiarazione per una definizione

Dichiarazione e definizione di tipi

■ Dichiarazioni di tipi

- sono costrutti che annunciano al compilatore ciò che potrà essere utilizzato
- non introducono codice nel file oggetto e possono essere iterate
 - purché siano coerenti con le precedenti

■ Definizioni di tipi

- sono costrutti che descrivono in dettaglio come sono composti i tipi che vengono definiti
- le definizioni di tipi non introducono codice nel file oggetto e possono essere iterate
 - purché siano coerenti con le precedenti

Esempio di ordine di definizione errato

```
int main(int argc, char** argv) {  
    float a = doppio(2.5);  
}  
float doppio(float v) {  
    return v * 2.0;  
}
```

funzione ancora
non dichiarata

```
localhost:~$ cc -c file.c  
file.c:4:7: error: conflicting types for 'doppio'  
    float doppio(float v) {  
      ^  
file.c:2:13: note: previous implicit declaration of  
'doppio' was here  
    float a = doppio(2.5);  
              ^  
localhost:~$
```

Ordine di definizione corretto

```
float doppio(float v) {  
    return v * 2.0;  
}  
  
int main(int argc, char** argv) {  
    float a = doppio(2.5);  
}
```

```
localhost:~$ ls  
file.c  
localhost:~$ cc -c file.c  
localhost:~$ ls  
file.c file.o  
localhost:~$
```

Alternativamente: uso di una dichiarazione

```
float doppio(float v);
```

dichiarazione

```
int main(int argc, char** argv) {  
    float a = doppio(2.5);  
}
```

```
float doppio(float v) {  
    return v * 2.0;  
}
```

definizione

```
localhost:~$ ls  
file.c  
localhost:~$ cc -c file.c  
localhost:~$ ls  
file.c file.o  
localhost:~$
```


Uso obbligatorio di una dichiarazione

```
float funzione1(float v) {  
    if ( v > 10 ) return funzione2(v)-1;  
    return v;  
}  
  
float funzione2(float v) {  
    if ( v > 10 ) return 0.5*funzione1(v);  
    return v;  
}
```

- Non esiste un ordine di definizione delle due funzioni che metta al riparo da un errore di compilazione

Uso obbligatorio di una dichiarazione

```
float funzione2(float v);
```

dichiarazione

```
float funzione1(float v) {  
    if ( v > 10 ) return funzione2(v)-1;  
    return v;  
}
```

```
float funzione2(float v) {  
    if ( v > 10 ) return 0.5*funzione1(v);  
    return v;  
}
```

Dichiarazione di un tipo

- Poiché la definizione può essere iterata è più raro trovarsi nella necessità di dichiarare un tipo
- In alcuni casi, però, ciò è indispensabile

```
typedef struct str1 strutturale1;  
typedef struct str2 strutturale2;
```

```
typedef struct str1 {  
    strutturale2* puntatore;  
} strutturale1;
```

```
typedef struct str2 {  
    strutturale1* puntatore;  
} strutturale2;
```

forward
declarations

Localizzazione delle dichiarazioni

- Generalmente, tutte le dichiarazioni vengono messe in un file `.h` che può essere importato da altri file `.c`

```
#ifndef _NOME_DEL_FILE
#define _NOME_DEL_FILE

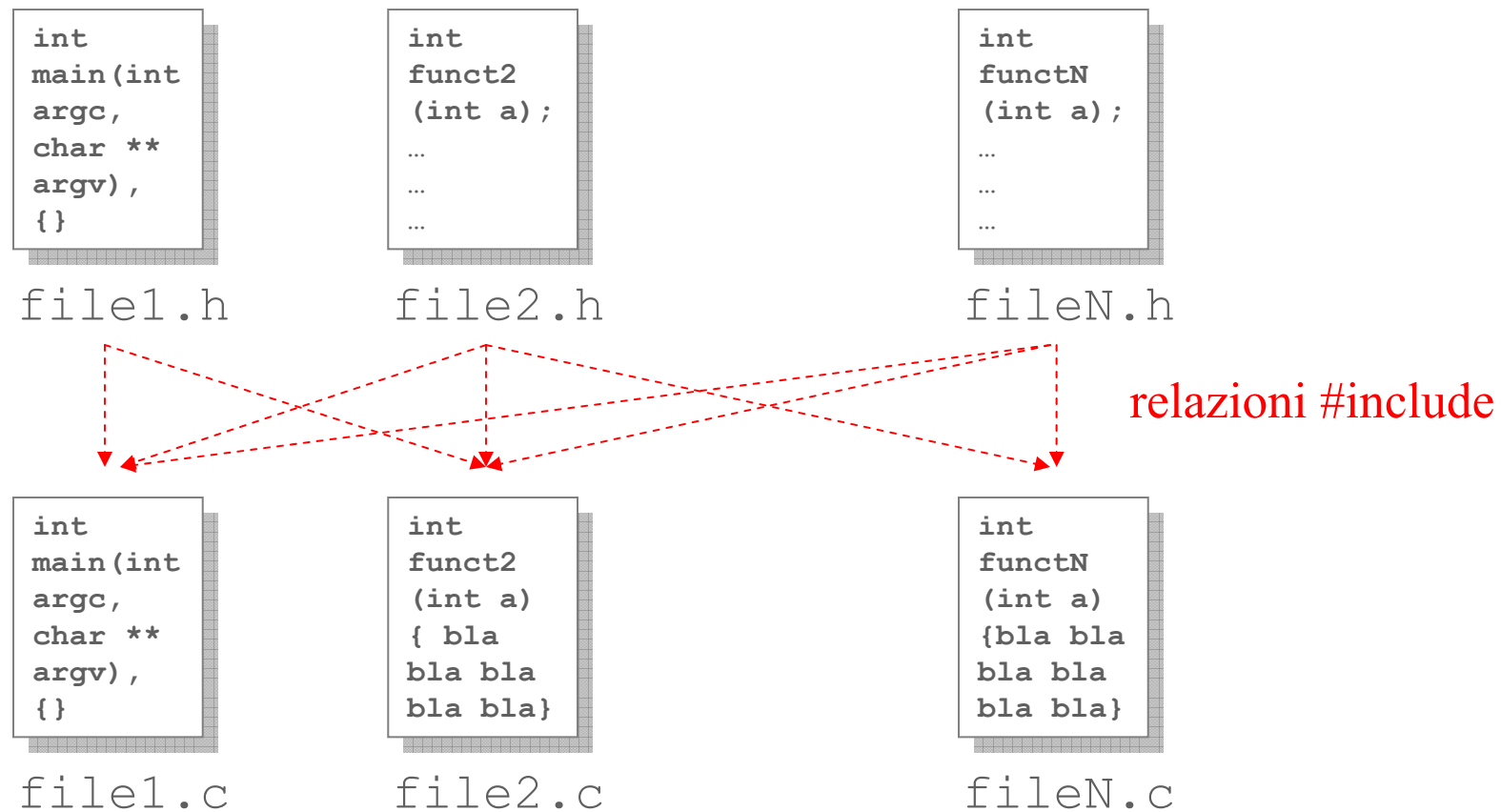
/* dichiarazioni varie */

#endif
```

- Le variabili e le funzioni dichiarate nel file `.h` devono essere contenute in un file `.o` (quindi definite nel corrispondente file `.c`) perché la fase di linking vada a buon fine

Distribuzione del codice nei file

- Ogni file .c contiene il codice relativo alle dichiarazioni del corrispondente .h



Variabili globali

- Le variabili globali
 - sono dichiarate e definite in un solo file .c (tipicamente il file principale)
 - sono dichiarate come “extern” in tutti i file .c che le utilizzano

```
int status = 0;

int main(int argc,
char ** argv){
...
...
...
}
```

file1.c

```
extern int status;

int functK (int a)
{
...
...
...
}
```

fileK.c

Compilazione secondo lo standard ANSI

- Lo standard ANSI C originale (X3.159-1989) fu ratificato nel 1989 e pubblicato nel 1990
 - per questo motivo è spesso chiamato C89 o C90
- Se si vuole che il compilatore si attenga strettamente allo standard ANSI originale occorre compilare con l'opzione **-pedantic-errors**

```
localhost:~$ cc file.c -pedantic-errors -o file
```

Compilazione secondo lo standard ANSI

```
int main(int argc, char** argv) {  
    int a = 10;    // commento  
    int b = a;  
}
```

```
localhost:~$ cc file.c -pedantic-errors -o file  
file.c: In function 'main':  
file.c:3:16: error: C++ style comments are not allowed  
in ISO C90  
    int a = 10;    // commento  
                  ^  
file.c:3:16: error: (this will be reported only once  
per input file)  
localhost:~$
```


Compilazione secondo lo standard ANSI

```
int main(int argc, char** argv) {  
    int a = 10, b = 5, c;  
    c = a + b;  
    char ch;  
}
```

```
localhost:~$ cc file.c -pedantic-errors -o file  
file.c: In function 'main':  
file.c:5:3: error: ISO C90 forbids mixed declarations  
and code [-Wpedantic]  
    char ch;  
    ^  
localhost:~$
```

Esercizi

1. Scrivi una funzione in linguaggio C che verifichi se in un array di interi ci sia almeno un intero ripetuto due volte
2. Scrivi una funzione in linguaggio C che verifichi se in un array di interi ci sia almeno un intero ripetuto **tre** volte

Algoritmi e Strutture di Dati

Notazione asintotica

M. Patrignani

Nota di copyright

- Queste slides sono protette dalle leggi sul copyright
- Il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- Le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- Ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- Gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- Questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Panoramica

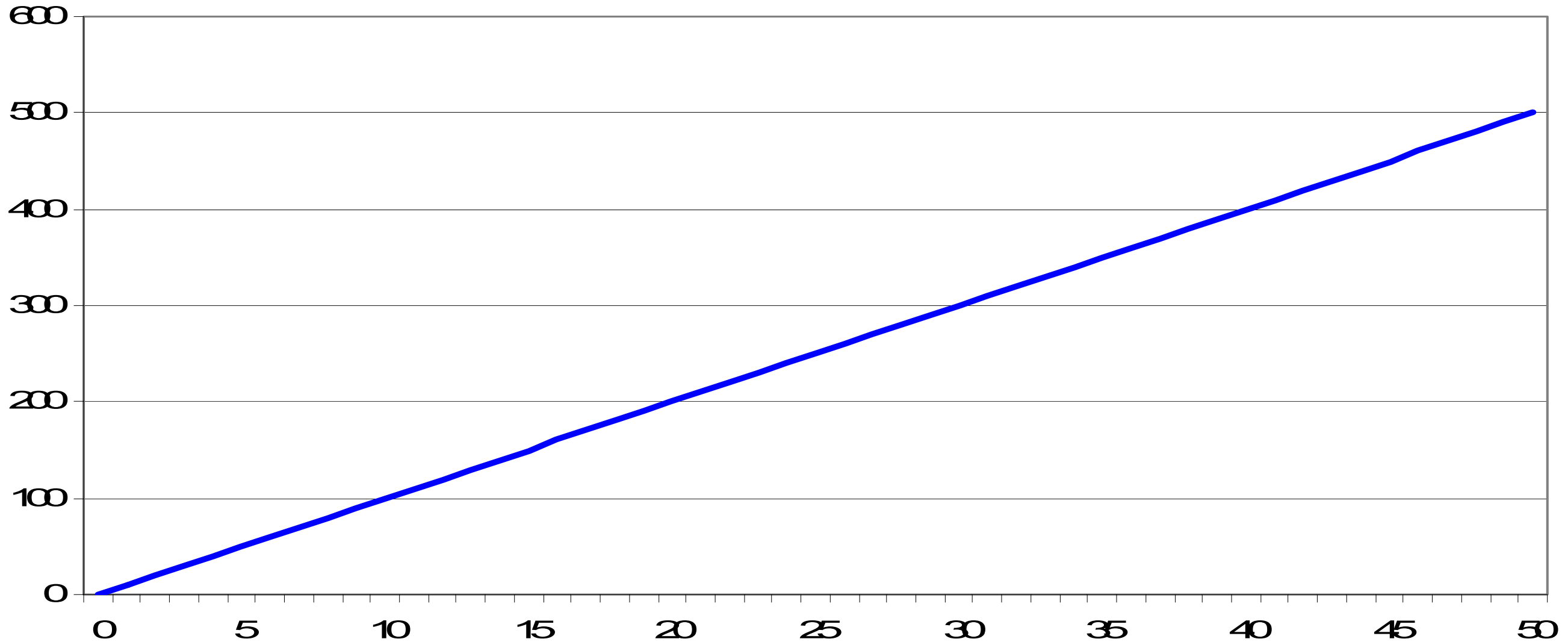
- Definizioni
- Proprietà delle notazioni asintotiche
- Uso esteso (o improprio) della notazione

Studio di funzioni

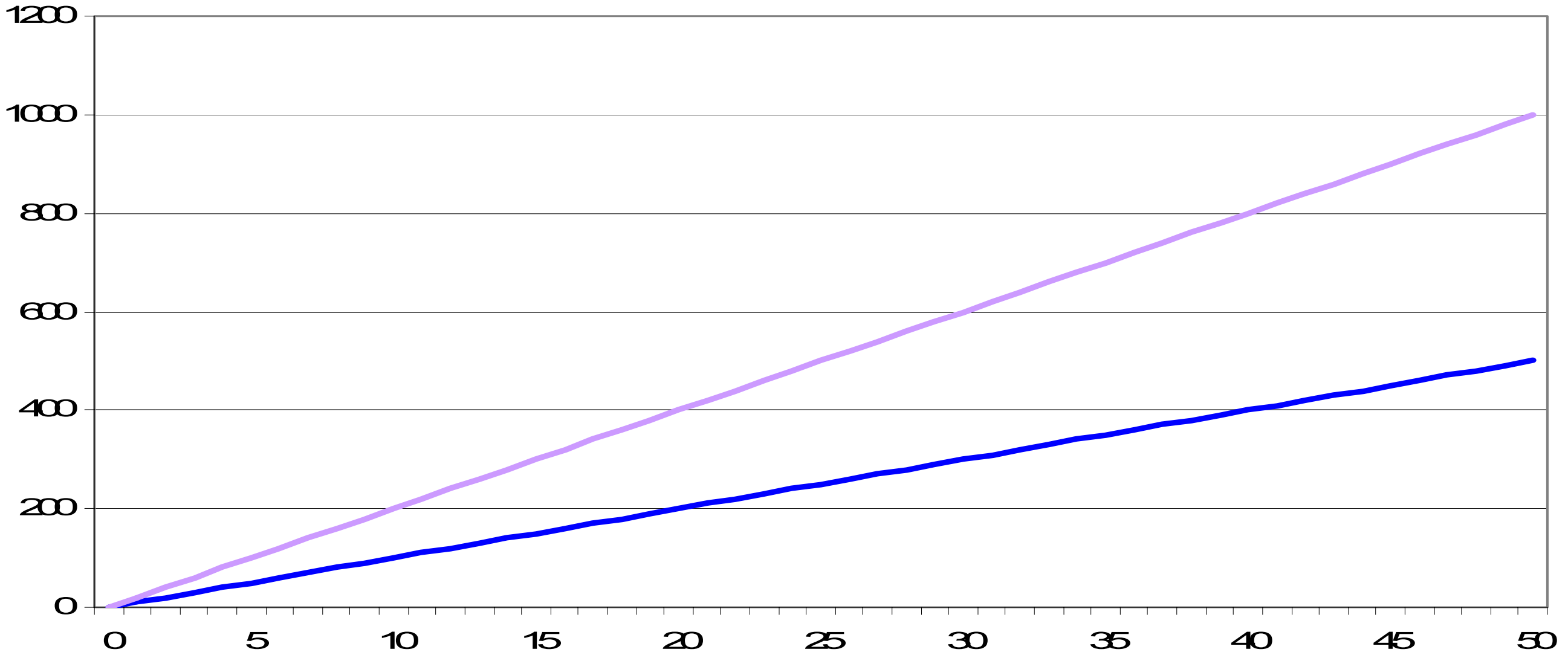
- Intersezioni con gli assi e segno
- Simmetrie e periodicità
- Continuità, discontinuità, derivazione
- Massimi, minimi e punti di flesso
- Comportamento agli estremi del dominio
 - asintoti orizzontali, verticali, obliqui
 - notazione asintotica



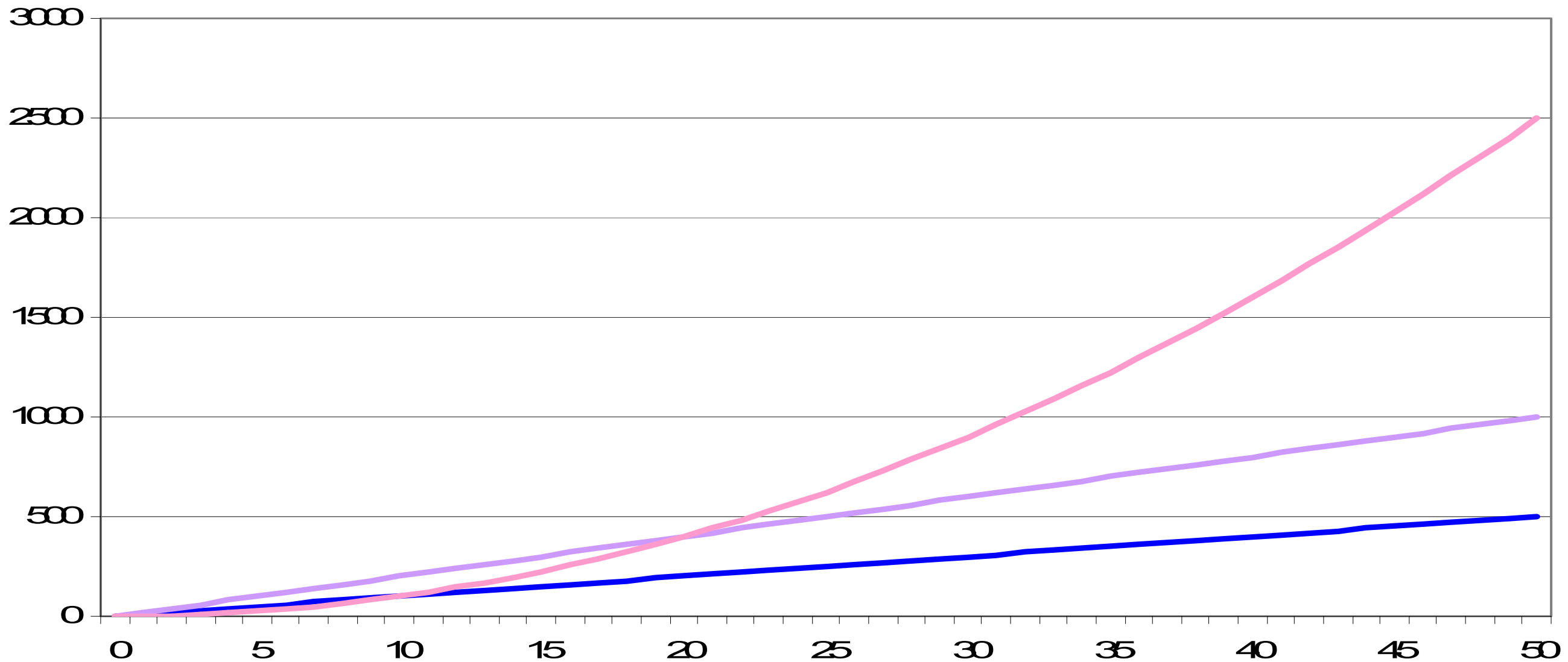
La funzione lineare $10n$



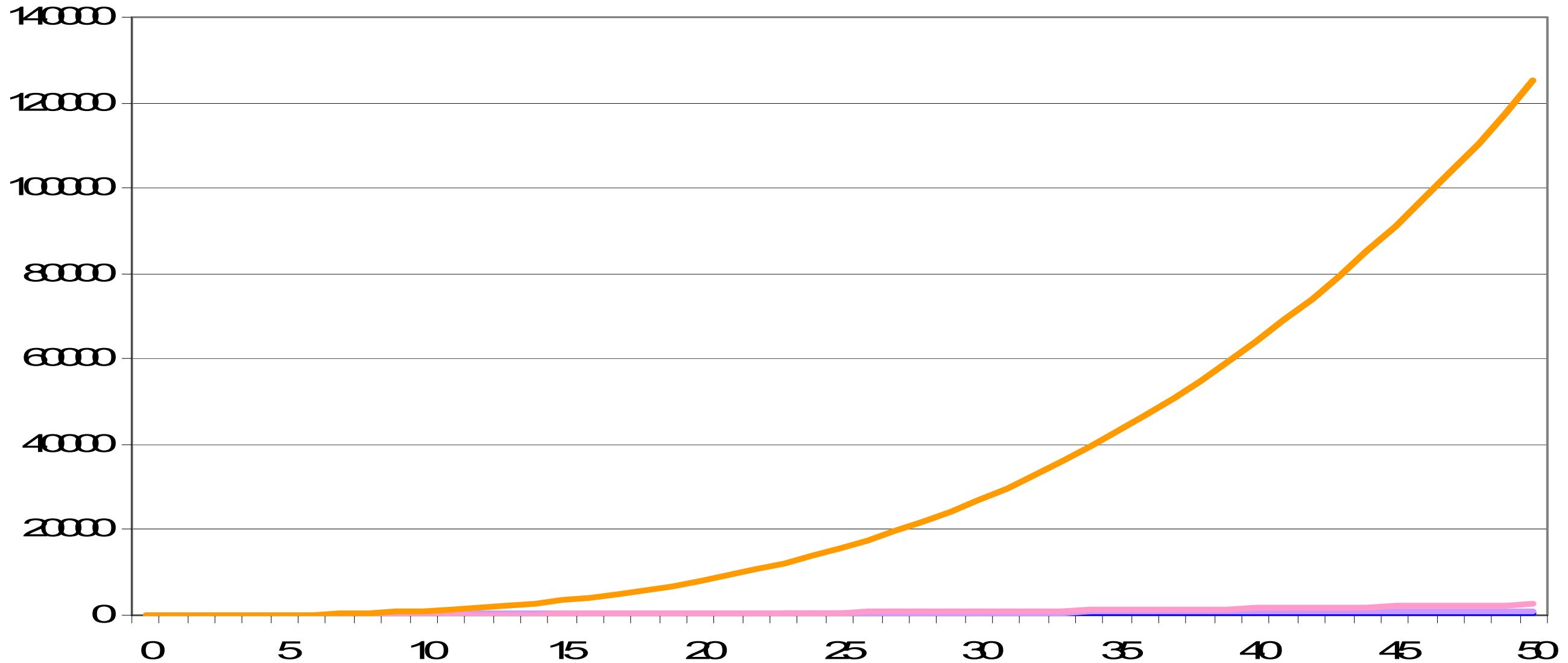
La funzione lineare $20n$



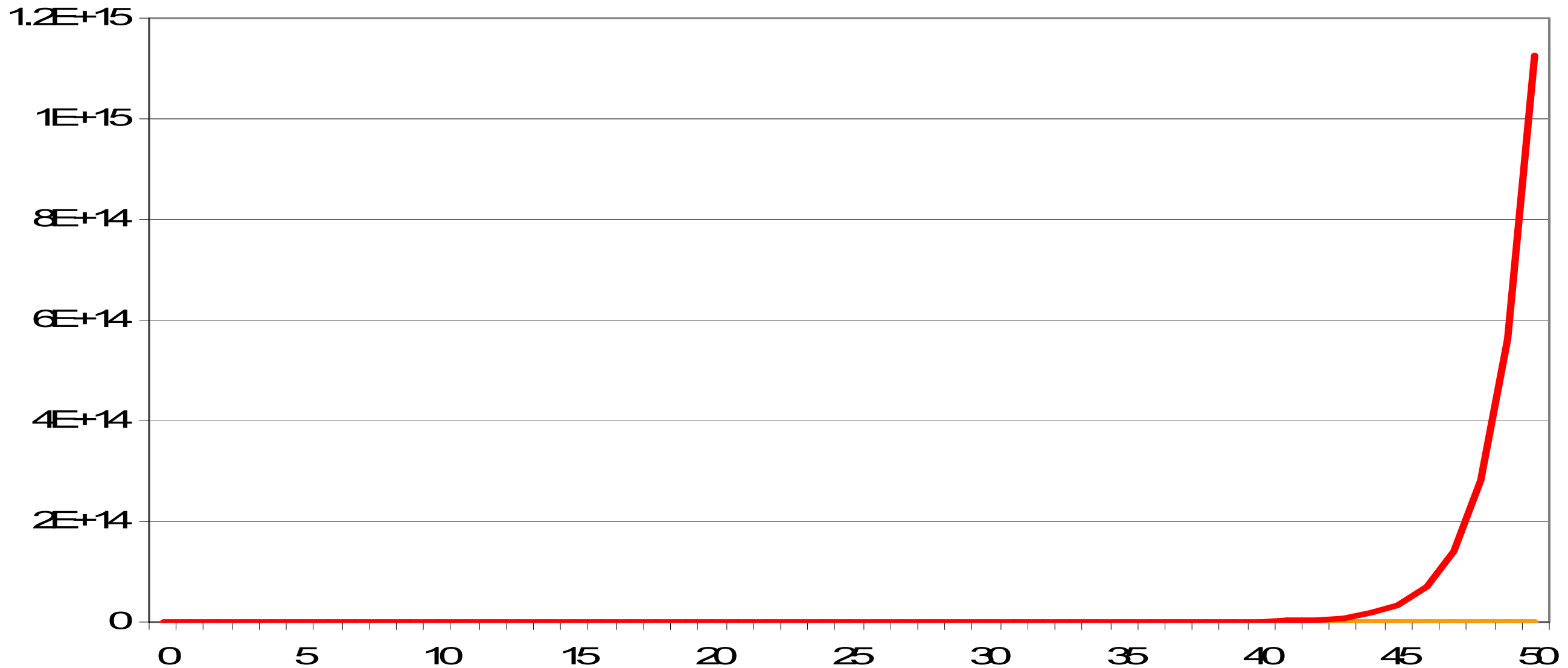
la funzione quadratica n^2



la funzione cubica n^3



La funzione esponenziale 2^n



Scopo delle notazioni asintotiche

- Si applicano alle funzioni $f(n)$ il cui dominio è l'insieme N dei naturali
 - possono essere facilmente estese ai reali
- Classificano le funzioni dal punto di vista del loro comportamento per grandi valori di n
- Forniscono un limite superiore e/o inferiore della funzione
 - la limitazione avviene per confronto con altre funzioni

Notazione O-grande

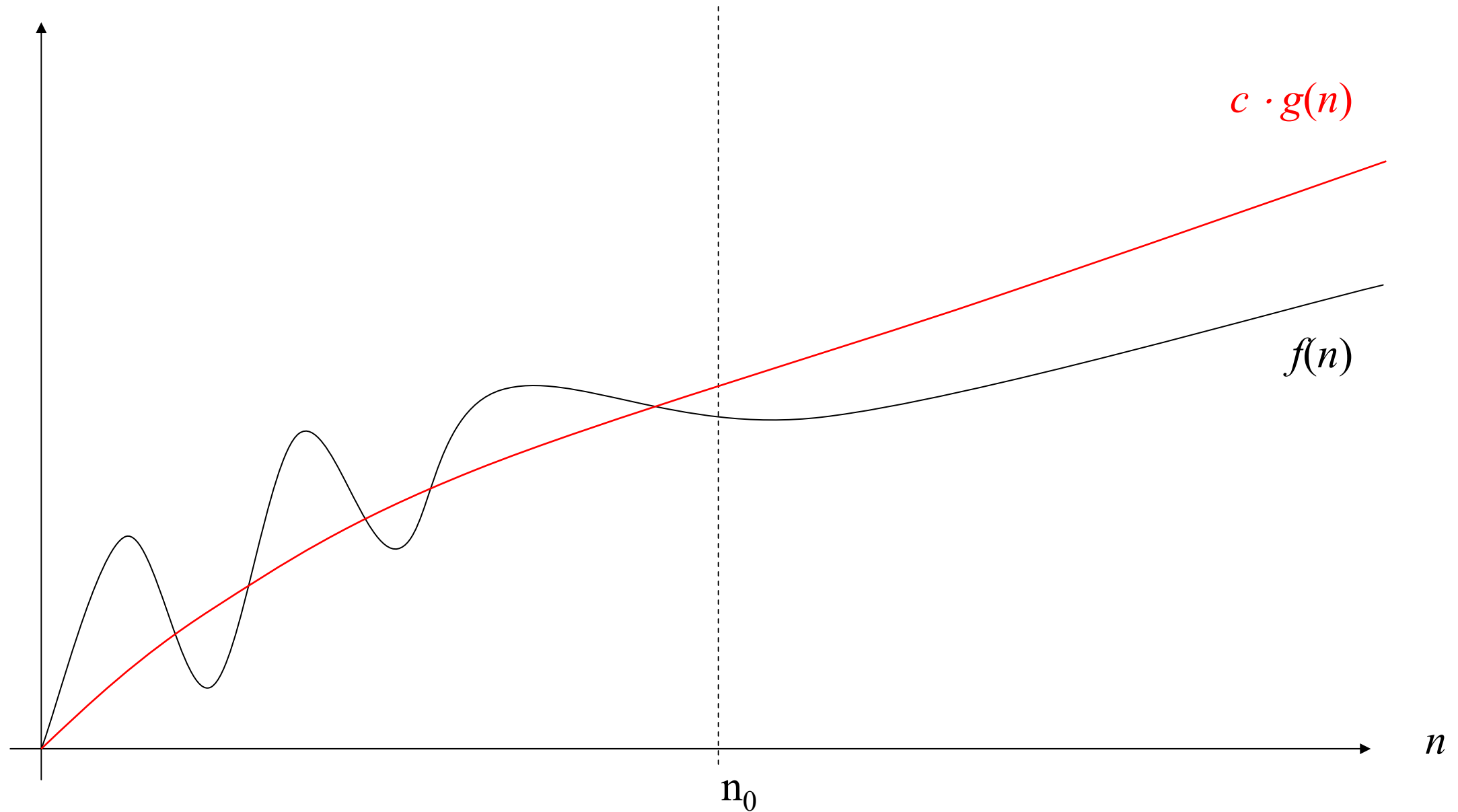
- Denotiamo $O(g(n))$ (“O grande di g di n ”) l’insieme delle funzioni “*limitate superiormente da $g(n)$* ”
- Definite come segue:

$$f(n) \in O(g(n)) \Leftrightarrow \text{esistono due costanti positive } c \text{ ed } n_0 \text{ tali che per ogni } n \geq n_0 \text{ si verifica}$$
$$0 \leq f(n) \leq c \cdot g(n)$$

- Oppure, più formalmente:

$$O(g(n)) = \{ f(n) : \exists c > 0, \exists n_0 > 0, \text{ tali che } \forall n \geq n_0 \\ 0 \leq f(n) \leq c \cdot g(n) \}$$

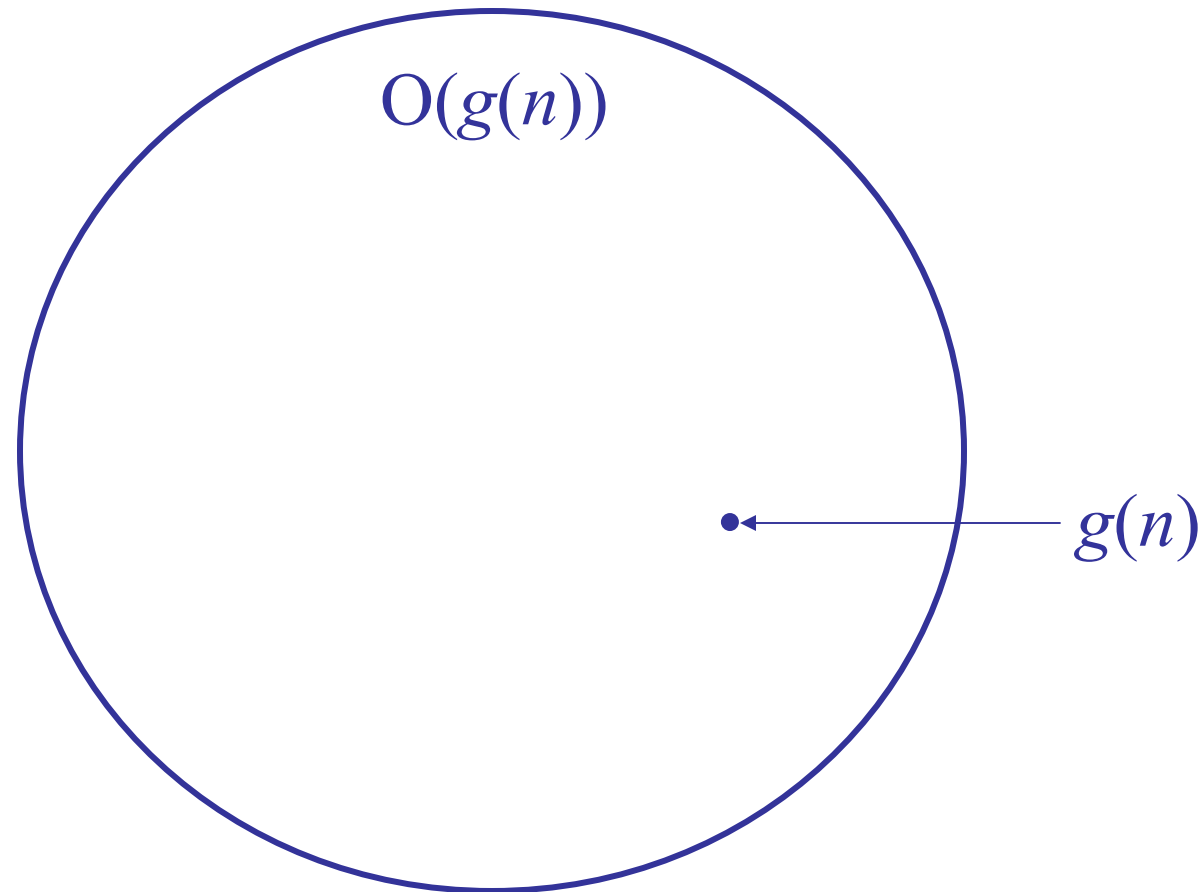
Notazione O-grande



Osservazioni sulla definizione di $O(g(n))$

- $O(g(n)) = \emptyset$ (l'insieme vuoto) se $g(n)$ è una funzione asintoticamente negativa
 - conveniamo che $g(n)$ non sia mai asintoticamente negativa
- Le costanti c ed n_0 dipendono dalla specifica $f(n)$
- Qual è il ruolo della costante c ?
 - se la costante c non ci fosse
 - correttamente avremmo $2n \in O(n^2)$
 - ma avremmo anche $2n \notin O(n)$, oppure $n^2+1 \notin O(n^2)$
- Vale la proprietà riflessiva: $g(n) \in O(g(n))$

Funzioni limitate superiormente da $g(n)$



Esempio di funzione $\in O(n^2)$

- dimostriamo che $\frac{1}{2} n^2 - 3n \in O(n^2)$
 - dobbiamo trovare almeno una $c > 0$ ed una $n_0 > 0$ tali che

$$\forall n \geq n_0, \quad 0 \leq f(n) \leq c \cdot g(n)$$

$$0 \leq \frac{1}{2} n^2 - 3n \leq c \cdot n^2$$

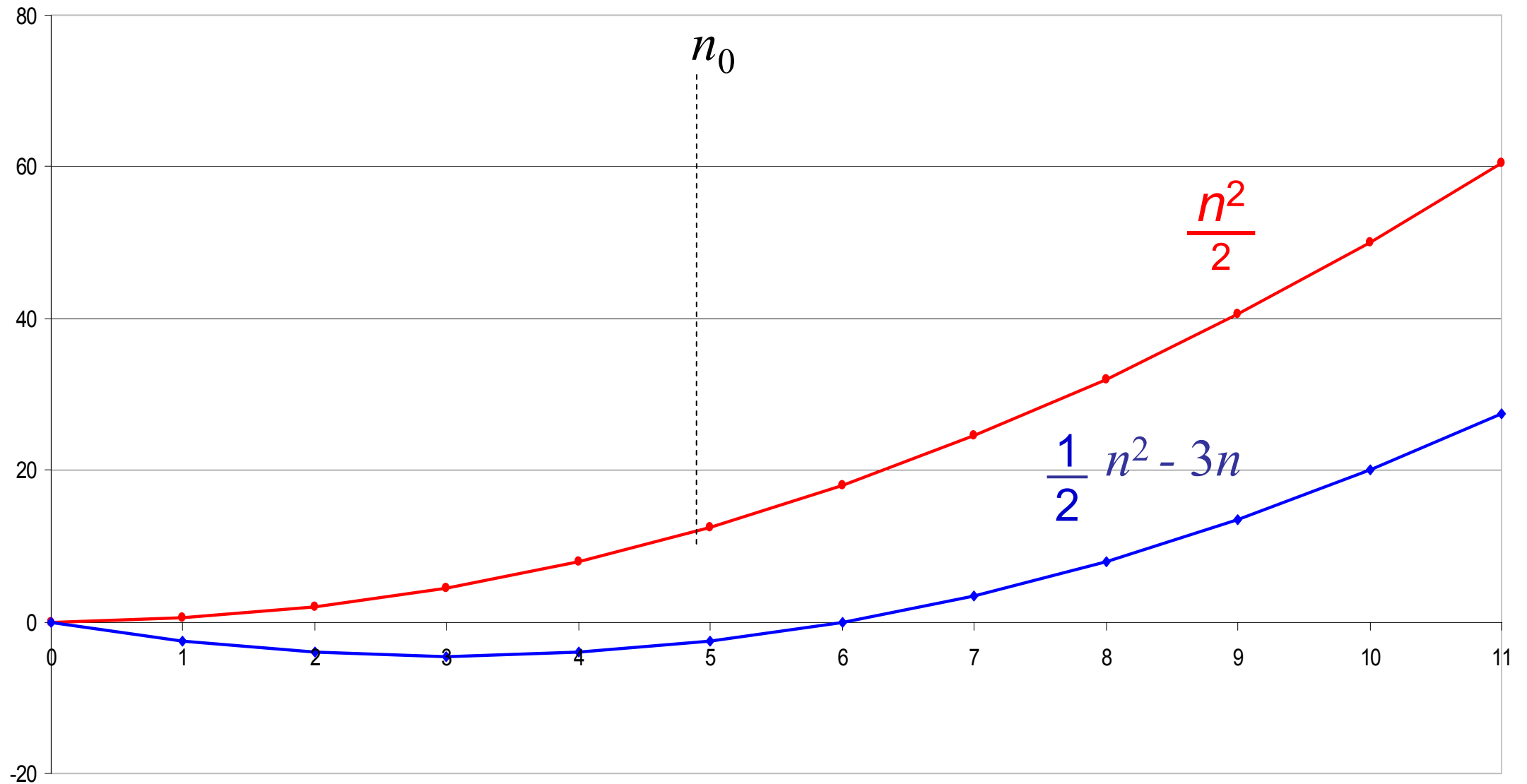
- dividiamo per n^2 e otteniamo $0 \leq \frac{1}{2} - \frac{3}{n} \leq c$
- proviamo a fissare $c = \frac{1}{2}$

$$\frac{1}{2} - \frac{3}{n} \leq \frac{1}{2} \text{ è soddisfatta per } n > 0$$

$$0 \leq \frac{1}{2} - \frac{3}{n} \text{ è soddisfatta per } n \geq 6$$

- dunque $c = 0.5$ e $n_0 = 6$ dimostrano l'asserto

Esempio di funzione $\in O(n^2)$



Generalizzazione

- per $c_1 > 0$ e $c_2 > 0$ si ha $c_1 n^k - c_2 n^{k-1} \in O(n^k)$
 - dobbiamo trovare almeno una $c > 0$ ed una $n_0 > 0$ tali che

$$\forall n \geq n_0, \quad 0 \leq f(n) \leq c \cdot g(n)$$

$$0 \leq c_1 n^k - c_2 n^{k-1} \leq c \cdot n^k$$

- dividiamo per n^k e otteniamo: $0 \leq c_1 - \frac{c_2}{n} \leq c$
- proviamo a fissare $c = c_1$

$$c_1 - \frac{c_2}{n} \leq c_1 \quad \text{è soddisfatta per } n \geq 0$$

$$0 \leq c_1 - \frac{c_2}{n} \quad \text{è soddisfatta per } n \geq \frac{c_2}{c_1}$$

- dunque la coppia $c = c_1$ e $n_0 = c_2/c_1$ dimostrano l'asserto

SEI
STUPIDO!
MA TI
AMO

Esercizio: $n^3 \notin O(n^2)$

- dimostriamo che $n^3 \notin O(n^2)$

- dovremmo trovare c ed n_0 tali che

$$\forall n \geq n_0, \quad 0 \leq f(n) \leq c \cdot g(n)$$
$$0 \leq n^3 \leq c \cdot n^2$$

- dividiamo per n^2

$$0 \leq n \leq c$$

- assurdo

- quale che sia c esiste sempre un valore di n per cui $n > c$

- analogamente, è facile dimostrare che

$$n^{k+1} \notin O(n^k)$$

Esercizio: $n^2 \in O(n^3)$

- dimostriamo, viceversa che $n^2 \in O(n^3)$

- dobbiamo trovare c ed n_0 tali che

$$\forall n \geq n_0, 0 \leq f(n) \leq c \cdot g(n)$$

$$0 \leq n^2 \leq c \cdot n^3$$

- dividiamo per n^2

$$0 \leq 1 \leq c \cdot n$$

- che è soddisfatta, per esempio, per $c = 1$ ed $n_0 = 1$

- analogamente, è facile dimostrare che

$$n^k \in O(n^{k+1})$$

Funzioni incommensurabili

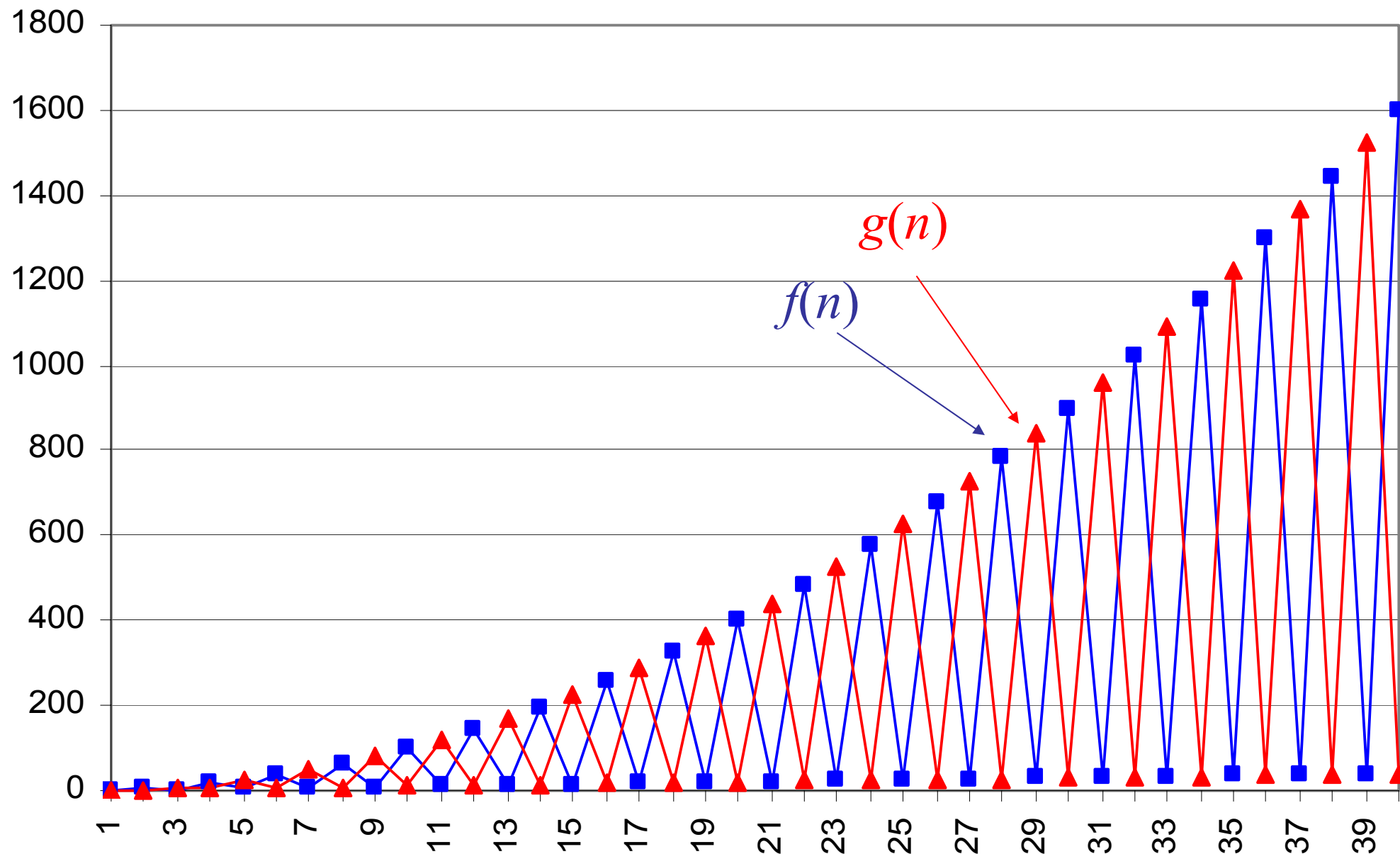
- è sempre vero che: $f(n) \in O(g(n))$
oppure: $g(n) \in O(f(n))$?
- consideriamo le seguenti funzioni

$$f(n) = \begin{cases} n & \text{se } n \text{ è dispari} \\ n^2 & \text{se } n \text{ è pari} \end{cases}$$

$$g(n) = \begin{cases} n^2 & \text{se } n \text{ è dispari} \\ n & \text{se } n \text{ è pari} \end{cases}$$

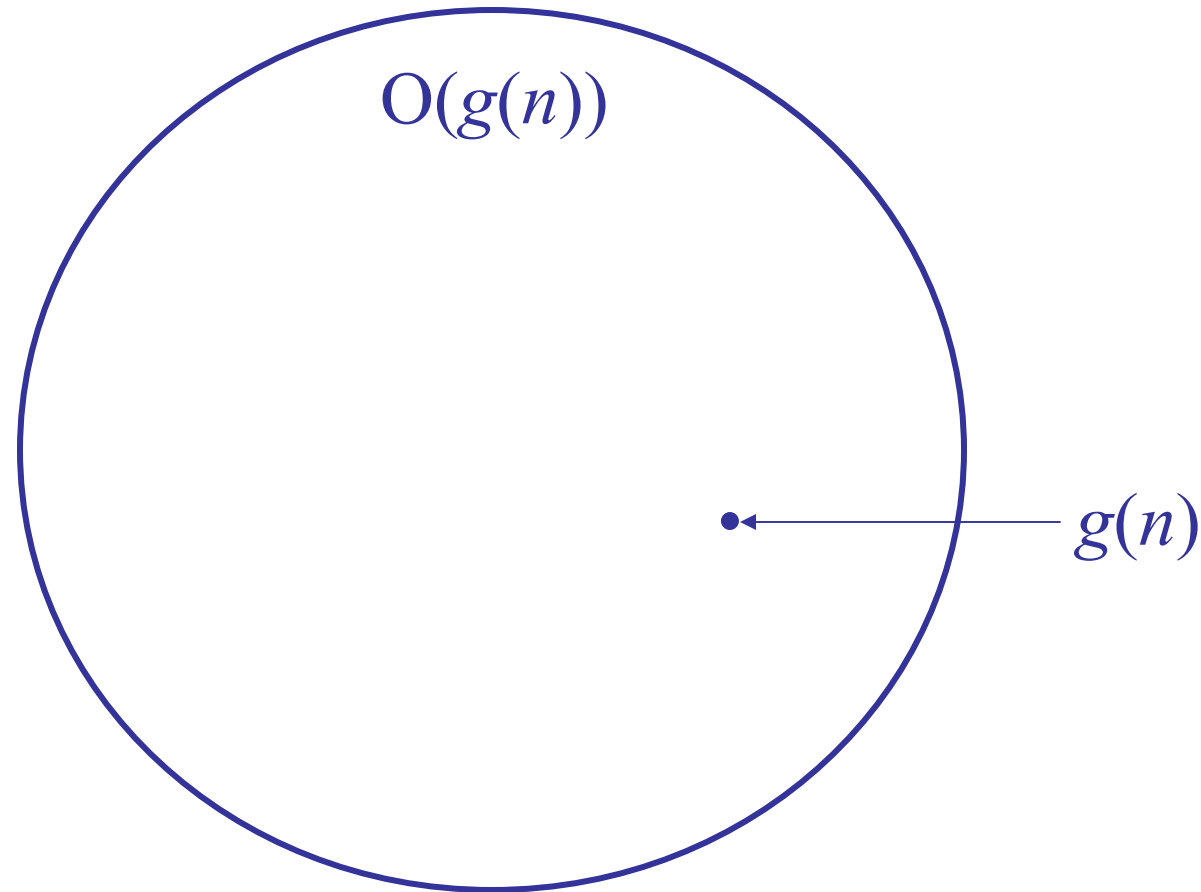
- poiché $n^2 \notin O(n)$
 - $f(n) \notin O(g(n))$ per via degli n pari
 - $g(n) \notin O(f(n))$ per via degli n dispari

Due funzioni incommensurabili



Esploriamo $O(g(n))$

- quali funzioni (oltre a $g(n)$) sono in $O(g(n))$?



Funzioni in $O(g(n))$

- dimostriamo che appartengono ad $O(g(n))$ le seguenti funzioni $f(n)$:

proprietà transitiva

$f(n) \in O(h(n))$ per qualche $h(n) \in O(g(n))$

regola dei fattori costanti positivi

$f(n) = d \cdot h(n)$ per qualche $h(n) \in O(g(n))$ e $d > 0$

regola della somma

$f(n) = h(n) + k(n)$ con $h(n)$ e $k(n) \in O(g(n))$

Proprietà transitiva

- dimostriamo che:
$$\left. \begin{array}{c} f(n) \in O(h(n)) \\ \wedge \\ h(n) \in O(g(n)) \end{array} \right\} \Rightarrow f(n) \in O(g(n))$$
- per ipotesi
$$\exists c' > 0, \exists n'_0 > 0, \text{ t.c. } \forall n \geq n'_0, 0 \leq f(n) \leq c' \cdot h(n)$$
$$\exists c'' > 0, \exists n''_0 > 0, \text{ t.c. } \forall n \geq n''_0, 0 \leq h(n) \leq c'' \cdot g(n)$$
- componendo le due
$$0 \leq f(n) \leq c' \cdot c'' \cdot g(n)$$
- e dunque
$$\exists c''' > 0, \exists n'''_0 > 0, \text{ t.c. } \forall n \geq n'''_0, 0 \leq f(n) \leq c''' \cdot g(n)$$

con $c''' = c' \cdot c''$ e con $n'''_0 = \max(n'_0, n''_0)$

Regola dei fattori costanti positivi

- se $d > 0$ è una costante

$$f(n) \in O(g(n)) \iff d \cdot f(n) \in O(g(n))$$

- infatti, per ipotesi si ha:

$$\exists c > 0, \exists n_0 > 0, \text{ t.c. } \forall n \geq n_0, 0 \leq f(n) \leq c \cdot g(n)$$

- definisco

$$c' = c \cdot d \quad (c' > 0 \text{ dato che } d > 0)$$

- sostituendo $c = c'/d$ ottengo

$$\exists c' > 0, \exists n_0 > 0, \text{ t.c. } \forall n \geq n_0, 0 \leq f(n) \leq c'/d \cdot g(n)$$

- finalmente moltiplicando per d

$$\exists c' > 0, \exists n_0 > 0, \text{ t.c. } \forall n \geq n_0, 0 \leq d \cdot f(n) \leq c' \cdot g(n)$$

Regola della somma

- dimostriamo che:
$$\left. \begin{array}{l} h(n) \in O(g(n)) \\ \wedge \\ k(n) \in O(g(n)) \end{array} \right\} \Rightarrow h(n) + k(n) \in O(g(n))$$

- per ipotesi

$$\exists c' > 0, \exists n'_0 > 0, \text{ t.c. } \forall n \geq n'_0, 0 \leq h(n) \leq c' \cdot g(n)$$

$$\exists c'' > 0, \exists n''_0 > 0, \text{ t.c. } \forall n \geq n''_0, 0 \leq k(n) \leq c'' \cdot g(n)$$

- sommando le due disequazioni si ottiene

$$0 \leq h(n) + k(n) \leq c' \cdot g(n) + c'' \cdot g(n)$$

- da cui

$$\begin{array}{l} \exists c''' > 0, \exists n'''_0 > 0, \text{ t.c. } \forall n \geq n'''_0, 0 \leq h(n) + k(n) \leq c''' \cdot g(n) \\ \text{con } c''' = c' + c'' \text{ e con } n'''_0 = \max(n'_0, n''_0) \end{array}$$

Usi estesi (o impropri) della notazione

- abuso della notazione

- spesso in luogo di $f(n) \in O(g(n))$ si trova

$$f(n) = O(g(n))$$

- questo corrisponde alla lettura “ $f(n)$ è $O(g(n))$ ” piuttosto che “ $f(n)$ è un elemento di $O(g(n))$ ”

- operazioni con la notazione asintotica

$3n^3 + O(n)$ si intende: $3n^3$ sommata con una qualche funzione appartenente ad $O(n)$

Esercizio

- dimostriamo che

$$6n^4 - 3n^3 + 2n^2 + 5n + 6 \in O(n^4)$$

fattori costanti positivi

appartenenze note

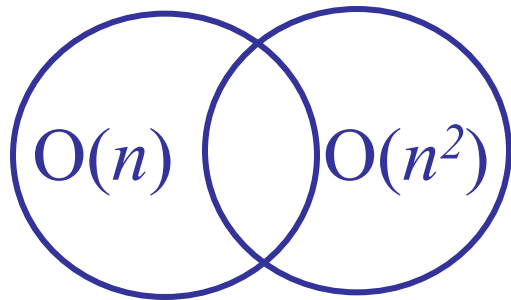
regola della somma

regola della somma

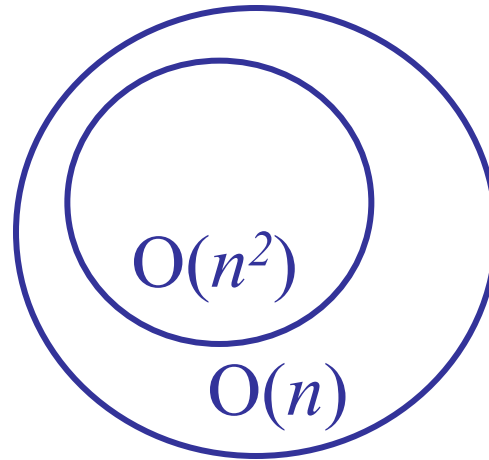
$$\begin{array}{c} O(n^4) - O(n^3) + O(n^2) + O(n) + O(1) \\ \underbrace{\hspace{10em}} \\ O(n^4) + O(n^2) + O(n) \\ \underbrace{\hspace{10em}} \\ O(n^4) + O(n) \\ \underbrace{\hspace{10em}} \\ O(n^4) \end{array}$$

Esercizi sulla notazione O-grande

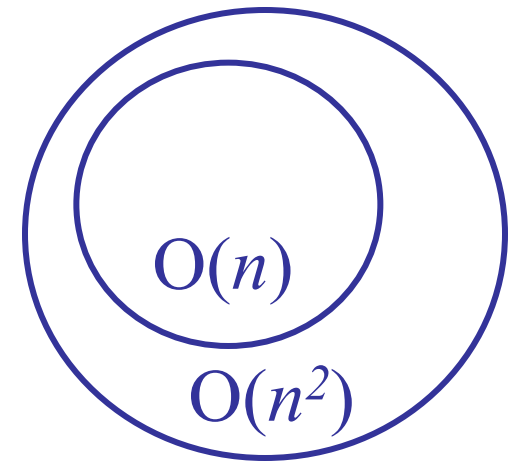
- Quali di questi rapporti di contenimento sono corretti?



Risposta 1



Risposta 2



Risposta 3

Notazione Ω

- Denotiamo $\Omega(g(n))$ (“Omega di g di n ”) l’insieme delle funzioni “*limitate inferiormente da $g(n)$* ”
- Definite come segue:

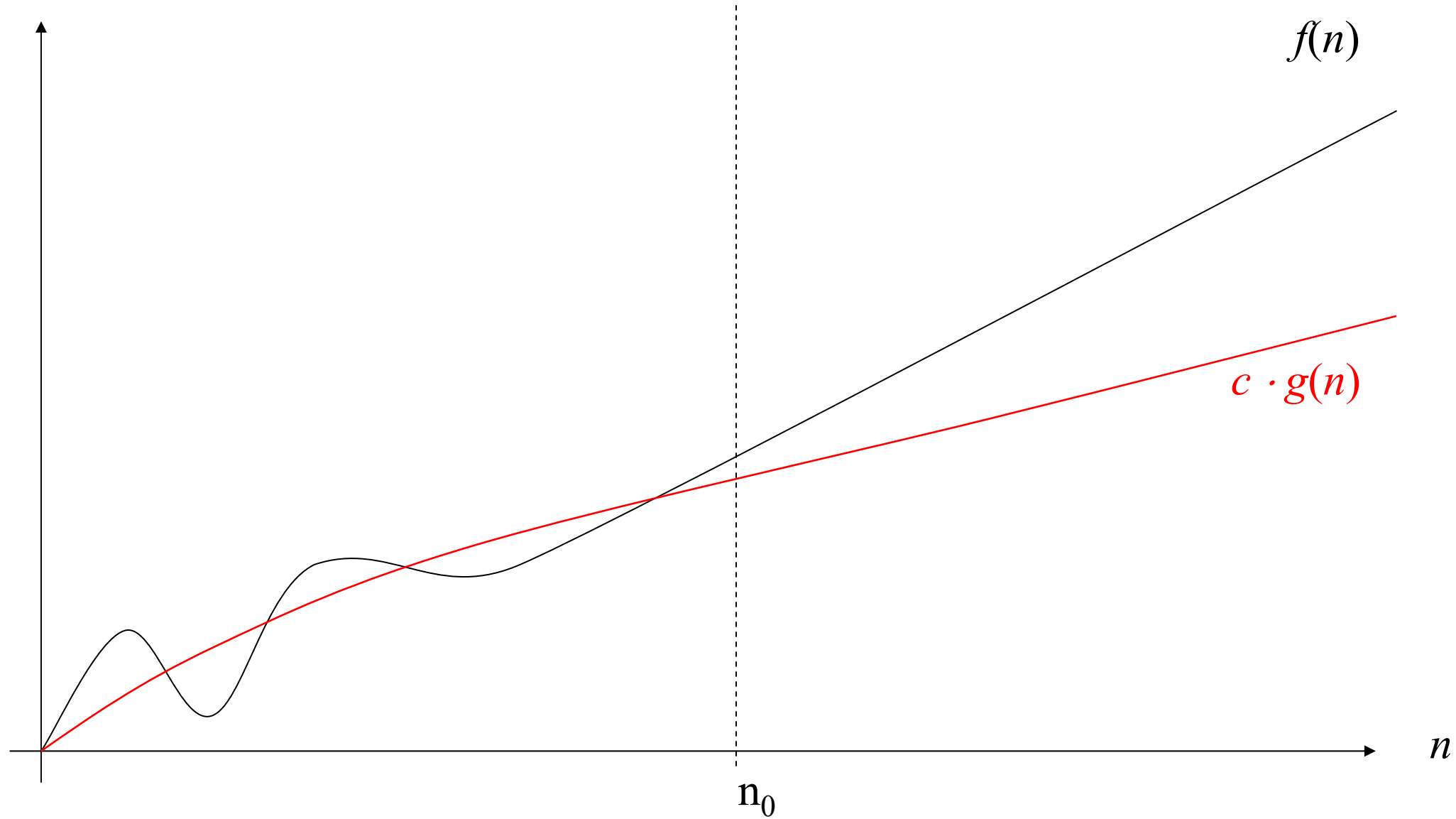
$f(n) \in \Omega(g(n)) \Leftrightarrow$ esistono due costanti positive c ed n_0 tali che per ogni $n \geq n_0$ si verifica

$$0 \leq c \cdot g(n) \leq f(n)$$

- Oppure, più formalmente:

$$\Omega(g(n)) = \{ f(n) : \exists c > 0, \exists n_0 > 0, \text{ tali che } \forall n \geq n_0 \\ 0 \leq c \cdot g(n) \leq f(n) \}$$

Notazione Ω



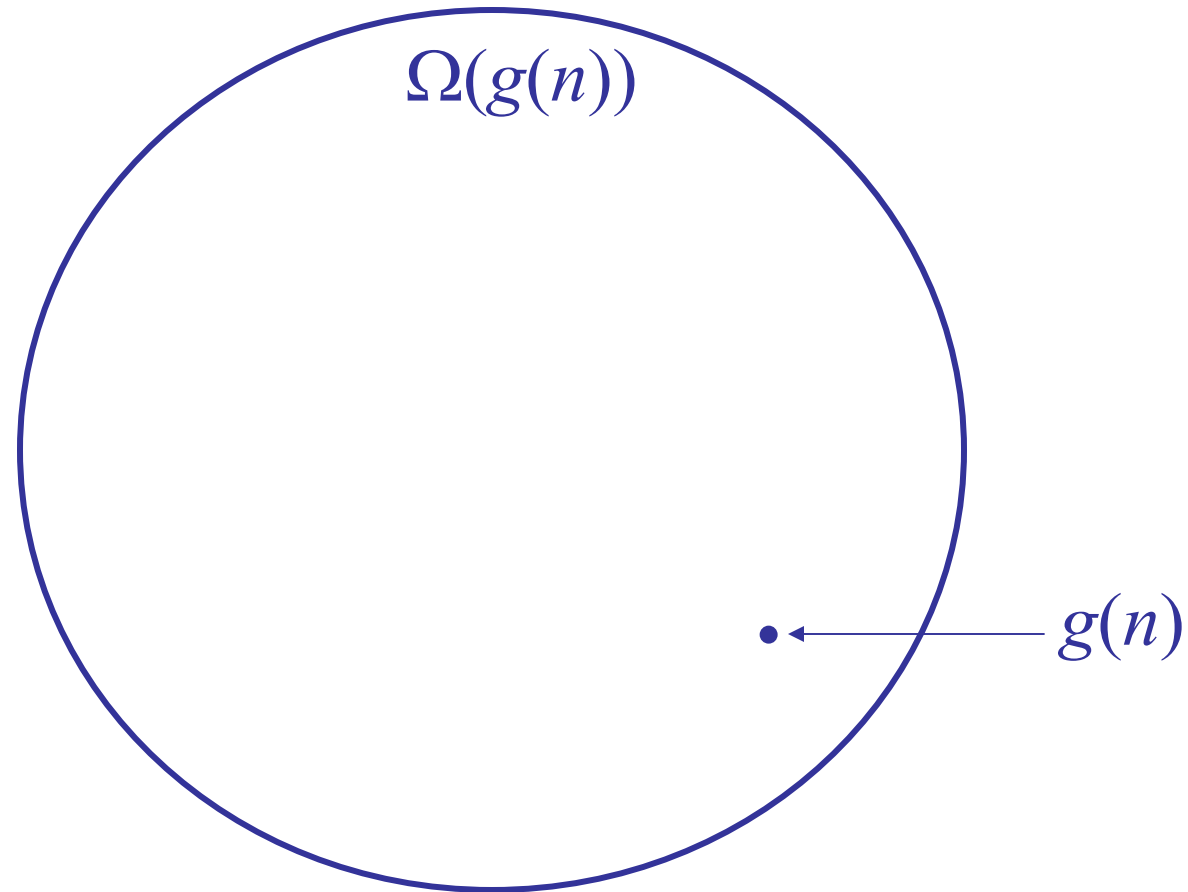
Osservazioni sulla definizione di $\Omega(g(n))$

- si può facilmente dimostrare che

$$f(n) \in \Omega(g(n)) \Leftrightarrow g(n) \in O(f(n))$$

- nel caso della notazione Ω occorre spesso ricorrere a valori minori di uno per la costante c
 - la costante c non è necessariamente un intero
- sarebbe stato analogo scrivere: $0 \leq g(n) \leq c \cdot f(n)$
- anche per $\Omega(g(n))$ esistono funzioni incommensurabili
- vale la proprietà riflessiva: $g(n) \in \Omega(g(n))$

Funzioni limitate inferiormente da $g(n)$



Funzioni in $\Omega(g(n))$

- Per $\Omega(g(n))$ valgono proprietà analoghe a quelle che abbiamo dimostrato per $O(g(n))$
- Appartengono ad $\Omega(g(n))$ le seguenti funzioni:

proprietà transitiva

$$f(n) \in \Omega(h(n)) \quad \text{per qualche} \quad h(n) \in \Omega(g(n))$$

regola dei fattori costanti positivi

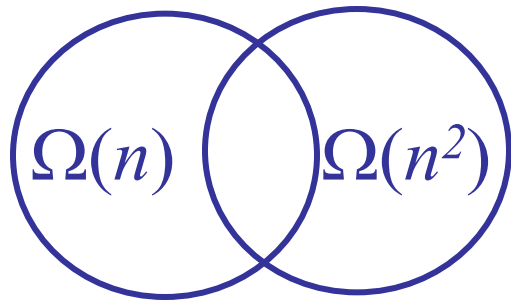
$$f(n) = d \cdot h(n) \quad \text{per qualche} \quad h(n) \in \Omega(g(n)) \text{ e } d > 0$$

regola della somma

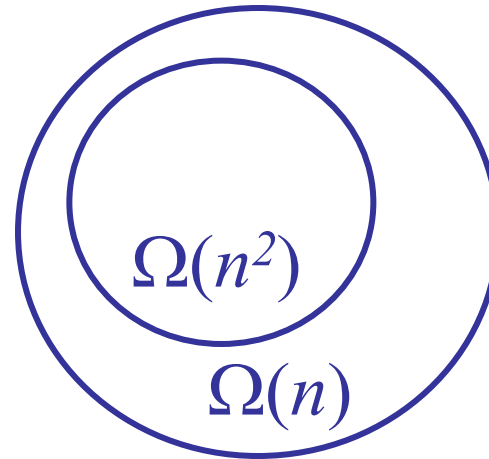
$$f(n) = h(n) + k(n) \quad \text{con} \quad h(n) \text{ e } k(n) \in \Omega(g(n))$$

Esercizi sulla notazione Ω

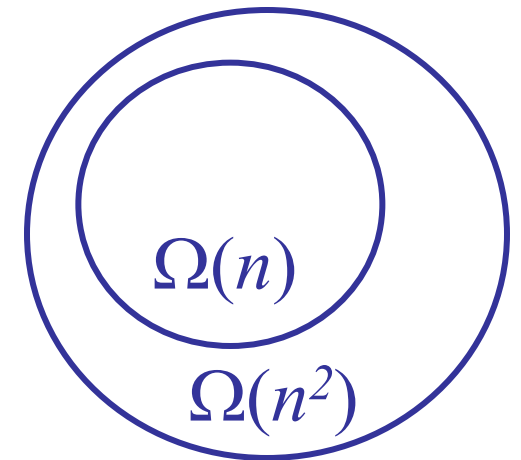
- Quali di questi rapporti di contenimento sono corretti?



Risposta 1



Risposta 2



Risposta 3

Notazione Θ

- Denotiamo $\Theta(g(n))$ (“Teta di g di n ”) l’insieme delle funzioni “*limitate inferiormente e superiormente da $g(n)$* ”
- Definite come segue:

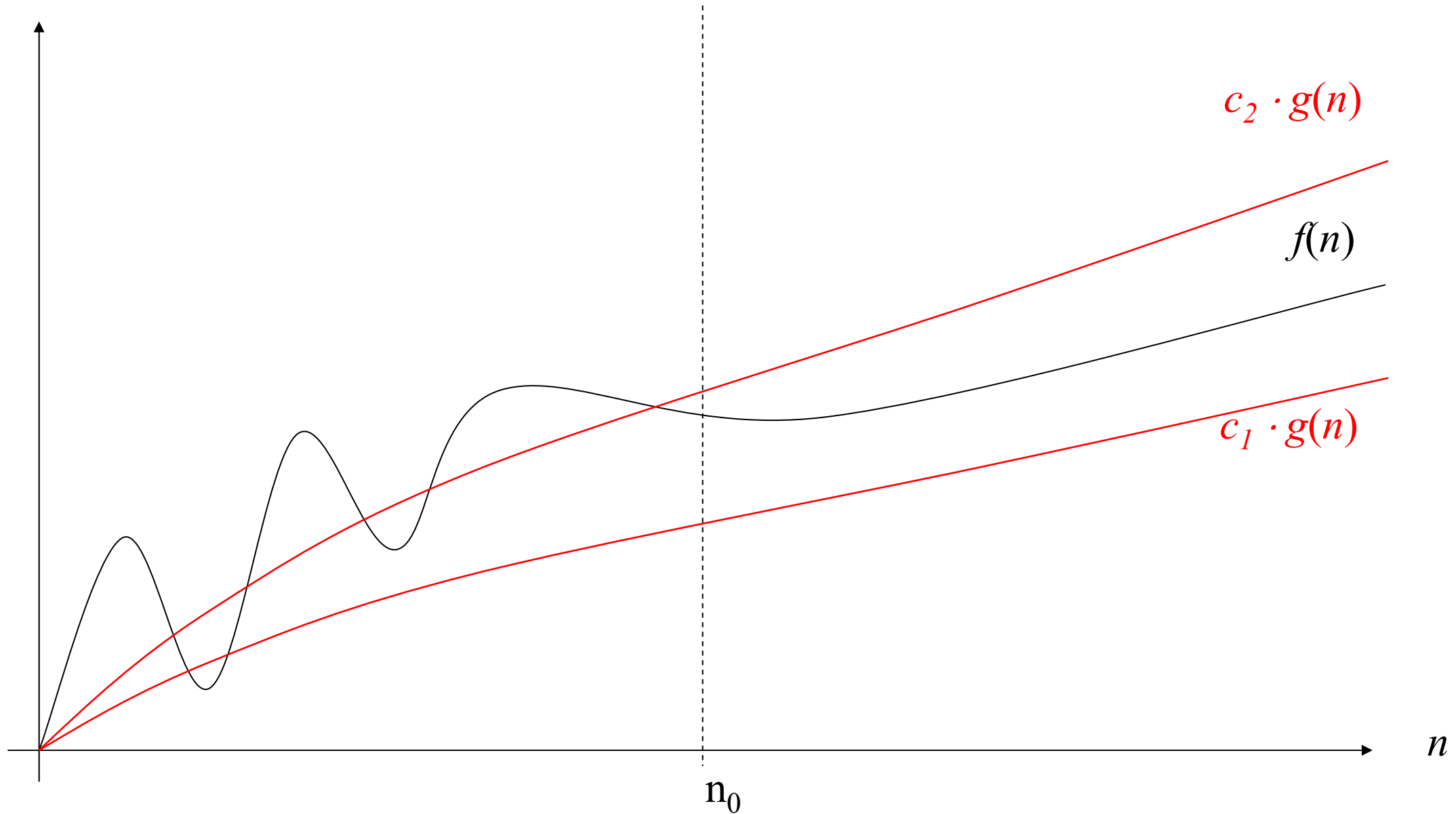
$f(n) \in \Theta(g(n)) \Leftrightarrow$ esistono tre costanti positive c_1 , c_2 , ed n_0 tali che per ogni $n \geq n_0$ si verifica

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

- Oppure, più formalmente:

$\Theta(g(n)) = \{ f(n) : \exists n_0 > 0, \exists c_1 > 0, \exists c_2 > 0, \text{ tali che}$
$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \forall n \geq n_0 \}$$

Notazione Θ



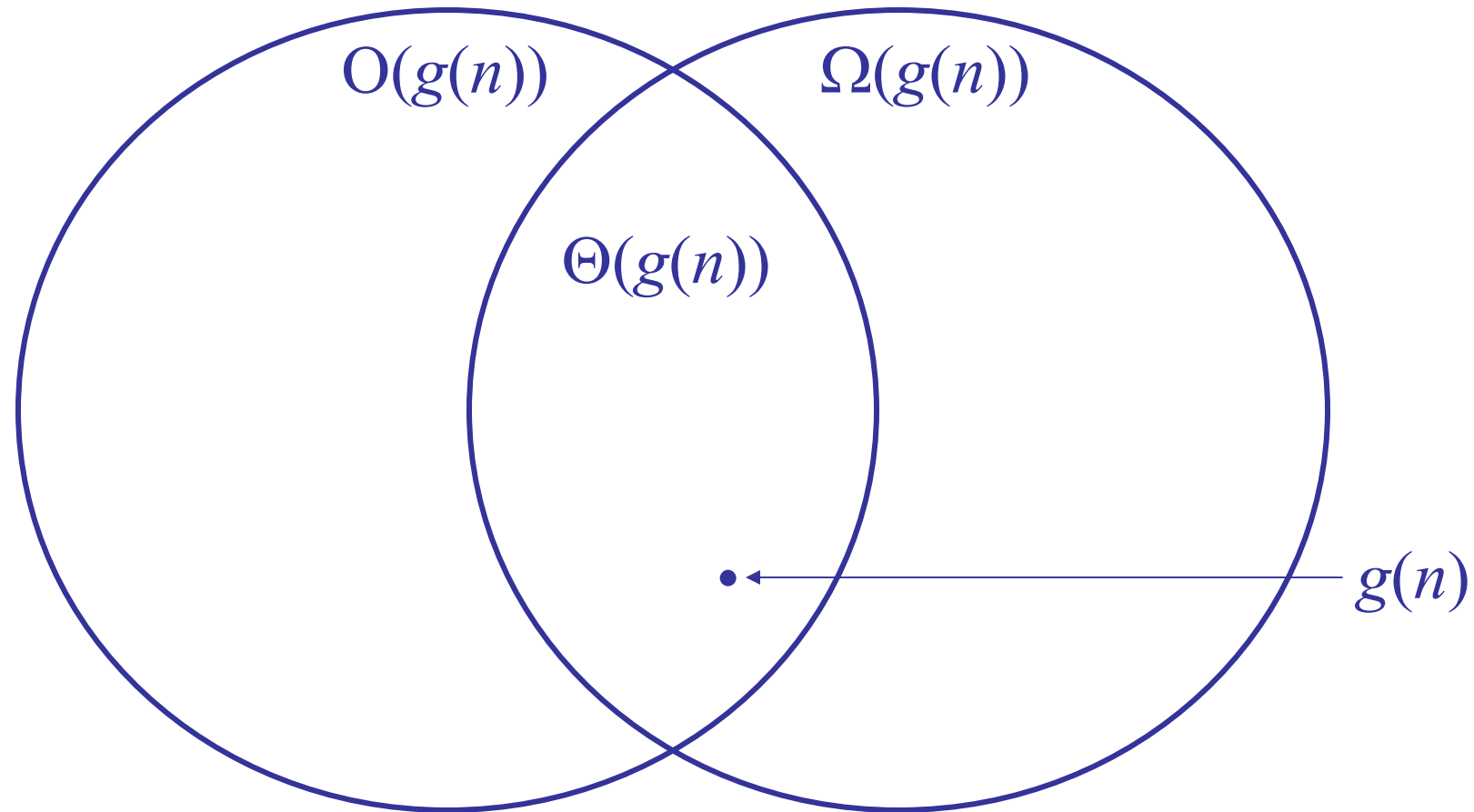
Osservazioni sulla definizione di $\Theta(g(n))$

- dalla definizione si ricava immediatamente che

$$f(n) \in \Theta(g(n)) \Leftrightarrow \begin{cases} f(n) \in O(g(n)) \\ \wedge \\ f(n) \in \Omega(g(n)) \end{cases}$$

- questa considerazione offre una definizione alternativa di $\Theta(g(n))$
- vale la proprietà riflessiva: $g(n) \in \Theta(g(n))$
- valgono tutte le proprietà che abbiamo dimostrato per O-grande e per Ω

Funzioni $\in \Theta(g(n))$



Proprietà simmetrica

- è immediato dimostrare che

$$f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$$

- infatti

$$f(n) \in O(g(n)) \Rightarrow g(n) \in \Omega(f(n))$$

$$f(n) \in \Omega(g(n)) \Rightarrow g(n) \in O(f(n))$$

- dunque

$$f(n) \in \Theta(g(n)) \Rightarrow g(n) \in \Theta(f(n))$$

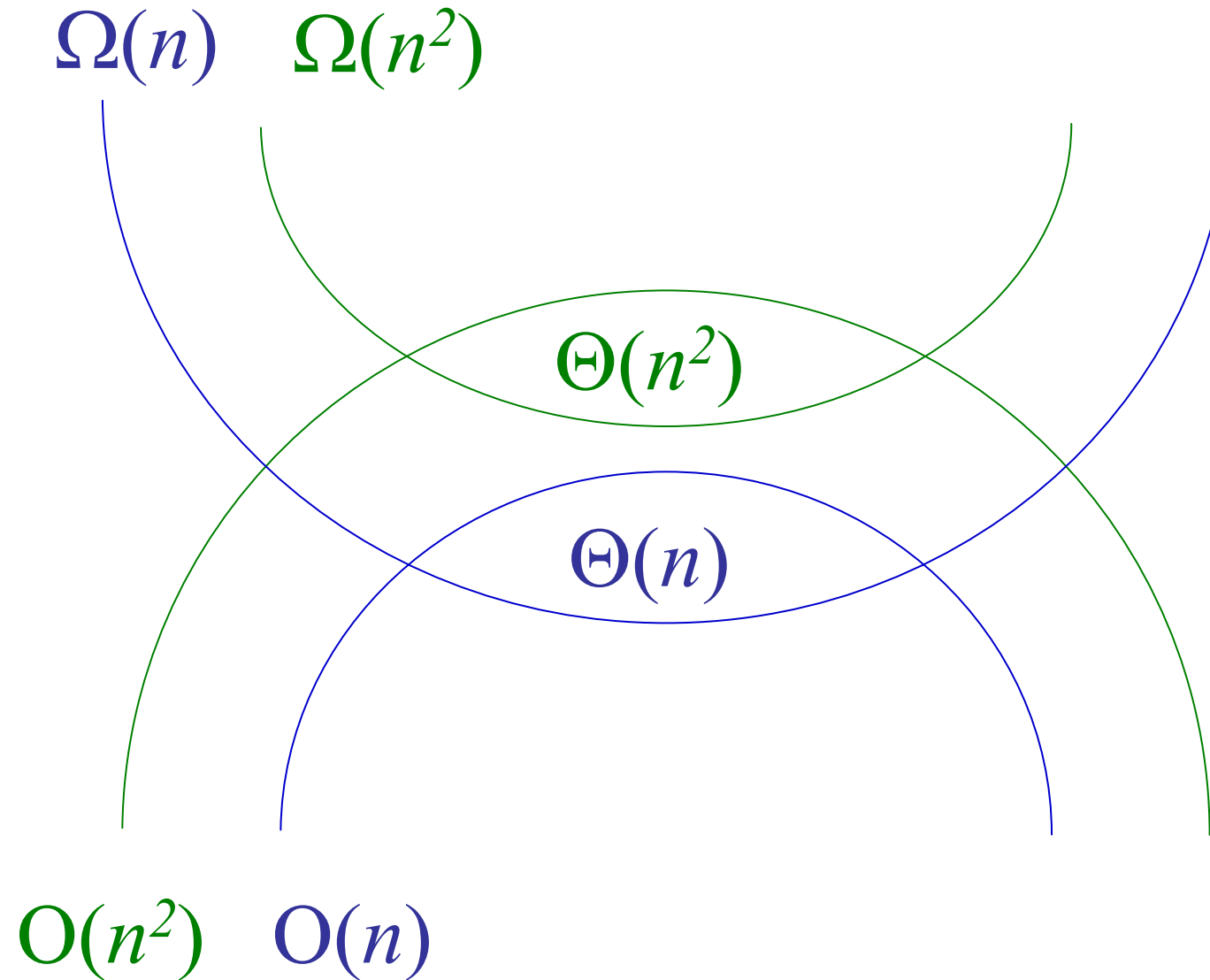
- in maniera analoga si dimostra che

$$g(n) \in \Theta(f(n)) \Rightarrow f(n) \in \Theta(g(n))$$

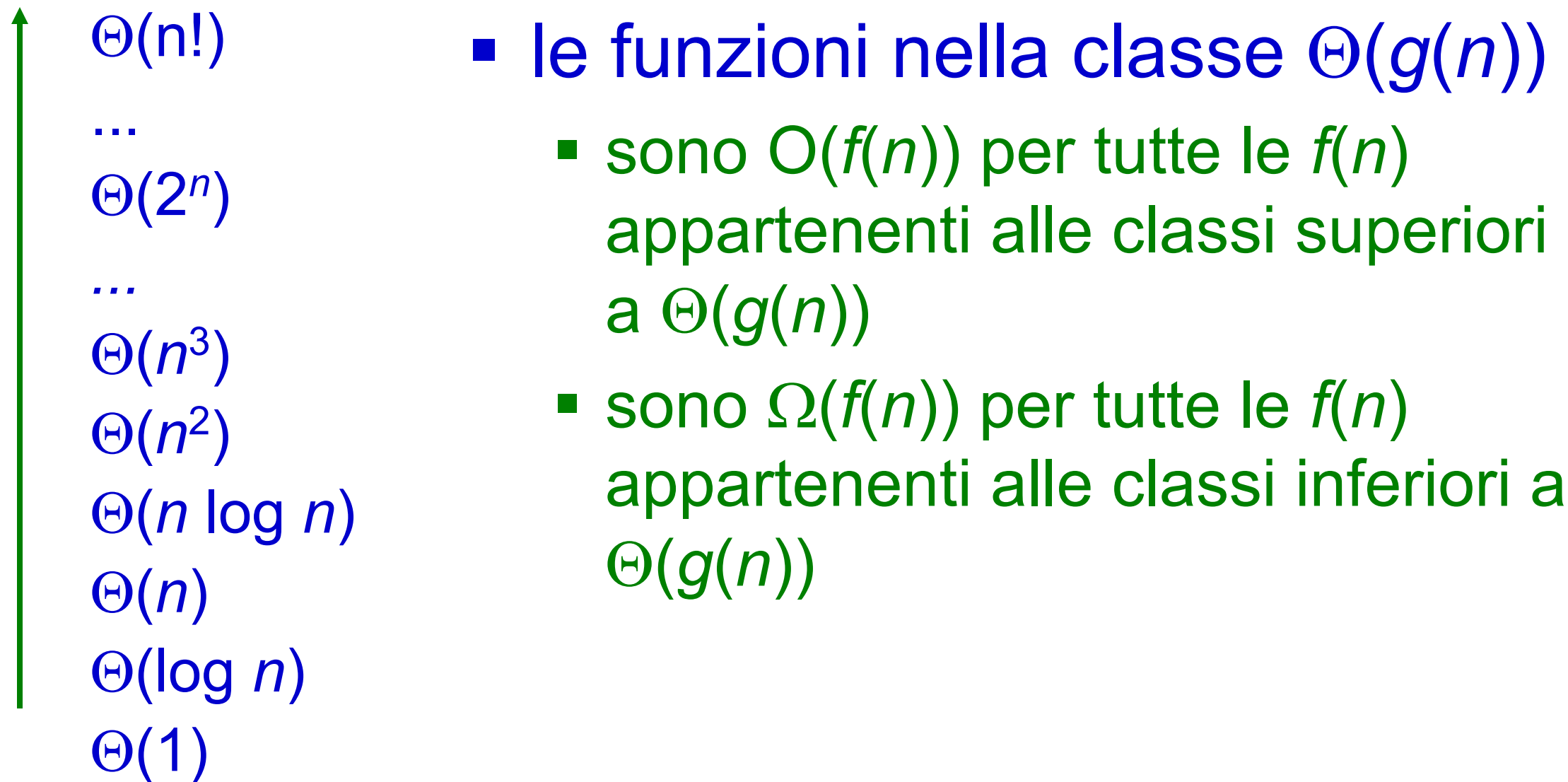
La relazione di equivalenza Θ

- poiché $f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$, la relazione $f(n) \in \Theta(g(n))$ tra $f(n)$ e $g(n)$ gode della proprietà simmetrica
- dunque la notazione Θ definisce una relazione di equivalenza
 - valgono infatti le tre proprietà riflessiva, simmetrica e transitiva
 - la notazione Θ consente di classificare tutte le funzioni in classi di equivalenza, che descrivono il loro comportamento al crescere di n

Rapporti tra classi



Gerarchia delle funzioni



Algoritmi e Strutture di Dati

Complessità degli algoritmi

M. Patrignani

Nota di copyright

- Queste slides sono protette dalle leggi sul copyright
- Il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- Le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- Ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- Gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- Questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Algoritmi e programmi

- un *algoritmo* coincide per noi con la sua descrizione in pseudocodice
 - sappiamo che ciò è equivalente a definire una Random Access Machine
- lo pseudocodice può essere facilmente tradotto in un linguaggio di programmazione arbitrario per ottenere un *programma*
- l'operazione di traduzione di un algoritmo in un programma viene detta *implementazione*

Esecuzione dei programmi

- il programma può essere eseguito
 - su una piattaforma qualsiasi
 - con dati di input qualsiasi
- la sua esecuzione ha un costo (economico) che può essere espresso tramite le risorse di calcolo utilizzate
 - tempo
 - memoria
 - traffico generato su rete
 - trasferimento dati da/su disco
 -
- nella maggior parte dei casi la risorsa più critica è il tempo di calcolo

Tempo di calcolo e dimensione dell'input

- si riscontra che il tempo di calcolo cresce al crescere della dimensione n dell'input
 - è legittimo misurarlo come una funzione di n
- la nozione di dimensione dell'input dipende dal problema
 - ordinamento:
 - numero di elementi da ordinare
 - operazioni su liste:
 - lunghezza della lista
 - operazioni su matrici:
 - dimensione massima delle matrici coinvolte
 - valutazione di un polinomio in un punto:
 - grado del polinomio

Fattori che influenzano il tempo di calcolo

- **dimensione dell'input**
 - maggiore è la quantità di dati in input maggiore è il tempo necessario per processarli
- **algoritmo**
 - può essere più o meno efficiente
- **hardware**
 - un supercalcolatore è più veloce di un personal computer
- **linguaggio**
 - un'implementazione diretta in linguaggio macchina è più veloce di un'implementazione in un linguaggio ad alto livello
 - un programma compilato è più veloce di un programma interpretato
- **compilatore**
 - alcuni compilatori sono progettati per generare codice efficiente
- **programmatore**
 - a parità di algoritmo e di linguaggio, programmatori esperti scelgono costrutti più veloci

Tempo di calcolo e algoritmi

- esperimento: confronto tra due algoritmi di ordinamento
 - stesso input: 1.000.000 numeri interi

algoritmo	insertion sort	merge sort
hardware	supercalcolatore	personal computer
linguaggio	linguaggio macchina	linguaggio ad alto livello
compilatore	—	non efficiente
programmatore	esperto	medio

tempo	5,56 ore	16,67 minuti
-------	----------	--------------

- conclusione:
 - il tempo di calcolo di un programma su uno specifico input è influenzato dall'algoritmo che implementa più che dagli altri fattori

Progetto di algoritmi efficienti

■ motivazione

- ha un impatto economico diretto per gli utilizzatori dei programmi
 - il tempo di calcolo si traduce in un investimento economico
- è fondamentale per ottenere implementazioni di uso pratico
 - l'usabilità di un programma può essere compromessa da un algoritmo inefficiente

■ problema

- nel momento in cui progettiamo un algoritmo non possiamo misurare direttamente l'efficienza delle sue future implementazioni

■ soluzione

- previsione del tempo di calcolo delle implementazioni di un algoritmo (analisi)

Analisi degli algoritmi

■ obiettivo

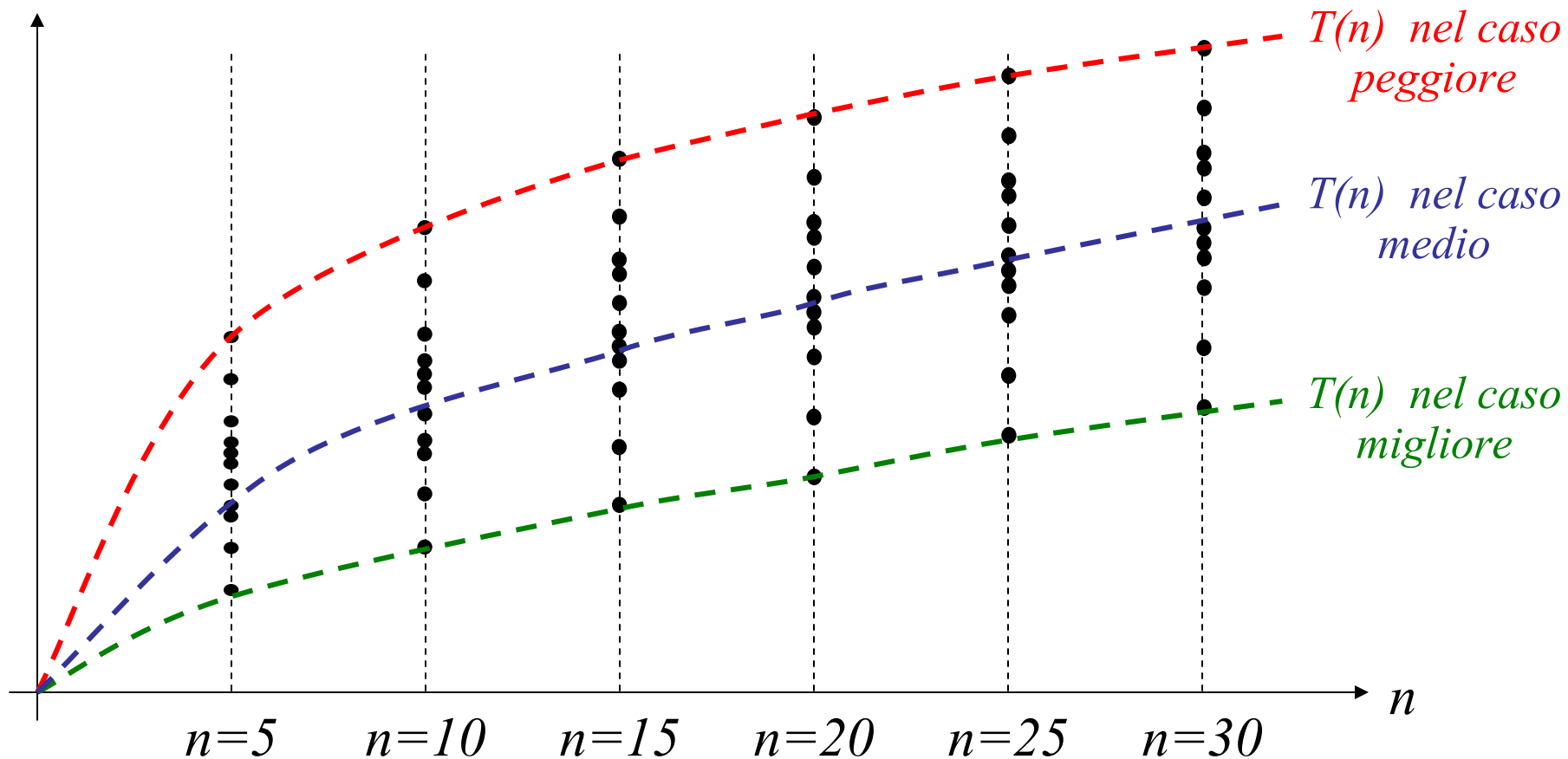
- prevedere il tempo di calcolo richiesto dall'esecuzione di un programma che implementa il nostro algoritmo
 - in funzione della dimensione dell'input
 - per piccoli input il tempo di calcolo sarà comunque basso
 - qual è il tempo di calcolo per input di grandi dimensioni?

■ strumenti

- ipotesi sul tempo di esecuzione di ogni istruzione
- analisi asintotica delle funzioni
 - quale funzione consideriamo?

Il tempo di calcolo non è una funzione

- In generale il tempo di calcolo per un input di dimensione n non è una funzione



Tempo di calcolo e analisi asintotica

- vogliamo studiare il tempo di calcolo con gli strumenti dell'analisi asintotica
 - ma l'analisi asintotica si applica solo alle funzioni
 - dobbiamo trasformare il tempo di calcolo in una funzione
 - per questo consideriamo il caso peggiore/medio/migliore

		O	Ω	Θ
←funzioni→	caso peggiore			
	caso medio			
	caso migliore			

Uso del caso peggiore

- in generale è di maggiore interesse il *caso peggiore* rispetto al *caso migliore* o al *caso medio*
 - si preferisce un errore per eccesso ad un errore per difetto
 - il caso migliore non dà nessuna garanzia sul tempo di calcolo con un input generico
 - è di interesse solamente teorico
 - spesso il tempo di calcolo del caso medio è più vicino al caso peggiore che al caso migliore
 - conoscere il costo del caso medio può essere utile solo qualora si debba ripetere un'operazione un numero elevato di volte

Stima del tempo di calcolo

- denotiamo $T(n)$ il tempo di calcolo di una implementazione dell'algoritmo nel caso peggiore su un input di dimensione n
- vogliamo stimare $T(n)$ a partire dallo pseudocodice
- quanto costa ogni operazione elementare?
- ipotesi semplificativa:
 - per eseguire una linea (o istruzione) di pseudocodice è richiesto tempo costante
 - denotiamo con c_i il tempo necessario per eseguire la riga i

Strategie per la stima del tempo di calcolo

- strategia più onerosa

- calcoliamo esplicitamente $T(n)$ a partire dallo pseudocodice
 - $T(n)$ dipende, oltre dalla dimensione dell'input n , anche dal costo di esecuzione associato alle singole righe dello pseudocodice $c_1, c_2, c_3, c_4, \dots$
- studiamo il comportamento asintotico di $T(n)$

- strategia più veloce

- calcoliamo il costo asintotico di ogni porzione dello pseudocodice
- otteniamo il costo asintotico dell'intero algoritmo componendo i costi calcolati
- otteniamo il comportamento asintotico di $T(n)$ senza mai calcolare esplicitamente $T(n)$

Esempio di algoritmo

- algoritmo DUE_OCCORRENZE
 - accetta in input un array di interi
 - ritorna TRUE se esiste un valore che occorre almeno due volte
 - ritorna FALSE se tutti gli elementi sono diversi

DUE_OCCORRENZE (A)

1. output = **false**

2. **for** i = 0 **to** A.length-2

3. **for** j = i+1 **to** A.length-1

4. **if** (A[i] == A[j])

5. output = **true**

6. **return** output

Esempio di calcolo di $T(n)$

DUE_OCCORRENZE (A)	costo	n° di volte
1. output = false	C_1	1
2. for i = 0 to A.length-2	C_2	
3. for j = i+1 to A.length-1	C_3	
4. if (A[i] == A[j])	C_4	
5. output = true	C_5	
6. return output	C_6	1

supponiamo che ogni riga i
abbia un costo di esecuzione c_i

contiamo quante volte ogni riga
viene eseguita

Esempio di calcolo di $T(n)$

DUE_OCCORRENZE (A)

```
1. output = false
2. for i = 0 to A.length-2
3.     for j = i+1 to A.length-1
4.         if (A[i] == A[j])
5.             output = true
6. return output
```

costo

C_1

C_2

C_3

C_4

C_5

C_6

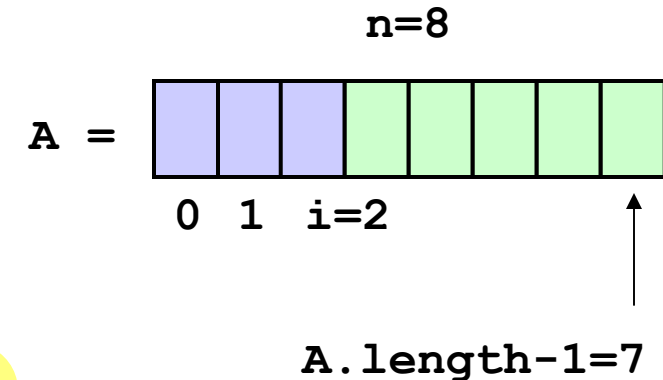
n° di volte

1

n

1

- il ciclo esterno viene eseguito $n-1$ volte
 - il test, dunque, viene eseguito n volte
 - l'ultimo test determina l'uscita dal ciclo for
- il ciclo interno viene eseguito $n-i-1$ volte



Esempio di calcolo di $T(n)$

DUE_OCCORRENZE (A)	costo	n° di volte
1. output = false	C_1	1
2. for i = 0 to A.length-2	C_2	n
3. for j = i+1 to A.length-1	C_3	
4. if (A[i] == A[j])	C_4	
5. output = true	C_5	
6. return output	C_6	1

- per ogni i , il ciclo interno viene eseguito $n-i-1$ volte, cioè: $\sum_{i=0..n-2}(n-i-1)$
 $= \sum_{i=0..n-2}(n) - \sum_{i=0..n-2}(i) - \sum_{i=0..n-2}(1)$
- dove

$$\sum_{i=0..n-2}(n) = (n-1)n$$

$$\sum_{i=0..n-2}(i) = (n-1)(n-2)/2$$

$$\sum_{i=0..n-2}(1) = (n-1)$$

formula di Gauss

$$\sum_{i=1..n}(i) = \frac{(n+1)n}{2}$$

Esempio di calcolo di $T(n)$

DUE_OCCORRENZE (A)	costo	n° di volte
1. output = false	C_1	1
2. for i = 0 to A.length-2	C_2	n
3. for j = i+1 to A.length-1	C_3	$0.5n^2+1.5n+1$
4. if (A[i] == A[j])	C_4	$0.5n^2+1.5n$
5. output = true	C_5	$0.5n^2+1.5n$
6. return output	C_6	1

- il ciclo interno viene eseguito $(n-1)n - (n-1)(n-2)/2 - (n-1) = n^2-n - (n^2-3n+2)/2 - n+1 = n^2-n - 0.5n^2+1.5n-1+n+1 = 0.5n^2+1.5n$ volte
- il test alla riga 3 viene eseguito $0.5n^2+1.5n+1$ volte

Esempio di calcolo di $T(n)$

DUE_OCCORRENZE (A)	costo	n° di volte
1. output = false	c_1	1
2. for i = 0 to A.length-2	c_2	n
3. for j = i+1 to A.length-1	c_3	$0.5n^2+1.5n+1$
4. if (A[i] == A[j])	c_4	$0.5n^2+1.5n$
5. output = true	c_5	$0.5n^2+1.5n$
6. return output	c_6	1

$$\begin{aligned} T(n) &= c_1 + c_2(n) + c_3(0.5n^2 + 1.5n + 1) + (c_4 + c_5)(0.5n^2 + 1.5n) + c_6 \\ &= n^2 (0.5c_3 + 0.5c_4 + 0.5c_5) + n (c_2 + 1.5c_3 + 1.5c_4 + 1.5c_5) + \\ &\quad (c_1 + c_3 + c_6) \end{aligned}$$

Dunque: $T(n) \in O(n^2)$, $T(n) \in \Omega(n^2) \Rightarrow T(n) \in \Theta(n^2)$

Algoritmi e complessità $O(f(n))$

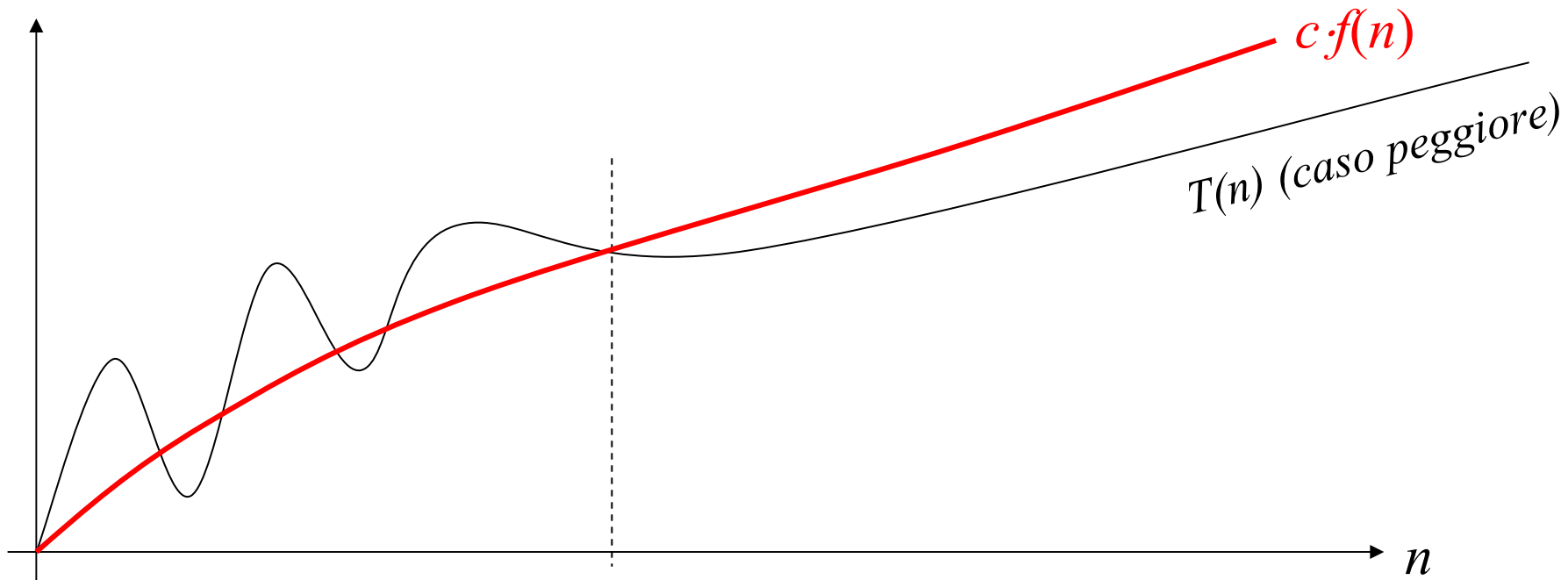
- sia $T(n)$ il tempo di esecuzione di un algoritmo A su un'istanza di dimensione n nel caso peggiore

l'algoritmo A ha *complessità temporale* $O(f(n))$ se
 $T(n) \in O(f(n))$

- diciamo anche che
 - il tempo di esecuzione dell'algoritmo A è *al più* $f(n)$
 - $f(n)$ è un *limite superiore*, o *upper-bound*, al tempo di esecuzione dell'algoritmo A
 - $f(n)$ è la quantità di tempo *sufficiente* (in ogni caso) all'esecuzione dell'algoritmo A

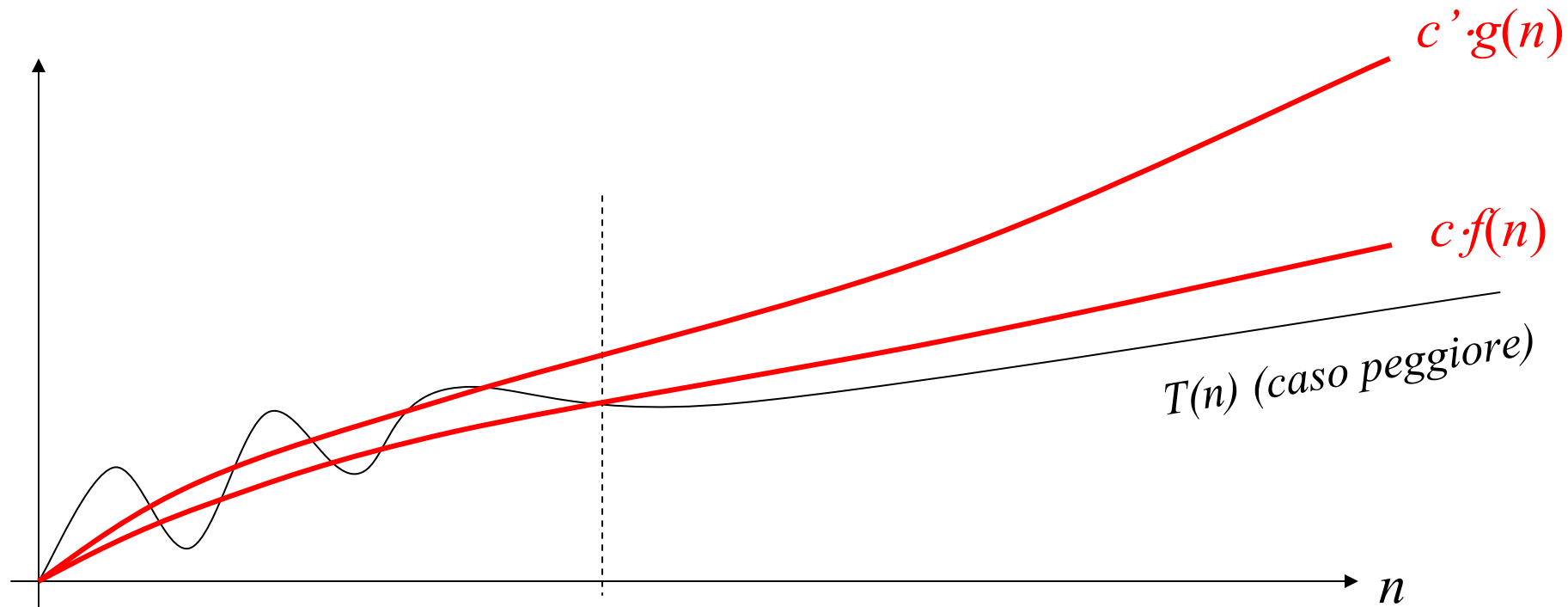
Algoritmi e complessità $O(f(n))$

l'algoritmo A ha *complessità temporale* $O(f(n))$ se
 $T(n) \in O(f(n))$



Significatività della funzione $f(n)$

- se un algoritmo ha complessità $f(n)$ allora ha anche complessità $g(n)$ per ogni $g(n)$ tale che $f(n) \in O(g(n))$
- la complessità espressa tramite la notazione O-grande è tanto più significativa quanto più $f(n)$ è stringente (piccolo)



Algoritmi e complessità $\Omega(f(n))$

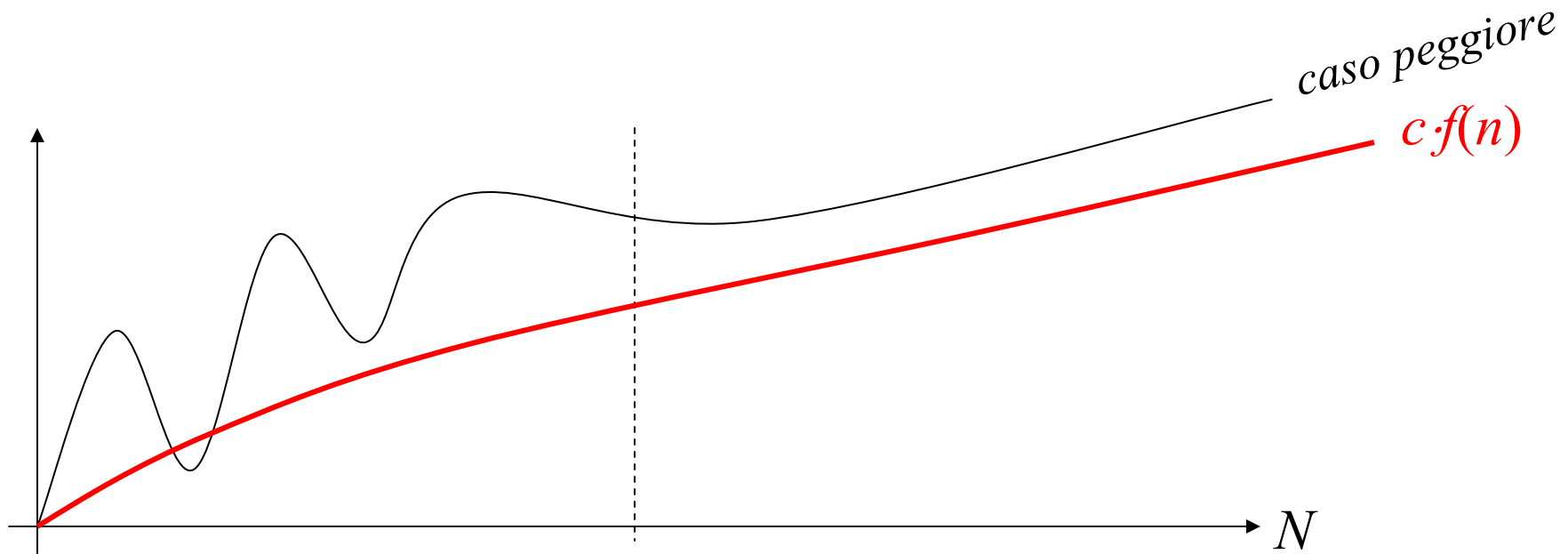
- sia $T(n)$ il tempo di esecuzione di un algoritmo A su un'istanza di dimensione n nel caso peggiore

l'algoritmo A ha *complessità temporale* $\Omega(f(n))$ se
 $T(n) \in \Omega(f(n))$

- diciamo anche che
 - il tempo di esecuzione dell'algoritmo A è *almeno* $f(n)$
 - $f(n)$ è un *limite inferiore*, o *lower-bound*, al tempo di esecuzione dell'algoritmo A
 - $f(n)$ è la quantità di tempo *necessaria* (in almeno un caso) all'esecuzione dell'algoritmo A

Algoritmi e complessità $\Omega(f(n))$

l'algoritmo A ha *complessità temporale* $\Omega(f(n))$ se
 $T(n) \in \Omega(f(n))$



Algoritmi e complessità $\Theta(f(n))$

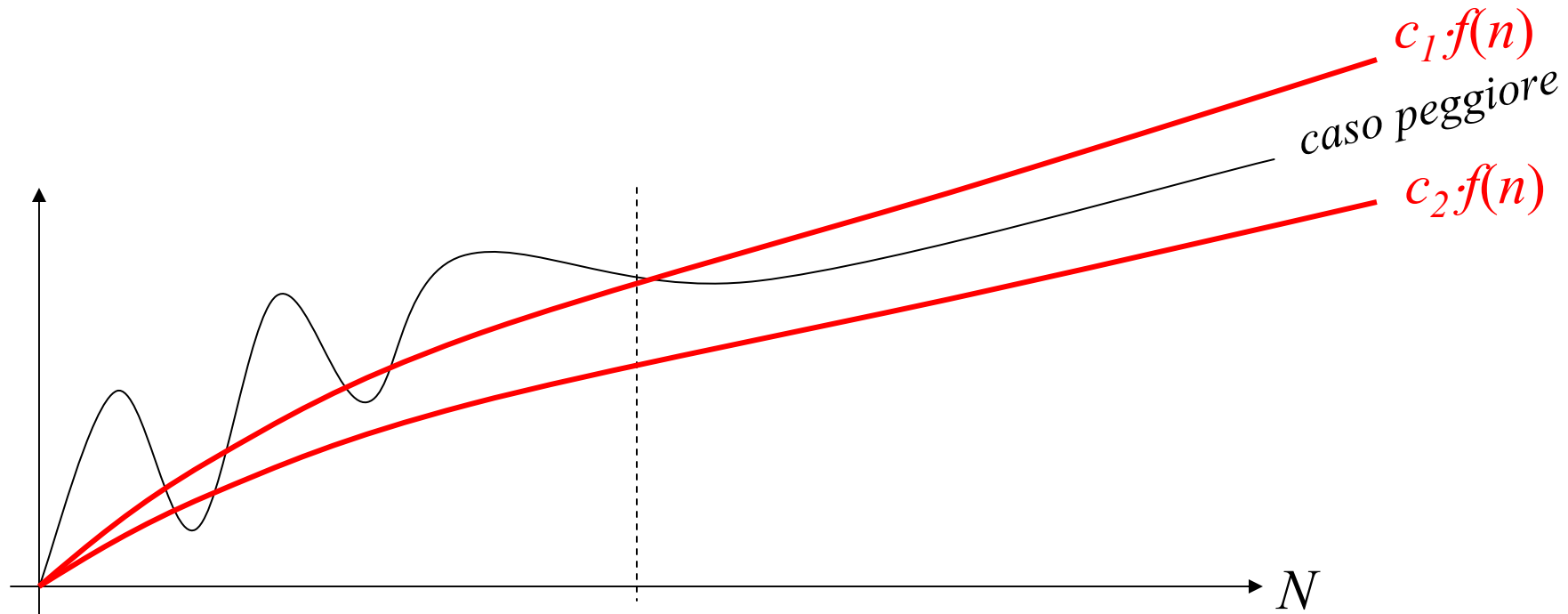
- sia $T(n)$ il tempo di esecuzione di un algoritmo A su un'istanza di dimensione n nel caso peggiore

l'algoritmo A ha *complessità temporale* $\Theta(f(n))$ se
$$T(n) \in \Theta(f(n))$$

- diciamo anche che
 - il tempo di esecuzione dell'algoritmo è $f(n)$
 - $f(n)$ è un *limite inferiore e superiore* (*lower-bound* e *upper-bound*), al tempo di esecuzione dell'algoritmo
 - $f(n)$ è la quantità di tempo *necessaria e sufficiente* all'esecuzione dell'algoritmo

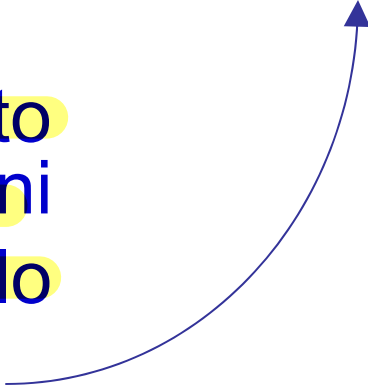
Algoritmi e complessità $\Theta(f(n))$

l'algoritmo A ha *complessità temporale* $\Theta(f(n))$ se
 $T(n) \in \Theta(f(n))$



Strategie di analisi più efficienti

DUE_OCCORRENZE (A)	n° di volte	ordine
1. output = false	1	$\Theta(1)$
2. for i = 0 to A.length-2	n	$\Theta(n)$
3. for j = i+1 to A.length-1	$0.5n^2+1.5n+1$	$\Theta(n^2)$
4. if (A[i] == A[j])	$0.5n^2+1.5n$	$\Theta(n^2)$
5. output = true	$0.5n^2+1.5n$	$\Theta(n^2)$
6. return output	1	$\Theta(1)$

- la regola della somma mi assicura che l'andamento asintotico di una somma di termini può essere ottenuto considerando l'andamento asintotico dei singoli termini
 - una strategia più efficiente si basa, dunque, sul calcolo diretto del costo asintotico di ogni riga
- 

Calcolo efficiente del costo asintotico

- **istruzioni semplici**
 - tempo di esecuzione costante: $\Theta(1)$
- **sequenza (necessariamente finita) di istruzioni semplici**
 - tempo di esecuzione costante: $\Theta(1)$
- **sequenza di istruzioni generiche**
 - somma dei tempi di esecuzione di ciascuna istruzione

Istruzioni condizionali

1. if <condizione> then
2. <parte-then>
3. else
4. <parte-else>

per calcolare $T(n)$
occorrerebbe sapere se
la condizione si verifica o
meno

- troviamo un limite superiore O -grande al tempo di esecuzione $T(n)$ come somma dei costi seguenti
 - costo O -grande della valutazione della condizione
 - costo O -grande maggiore tra <parte-then> e <parte-else>
- troviamo un limite inferiore Ω al tempo di esecuzione $T(n)$ come somma dei costi seguenti
 - costo Ω della valutazione della condizione
 - costo Ω minore tra <parte-then> e <parte-else>

Istruzioni ripetitive

- il nostro pseudocodice ci offre tre istruzioni ripetitive
 - `for`, `while` e `repeat`
- per il limite superiore O-grande occorre determinare
 - un limite superiore $O(f(n))$ al numero di iterazioni del ciclo
 - un limite superiore $O(g(n))$ al tempo di esecuzione di ogni iterazione
 - si compone del costo dell'esecuzione del blocco di istruzioni più il costo di esecuzione del test
- il costo del ciclo sarà: $O(g(n) \cdot f(n))$
- analogamente sarà: $\Omega(g'(n) \cdot f'(n))$
 - dove le iterazioni sono $\Omega(g'(n))$ ed il costo di una iterazione è $\Omega(f'(n))$

Istruzioni ripetitive: esempio

FACT (n)

1. `f = 1`

2. `for k = 2 to n`

3. `f = f * k`

4. `return f`

- numero di iterazioni del ciclo for: $\Theta(n)$
- costo di una singola iterazione: $\Theta(1)$
- costo complessivo del ciclo for: $\Theta(n \cdot 1) = \Theta(n)$
- costo complessivo della procedura: $\Theta(n)$

Istruzioni ripetitive annidate

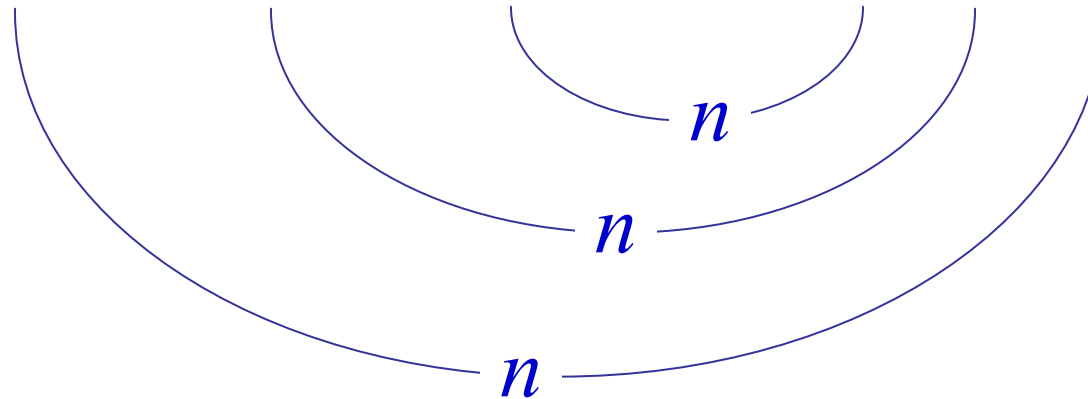
```
2. for i = 0 to n
3.   for j = i+1 to n
4.     <istruzione o blocco>
```

Un caso frequente è quando il secondo ciclo viene eseguito meno volte del primo

Quante volte viene eseguita l'istruzione?

Abbiamo $n/2$ contributi di valore n

$$(n-1) + (n-2) + (n-3) + \dots + 3 + 2 + 1$$



$$n/2 \times n = n^2/2 = \Theta(n^2)$$

Chiamata a funzione o procedura

1 .	P (...)
2 .	...
3 .	Q (...)
4 .	...

supponiamo che un programma P
invochi la procedura Q

- sia $T_Q(n)$ il tempo di esecuzione della procedura Q
- il tempo di esecuzione dell'invocazione della procedura Q in P è $T_Q(m)$, dove m è la dimensione dell'input passato alla procedura Q
 - attenzione: occorre determinare la relazione tra m e la dimensione n dell'input di P

Esempio di chiamata a funzione

```
SUM_OF_FACT (n)
```

```
1. sum = 0
```

```
2. for m = 0 to n
```

```
3.     sum = sum + FACT (m)
```

```
4. return sum
```

- il corpo del ciclo for ha complessità $\Theta(1) + \Theta(m) = \Theta(m)$
- il ciclo viene eseguito n volte
- il costo complessivo del ciclo è dunque:
 $n + (n-1) + (n-2) + \dots + 2 + 1 = \Theta(n^2)$
- il costo totale è $\Theta(n^2)$

Limiti del modello

- il calcolo asintotico della complessità non produce sempre una stima soddisfacente del tempo di calcolo di un algoritmo
- abbiamo almeno tre eccezioni
 - situazioni in cui non si applica il modello RAM
 - comportamento dell'algoritmo per input di piccole dimensioni
 - impatto delle costanti

Problemi derivanti dal modello RAM

FACT (n)

1. $f = 1$

2. for $k = 2$ **to** n

3. $f = f * k$

4. return f

- il modello RAM prevede che i registri della memoria (le variabili dello pseudocodice) possano contenere numeri arbitrariamente grandi
- in questo esempio la variabile f cresce in maniera drammatica
 - 1, 2, 6, 24, 120, 720, 5.040, 40.320, 362.880, 3.628.800, 39.916.800, 479.001.600, 6.227.020.800, ...
- per tutti i linguaggi in cui gli interi hanno un valore massimo, si raggiunge presto tale limite
- per tutti i linguaggi in cui gli interi hanno una rappresentazione telescopica, la grandezza del numero influenza il costo delle operazioni

Problemi derivanti dal modello RAM

FACT (n)

1. $f = 1$

2. for $k = 2$ **to** n

3. $f = f * k$

4. return f

- potremmo modificare il modello considerando il fatto che la rappresentazione di un numero n necessita di un numero di bit $k = \lceil \log_2 n \rceil$
- se usassimo come misura dell'input il numero k di bit necessari per rappresentare n in binario avremmo $k = \lceil \log_2 n \rceil$ e il costo complessivo sarebbe $\Theta(n) = \Theta(2^k)$
 - secondo questo modello più realistico l'algoritmo avrebbe un tempo esponenziale ammesso che il linguaggio preveda una rappresentazione dinamica degli interi

Input di piccole dimensioni

- il calcolo asintotico ci dice cosa succede al crescere della dimensione n dell'input
- in alcune circostanze applicative sappiamo che il valore di n è limitato superiormente da una costante
- in questi casi un algoritmo asintoticamente inefficiente potrebbe essere appetibile, perché avrebbe comunque un tempo complessivo costante

Impatto delle costanti

- il calcolo asintotico della complessità trascura le costanti moltiplicative
- è chiaro però che nei contesti applicativi queste costanti possono essere significative
 - se le costanti fossero ridotte della metà, anche i tempi di calcolo complessivi lo sarebbero
- caso limite: un algoritmo $\Theta(1)$ per cui la costante è 10.000 anni
 - avrebbe tempi di calcolo indipendenti dalla dimensione dell'input
 - ovviamente sarebbe un algoritmo proibitivo

Algoritmi e Strutture di Dati

Esercitazioni in linguaggio C

Array e puntatori

m.patrignani

Nota di copyright

- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Richiami di linguaggio C

- Puntatori in linguaggio C
- Gli operatori $*$ e $\&$
- Algebra dei puntatori
- Array
- Allocazione statica e dinamica di array

Puntatori in linguaggio C

- Molti linguaggi supportano i riferimenti
 - un riferimento è una variabile che viene utilizzata per identificare un'altra variabile (o un oggetto)



- Nel linguaggio C i riferimenti sono i puntatori
 - un puntatore è una variabile (cioè una cella di memoria con un tipo e un nome) che contiene l'indirizzo di un'altra cella di memoria
 - per esempio l'indirizzo di un'altra variabile



L'operatore *

- Nel linguaggio C il tipo di una variabile puntatore si ottiene dal tipo della variabile puntata seguito dal simbolo * (leggi “star”)
- Esempi
 - un puntatore ad intero ha tipo `int*`
 - leggi “int star”
 - è indifferente la presenza di spazi o meno tra la stringa “int” e il simbolo “*”
 - un puntatore a carattere ha tipo `char*`
 - leggi “char star”
 - un puntatore a puntatore ad intero ha tipo `int**`
 - leggi “int star star”

Esempi di dichiarazioni di puntatori

- La variabile `a` è un puntatore ad intero

```
int* a;
```

- La variabile `b` è un puntatore a puntatore ad intero

```
int** b;
```

- La variabile `c` è un puntatore ad un float

```
float* c;
```

- La variabile `d` è un puntatore ad una struttura

```
struct {  
    int minimo;  
    int massimo;  
}* d;
```

La costante `NULL`

- Il valore costante `NULL` è un valore convenzionale che si suppone assegnato ad un puntatore che non contiene alcun indirizzo significativo
 - nella rappresentazione interna `NULL` corrisponde al valore 0 (zero), che non è legittimo per un indirizzo di memoria a disposizione dell'utente

La costante NULL

- Quando viene dichiarato un puntatore senza inizializzarlo, si suppone che il suo valore sia NULL

```
int* a; /* a è ancora uguale a NULL */
```

- Questo può essere anche esplicitato da un'assegnazione

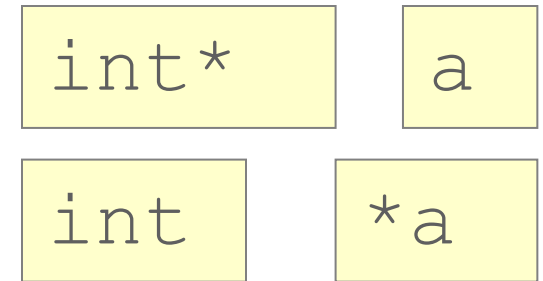
```
int* a = NULL;
```

Due diverse interpretazioni dell'operatore *

- L'espressione `int* a` può essere interpretata in due modi equivalenti

`int* a`

1. la variabile `a` è di tipo `int*`
2. `*a`, cioè "l'oggetto puntato da `a`" è di tipo `int`



- Infatti, l'operatore `*` anteposto ad un indirizzo di memoria identifica il contenuto della memoria stessa
 - `*(a+b)` è il contenuto della cella di memoria che si trova in posizione `a+b`

L'operatore &

- L'operatore & (e commerciale) anteposto ad una variabile ne estrae l'indirizzo
- Esempio
 - l'indirizzo della variabile intera "a" è "&a"
 - "&a" è un'espressione di tipo "int*"
- Non ha senso utilizzare l'operatore & con espressioni generiche
 - la scrittura "& (a+b)" non ha senso perché "a+b" non è una variabile, ma solo un valore
 - tecnicamente: non ha left-value ma solo right-value

Algebra degli operatori * e &

■ Regola mnemonica

- ogniqualevolta si incontra l'operatore * lo si può sostituire con "il puntato da" oppure "il contenuto di"
- ogniqualevolta si incontra l'operatore & lo si può sostituire con "l'indirizzo di"

```
int a;  
int* b;  
b = &a;      /* "l'indirizzo di a" */  
*b = 3;      /* "il puntato da b" */
```


Algebra degli operatori * e &

- I due operatori possono essere combinati insieme in modo spesso complesso

```
int a = 3;  
int* b = &a;  
int** c = &b;  
*b = *b+1;           /* ora a vale 4 */  
**c = **c+1          /* ora a vale 5 */
```

- Osservazione

- *x identifica un right value (cioè un valore)
- &y identifica un left value (cioè una variabile)

Passaggio di parametri

- Tutti i passaggi di parametri in linguaggio C sono per valore
 - esempio

```
...  
int a = 3;  
mia_funzione(a); /* viene passato 3 */  
                /* qui "a" vale sempre 3 */  
...
```

```
void mia_funzione(int a){  
    a = a + 1; /* non ha side effects */  
}
```

Passaggio di parametri

- Passaggio di parametri “per riferimento”
 - in realtà il passaggio è sempre per valore
 - esempio

```
...  
int a = 3;  
mia_funzione(&a);    /* l'indirizzo */  
                    /* qui "a" vale 4 */  
...
```

```
void mia_funzione(int* a) {  
    (*a) = (*a) + 1;  /* side effects */  
}
```

Gli array

- Un array è una sequenza di variabili omogenee, tutte memorizzate in celle contigue di memoria e indicizzate con un indice intero
- In linguaggio C un array si dichiara premettendo il tipo degli elementi e posponendo il numero degli elementi tra parentesi quadre

```
int a[10];    /* a array di 10 interi */  
float b[5];   /* b array di 5 float */
```

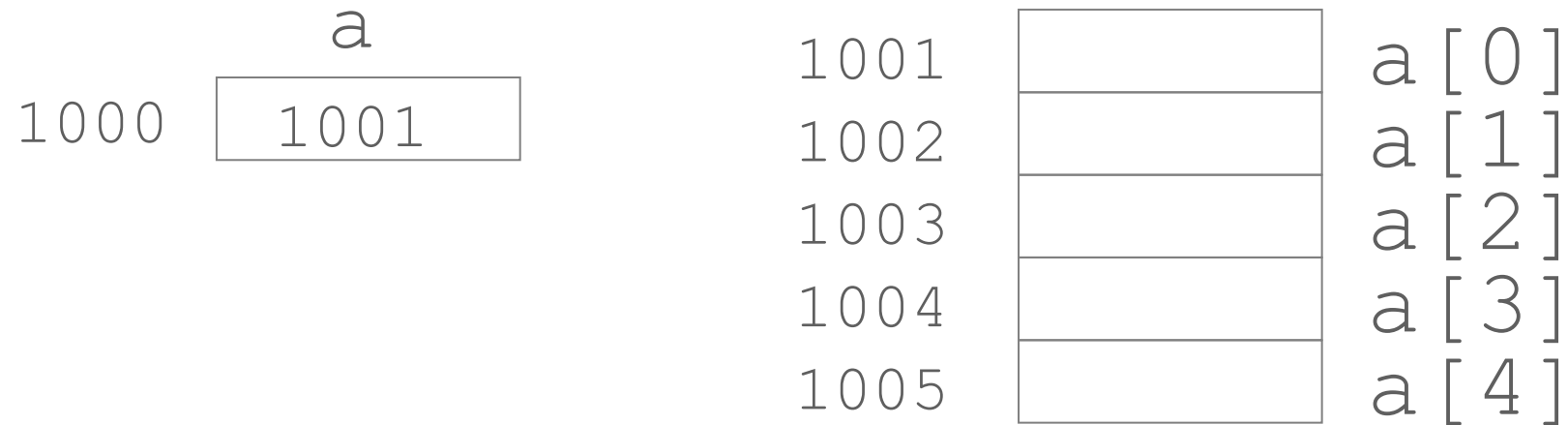
- in ansi C il numero di elementi deve essere una costante, non può essere una variabile o un'espressione generica

```
int n=10;     /* numero di celle */  
int a[n];     /* errore! */
```

Array e allocazione della memoria

- Cosa succede in memoria quando dichiariamo un array?

```
int a[5]; /* a array di 5 interi */
```



- Quante celle di memoria vengono allocate quando dichiaro un array di 5 interi?

Array e puntatori

- Un array è a tutti gli effetti un puntatore
 - è l'indirizzo della prima cella di memoria dell'array
- In linguaggio C l'identità tra array e puntatori è esplicita

```
int a[5];      /* a è di tipo int* */  
int* b = a;    /* b ha lo stesso valore  
                di a */
```

- Tuttavia un array è un puntatore costante
 - non è legittimo sovrascrivere o modificare l'indirizzo di memoria associato ad un array

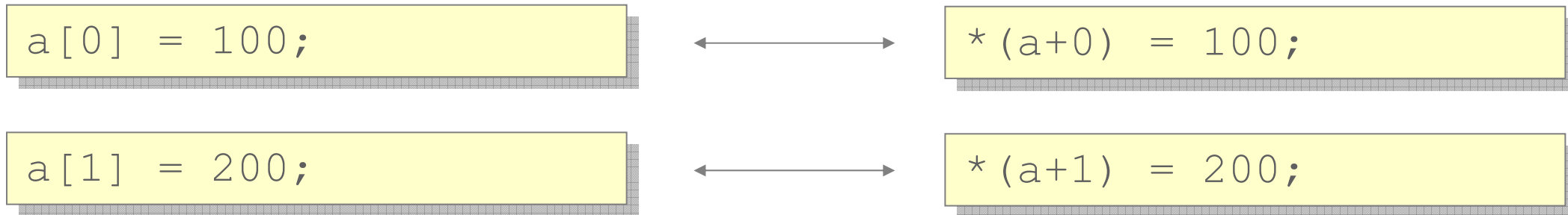
Incremento di un puntatore

- Incrementando un puntatore si salta una porzione di memoria pari alla dimensione dell'oggetto puntato

```
int a[5];      /* a array di 5 interi */
int* b = a;    /* b punta alla prima
                cella di a */
b = b + 1;     /* b punta alla seconda
                cella di a */
b++;           /* b punta alla terza
                cella di a */
```

Interpretazione delle parentesi quadre

- Date due espressioni x e y , il costrutto $x[y]$ è equivalente all'operazione $*(x+y)$
- Quindi queste espressioni sono equivalenti



- Si può verificare l'equivalenza persino delle seguenti espressioni



- infatti $*(a+1)$ è uguale a $*(1+a)$

Tempi di accesso agli array

- Consideriamo il tempo di esecuzione di

```
a[0] = 100;
```



```
*(a+0) = 100;
```

- è evidente che le due operazioni sulla destra si possono fare in tempo costante

```
a[9999] = 200;
```



```
*(a+9999) = 200;
```

- il tempo di accesso è costante anche se si afferisce ad una cella lontana dalla prima
- Il tempo di accesso ad un array è sempre $\theta(1)$

Allocazione statica degli array

- La dichiarazione di array che abbiamo visto alloca le celle dell'array direttamente sullo stack

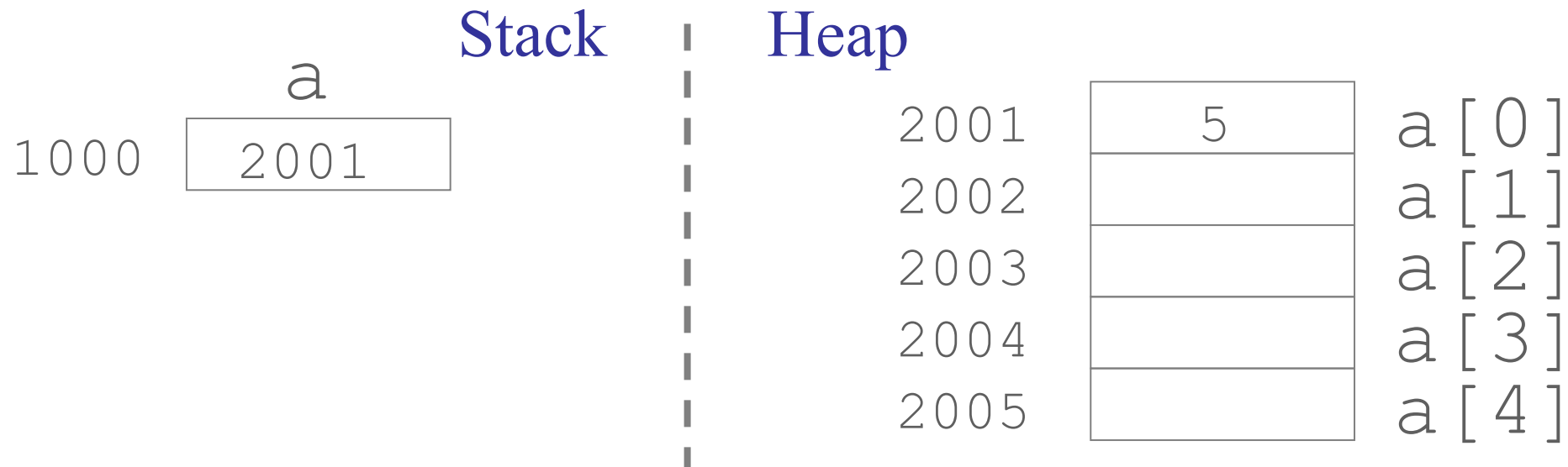
```
int a[5]; /* a è nello stack.  
          Anche a[0],a[1],... ,a[4]  
          sono nello stack */
```

- Questo comporta due vincoli
 - il numero delle celle non può essere modificato
 - il valore di `a` non può essere modificato

Allocazione dinamica degli array

- Nell'allocazione dinamica l'array viene posto nello heap tramite la funzione `calloc`

```
int* a;          /* a è nello stack */  
a = (int *)calloc(5, sizeof(int));  
a[0] = 5;
```



Vantaggi della dichiarazione dinamica

- La dichiarazione dinamica degli array comporta diversi vantaggi
 - il numero di celle dell'array può essere specificato tramite una variabile o un'espressione generica
 - l'indirizzo dell'array può essere modificato
 - il numero delle celle dell'array può essere modificato

```
int* a;  
a = (int *)calloc(5, sizeof(int));  
a[0] = 100;  
...  
a = realloc(a, 10 * sizeof(int));  
a[8] = a[0];          /* cioè 100 */
```

Esercizi

1. Implementa in linguaggio C uno stack di interi con le funzioni `NEW_STACK`, `IS_EMPTY`, `PUSH`, e `POP` e con la gestione telescopica della memoria
 - operazione `PUSH` con complessità ammortizzata $O(1)$
2. Implementa in linguaggio C una coda di interi con le funzioni `NEW_QUEUE`, `IS_EMPTY`, `ENQUEUE`, e `DEQUEUE` e con la gestione telescopica della memoria
 - operazione `ENQUEUE` con complessità ammortizzata $O(1)$

Algoritmi e Strutture di Dati

Notazione asintotica - esercizi

M. Patrignani

Nota di copyright

- Queste slides sono protette dalle leggi sul copyright
- Il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- Le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- Ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- Gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- Questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Esercizi sulla notazione O-grande

1. la funzione $f(n) = n^2 + n^4$ appartiene ad $O(n^3)$? *No*
2. la funzione $f(n) = 0.5n^2 + 3n$ appartiene ad $O(n)$? *No*
3. la funzione $f(n) = 4n^2 + 3 \log(n)$ appartiene ad $O(n)$? *No*
4. la funzione $f(n) = n \log(n)$ appartiene ad $O(n^2)$? *Sì*
5. la funzione $f(n) = 5$ (costante per ogni n) appartiene ad $O(n)$? *Sì*
6. la funzione $f(n) = 5$ (costante per ogni n) appartiene ad $O(1)$? *Sì*
7. una funzione appartenente ad $O(n^2)$ appartiene anche ad $O(n \log(n))$? *Non è detto*
8. una funzione appartenente ad $O(n)$ appartiene anche ad $O(n \log(n))$? *Sì*
9. se $h(n)$ e $k(n)$ sono due funzioni appartenenti a $O(g(n))$, allora la funzione $f(n) = 2h(n) + 2k(n)$ appartiene anch'essa ad $O(g(n))$? *Sì*
10. la funzione $f(n) = \log(n)$ appartiene ad $O(2^n)$? *Sì*

Esercizi sulla notazione O-grande

11. una funzione appartenente a $O(\log n)$ appartiene anche ad $O(n)$? *SI*
12. una funzione appartenente a $O(n^2)$ appartiene anche ad $O(n^3)$? *SI*
13. se la funzione $f(n) \in O(g(n))$ allora è sempre vero che $g(n) \in O(f(n))$? *NO*
14. se la funzione $f(n) \notin O(g(n))$ allora necessariamente $g(n) \in O(f(n))$? *NO*
15. se $h(n)$ e $k(n)$ sono due funzioni appartenenti a $O(g(n))$, allora la funzione $f(n) = h(n) + k(n)$ appartiene anch'essa ad $O(g(n))$? *SI*
16. la funzione $f(n) = n^3 + 3n$ appartiene ad $O(n^2)$? *NO*
17. $O(23) = O(1)$? *SI*
18. la funzione $f(n) = 1$ (costante per tutti gli n) appartiene ad $O(n)$? *SI*
19. la funzione $f(n) = 2n + 3n^2$ appartiene ad $O(n)$? *NO*

Esercizi sulla notazione Ω

SI NO SI
SI SI SI
SI

1. la funzione $f(n) = n^2 + n^4$ appartiene ad $\Omega(n^3)$? SI
2. la funzione $f(n) = 3$ (costante per ogni n) appartiene ad $\Omega(n)$? NO
3. una funzione appartenente ad $\Omega(n^3)$ appartiene anche ad $\Omega(n^2)$? SI
4. la somma di due funzioni appartenenti ad $\Omega(n)$ produce sempre una funzione appartenente ad $\Omega(n)$? SI
5. se la funzione $f(n) \in O(g(n))$, allora necessariamente la funzione $g(n) \in \Omega(f(n))$? SI
6. la funzione n^2 appartiene ad $\Omega(n)$? SI
7. la funzione $f(n) = 4n + 6n^2 + 8n^3$ appartiene ad $\Omega(n^2)$? SI

Esercizi di ricapitolazione

NO, IL TERMINE MAGGIORE È n^4
SI SI SI SI
SI NO SI

1. la funzione $f(n) = n^2 + n^4$ appartiene a $\Theta(n^3)$? NO
2. la funzione $f(n) = n^2 + n^3$ appartiene a $\Theta(n^3)$? SI
3. le funzioni appartenenti ad $O(g(n))$ sono quelle limitate superiormente da $g(n)$, cioè che crescono “al più come $g(n)$ ”? SI
4. le funzioni appartenenti ad $\Omega(g(n))$ sono quelle limitate inferiormente da $g(n)$, cioè che crescono “almeno come $g(n)$ ”? SI
5. la somma di due funzioni appartenenti a $\Theta(n)$ produce sempre una funzione appartenente a $\Theta(n)$? SI
6. se la funzione $f(n) \in O(g(n))$ e $f(n) \in \Omega(g(n))$ allora $f(n) \in \Theta(g(n))$? SI
7. $O(g(n))$ denota l'insieme delle funzioni limitate inferiormente da $g(n)$, che cioè crescono “almeno come $g(n)$ ”? NO
8. se $f(n)$ appartiene a $\Theta(g(n))$, allora $g(n)$ appartiene a $\Theta(f(n))$? SI

Soluzioni degli esercizi sulla notazione O-grande

1. No

2. No

3. No

4. Sì

5. Sì

6. Sì

7. Non è detto

- per esempio n^2

8. Sì

9. Sì

10. Sì

11. Sì

12. Sì

13. No

14. No

- esistono le
incommensurabili

15. Sì

16. No

17. Sì

18. Sì

19. No

Algoritmi e Strutture di Dati

Il problema dell'ordinamento
Algoritmi greedy e algoritmi iterativi

M. Patrignani

Nota di copyright

- Queste slides sono protette dalle leggi sul copyright
- Il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- Le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- Ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- Gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- Questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Panoramica

- il problema dell'ordinamento
- gli algoritmi greedy
 - l'algoritmo selection sort
- gli algoritmi iterativi
 - l'algoritmo insertion sort

Il problema dell'ordinamento

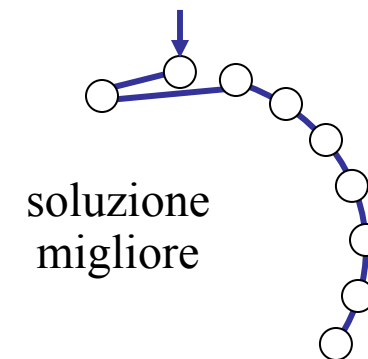
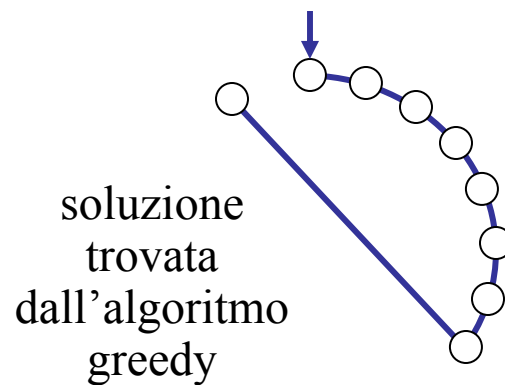
- **input:** una sequenza di n numeri $\langle a_1, a_2, \dots, a_n \rangle$
- **output:** una permutazione $\langle a_1', a_2', \dots, a_n' \rangle$ della sequenza tale che $a_1' \leq a_2' \leq \dots \leq a_n'$
- **esempio**
 - un'istanza
 $\langle 31, 41, 59, 26, 41, 58 \rangle$
 - la soluzione dell'istanza qui sopra
 $\langle 26, 31, 41, 41, 58, 59 \rangle$
- nel seguito supporremo che l'istanza sia fornita tramite un array A con n posizioni

Selection sort

La tecnica greedy

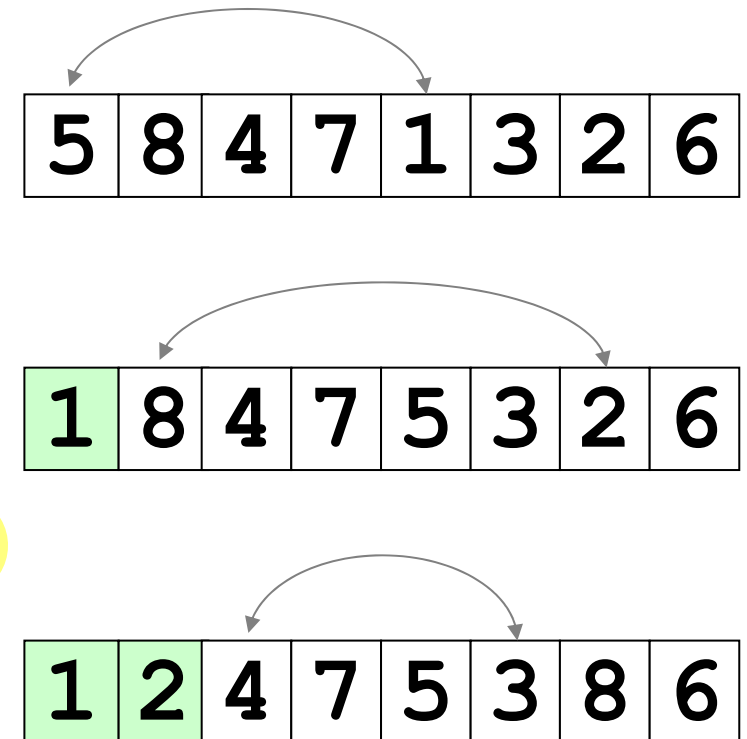
La tecnica greedy

- la tecnica greedy (golosa) consiste nel scegliere sempre l'alternativa che al momento sembra più appetibile
 - corrisponde ad eseguire una scelta localmente ottima
 - ciò non sempre comporta una scelta globalmente ottima
- esempio in cui greedy non dà la soluzione ottima
 - trovare il cammino più breve tra n città
 - algoritmo greedy: muoviti sempre verso la città più vicina non ancora visitata



Algoritmo selection sort

- utilizza una tecnica greedy per ordinare un array
- strategia generale
 - seleziona l'elemento più piccolo e mettilo al primo posto
 - seleziona l'elemento più piccolo dei rimanenti e mettilo al secondo posto
 - ...



Algoritmo SELECTION_SORT

- pseudocodice dell'algoritmo
 - si fa uso di due cicli annidati

SELECTION_SORT(A)

1. **for** $i = 0$ **to** $A.length-2$

2. $min = i$ ▷ indice elemento minimo in $A[i..n-1]$

3. **for** $j = i + 1$ **to** $A.length-1$ ▷ scorro l'array

4. **if** $A[j] < A[min]$ ▷ devo aggiornare min

5. $min = j$

6. $temp = A[i]$ ▷ scambio $A[i]$ con $A[min]$

7. $A[i] = A[min]$

8. $A[min] = temp$

Complessità del SELECTION_SORT

SELECTION_SORT(A)

1. **for** $i = 0$ **to** $A.length-2$

2. $min = i$ ▷ indice elemento minimo in $A[i..n-1]$

3. **for** $j = i + 1$ **to** $A.length-1$ ▷ scorro l'array

4. **if** $A[j] < A[min]$ ▷ devo aggiornare min

5. $min = j$

6. $temp = A[i]$ ▷ scambio $A[i]$ con $A[min]$

7. $A[i] = A[min]$

8. $A[min] = temp$

- i valori dell'input non modificano il numero delle iterazioni del ciclo esterno e del ciclo interno
 - quindi il caso migliore, il caso peggiore ed il caso medio hanno la stessa complessità

Complessità del SELECTION_SORT

SELECTION_SORT(A)

```
1. for i = 0 to A.length-2
2.   min = i    ▷ indice elemento minimo in A[i..n-1]
3.   for j = i + 1 to A.length-1 ▷ scorro l'array
4.     if A[j] < A[min]          ▷ devo aggiornare min
5.       min = j
6.   temp = A[i]                ▷ scambio A[i] con A[min]
7.   A[i] = A[min]
8.   A[min] = temp
```

- l'algoritmo esegue $O(n)$ cicli esterni e $O(n)$ cicli interni
 - dunque SELECTION-SORT ha complessità $O(n^2)$
- la riga 4 viene eseguita $(n-1)+(n-2)+\dots+1 = [n(n-1)]/2$ volte
 - dunque SELECTION-SORT ha complessità $\Omega(n^2)$
- il tempo di esecuzione dell'algoritmo è $\Theta(n^2)$

Algoritmi che operano *in loco*

- gli algoritmi che operano *in loco* non necessitano di copiare l'input in strutture di dati diverse da quella utilizzata per l'input
 - questa caratteristica è utile per input di grosse dimensioni
- l'algoritmo `SELECTION_SORT` opera in loco
 - gli elementi vengono solo scambiati
 - l'algoritmo necessita di una quantità di memoria costante oltre a quella per memorizzare A
 - la memoria utilizzata dall'algoritmo è $O(1)$

Algoritmi di ordinamento *stabili*

- un algoritmo di ordinamento si dice *stabile* se non modifica l'ordine degli elementi che hanno lo stesso valore
 - in alcune applicazioni ciò può essere utile
 - quando agli elementi sono collegati dei dati satellite
 - quando gli elementi con la stessa chiave hanno una posizione reciproca significativa
- l'algoritmo `SELECTION_SORT` è stabile
 - se $A[j] = A[k]$ con $j < k$, allora $A[j]$ viene selezionato per primo

Insertion sort

Un algoritmo incrementale

Gli algoritmi incrementali

- si basano sulla seguente osservazione
 - la soluzione di un'istanza di dimensione $n-1$ può essere utile per risolvere un'istanza di dimensione n

- esempio

a) istanza di dimensione cinque:

5	2	4	6	1
---	---	---	---	---

b) istanza di dimensione sei:

5	2	4	6	1
---	---	---	---	---

 +

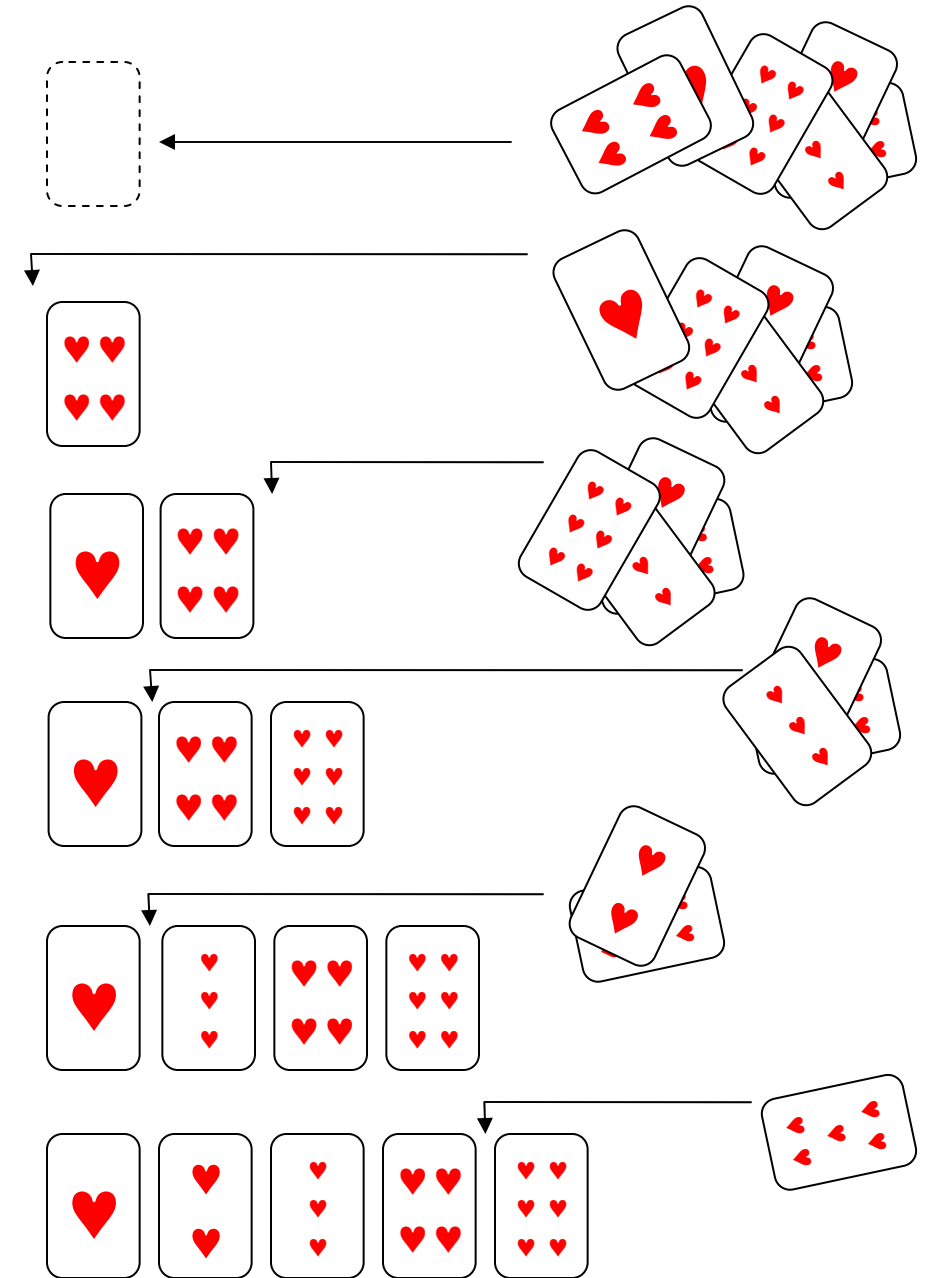
3

la soluzione di (a) mi aiuta a risolvere (b) ?

$$\begin{array}{|c|c|c|c|c|} \hline 1 & 2 & 4 & 5 & 6 \\ \hline \end{array} + \begin{array}{|c|} \hline 3 \\ \hline \end{array} = ?$$

Insertion sort

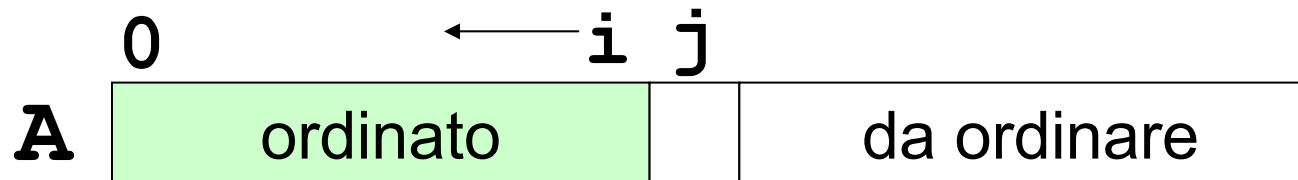
- manteniamo un sottoinsieme ordinato di elementi
 - cominciando da un singolo elemento
- inseriamo un elemento alla volta
 - il sottoinsieme ordinato cresce
 - i suoi elementi vengono traslati per far posto al nuovo elemento
- quando tutti gli elementi sono inseriti il problema è risolto



INSERTION_SORT

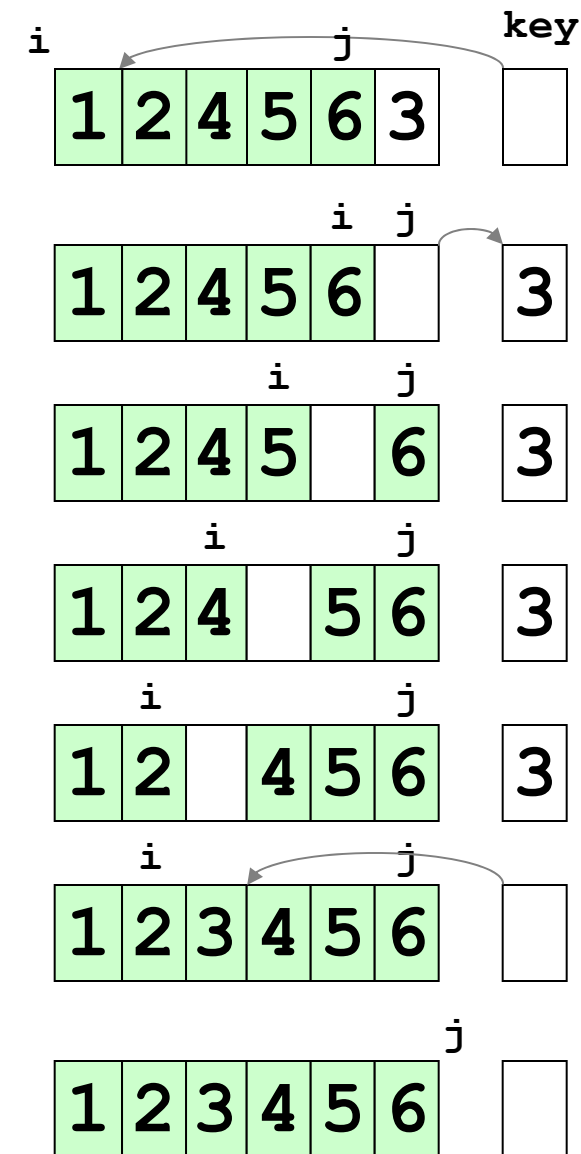
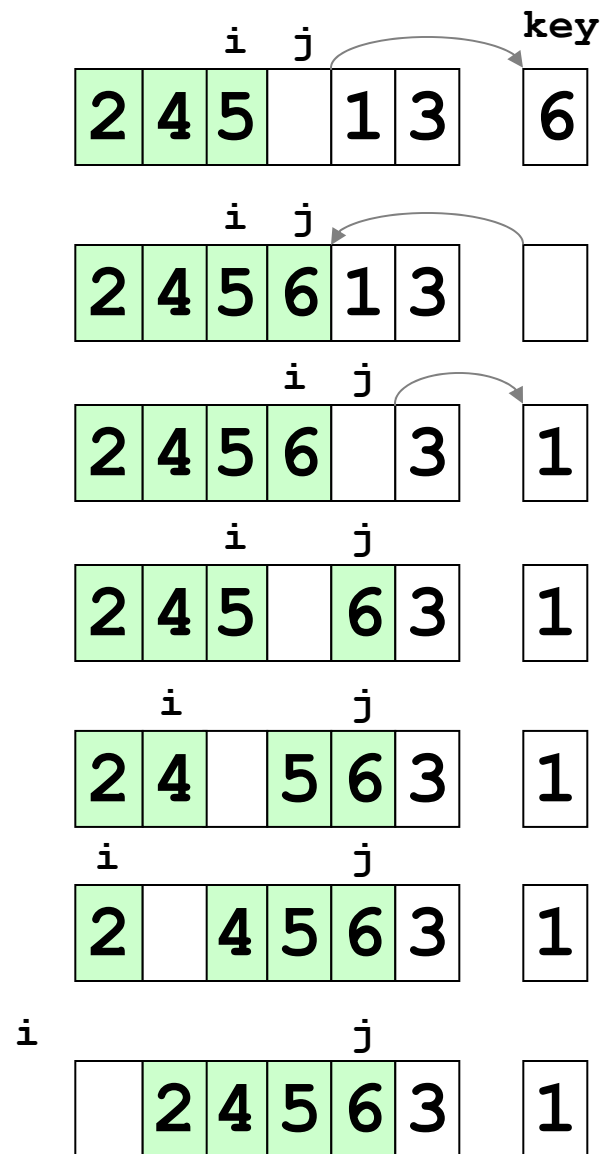
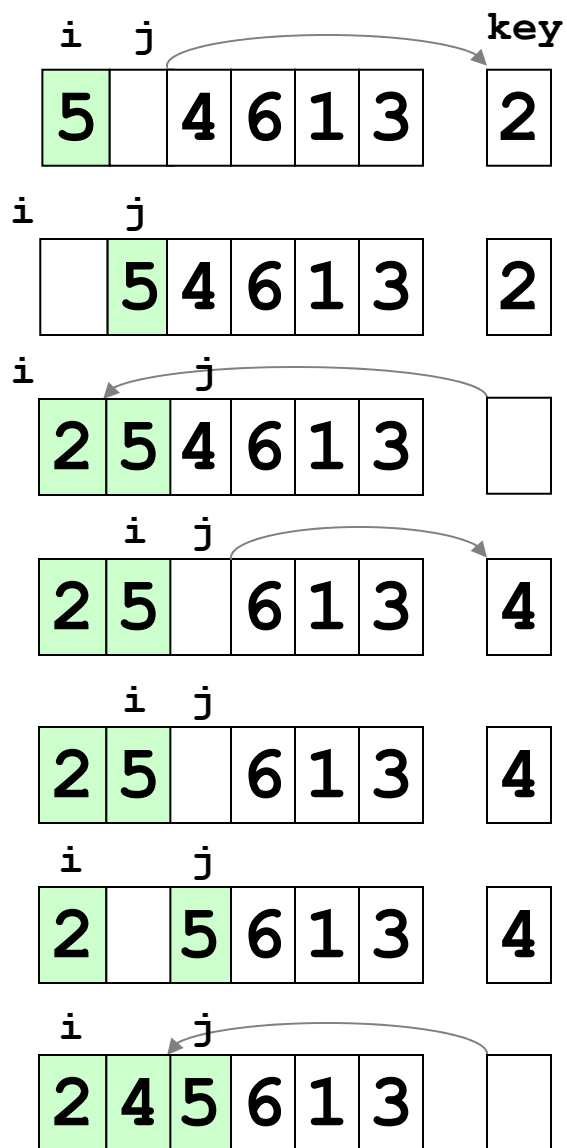
INSERTION_SORT (A)

```
1. for j = 1 to A.length-1
2.     key = A[j]
3.     ▷ inserisce key nella sequenza ordinata A[0..j-1]
4.     i = j-1
5.     while i > -1 and A[i] > key
6.         A[i+1] = A[i]
7.         i = i-1
8.     A[i+1] = key
```



Insertion sort con input

5	2	4	6	1	3
---	---	---	---	---	---



Proprietà di INSERTION_SORT

■ INSERTION_SORT è stabile

- supponiamo che per $i < j$ si abbia $A[i] = A[j]$
- quando l'algoritmo inserisce $A[j]$, l'elemento $A[i]$ è già stato inserito
- l'algoritmo inserisce $A[j]$ dopo $A[i]$ preservando dunque la loro posizione reciproca originale

■ INSERTION_SORT opera in loco

- richiede $O(1)$ memoria aggiuntionale rispetto alla memoria utilizzata per l'input

Complessità nel caso peggiore

INSERTION_SORT (A)

```
1. for j = 1 to A.length-1
2.     key = A[j]
3.     ▷ inserisce key nella sequenza ordinata A[0..j-1]
4.     i = j-1
5.     while i > -1 and A[i] > key
6.         A[i+1] = A[i]
7.         i = i-1
8.     A[i+1] = key
```

- il ciclo esterno viene eseguito $O(n)$ volte
- nel caso peggiore
 - l'elemento corrente deve essere inserito sempre al primo posto
 - l'array A in input è ordinato in maniera decrescente
 - il numero delle operazioni è $\Theta(n^2)$

Complessità nel caso migliore

INSERTION_SORT (A)

```
1. for j = 1 to A.length-1
2.     key = A[j]
3.     ▷ inserisce key nella sequenza ordinata A[0..j-1]
4.     i = j-1
5.     while i > -1 and A[i] > key
6.         A[i+1] = A[i]
7.         i = i-1
8.     A[i+1] = key
```

- il ciclo esterno viene eseguito comunque $O(n)$ volte
- nel caso migliore
 - l'elemento corrente è già posizionato al punto giusto
 - l'array A in input è già ordinato
 - il numero delle operazioni è $\Theta(n)$

Complessità nel caso medio

INSERTION_SORT (A)

```
1. for j = 1 to A.length-1
2.     key = A[j]
3.     ▷ inserisce key nella sequenza ordinata A[0..j-1]
4.     i = j-1
5.     while i > -1 and A[i] > key
6.         A[i+1] = A[i]
7.         i = i-1
8.     A[i+1] = key
```

- il ciclo esterno viene eseguito comunque $O(n)$ volte
- nel caso medio
 - l'elemento corrente va posizionato nel mezzo di $A[0..j]$
 - il numero delle operazioni è $\Theta(n^2)$

Proprietà dell'insertion sort

- efficiente su piccole istanze
 - più efficiente in pratica che il selection sort
 - nel caso migliore ha complessità lineare
 - si può calcolare che la complessità media è $n^2/4$
- adattivo
 - veloce su istanze già parzialmente ordinate
 - complessità $O(n + d)$ dove d è il numero delle inversioni
- stabile
 - non cambia l'ordine degli elementi che hanno lo stesso valore
- in loco
 - la memoria aggiuntiva richiesta è $O(1)$
- online
 - può essere utilizzato quando i numeri arrivano uno alla volta

Algoritmi di ordinamento visti finora

	migliore caso	caso medio	caso peggiore	in loco	stabile
SELECTION-SORT	$\Theta(n^2)$			<i>si</i>	<i>si</i>
INSERTION-SORT	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	<i>si</i>	<i>si</i>

Algoritmi e Strutture di Dati

Ricorsione e complessità

m.patrignani

Nota di copyright

- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Sommario

- funzioni e record di attivazione
- ricorsione e record di attivazione
- formule di ricorrenza
 - teorema dell'esperto
- strategie algoritmiche
 - algoritmi divide et impera e merge sort

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

SUM_OF_FACT (3)

istruzione	3
variabile sum	0
variabile i	0

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

FACT (0)

istruzione	5
variabile f	1
variabile i	

SUM_OF_FACT (3)

istruzione	3
variabile sum	0
variabile i	0

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

FACT (0)

istruzione	8
variabile f	1
variabile i	2

SUM_OF_FACT (3)

istruzione	3
variabile sum	0
variabile i	0

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

SUM_OF_FACT (3)

istruzione	3
variabile sum	1
variabile i	0

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

SUM_OF_FACT (3)

istruzione	2
variabile sum	1
variabile i	1

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

FACT (1)

istruzione	8
variabile f	1
variabile i	2

SUM_OF_FACT (3)

istruzione	3
variabile sum	1
variabile i	1

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

SUM_OF_FACT (3)

istruzione	3
variabile sum	2
variabile i	1

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

SUM_OF_FACT (3)

istruzione	2
variabile sum	2
variabile i	2

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

FACT (2)

istruzione	8
variabile f	2
variabile i	3

SUM_OF_FACT (3)

istruzione	3
variabile sum	2
variabile i	2

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

SUM_OF_FACT (3)

istruzione	3
variabile sum	4
variabile i	2

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

SUM_OF_FACT (3)

istruzione	2
variabile sum	4
variabile i	3

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

FACT (3)

istruzione	8
variabile f	6
variabile i	4

SUM_OF_FACT (3)

istruzione	3
variabile sum	4
variabile i	3

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

SUM_OF_FACT (3)

istruzione	3
variabile sum	10
variabile i	3

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

SUM_OF_FACT (3)

istruzione	2
variabile sum	10
variabile i	4

Effetti di una chiamata a funzione

SUM_OF_FACT (n)

```
1. sum = 0
2. for i = 0 to n
3.     sum = sum + FACT(i)
4. return sum
```

FACT (n)

```
5. f = 1
6. for i = 2 to n
7.     f = f * i
8. return f
```

- supponiamo di eseguire SUM_OF_FACT(3)
- seguiamo l'evoluzione dello stack dei record di attivazione

SUM_OF_FACT (3)

istruzione	4
variabile sum	10
variabile i	4

Funzioni ricorsive

FACT (n)

5. `f = 1`

6. `for i = 2 to n`

7. `f = f * i`

8. `return f`

FACT_RIC (n)

1. `if n == 0`

2. `f = 1`

3. `else`

4. `f = n * FACT_RIC(n-1)`

5. `return f`

- abbiamo già visto che l'algoritmo iterativo FACT per il calcolo del fattoriale ha complessità $\Theta(n)$
- il calcolo del fattoriale può essere facilmente realizzato anche tramite un algoritmo ricorsivo

Esecuzione di funzioni ricorsive

FACT_RIC (n)	
1.	if n == 0
2.	f = 1
3.	else
4.	f = n * FACT_RIC (n-1)
5.	return f

- supponiamo di eseguire **FACT_RIC(3)**
- seguiamo l'evoluzione dello stack dei record di attivazione

FACT_RIC (3)	
istruzione	4
variabile f	0

Esecuzione di funzioni ricorsive

FACT_RIC (n)	
1.	if n == 0
2.	f = 1
3.	else
4.	f = n * FACT-RIC (n-1)
5.	return f

- supponiamo di eseguire **FACT_RIC(3)**
- seguiamo l'evoluzione dello stack dei record di attivazione

FACT_RIC (2)

istruzione	4
variabile f	0

FACT_RIC (3)

istruzione	4
variabile f	0

Esecuzione di funzioni ricorsive

FACT_RIC (n)

1. **if** n == 0

2. f = 1

3. **else**

4. f = n * **FACT-RIC** (n-1)

5. **return** f

- supponiamo di eseguire **FACT_RIC(3)**
- seguiamo l'evoluzione dello stack dei record di attivazione

FACT_RIC (1)

istruzione	4
variabile f	0

FACT_RIC (2)

istruzione	4
variabile f	0

FACT_RIC (3)

istruzione	4
variabile f	0

Esecuzione di funzioni ricorsive

FACT_RIC (n)

1. **if** n == 0

2. f = 1

3. **else**

4. f = n * **FACT-RIC** (n-1)

5. **return** f

- supponiamo di eseguire **FACT_RIC(3)**
- seguiamo l'evoluzione dello stack dei record di attivazione

FACT_RIC (0)

istruzione	2
variabile f	1

FACT_RIC (1)

istruzione	4
variabile f	0

FACT_RIC (2)

istruzione	4
variabile f	0

FACT_RIC (3)

istruzione	4
variabile f	0

Esecuzione di funzioni ricorsive

FACT_RIC (n)

1. **if** n == 0

2. f = 1

3. **else**

4. f = n * **FACT-RIC** (n-1)

5. **return** f

- supponiamo di eseguire **FACT_RIC(3)**
- seguiamo l'evoluzione dello stack dei record di attivazione

FACT_RIC (1)

istruzione	4
variabile f	1

FACT_RIC (2)

istruzione	4
variabile f	0

FACT_RIC (3)

istruzione	4
variabile f	0

Esecuzione di funzioni ricorsive

FACT_RIC (n)	
1.	if n == 0
2.	f = 1
3.	else
4.	f = n * FACT-RIC (n-1)
5.	return f

- supponiamo di eseguire **FACT_RIC(3)**
- seguiamo l'evoluzione dello stack dei record di attivazione

FACT_RIC (2)

istruzione	4
variabile f	2

FACT_RIC (3)

istruzione	4
variabile f	0

Esecuzione di funzioni ricorsive

FACT_RIC (n)	
1.	if n == 0
2.	f = 1
3.	else
4.	f = n * FACT-RIC (n-1)
5.	return f

- supponiamo di eseguire **FACT_RIC(3)**
- seguiamo l'evoluzione dello stack dei record di attivazione

FACT_RIC (3)	
istruzione	4
variabile f	6

Costo di FACT_RIC

FACT_RIC (n)	
1.	if $n == 0$
2.	$f = 1$
3.	else
4.	$f = n * \text{FACT_RIC}(n-1)$
5.	return f

$$T(0) = \Theta(1)$$

$$T(n) = T(n-1) + \Theta(1)$$

- il costo di $\text{FACT_RIC}(n)$ è
 - $\Theta(1)$ quando n è zero
 - pari al costo di $\text{FACT_RIC}(n-1) + \Theta(1)$ negli altri casi

Formule di ricorrenza

- equazioni o disequazioni che descrivono una funzione in termini del suo valore su input più piccoli
 - prevedono sempre dei casi base e dei casi induttivi
- esempi

$$T(n) = \begin{cases} a & \text{per } n = 0 \\ T(n-1) + g(n) & \text{per } n > 0 \end{cases}$$

$$T(n) = \begin{cases} a & \text{per } n = 0 \text{ o } n = 1 \\ 2T(n/2) + f(n) & \text{per } n > 1 \end{cases}$$

Formule di ricorrenza

- le soluzioni delle formule di ricorrenza non sempre sono facili da trovare
- quando esprimono delle complessità asintotiche talvolta i casi base vengono omessi
 - se $T(n)$ esprime il tempo di esecuzione di un algoritmo, $T(n)$ è sempre $\Theta(1)$ per n piccolo
- esempio

$$T(n) = 2T(n/2) + \Theta(n)$$

- è sottointeso che $T(n) = \Theta(1)$ per $n = 0$ e $n = 1$

Soluzione di una equazione di ricorrenza

- dimostriamo che l'equazione di ricorrenza

$$T(n) = \begin{cases} a & \text{per } n = 0 \\ T(n-1) + g(n) & \text{per } n > 0 \end{cases}$$

- ammette come soluzione

$$T(n) = a + \sum_{k=1}^n g(k)$$

- per dimostrarlo sostituiamo la soluzione proposta a destra e sinistra dell'equazione di ricorrenza

Verifica della correttezza della soluzione

- caso base per $n=0$

$$T(n=0) = a + \sum_{k=1}^0 g(k) = a + 0 = a \quad (\text{verificato})$$

- caso induttivo

$$\text{so che } T(n-1) = a + \sum_{k=1}^{n-1} g(k) \quad (\text{ipotesi induttiva})$$

$$T(n) = T(n-1) + g(n) \quad (\text{dalla definizione})$$

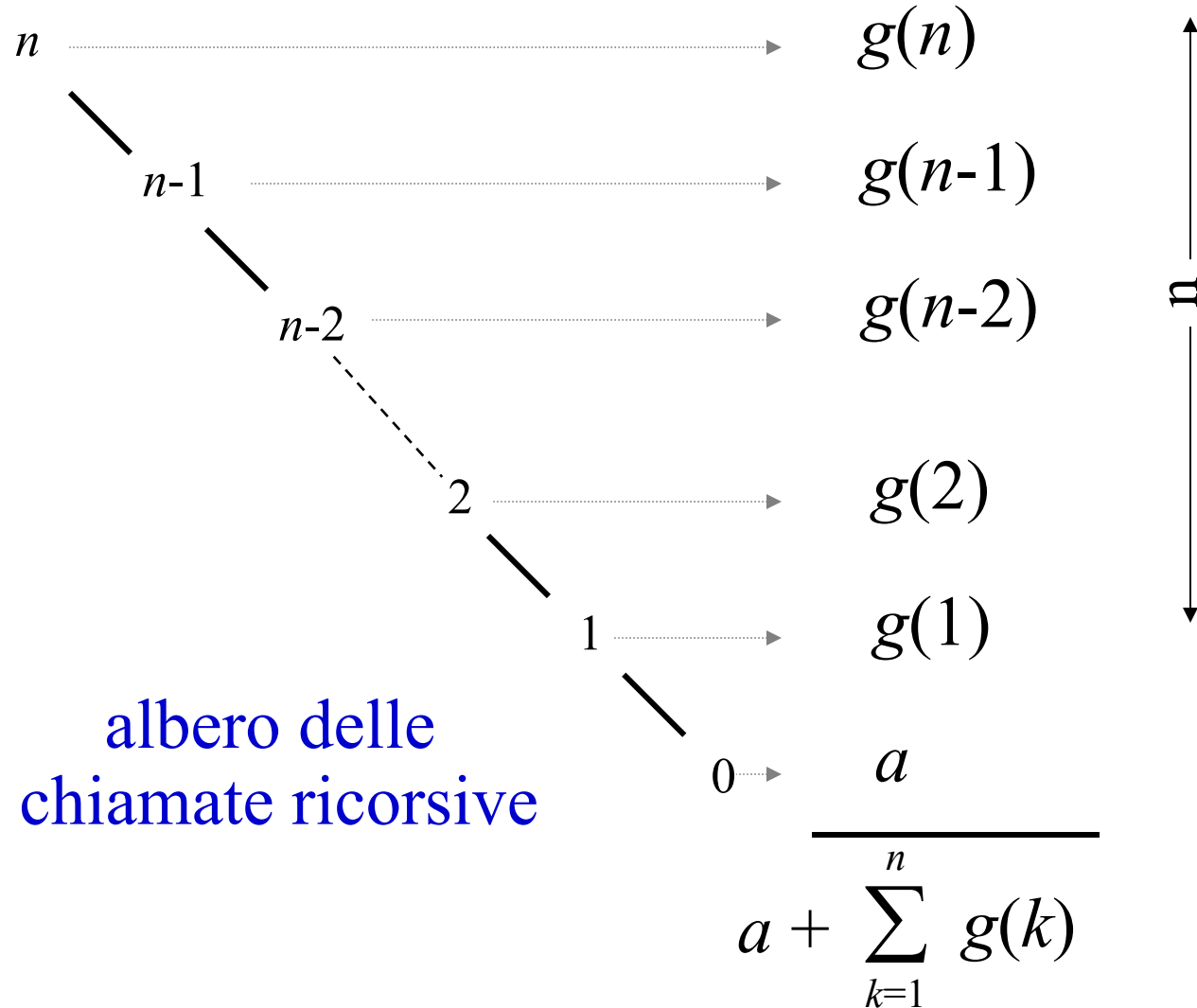
$$T(n) = a + \sum_{k=1}^{n-1} g(k) + g(n)$$

$$T(n) = a + \sum_{k=1}^n g(k) \quad (\text{verificato})$$

Graficamente

contributi

dimensione del problema



Complessità di FACT_RIC

- sappiamo che FACT_RIC ha complessità

$$T(n) = \begin{cases} \Theta(1) & \text{per } n = 0 \\ T(n-1) + \Theta(1) & \text{per } n > 0 \end{cases}$$

- sappiamo che l'equazione di ricorrenza

$$T(n) = \begin{cases} a & \text{per } n = 0 \\ T(n-1) + g(n) & \text{per } n > 0 \end{cases}$$

- ammette come soluzione $T(n) = a + \sum_{k=1}^n g(k)$
- la complessità di FACT_RIC è dunque

$$T(n) = \Theta(1) + \sum_{k=1}^n \Theta(1) = \Theta(n)$$

Versione ricorsiva del selection sort

SELECTION(A)

▷ ordina A

1. SELECTION_RIC(A, 0)

▷ ordina A da 0 in poi

SELECTION_RIC(A, i)

▷ ordina A da i a A.length-1

1. if i < A.length-1

▷ altrimenti è già ordinato

2. min = i

▷ indice elemento minimo in A[i..n-1]

3. for j = i + 1 **to** A.length-1

▷ scorro l'array

4. if A[j] < A[min]

▷ devo aggiornare min

5. min = j

6. temp = A[i]

▷ scambio A[i] con A[min]

7. A[i] = A[min]

8. A[min] = temp

9. SELECTION-RIC(A, i+1)

Complessità di SELECTION_RIC

- possiamo scrivere la seguente equazione di ricorrenza, in cui n è il numero degli elementi di A ancora da ordinare

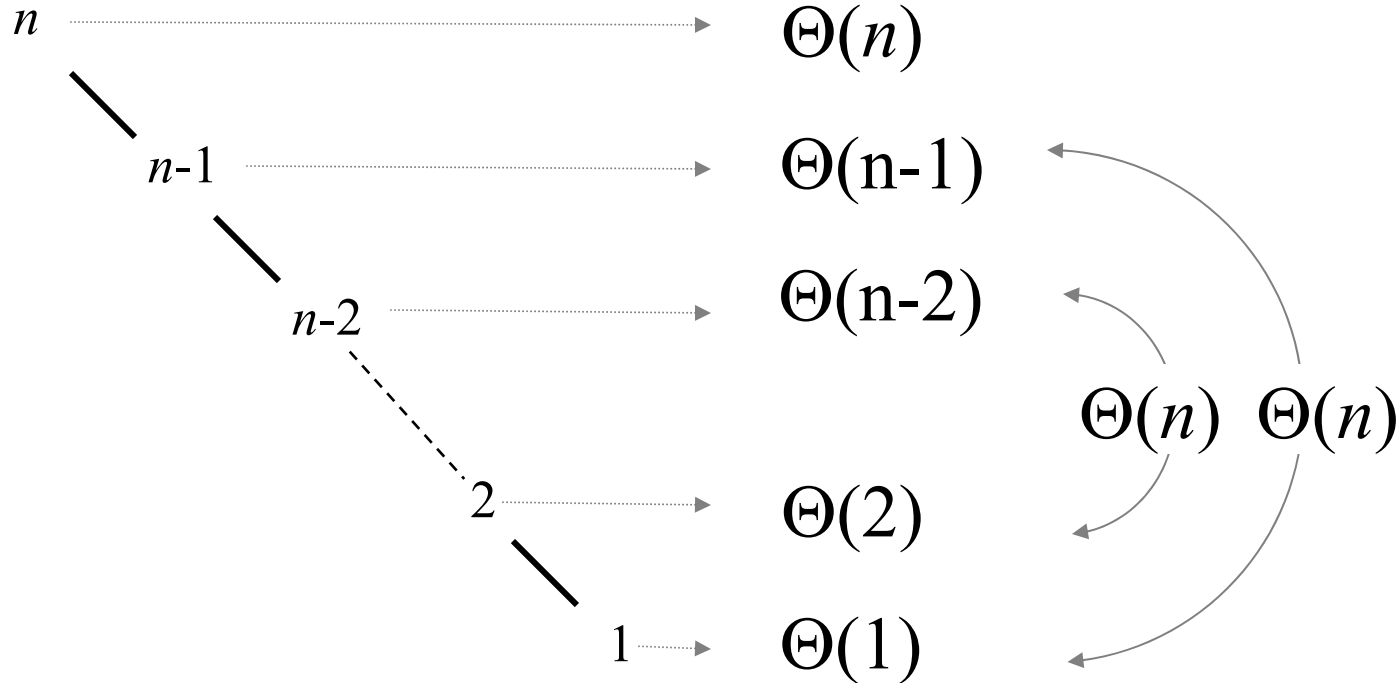
$$T(n) = \begin{cases} \Theta(1) & \text{per } n = 1 \\ T(n-1) + \Theta(n) & \text{per } n > 1 \end{cases}$$

- la complessità di SELECTION_RIC è dunque

$$T(n) = \Theta(1) + \sum_{k=1}^n \Theta(k) = \Theta(n^2)$$

Graficamente

dimensione del problema



contributi

albero delle
chiamate ricorsive

$$n/2 \cdot \Theta(n) = \Theta(n^2)$$

La tecnica divide et impera

- detta anche “divide and conquer”
- consiste nel suddividere il problema in diversi sottoproblemi
 - i sottoproblemi sono dello stesso tipo del problema originale
 - ma di dimensioni più piccole
 - i sottoproblemi possono essere risolti in maniera ricorsiva
 - suddividendoli a loro volta
 - caso base
 - quando i sottoproblemi sono di dimensioni ridottissime la loro soluzione è banale

Ricorsione del divide et impera

- a ciascun passo della ricorsione
 - divide
 - l'istanza corrente viene divisa in due o più istanze più piccole
 - impera
 - l'algoritmo viene lanciato sulle istanze più piccole
 - combina
 - le soluzioni delle istanze più piccole vengono utilizzate per produrre una soluzione dell'istanza corrente

Merge sort

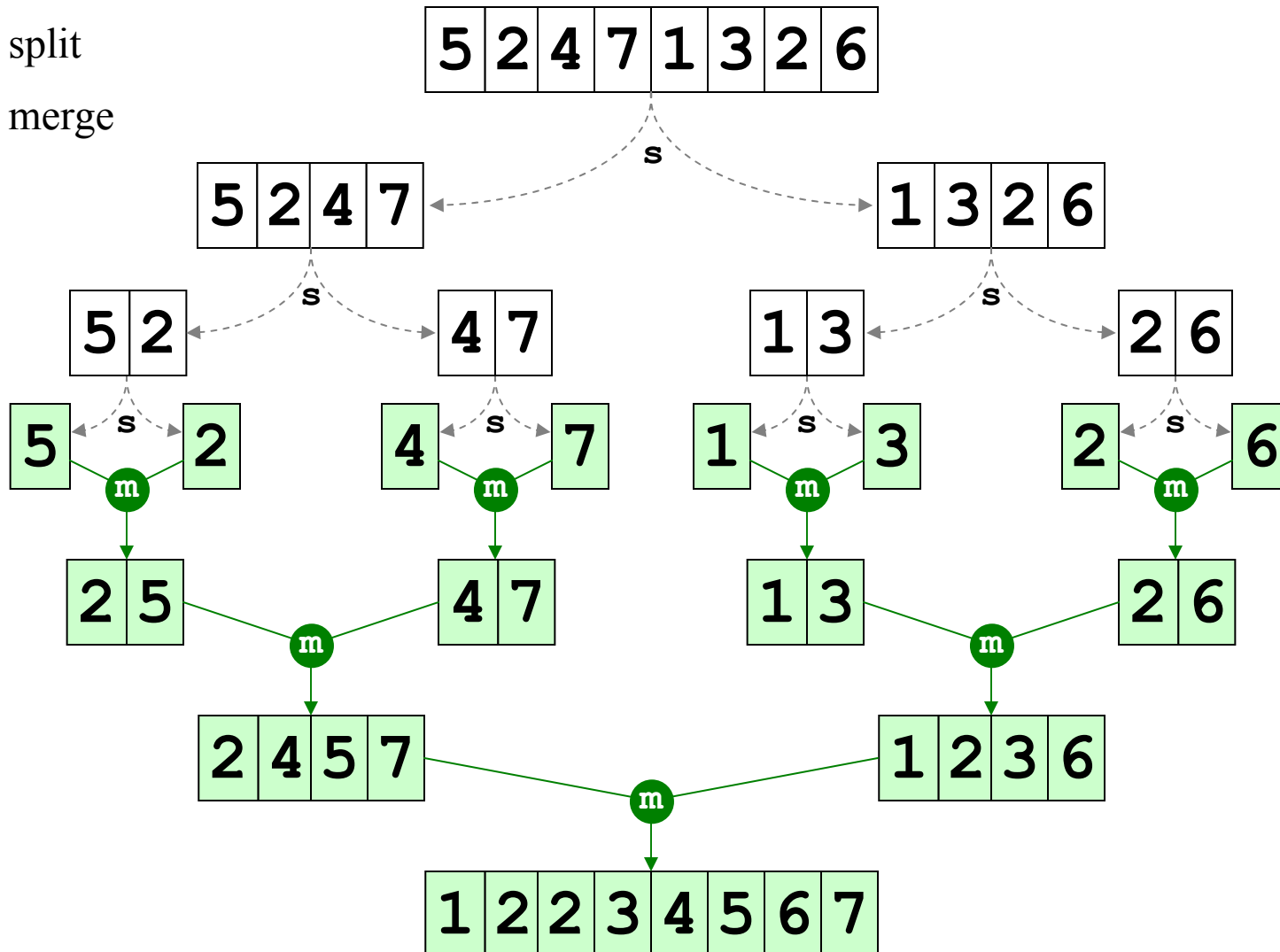
- introdotto da John von Neumann nel 1945
- osservazione elementare
 - due sequenze ordinate possono essere fuse in un'unica sequenza ordinata molto facilmente
- un possibile algoritmo
 - dividere la sequenza di input in due sottosequenze
 - ordinare le due sottosequenze
 - tramite lo stesso merge sort
 - fondere le due sottosequenze ordinate
- caso base
 - un array di un solo elemento è ordinato per definizione

Merge sort con input

5	2	4	7	1	3	2	6
---	---	---	---	---	---	---	---

s = split

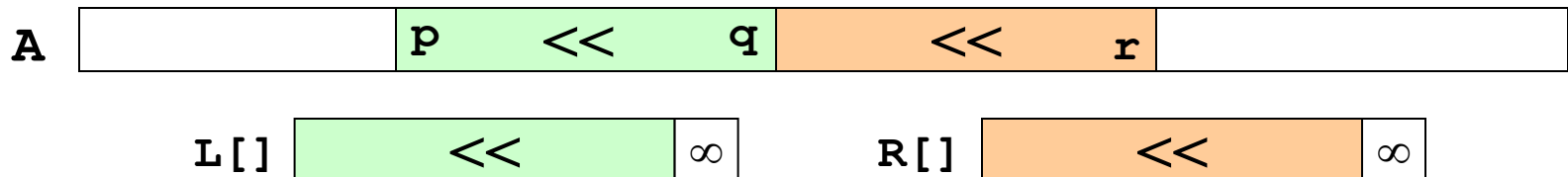
m = merge



Fusione: l'algoritmo MERGE

MERGE (A, p, q, r)

1. $n_1 = q - p + 1$ $\triangleright$ lunghezza della prima sequenza
 2. $n_2 = r - q$ $\triangleright$ lunghezza della seconda sequenza
 3. $\triangleright$ creo array $L[0...n_1]$ e $R[0...n_2]$ (con una casella in +)
 4. **for** $i = 0$ **to** $n_1 - 1$
 5. $L[i] = A[p+i]$ $\triangleright$ copio la 1^a sequenza
 6. **for** $j = 0$ **to** $n_2 - 1$
 7. $R[j] = A[q+j+1]$ $\triangleright$ copio la 2^a sequenza
 8. $L[n_1] = \infty$ $\triangleright$ chiudo con "infinito"
 9. $R[n_2] = \infty$ $\triangleright$ chiudo con "infinito"
- ...(continua nella prossima slide)...



Fusione (continua)

...(dalla slide precedente)...

```
10. i = 0           ▷ iteratore per array L
11. j = 0           ▷ iteratore per array R
12. for k = p to r
13.     if L[i] ≤ R[j] then
14.         A[k] = L[i]       ▷ pesca da L
15.         i = i + 1
16.     else
17.         A[k] = R[j]       ▷ pesca da R
18.         j = j + 1
```

- il confronto con “ $\leq$ ” sulla riga 13 garantisce la stabilità dell’algoritmo
 - se $L[i] = R[j]$ allora $L[i]$ ha la precedenza

L'algoritmo MERGE_SORT

- l'algoritmo MERGE_SORT esegue la parte “divide”, risolve i sottoproblemi ed esegue la parte “combine”

MERGE_SORT (A, p, r)

- if** $p < r$ **then** ▷ nel caso base esco subito
- $q = \lfloor (p+r)/2 \rfloor$ ▷ divido l'array in due
- MERGE_SORT** (A, p, q)
- MERGE_SORT** (A, q+1, r)
- MERGE** (A, p, q, r)

- all'inizio della computazione lanciamo

MERGE (A) ▷ ordina A

- MERGE_SORT** (A, 0, A.length-1)

Tempo di esecuzione di merge sort

- calcoliamo il costo $T(n)$ di esecuzione del merge sort su un'istanza con n elementi
- caso base
 - costo $\Theta(1)$
- divide
 - calcolo di $n/2$: costo $D(n) = \Theta(1)$
- impera
 - ogni sottoproblema ha dimensione $n/2$
 - i sottoproblemi sono 2
 - costo: $2 \cdot T(n/2)$
- combina
 - l'algoritmo MERGE ha costo lineare: $C(n) = \Theta(n)$

Tempo di esecuzione di merge sort

- complessivamente

$$T(n) = \begin{cases} \Theta(1) & \text{per } n = 0 \text{ o } n = 1 \\ 2 \cdot T(n/2) + D(n) + C(n) & \text{per } n > 1 \end{cases}$$

- poiché $D(n) + C(n) = \Theta(1) + \Theta(n) = \Theta(n)$ si ha

$$T(n) = \begin{cases} \Theta(1) & \text{per } n = 0 \text{ o } n = 1 \\ 2 \cdot T(n/2) + \Theta(n) & \text{per } n > 1 \end{cases}$$

- dimostreremo che questa particolare equazione di ricorrenza ammette come soluzione

$$T(n) = \Theta(n \log n)$$

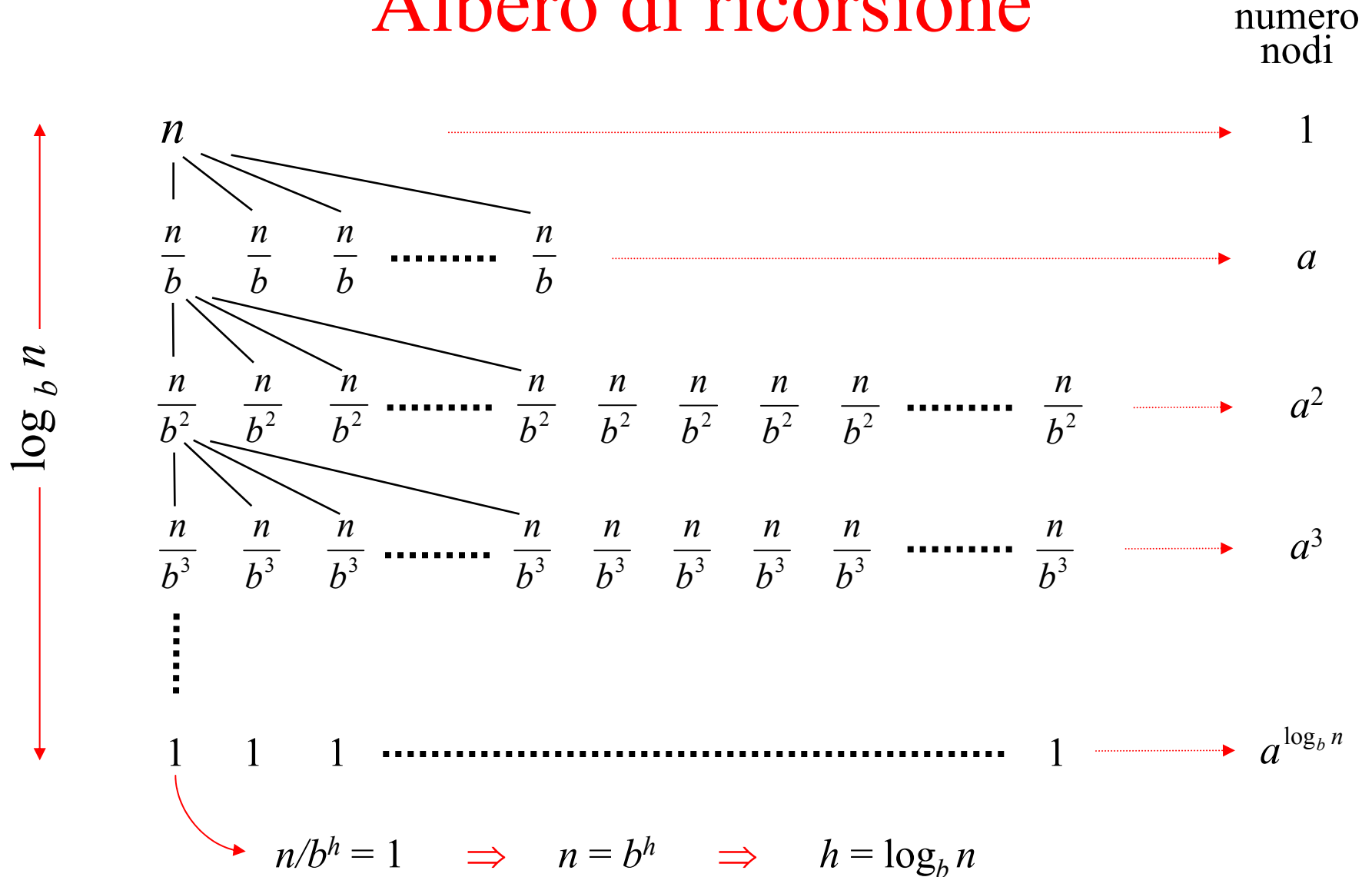
Master theorem (teorema dell'esperto)

- siano $a, b \geq 1$
- il master theorem considera l'equazione di ricorrenza seguente

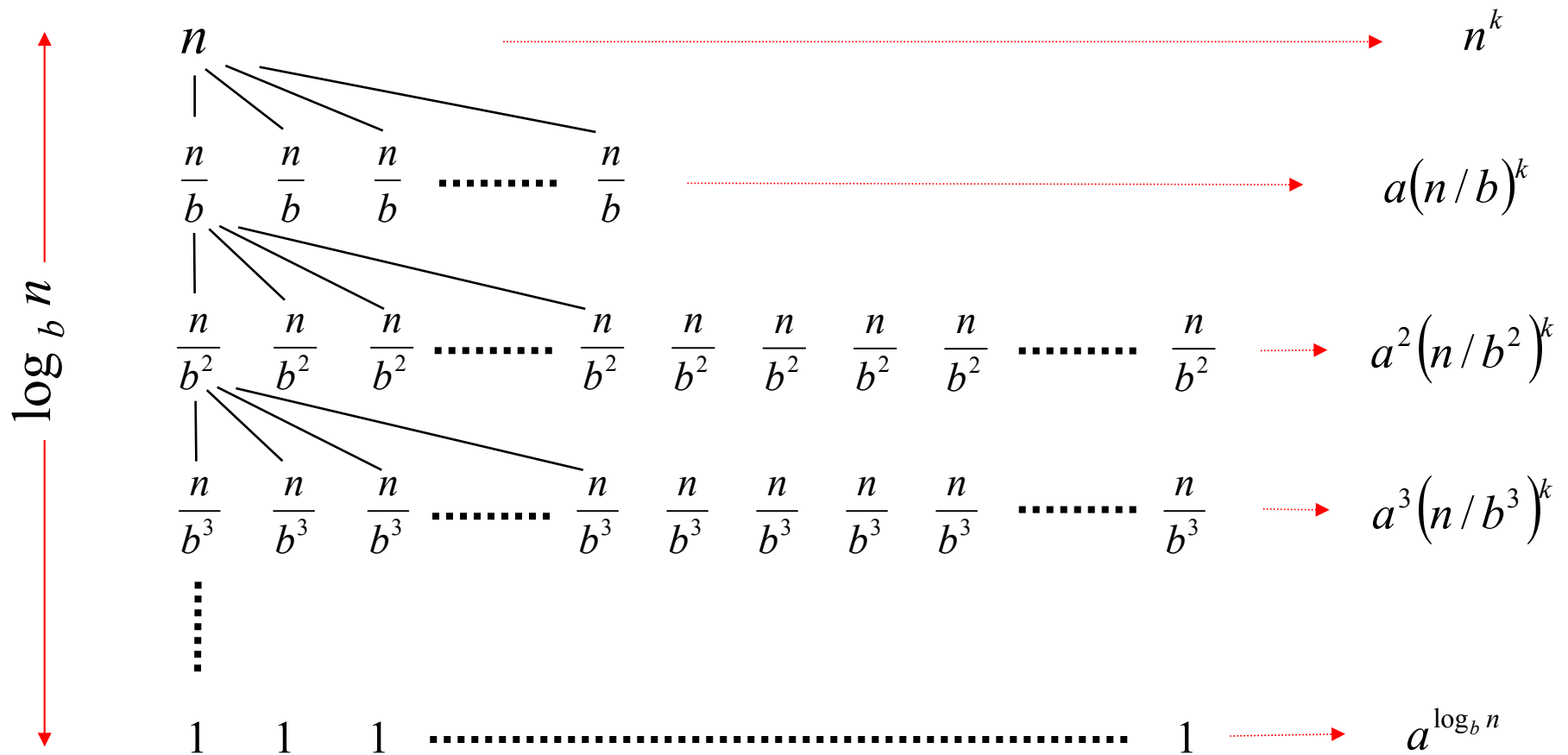
$$T(n) = \begin{cases} \Theta(1) & \text{per } n = 0 \\ a \cdot T(n/b) + O(n^k) & \text{per } n > 0 \end{cases}$$

- il master theorem afferma che tale equazione di ricorrenza ammette le soluzioni seguenti
 1. se $a < b^k$ allora $T(n) = \Theta(n^k)$
 2. se $a = b^k$ allora $T(n) = \Theta(n^k \log n)$
 3. se $a > b^k$ allora $T(n) = \Theta(n^{\log_b a})$

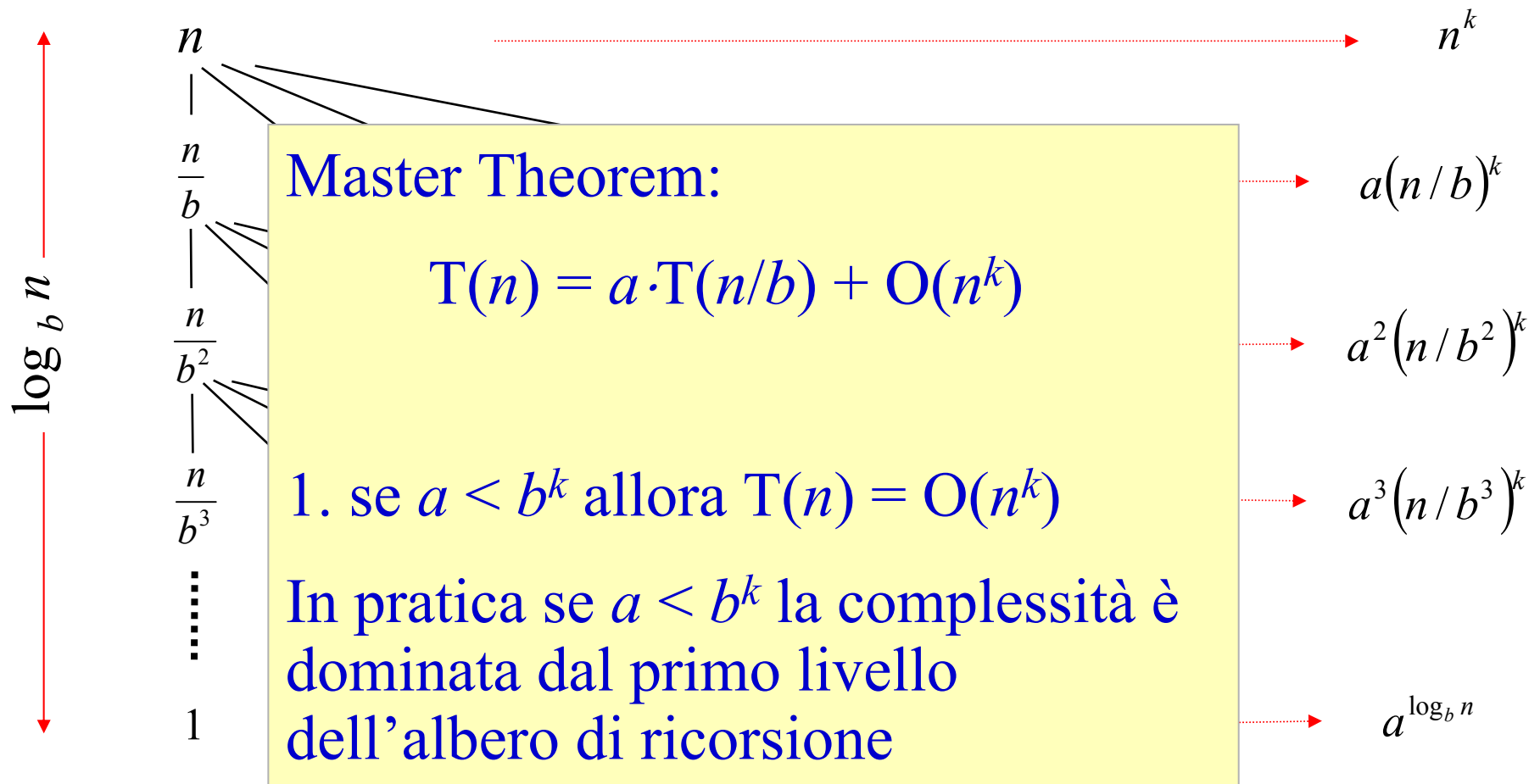
Albero di ricorsione



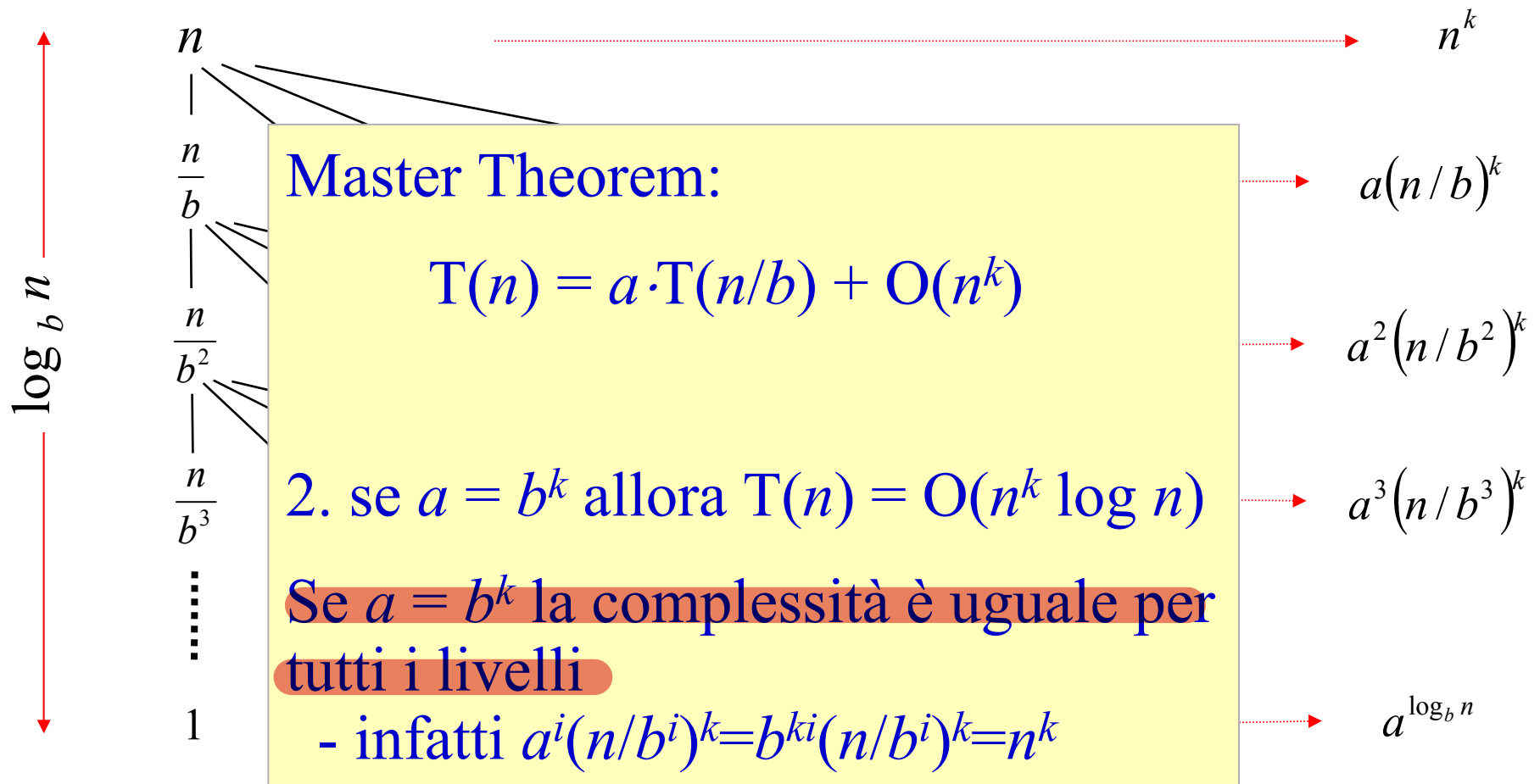
Costo per ogni livello



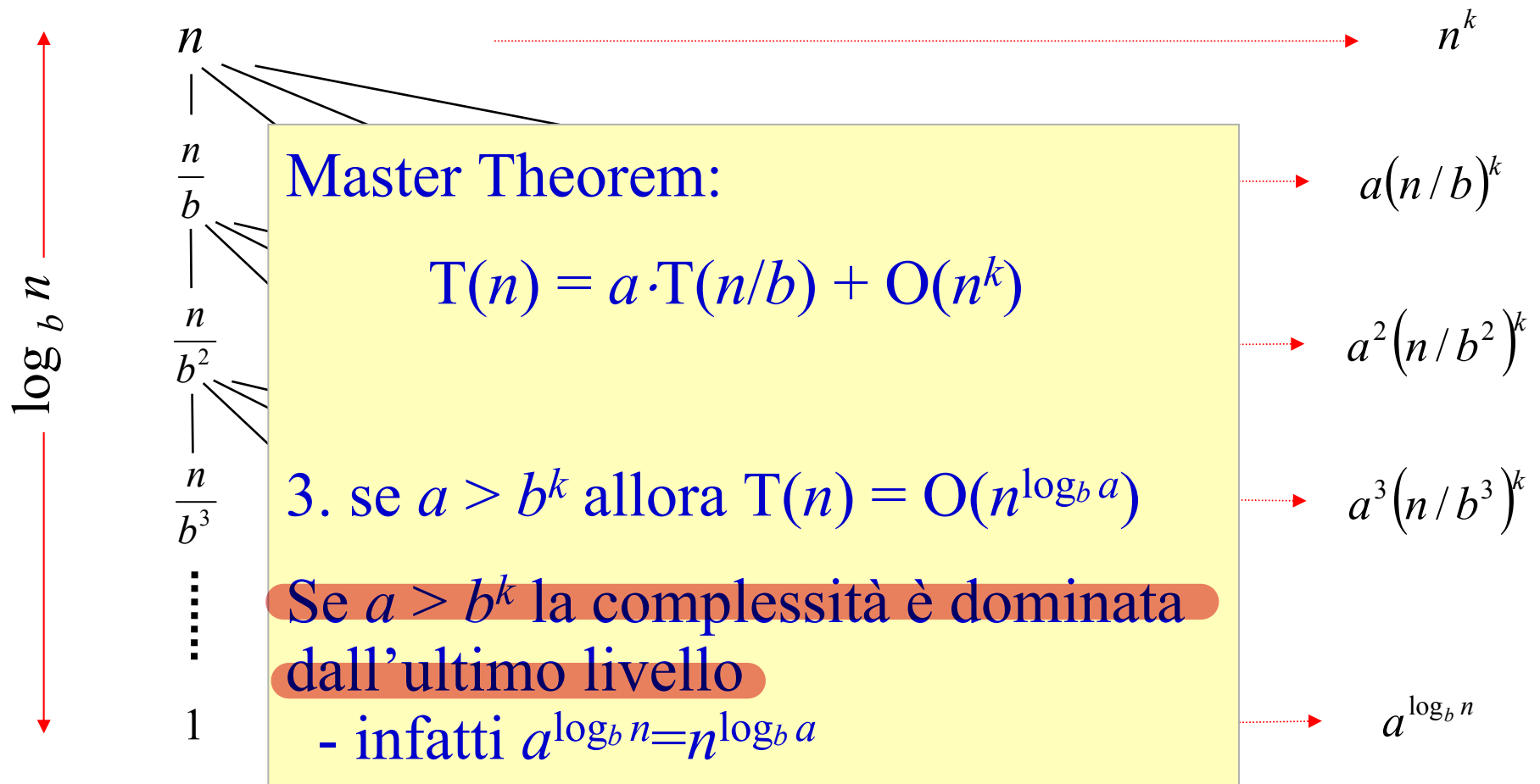
Costo per ogni livello



Costo per ogni livello



Costo per ogni livello



Dimostriamo che $x^{\log y} = y^{\log x}$

- Partiamo da

$$x^{\log y} = y^{\log x}$$

- Facciamo il logaritmo da entrambe le parti

$$\log(x^{\log y}) = \log(y^{\log x})$$

- Ricordando che

$$\log a^b = b \log a$$

- Otteniamo

$$(\log y)(\log x) = (\log x)(\log y)$$

- Che è vera per la proprietà commutativa del prodotto

Dimostrazione del primo caso ($a < b^k$)

- La somma del costo di tutti i livelli è

$$T(n) = \sum_{i=0}^h a^i \left(\frac{n}{b^i} \right)^k = \sum_{i=0}^h a^i \frac{n^k}{b^{ik}} = n^k \sum_{i=0}^h \left(\frac{a}{b^k} \right)^i$$

- $\sum_{i=0}^h \left(\frac{a}{b^k} \right)^i$ è una serie geometrica con ragione $r = \frac{a}{b^k}$
- Se $r < 1$, cioè se $a < b^k$, la sommatoria, anche se avesse infiniti termini, sarebbe comunque una costante $1/(1-r)$
- Dunque $T(n) = O(n^k)$

Dimostrazione del terzo caso ($a > b^k$)

- Torniamo alla serie geometrica con $r = \frac{a}{b^k}$

$$T(n) = n^k \sum_{i=0}^h \left(\frac{a}{b^k} \right)^i$$

- Se $a > b^k$, cioè se $r > 1$, la sommatoria vale

$$\frac{1 - r^h}{1 - r} = \frac{r^h - 1}{r - 1} \in O(r^h)$$

$$r^h = \left(\frac{a}{b^k} \right)^{\log_b n} = \frac{a^{\log_b n}}{b^{k \log_b n}} = \frac{a^{\log_b n}}{(b^{\log_b n})^k} = \frac{a^{\log_b n}}{n^k}$$

Dimostrazione del terzo caso ($a > b^k$)

- Dunque

$$T(n) = O(n^k) \cdot O(r^h) = O(n^k) \cdot O\left(\frac{a^{\log_b n}}{n^k}\right)$$

- Da cui

$$T(n) = O(a^{\log_b n}) = O(n^{\log_b a})$$

Esempi di applicazione del master theorem

- $T(n) = 9T(n/3) + n$
 - abbiamo: $a = 9$; $b = 3$; $p(n^k) = n$; $k = 1$
 - quindi $a > b^k$
 - si ha $T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_3 9}) = \Theta(n^2)$
- $T(n) = T(2n/3) + 1$
 - abbiamo: $a = 1$; $b = 3/2$; $p(n^k) = 1$; $k = 0$
 - quindi $a = b^k$
 - si ha $T(n) = \Theta(n^k \log n) = \Theta(n^0 \log n) = \Theta(\log n)$

Complessità del merge sort

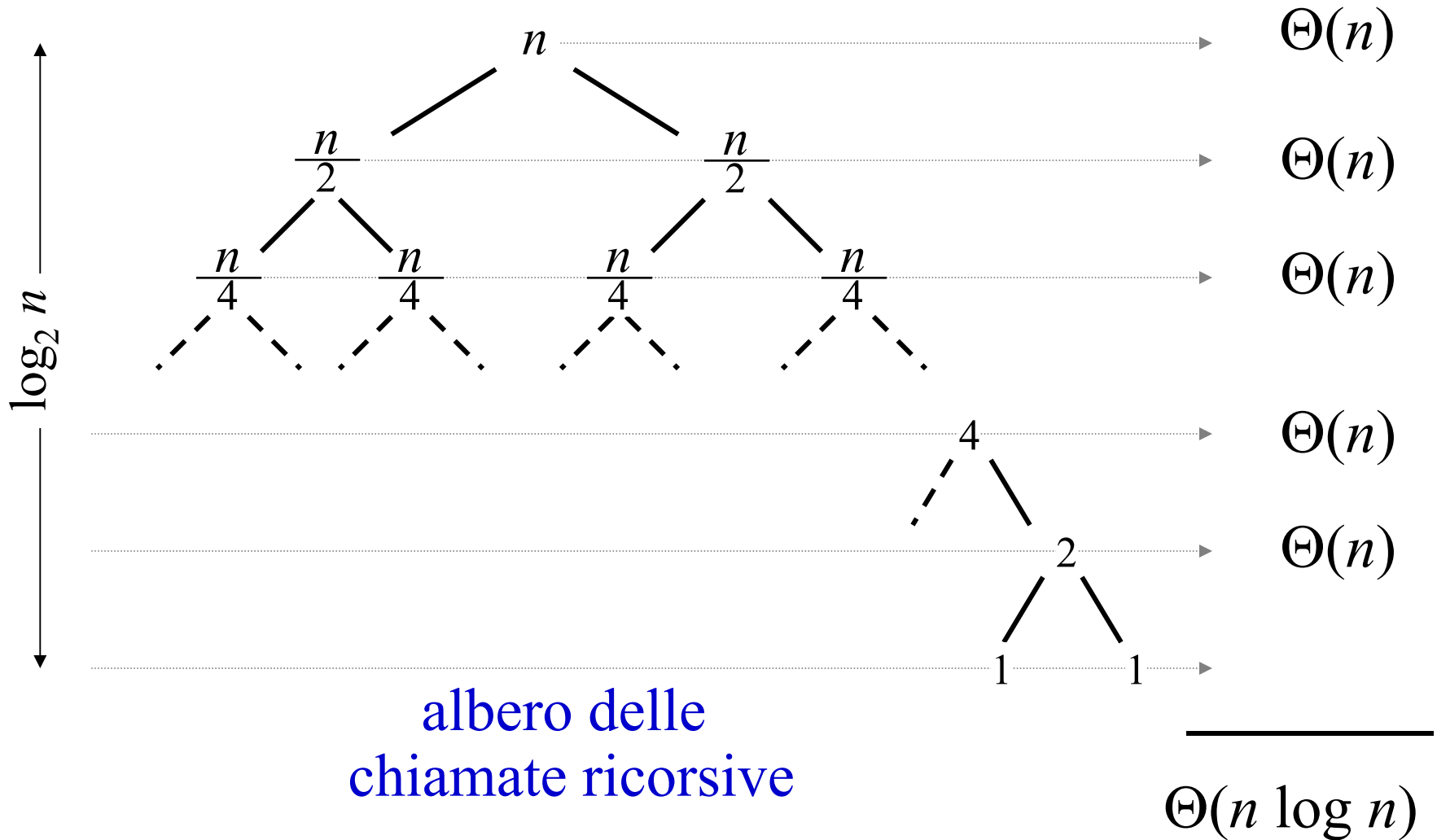
- la complessità del merge sort è data dalla formula di ricorrenza

$$T(n) = 2 \cdot T(n/2) + \Theta(n)$$

- applichiamo il teorema dell'esperto
 - abbiamo: $a = 2$; $b = 2$; $p(n^k) = n$; $k = 1$
 - quindi $a = b^k$
 - si ha $T(n) = \Theta(n^k \log n) = \Theta(n \log n)$

Graficamente

contributi



Algoritmi di ordinamento visti finora

	caso migliore	caso medio	caso peggiore	in loco	stabile
SELECTION-SORT	$\Theta(n^2)$			<i>si</i>	<i>si</i>
INSERTION-SORT	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	<i>si</i>	<i>si</i>
MERGE-SORT	$\Theta(n \log n)$			<i>no</i>	<i>si</i>

Algoritmi e Strutture di Dati

Tipi astratti di dato

(pile e code realizzate tramite array)

m.patrignani

Nota di copyright

- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Sommario

- Tipi astratti di dato
- Strutture di dati elementari
 - le pile
 - le code
- Realizzazione con array di queste strutture
- La complessità ammortizzata

Tipo astratto di dato

- Un *tipo astratto di dato* (o ADT, abstract data type) è una descrizione di un tipo di dato indipendente dalla sua realizzazione in un linguaggio di programmazione
- Un tipo astratto di dato è costituito da:
 - i domini interessati
 - tra cui il dominio di interesse ed eventualmente altri domini di supporto
 - un insieme di costanti
 - una collezione di operazioni

Il tipo astratto di dato pila

- Le pile (o stack) realizzano una strategia LIFO (last-in first-out)
- Tipo astratto: pila di interi
 - domini
 - il dominio di interesse è l'insieme delle pile P di interi
 - dominio di supporto: gli interi $Z = \{0, 1, -1, 2, -2, \dots\}$
 - dominio di supporto: i booleani $B = \{\text{true}, \text{false}\}$
 - costanti
 - la pila vuota

Il tipo astratto di dato pila

- Tipo astratto: pila di interi
 - operazioni
 - verifica se una pila è vuota
$$\text{IS_EMPTY: } P \rightarrow B$$
 - inserimento di un elemento nella pila
$$\text{PUSH: } P \times Z \rightarrow P$$
 - rimozione e restituzione dell'elemento affiorante della pila
$$\text{POP: } P \rightarrow P \times Z$$
 - operazioni aggiuntive
 - lettura dell'elemento affiorante della pila
$$\text{TOP: } P \rightarrow Z$$
 - svuotamento della pila
$$\text{EMPTY: } P \rightarrow P$$
 - numero degli elementi nella pila
$$\text{SIZE: } P \rightarrow Z$$

Realizzazione di un tipo astratto di dato

- Un tipo astratto di dato può essere *realizzato* (o *implementato*) in uno specifico linguaggio di programmazione tramite la definizione di
 - tipi concreti o strutture dati nello specifico linguaggio di programmazione corrispondenti ai *domini* necessari
 - costrutti che consentono di codificare le *costanti*
 - funzioni che realizzano le *operazioni* previste
- Ovviamente uno stesso tipo astratto di dato può avere diverse realizzazioni con diverse proprietà

Realizzazione di un ADT: osservazioni

- La definizione dei tipi concreti o delle strutture che corrispondono ai domini può comportare delle limitazioni
 - esempi di limitazioni
 - il dominio P delle pile viene ristretto alle pile di dimensione massima `maxsize`, nota a priori
 - il dominio Z degli interi viene realizzato tramite il tipo concreto `int` che ha un valore minimo `MININT` e massimo `MAXINT`
- Le costanti vengono spesso codificate tramite delle funzioni che ritornano la struttura corrispondente
 - esempio di costante:
 - la pila vuota viene realizzata tramite
`CREATE-STACK(maxsize)`
che ritorna il riferimento ad una pila vuota che potrà contenere al massimo `maxsize` elementi

Funzioni che operano su pile

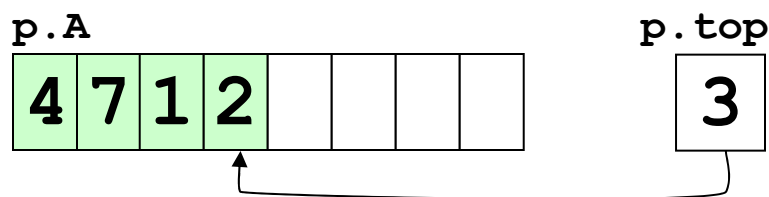
- `NEW_STACK(maxsize)`
 - ritorna il riferimento ad una pila vuota che potrà contenere al massimo `maxsize` elementi
- `IS_EMPTY(p)`
 - ritorna `true` se la pila è vuota, `false` altrimenti
- `PUSH(p, x)`
 - inserimento di un elemento nella pila
 - può dare un errore di “overflow” se l’implementazione prevede un numero massimo di elementi nella pila
- `POP(p)`
 - rimozione e restituzione dell’elemento affiorante della pila
 - dà un errore di “underflow” se la pila è vuota

Altre funzioni su pile

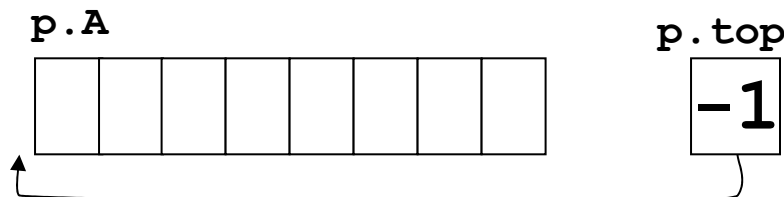
- $TOP(p)$
 - ritorna l'elemento affiorante senza rimuoverlo
 - può dare errore se la pila è vuota
- $EMPTY(p)$
 - svuota la pila
- $SIZE(p)$
 - ritorna il numero degli elementi in pila

Realizzazione di una pila con array

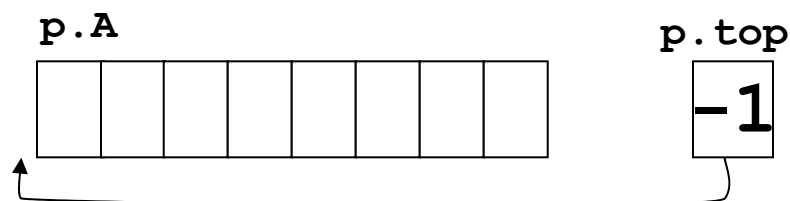
- Una pila p può essere realizzata tramite un oggetto contenente un array $p.A$ e un intero $p.top$ che specifica l'indice dell'elemento affiorante



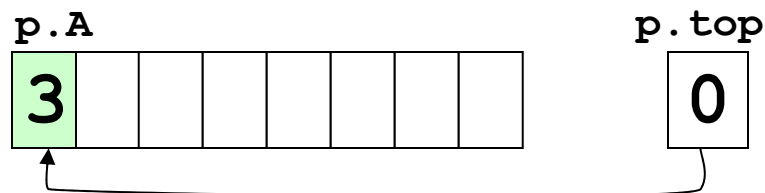
- Quando la pila è vuota $p.top$ vale -1



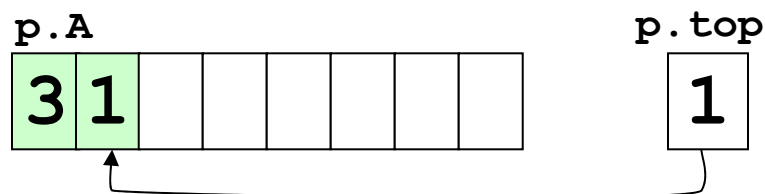
Sequenza di operazioni su una pila



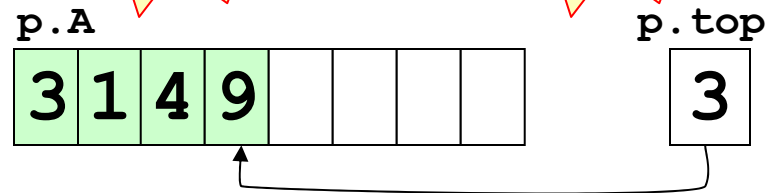
PUSH(p,3)



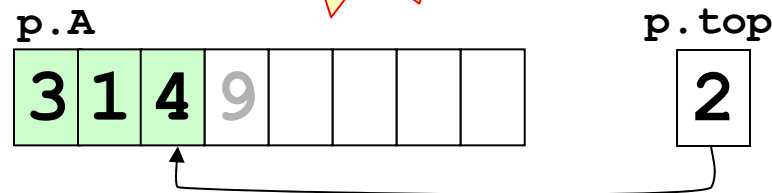
PUSH(p,1)



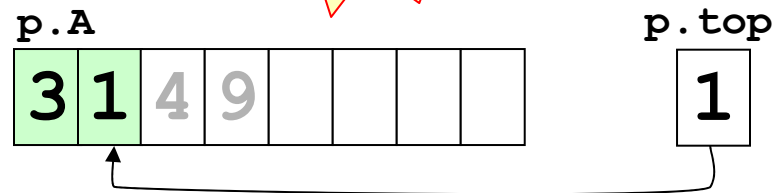
PUSH(p,4) **PUSH(p,9)**



POP(p)



POP(p)



Operazione NEW_STACK

- Implementazione in pseudocodice della funzione di creazione

NEW_STACK(maxsize)

1. ▷ creo un oggetto p con: p.A array di **maxsize** interi
2. ▷ p.top intero
3. p.top = -1
4. **return** p

- L'uso della funzione NEW_STACK è il seguente

```
...  
p = NEW_STACK(10)    ▷ creo una pila con 10 posizioni  
...
```

Operazioni PUSH e POP

- Implementazione delle funzioni di modifica

PUSH(p, x)

```
1. if p.top == p.A.length-1
2.     error("overflow")
3. else
4.     p.top = p.top + 1
5.     p.A[p.top] = x
```

POP(p)

```
1. if p.top == -1
2.     error("underflow")
3. else
4.     p.top = p.top - 1
5.     return p.A[p.top + 1]
```

Altre operazioni sulle pile

IS_EMPTY (p)

1. **return** p.top == -1 ▷ true se la pila è vuota

EMPTY (p)

1. p.top = -1 ▷ vuoto la pila

TOP (p)

1. **return** p.A[p.top] ▷ l'elemento affiorante

SIZE (p)

1. **return** p.top + 1 ▷ il numero di elementi

- Il tempo di esecuzione di ogni operazione è $\Theta(1)$

Gestione telescopica della pila

- Strategia utilizzata per non avere limiti sulla dimensione della pila
- Si adotta una dimensione iniziale di default
 - può essere un numero fissato dal programmatore
 - per esempio 128 posizioni
 - può essere il numero di celle specificato dall'utente nella funzione `NEW_STACK`
- Quando viene eseguita una `PUSH` sulla pila piena si raddoppia la dimensione della pila corrente per poter inserire ulteriori elementi

Nuova funzione PUSH

- Questa funzione PUSH raddoppia la dimensione dell'array `p.A` quando l'array è pieno

PUSH(p, x)

1. **if** `p.top == p.A.length-1`

2. ▷ B nuovo array di `2*p.A.length` posizioni

3. **for** `i = 0 to p.A.length-1`

4. `B[i] = p.A[i]` ▷ copio `p.A` dentro `B`

5. `p.A = B` ▷ sostituisco `B` a `p.A`

6. `p.top = p.top + 1` ▷ inserisco l'elemento `x`

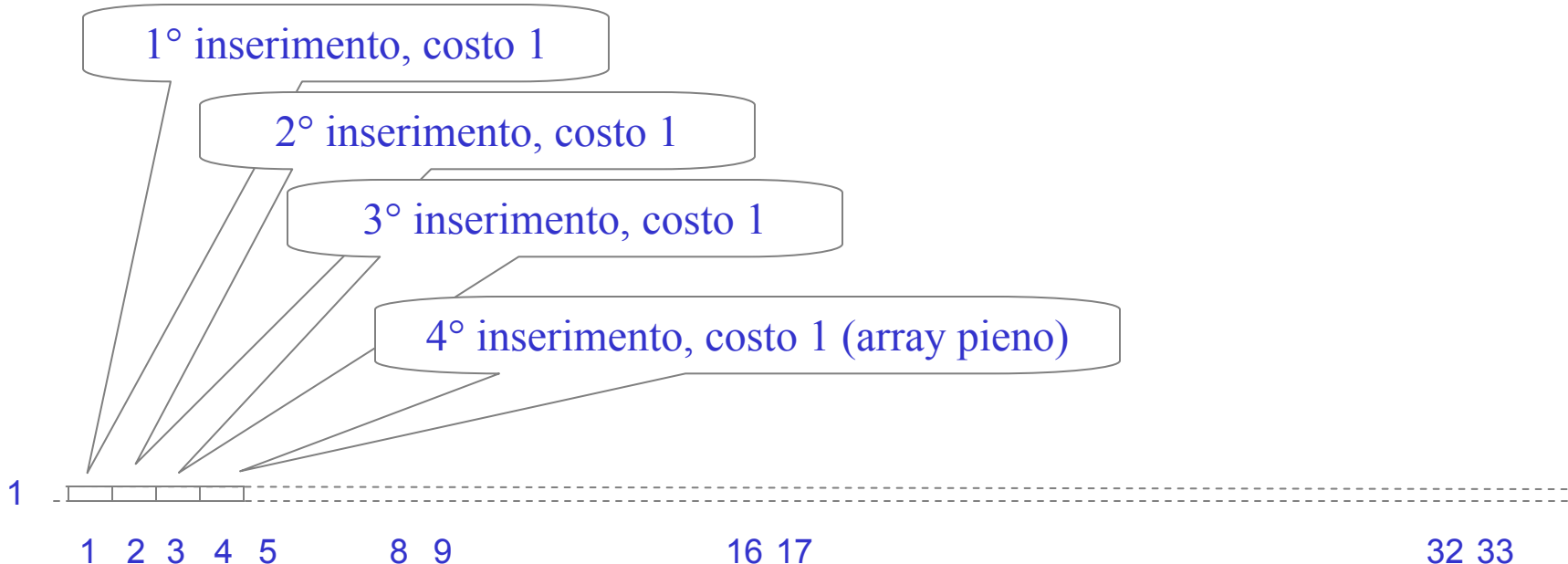
7. `p.A[p.top] = x`

Analisi ammortizzata

- Calcoliamo il costo degli inserimenti partendo da un array da 4 elementi

1	2	3	4
---	---	---	---

- Diagramma dei costi:



Analisi ammortizzata

- Calcoliamo il costo degli inserimenti partendo da un array da 4 elementi



5

5° inserimento

- Diagramma dei costi:

raddoppio la dimensione dell'array e ci copio dentro i primi 4 elementi

5° inserimento, costo 5
(4 per la copia, 1 per l'inserimento)

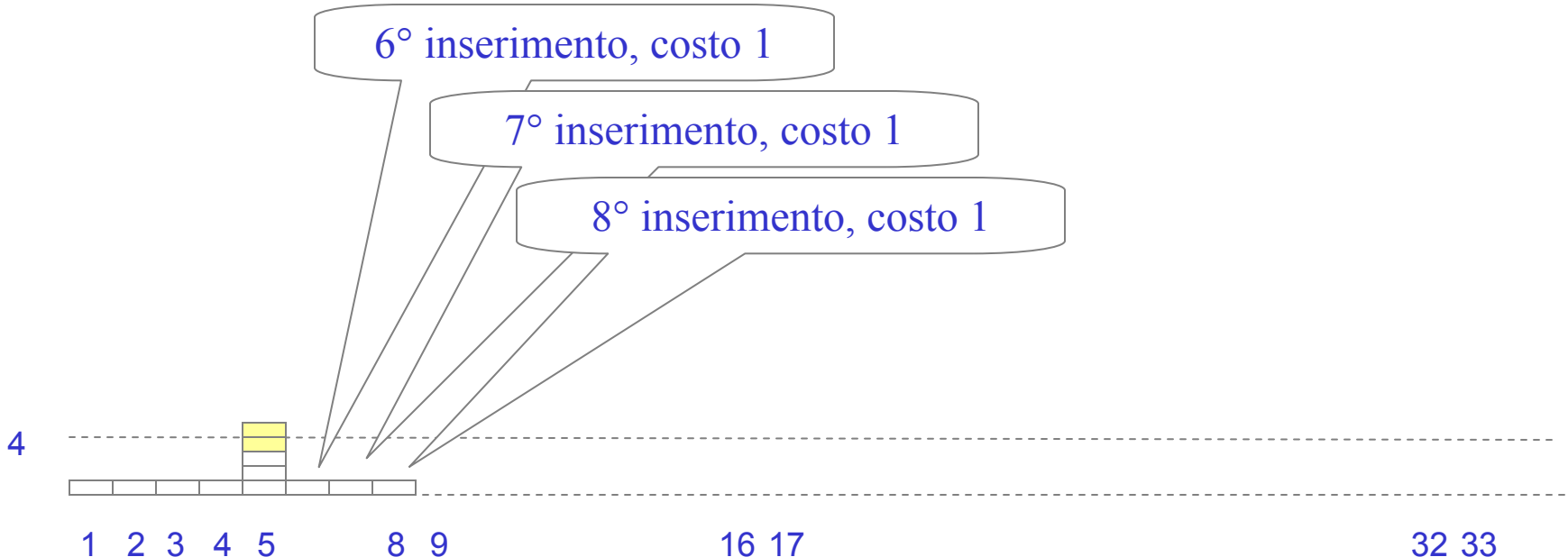


Analisi ammortizzata

- Calcoliamo il costo degli inserimenti partendo da un array da 4 elementi

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

- Diagramma dei costi:



Analisi ammortizzata

- Calcoliamo il costo degli inserimenti partendo da un array da 4 elementi



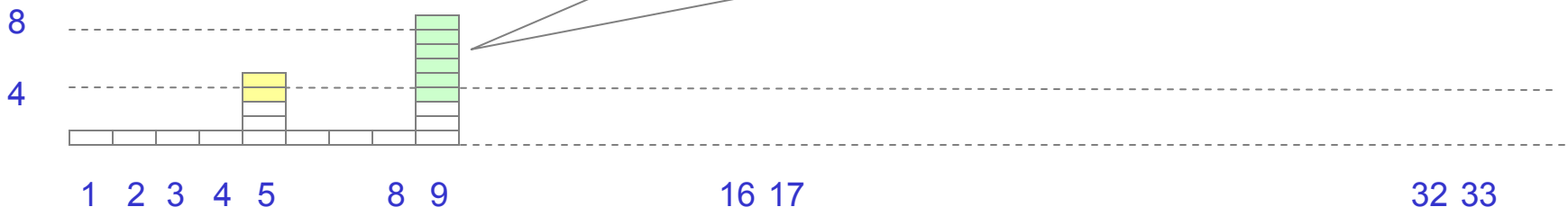
9

9° inserimento

- Diagramma dei costi:

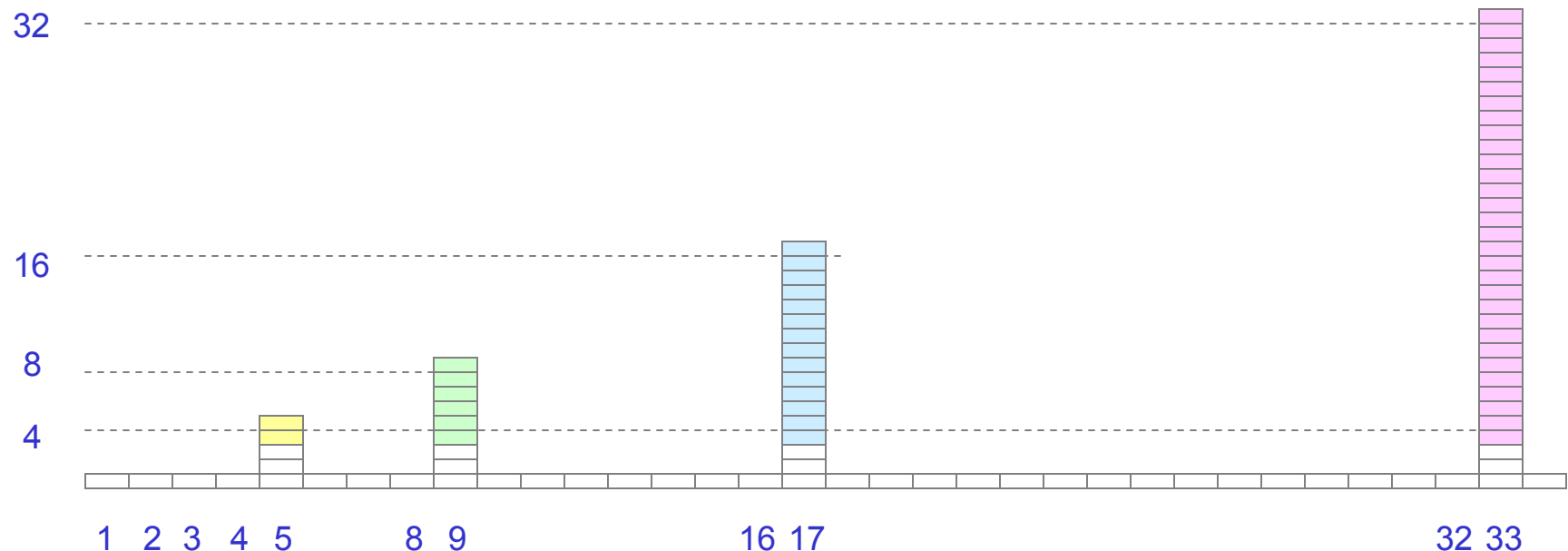
raddoppio la dimensione dell'array e ci copio dentro i primi 8 elementi

9° inserimento, costo 9
(8 per la copia, 1 per l'inserimento)



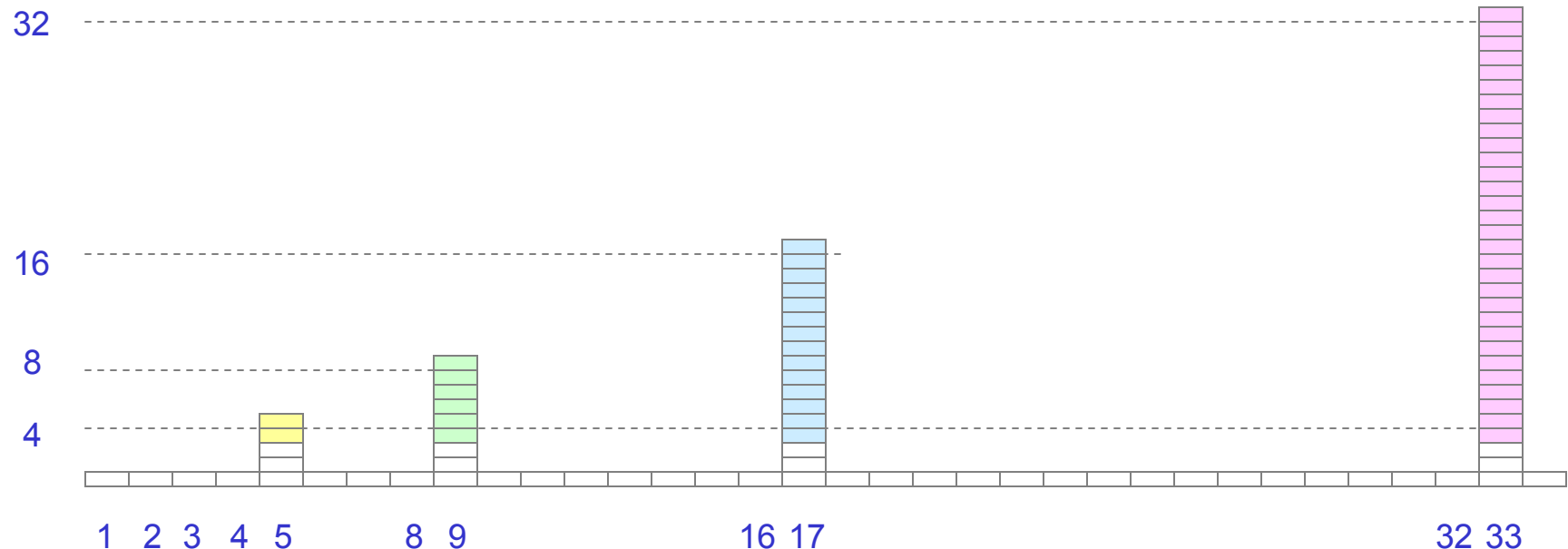
Analisi ammortizzata

- Calcoliamo il costo degli inserimenti partendo da un array da 4 elementi
- Diagramma dei costi dopo molti inserimenti



Analisi ammortizzata

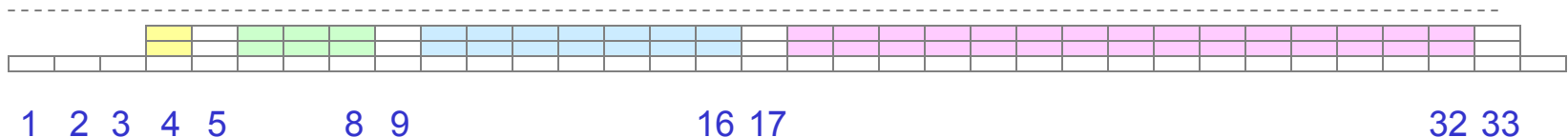
- Calcoliamo il costo degli inserimenti partendo da un array da 4 elementi
- Diagramma dei costi dopo molti inserimenti
- Distribuiamo i costi sugli inserimenti



Complessità ammortizzata

- Alcuni inserimenti costano effettivamente $\Theta(n)$, dove n è il numero di elementi già contenuti nella pila
 - è corretto dire che il costo di un inserimento nel caso peggiore è $\Theta(n)$
- Una sequenza di n inserimenti costa $\Theta(n)$
 - si dice che la complessità “ammortizzata” di un inserimento è $\Theta(1)$

4



Uso delle realizzazioni di un ADT

- Dopo aver realizzato un ADT possiamo mettere l'implementazione a disposizione di altri programmatori specificando
 - le eventuali limitazioni della realizzazione
 - l'elenco delle operazioni supportate e la loro complessità asintotica o ammortizzata
- Esempio
 - si dispone di una realizzazione di una pila con le seguenti funzioni e complessità nel caso peggiore
 - `NEW_STACK(maxsize)` $\Theta(1)$
 - `IS_EMPTY(p)` $\Theta(1)$
 - `PUSH(p, x)` $\Theta(1)$
 - `POP(p)` $\Theta(1)$
 - `EMPTY(p)` $\Theta(1)$

Il tipo astratto di dato coda

- Le code (o queue) realizzano una strategia FIFO (first-in first-out)
- Tipo astratto: coda di interi
 - domini
 - il dominio di interesse è l'insieme delle code Q di interi
 - dominio di supporto: gli interi $Z = \{0, 1, -1, 2, -2, \dots\}$
 - dominio di supporto: i booleani $B = \{\text{true}, \text{false}\}$
 - costanti
 - la coda vuota

Il tipo astratto di dato coda

- Tipo astratto: coda di interi

- operazioni

- verifica se una coda è vuota

$$\text{IS_EMPTY: } Q \rightarrow B$$

- inserimento di un elemento nella coda

$$\text{ENQUEUE: } Q \times Z \rightarrow Q$$

- rimozione e restituzione dell'elemento più vecchio della coda

$$\text{DEQUEUE: } Q \rightarrow Q \times Z$$

- operazioni aggiuntive

- lettura dell'elemento più vecchio della coda

$$\text{FRONT: } Q \rightarrow Z$$

- svuotamento della coda

$$\text{EMPTY: } Q \rightarrow Q$$

- numero degli elementi nella coda

$$\text{SIZE: } Q \rightarrow Z$$

Realizzazione dell'ADT coda

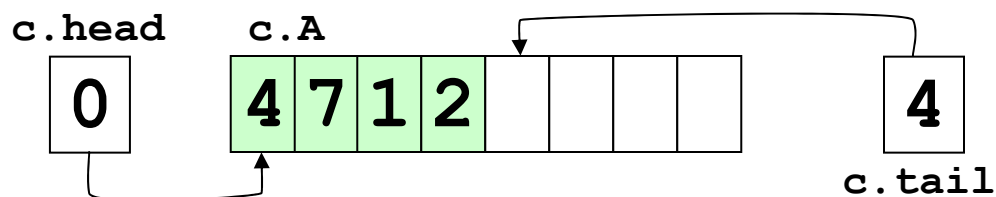
- `NEW_QUEUE(maxsize)`
 - ritorna un riferimento ad una coda vuota che può contenere al massimo `maxsize` interi
- `IS_EMPTY(c)`
 - ritorna `true` se la coda è vuota, altrimenti ritorna `false`
- `ENQUEUE(c, x)`
 - inserisce un elemento `x` nella coda
 - può dare un errore di “overflow” se l’implementazione prevede un numero massimo di elementi
- `DEQUEUE(c)`
 - rimuove l’elemento più vecchio della coda e lo restituisce
 - dà un errore di “underflow” se la coda è vuota

Altre operazioni sulle code

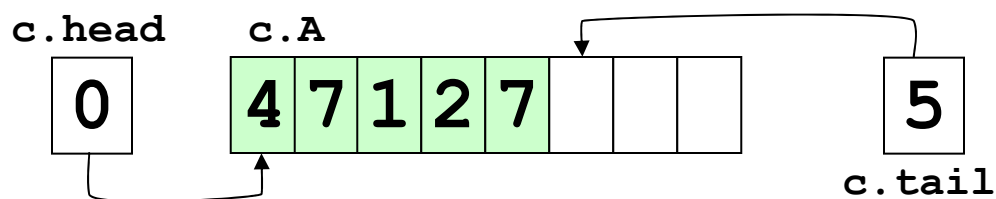
- FRONT (c)
 - ritorna l'elemento più vecchio senza rimuoverlo
 - può dare errore se la coda è vuota
- EMPTY (c)
 - svuota la coda
- SIZE (c)
 - ritorna il numero degli elementi in coda

Realizzazione di una coda tramite array

- Una coda c può essere realizzata con un array $c.A$ arricchito da due attributi $c.head$ e $c.tail$ che contengono gli indici dell'elemento più vecchio e della prima posizione utile

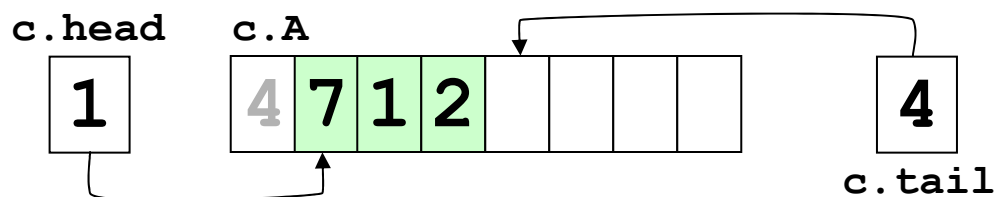
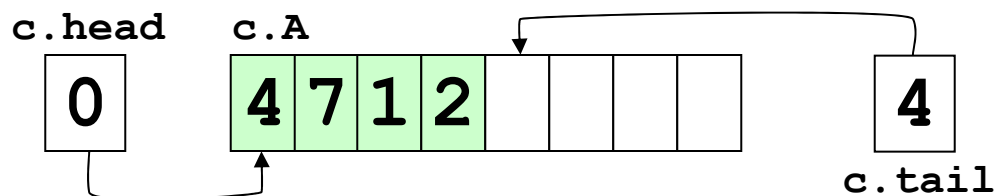


- Un nuovo elemento viene aggiunto da ENQUEUE nella posizione $c.tail$ (che viene incrementato)

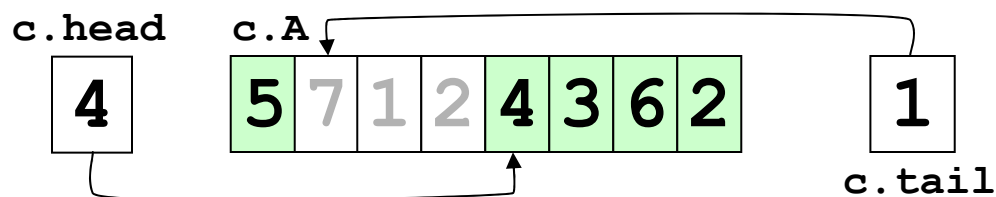


Realizzazione di una coda tramite array

- L'elemento ritornato da DEQUEUE è quello in posizione `c.head` (che viene incrementato)

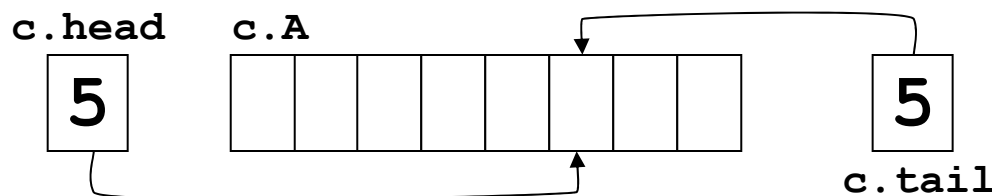


- L'array è gestito come una lista circolare

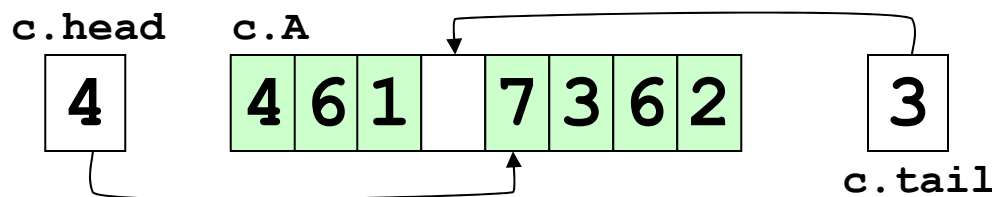


Realizzazione di una coda tramite array

- La coda è vuota quando `c.head` e `c.tail` puntano alla stessa casella



- La coda non può avere più di $n-1$ elementi
 - se avesse n elementi, che valore potremmo dare a `c.tail` senza creare equivoco con la coda vuota?



Operazioni ENQUEUE e DEQUEUE

ENQUEUE (c, x)

```
1.  if c.head==c.tail+1 or (c.tail==c.A.length-1 and c.head==0)
2.      error("overflow")
3.  else  c.A[c.tail] = x
4.      if c.tail == c.A.length-1
5.          c.tail = 0
6.      else  c.tail = c.tail + 1
```

DEQUEUE (c)

```
1.  if c.head == c.tail
2.      error("underflow")
3.  else  x = c.A[c.head]
4.      if c.head == c.A.length-1
5.          c.head = 0
6.      else  c.head = c.head + 1
7.      return x
```

Altre operazioni sulle code

IS_EMPTY(c)

```
1. return c.head == c.tail
```

EMPTY(c)

```
1. c.head = c.tail = 0
```

FRONT(c)

```
1. if c.head == c.tail
2.     error("empty queue")
3. else
4.     return c.A[c.head]
```

- Il tempo di esecuzione di ogni operazione è $\Theta(1)$

Esercizi sulle code

1. Scrivi lo pseudocodice della procedura `NEW_QUEUE()` che restituisce il riferimento ad una coda vuota
2. Scrivi lo pseudocodice della procedura `SIZE(c)` che restituisce il numero di elementi in una coda
3. Scrivi lo pseudocodice della procedura `ENQUEUE (q, x)` che abbia complessità ammortizzata $\Theta(1)$

Esercizi su pile e code

4. Supponi di disporre di una realizzazione di una pila con le seguenti caratteristiche

- funzioni `NEW_STACK`, `IS_EMPTY` e `POP` di complessità $\Theta(1)$
- funzione `PUSH` di complessità nel caso peggiore $\Theta(n)$ e complessità ammortizzata $\Theta(1)$

Descrivi come sia possibile implementare un coda utilizzando esclusivamente pile

- ti occorreranno due pile `p1` e `p2` da usare simultaneamente
- l'operazione `NEW_QUEUE` creerà le due pile `p1` e `p2`
- le operazioni `ENQUEUE` e `DEQUEUE` si tradurranno in opportune operazioni di `PUSH` e `POP` sulle due pile `p1` e `p2`

Discuti la complessità delle operazioni `ENQUEUE` e `DEQUEUE`

Esercizi su pile e code

5. Supponi di disporre di una realizzazione di una coda con le seguenti caratteristiche
- funzioni `NEW_QUEUE`, `IS_EMPTY` e `DEQUEUE` di complessità $\Theta(1)$
 - funzione `ENQUEUE` di complessità nel caso peggiore $\Theta(n)$ e complessità ammortizzata $\Theta(1)$

Descrivi come sia possibile implementare un pila utilizzando esclusivamente code

- ti occorreranno due code `q1` e `q2` da usare simultaneamente
- le operazioni `PUSH` e `POP` si tradurranno in opportune operazioni di `ENQUEUE` e `DEQUEUE` sulle due code `q1` e `q2`

Discuti la complessità delle operazioni `PUSH` e `POP`

Esercizi su pile e code

6. Scrivi lo pseudocodice della procedura `CONTAINS (p, x)` che ritorna `true` se l'elemento `x` è contenuto nella pila `p` e ritorna `false` se `x` non è contenuto (lasciando la pila invariata)
7. Scrivi lo pseudocodice di una struttura dati in cui si possa inserire/rimuovere elementi sia in testa che in coda
 - deve avere contemporaneamente `PUSH`, `POP`, `DEQUEUE`, `ENQUEUE` (dove evidentemente `ENQUEUE` è uguale a `PUSH`)

Algoritmi e Strutture di Dati

Liste implementate tramite array

m.patrignani

Nota di copyright

- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Sommario

- Il tipo astratto di dato lista
 - tipologie di liste e strategie di realizzazione
- Realizzazione delle liste con array
 - uso di tre array
 - uso di un solo array
 - liste disomogenee

Liste

- Le liste sono strutture di dati in cui gli oggetti sono disposti in una sequenza lineare
 - si assume che l'utente voglia scorrere gli elementi tramite un iteratore
- Tipo astratto: lista di interi
 - domini
 - il dominio di interesse è l'insieme delle liste L di interi
 - dominio di supporto: gli iteratori I che identificano le posizioni
 - dominio di supporto: gli interi $Z = \{0, 1, -1, 2, -2, \dots\}$
 - dominio di supporto: i booleani $B = \{\text{true}, \text{false}\}$

Realizzazione di una lista

- Costanti

- la lista vuota viene realizzata tramite la funzione `NEW_LIST(maxsize)`
 - ritorna il riferimento ad una lista vuota che può contenere al massimo `maxsize` elementi
- l'iteratore non valido è solitamente uno specifico valore dell'iteratore

- Operazioni di aggiornamento

- l'inserimento in testa alla lista `INSERT: L × Z → L` viene realizzato tramite la funzione `INSERT(l,x)`
- l'inserimento in coda `ADD: L × Z → L` viene realizzato tramite la funzione `ADD(l,x)`
- l'eliminazione di un elemento a partire dal suo iteratore `DELETE: L × I → L` viene realizzata tramite la funzione `DELETE(l,i)`
- la ricerca e l'eliminazione di un elemento `DELETE: L × Z → L` viene realizzata tramite la funzione `DELETE(l,x)`

Operazioni possibili sulle liste

- Altre operazioni di aggiornamento
 - l'inserimento prima di una posizione specifica
 $\text{INSERT_BEFORE: } L \times I \times Z \rightarrow L$ viene realizzato tramite la funzione $\text{INSERT_BEFORE}(l, x, i)$
 - l'inserimento dopo una posizione specifica
 $\text{ADD_AFTER: } L \times I \times Z \rightarrow L$ viene realizzato tramite la funzione $\text{ADD_AFTER}(l, x, i)$
 - lo svuotamento della lista $\text{EMPTY: } L \rightarrow L$ viene realizzato tramite la funzione $\text{EMPTY}(l)$

Operazioni possibili sulle liste

- Operazioni di consultazione

- l'iteratore del 1° elemento della lista HEAD: $L \rightarrow I$ si ottiene tramite la funzione HEAD(l)
 - ritorna l'iteratore non valido se la lista è vuota
- l'iteratore successivo all'iteratore corrente NEXT: $L \times I \rightarrow I$ si ottiene tramite la funzione NEXT(l, i)
 - ritorna l'iteratore non valido se i è l'iteratore dell'ultimo elemento
- l'iteratore precedente all'iteratore corrente PREV: $L \times I \rightarrow I$ si ottiene tramite la funzione PREV(l, i)
 - ritorna l'iteratore non valido se i è l'iteratore del primo elemento
- l'intero associato alla posizione specificata da un iteratore INFO: $L \times I \rightarrow Z$ si ottiene con la funzione INFO(l, i)

Operazioni possibili sulle liste

- Altre operazioni di consultazione
 - la ricerca della posizione di un elemento $\text{SEARCH: } L \times Z \rightarrow I$ si ottiene tramite la funzione $\text{SEARCH}(l, k)$
 - ritorna l'iteratore dell'elemento con chiave k nella lista l oppure l'iteratore non valido
 - l'iteratore associato all'ultimo elemento della lista $\text{LAST: } L \rightarrow I$ si ottiene tramite la funzione $\text{LAST}(l)$
 - ritorna l'iteratore non valido se la lista è vuota
 - la verifica se una lista è vuota $\text{IS_EMPTY: } L \rightarrow B$ è realizzata dalla funzione $\text{IS_EMPTY}(l)$
 - il numero degli elementi in lista $\text{SIZE: } L \rightarrow Z$ si ottiene tramite la funzione $\text{SIZE}(l)$

Strategie di realizzazione delle liste

- Dipendentemente dal tipo di operazioni che è necessario compiere in maniera efficiente sulla lista esistono diverse strategie implementative:

- lista concatenata

- consente lo scorrimento efficiente della lista in avanti ma non consente un efficiente scorrimento all'indietro

- lista doppiamente concatenata

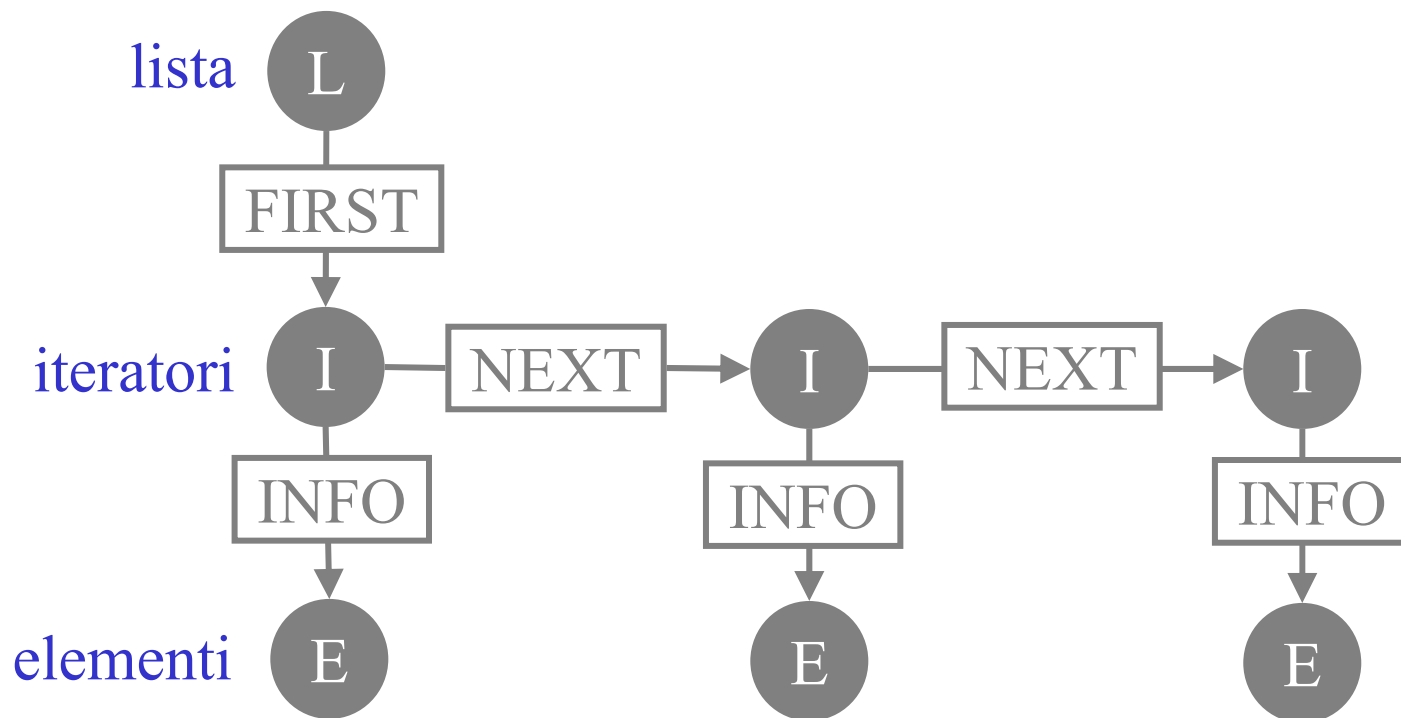
- supporta in maniera efficiente lo scorrimento in avanti e indietro

- accesso agli estremi

- consente un veloce accesso sia al primo che all'ultimo elemento della lista

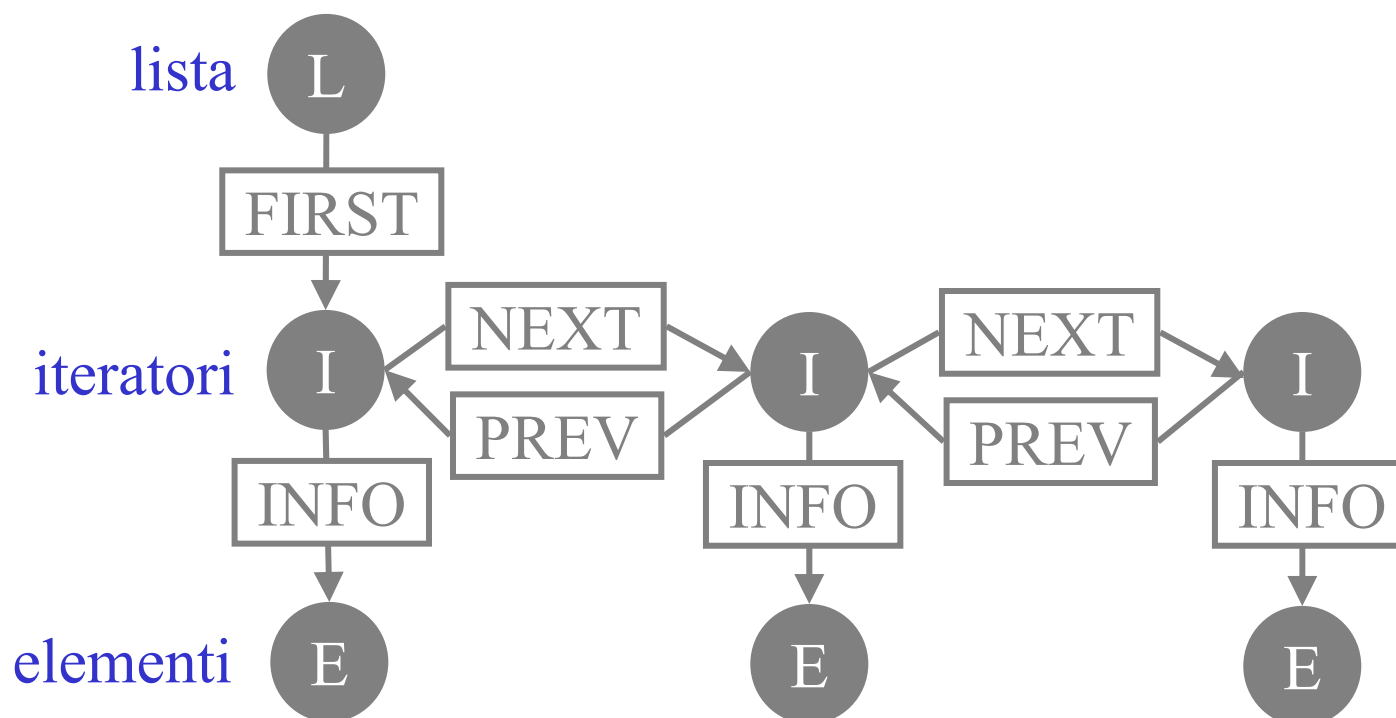
Lista semplicemente concatenata

- La struttura di dati supporta il passaggio diretto da un iteratore all'iteratore successivo
 - si vogliono le operazioni FIRST e NEXT in $\Theta(1)$



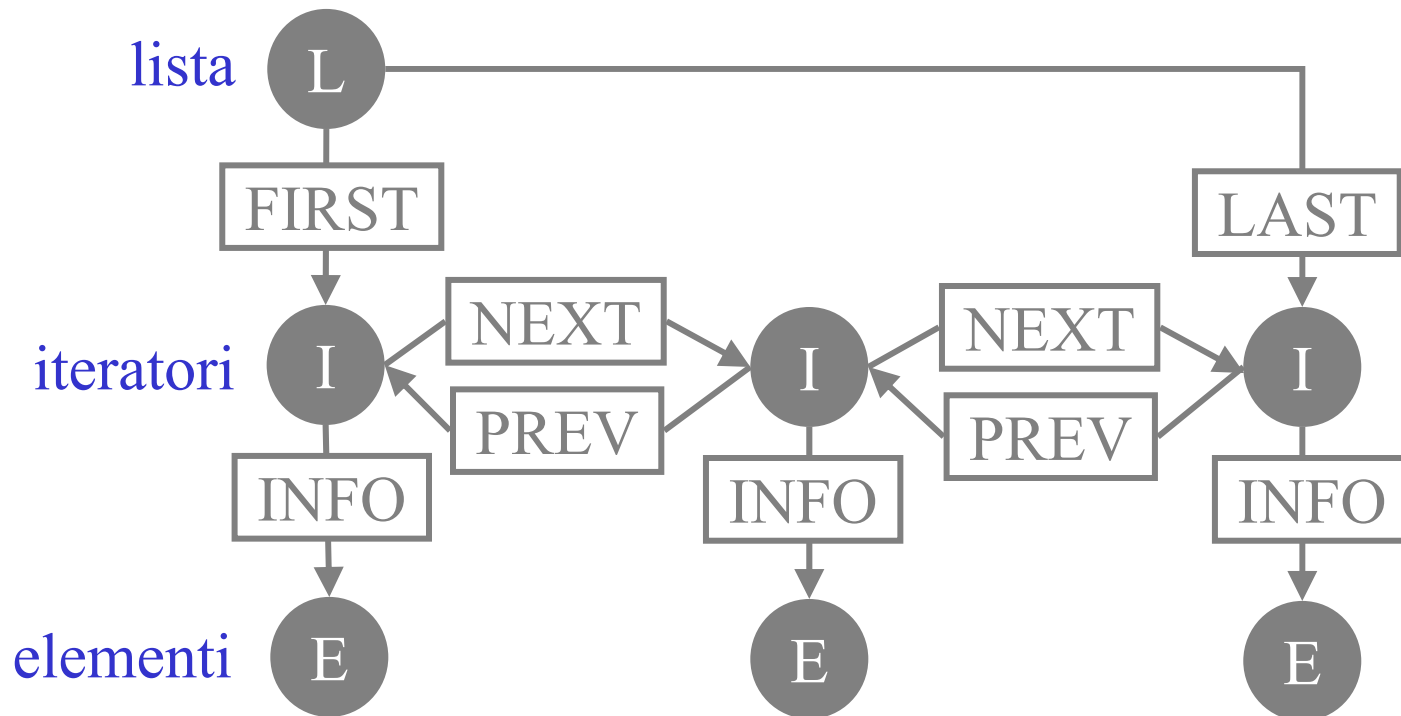
Lista doppiamente concatenata

- La struttura di dati supporta il passaggio diretto da un iteratore al successivo e al precedente
 - si vogliono le operazioni FIRST, NEXT e PREV in $\Theta(1)$



Liste con accesso agli estremi

- La struttura di dati supporta l'accesso diretto al primo e all'ultimo iteratore della lista
 - si vogliono le operazioni FIRST e LAST in $\Theta(1)$

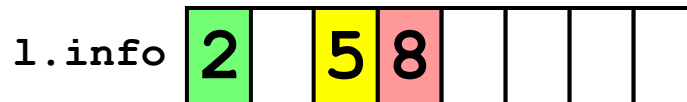


Realizzazione di una lista con array

- Supponiamo di mettere gli elementi della lista nelle celle successive di un array `l.info`



- come facciamo a gestire la cancellazione di un elemento intermedio della lista?

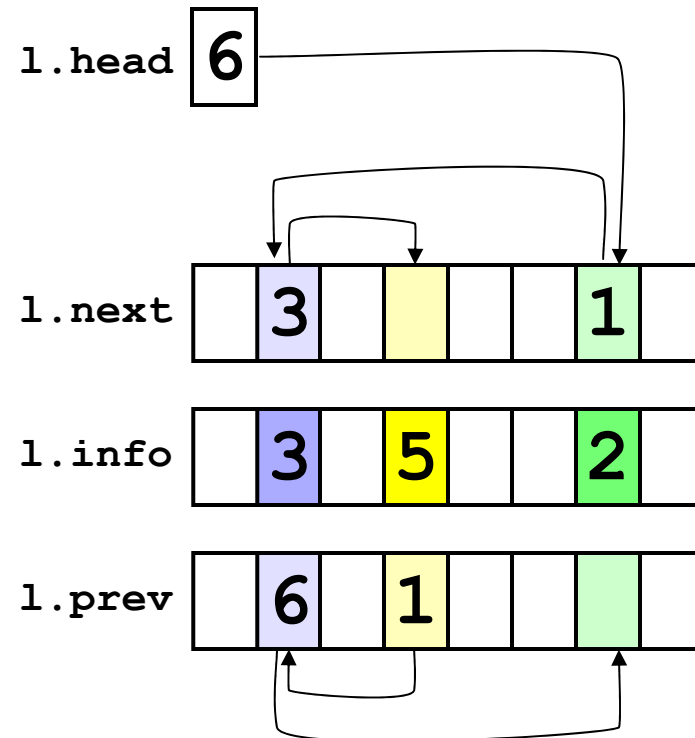


- come facciamo a sapere quali celle dell'array sono utilizzate?
 - non c'è nessun valore intero che possiamo associare ad una cella vuota

Realizzazione di una lista con array

- Gli elementi della lista sono memorizzati in un array `l.info`
- L'array `l.next` contiene l'indice dell'elemento che segue
- L'array `l.prev` contiene l'indice dell'elemento che segue

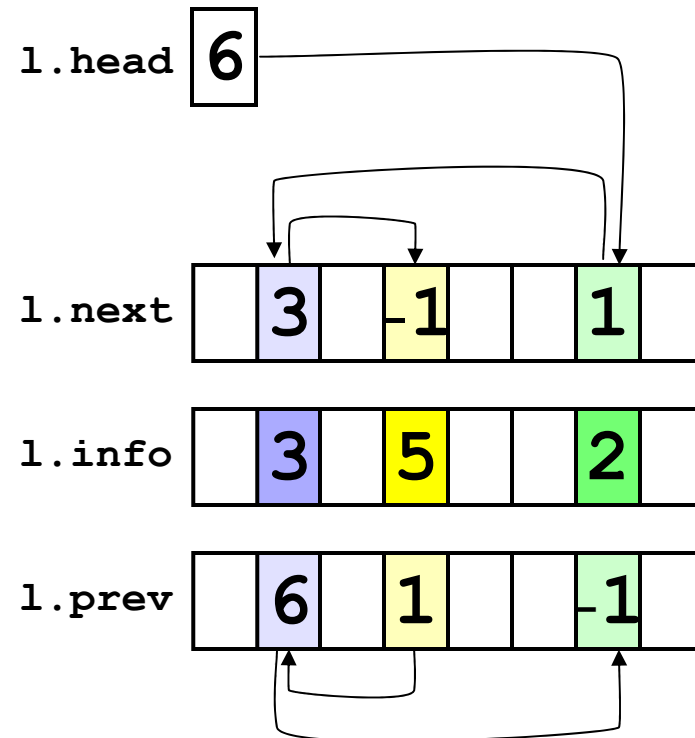
sequenza: 2 , 3 , 5



Realizzazione di una lista con array

- L'iteratore della lista è un intero
 - è l'indice della posizione dell'elemento corrispondente
- L'iteratore non valido è rappresentato dal valore -1

sequenza: 2 , 3 , 5

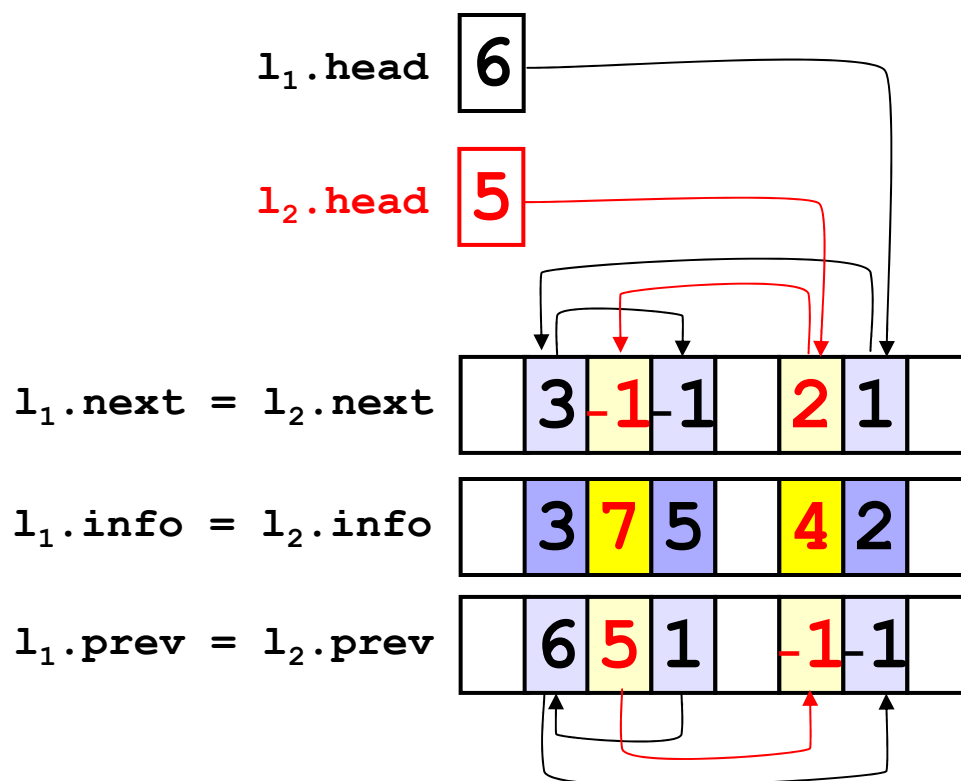


Più liste con gli stessi array?

- Gli array `l.next`, `l.info` ed `l.prev` possono essere condivisi da due o più liste
 - le liste non interferiscono, perché utilizzano posizioni diverse degli array

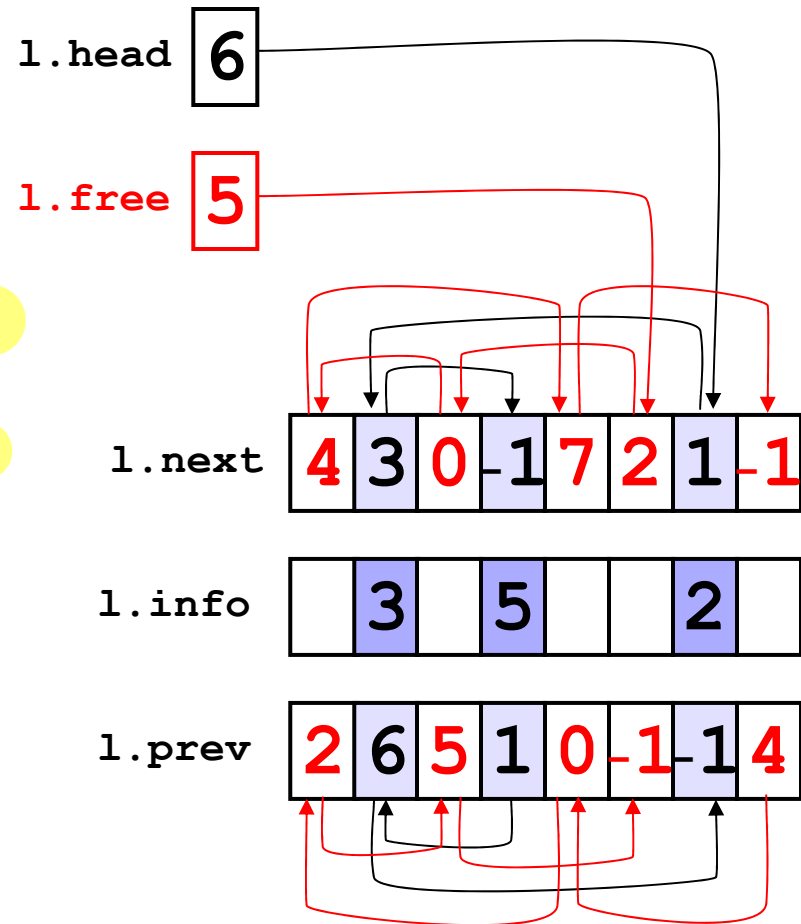
sequenza 1: 2 , 3 , 5

sequenza 2: 4 , 7



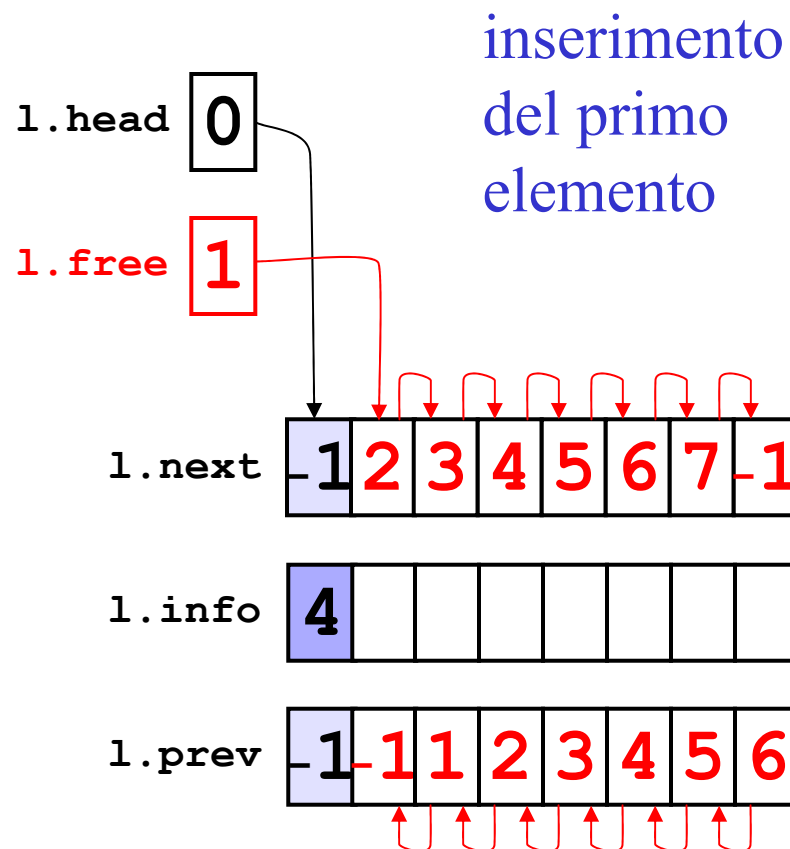
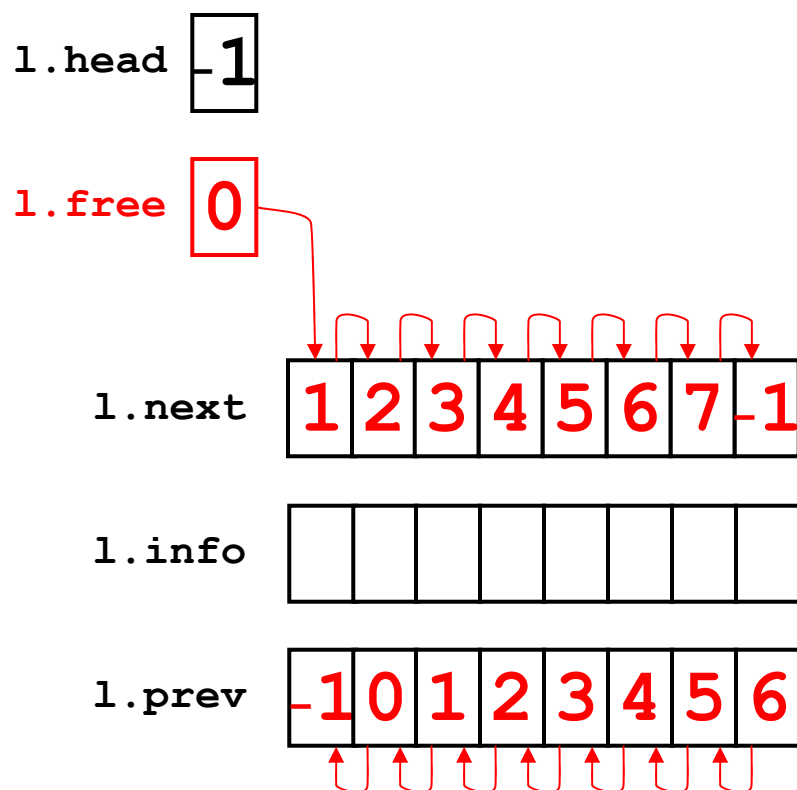
Uso della lista libera

- Per inserire un nuovo elemento in lista occorre sapere quali posizioni degli array sono ancora libere
- Tutte le posizioni libere possono essere memorizzate in una seconda lista `l.free`
 - un inserimento di un elemento in `l` è un trasferimento di una posizione dalla lista `l.free` alla lista `l.head`
 - una cancellazione da `l` è un trasferimento di una posizione dalla lista `l.head` alla lista `l.free`



Configurazione iniziale: lista vuota

- Quando la lista `l` è vuota tutte le posizioni sono assegnate alla lista `l.free`



Creazione di una lista vuota di interi

- Procedura per inizializzare una lista vuota di interi

NEW_LIST(maxsize)

```
1. ▷ creo un nuovo oggetto l con:
2. ▷      l.next, l.info, l.prev array di maxsize interi
3. ▷      l.head, l.free interi
4. for i = 0 to maxsize-1
5.     l.next[i] = i+1
6.     l.prev[i] = i-1
7. l.next[maxsize-1] = -1
8. l.head = -1
9. l.free = 0
10. return l
```

Gestione della lista `l.free`

- Procedura di servizio per ottenere una posizione libera dalla lista `l.free`

```
ALLOCATE-COLUMN(l)
```

```
1. if l.free == -1
```

```
2.     error("overflow")
```

```
3. else
```

```
4.     i = l.free      ▷ i è l'indice della nuova posizione
```

```
5.     l.free = l.next[l.free]
```

```
6.     if l.free != -1
```

```
7.         l.prev[l.free] = -1
```

```
8. return i
```


Gestione della lista `l.free`

- Procedura di servizio per restituire una posizione alla lista `l.free`

```
FREE-COLUMN(l, i)
```

```
1. l.prev[i] = -1
```

```
2. l.next[i] = l.free
```

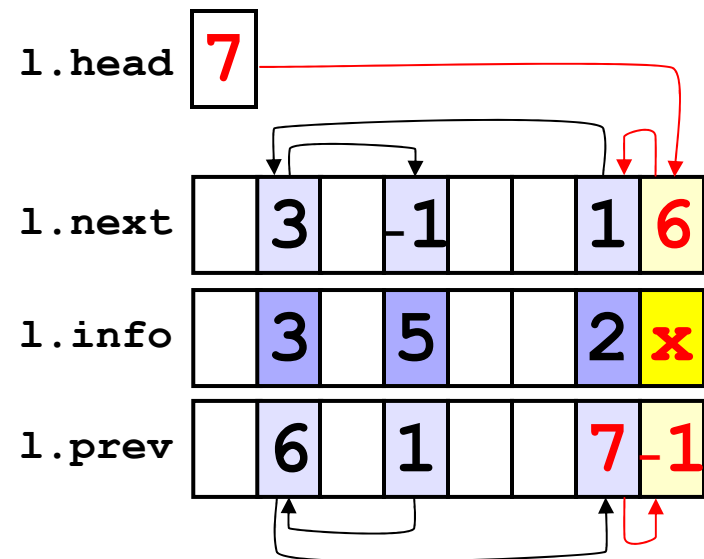
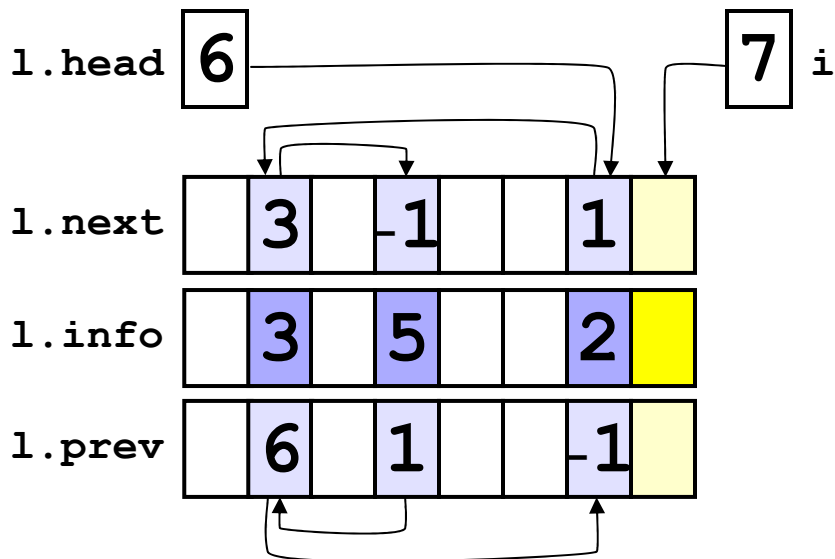
```
3. if l.free != -1
```

```
4.     l.prev[l.free] = i
```

```
5. l.free = i
```

Codice di INSERT(l,x)

```
INSERT(l,x)      ▷ x è il valore da aggiungere in lista  
1.  i = ALLOCATE-COLUMN(l)      ▷ indice di una nuova colonna libera  
2.  l.info[i] = x  
3.  l.prev[i] = -1                ▷ i diventa primo elemento  
4.  l.next[i] = l.head           ▷ il resto della lista segue i  
5.  if l.head != -1  
6.      l.prev[l.head] = i  
7.  l.head = i
```

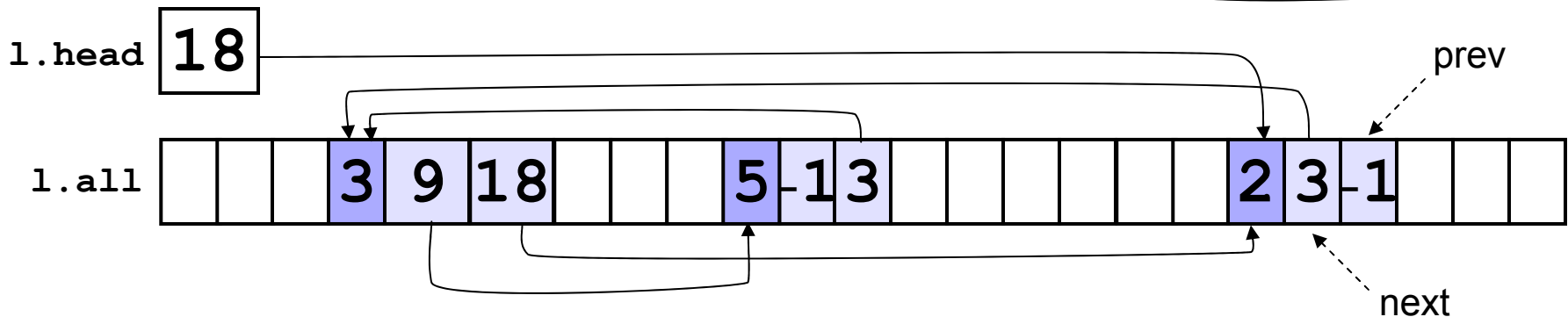
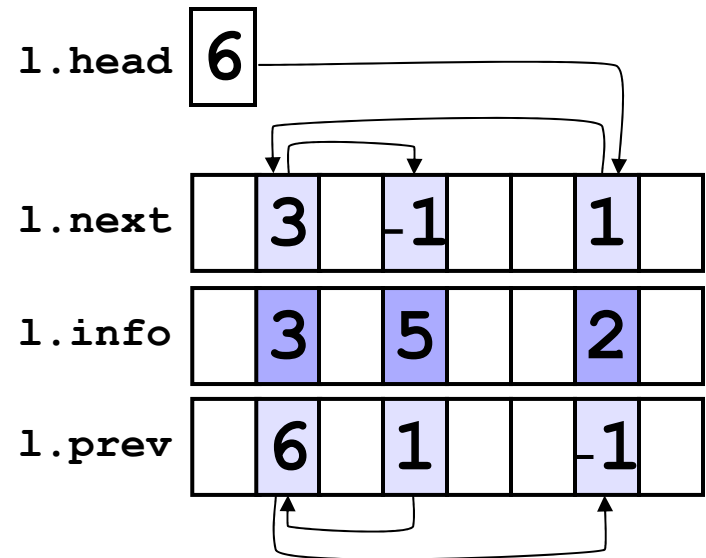


Esercizi: liste implementate con array

1. Scrivi lo pseudocodice della procedura `SIZE(l)` che conta gli elementi della lista `l`
2. Scrivi lo pseudocodice della procedura `SEARCH(l,k)` che restituisce la posizione del primo elemento di `l` con valore della chiave `k`
3. Scrivi lo pseudocodice della procedura `DELETE(l,i)` che rimuove da `l` l'elemento in posizione `i`
4. Scrivi lo pseudocodice della procedura `DELETE(l,x)` che rimuove da `l` il primo elemento che ha valore `x`

Rappresentazione con un solo array

- Un solo array è sufficiente a rappresentare le informazioni degli array `l.next`, `l.info` ed `l.prev`
- Ovviamente è anche necessaria una lista `l.free` (non rappresentata in figura)



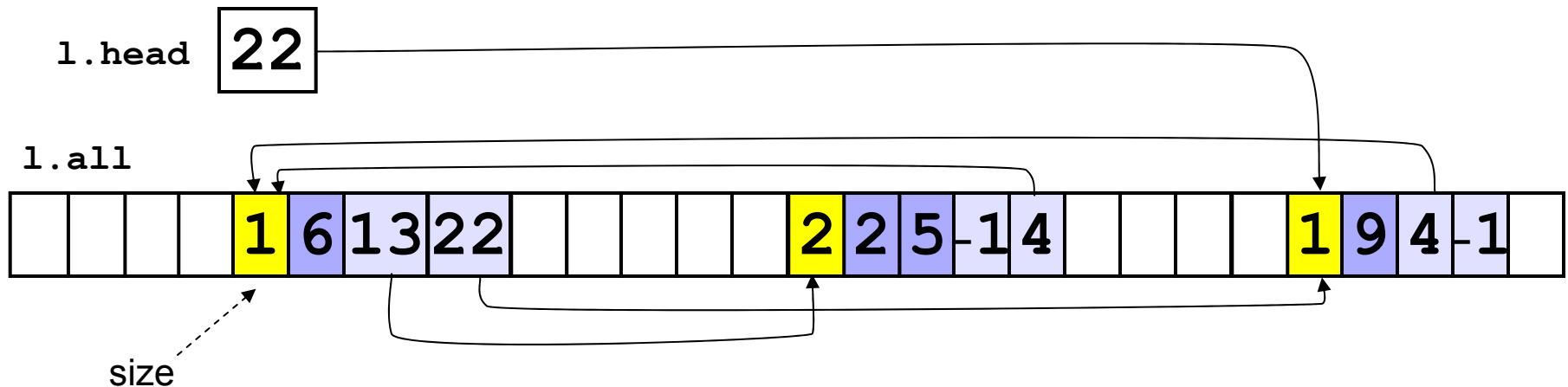
Esercizi: liste su un solo array

Negli esercizi seguenti supponi che la lista l sia doppiamente concatenata ed implementata tramite un solo array

5. Scrivi lo pseudocodice della procedura `ALLOCATE_OBJECT(l)`
6. Scrivi lo pseudocodice della procedura `FREE_OBJECT(l,i)`
7. Scrivi lo pseudocodice della procedura `LOWER_FREE_POSITION(l)` che trova la posizione con indice più basso tra gli elementi della lista libera `l.free`

Liste con elementi eterogenei

- L'uso di un singolo array consente la gestione di liste di elementi eterogenei
 - un campo aggiuntivo specifica la dimensione di ogni elemento
 - la lista `l.free` (non rappresentata in figura) viene gestita con criteri analoghi



Esercizi: liste eterogenee

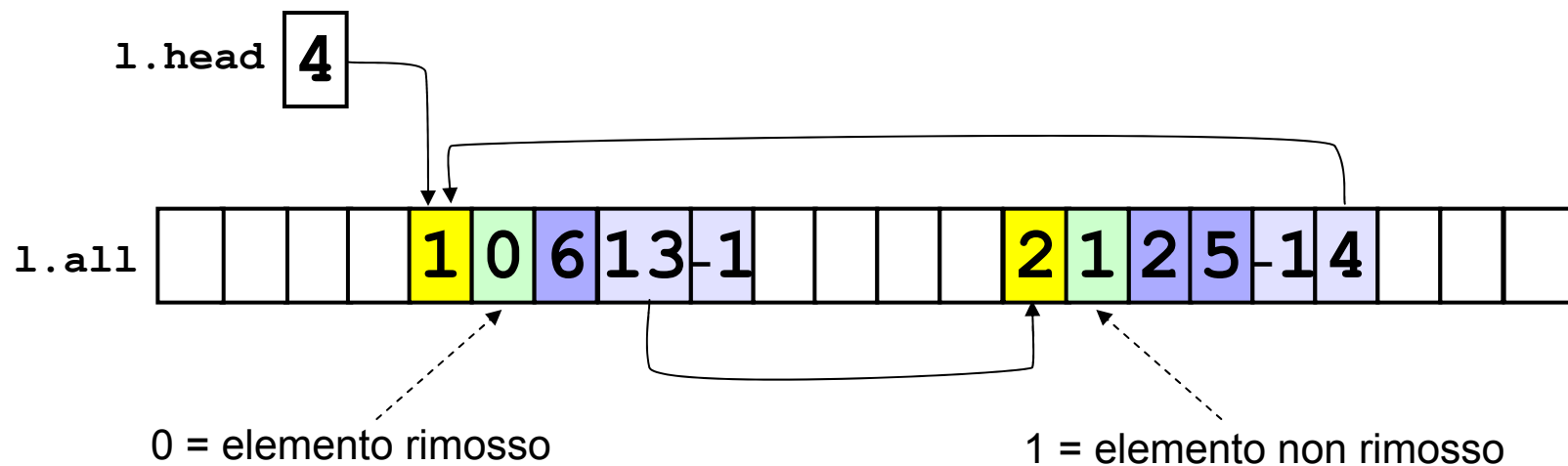
negli esercizi seguenti supponi che la lista l sia eterogenea, doppiamente concatenata, ed implementata tramite un singolo array

8. Scrivi lo pseudocodice della procedura `ALLOCATE-OBJECT-WITH-SIZE( $l, x$ )` che trova nella lista libera `l.free` una posizione adatta ad ospitare un elemento di dimensione `x`
9. Scrivi lo pseudocodice della procedura `FREE-OBJECT( $l, i$ )` che inserisce nella lista libera `l.free` l'oggetto in posizione `i`
10. Scrivi lo pseudocodice della procedura `INSERT( $l, A$ )` che inserisce nella lista `l` un elemento di dimensione `A.length` che contiene tutti i valori dell'array `A`

Esercizio: cancellazione *lazy* su liste eterogenee

Negli esercizi seguenti supponi che una lista eterogenea doppiamente concatenata ed implementata con un singolo array preveda, per ogni elemento, un valore che specifica se l'elemento è da considerarsi “rimosso” oppure no

11. Scrivi lo pseudocodice della procedura DELETE(*l*,*i*) dove *i* è l'indice della posizione di un elemento da marcare come “rimosso”
12. Scrivi lo pseudocodice della procedura GARBAGE-COLLECTION(*l*) che trasferisce nella lista *l.free* tutti gli elementi marcati come “rimossi”



Algoritmi e Strutture di Dati

Implementazioni di liste con
oggetti e riferimenti

M. Patrignani

Nota di copyright

- Queste slides sono protette dalle leggi sul copyright
- Il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- Le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- Ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- Gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- Questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Strutture di dati con oggetti e riferimenti

- alcuni linguaggi supportano oggetti e riferimenti
 - lo pseudocodice è uno di questi
- con oggetti e riferimenti si possono realizzare strutture di dati elementari in modo più naturale
- vedremo la realizzazione del tipo astratto di dato lista
 - pile e code possono essere rivisti come casi particolari di liste
 - due principali varianti implementative:
 - lista singolarmente concatenata
 - lista doppiamente concatenata

Operazioni su una lista di interi

NEW_LIST()	ritorna il riferimento ad una lista vuota
HEAD(l)	ritorna l'iteratore del primo elemento della lista ritorna un iteratore invalido se la lista è vuota
LAST(l)	ritorna l'iteratore dell'ultimo elemento della lista ritorna un iteratore invalido se la lista è vuota
NEXT(l,i)	ritorna l'iteratore dell'elemento che segue i nella lista ritorna un iteratore invalido se i è l'ultimo elemento
PREV(l,i)	ritorna l'iteratore dell'elemento che precede i nella lista ritorna un iteratore invalido se i è il primo elemento

Operazioni su una lista di interi

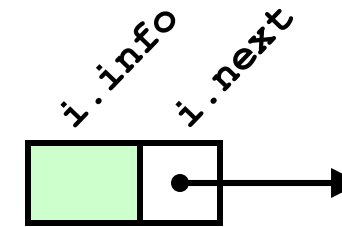
<code>INSERT(l,n)</code>	inserisce l'elemento <code>n</code> in testa alla lista <code>l</code>
<code>INSERT_BEFORE(l,n,i)</code>	inserisce l'elemento <code>n</code> prima della posizione <code>i</code>
<code>ADD(l,n)</code>	aggiunge <code>n</code> in coda alla lista <code>l</code>
<code>ADD_AFTER(l,n,i)</code>	aggiunge l'elemento <code>n</code> dopo la posizione <code>i</code>
<code>DELETE(l,i)</code>	rimuove l'elemento in posizione <code>i</code> dalla lista <code>l</code>
<code>DELETE(l,n)</code>	rimuove l'elemento <code>n</code> dalla lista <code>l</code>
<code>EMPTY(l)</code>	vuota la lista
<code>SEARCH(l,n)</code>	ritorna l'iteratore dell'elemento <code>n</code> nella lista <code>l</code>
<code>IS_EMPTY(l)</code>	ritorna <code>true</code> se la lista è vuota, altrimenti ritorna <code>false</code>

Lista concatenata (singly linked list)

- l'iteratore `i` è un riferimento ad un nodo della lista, che è un oggetto composto dai seguenti attributi

- `i.info`

- elemento in lista del tipo opportuno
- può essere un riferimento ad un oggetto esterno con dati satellite



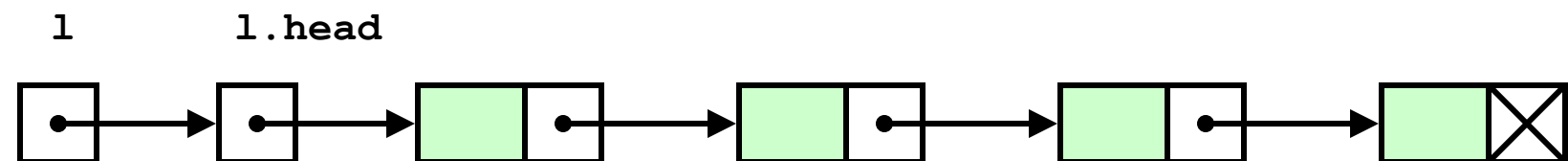
- `i.next`

- riferimento al nodo seguente o `NULL`

- una lista `l` ha un solo attributo

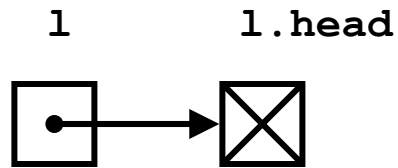
- `l.head`

- riferimento al primo nodo



Lista concatenata: lista vuota

- Quando la lista `l` è vuota `l.head` è `NULL`



- Pseudocodice delle procedure `IS_EMPTY` e `EMPTY`

```
IS_EMPTY(l)
```

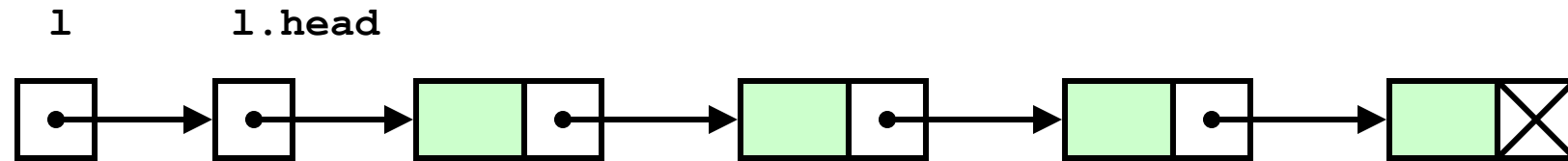
```
1. return l.head == NULL
```

```
EMPTY(l)
```

```
1. l.head = NULL      ▷ lo pseudocodice non dealloca memoria
```

Lista concatenata: first e next

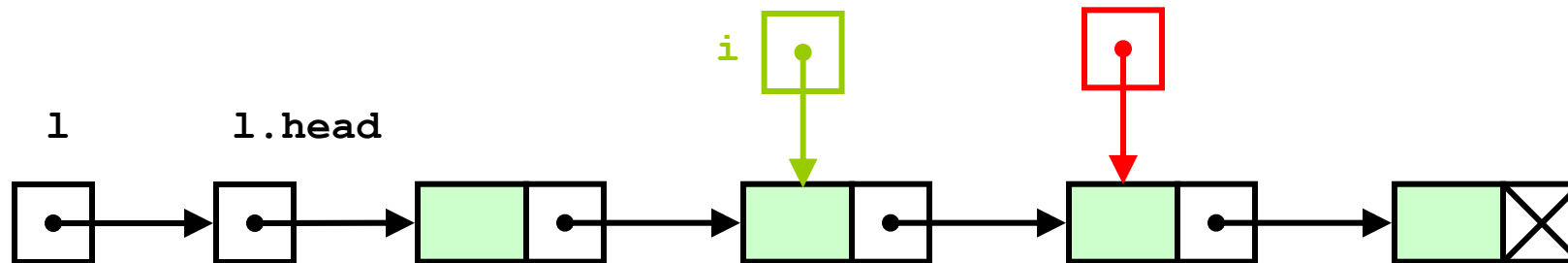
- **FIRST:** iteratore dell'elemento affiorante



FIRST(1)

1. return l.head ▷ potrebbe essere NULL

- **NEXT:** prossimo elemento



NEXT(l, i)

1. return i.next ▷ il parametro l non è utilizzato

Realizzazione di funzioni elementari

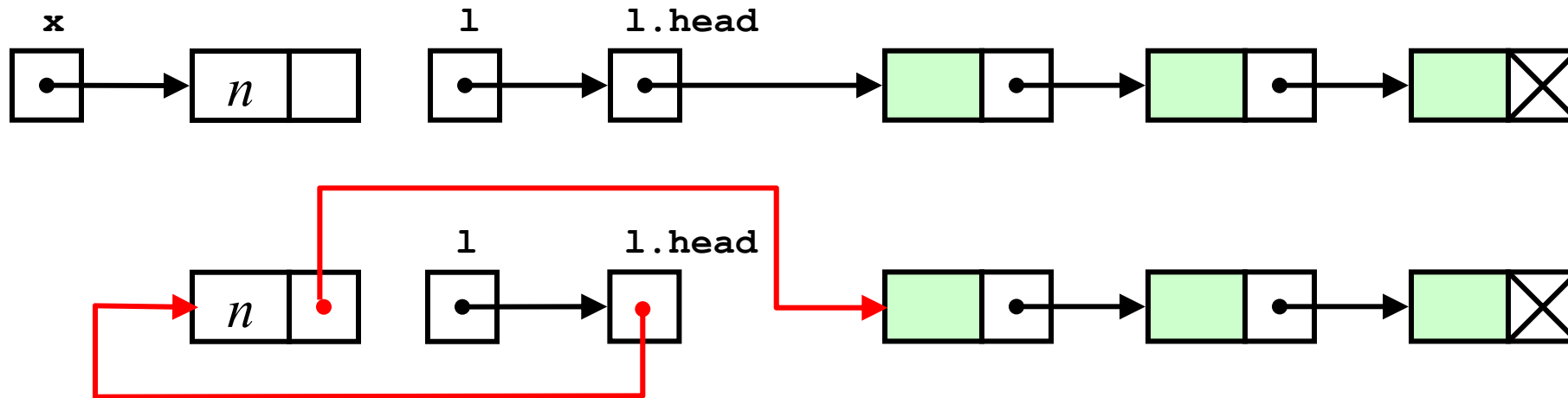
- La semplicità di alcune funzioni (come `IS_EMPTY`, `EMPTY`, `FIRST`, `NEXT`, ecc) induce a sostituirle con le istruzioni opportune direttamente nello pseudocodice
 - questo ovviamente fa perdere di generalità al codice scritto
- Esempio

```
...  
if !IS-EMPTY(l)  
then ...  
...  
x = NEXT(l, x)  
...
```

```
...  
if l.head != NULL  
then ...  
...  
x = x.next  
...
```

Lista concatenata: inserimento in testa

- INSERT: inserimento di n in testa alla lista



INSERT (l, n)

▷ n è un intero

1. $x.info = n$

▷ x è un nuovo oggetto con due campi:

2.

▷ $x.info$ (intero)

3.

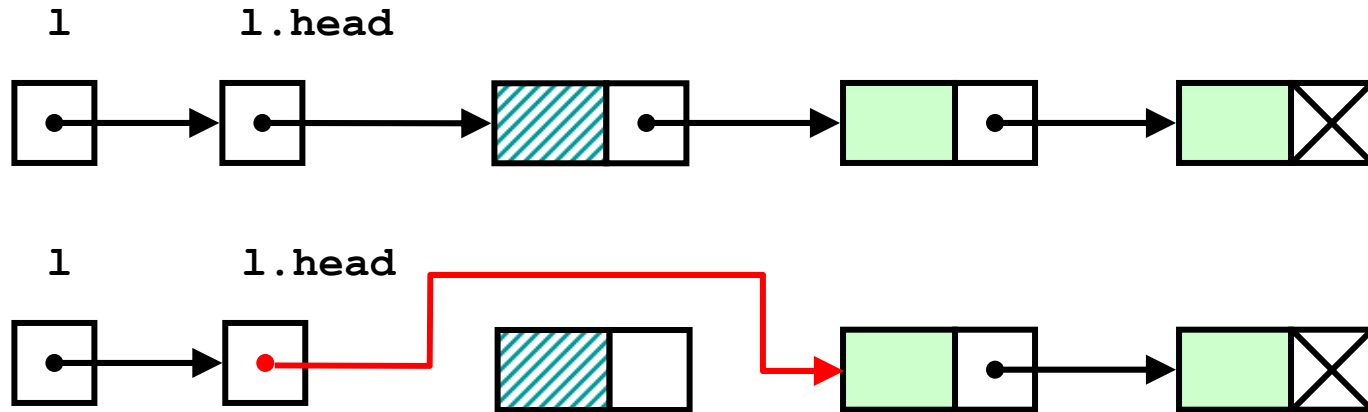
▷ $x.next$ (rif. ad analogo oggetto)

4. $x.next = l.head$

5. $l.head = x$

Lista concatenata: cancellazione

■ DELETE_FIRST: rimozione del primo nodo



```
DELETE_FIRST(l)
```

```
1. ▷ NOTA: lo pseudocodice non dealloca l'elemento
```

```
2. if l.head == NULL
```

```
3.     error("lista vuota")
```

```
4. else
```

```
5.     l.head = l.head.next
```

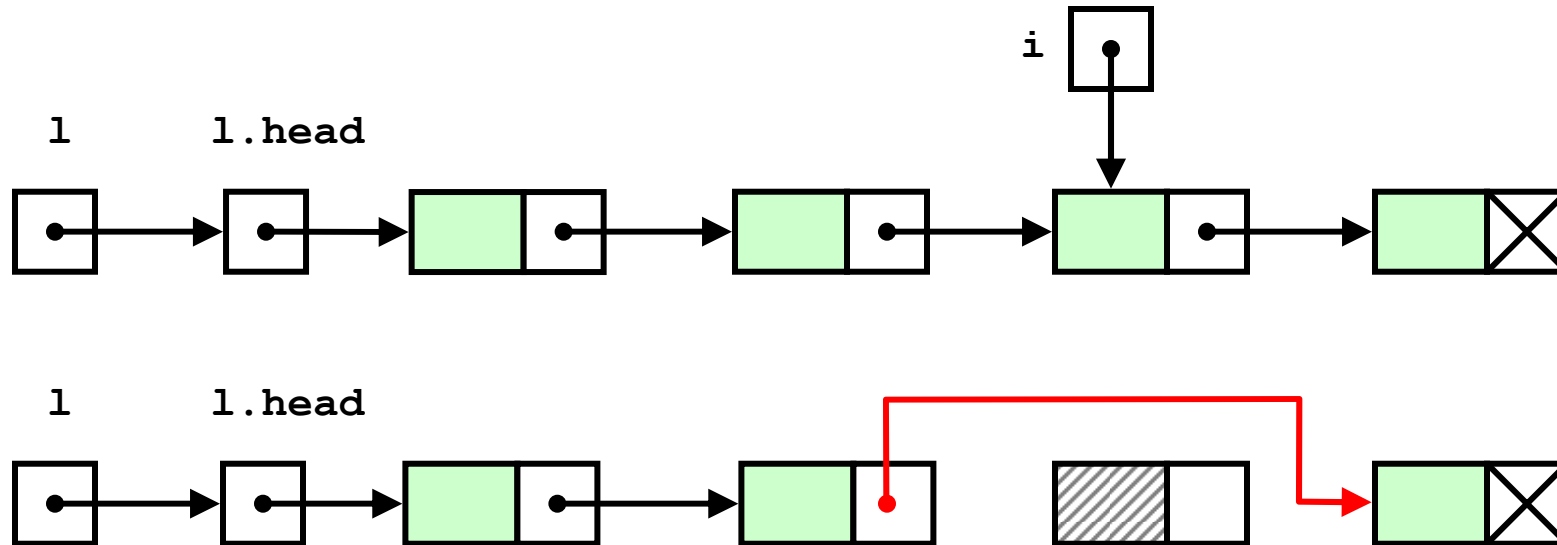
Esercizi: liste singolarmente concatenate

Esercizi sullo scorrimento delle liste

1. Scrivi lo pseudocodice della procedura `SOMMA(l)` che ritorna la somma degli elementi contenuti in una lista singolarmente concatenata di interi
2. Scrivi lo pseudocodice della procedura `MASSIMO(l)` che ritorna il valore del massimo elemento contenuto in una lista singolarmente concatenata di interi
 - assumi che la lista non sia mai vuota

Lista concatenata: cancellazione

- `DELETE(l,i)`: cancellazione del nodo i
- La cancellazione di un nodo diverso dal primo è poco efficiente in una lista singolarmente concatenata



- Occorre infatti modificare l'attributo `next` del nodo che lo precede

Esercizi: liste singolarmente concatenate

3. Scrivi lo pseudocodice della procedura `SEARCH(l,u)` che ritorna il riferimento all'elemento `i` che contiene il valore intero `u` in una lista singolarmente concatenata di interi (oppure `NULL` se `u` non è nella lista)
 - discuti la complessità dell'algoritmo in funzione del numero `n` degli elementi in lista
4. Scrivi lo pseudocodice della procedura `PREV(l,i)` che ritorna il riferimento all'elemento che precede l'elemento identificato dall'iteratore `i` in una lista singolarmente concatenata di interi (oppure `NULL` se `i` corrisponde al primo elemento della lista)

Esercizi: liste singolarmente concatenate

5. Scrivi lo pseudocodice dell'operazione $\text{DELETE}(l, i)$ che cancella il nodo i di una lista singolarmente concatenata
 - discuti della complessità dell'algoritmo
6. Scrivi lo pseudocodice dell'operazione $\text{DELETE}(l, u)$ che cancella il nodo che contiene il valore intero u in una lista singolarmente concatenata di interi
 - discuti della complessità dell'algoritmo

Esercizi: liste singolarmente concatenate

7. Implementa una pila di interi utilizzando oggetti e riferimenti

- devi realizzare le funzioni `NEW_STACK()`, `IS_EMPTY(p)`, `PUSH(p,u)`, e `POP(p)` facendo uso di oggetti e riferimenti

8. Implementa una coda di interi utilizzando oggetti e riferimenti

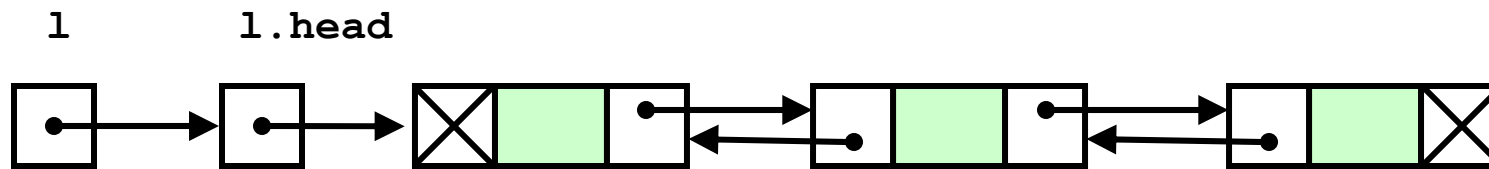
- devi realizzare le funzioni `NEW_QUEUE()`, `IS_EMPTY(c)`, `ENQUEUE(c,u)`, e `DEQUEUE(c)` facendo uso di oggetti e riferimenti

Esercizi: liste singolarmente concatenate

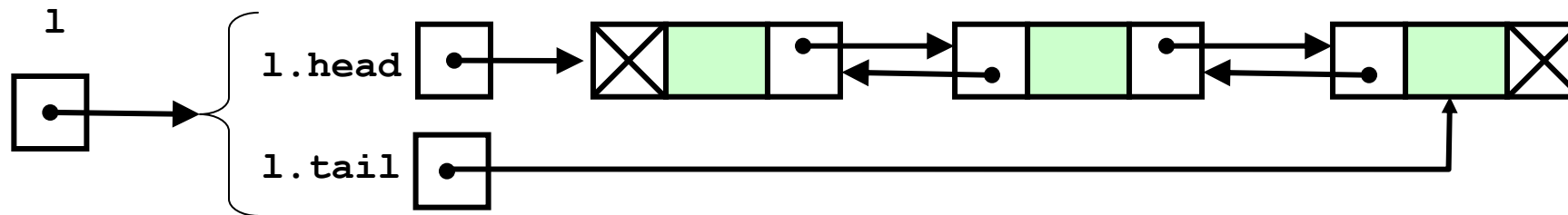
9. Scrivi lo pseudocodice della procedura $\text{COMUNI}(l_1, l_2)$ che ritorna il numero di elementi della lista l_1 che sono anche contenuti nella lista l_2
 - discuti la complessità dell'algoritmo proposto
10. Scrivi lo pseudocodice della procedura non ricorsiva $\text{INVERSA}(l)$ che ritorna una nuova lista singolarmente concatenata in cui gli elementi sono in ordine inverso
11. Scrivi lo pseudocodice della procedura $\text{ACCODA}(l_1, l_2)$ che accoda gli elementi della lista l_2 alla lista l_1 mantenendo l'ordine relativo che gli elementi avevano nelle liste originarie
 - puoi supporre di poter modificare le liste in input

Lista doppiamente concatenata

- Oltre all'attributo `next` i nodi dispongono anche dell'attributo `prev`

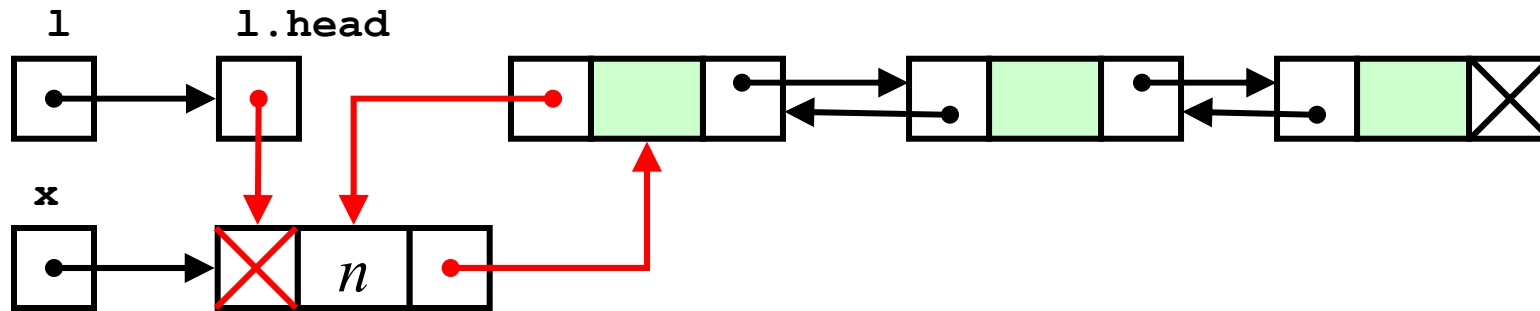
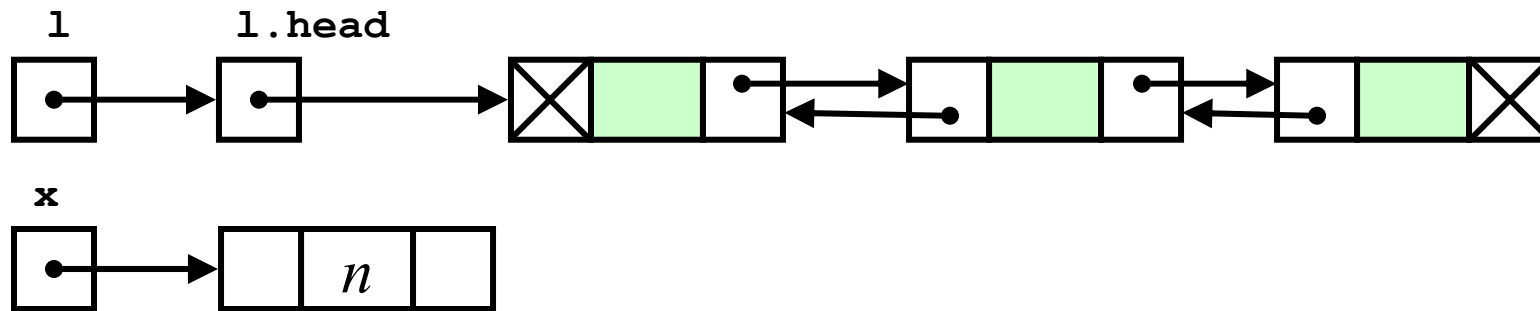


- Talvolta la lista `l` dispone anche di un attributo `l.tail`



Inserimento nella lista

- INSERT(l, n): inserimento in testa alla lista



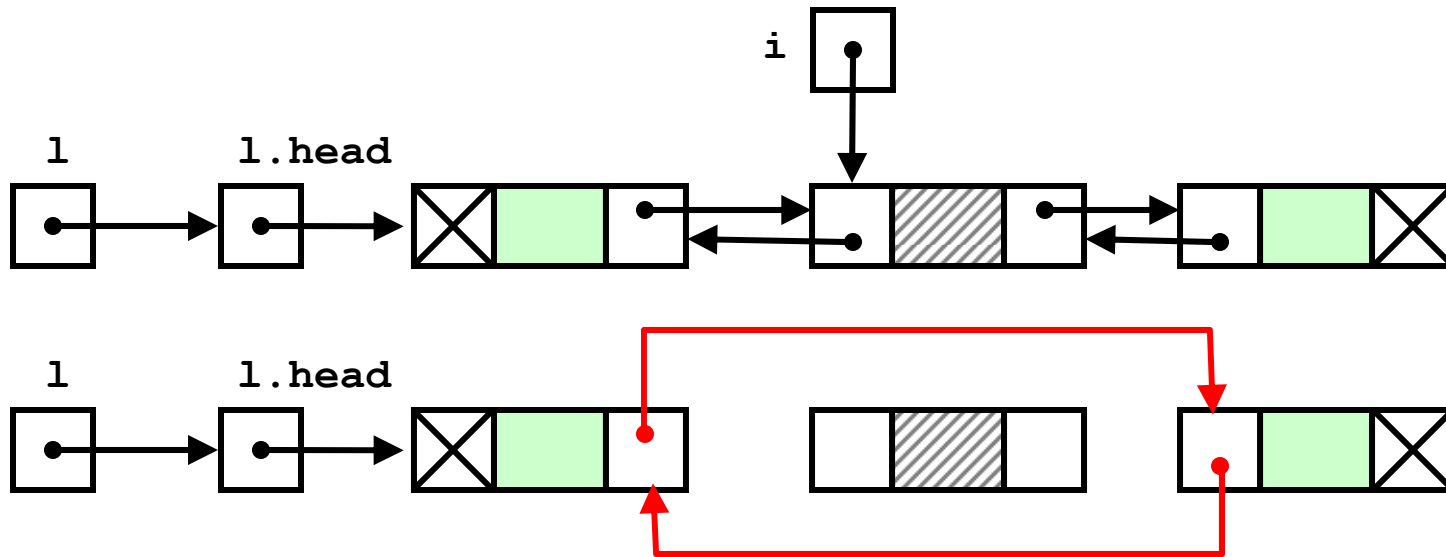
Inserimento nella lista

- **INSERT(l,n):** inserimento in testa alla lista

INSERT (l,n)

```
1. ▷ x è un nuovo oggetto con tre campi:
2. ▷   x.info (intero)
3. ▷   x.prev, x.next (riferimenti ad oggetti analoghi)
4. x.info = n
5. x.next = l.head
6. x.prev = NULL
7. if l.head != NULL
8.   l.head.prev = x
9. l.head = x
```

Cancellazione di un elemento



```
DELETE(l, i)
```

```
1. if i.prev != NULL
```

```
2.   i.prev.next = i.next
```

```
3. else
```

```
4.   l.head = i.next
```

```
5. if i.next != NULL
```

```
6.   i.next.prev = i.prev
```

Esercizi su liste doppiamente concatenate

12. Scrivi lo pseudocodice dell'operazione

`INSERT_BEFORE(l, n, i)` che riceva come parametri una lista doppiamente concatenata l , un intero n ed un iteratore i , e inserisca n nella lista prima dell'elemento riferito da i

- discuti la complessità della procedura

13. Scrivi lo pseudocodice dell'operazione

`ADD_AFTER(l, n, i)` che riceva come parametri una lista doppiamente concatenata l , un intero n ed un iteratore i , e inserisca n nella lista dopo l'elemento riferito da i

- discuti la complessità della procedura

Esercizi su liste doppiamente concatenate

14. Implementa una coda utilizzando una lista doppiamente concatenata

- è possibile che le operazioni `ENQUEUE` e `DEQUEUE` abbiano entrambe complessità $\Theta(1)$?
- come si potrebbe fare per ottenere questo risultato?

15. Scrivi lo pseudocodice della procedura `DELETE(l,u)` che rimuova l'elemento che ha valore `u` da una lista doppiamente concatenata di interi

- discuti la complessità dell'algoritmo

Esercizi su liste doppiamente concatenate

16. Scrivi lo pseudocodice della procedura `INSERT_ORDERED( $l, u$ )` che inserisca nella lista l (che si suppone ordinata in senso crescente) un intero u mantenendo l'ordinamento crescente della lista
17. Scrivi lo pseudocodice della procedura `MERGE( $l_1, l_2$ )` che accetti come parametri due liste doppiamente concatenate di interi ordinate in senso crescente e restituisca una lista ordinata in senso crescente con gli elementi di entrambe
- puoi supporre che tutti gli elementi delle liste siano diversi

Esercizi sulle liste ordinate

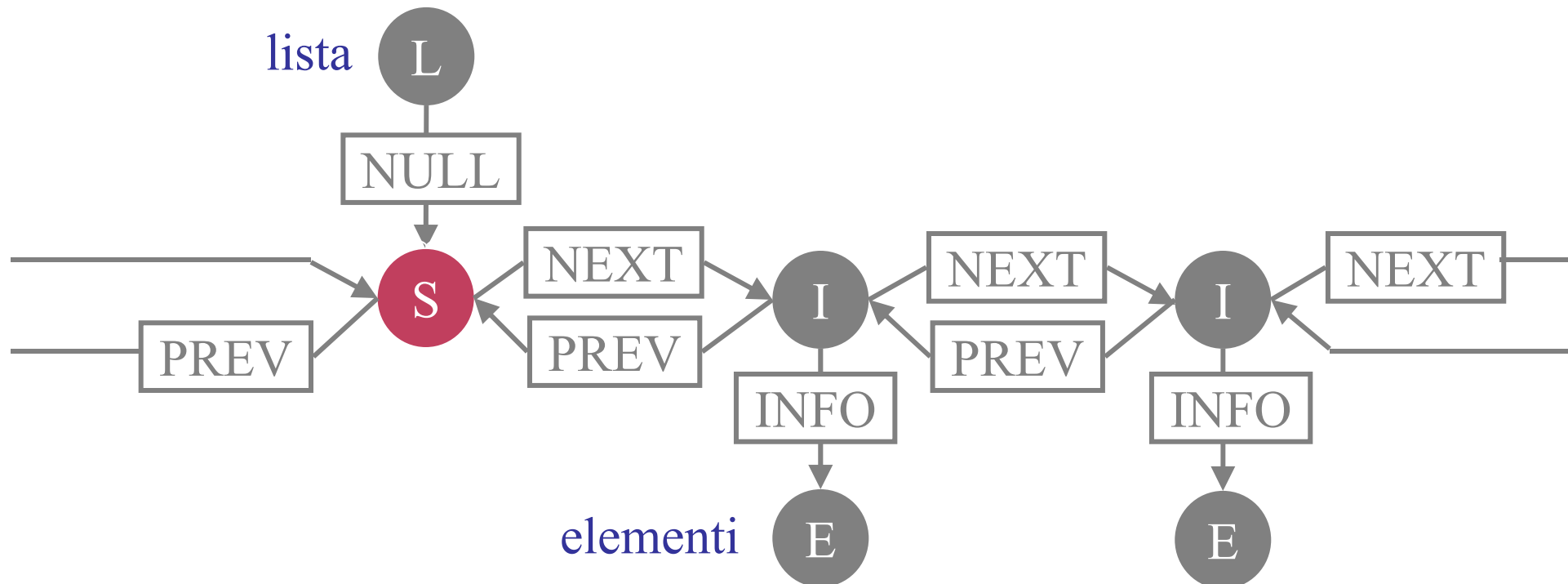
18. Scrivi lo pseudocodice della procedura `DOPPIONI(l)` che verifichi che una lista (non ordinata) doppiamente concatenata di interi non abbia doppioni
19. Scrivi lo pseudocodice della procedura `DOPPIONI_SORTED(l)` che verifichi che una lista doppiamente concatenata di interi ordinata in senso non-decrescente non abbia doppioni

Liste con sentinelle

- Le liste realizzate con oggetti e puntatori offrono l'opportunità di introdurre speciali iteratori chiamati “sentinelle”
- Il primo iteratore della lista (la “sentinella”) è sempre presente e non ha nessun elemento associato
- L'iteratore non valido coincide con l'iteratore che identifica la sentinella
- La struttura dati è circolare

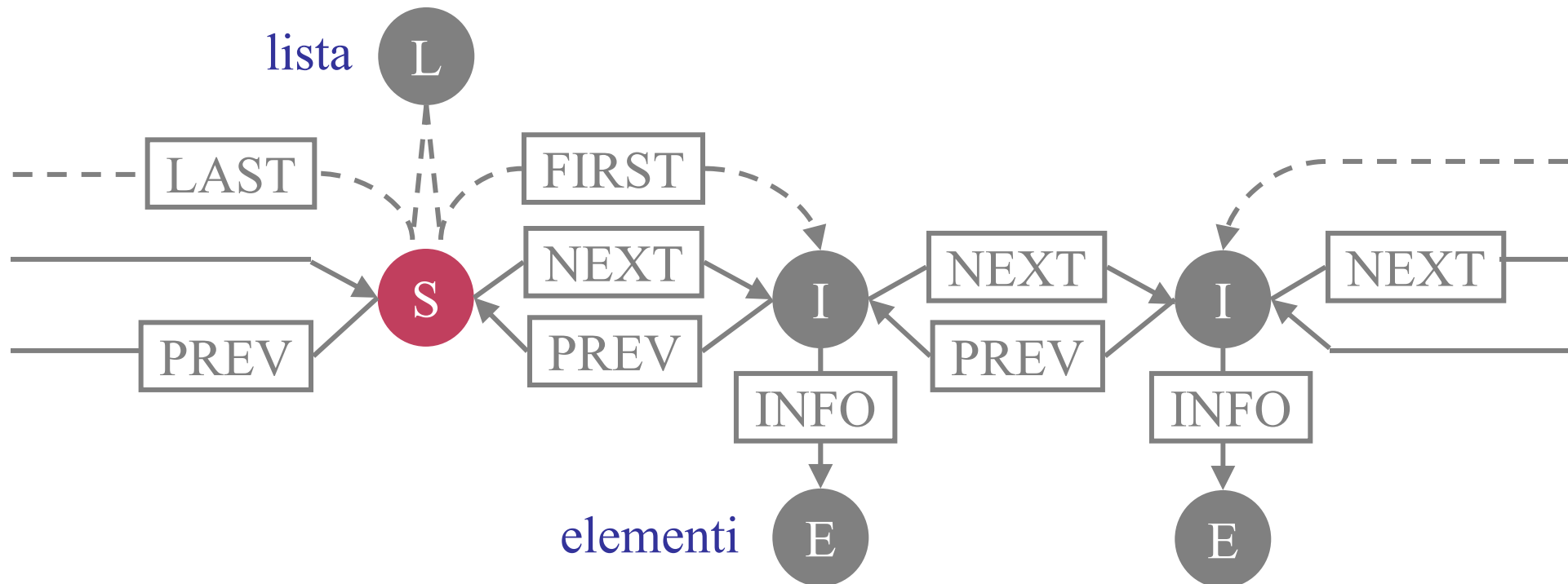
Liste con sentinelle

- Dalla lista si accede direttamente (ed esclusivamente) all'iteratore non-valido, cioè alla sentinella



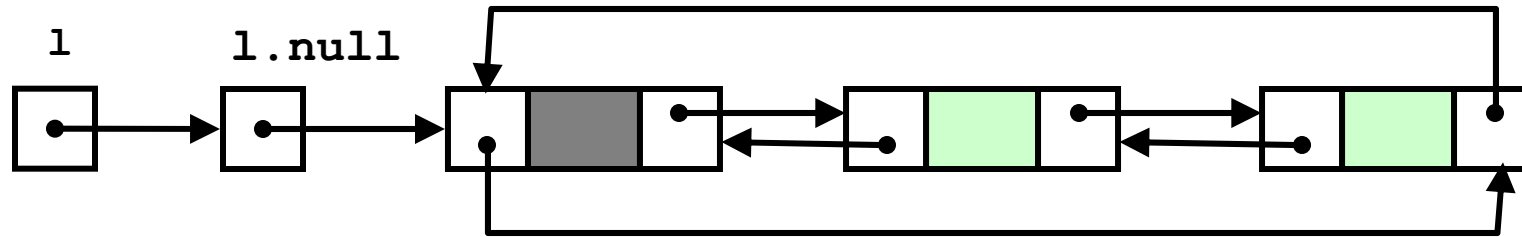
Uso della sentinella

- Concatenando NULL+NEXT si ottiene FIRST
- Concatenando NULL+PREV si ottiene LAST
- Questa strategia comporta diversi altri vantaggi
 - molte procedure risulteranno semplificate

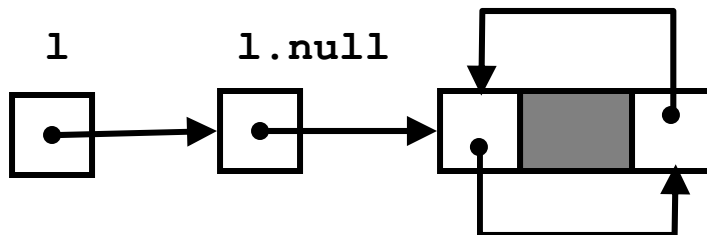


Realizzazione della sentinella

- La sentinella è un nodo fittizio introdotto in testa alla lista



- La lista vuota contiene solamente la sentinella



Procedure nelle liste con sentinelle

- Esempi di procedura semplificata dall'uso di sentinelle
 - lista doppiamente concatenata (senza sentinella)

DELETE(l,i) ▷ versione senza sentinella ($i \neq \text{NULL}$)

```
1. if i.prev != NULL
2.     i.prev.next = i.next
3. else
4.     l.head = i.next
5. if i.next != NULL
6.     i.next.prev = i.prev
```

- lista doppiamente concatenata con sentinella

DELETE(l,i) ▷ versione con sentinella ($i \neq l.\text{null}$)

```
1. i.prev.next = i.next
2. i.next.prev = i.prev
```

Esercizi sulle liste con sentinelle

20. Scrivi lo pseudocodice della procedura `INSERT( $l, n$ )` che inserisce in testa ad una lista con sentinella l un intero n
21. Scrivi lo pseudocodice della procedura `SEARCH( $l, n$ )` che ritorna un iteratore all'elemento della lista con sentinella l che ha valore n
- `SEARCH( $l, n$ )` ritorna `$l.null$` se n non è presente nella lista l

Algoritmi e Strutture di Dati

Alberi radicati

m.patrignani

Nota di copyright

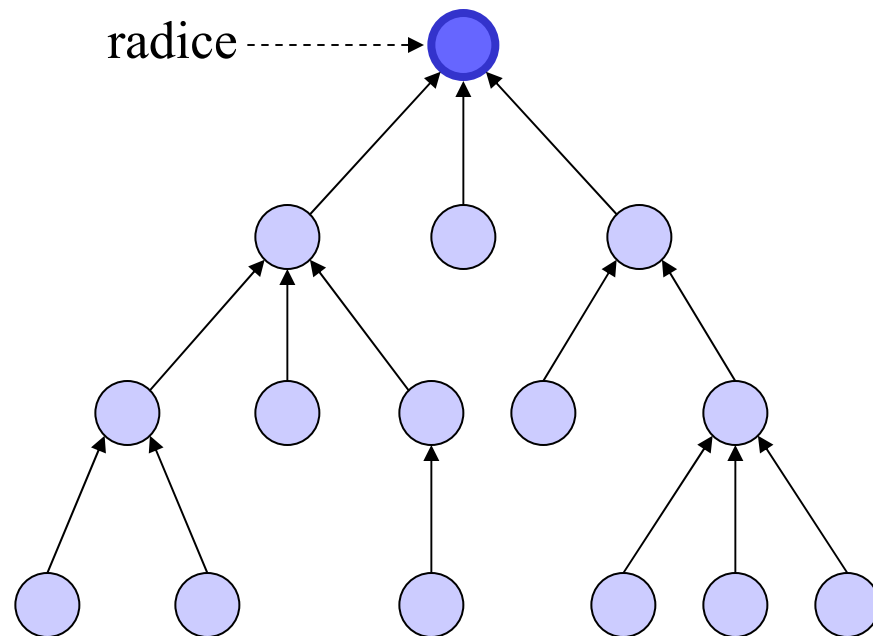
- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Sommario

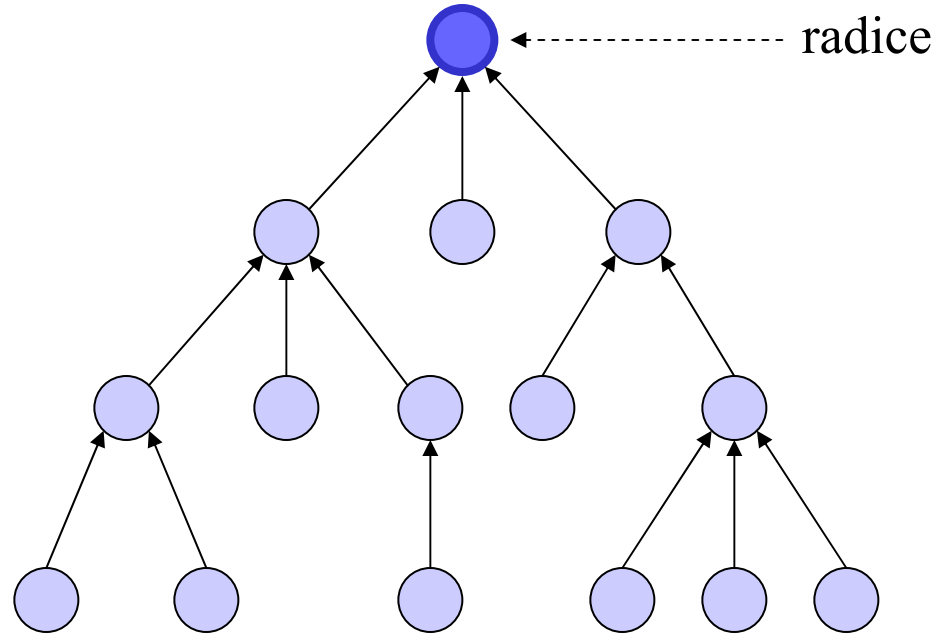
- Alberi radicati
 - definizione e uso
- Strutture di dati per rappresentare alberi
 - alberi binari, alberi di grado arbitrario
- Esercizi sugli alberi

Definizione di albero radicato (rooted tree)

- Un *albero radicato* è un insieme di nodi, su cui è definita una relazione binaria “ x è figlio di y ” (oppure “ y è genitore di x ”) tale che:
 1. ogni nodo ha un solo genitore, con l’eccezione della radice che non ha genitori
 2. c’è un cammino diretto da ogni nodo alla radice
 - l’albero, cioè, è connesso



Esempio di albero radicato

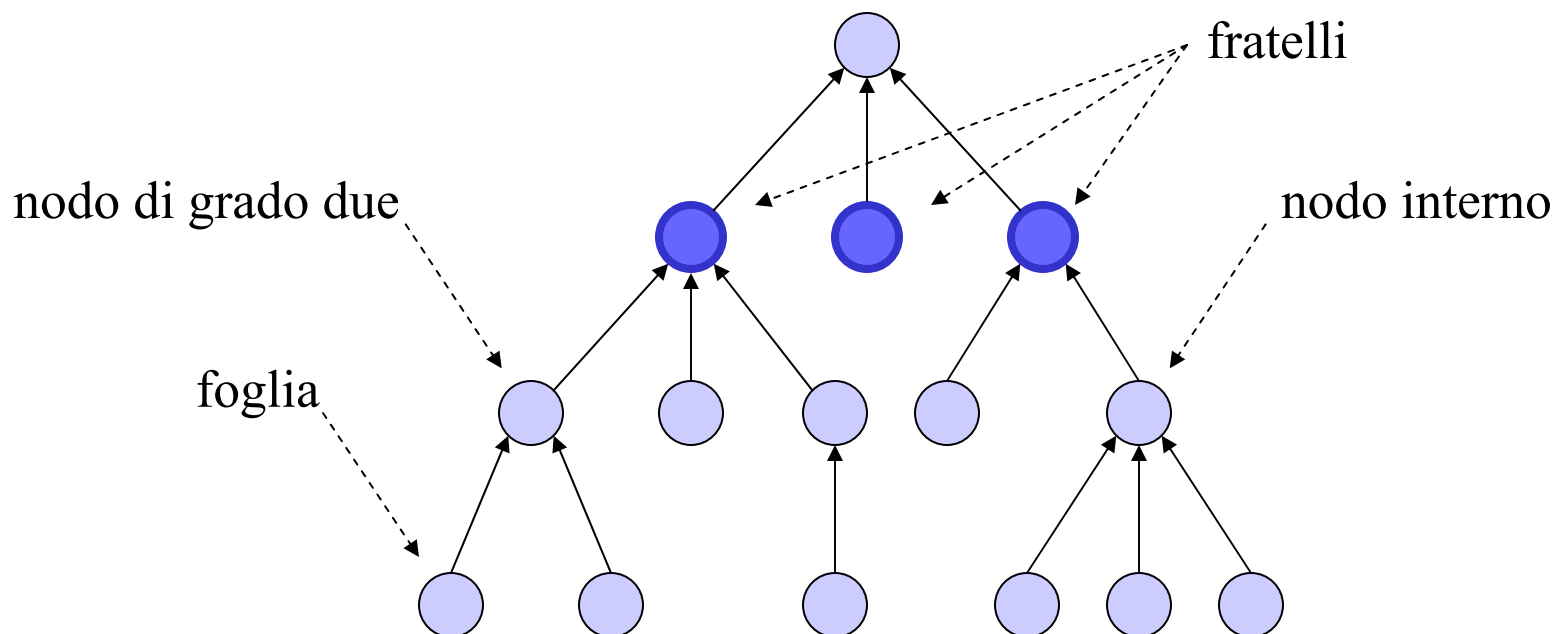


- Un albero può essere costruito a partire dalla radice aggiungendo ogni volta un nodo x come figlio di un nodo y già esistente
 - ciò giustifica il fatto che, se l'albero ha n nodi, allora ci sono $n-1$ relazioni genitore/figlio

Numerose applicazioni usano alberi

- I rapporti di ereditarietà determinano alberi
 - alberi genealogici o filogenetici
 - ereditarietà di classi nella programmazione ad oggetti
- I rapporti gerarchici sono alberi
 - gerarchie organizzative, di controllo, di responsabilità
- I rapporti di contenimento formano alberi
 - la classificazione scientifica degli organismi (tassonomie)
 - le directory del filesystem
 - i cammini minimi da una sorgente a tutti i nodi di una rete
- La struttura sintattica di una frase è un'albero
 - alberi sintattici
- ...

Alberi: definizioni

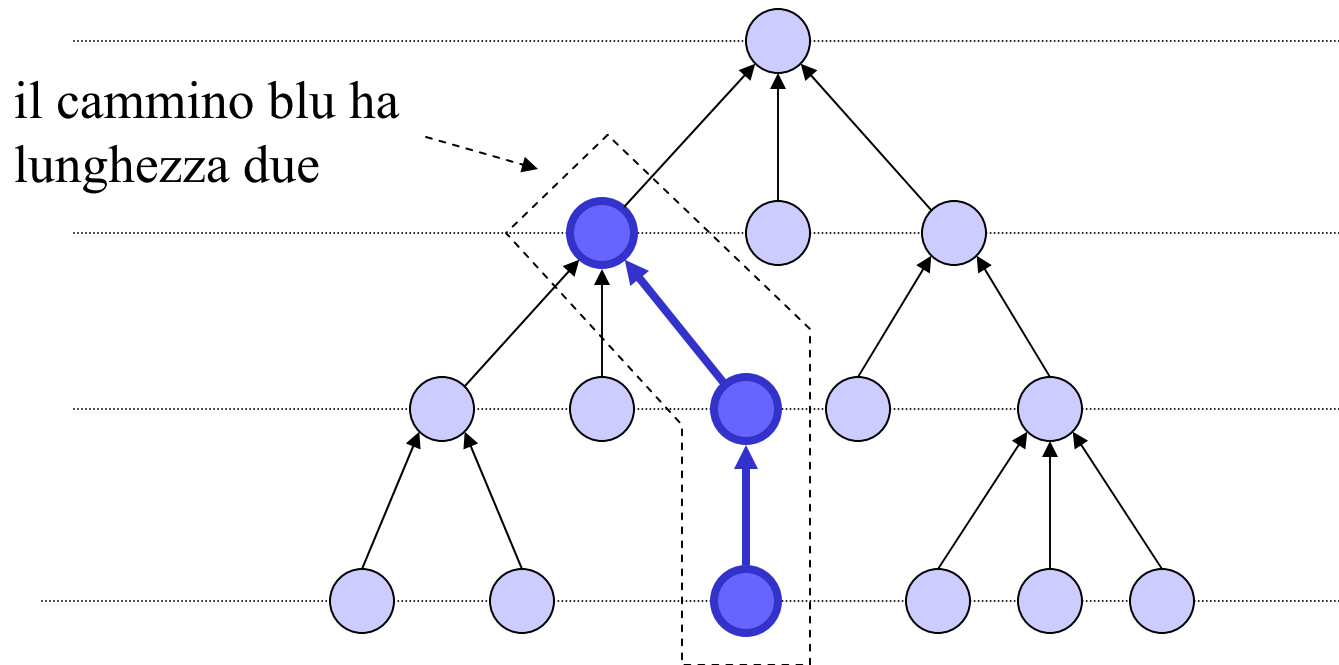


- Due nodi che hanno lo stesso genitore si dicono *fratelli*
- Il numero di figli di un nodo è il suo *grado*
- I nodi di grado zero sono *foglie*
- Un nodo non foglia è detto *nodo interno*

Tipi di alberi

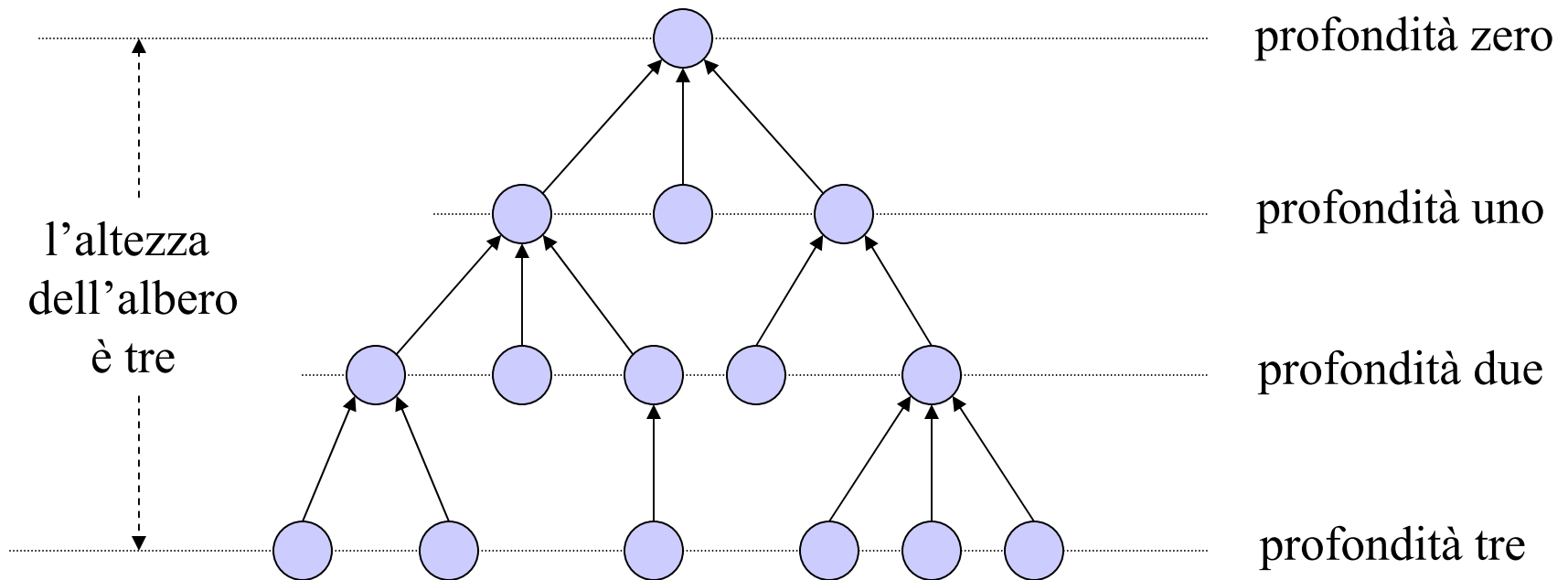
- Alberi binari
 - ogni nodo può avere solamente un figlio sinistro e un figlio destro
 - l'ordine dei figli è generalmente significativo
 - si distingue tra avere il solo figlio sinistro e avere il solo figlio destro
- Alberi di grado arbitrario
 - non è noto a priori il numero massimo dei figli di un nodo
 - l'ordine dei figli generalmente non è significativo

Alberi: definizioni



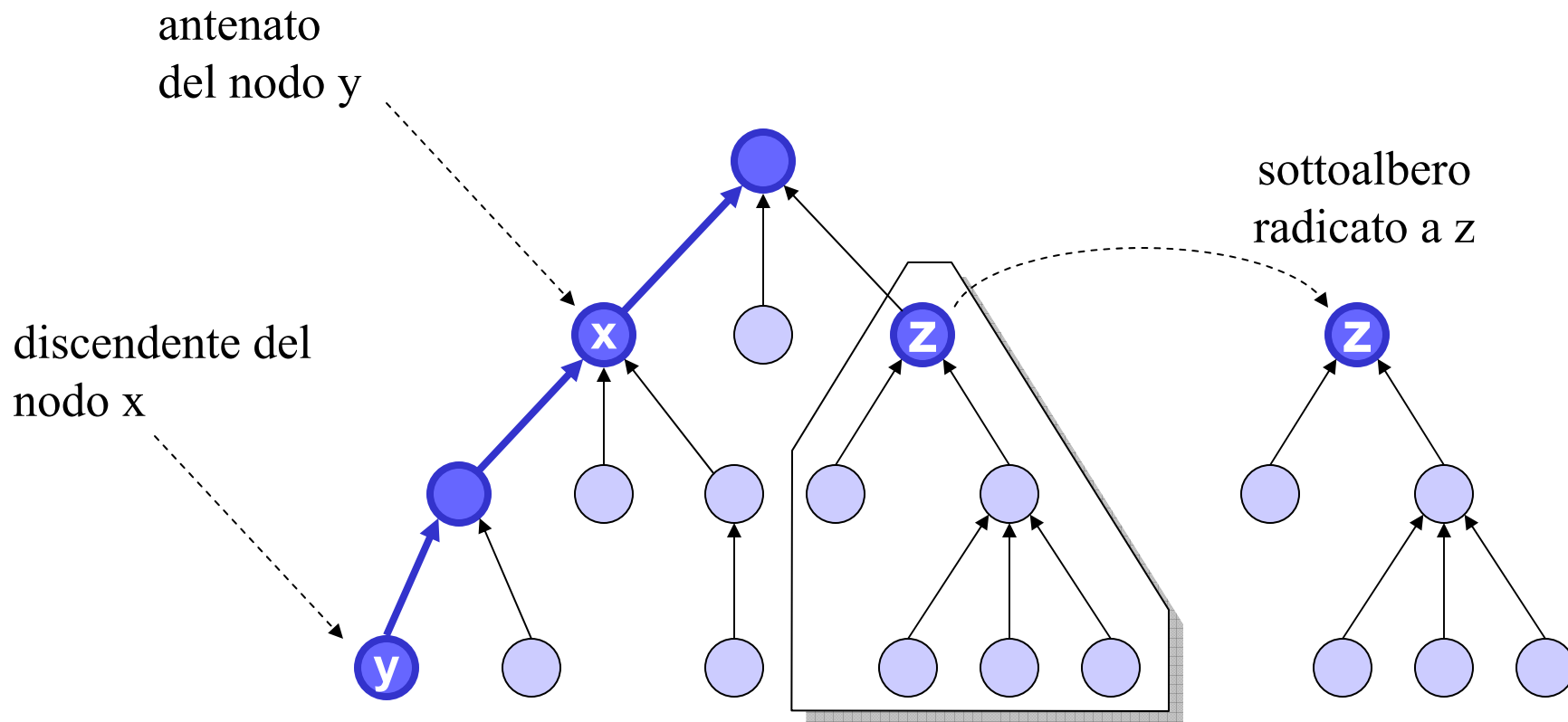
- Una sequenza di nodi tali che uno è il genitore del successivo è detta *cammino*
 - il cammino percorre gli archi alla rovescia rispetto alla figura qui sopra
- Il numero degli archi di un cammino è la sua *lunghezza*

Alberi: definizioni



- La **profondità** di un nodo è la lunghezza del cammino dal nodo alla radice
- La profondità del nodo più profondo è **l'altezza** dell'albero

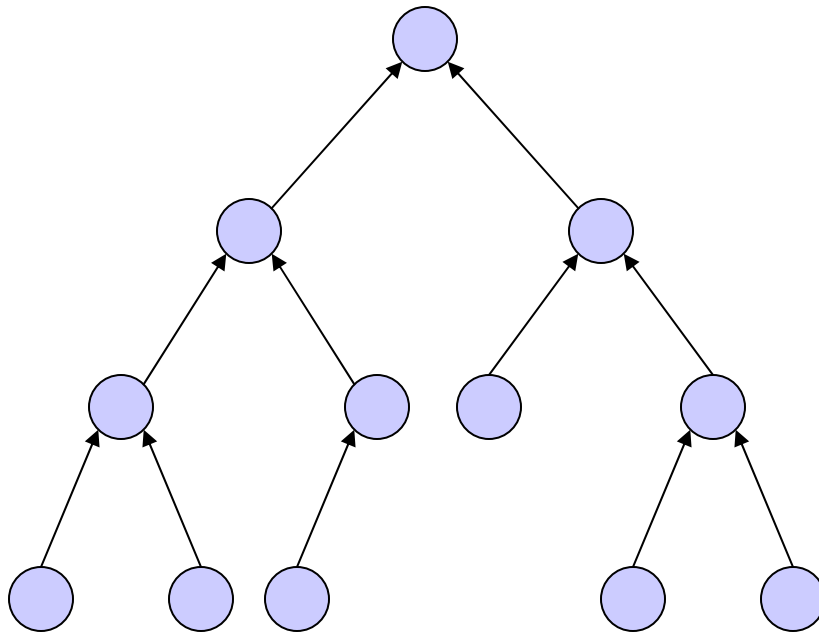
Alberi: definizioni



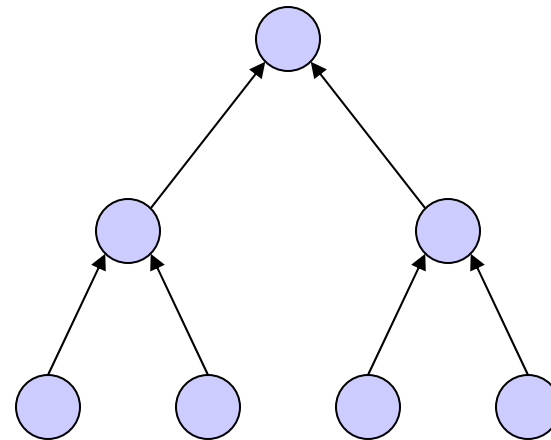
- Qualunque nodo x sul cammino (unico) dalla y alla radice è un *antenato* di y , mentre y è un *discendente* di x ;
- L'insieme costituito da un nodo z e da tutti i suoi discendenti è il *sottoalbero radicato a z*

Alberi: definizioni

albero binario

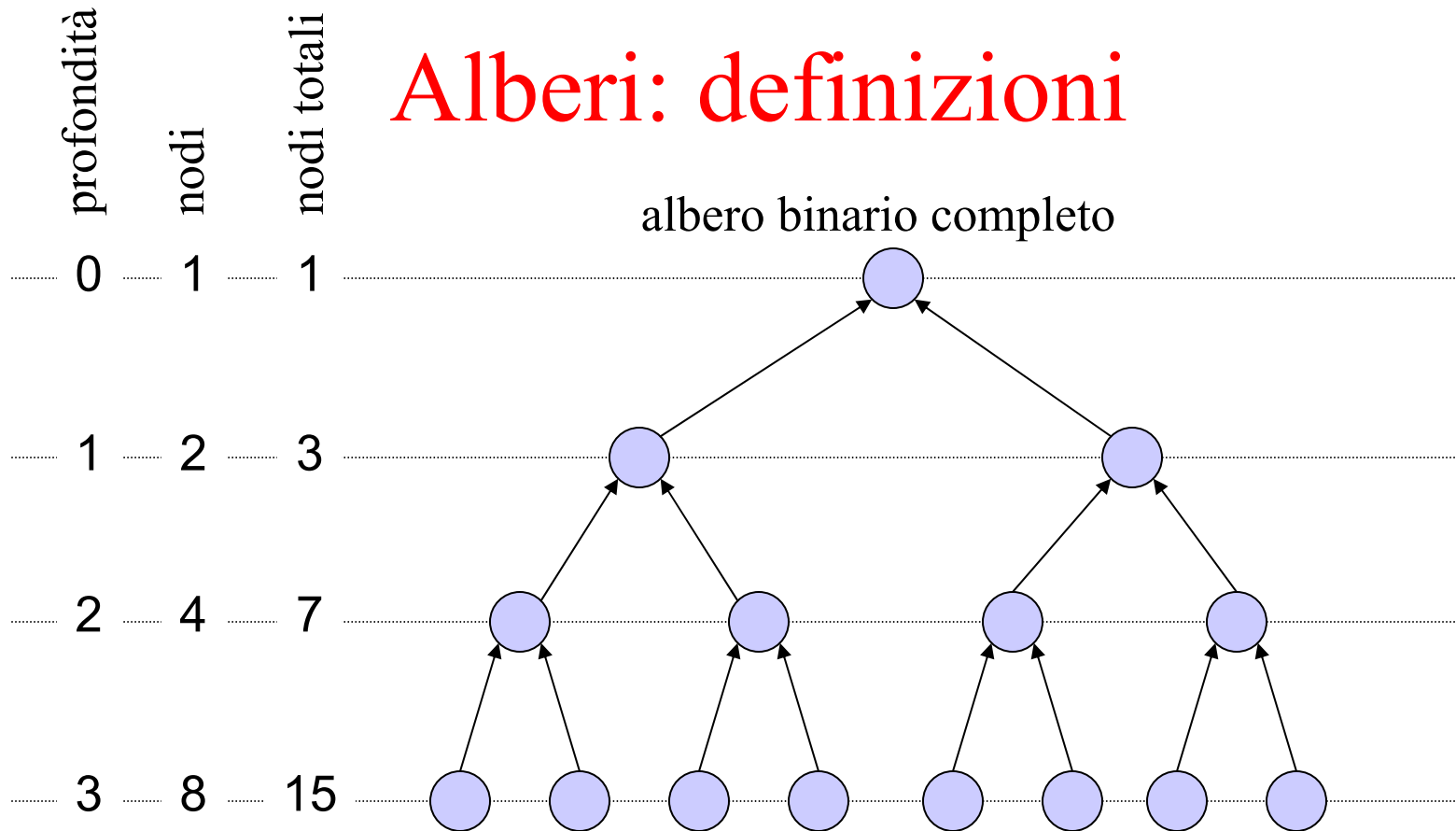


albero binario completo



- Un albero *ordinato* è un albero per il quale l'ordine dei figli di ogni nodo è significativo (non possono essere permutati)
- Un albero *binario* è un albero ordinato in cui i nodi hanno grado al più due
- Un albero binario è *completo* se ogni livello presenta tutti i nodi possibili

Alberi: definizioni



- Un albero binario completo di altezza h
 - ha 2^h foglie, dunque $h = \log_2(\text{numero foglie})$
 - ha $2^h - 1$ nodi interni
 - ha $2^{h+1} - 1$ nodi

Il tipo astratto albero

- Tipo astratto albero di interi
 - domini
 - il dominio di interesse è l'insieme degli alberi di interi
 - dominio di supporto: i riferimenti R che identificano le posizioni nell'albero
 - dominio di supporto: gli interi $Z = \{0, 1, -1, 2, -2, \dots\}$
 - dominio di supporto: i booleani $B = \{\text{true}, \text{false}\}$
 - costanti
 - l'albero vuoto

Operazioni del tipo astratto albero

- Operazioni sugli alberi di interi

- ritorna il riferimento alla radice:
- ritorna il riferimento al figlio sinistro:
- ritorna il riferimento al figlio destro:
- ritorna l'intero nel nodo specificato:
- verifica se un albero è vuoto:
- aggiunge un nodo come radice:
- aggiunge un nodo come figlio sinistro:
- aggiunge un nodo come figlio destro:
- elimina una foglia:
- cerca un nodo:
- svuota l'albero:
- conta i nodi dell'albero:
- ...

ROOT: $T \rightarrow R$

LEFT: $T \times R \rightarrow R$

RIGHT: $T \times R \rightarrow R$

INFO: $T \times R \rightarrow Z$

IS_EMPTY: $T \rightarrow B$

ADD_ROOT: $T \times Z \rightarrow T$

ADD_LEFT: $T \times R \times Z \rightarrow T$

ADD_RIGHT: $T \times R \times Z \rightarrow T$

DELETE_LEAF: $L \times R \rightarrow L$

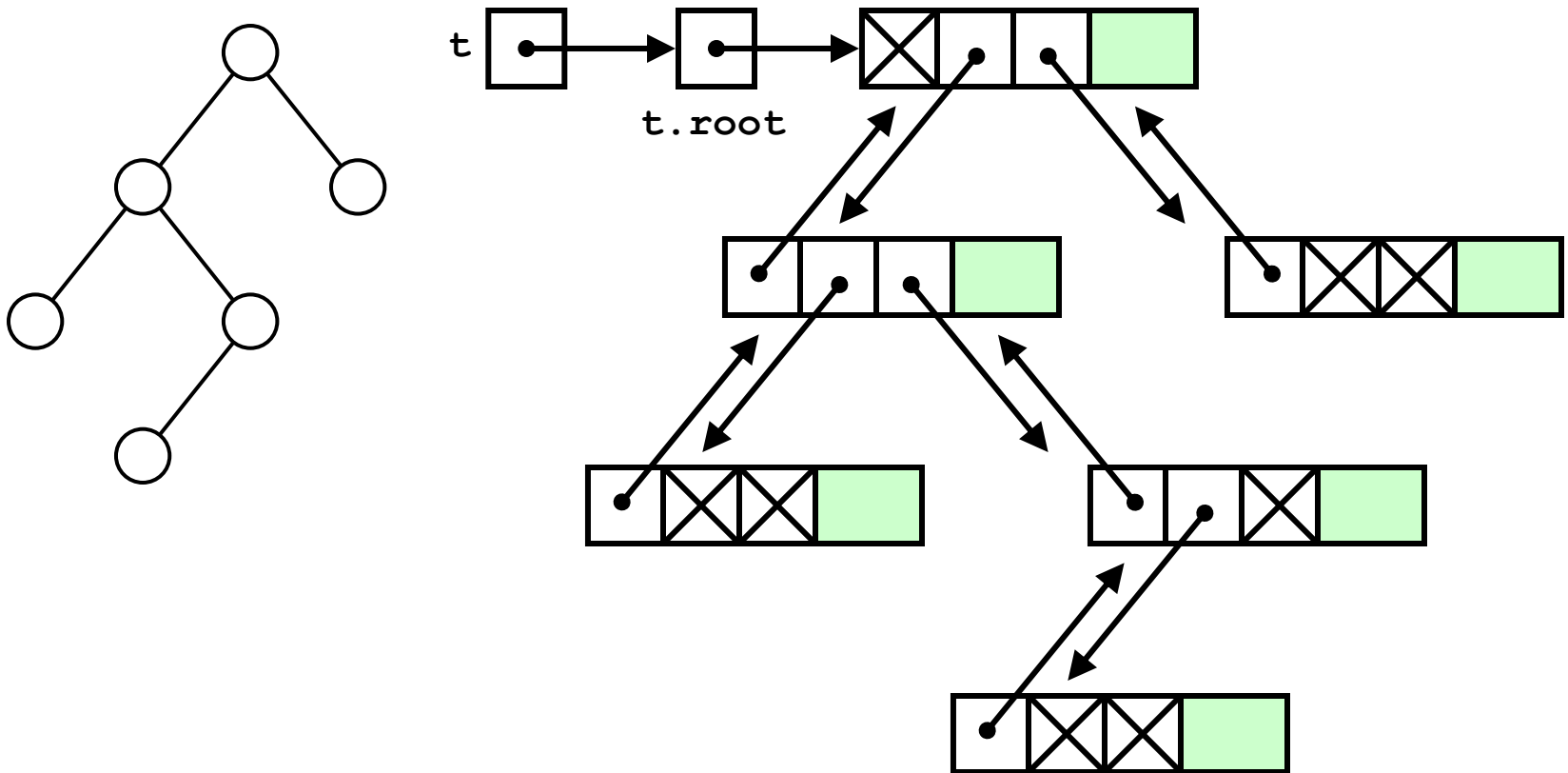
SEARCH: $T \times Z \rightarrow R$

EMPTY: $T \rightarrow T$

SIZE: $T \rightarrow Z$

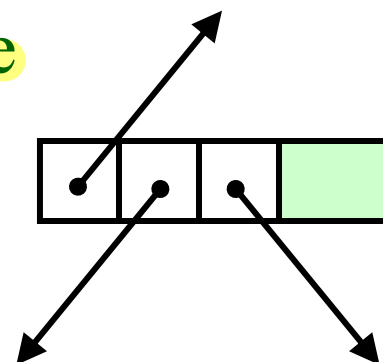
Rappresentazione di alberi binari

- Analogamente alle liste, gli alberi binari possono essere rappresentati mediante oggetti e riferimenti



Rappresentazione di alberi binari

- Un nodo dell'albero binario è un oggetto con i quattro campi
 - `parent`: riferimento al nodo genitore
 - `left`: riferimento al figlio sinistro
 - `right`: riferimento al figlio destro
 - `info`: dati satellite
- Un albero binario è un oggetto con un solo campo `root` che è un riferimento al nodo radice



Operazioni sugli alberi binari

- `NEW_TREE()`
 - restituisce una struttura rappresentante l'albero vuoto
 - questa funzione rappresenta la costante
- `IS_EMPTY(t)`
 - restituisce `TRUE` se l'albero è vuoto
- `ROOT(t)`
 - restituisce il riferimento alla radice dell'albero (`NULL` se `t` è vuoto)
- `LEFT(t,n)`
 - restituisce il riferimento (può essere `NULL`) al figlio sinistro del nodo n
- `RIGHT(t,n)`
 - restituisce il riferimento (può essere `NULL`) al figlio destro del nodo n
- `INFO(t,n)`
 - restituisce le informazioni (dati satellite) memorizzate nel nodo n
- ...

Esercizi sugli alberi binari

1. Scrivi lo pseudocodice delle funzioni

NEW_TREE()
IS_EMPTY(t)
ROOT(t)
LEFT(t, n)
RIGHT(t, n)
INFO(t, n)

descritte nella slide precedente

2. Scrivi lo pseudocodice della funzione TWO_CHILDREN(n) che ritorna TRUE se il nodo n ha due figli, FALSE altrimenti

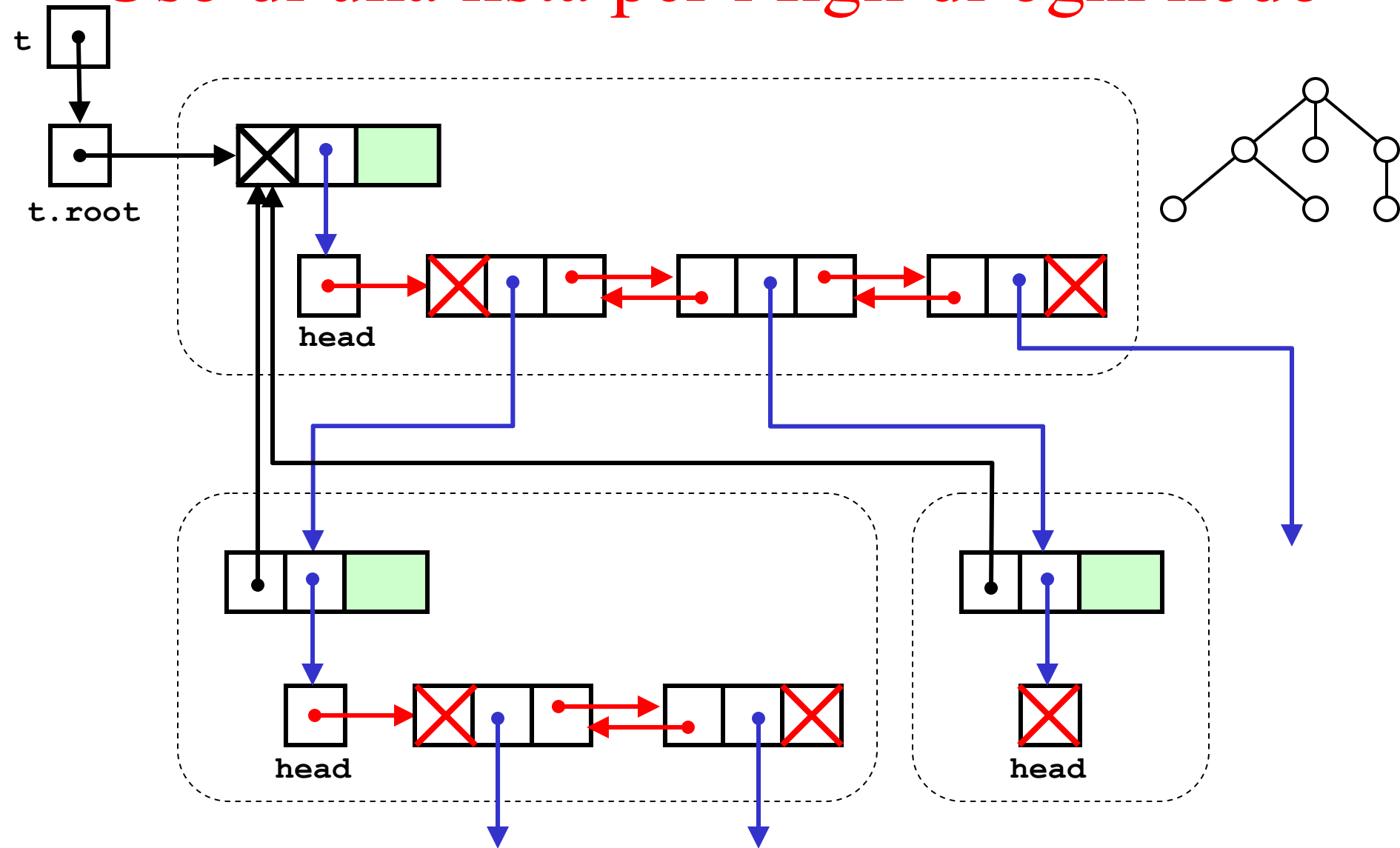
Esercizi sugli alberi binari

3. Scrivi lo pseudocodice della procedura `ADD_ROOT(t,z)` che aggiunga il nodo radice con valore z all'albero binario t
 - assumi che t sia vuoto
4. Scrivi lo pseudocodice delle procedure `ADD_LEFT(t,n,z)` e `ADD_RIGHT(t,n,z)` che aggiungono il figlio sinistro e destro al nodo n , contenente il valore z
5. Scrivi lo pseudocodice della funzione `ONLY_LEFT(t)` che restituisce `TRUE` se tutti i nodi dell'albero binario t hanno solamente il figlio sinistro (o nessun figlio), `FALSE` altrimenti
 - se l'albero è vuoto restituisci `TRUE`

Rappresentazione di alberi di grado arbitrario

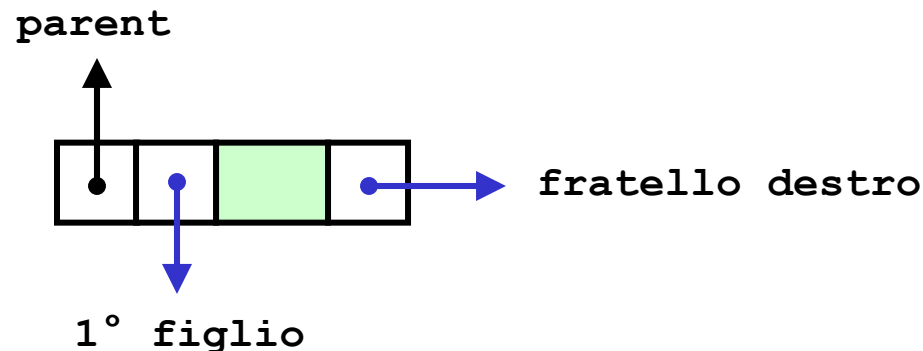
- Per rappresentare alberi di grado arbitrario si possono utilizzare diverse strategie
 - uso di una lista per i figli di ogni nodo
 - poco usato perché molto prolisso
 - uso di una struttura detta “figlio-sinistro-fratello-destro”
 - più sintetico

Uso di una lista per i figli di ogni nodo

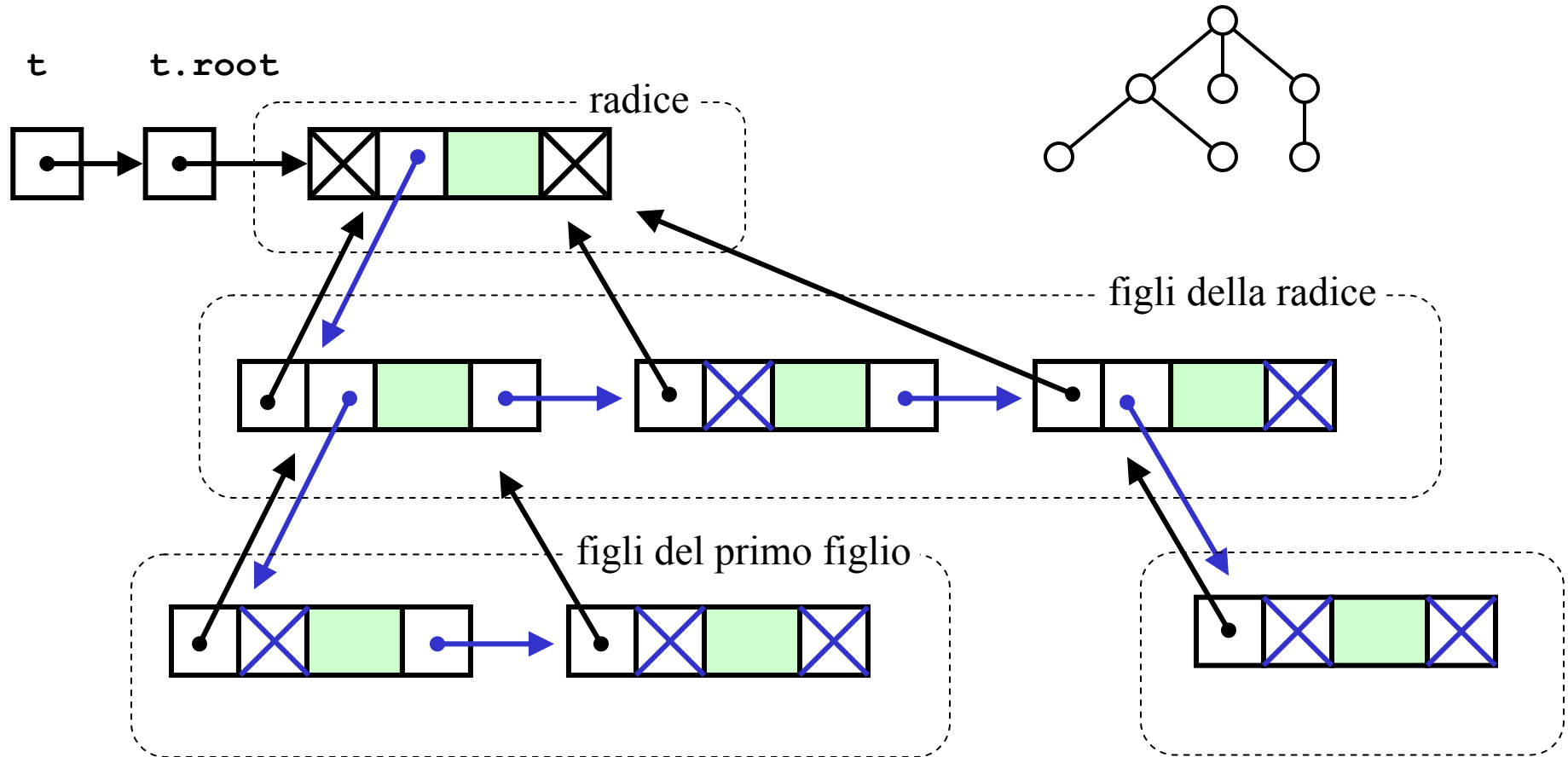


Struttura “figlio-sinistro-fratello-destro”

- I nodi hanno gli usuali campi `parent`, `left`, `right` e `info`
 - i campi `parent` e `info` hanno il significato usuale
 - il campo `left` è un riferimento al figlio di sinistra (cioè al primo figlio)
 - il campo `right`, invece di essere un riferimento al figlio destro, è un riferimento al prossimo fratello



Figlio-sinistro-fratello-destro



Operazioni sugli alberi qualsiasi

- `NEW_TREE()`
 - restituisce una struttura rappresentante l'albero vuoto
- `IS_EMPTY(t)`
 - restituisce `TRUE` se l'albero è vuoto
- `ROOT(t)`
 - restituisce il riferimento alla radice dell'albero (`NULL` se `t` è vuoto)
- `FIRST_CHILD(t,n)`
 - restituisce il riferimento (può essere `NULL`) al figlio sinistro del nodo `n`
- `NEXT_SIBLING(t,n)`
 - restituisce il riferimento (può essere `NULL`) al fratello destro del nodo `n`
- `INFO(t,n)`
 - restituisce l'intero memorizzato nel nodo `n`
- ...

Esercizi sugli alberi qualsiasi

6. Scrivi lo pseudocodice della procedura $\text{ADD_ROOT}(t, z)$ che aggiunga un nodo radice con valore z all'albero t
 - supponi che l'albero t sia vuoto
7. Scrivi lo pseudocodice della procedura $\text{ADD_SIBLING}(t, n, z)$ che aggiunge al nodo n un figlio che contiene il valore z

Algoritmi e Strutture di Dati

Code di priorità
(Heap e HEAP_SORT)

m.patrignani

Nota di copyright

- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Sommario

- Il tipo astratto di dato coda di priorità
- La struttura di dati heap
 - procedura `MAX_HEAPIFY`
 - procedura `BUILD_MAX_HEAP`
- Coda di priorità realizzata con un heap
- Algoritmo di ordinamento `HEAP_SORT`
 - analisi della sua complessità

Code di priorità

- Una coda di priorità (priority queue) è una collezione di elementi
 - ad ogni elemento è associato un valore di priorità
 - i valori di priorità definiscono un ordinamento
- Operazioni sulle code di priorità
 - l'utente vuole inserire efficientemente nuovi elementi con valori arbitrari di priorità
 - l'utente vuole estrarre efficientemente l'elemento a più alta priorità

Due applicazioni delle code di priorità

- Allocazione ai processi delle risorse condivise
 - gli elementi della coda sono le richieste da parte dei processi di una specifica risorsa
 - per esempio l'accesso all'hard disk o ad una periferica
 - i processi in esecuzione generano nuove richieste con priorità dipendenti dall'utente o dal tipo di operazione richiesta
 - la risorsa è assegnata al processo con più alta priorità
- Simulazione di un sistema complesso guidata dagli eventi
 - gli elementi della coda sono eventi, con associato il tempo in cui si devono verificare
 - gli eventi vengono simulati in ordine temporale
 - la simulazione di un evento può provocare l'inserimento nella coda di altri eventi a distanza di tempo

Coda di priorità di interi

- Domini

- il dominio di interesse Q di tutte le code di priorità di interi
- dominio di supporto: l'insieme degli interi Z
- dominio di supporto: l'insieme dei booleani $\{true, false\}$

- Costanti

- la coda di priorità vuota
 - `NEW_QUEUE()`: inizializza e ritorna una coda di priorità vuota

- Operazioni

`INSERT(Q,x)`: inserisce l'elemento x nella coda Q

`MAXIMUM(Q)`: restituisce l'elemento di Q con chiave più grande

`EXTRACT_MAX(Q)`: restituisce l'elemento di Q con chiave più grande e lo rimuove da Q

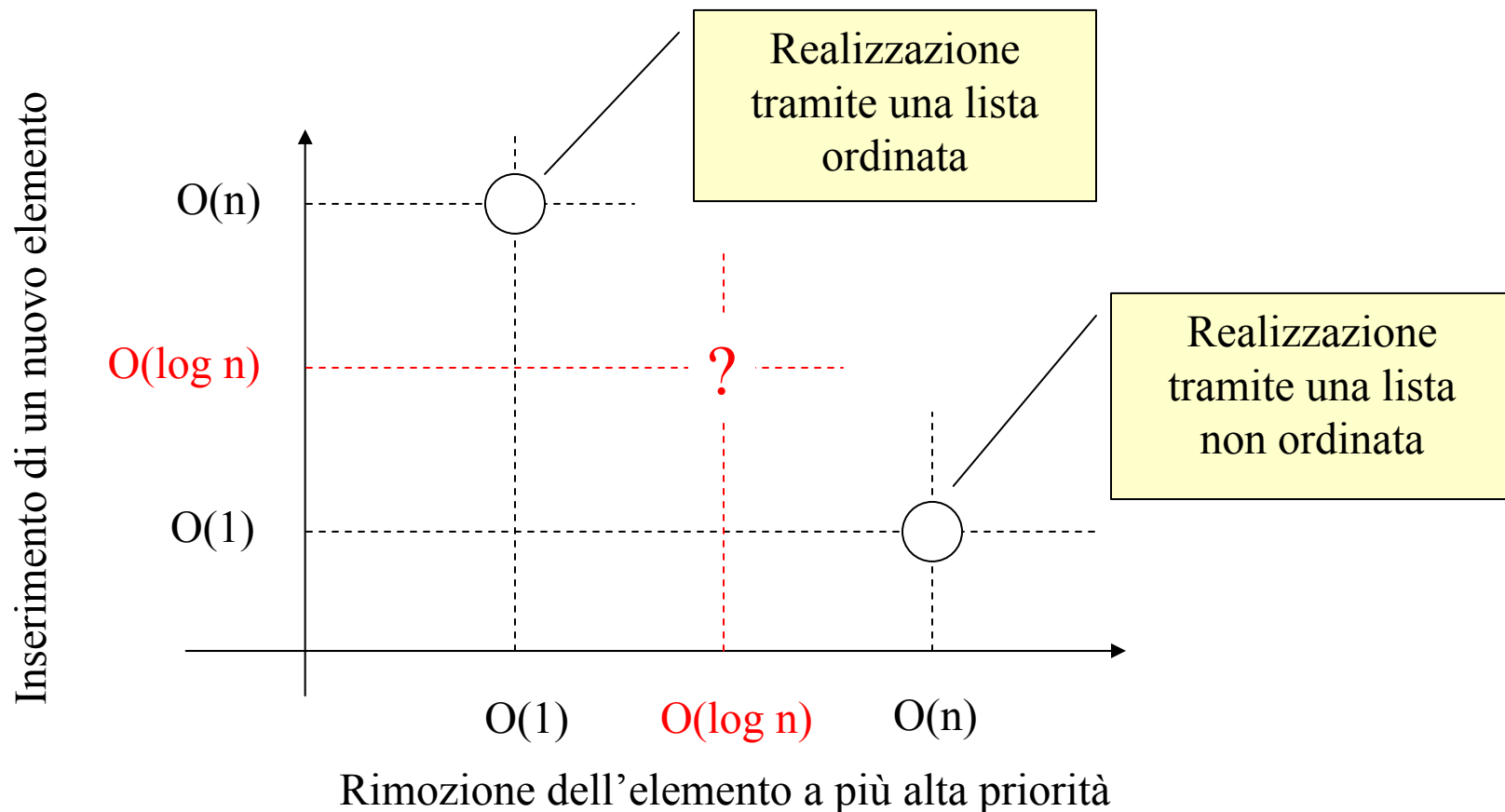
`IS_EMPTY(Q)`: riporta `true` se la coda Q è vuota, `false` altrimenti

Realizzazioni inefficienti di code di priorità

- Si potrebbe realizzare una coda di priorità tramite una lista ordinata
 - l'inserimento nella lista di un nuovo elemento avrebbe complessità $\Theta(n)$
 - la rimozione dell'elemento a più alta priorità (il primo della lista) avrebbe complessità $\Theta(1)$
- Si potrebbe realizzare una coda di priorità tramite una lista non ordinata
 - l'inserimento (in testa) di un nuovo elemento avrebbe complessità $\Theta(1)$
 - la ricerca e la rimozione dell'elemento a più alta priorità avrebbe complessità $\Theta(n)$

Sono possibili realizzazioni più efficienti?

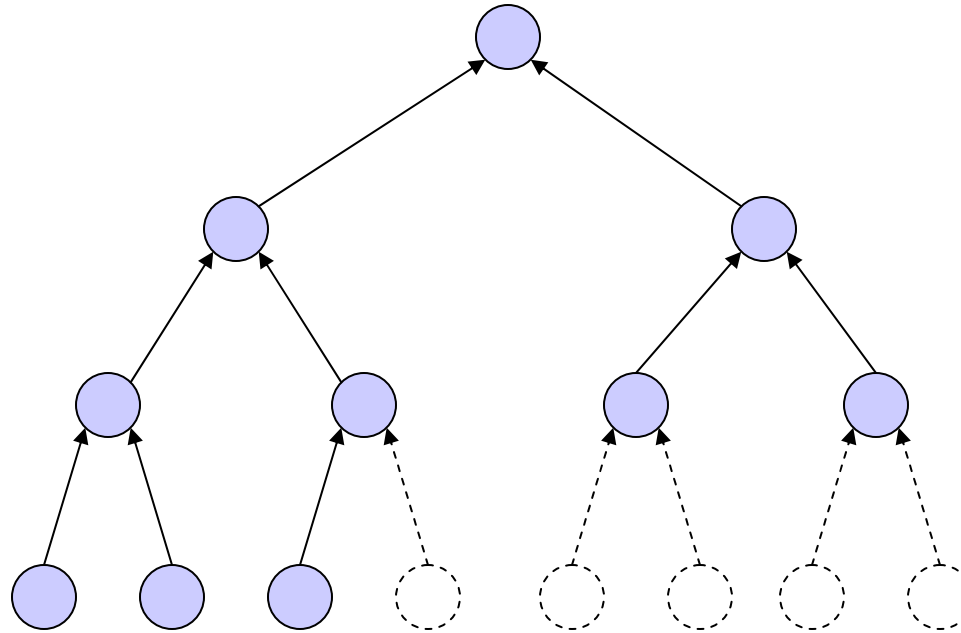
- L'obiettivo è quello di bilanciare i due costi



La struttura dati heap

- Un heap
 - è una struttura dati che può essere utilizzata per realizzare una coda di priorità
 - è uno speciale array i cui valori sono in rapporto con la loro posizione nell'array
 - può essere un max-heap o un min-heap
 - noi vedremo in dettaglio il max-heap

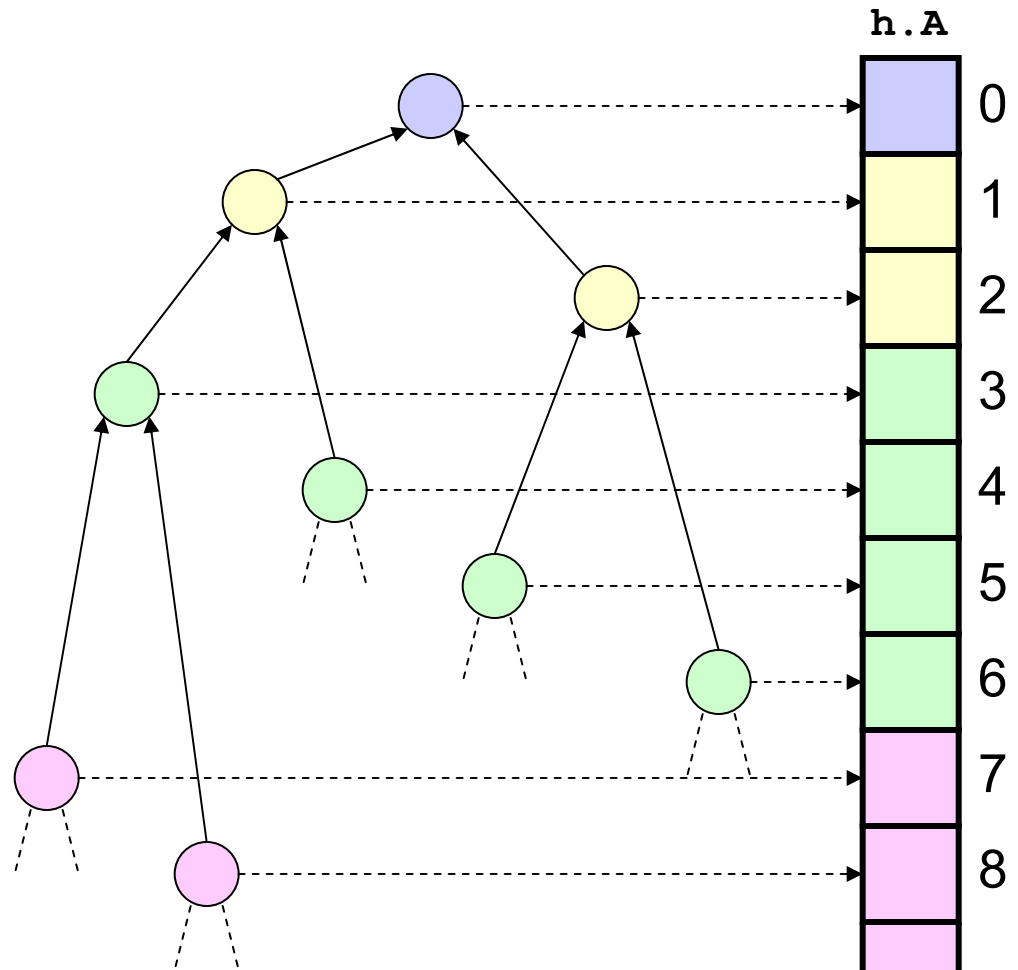
Alberi binari “quasi completi”



- Gli heap rappresentano alberi binari quasi completi
- Un albero binario è *quasi completo* se l'ultimo livello può essere incompleto nella sua parte destra

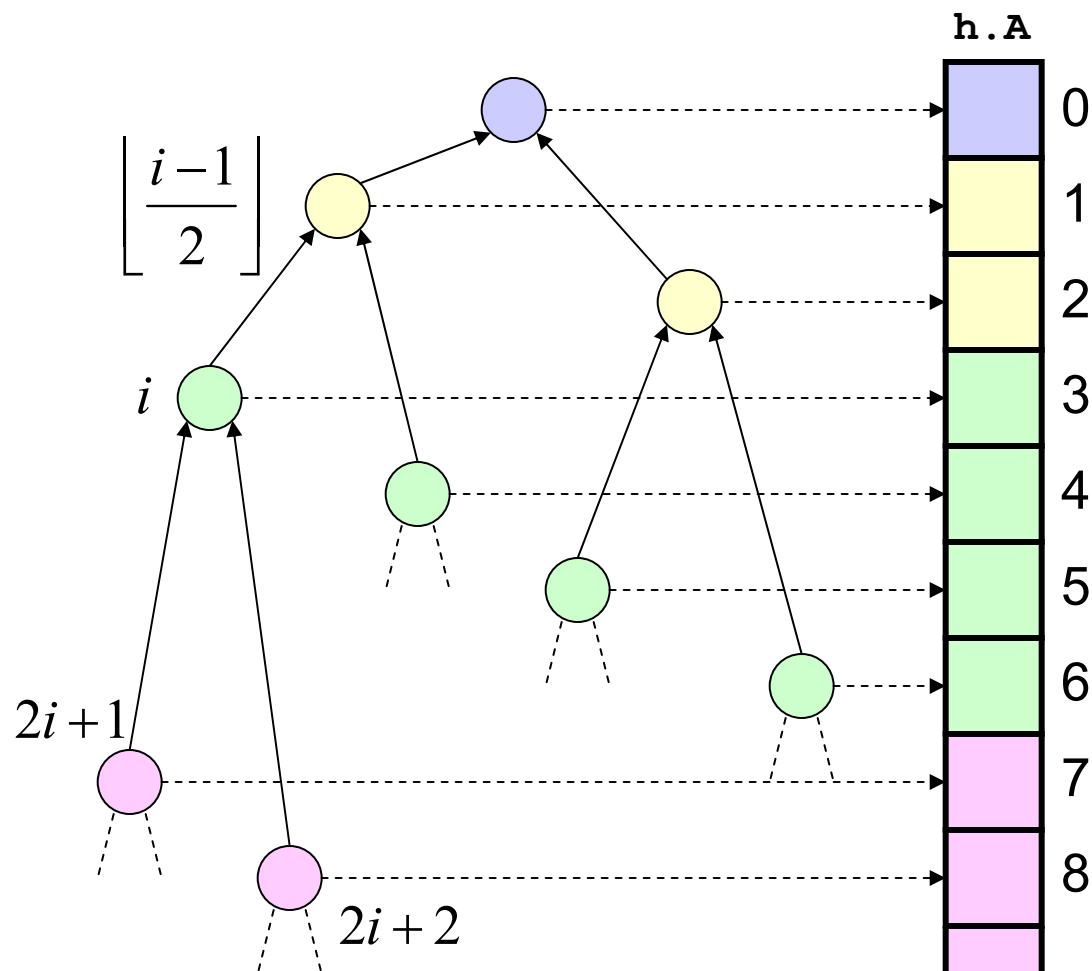
Un heap codifica un albero

- L'heap consiste di un array $h.A$ che codifica, livello per livello, un albero binario quasi completo



Un heap codifica un albero

- $h.A[0]$ è la radice dell'albero
- Dato il nodo associato alla posizione i :
 - i nodi figli si trovano in posizione $2i+1$ e $2i+2$
 - il nodo genitore (se $i \neq 0$) si trova in posizione $\lfloor (i-1)/2 \rfloor$



Semplificazione dello pseudocodice

- Per rendere più leggibile lo pseudocodice definiamo le seguenti funzioni

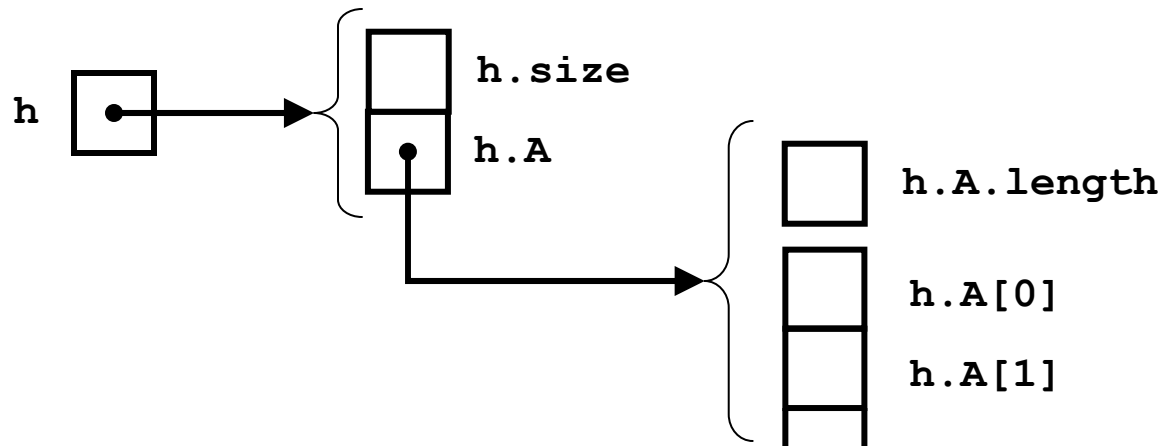
```
PARENT(i)  /* ritorna l'indice del parent del nodo i */  
1. return  $\lfloor (i-1)/2 \rfloor$ 
```

```
LEFT(i)    /* ritorna l'indice del figlio sinistro di i */  
1. return  $2i + 1$ 
```

```
RIGHT(i)   /* ritorna l'indice del figlio destro di i */  
1. return  $2i + 2$ 
```

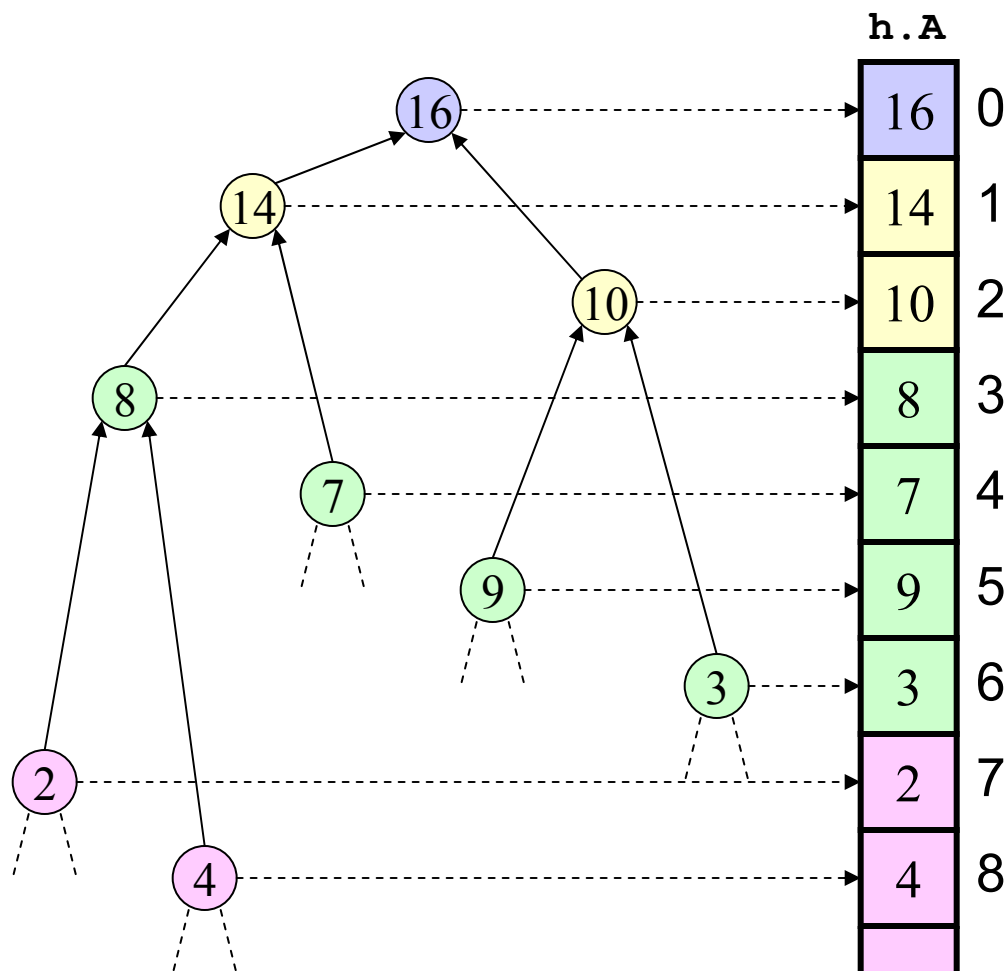
Dettagli implementativi

- Per maggiore flessibilità, anche se l'array è lungo `h.A.length`, supponiamo che solo i valori compresi tra 0 e `h.size-1` siano significativi
 - dove ovviamente $h.size \leq h.A.length$



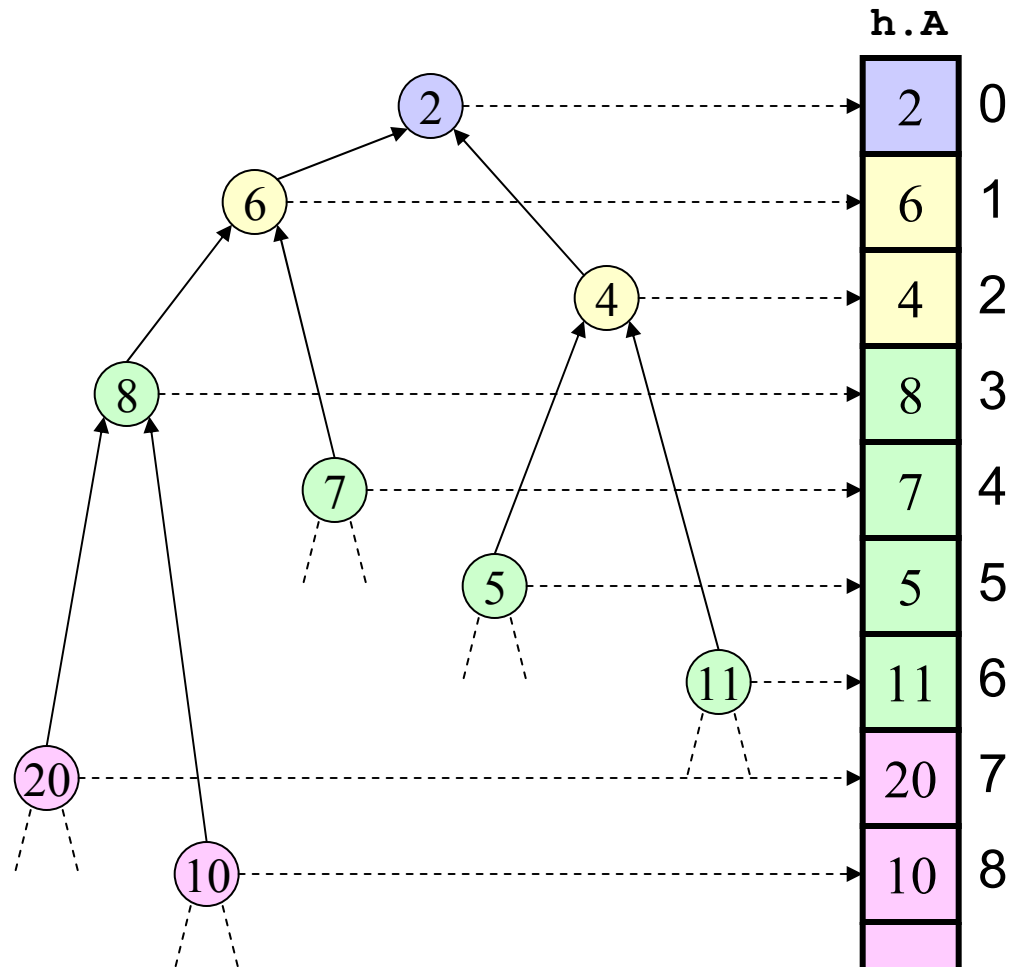
Valori contenuti in un max-heap

- In un *max-heap* l'elemento memorizzato nel nodo i ha valore maggiore o uguale degli elementi memorizzati nei suoi figli
 - la radice contiene il valore più alto dell'array
 - per $j > 0$, $h.A[\text{PARENT}(j)] \geq h.A[j]$



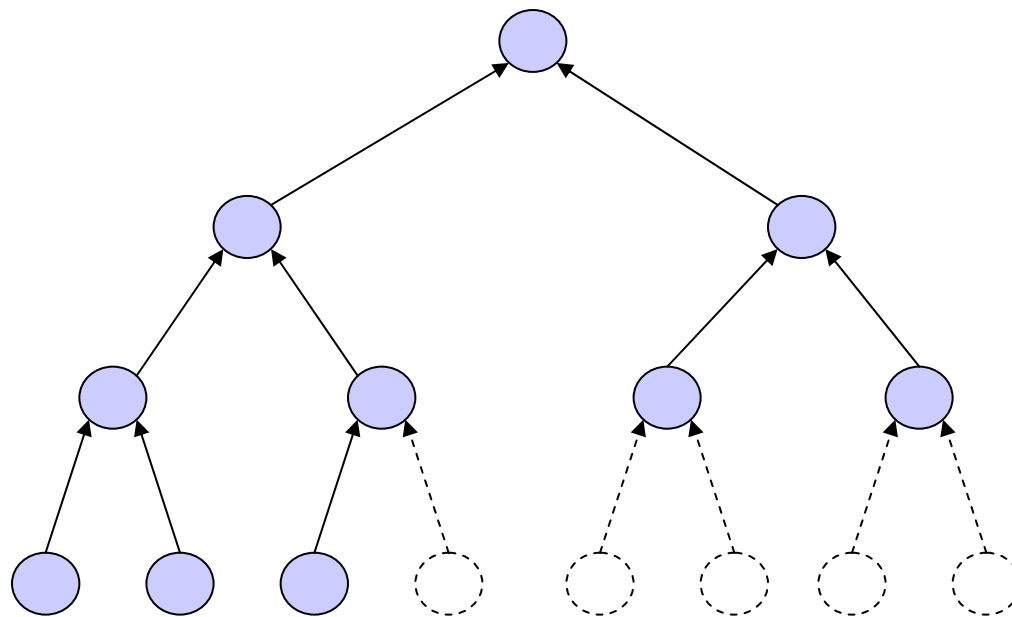
Min-heap

- Esiste anche il *min-heap* che ha la proprietà *simmetrica*
 - la radice contiene il valore minore dell'array



Proprietà degli heap

- Se h è un heap che codifica un albero quasi-completo con n elementi, gli $\lfloor n/2 \rfloor$ elementi da 0 a $\lfloor n/2 \rfloor - 1$ sono nodi interni

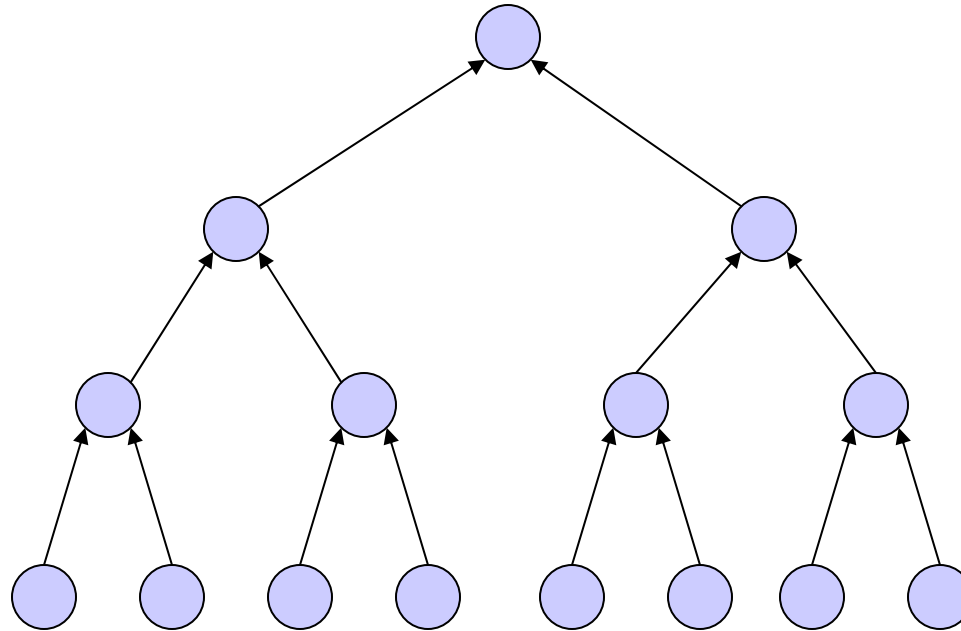


Dimostrazione della proprietà

Dimostriamo per induzione che i nodi interni sono $\lfloor n/2 \rfloor$

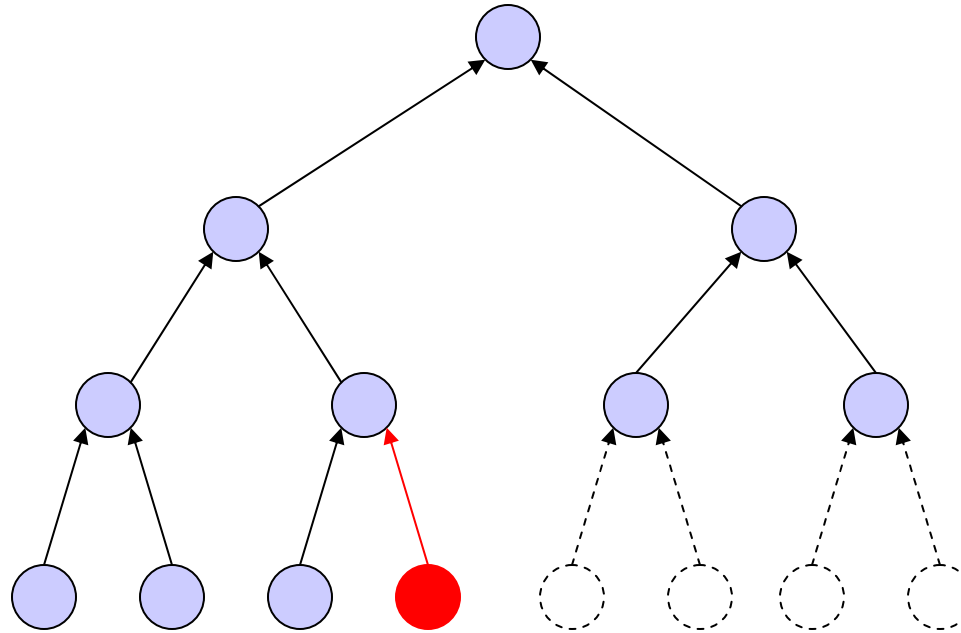
- Passo base
 - dimostriamo l'asserto per gli alberi completi
 - un albero completo è un particolare albero quasi completo
- Passo induttivo
 - dimostriamo che se vale per un albero quasi completo con n nodi, vale anche per un albero quasi completo con $n-1$ nodi
 - distinguiamo due casi: rimozione del figlio destro e rimozione del figlio sinistro

Passo base: albero binario completo



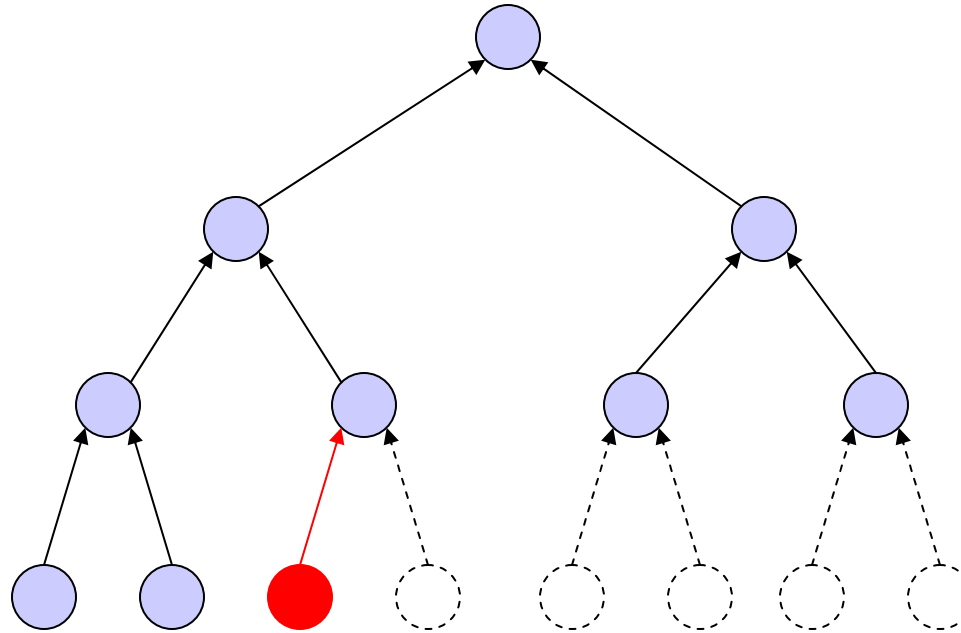
- Sappiamo che un albero binario completo di altezza h ha 2^h foglie e $2^h - 1$ nodi interni e dunque $n = 2^{h+1} - 1$ nodi totali
- Verifichiamo la formula:
$$\text{nodi interni} = \lfloor n/2 \rfloor = \lfloor (2^{h+1} - 1)/2 \rfloor = \lfloor 2^h - 1/2 \rfloor = 2^h - 1$$

Passo induttivo: rimuovo un figlio destro



- Prima della rimozione avevo n nodi e $\lfloor n/2 \rfloor$ nodi interni (con n dispari)
 - dalla disparità di n segue che $\lfloor n/2 \rfloor = \lfloor (n-1)/2 \rfloor$
- Dopo la rimozione ho $n' = n-1$ nodi e il numero dei nodi interni non è cambiato
 - ne segue che i nodi interni sono $\lfloor n/2 \rfloor = \lfloor (n-1)/2 \rfloor = \lfloor n'/2 \rfloor$

Passo induttivo: rimuovo un figlio sinistro



- Prima della rimozione avevo n nodi e $\lfloor n/2 \rfloor$ nodi interni (con n pari)
 - dalla parità di n segue che $\lfloor n/2 \rfloor = \lfloor (n-1)/2 \rfloor + 1$
- Dopo la rimozione ho $n' = n-1$ nodi e i nodi interni sono diminuiti di uno
 - dunque i nodi interni sono $\lfloor (n-1)/2 \rfloor = \lfloor n'/2 \rfloor$

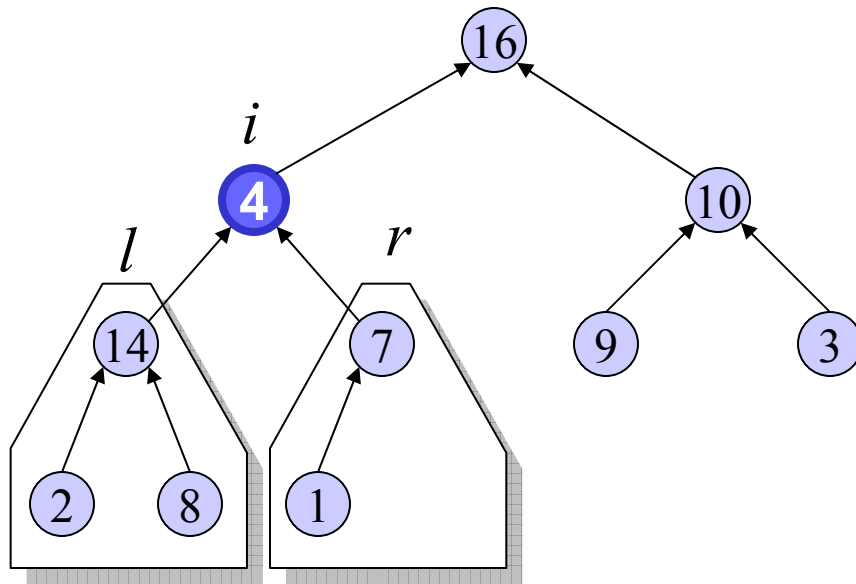
Procedura **MAX_HEAPIFY**

- Se i due sottonodi radicati a **LEFT(*i*)** e a **RIGHT(*i*)** sono dei max-heap, allora la procedura **MAX-HEAPIFY(*h*,*i*)** trasforma il sottonodo radicato ad *i* in un max-heap

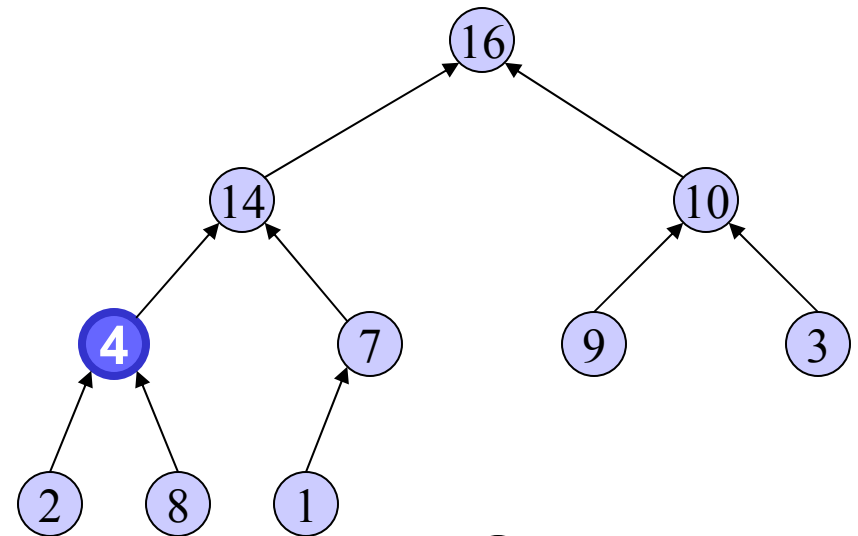
MAX_HEAPIFY(*h*, *i*)

```
1.  l = LEFT(i)           ▷ indice del figlio sinistro
2.  r = RIGHT(i)          ▷ indice del figlio destro
3.  if (l ≤ h.size-1 and h.A[l] > h.A[i])    massimo = l
4.  else                                                         massimo = i
5.  if (r ≤ h.size-1 and h.A[r] > h.A[massimo]) massimo = r
6.  /* ora massimo è il massimo tra h.A[l], h.A[r] ed h.A[i]
7.  if massimo ≠ i
8.      SCAMBIA_CASELLE(h.A, i, massimo)
9.      MAX_HEAPIFY(h, massimo)
```

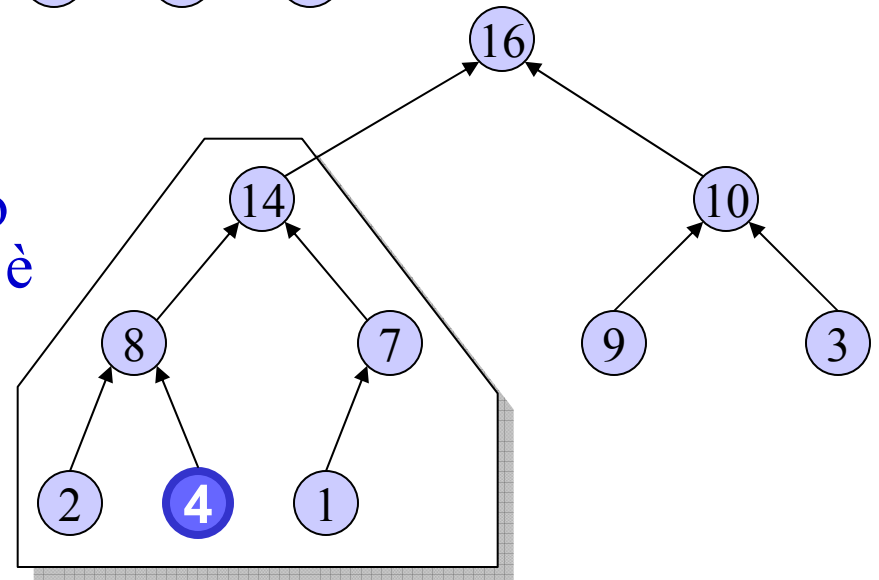
Esecuzione di MAX_HEAPIFY sul nodo i



i sottoalberi
radicati ad l
ed r sono
max-heap



il sottoalbero
radicato ad i è
diventato un
max-heap



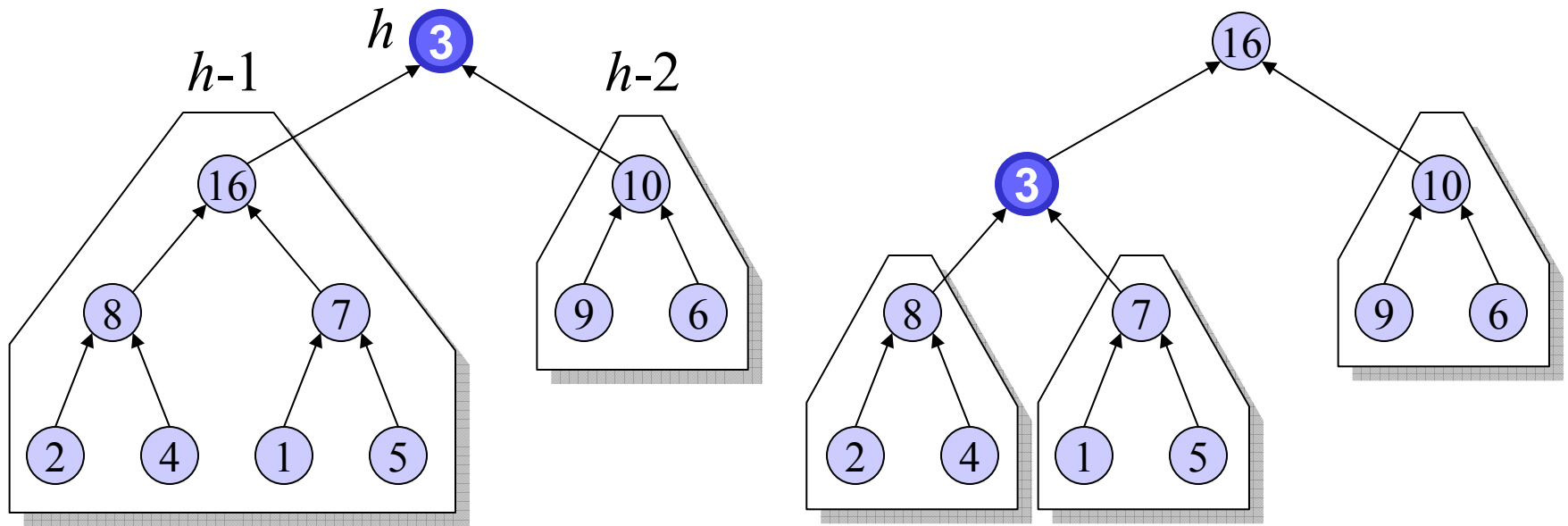
Analisi di MAX_HEAPIFY

MAX_HEAPIFY(h, i)

```
1. l = LEFT(i)           ▷ indice del figlio sinistro
2. r = RIGHT(i)          ▷ indice del figlio destro
3. if (l ≤ h.size-1 and h.A[l] > h.A[i])    massimo = l
4. else                                                            massimo = i
5. if (r ≤ h.size-1 and h.A[r] > h.A[massimo]) massimo = r
6. /* ora massimo è il massimo tra h.A[l], h.A[r] ed h.A[i]
7. if massimo ≠ i
8.     SCAMBIA_CASELLE(h.A, i, massimo)
9.     MAX_HEAPIFY(h, massimo)
```

- Il tempo di esecuzione di MAX_HEAPIFY(h, i) si ottiene sommando
 - il tempo di calcolo di massimo (linee 1-8), che è evidentemente $\Theta(1)$
 - il tempo di calcolo MAX_HEAPIFY(h, massimo) dove il sottoalbero radicato a massimo ha dimensione ridotta rispetto a quello radicato ad i

Analisi di MAX_HEAPIFY



- Il caso peggiore si presenta quando occorre ricorrere su un sottoalbero di profondità $h-1$, mentre il sottoalbero radicato al nodo fratello ha profondità $h-2$
 - ricorda che l'albero è quasi-completo
- In questo caso, se i nodi dell'albero sono n , i nodi del sottoalbero più pesante sono $n \cdot 2/3$

Analisi di MAX_HEAPIFY

- Il tempo di calcolo di MAX_HEAPIFY su un sottoalbero con n nodi è

$$T(n) \leq T(2n/3) + c$$

- Questa disequazione di ricorrenza può essere risolta con il master theorem

$$T(n) = a \cdot T(n/b) + p(n^k)$$

nello speciale caso in cui

$$a=1 \qquad b=3/2 \qquad k=0$$

che per $a = b^k$ si risolve in

$$T(n) = \Theta(n^k \log n) = \Theta(\log n)$$

- Dunque la complessità di MAX_HEAPIFY è $\Theta(\log n)$

Analisi alternativa di MAX_HEAPIFY

- Il tempo di calcolo di MAX_HEAPIFY su un sottoalbero con n nodi è chiaramente pari a $\Theta(1)$ moltiplicato per il numero di lanci ricorsivi di MAX_HEAPIFY
 - $\Theta(1)$ è dovuto alle linee 1-8 dello pseudocodice
- Poiché un albero binario quasi-completo ha altezza $\Theta(\log n)$, il numero di lanci ricorsivi nel caso peggiore è $\Theta(\log n)$
- La complessità di MAX_HEAPIFY nel caso peggiore è dunque:

$$\Theta(1) \cdot \Theta(\log n) = \Theta(\log n)$$

Procedura BUILD_MAX_HEAP

- BUILD_MAX_HEAP trasforma un array A in un heap
- Se $n = h.A.length$, gli elementi con indice $\geq \lfloor n/2 \rfloor$ sono tutte foglie
 - ognuna è un heap con un solo elemento
- BUILD_MAX_HEAP esegue MAX_HEAPIFY sui nodi che non sono foglie, dal basso verso l'alto

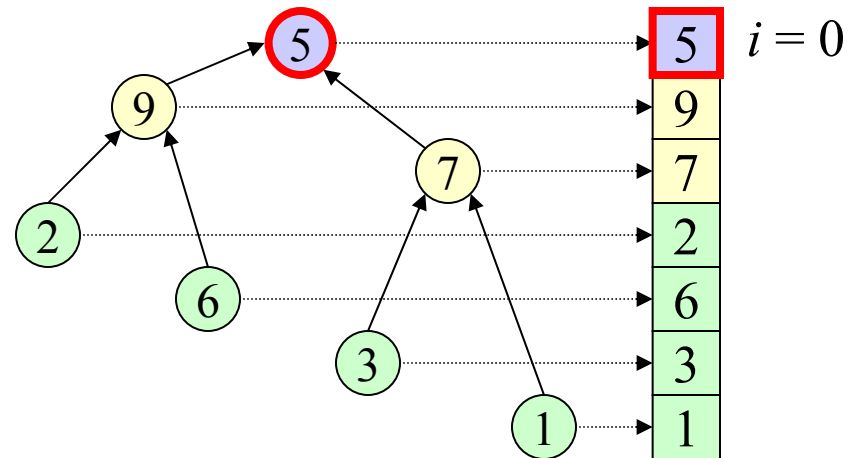
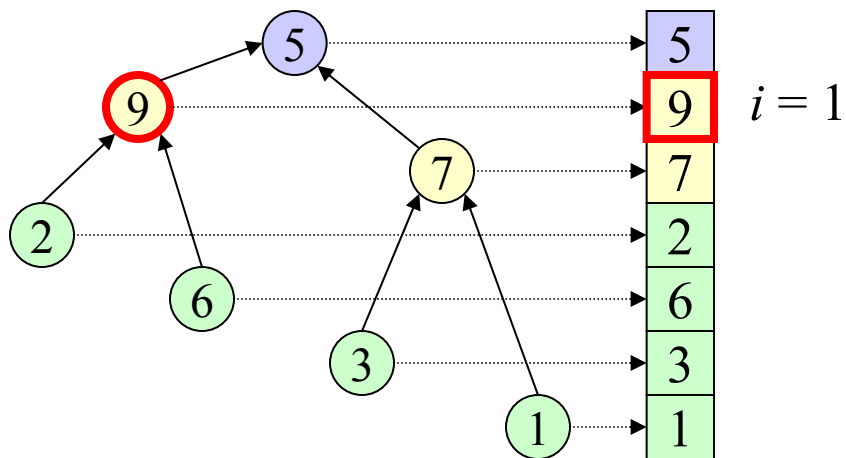
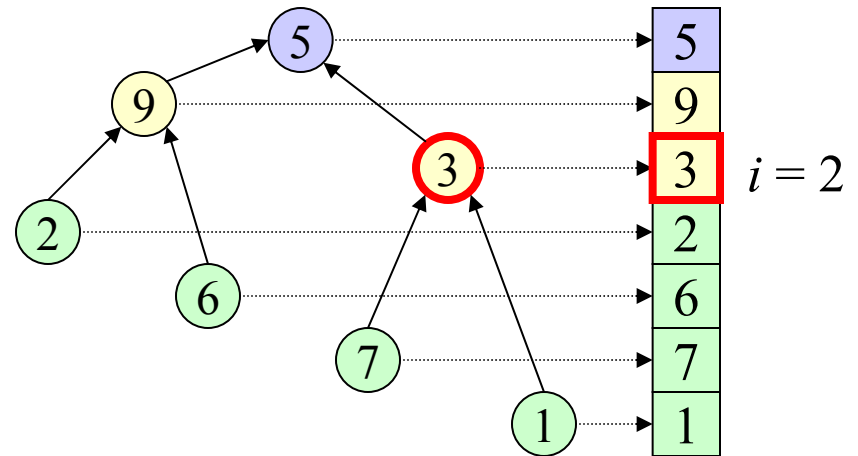
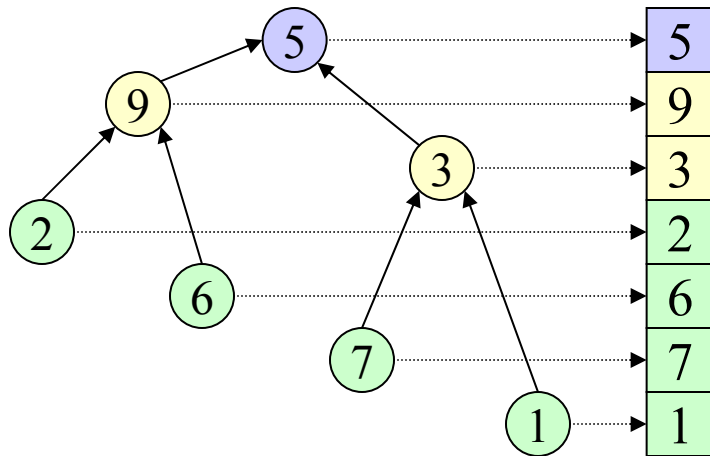
```
BUILD_MAX_HEAP(h)
```

```
1. h.size = h.A.length
```

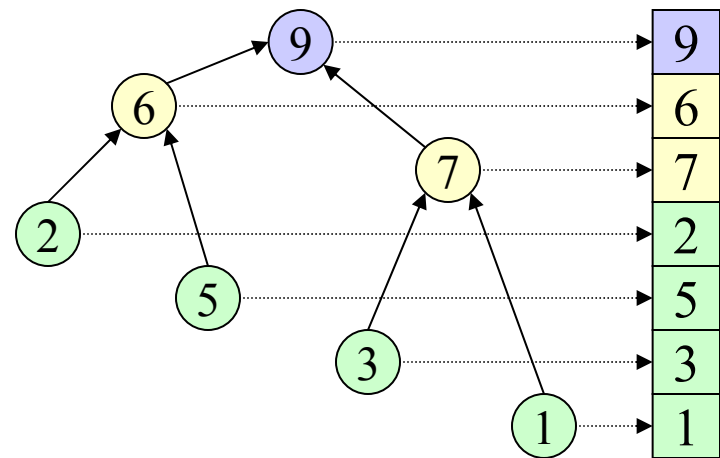
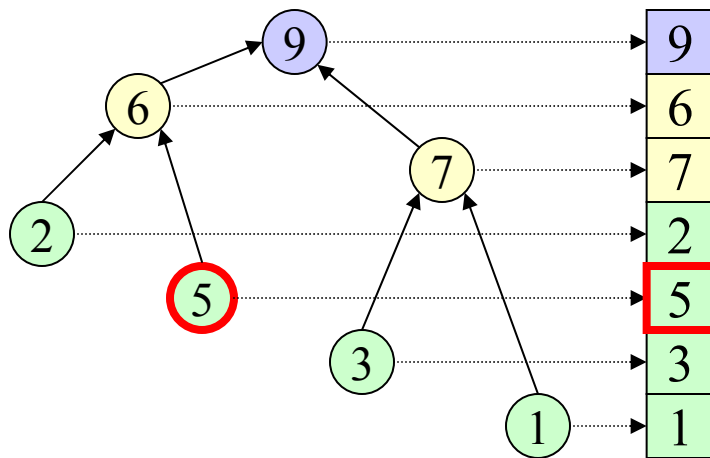
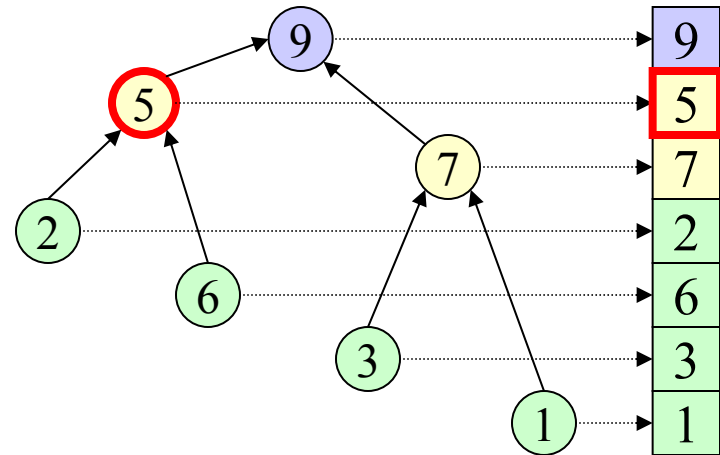
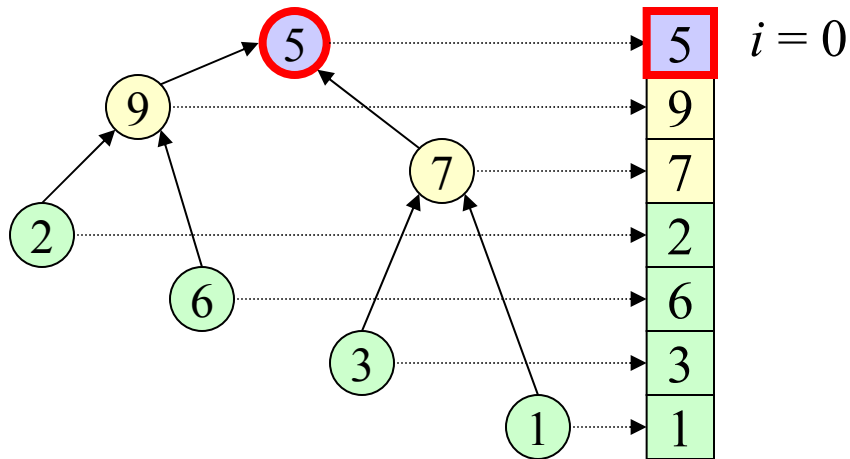
```
2. for i =  $\lfloor h.A.length/2 \rfloor - 1$  downto 0 // i nodi interni
```

```
3.     MAX_HEAPIFY(h, i)
```

Esecuzione di BUILD_MAX_HEAP (1/2)



Esecuzione di BUILD_MAX_HEAP (2/2)



Analisi di BUILD_MAX_HEAP

```
BUILD_MAX_HEAP (h)
```

```
1. h.size = h.A.length
```

```
2. for i =  $\lfloor h.A.length/2 \rfloor - 1$  downto 0 // i nodi interni
```

```
3.     MAX_HEAPIFY (h, i)
```

- BUILD_MAX_HEAP lancia MAX_HEAPIFY un numero $\Theta(n)$ di volte
 - il tempo di esecuzione di MAX_HEAPIFY nel caso peggiore è $\Theta(\log n')$
 - dove n' è il numero dei nodi del sottoalbero radicato al nodo sul quale è lanciato MAX_HEAPIFY
- Siccome $\Theta(\log n') \subseteq O(\log n)$ possiamo dire che la complessità di BUILD_MAX_HEAP nel caso peggiore è $O(n \log n)$
 - $O(n \log n)$ non è un limite asintoticamente stretto
 - con un'analisi più rigorosa dimostreremo che la complessità di BUILD_MAX_HEAP nel caso peggiore è $\Theta(n)$

Complessità di BUILD_MAX_HEAP

- La complessità di BUILD_MAX_HEAP coincide con la somma delle altezze di tutti i sottoalberi radicati ai nodi dell'albero
- Dimostriamo che tale somma sia $\Theta(n)$ per un albero completo con n nodi
- Sia $S(n)$ la somma delle altezze di tutti i sottoalberi di un albero binario completo con n nodi
 - ricorda che la sua altezza è $h = \log_2(n+1) - 1$
- Dimostriamo per induzione che

$$S(n) = n - h - 1 = n - \log_2(n+1) \in \Theta(n)$$

Complessità di BUILD_MAX_HEAP

- Caso base

- per un albero con la sola radice abbiamo

$$n = 1 \quad \text{●} \quad \updownarrow \quad h = 0$$

$$S(n) = n - h - 1$$

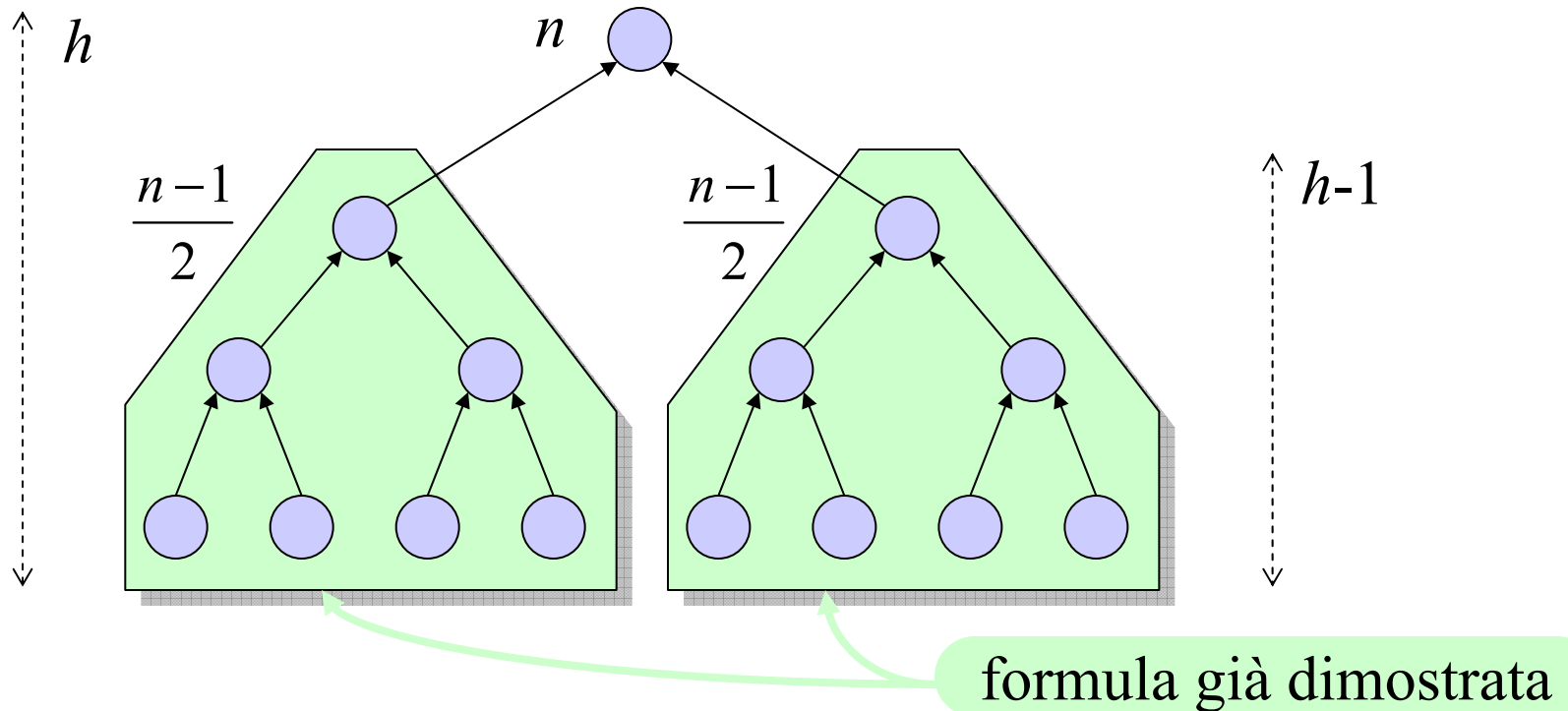
$$S(1) = 1 - 0 - 1 = 0$$

- infatti un albero con la sola radice ha un solo sottoalbero (l'albero stesso) che ha altezza zero

Complessità di BUILD_MAX_HEAP

- Caso induttivo

- supponiamo che la formula sia vera per tutti gli alberi con un numero di nodi minore di n



Complessità di BUILD_MAX_HEAP

- Caso induttivo

$$\begin{aligned} S(n) &= 2S\left(\frac{n-1}{2}\right) + h = \\ &= 2\left(\frac{n-1}{2} - (h-1) - 1\right) + h = \\ &= 2\left(\frac{n-1}{2} - h + 1 - 1\right) + h = \\ &= n - 1 - 2h + h \\ &= n - h - 1 \end{aligned}$$

ipotesi induttiva
 $S(n) = n - h - 1$

Realizzazione di una coda di priorità

- Le code di priorità possono essere gestite tramite un heap

```
NEW_QUEUE ()
```

```
1. /* h è un nuovo oggetto con i campi size (intero) ed A  
2.    (array di 100 interi) */  
3. h.size = 0  
4. return h
```

- Supponiamo di gestire le dimensioni dell'array h.A tramite una crescita telescopica
- La complessità di questa funzione è costante $\Theta(1)$

Realizzazione di una coda di priorità

```
IS_EMPTY(h)
```

```
1. return h.size == 0
```

```
MAXIMUM(h)
```

```
1. return h.A[0]
```

- Le due funzioni qui sopra hanno evidentemente una complessità costante $\Theta(1)$

Procedura `EXTRACT_MAX`

EXTRACT_MAX(h)

```
1. if IS_EMPTY(h)
2.     error("heap underflow")
3. max = h.A[0]
4. h.A[0] = h.A[h.size - 1]
5. h.size = h.size - 1
6. MAX_HEAPIFY(h, 0)
7. return max
```

- Viene eliminato il primo elemento dalla coda
- L'ultimo elemento viene messo al suo posto
- Viene decrementato `h.size`
- La complessità totale è quella di `MAX_HEAPIFY`, cioè $\Theta(\log n)$ nel caso peggiore

Procedura INSERT

INSERT (h, key)

1. **if** h.size == h.A.length

2. error("overflow")

3. h.size = h.size + 1

4. i = h.size - 1

5. **while** i>0 **and** h.A[PARENT(i)] < key

6. h.A[i] = h.A[PARENT(i)]

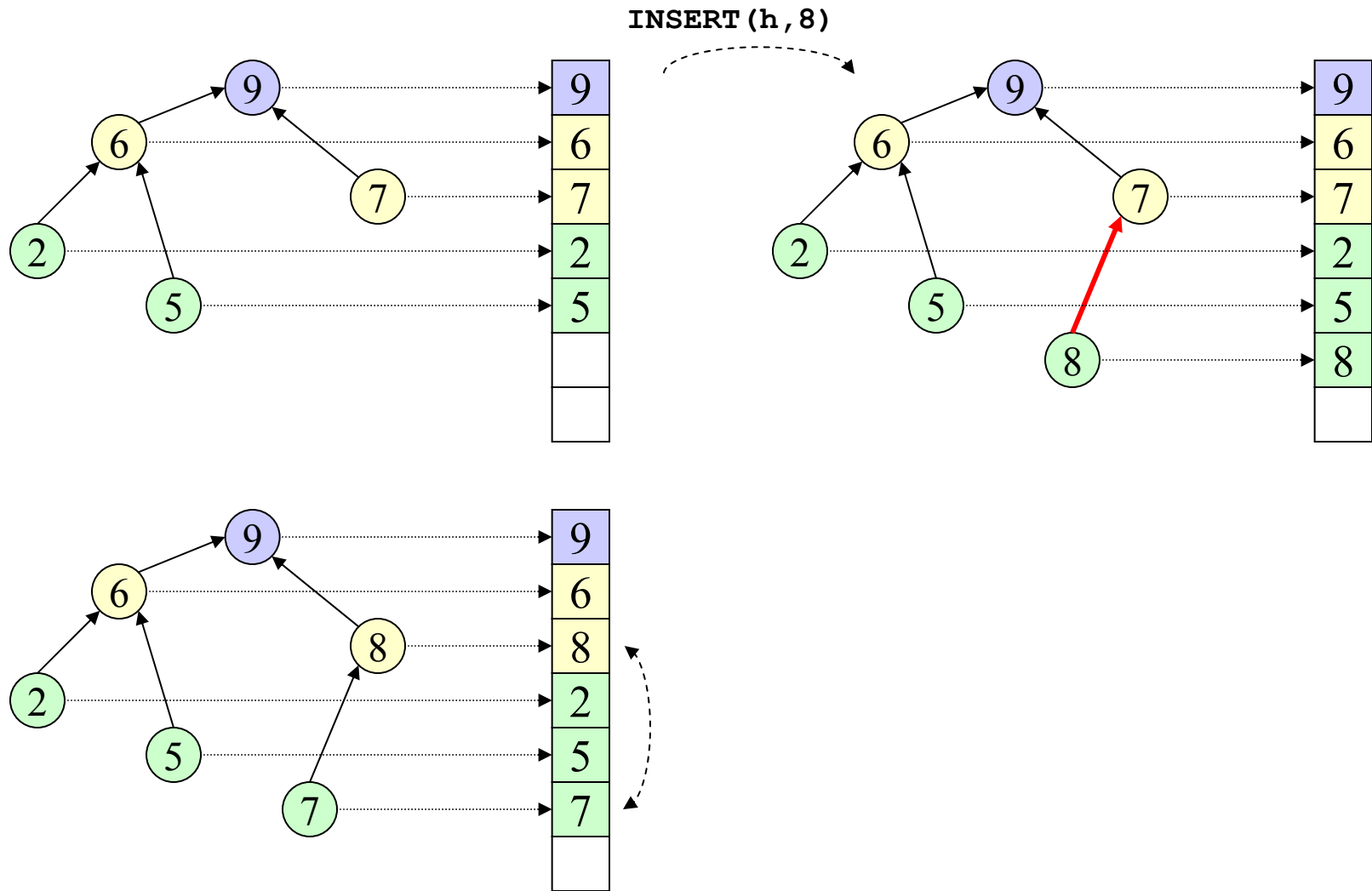
6. /* il genitore di i è stato spostato in basso */

8. i = PARENT(i)

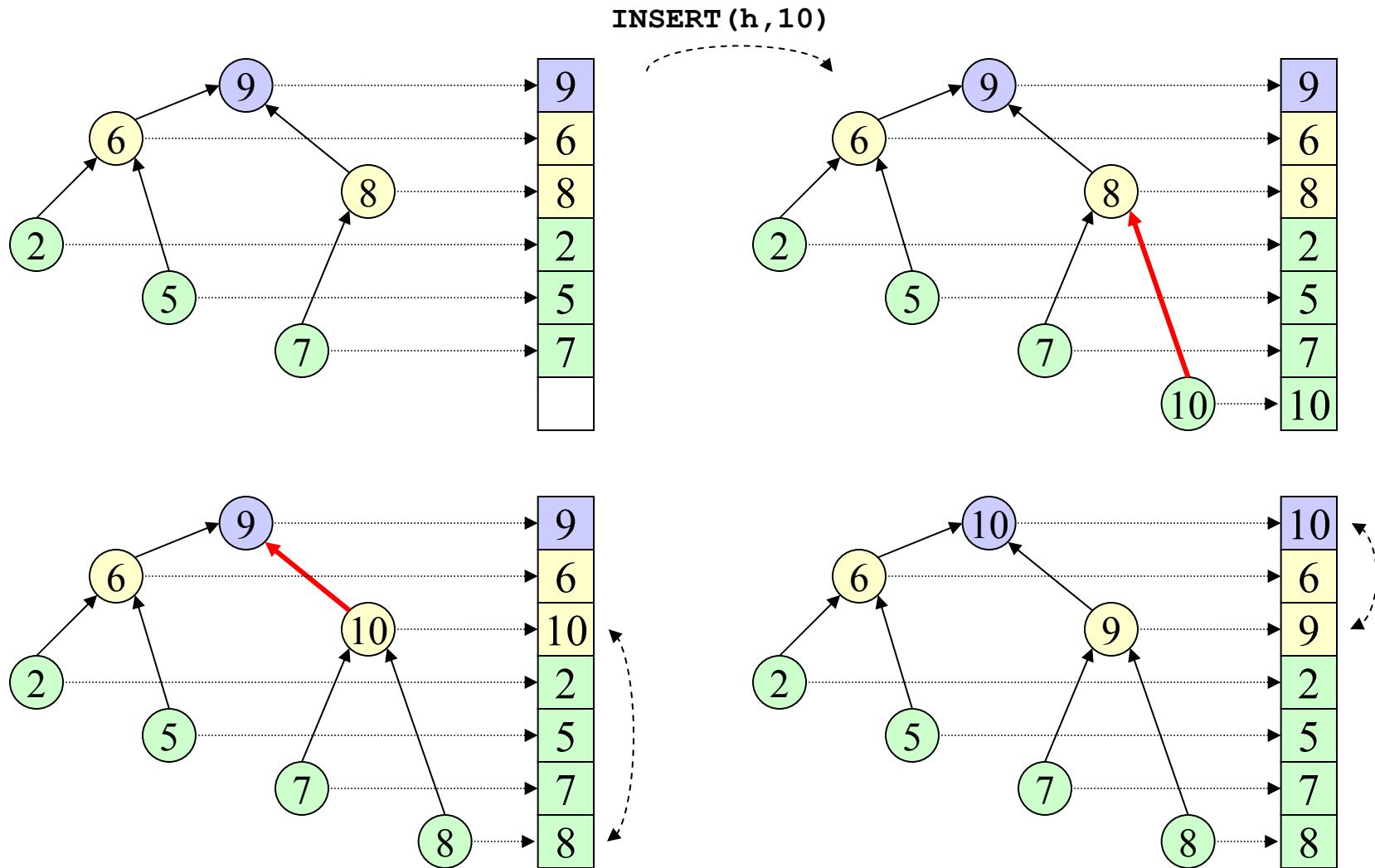
9. h.A[i] = key

- h.size viene incrementato di 1
- Il nuovo elemento viene “spinto in alto” fino a trovare la posizione giusta
- La complessità nel caso peggiore è data dall’altezza dell’albero, cioè $\Theta(\log n)$

Esecuzione di INSERT (1)



Esecuzione di INSERT (2)



Conclusioni sulle strutture di dati heap

- Consentono di realizzare delle code di priorità in cui
 - la creazione della coda di priorità ha complessità $\Theta(n)$
 - procedura `BUILD_MAX_HEAP(h)`
 - l'inserimento di un elemento con priorità arbitraria ha complessità $\Theta(\log n)$
 - procedura `INSERT(h, key)`
 - l'estrazione dell'elemento con chiave maggiore ha complessità $\Theta(\log n)$
 - procedura `EXTRACT_MAX(h)`

Esercizi sugli heap

1. Illustra le operazioni di `INSERT(h,10)` sullo heap

$h.A = \langle 15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$

2. Illustra le operazioni di `EXTRACT_MAX(h)` sullo heap

$h.A = \langle 15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$

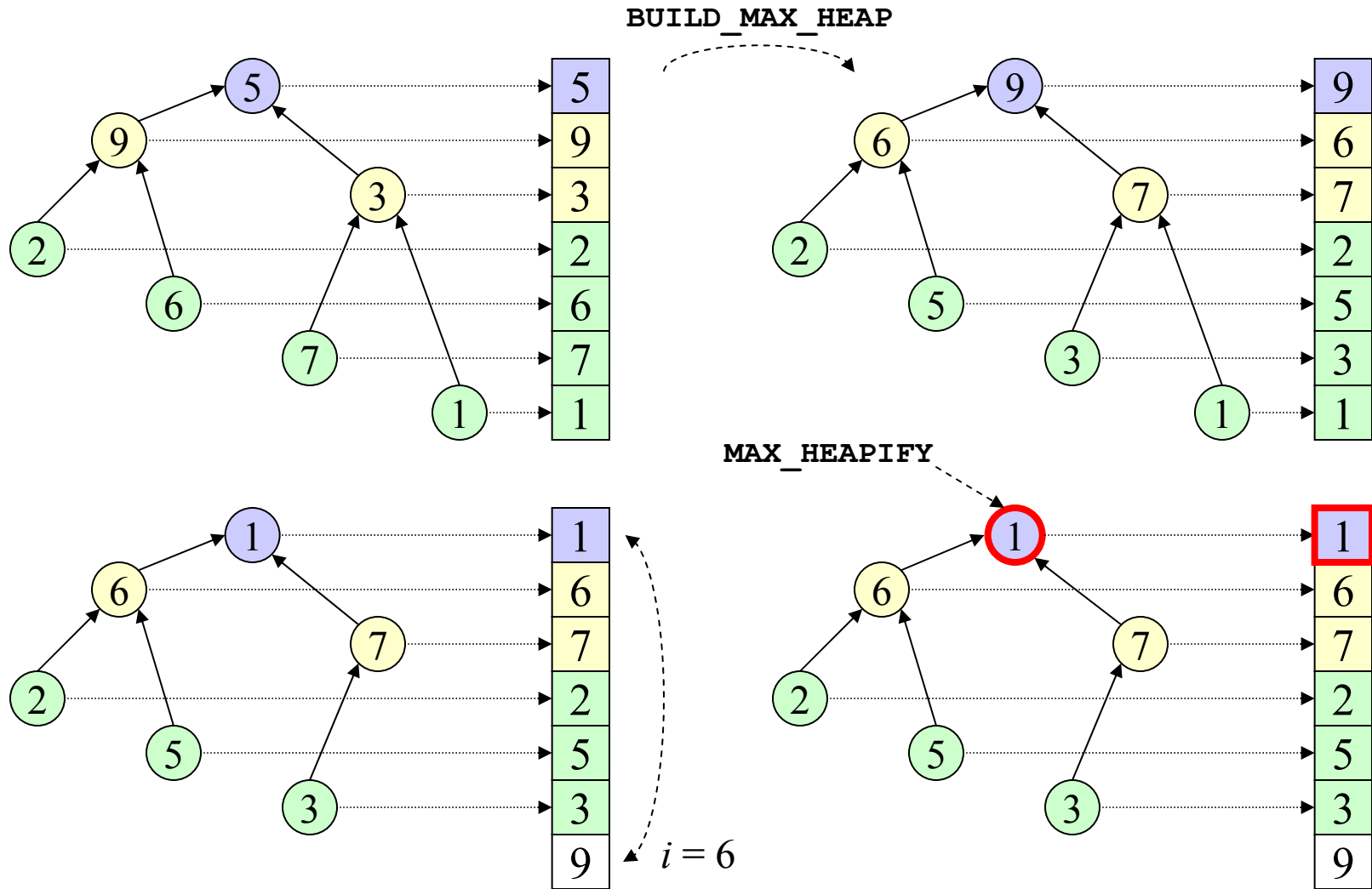
Procedura HEAP_SORT

HEAP_SORT (A)

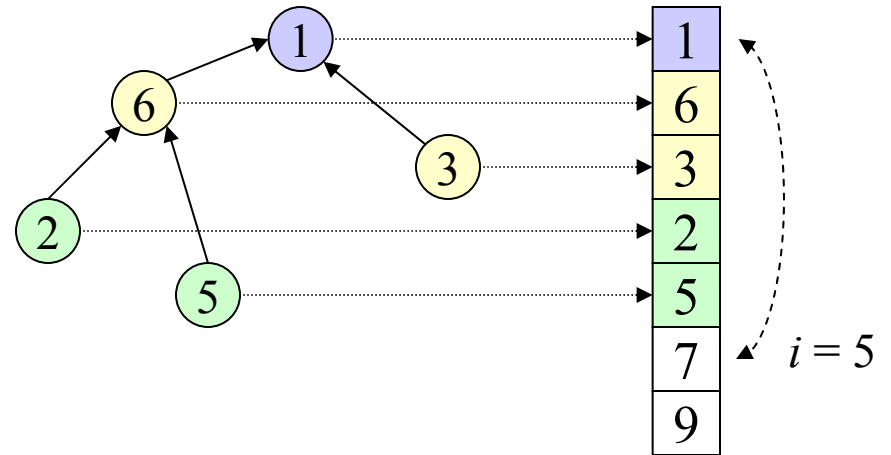
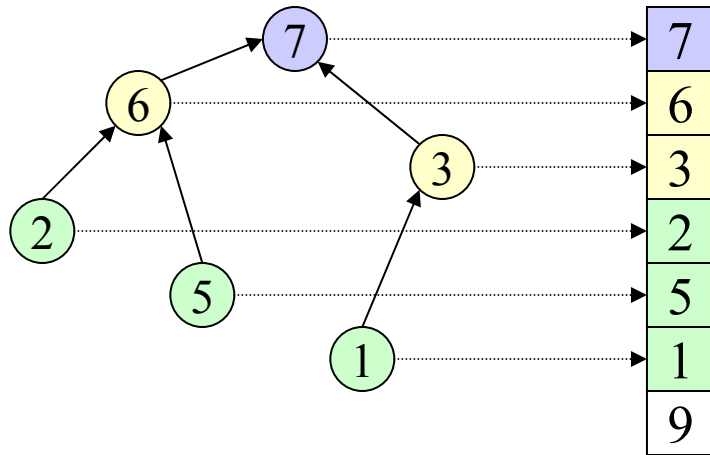
```
1. h.A = A    /* h è un nuovo heap */
2. h.size = A.length
3. BUILD_MAX_HEAP(h)
4. for i = h.A.length-1 downto 1
5.     SCAMBIA_CASELLE(A, 0, i)
6.     h.size = h.size - 1
7.     MAX_HEAPIFY(h, 0)
```

- A viene trasformato in un heap ($\Theta(n)$)
- Per i che va da 0 ad $A.length-1$ (cioè $\Theta(n)$ volte)
 - viene estratto il primo elemento di A e viene posto in coda all'array ($\Theta(1)$)
 - viene lanciato MAX_HEAPIFY per ripristinare le proprietà dell'heap (tempo $\Theta(\log n)$ se gli elementi sono tutti distinti)

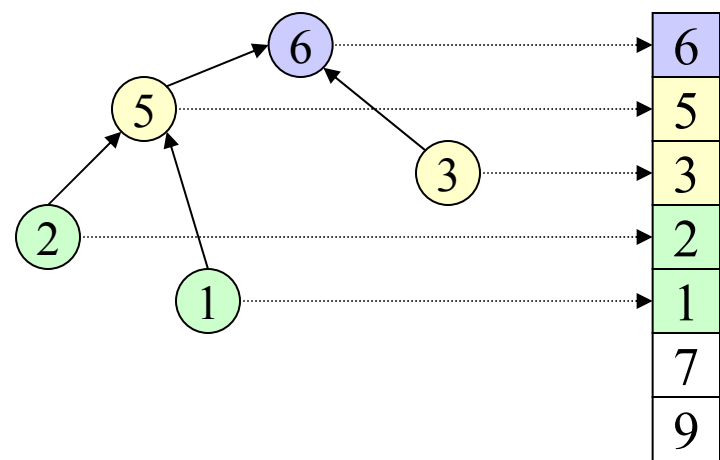
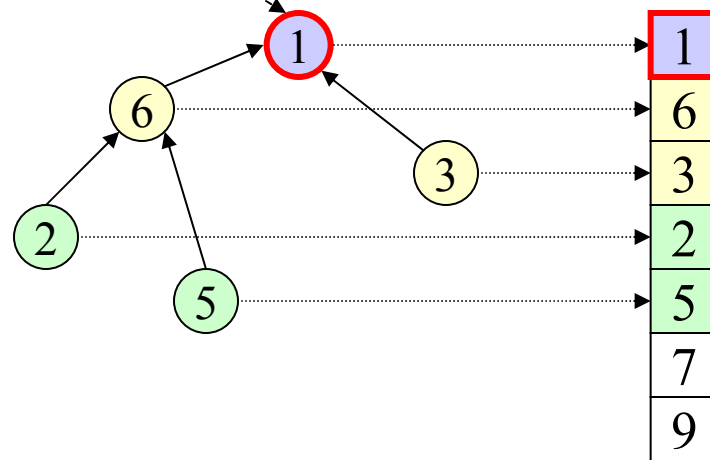
Esecuzione di HEAP_SORT (1/5)



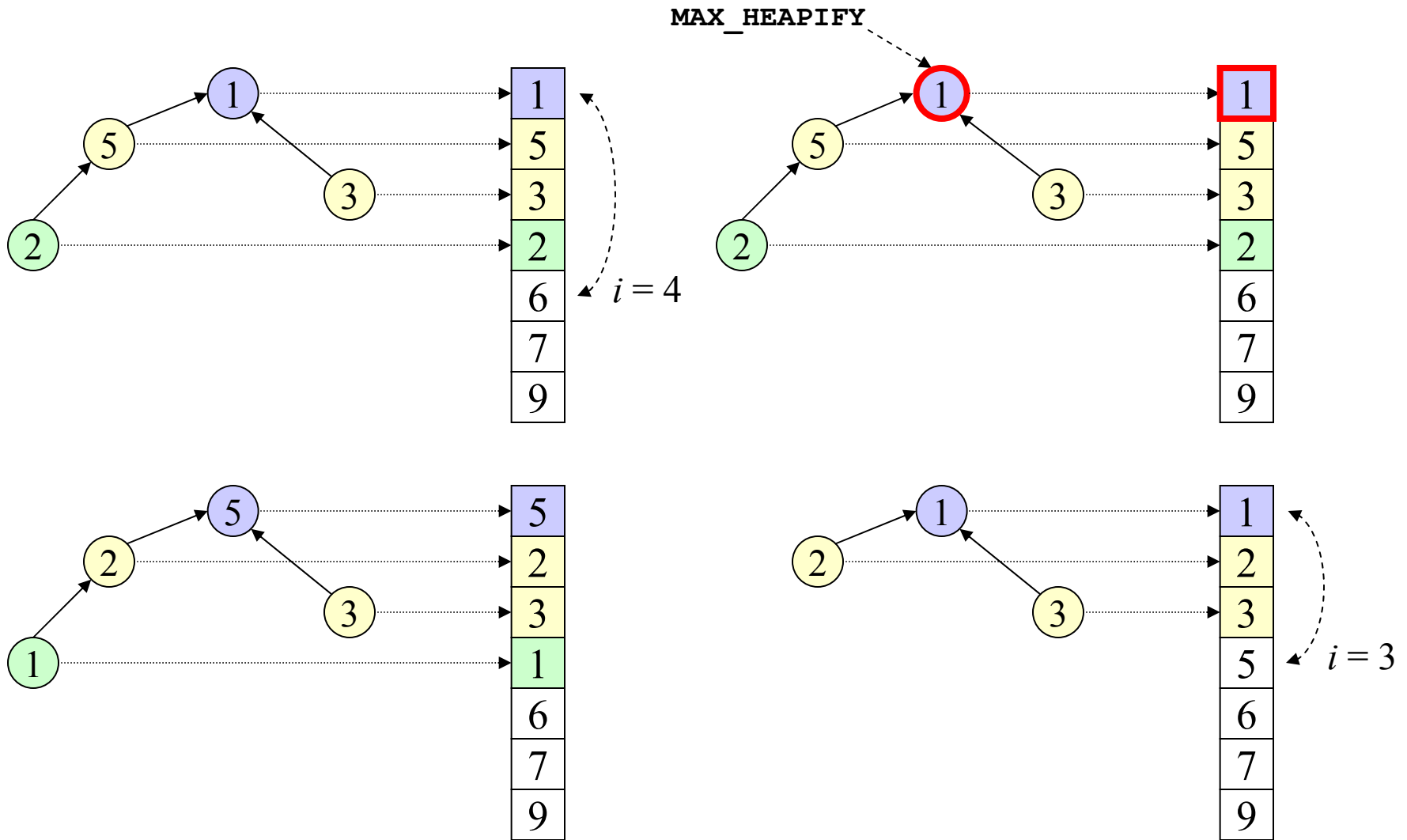
Esecuzione di HEAP_SORT (2/5)



MAX_HEAPIFY

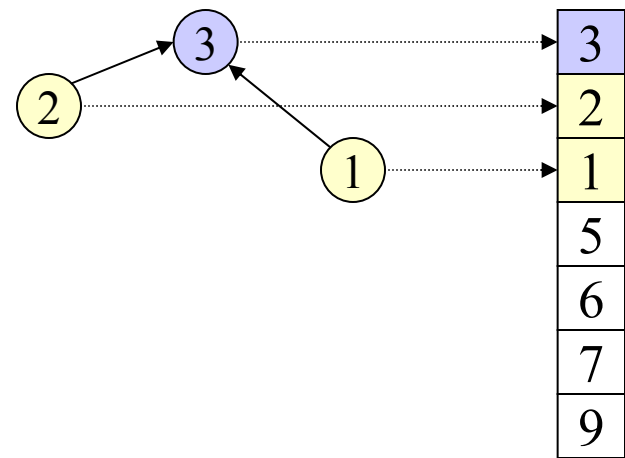
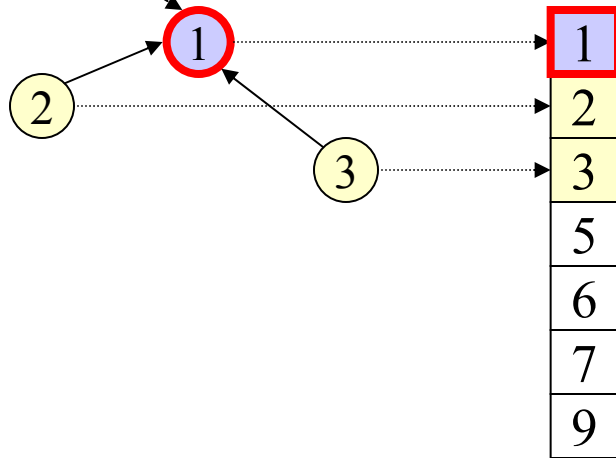


Esecuzione di HEAP_SORT (3/5)

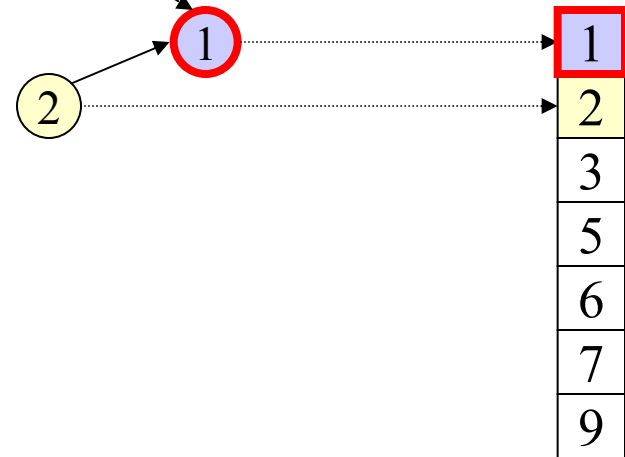
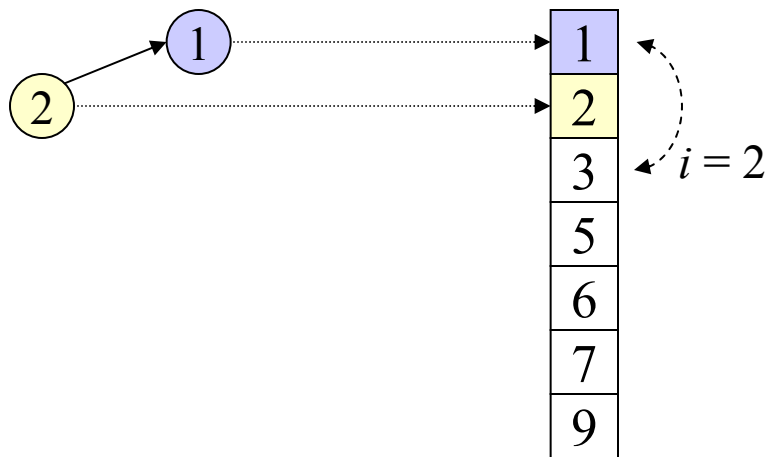


Esecuzione di HEAP_SORT (4/5)

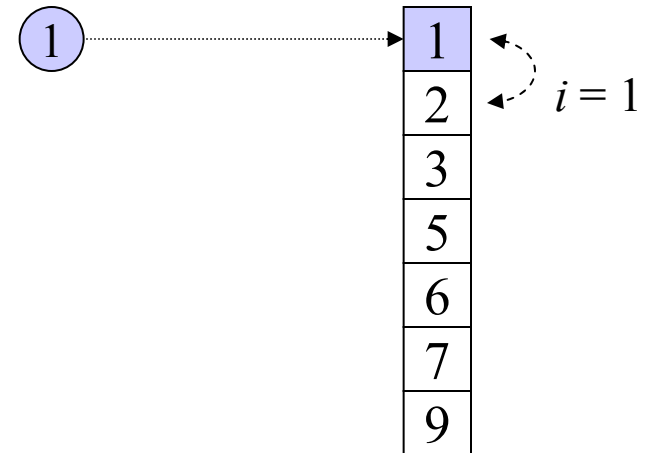
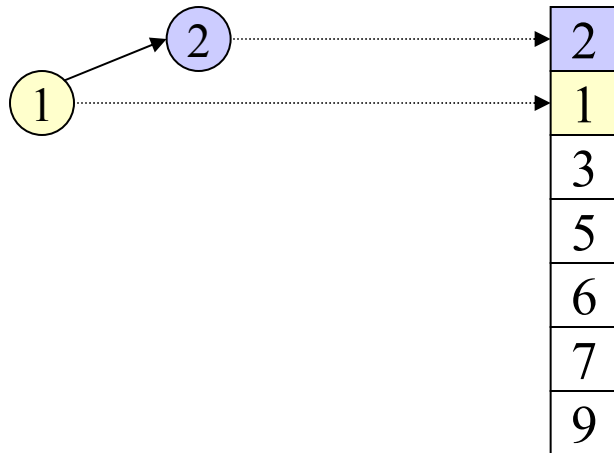
MAX_HEAPIFY



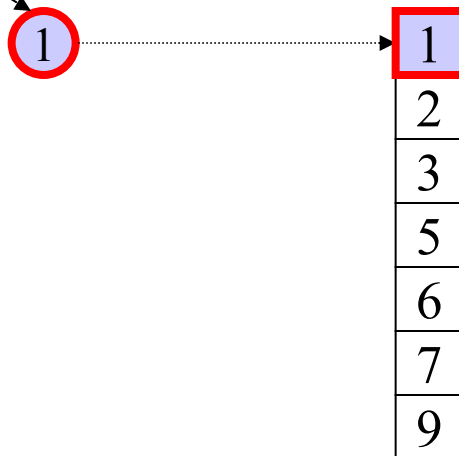
MAX_HEAPIFY



Esecuzione di HEAP_SORT (5/5)

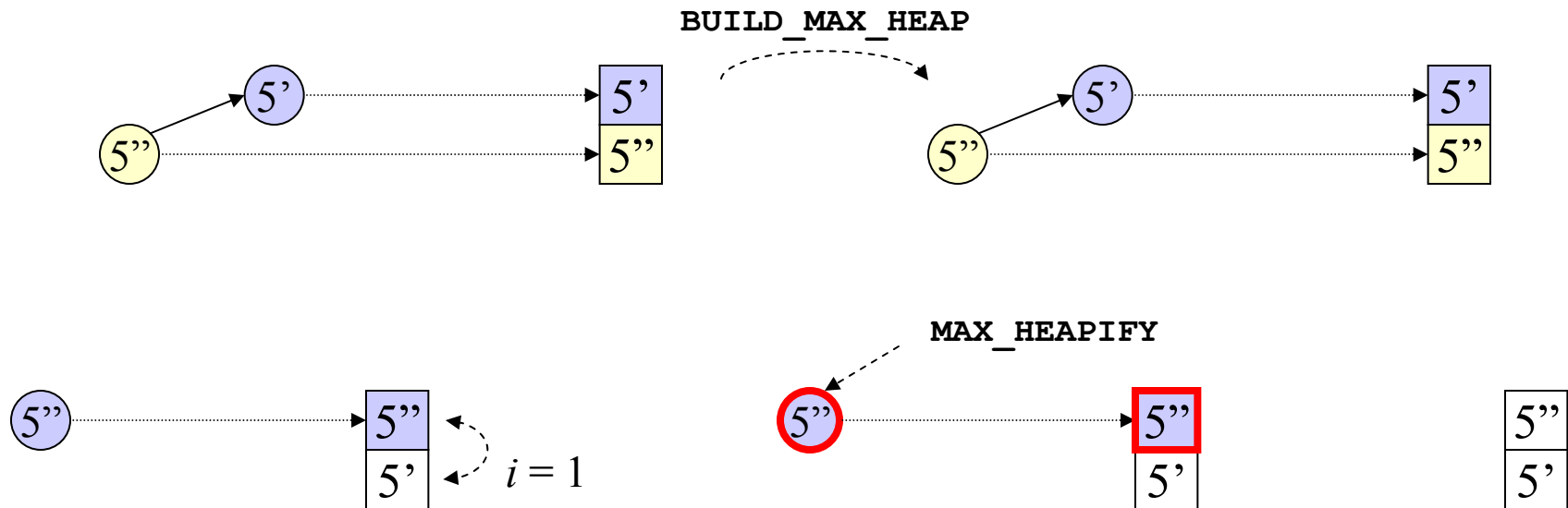


MAX_HEAPIFY



HEAP_SORT non è stabile

- Lo dimostriamo con un controesempio



- Ora la posizione dei due elementi è invertita

Algoritmi di ordinamento visti finora

	caso migliore	caso medio	caso peggiore	in loco	stabile
SELECTION_SORT	$\Theta(n^2)$			<i>si</i>	<i>si</i>
INSERTION_SORT	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	<i>si</i>	<i>si</i>
MERGE_SORT	$\Theta(n \log n)$			<i>no</i>	<i>si</i>
HEAP_SORT	$\Theta(n \log n)$			<i>si</i>	<i>no</i>

Nota: nel caso migliore HEAP_SORT ha complessità $\Theta(n \log n)$ se gli elementi sono tutti distinti e complessità $\Theta(n)$ se gli elementi sono tutti uguali

Domande sugli heap

3. Quali sono il numero minimo ed il numero massimo di elementi in uno heap di altezza h ?
4. In un max-heap, dove potrebbe risiedere l'elemento più piccolo, assumendo che siano tutti distinti?
5. Un heap in cui l'array è ordinato in ordine inverso è un max-heap?
6. La sequenza $\langle 23, 17, 14, 6, 13, 10, 1, 5, 7, 12 \rangle$ è un max-heap?
7. Qual è l'effetto di $\text{MAX_HEAPIFY}(h,i)$ se l'elemento $h.A[i]$ è più grande dei suoi figli?
8. Qual è l'effetto di $\text{MAX_HEAPIFY}(h,i)$ se $i > h.\text{size}/2 - 1$?

Esercizi sulle code di priorità

9. Illustra le operazioni di $\text{MAX_HEAPIFY}(h,2)$ sullo heap
 $h.A = \langle 27, 17, 3, 16, 13, 10, 1, 5, 7, 12, 4, 8, 9, 0 \rangle$
10. Illustra le operazioni di $\text{BUILD_MAX_HEAP}(h)$ sullo heap
 $h.A = \langle 5, 3, 17, 10, 84, 19, 6, 22, 9 \rangle$
11. Illustra le operazioni di HEAP_SORT sull'array
 $A = \langle 5, 13, 2, 25, 7, 17, 20, 8, 4 \rangle$

Algoritmi e Strutture di Dati

Quick-sort

m.patrignani

Nota di copyright

- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Quick-sort

- Algoritmo di ordinamento in loco ma non stabile
- Tempo di esecuzione
 - nel caso peggiore $\Theta(n^2)$
 - nel caso migliore e medio $\Theta(n \log n)$
 - i fattori costanti nascosti nella notazione Θ sono abbastanza piccoli
- Introdotto da Hoare nel 1962
 - la versione che vedremo è una variante dovuta a Lomuto
- Basato sul paradigma divide et impera

Divide et impera nel quick-sort

Per ordinare un sottoarray $A[p..r]$

- **Divide**
 - $A[p..r]$ viene ripartito (e risistemato) in due sottoarray non vuoti $A[p..q-1]$ e $A[q+1..r]$, in modo che ogni elemento del primo sia minore o uguale ad $A[q]$ e ogni elemento del secondo sia maggiore ad $A[q]$
 - l'indice q viene calcolato dalla procedura di partizionamento
- **Impera**
 - i due sottoarray $A[p..q-1]$ e $A[q+1..r]$ sono ordinati, ricorsivamente
- **Combina**
 - non c'è niente da fare: $A[p..r]$ è ordinato

Procedura QUICK_SORT

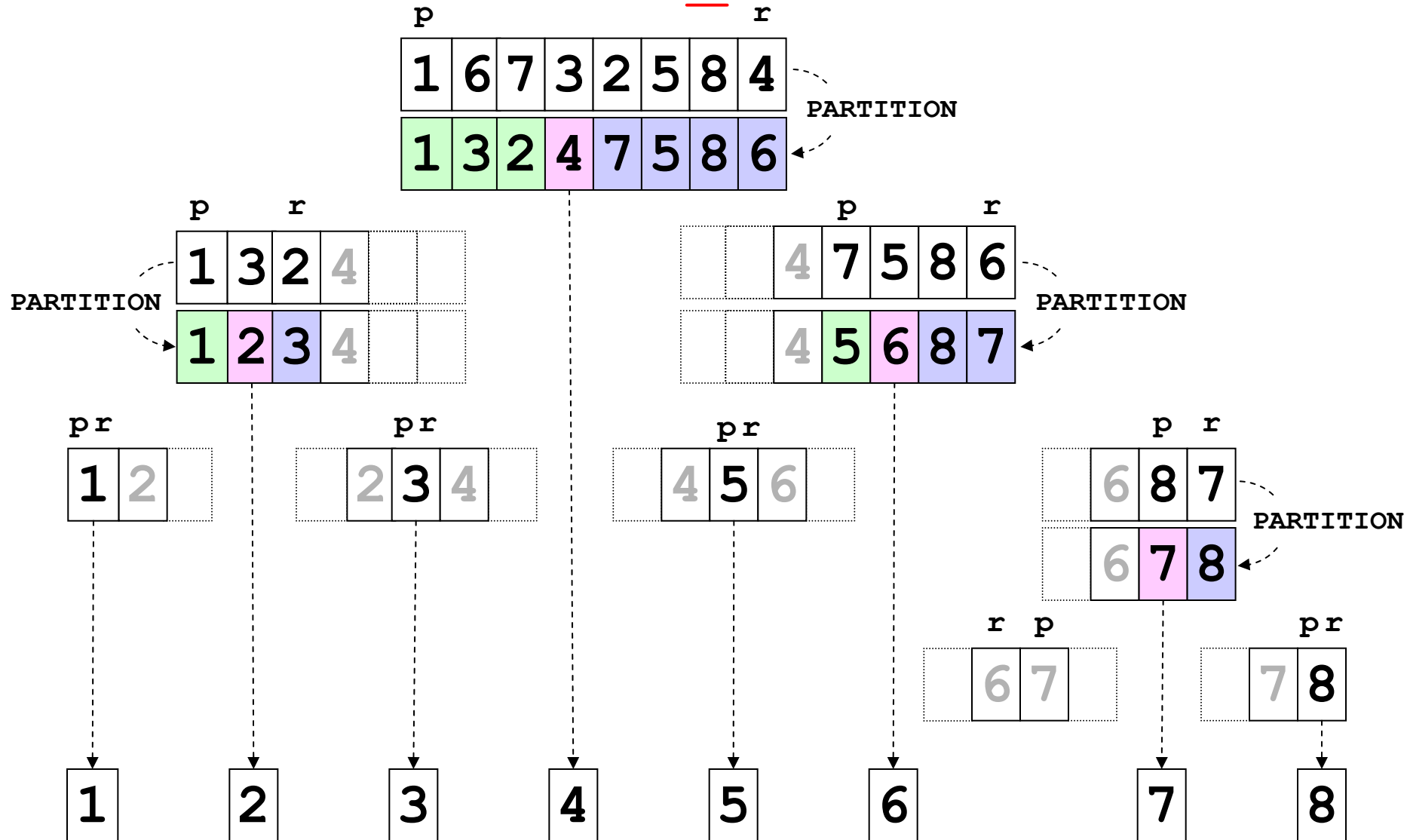
QUICK_SORT (A, p, r)

1. **if** $p < r$
2. $q = \text{PARTITION}(A, p, r)$
3. **QUICK_SORT** ($A, p, q-1$)
4. **QUICK_SORT** ($A, q+1, r$)

- La procedura **QUICK_SORT** ordina in loco l'intervallo $A[p..r]$
 - se $p = r$, allora l'intervallo contiene una sola casella ed è già ordinato: l'invocazione di **QUICK_SORT** non ha effetto
 - se $p > r$, allora l'intervallo è un intervallo degenere e l'invocazione di **QUICK_SORT** non ha effetto
- Il valore q ritornato da **PARTITION** è tale che $p \leq q \leq r$
- Per ordinare l'intero array viene invocata la procedura:
QUICK_SORT($A, 0, A.length-1$)

Esecuzione di QUICK_SORT su

1	6	7	3	2	5	8	4
---	---	---	---	---	---	---	---

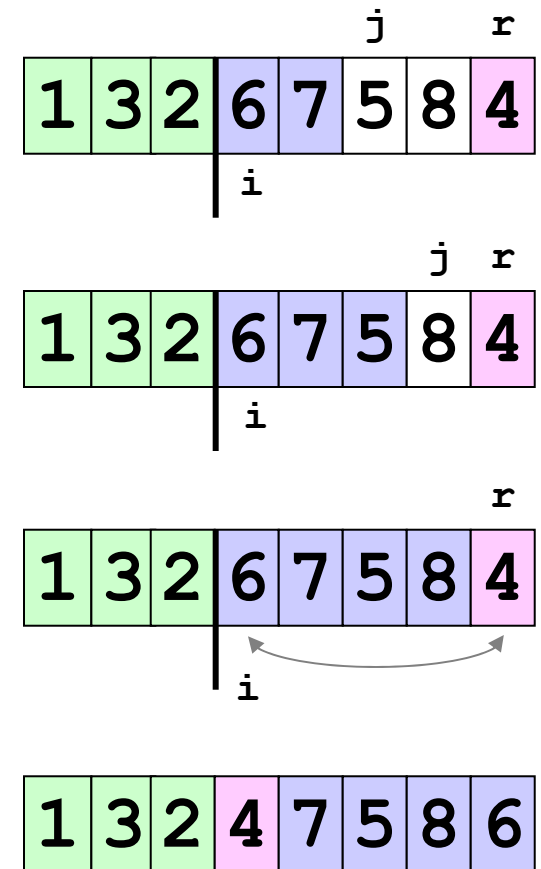
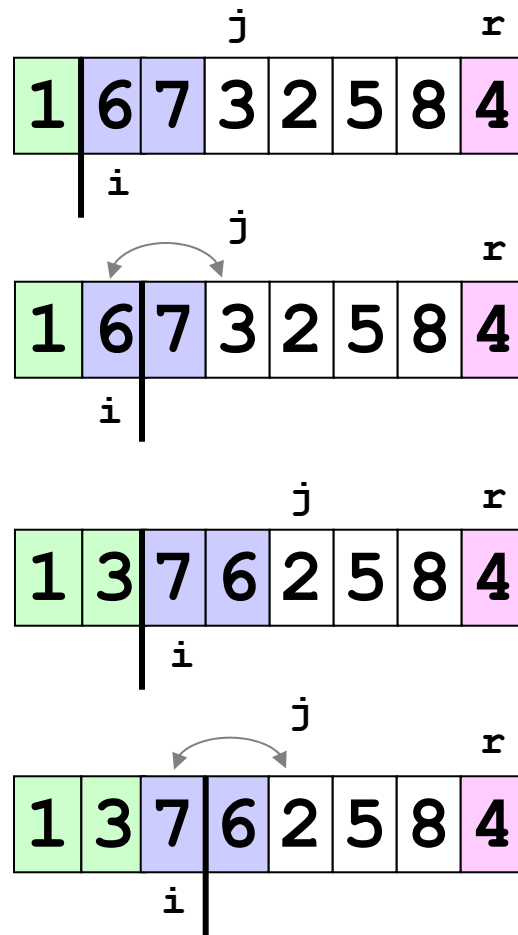
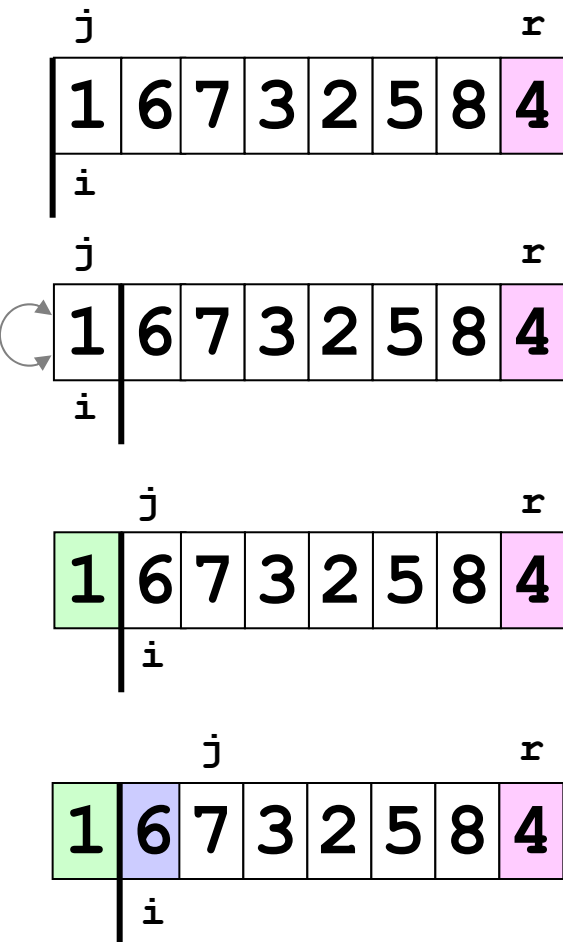


Procedura PARTITION

```
PARTITION(A, p, r)      /* si assume  $p < r$  */  
1.  i = p      /* i è il primo elemento > A[r] = pivot */  
2.    for j = p to r - 1 /* scorro l'array (non il pivot) */  
3.        if A[j] ≤ A[r] /* A[r] è il pivot */  
4.            SCAMBIA(A, i, j)  
5.            i = i + 1  
6.  SCAMBIA(A, i, r)      /* metto il pivot al centro */  
7.  return i      /* ritorno la posizione del pivot */
```

- La procedura PARTITION viene invocata su un intervallo di almeno due elementi ($p < r$)
 - due casi base
 - i due elementi sono ordinati
 - i due elementi non sono ordinati

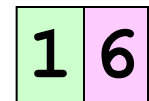
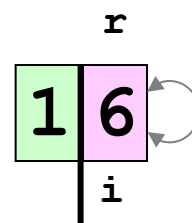
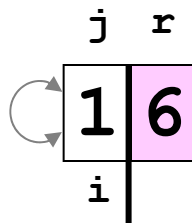
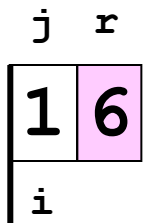
Esecuzione di PARTITION su | | | | | | | | | |---|---|---|---|---|---|---|---| | 1 | 6 | 7 | 3 | 2 | 5 | 8 | 4 | |---|---|---|---|---|---|---|---|



Esecuzione di PARTITION su | | | |---|---| | 1 | 6 | |---|---|

- Caso base 1: PARTITION su una coppia ordinata

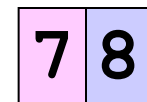
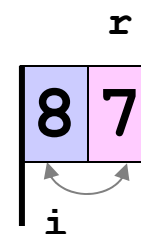
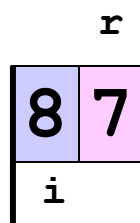
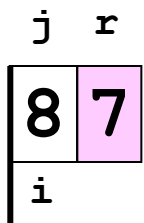
```
PARTITION(A,p,r)    /* si assume  $p < r$  */  
1.  $i = p$           /*  $i$  è il primo elemento  $> A[r] = \text{pivot}$  */  
2.   for  $j = p$  to  $r - 1$  /* scorro l'array (non il pivot) */  
3.       if  $A[j] \leq A[r]$  /*  $A[r]$  è il pivot */  
4.           SCAMBIA(A,i,j)  
5.            $i = i + 1$   
6. SCAMBIA(A,i,r)    /* metto il pivot al centro */  
7. return  $i$         /* ritorno la posizione del pivot */
```



Esecuzione di PARTITION su | | | |---|---| | 8 | 7 | |---|---|

- Caso base 2: PARTITION su una coppia non ordinata

```
PARTITION(A,p,r)    /* si assume  $p < r$  */  
1.  $i = p$           /*  $i$  è il primo elemento  $> A[r] = \text{pivot}$  */  
2.   for  $j = p$  to  $r - 1$  /* scorro l'array (non il pivot) */  
3.       if  $A[j] \leq A[r]$  /*  $A[r]$  è il pivot */  
4.           SCAMBIA(A,i,j)  
5.            $i = i + 1$   
6. SCAMBIA(A,i,r)    /* metto il pivot al centro */  
7. return  $i$         /* ritorno la posizione del pivot */
```



Tempo di esecuzione di PARTITION

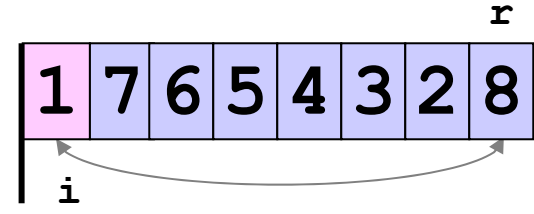
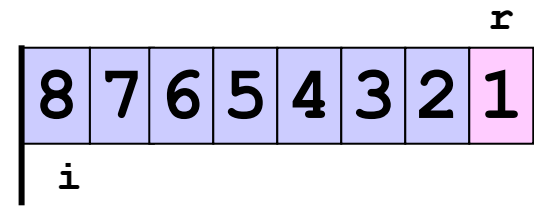
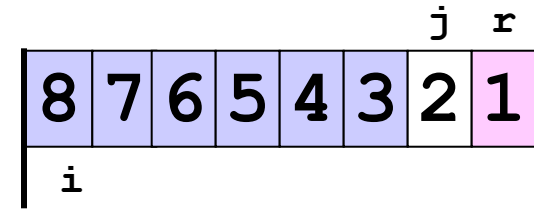
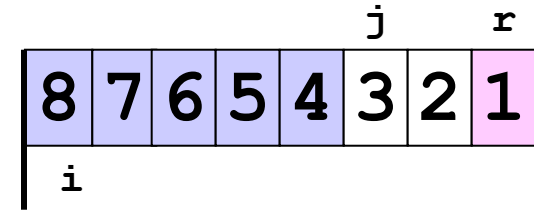
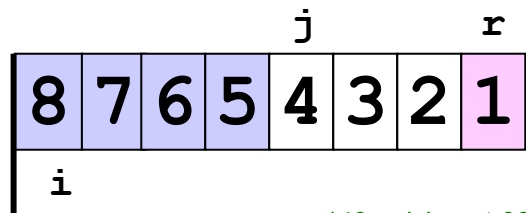
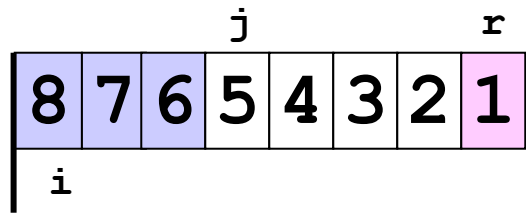
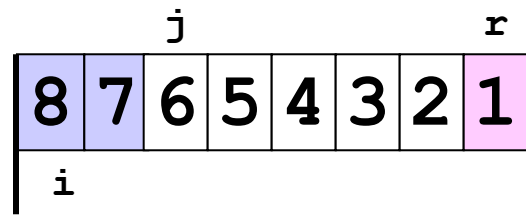
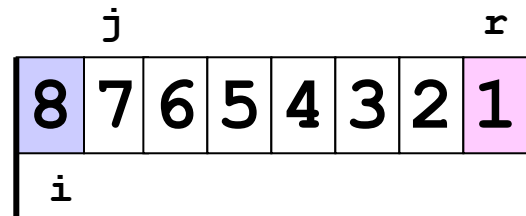
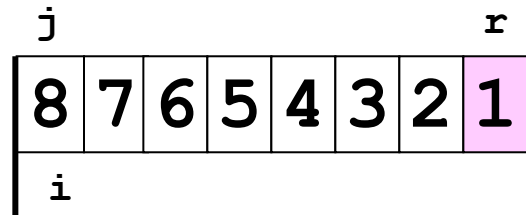
```
PARTITION(A, p, r)      /* si assume  $p < r$  */  
1.   $i = p$           /*  $i$  è il primo elemento  $> A[r] = \text{pivot}$  */  
2.    for  $j = p$  to  $r - 1$  /* scorro l'array (non il pivot) */  
3.      if  $A[j] \leq A[r]$  /*  $A[r]$  è il pivot */  
4.        SCAMBIA(A,  $i, j$ )  
5.       $i = i + 1$   
6.  SCAMBIA(A,  $i, r$ ) /* metto il pivot al centro */  
7.  return  $i$       /* ritorno la posizione del pivot */
```

- Le assegnazioni iniziali e finali richiedono tempo costante
- Nel caso peggiore, come nel caso migliore, il sottoarray $A[p \dots r]$ viene scorso per intero da sinistra verso destra
- Il tempo di esecuzione $T_{\text{PARTITION}}(n) \in \Theta(n)$

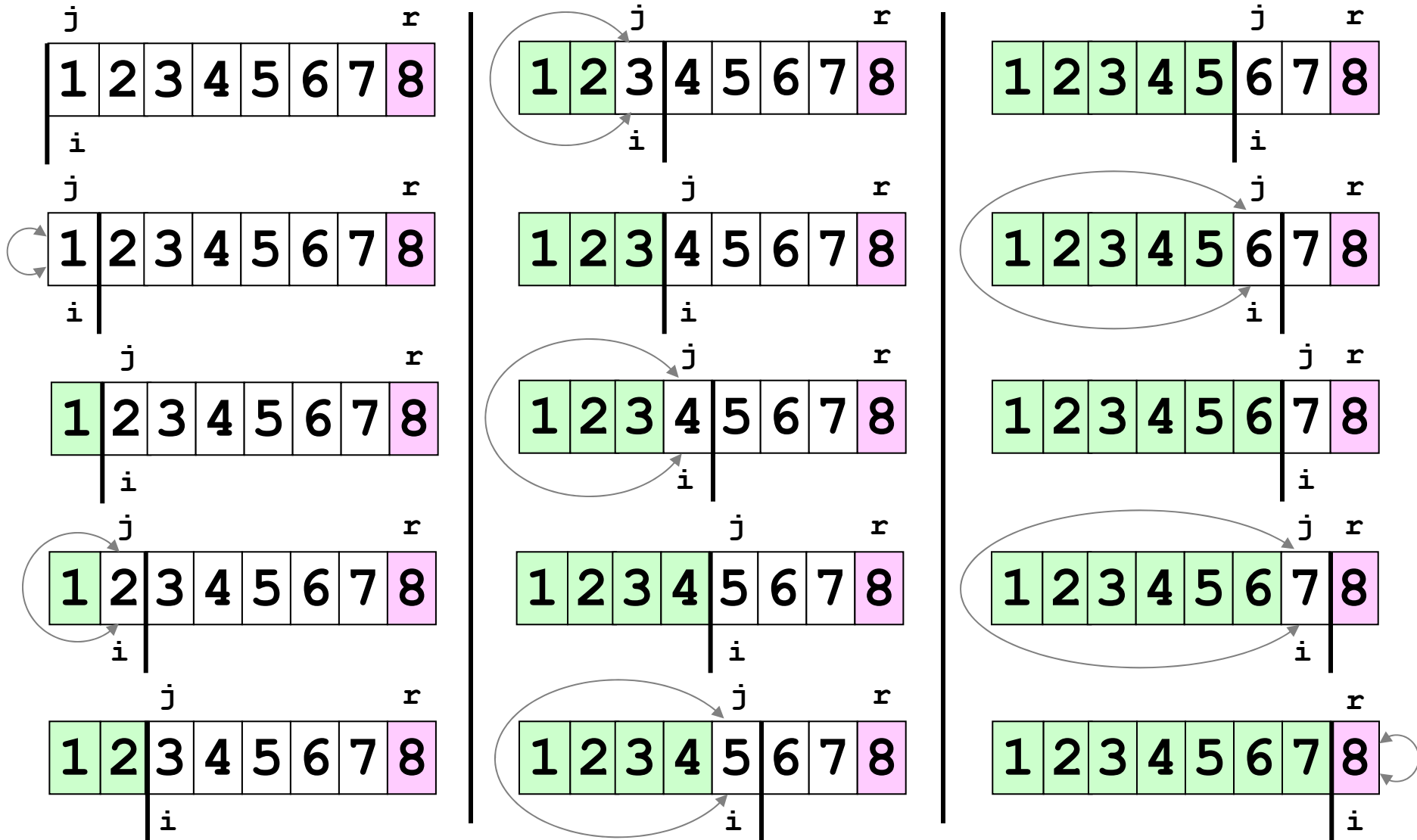
Esercizi

1. Che cosa succederebbe nel QUICK_SORT se $\text{PARTITION}(A, p, r)$ restituisse un valore q uguale a r ?
2. Illustrare le operazioni di PARTITION sull'array $A = \langle 13, 19, 9, 5, 12, 8, 7, 4, 11, 2, 6, 21 \rangle$
3. Illustrare le operazioni di PARTITION su un array
 - già ordinato in senso decrescente
 - già ordinato in senso crescente
4. Quale valore restituisce PARTITION se tutti gli elementi dell'array $A[p..r]$ hanno lo stesso valore?

Esecuzione di PARTITION su | | | | | | | | | |---|---|---|---|---|---|---|---| | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | |---|---|---|---|---|---|---|---|



Esecuzione di PARTITION su | | | | | | | | | |---|---|---|---|---|---|---|---| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | |---|---|---|---|---|---|---|---|



Caso peggiore e migliore per QUICK_SORT

- Il caso peggiore per QUICK_SORT è quando PARTITION elegge a *pivot* il valore massimo o minimo dell'array
 - in questo caso QUICK_SORT non ricorre su due sottoarray bilanciati, ma ricorre su un sottoarray più corto di una casella ed un sottoarray degenerare
- Il caso migliore per QUICK_SORT è invece quando PARTITION elegge a *pivot* il valore mediano dell'array
 - in questo caso QUICK_SORT ricorre su due sottoarray bilanciati

Analisi del caso migliore per QUICK_SORT

- Nel caso migliore il tempo di calcolo di QUICK_SORT su un array con n posizioni è

$$T(n) = 2 \cdot T(n/2) + \Theta(n)$$

- Questa equazione di ricorrenza può essere risolta con il teorema dell'esperto

$$T(n) = a \cdot T(n/b) + p(n^k)$$

- Nello speciale caso in cui

$$a=2 \qquad b=2 \qquad k=1$$

- Che per $a = b^k$ si risolve in

$$T(n) = \Theta(n^k \log n) = \Theta(n \log n)$$

Analisi del caso peggiore per QUICK_SORT

- Si ha

$$T(0) = a$$

$$T(n) = T(n-1) + \Theta(n)$$

- Sappiamo che la soluzione di questa equazione di ricorrenza è

$$T(n) = a + \sum_{k=1}^n g(k)$$

- E dunque

$$T(n) = \sum_{k=1}^n \Theta(k) = \Theta\left(\sum_{k=1}^n k\right) = \Theta(n^2)$$

Analisi del caso medio per QUICK_SORT

- Si può dimostrare formalmente che nel caso medio QUICK_SORT ha una complessità $\Theta(n \log n)$
 - l'analisi, però, è molto più complessa del caso migliore e del caso peggiore
- Nel seguito vedremo solamente due considerazioni intuitive che ci aiutano a giustificare questo risultato
 1. qual è la complessità nel caso in cui lo sbilanciamento della ricorsione non supera mai una determinata soglia
 2. qual è la complessità nel caso in cui ricorsioni sbilanciate si alternano a ricorsioni più bilanciate

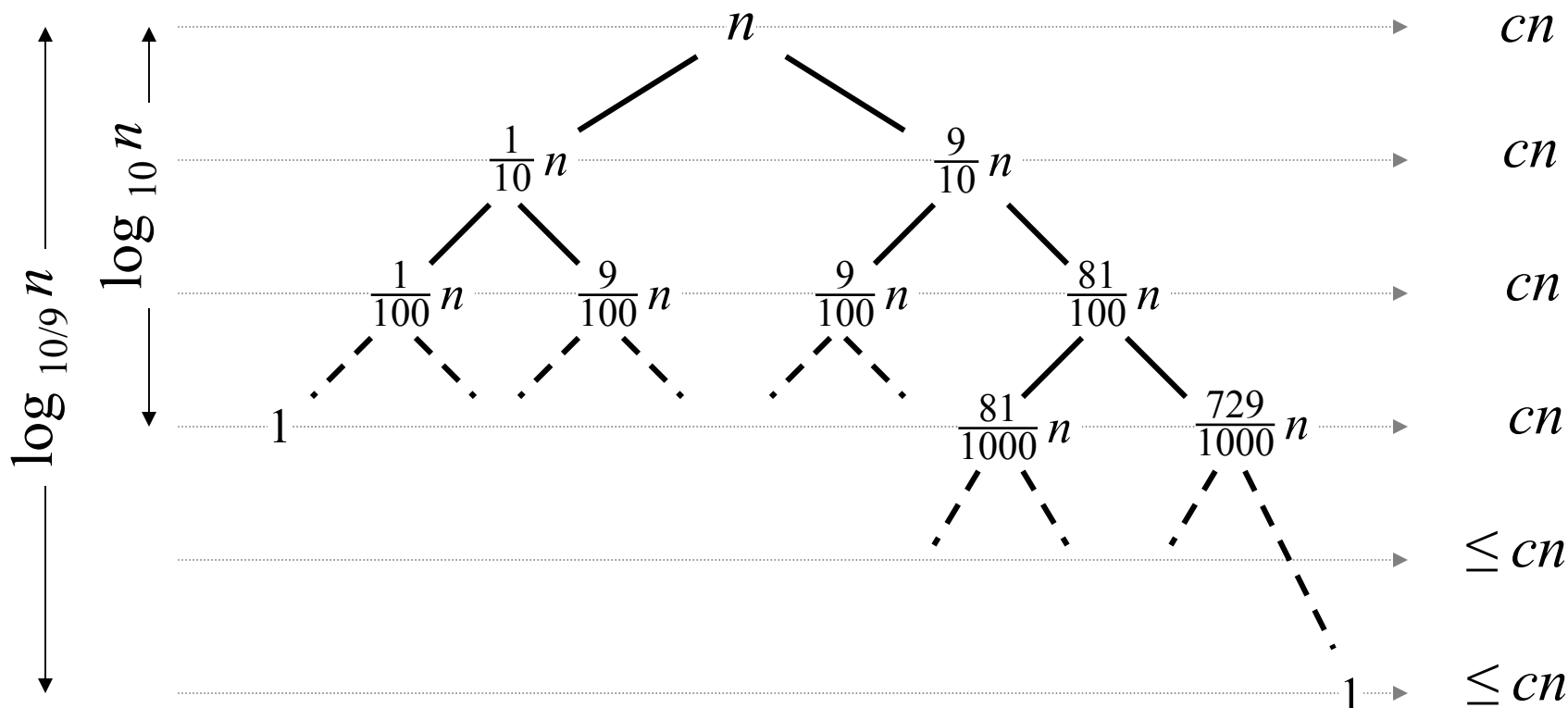
Caso bilanciato 9-a-1

- Supponiamo che PARTITION divida il sottoarray in due parti che hanno una proporzione fissa
 - supponiamo che la proporzione sia 9-a-1
- Abbiamo

$$T(n) \leq T(9n/10) + T(n/10) + cn$$

dove cn esplicita $\Theta(n)$

Ricorsione con proporzione 9-a-1

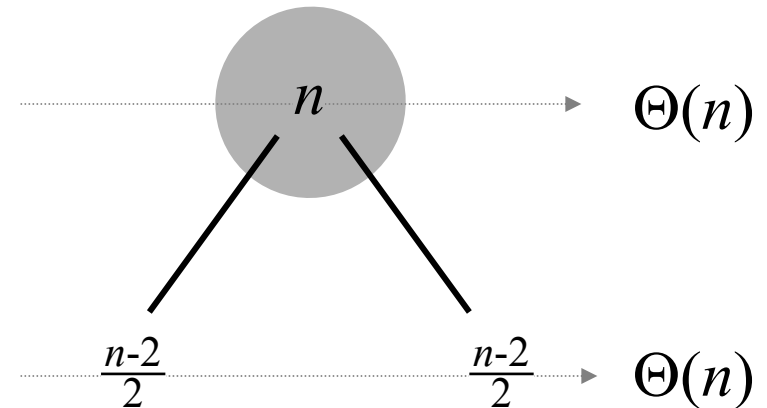
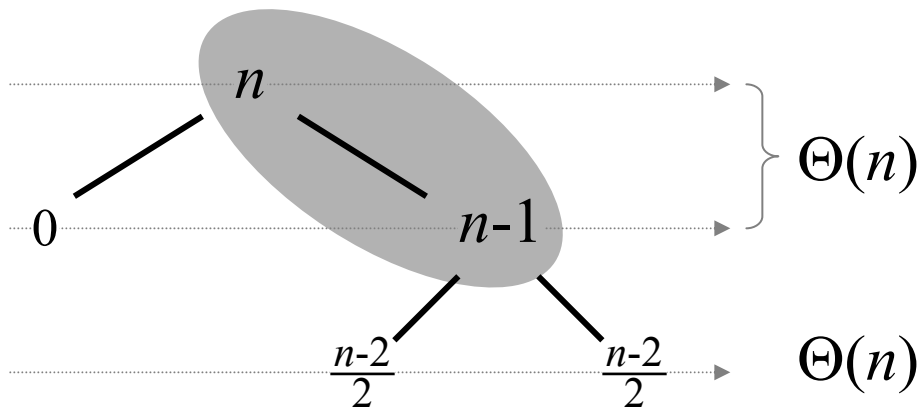


- Ciò fa presumere che il costo nel caso medio sia molto vicino al caso migliore

$$O(n \lg n)$$

Alternanza di ricorsioni bilanciate e sbilanciate

- Supponiamo che nel 20% dei casi PARTITION produca una partizione meno bilanciata di 9-a-1
- Supponiamo che nell'albero delle chiamate ricorsive una ripartizione sbilanciata sia sempre seguita da una bilanciata
- Il costo di una ripartizione sbilanciata può essere assorbito dal costo della ripartizione bilanciata



Versione randomizzata di QUICK_SORT

- E' possibile modificare QUICK_SORT in maniera che i casi peggiori non coincidano con disposizioni notevoli degli elementi

```
RANDOMIZED_PARTITION(A, p, r)
```

```
1. i = RANDOM(p, r)
```

```
2. SCAMBIA(A, r, i)
```

```
3. return PARTITION(A, p, r)
```

```
RANDOMIZED_QUICK_SORT(A, p, r)
```

```
1. if p < r then
```

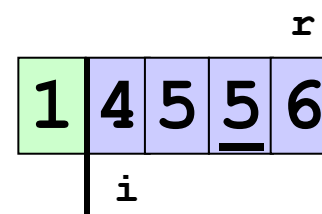
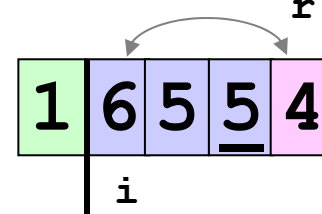
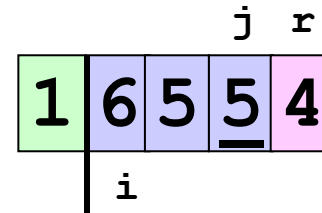
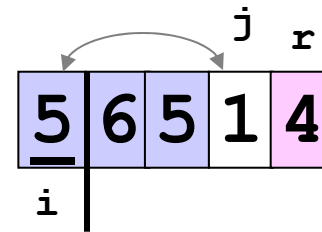
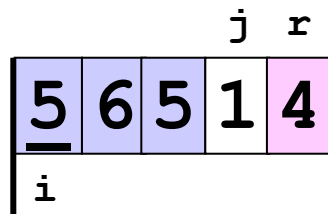
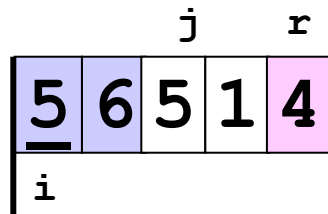
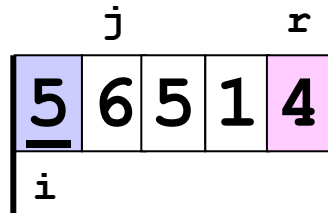
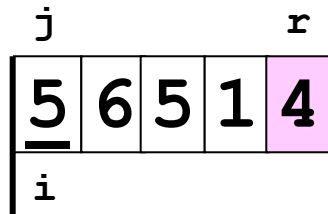
```
2.     q = RANDOMIZED_PARTITION(A, p, r)
```

```
3.     RANDOMIZED_QUICK_SORT(A, p, q-1)
```

```
4.     RANDOMIZED_QUICK_SORT(A, q+1, r)
```

Stabilità di QUICK_SORT

- QUICK_SORT non è stabile:



Algoritmi di ordinamento per confronto

	caso migliore	caso medio	caso peggiore	in loco	stabile
SELECTION-SORT	$\Theta(n^2)$			<i>si</i>	<i>si</i>
INSERTION-SORT	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	<i>si</i>	<i>si</i>
MERGE-SORT	$\Theta(n \log n)$			<i>no</i>	<i>si</i>
HEAP-SORT	$\Theta(n \log n)$			<i>si</i>	<i>no</i>
QUICK-SORT	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	<i>si</i>	<i>no</i>

Algoritmi e Strutture di Dati

Alberi binari di ricerca

m.patrignani

Nota di copyright

- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Contenuto

- Tipo astratto di dato array associativo
- Alberi binari di ricerca (abr)
 - loro uso per la realizzazione di tipi astratti di dato
 - consultazione di un abr
 - ricerca di un valore
 - calcolo del valore massimo o minimo
 - creazione e manutenzione di un abr
 - inserimento e cancellazione di nodi

Array associativi

- Sappiamo già cos'è un array
 - una sequenza di variabili omogenee accedute tramite un indice intero in un intervallo specificato
 - esempio: A è un array di 10 caratteri da A[0] ad A[9]
- Un *array associativo* (o *mappa*, o *dizionario*) è un insieme di variabili omogenee accedute tramite chiavi (omogenee ma di tipo qualsiasi)
 - la chiave può essere un intero, una stringa, un oggetto, ecc
 - si assume che la chiave sia unica
- Un array associativo è dunque costituito da un insieme di coppie $\langle \text{chiave}, \text{valore} \rangle$

Esempi di array associativi

- Dizionario della lingua italiana
 - la chiave è un lemma
 - il valore è un testo che descrive la sua classificazione, i suoi significati, alcuni esempi d'uso, ecc.
- Dati fiscali dei contribuenti
 - la chiave è il codice fiscale
 - il valore è composto dai dati anagrafici, contributivi, ecc
- Dati satellite associati ad oggetti software
 - la chiave è un oggetto
 - il valore è un insieme di dati specifici associati all'oggetto
- Voti degli studenti del corso di ASD
 - la chiave è un numero di matricola
 - il valore è un voto in trentesimi

Il tipo astratto di dato array associativo

- Domini

- il dominio di interesse è l'insieme A degli array associativi
- dominio di supporto: le chiavi K dell'array associativo
- dominio di supporto: i valori V dell'array associativo
 - comprensivo della costante “valore nullo”
- dominio di supporto: i booleani $B = \{\text{true}, \text{false}\}$

- Costanti

- l'array associativo vuoto

- Operazioni

- aggiunge una coppia $\langle \text{chiave}, \text{valore} \rangle$: $\text{PUT: } A \times K \times V \rightarrow A$
- restituisce il valore associato ad una chiave: $\text{GET: } A \times K \rightarrow V$
 - può restituire il valore nullo se nessun elemento è associato alla chiave
- rimuove la coppia $\langle \text{chiave}, \text{valore} \rangle$: $\text{DELETE: } A \times K \rightarrow A$
- verifica che un a chiave sia utilizzata: $\text{EXISTS: } A \times K \rightarrow B$
- ...

Realizzazioni di array associativi

- Un array associativo potrebbe essere realizzato tramite...
 - un array di coppie chiave-valore non ordinato
 - un array di coppie chiave-valore ordinato in base alle chiavi
 - una lista di coppie chiave-valore non ordinata
 - una lista di coppie chiave-valore ordinata in base alle chiavi
- Nessuna di queste realizzazioni garantisce tempi logaritmici sia per l'inserimento che per la ricerca

Realizzazione tramite array ordinato

- In un array di coppie chiave-valore ordinato in base alle chiavi
 - l'inserimento ha un costo lineare in quanto occorre traslare tutti gli elementi dell'array successivi alla posizione corrente
 - la ricerca ha un costo logaritmico tramite una strategia di ricerca binaria
- Strategia di ricerca binaria:
 - salto a metà dell'intervallo
 - se ho trovato il valore cercato la procedura termina
 - se trovo un valore maggiore ricorro nel primo sottointervallo
 - altrimenti ricorro nel secondo sottointervallo

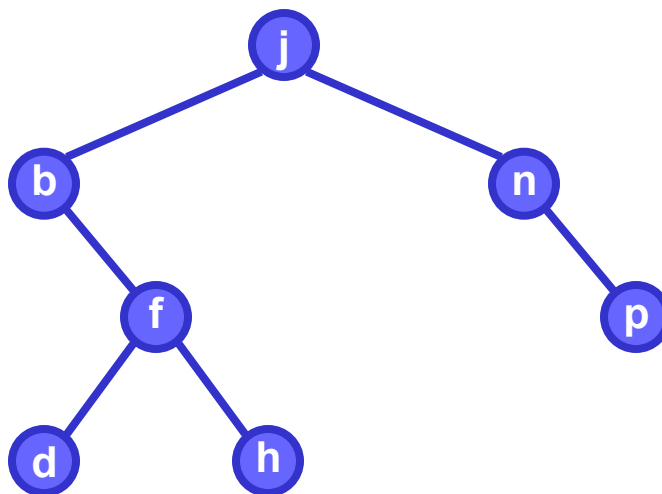
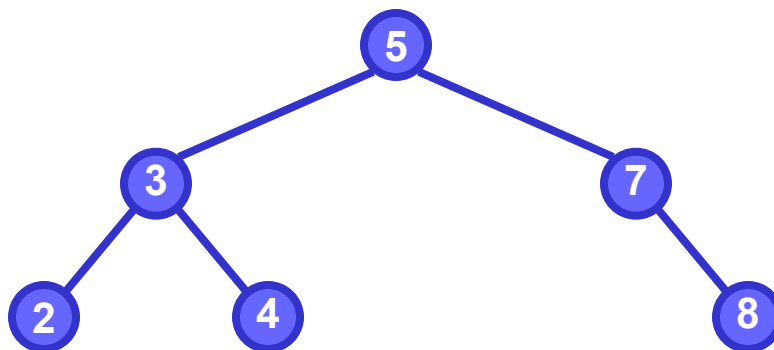
Alberi binari di ricerca

- Detti anche abr o bst (binary search trees)
- Sono strutture di dati utilizzate per implementare dizionari, cioè coppie $\langle \text{chiave}, \text{valore} \rangle$ nel caso in cui sulle chiavi, che si suppongono uniche, sia definita una relazione d'ordine totale (o “lineare”)
 - sono disponibili due funzioni $\text{MINORE}(x,y)$ e $\text{UGUALE}(x,y)$ che calcolano la posizione reciproca di due chiavi
 - per esempio: le chiavi sono interi e la relazione d'ordine è “ $<$ ”

Alberi binari di ricerca

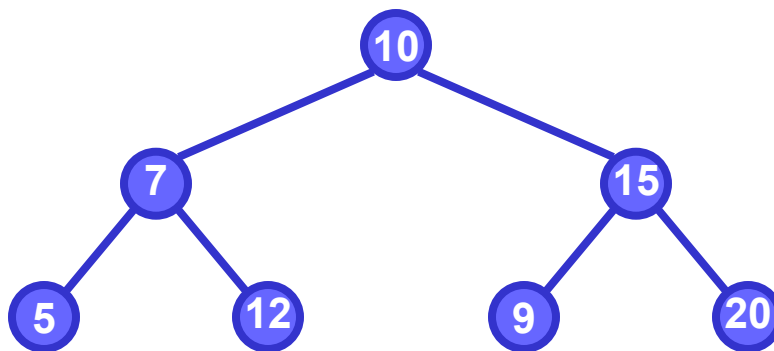
- Gli alberi sono realizzati con alberi binari radicati
 - ogni nodo dell'albero rappresenta un elemento e ha i campi
 - `parent`: riferimento al nodo genitore
 - `left`: riferimento al figlio sinistro
 - `right`: riferimento al figlio destro
 - `key`: valore della chiave
 - nel seguito supporremo che sia un intero
 - `value`: il valore associato alla chiave
 - nel seguito ignoreremo questo valore
- Per ogni nodo x dell'albero
 - tutti i nodi del sottoalbero sinistro di x hanno chiave minore di quella di x
 - tutti i nodi del sottoalbero destro di x hanno chiave maggiore di quella di x

Esempi di abr



Esercizi sugli alberi binari di ricerca

- Disegna abr di altezza 2, 3, 4, 5, 6 sull'insieme di chiavi $\{1, 4, 5, 10, 16, 17, 12\}$
- Questo albero è un abr?

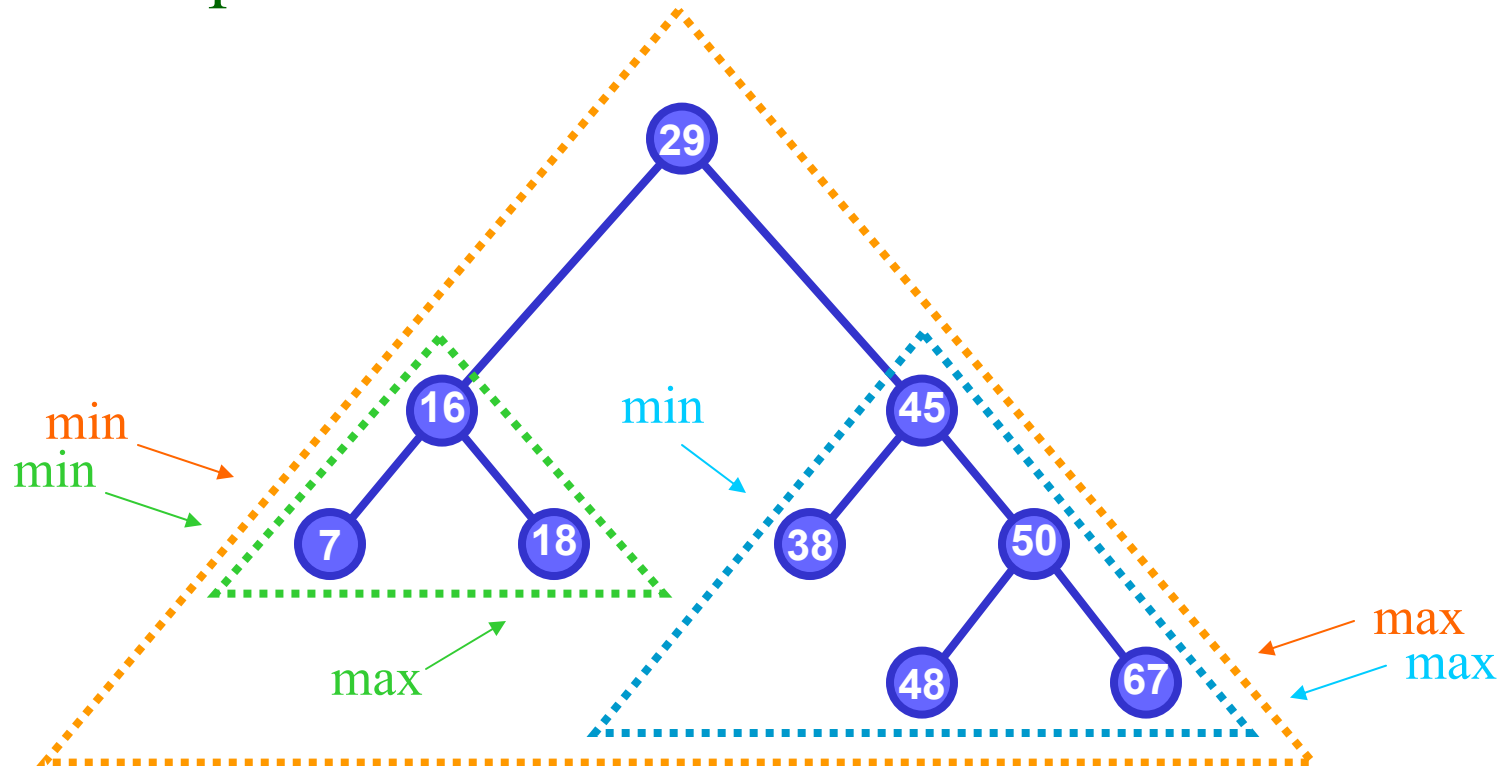


Operazioni sugli abr

- Gli abr supportano operazioni di
 - consultazione
 - calcolo del minimo valore contenuto
 - calcolo del massimo valore contenuto
 - ricerca del nodo (se esiste) contenente un particolare valore
 - modifica
 - inserimento di un elemento nell'inseme
 - cancellazione di un elemento dall'inseme
 - verifica di consistenza
 - verifica che un albero sia effettivamente un abr

Calcolo del minimo e del massimo

- In un abr
 - il ramo più a sinistra termina con il valore minimo
 - il ramo più a destra termina con il valore massimo



TREE_MINIMUM e TREE_MAXIMUM

- Queste funzioni ritornano il riferimento al nodo che contiene il minimo e il massimo valore dell'abr

```
TREE_MINIMUM(x)      /* minimo del sottoalbero radicato ad x */  
1.  while (x.left != NULL)  
2.      x = x.left      /* scendo a sinistra finché posso */  
3.  return x
```

```
TREE_MAXIMUM(x)      /* massimo del sottoalbero radicato ad x */  
1.  while (x.right != NULL)  
2.      x = x.right      /* scendo a destra finché posso */  
3.  return x
```

- Osservazione banale (che sarà utile in seguito)
 - il nodo minimo non ha figlio sinistro
 - il nodo massimo non ha figlio destro

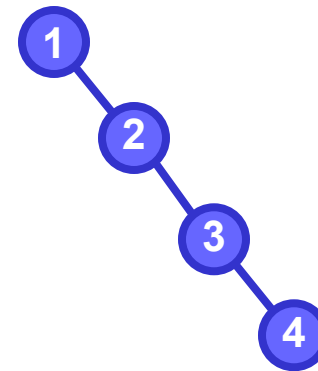
Ricerca del nodo con chiave k

```
ITERATIVE_TREE_SEARCH(x,k) /* suppongo x radice dell'abr */  
1. while (x != NULL) and k != x.key  
2.     if (k < x.key)  
3.         x = x.left  
4.     else  
5.         x = x.right  
6. return x      /* ritorna il riferim. al nodo */
```

```
RECURSIVE_TREE_SEARCH(x,k) /* suppongo x radice dell'abr */  
1. if (x == NULL) or (k == x.key)  
2.     return x /* ritorna il riferim. al nodo */  
3. if (k < x.key)  
4.     return RECURSIVE_TREE_SEARCH(x.left,k)  
5. else  
6.     return RECURSIVE_TREE_SEARCH(x.right,k)
```

Complessità delle operazioni viste

- Gli algoritmi per il calcolo del minimo e del massimo e per la ricerca di una chiave hanno tutti una complessità asintotica nel caso peggiore $\Theta(h)$, dove h è l'altezza dell'abr
 - poiché esistono abr la cui altezza è lineare nel numero dei nodi n , la complessità nel caso peggiore è $\Theta(n)$

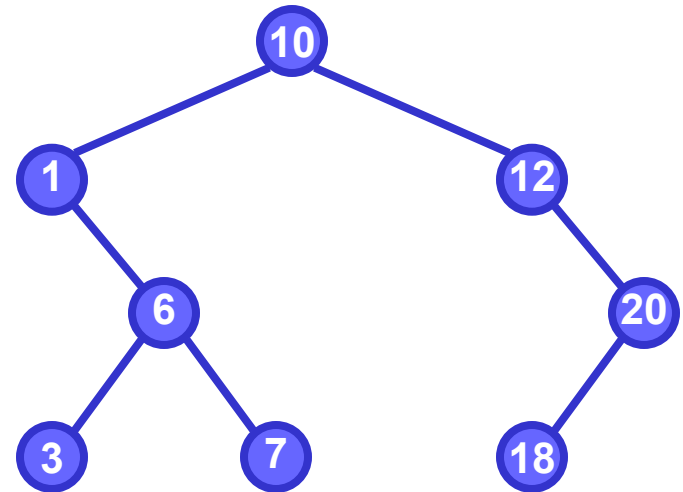
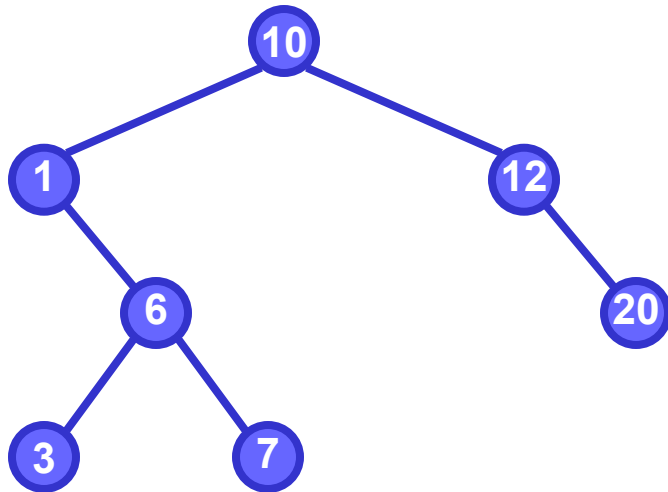


Inserimento di un nuovo elemento nell'abr

- Se l'abr è vuoto
 - crea un nodo che diventa radice dell'abr
- Altrimenti
 - se l'oggetto da inserire è uguale al nodo corrente
 - rifiuta l'inserimento (oppure sovrascrivi, a seconda dell'applicazione)
 - altrimenti
 - se l'oggetto da inserire è $<$ del nodo corrente
 - inserisci nel sottoalbero sinistro
 - altrimenti
 - inserisci nel sottoalbero destro
- Il nuovo nodo sarà sempre inserito come foglia
 - quindi con i due sottoalberi vuoti

Inserimento di un nodo in un abr

- Inserimento del nodo “18”



Inserimento di un nodo in un abr

- Suppongo di avere a disposizione una funzione per la creazione di nuovi nodi

```
NEW_TREE_ELEM(k)      /* costruisco un nodo con chiave k */  
1.  /* x è un oggetto con campi p, left, right, e key */  
2.  x.p = x.left = x.right = NULL      /* genitore e figli */  
3.  x.key = k  
4.  return x
```

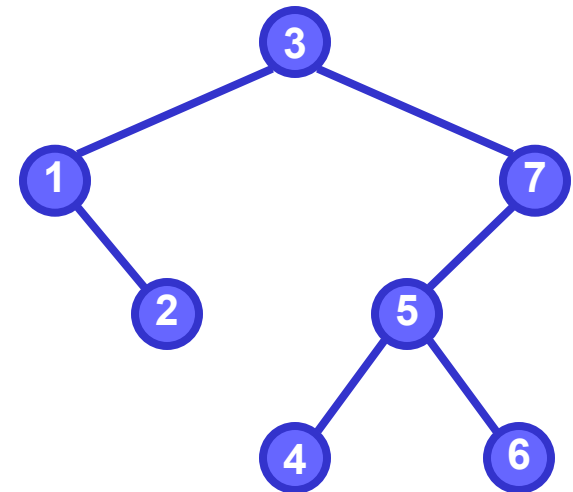
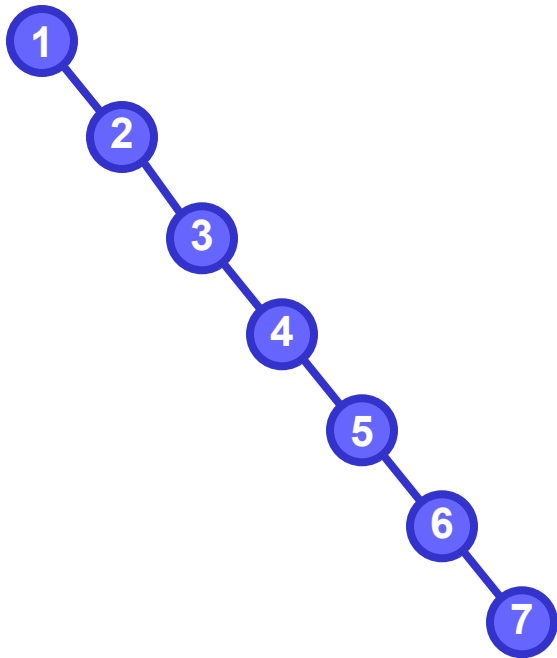
- Il caso in cui l'abr è vuoto viene trattato separatamente

```
TREE_INSERT(t, k)  
1.  if (t.root == NULL)  
2.      t.root = NEW_TREE_ELEM(k)  
3.      return TRUE      /* nodo inserito correttamente */  
4.  else return BST_INSERT(t.root, k)
```

```
BST_INSERT(x,k)    /* ins. k nell'abr radicato a x != NULL */  
1.  output = FALSE  /* nodo non inserito */  
2.  if (MINORE(k,x.key))    /* devo inserire k a sinistra */  
3.      if (x.left == NULL)  
4.          x.left = NEW_TREE_ELEM(k)  
5.          x.left.p = x  
6.          output = TRUE    /* nodo inserito */  
6.      else  
7.          output = BST_INSERT(x.left,k)  
8.  else if (MINORE(x.key,k)) /* devo inserire k a destra */  
9.      if (x.right == NULL)  
10.         x.right = NEW_TREE_ELEM(k)  
11.         x.right.p = x  
12.         output = TRUE    /* nodo inserito */  
12.      else  
13.         output = BST_INSERT(x.right,k)  
14.  return output /* è FALSE se il nodo era già presente */
```

La forma dell'albero dipende dall'ordine di inserimento

Inserimento di 1, 2, 3, 4, 5, 6, 7



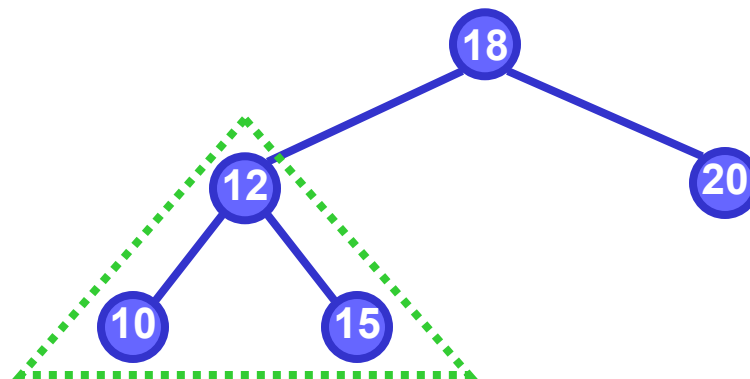
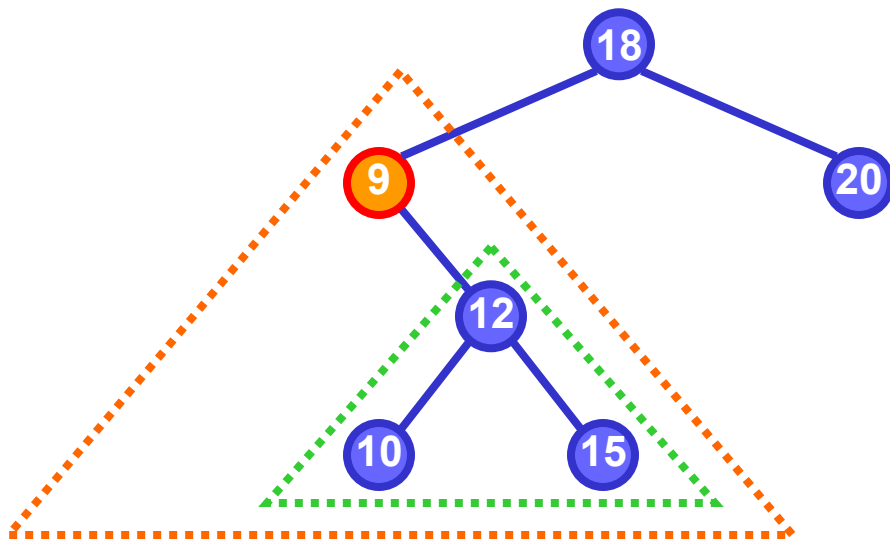
Inserimento di 3, 1, 7, 2, 5, 4, 6

Strategia per la cancellazione di un nodo x

- Se x non ha figli lo posso sempre rimuovere
- Se il nodo x ha un figlio
 - elimino x collegando il genitore con il figlio
- Se il nodo x ha due figli
 - cerco il nodo y , successore di x , nel sottoalbero destro
 - y ha al massimo un figlio
 - rimuovo y
 - sostituisco y ad x
- Sarebbe stato analogo cercare il nodo predecessore di x nel sottoalbero sinistro

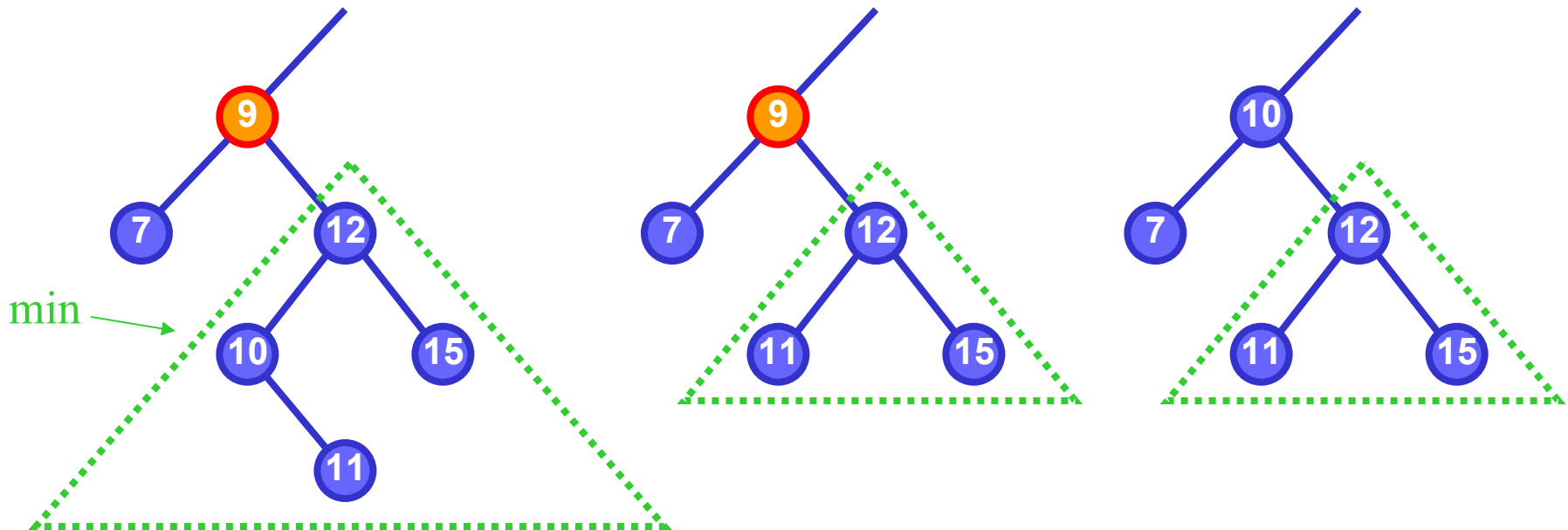
Rimozione di un nodo con un solo figlio

- devo rimuovere il nodo 9 (che ha un solo figlio)
- Il sottoalbero radicato al nodo 12:
 - è contenuto nel sottoalbero radicato al nodo 9
 - contiene tutti valori minori o uguali a 18
 - dunque può essere il sottoalbero sinistro del nodo 18



Rimozione di un nodo con due figli

- Devo rimuovere il nodo 9 che ha due figli
- Il nodo “successore” del nodo 9 è il nodo 10 (minimo del sottoalbero destro del nodo 9)
- Il nodo 10 è rimuovibile con la strategia precedente
- Sostituendo il nodo 10 al nodo 9 si ottiene un abr



Funzione TREE_BYPASS

```
TREE_BYPASS(t,x)      /* x ha al massimo un figlio */
1.  if (x.left != NULL)
2.      figlio = x.left
3.  else
4.      figlio = x.right      /* NULL se x non ha figli */
5.  if (figlio != NULL)
6.      figlio.p = x.p
7.  if (x.p != NULL) /* c'è il parent di x da aggiornare */
8.      if (x == x.p.left) /* x era il figlio sinistro */
9.          x.p.left = figlio
10.     else                /* x era il figlio destro */
11.         x.p.right = figlio
12. else t.root = figlio
```

- La complessità è $\Theta(1)$

Funzione TREE-DELETE

```
TREE_DELETE(t, x)      /* x qualsiasi */  
1.  if (x.left != NULL) and (x.right != NULL)  
2.      y = TREE_MINIMUM(x.right)  
3.      x.key = y.key  
4.  else  
5.      y = x  
6.  TREE_BYPASS(t, y)
```

- **TREE_DELETE** ha complessità $\Theta(h)$
 - la funzione **TREE_MINIMUM** ha complessità $\Theta(h)$

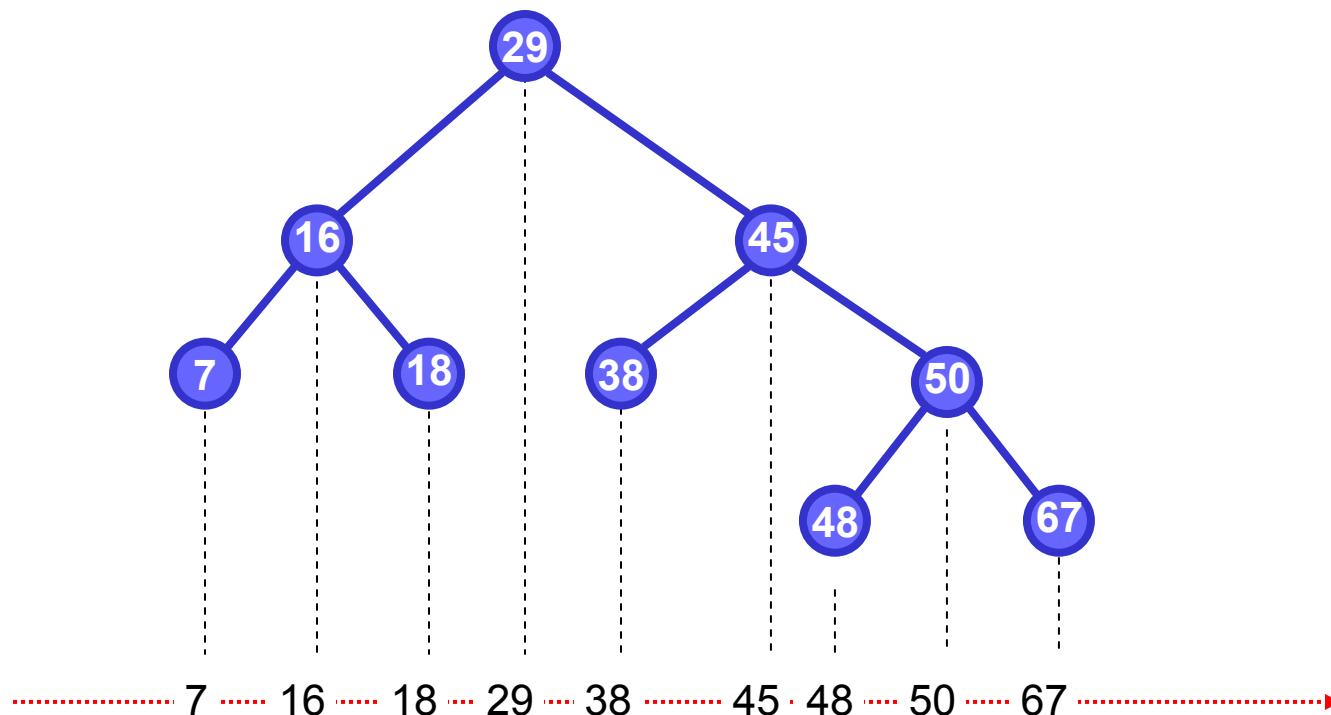
Complessità delle operazioni sugli abr

- Tutti gli algoritmi visti hanno una complessità asintotica nel caso peggiore $\Theta(h)$, dove h è l'altezza dell'abr
 - l'altezza dell'abr dipende dall'ordine di inserimento
 - se l'albero è sbilanciato (cosa che dobbiamo assumere nel caso peggiore) $h \in \Theta(n)$
 - se potessimo restringerci ad alberi bilanciati $h \in \Theta(\log n)$ ed il costo delle operazioni nel caso peggiore sarebbe $\Theta(\log n)$
 - sono note delle strategie (alberi rosso-neri) per mantenere bilanciati gli abr
- E' stato dimostrato che nel caso medio un abr ha altezza logaritmica
 - quindi nel caso medio il costo delle operazioni è $\Theta(\log n)$
 - noi non lo dimostriamo

Verifica che un albero sia un abr

- Osservazione

- se l'albero è un abr una visita simmetrica produce valori in ordine crescente



Visita simmetrica ABR_SYM

- La strategia è la seguente
 - compongo un array con tutti gli elementi dell'abr nell'ordine in cui sono processati da una visita simmetrica
 - verifico che l'array sia non decrescente

ABR_SYM(x)

1. **n = CONTA_NODI(x)**

2. /* creo l'array A con n posizioni */

3. **TREE_TO_ARRAY(A, x, 0)** /* "riverso" l'albero in A */

4. **return IS_SORTED(A)**

- La complessità asintotica è $\Theta(n)$ in quanto ogni singola fase ha costo $\Theta(n)$

Funzioni CONTA_NODI e IS_SORTED

CONTA_NODI (x)

```
1. if (x == NULL) return 0
2. l = CONTA_NODI (x.left)
3. r = CONTA_NODI (x.right)
4. return 1 + l + r
```

IS_SORTED (A)

```
1. for i = 0 to A.length-2
2.     if A[i] > A[i+1]
3.         return FALSE
4. return TRUE
```

- La complessità asintotica è $\Theta(n)$ per entrambe

Visita simmetrica TREE_TO_ARRAY

```
TREE_TO_ARRAY(A,x,i) /* uso A a partire dalla posizione i */  
1. if (x == NULL) return i  
2. i = TREE_TO_ARRAY(A,x.left,i)  
3. A[i] = x.key  
4. i = TREE_TO_ARRAY(A,x.right,i+1)  
5. return i // i è la prossima posizione libera dell'array
```

- La complessità asintotica è $\Theta(n)$
- Questa funzione può essere usata per creare un algoritmo di ordinamento chiamato TREE_SORT
 - costruisco un abr da un array di input
 - lancio TREE_TO_ARRAY sull'abr ottenuto
 - l'array ottenuto è ordinato

Domande

- In un abr bilanciato con n nodi quanto costa:
 - l'inserimento di un nodo?
 - la cancellazione di un nodo?
 - la ricerca di un nodo?
- Quanto costano le stesse operazioni se l'abr è fortemente sbilanciato?

Esercizi

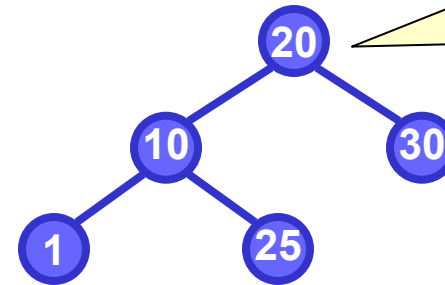
1. Scrivi lo pseudocodice della procedura $IS_ABR_PRE(t)$ che verifica se un albero binario t di interi sia un albero binario di ricerca con una visita in preordine
 - qual è la sua complessità nel caso peggiore?
2. Scrivi lo pseudocodice della procedura $IS_ABR_POST(t)$ che verifica se un albero binario t di interi sia un albero binario di ricerca con una visita in postordine
 - qual è la sua complessità nel caso peggiore?

Esercizi

3. Scrivi lo pseudocodice della procedura $\text{TREE_SORT}(A)$ che ordina un array A di interi utilizzando un albero binario di ricerca
- supponi di avere a disposizione le funzioni
 - $\text{INSERISCI}(t, k)$ che in tempo lineare nell'altezza dell'albero t inserisce un intero k nell'albero
 - $\text{TREE_TO_ARRAY}(A, t, i)$ che con una visita simmetrica riversa in tempo lineare l'albero t nell'array A a partire dalla posizione i
 - qual è la complessità di $\text{TREE_SORT}(A)$ nel caso peggiore?
 - qual è la complessità di $\text{TREE_SORT}(A)$ nel caso migliore?

Soluzione esercizio 1

- Si noti che il codice seguente è errato



io non
sono un
abr!

```
IS_ABR_PRE(t)          /* t è un albero */
```

```
1. return IS_ABR_ERRATO(t.root)
```

```
IS_ABR_ERRATO(x)       /* x è un nodo dell'albero */
```

```
1. if x == NULL
```

```
2.     return TRUE
```

```
3. else return ( (x.key >= x.left.key ) and
```

```
4.         (x.key <= x.right.key ) and
```

```
5.         IS_ABR_ERRATO(x.left) and
```

```
6.         IS_ABR_ERRATO(x.right) )
```

Soluzione esercizio 1

- Soluzione con una visita in preordine

```
IS_ABR_PRE(t)          /* t è un albero */  
1.  return ABR_PRE_RIC(t.root)
```

```
ABR_PRE_RIC(x)          /* x è un nodo dell'albero */  
1.  if x == NULL  
2.      return TRUE  
3.  else return ( NO_MAGGIORE(x.left,x.key) and  
4.                NO_MINORE(x.right,x.key) and  
5.                ABR_PRE_RIC(x.left) and  
6.                ABR_PRE_RIC(x.right) )
```

Soluzione esercizio 1

- Dove `NO_MAGGIORE` e `NO_MINORE` sono

```
NO_MAGGIORE(x,v)      /* x è un nodo, v è un intero */  
1. if x == NULL  
2.         return TRUE  
3. else   return ( (x.key <= v) and  
4.                NO_MAGGIORE(x.left,v) and  
5.                NO_MAGGIORE(x.right,v) )
```

```
NO_MINORE(x,v)      /* x è un nodo, v è un intero */  
1. if x == NULL  
2.         return TRUE  
3. else   return ( (x.key >= v) and  
4.                NO_MINORE(x.left,v) and  
5.                NO_MINORE(x.right,v) )
```

Soluzione esercizio 1

- Nel caso peggiore l'albero è un albero completamente sbilanciato

$$T(n) = T(n-1) + \Theta(n)$$

- Questa equazione di ricorrenza ha la forma

$$T(n) = T(n-1) + g(n)$$

- Che ammette soluzione

$$T(n) = c + \sum_{k=1}^n g(k)$$

- Che nel caso in esame produce

$$T(n) = \Theta(n^2)$$

Soluzione esercizio 2

- Eseguo le operazioni in postordine (prima i figli)
- Quando considero un nodo x ho già verificato che i sottoalberi destro e sinistro siano abr
- Avendo già percorso i sottoalberi posso essermi contestualmente calcolato il valore minimo e massimo in essi contenuto
- La funzione `ABR_POST` ritorna un oggetto con tre valori
 - `is_abr`: booleano che mi dice se il sottoalbero è un abr
 - `min`: il minimo valore contenuto nell'abr
 - `max`: il massimo valore contenuto nell'abr

Soluzione esercizio 2

```
ABR_POST_RIC(x) /* ritorna un oggetto (is_abr, min, max) */  
1.  if (x == NULL) return NULL  
2.  l = ABR_POST_RIC(x.left) // l ha campi is_abr, min, max  
3.  r = ABR_POST_RIC(x.right) // idem  
4.  if (l == NULL and r == NULL) /* x è una foglia */  
5.      return (TRUE, x.key, x.key)  
6.  if (l == NULL and r != NULL) // x ha il figlio destro  
7.      out = r.is_abr and (x.key < r.min)  
8.      return (out, x.key, r.max)  
9.  if (l != NULL and r == NULL) // x ha il figlio sinistro  
10.     out = l.is_abr and (x.key > l.max)  
11.     return (out, l.min, x.key)  
12. out = l.is_abr and r.is_abr // x ha entrambi i figli  
13. out = out and (x.key < r.min) and (x.key > l.max)  
14. return (out, l.min, r.max)
```

Soluzione esercizio 2

- Nel caso della visita in postordine tutti i test (linea 1 e dalla linea 4 alla linea 14) non prevedono chiamate a funzioni
 - la loro complessità è $\Theta(1)$
- Le chiamate ricorsive ad `ABR_POST_RIC` realizzano una visita in postordine
 - la complessità asintotica è $\Theta(n)$ come per tutte le visite in postordine

Soluzione esercizio 3

TREE_SORT (A)	▷ A è un array che deve essere ordinato
1. t.root = NULL	▷ t è un nuovo albero
2. for i = 1 to A.length-1	
3. INSERISCI (t, A[i])	▷ r ha i campi is_abr, min, max
4. TREE_TO_ARRAY (A, t, 0)	

- Complessità della procedura **TREE_SORT**
 - nel caso peggiore, poiché l’inserimento ha complessità lineare, la complessità totale è $\Theta(n^2)$
 - se l’albero fosse bilanciato l’inserimento avverrebbe in tempo $\Theta(\log n)$ e la complessità totale sarebbe $\Theta(n \log n)$ nel caso peggiore

Algoritmi e Strutture di Dati

Alberi rosso-neri

m.patrignani

Contenuto

- Definizione di alberi rosso-neri
- Proprietà degli alberi rosso-neri
- Complessità delle operazioni elementari
- Rotazioni
- Inserimenti e cancellazioni

Motivazioni

- Un dizionario realizzato con un albero binario di ricerca consente operazioni efficienti quando l'albero è bilanciato

alberi binari di ricerca (complessità nel caso peggiore)		
operazione	sbilanciati	bilanciati
ricerca	$\Theta(n)$	$\Theta(\log n)$
inserimento	$\Theta(n)$	$\Theta(\log n)$
cancellazione	$\Theta(n)$	$\Theta(\log n)$

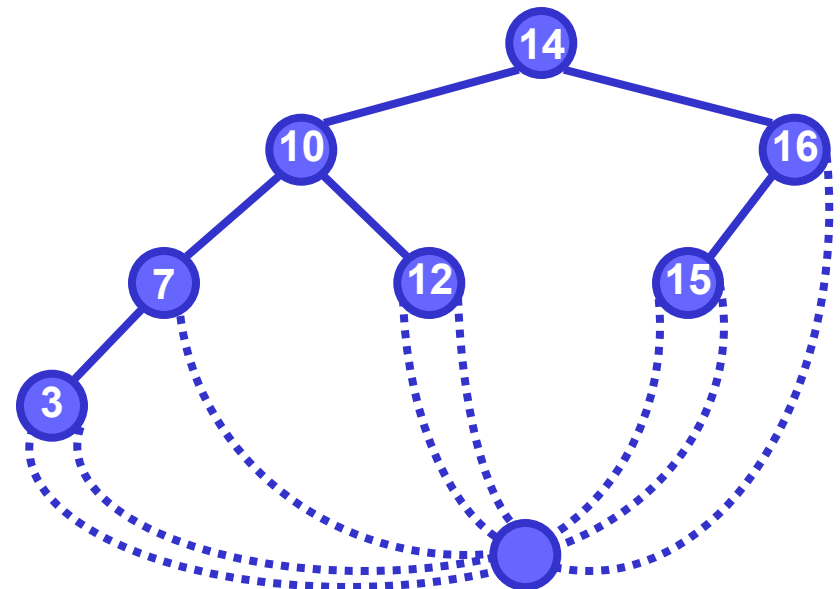
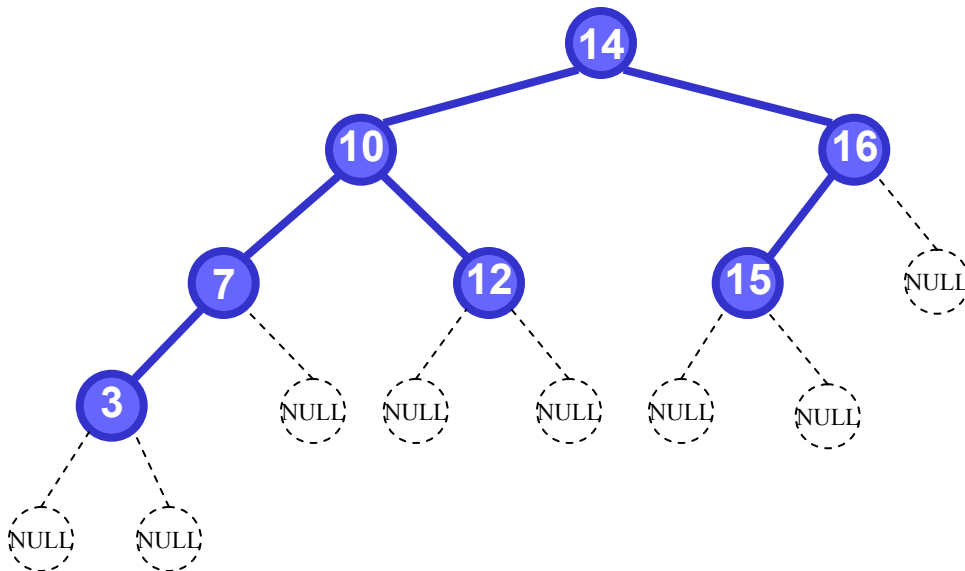
- Ha senso investire delle risorse per mantenere l'albero bilanciato

Storia e alternative

- Gli alberi rosso-neri sono una struttura di dati pensata per mantenere gli alberi binari di ricerca bilanciati
 - furono introdotti da Leonidas J. Guibas and Robert Sedgewick nel 1978
- Strutture dati alternative possono essere
 - AVL-trees
 - introdotti da Adelson-Velsky e Landis nel 1962
 - l'altezza del sottoalbero sinistro e destro differisce al massimo di uno
 - WAVL-trees
 - introdotti da Haeupler, Sen e Tarjan ne 2015
 - combinano caratteristiche degli alberi rosso-neri e degli AVL-trees
 - B-trees
 - introdotti da Bayer and McCreight nel 1970
 - consentono di avere nodi con più di due figli

Albero con sentinelle

- Gestire il bilanciamento di un albero è un obiettivo complesso
- Per semplicità vorremmo che non ci siano nodi con un solo figlio destro o un solo figlio sinistro
 - questo può essere realizzato aggiungendo all'albero t un nodo “sentinella” $t.\text{null}$ e sostituendo con un puntatore a $t.\text{null}$ ogni valore NULL del puntatore $x.\text{left}$ o $x.\text{right}$ di un nodo x



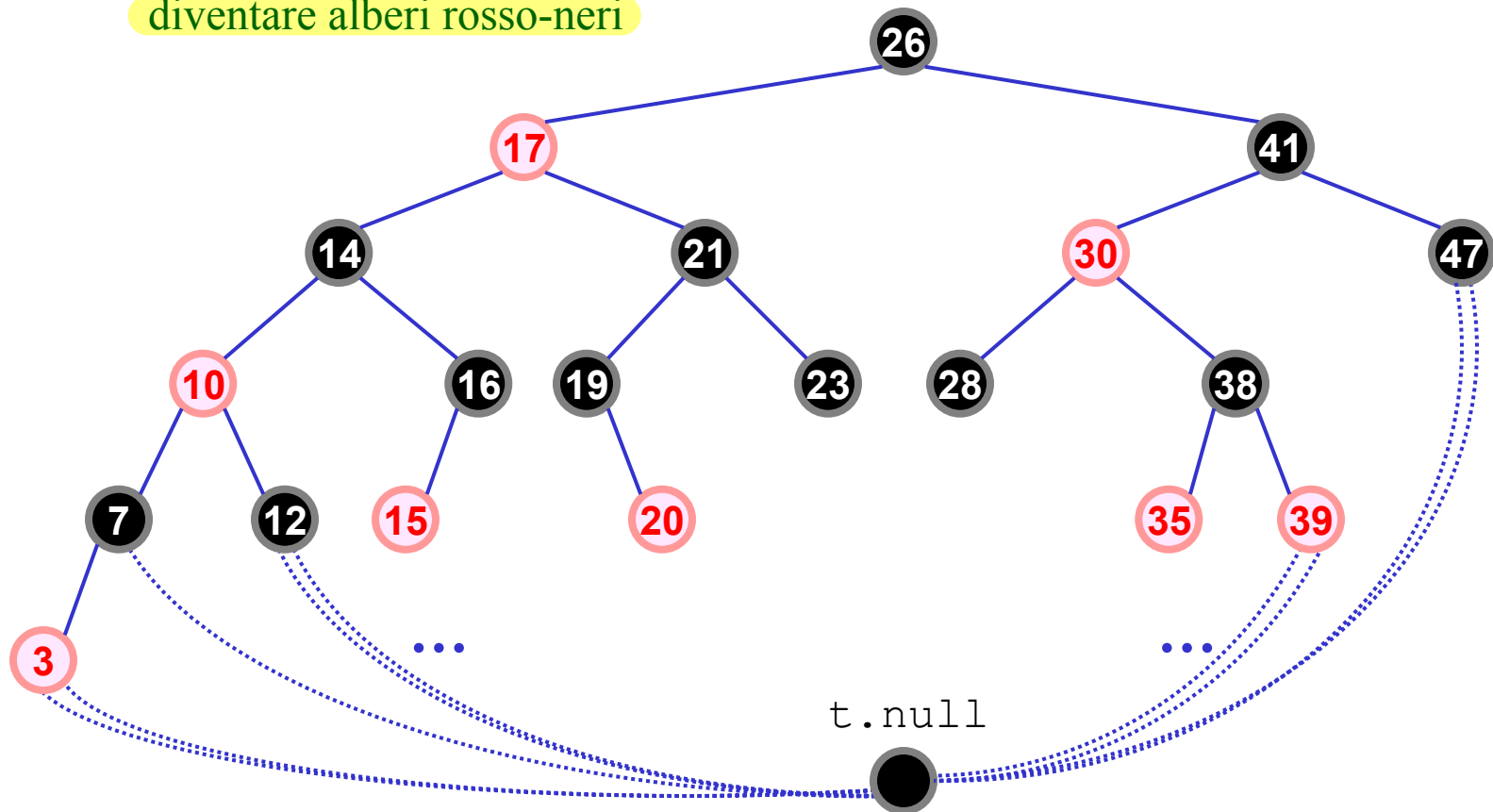
Definizione di alberi rosso-neri

- Un albero rosso-nero è un albero binario di ricerca nel quale
 1. ogni nodo è rosso o nero
 2. la radice `t.root` e la sentinella `t.null` sono nere
 3. se un nodo è rosso entrambi i suoi figli sono neri
 4. tutti i cammini che vanno dalla radice a `t.null` contengono lo stesso numero di nodi neri
- Convenzionalmente chiamiamo “altezza” dell'albero rosso-nero la lunghezza del cammino più lungo tra la radice e `t.null`
 - corrisponde in realtà all'altezza + 1

Esempio di albero rosso-nero

- Attenzione

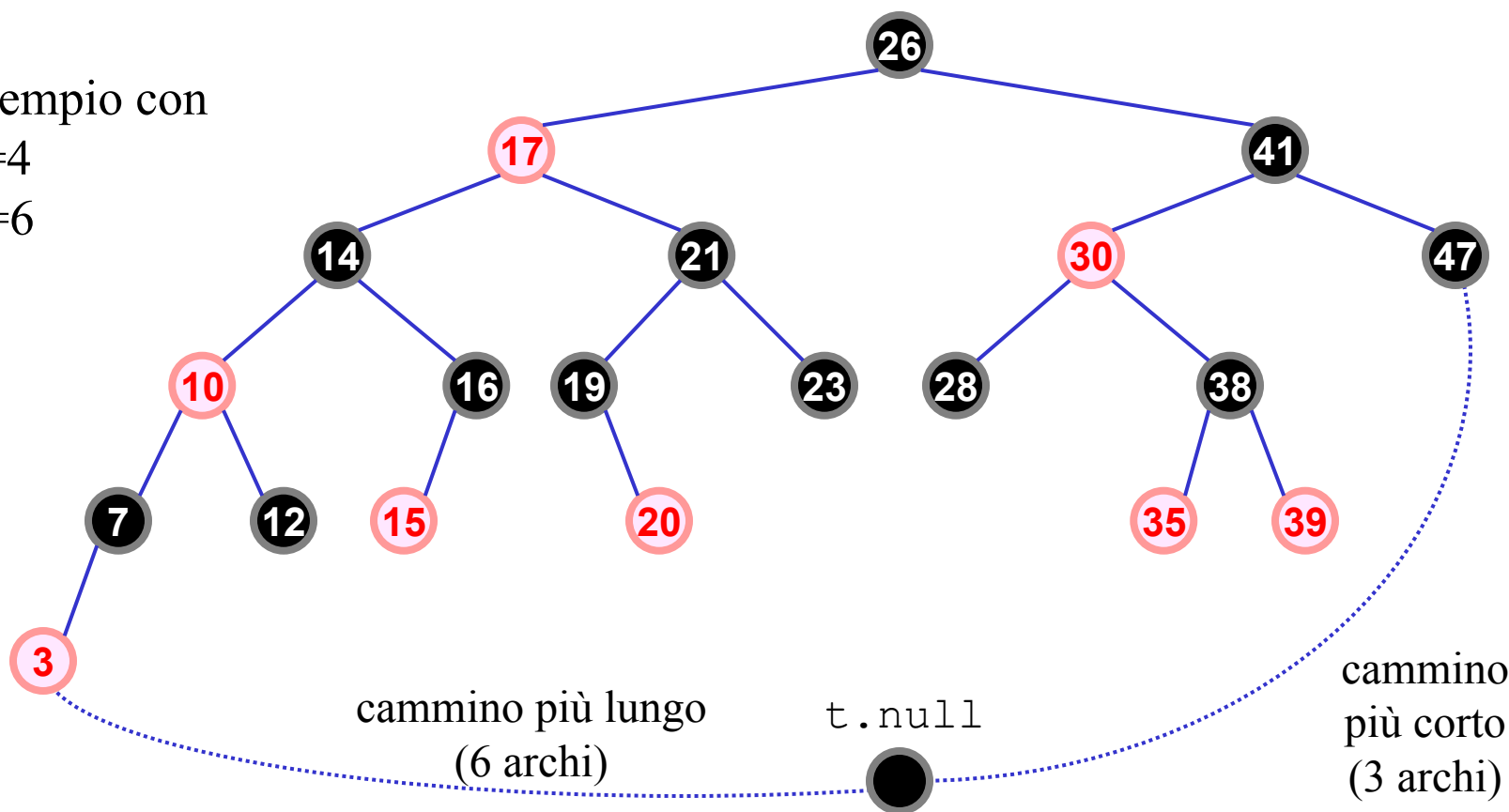
- l'albero deve essere un albero binario di ricerca
- non tutti gli alberi binari di ricerca possono essere colorati in maniera da diventare alberi rosso-neri



Alberi rosso-neri e bilanciamento

- Tutti i cammini dalla radice a `t.null` hanno k nodi neri (nell'esempio $k=4$)
 - ogni cammino ha almeno $k-1$ archi (nell'esempio: 3 archi)
 - il cammino più lungo alterna nodi neri e rossi e ha $2(k-1)$ archi (nell'esempio: 6 archi)

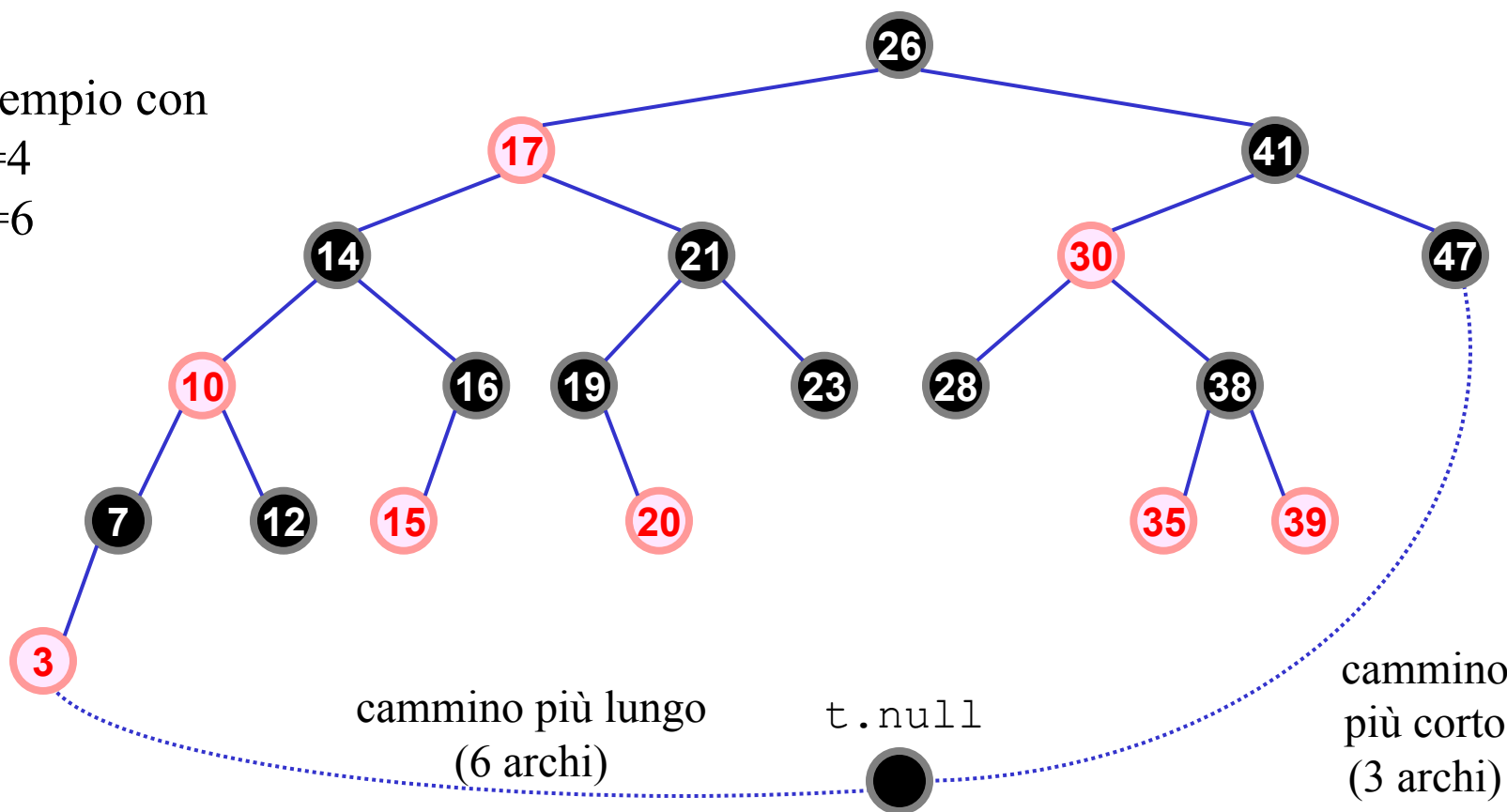
esempio con
 $k=4$
 $h=6$



Alberi rosso-neri e bilanciamento

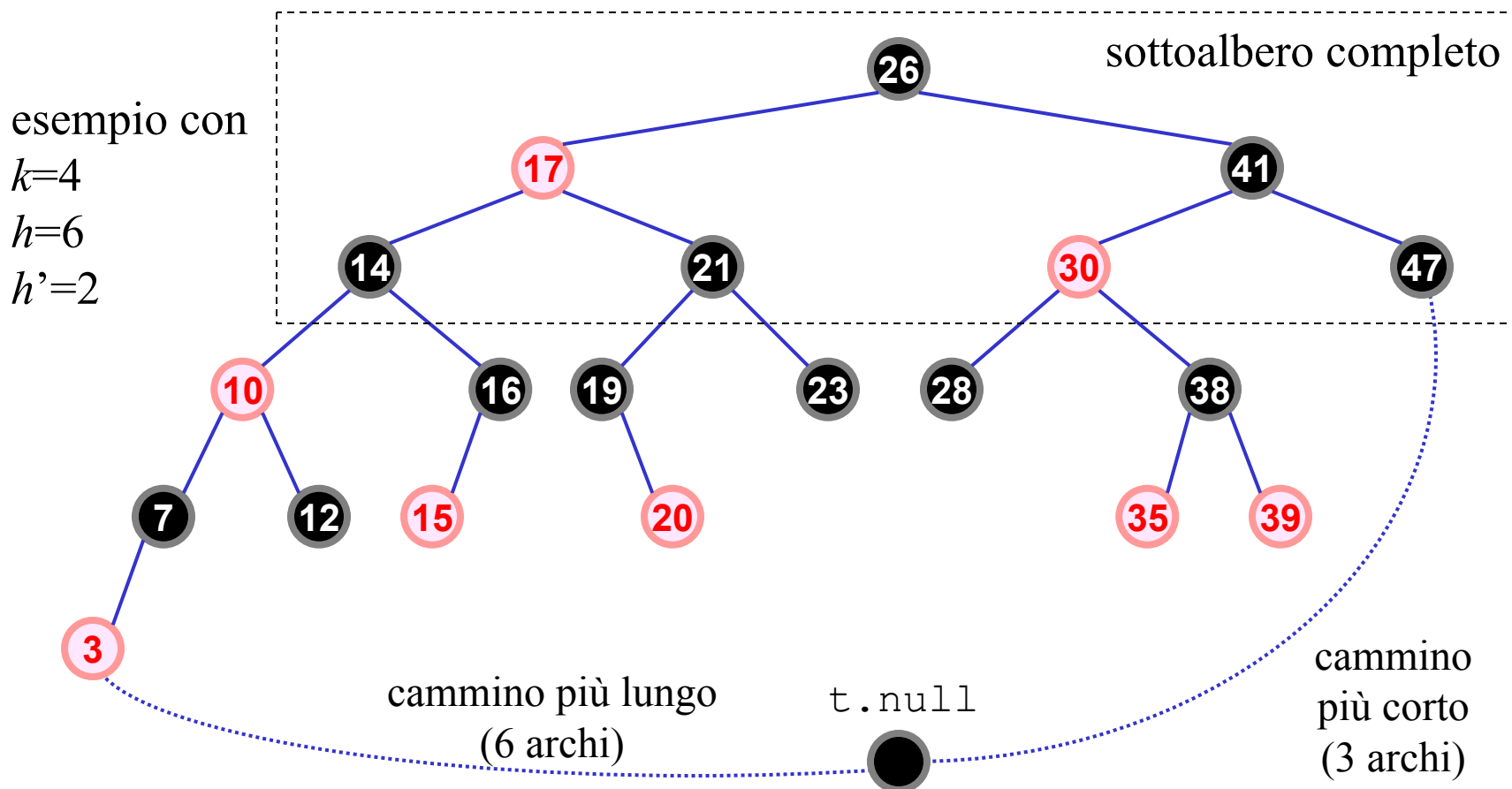
- Tutti i cammini dalla radice a `t.null` hanno k nodi neri (nell'esempio $k=4$)
 - la lunghezza del cammino più lungo ($2(k-1)$) è al massimo due volte la lunghezza del cammino più corto ($k-1$)

esempio con
 $k=4$
 $h=6$



Alberi rosso-neri e profondità

- Tutti i cammini dalla radice a `t.null` hanno k nodi neri (nell'esempio $k=4$)
 - l'albero contiene un sottoalbero completo di profondità $h' = h/2 - 1$



Alberi rosso-neri e numero dei nodi

- Tutti i cammini dalla radice a `t.null` hanno k nodi neri (nell'esempio $k=4$)
 - l'albero ha profondità massima $h = 2(k-1)$
 - l'albero contiene un sottoalbero completo di profondità $h' = h/2 - 1$
- I nodi dell'albero sono almeno quelli del sottoalbero completo
 - ricorda che un albero completo di altezza x ha $2^{x+1}-1$ nodi

$$n \geq 2^{h'+1} - 1 = 2^{\left(\frac{h}{2}-1\right)+1} - 1 = 2^{\frac{h}{2}} - 1$$

$$n + 1 \geq 2^{\frac{h}{2}}$$

$$\frac{h}{2} \leq \log(n + 1)$$

$$h \leq 2 \log(n + 1)$$

- Dunque $h \in O(\log n)$

Alberi rosso-neri e numero dei nodi

- Abbiamo appena dimostrato che in un albero rosso-nero $h \in O(\log n)$
- Sappiamo però che in un albero binario h è almeno l'altezza di un albero completo con n nodi, cioè $h \in \Omega(\log n)$
- Dunque in un albero rosso-nero $h \in \Theta(\log n)$

Operazioni sugli alberi rosso-neri

- L'altezza dell'albero è logaritmica nel numero dei nodi ($h \in \Theta(\log n)$)
- Tutte le operazioni di consultazione eseguibili in tempo $\Theta(h)$ su un albero binario di ricerca sono eseguibili in tempo $\Theta(\log n)$ su un albero rosso-nero:
 - SEARCH
 - MINIMUM
 - MAXIMUM

Operazioni INSERT e DELETE

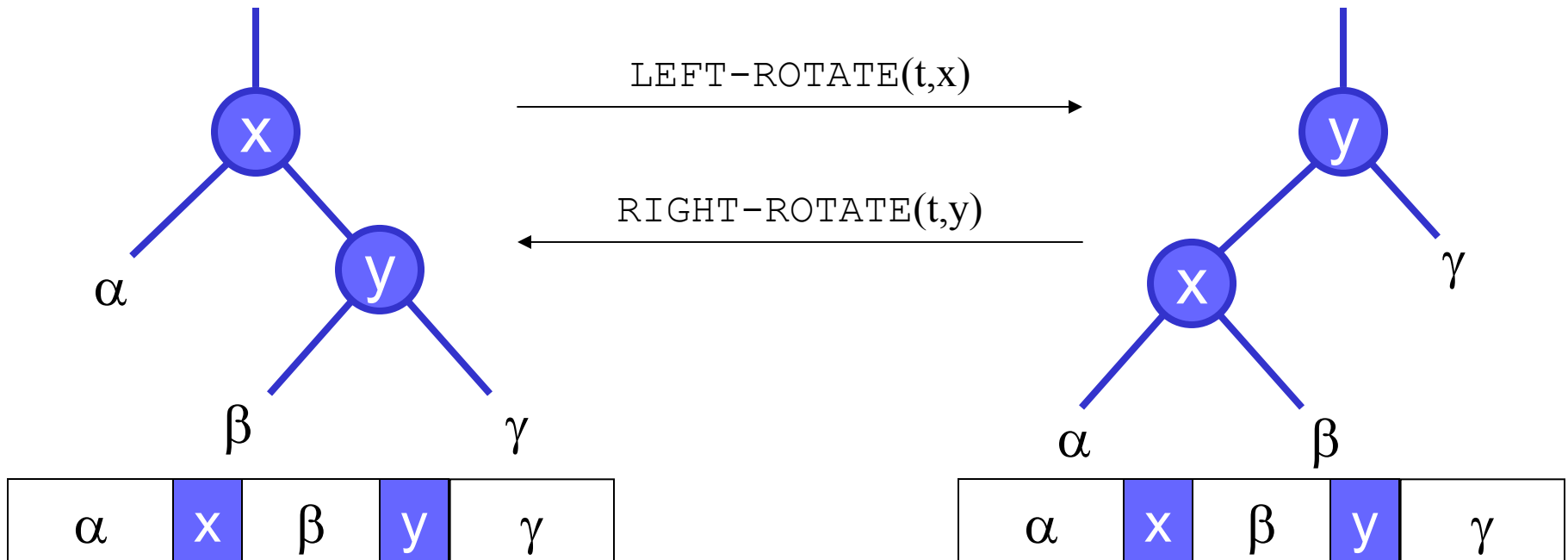
- Le operazioni INSERT e DELETE possono ugualmente essere eseguite in $\Theta(\log n)$
- TREE_INSERT e TREE_DELETE, però, non garantiscono la conservazione delle proprietà degli alberi rosso-neri
 - a valle delle operazioni di inserimento e cancellazione devono essere lanciate delle procedure che ripristinano tali proprietà in $\Theta(\log n)$
- Nel seguito vedremo a titolo di esempio la sola procedura RB_INSERT per l'inserimento di un nodo

Procedura RB_INSERT

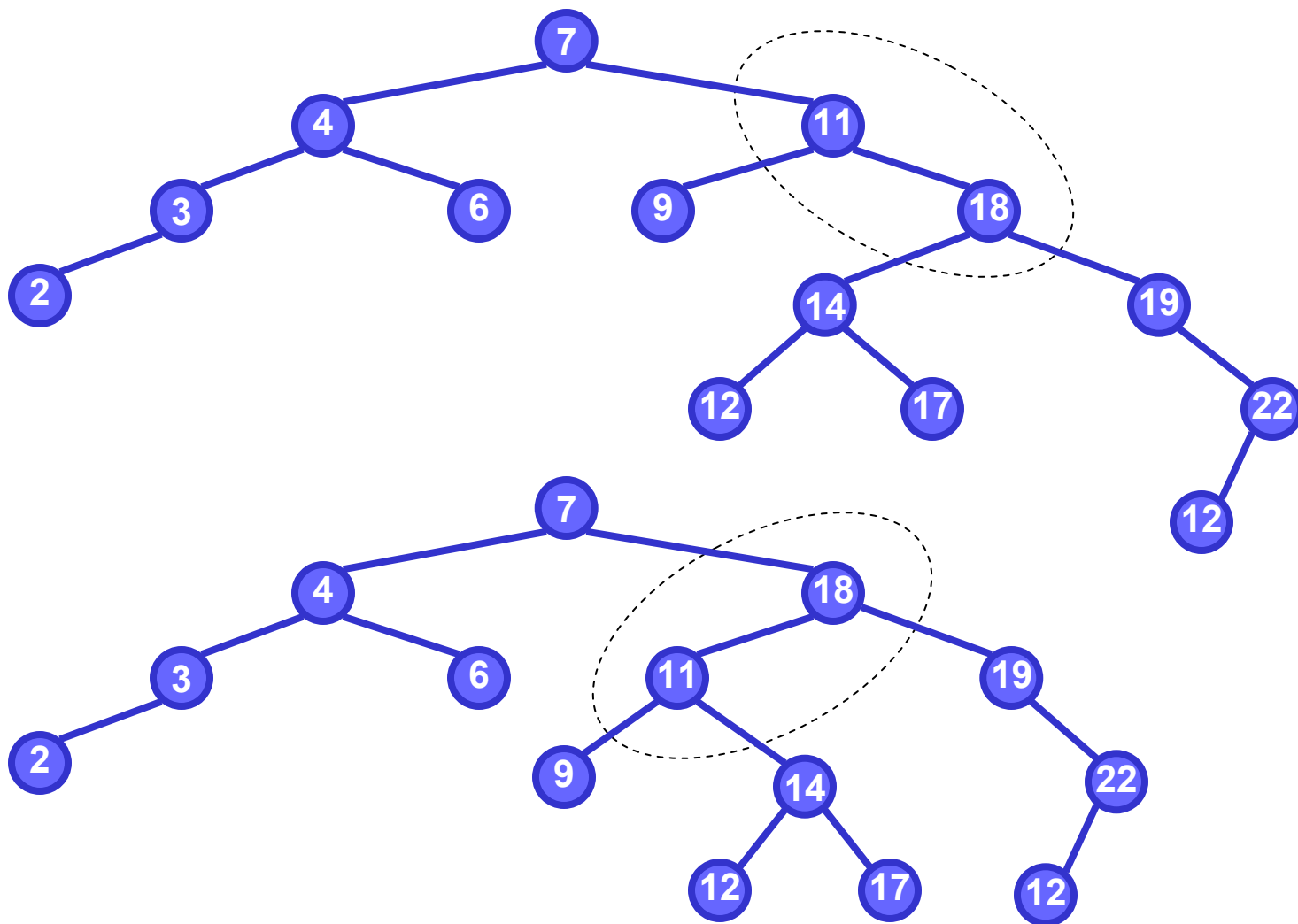
```
RB_INSERT(t,new)      /* inserisco il nodo new nell'albero t */
1.  y = t.null
2.  x = t.root
3.  while x != t.null    // finché non sono arrivato a t.null
4.      y = x            // cerco il padre y a cui appendere new
5.      if new.key < x.key
6.          x = x.left
7.      else x = x.right
8.  new.p = y            // aggiorno il genitore di new
9.  if y == t.null       // se new deve diventare la radice...
10.     t.root = new     // ...aggiorno t.root
11. else if new.key < y.key
12.     y.left = new
13. else y.right = new
14. new.left = new.right = t.null
15. new.color = RED      // i nuovi nodi sono sempre rossi
16. RB_INSERT_FIXUP(t,new) // ripristina le proprietà
```


Rotazioni

- L'operazione base che viene utilizzata per ripristinare le proprietà dell'albero rosso-nero è la rotazione
 - le rotazioni non alterano i colori dei nodi
 - l'albero rimane un albero binario di ricerca
 - l'operazione può essere eseguita in tempo $\Theta(1)$



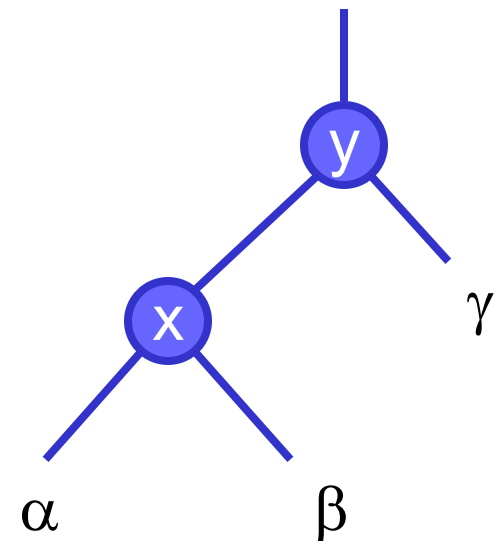
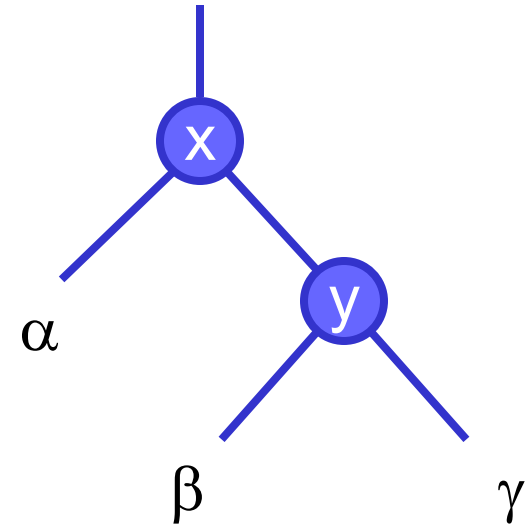
Esempio di rotazione a sinistra



Procedura LEFT_ROTATE

LEFT_ROTATE(t, x)

```
1. y = x.right          // trovo y
2. x.right = y.left     // sposto  $\beta$ 
3. if y.left != t.null
4.     y.left.p = x
5. y.p = x.p
6. if x.p == t.null
7.     t.root = y
8. else if x == x.p.left
9.     x.p.left = y
10.    else x.p.right = y
11. y.left = x
12. x.p = y
```



Ripristino dell'albero rosso-nero

- Il nuovo nodo aggiunto è una foglia e ha colore rosso
- Ricordiamo i vincoli di un albero rosso-nero
 1. ogni nodo è rosso o nero
 2. la radice e la sentinella `t.null` sono nere
 3. se un nodo è rosso entrambi i suoi figli sono neri
 4. tutti i cammini che vanno dalla radice a `t.null` contengono lo stesso numero di nodi neri
- Se l'albero era vuoto la proprietà 2 è violata
 - in questo caso è sufficiente colorare la radice di nero
- Altrimenti solo la proprietà 3 potrebbe essere violata
 - situazione più complicata

Violazione: nodo rosso con un figlio rosso

- Se `RB_INSERT` ha appeso il nuovo nodo `new` (che è sempre rosso) ad un genitore rosso
 - chiamiamo “zio di `new`” il nodo fratello del genitore di `new`
 - lo zio di `new` esiste sempre, eventualmente è `t.null`
 - sono possibili tre casi

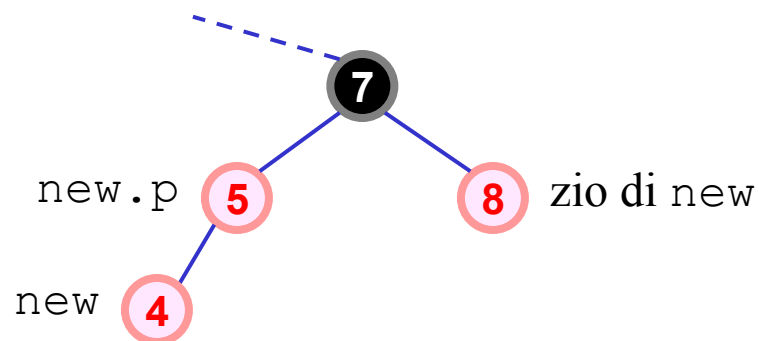
caso 1: `new` è un figlio sinistro e
lo zio è nero e figlio destro

caso 1': `new` è un figlio destro e
lo zio è nero e figlio sinistro

caso 2: `new` è un figlio destro e lo zio è nero e figlio destro

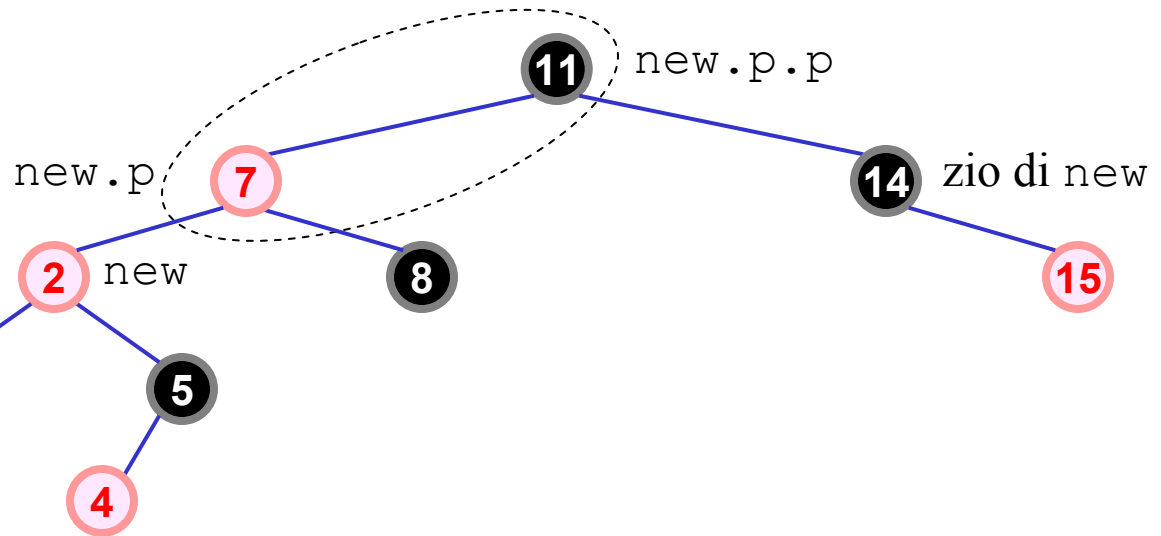
caso 2': `new` è un figlio sinistro e lo zio è nero e figlio sinistro

caso 3: lo zio di `new` è rosso



Violazione: caso 1

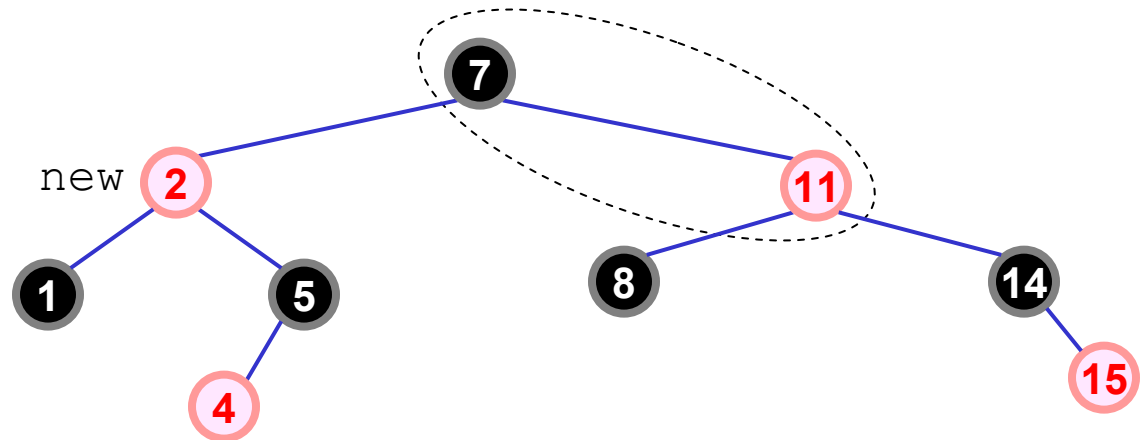
caso 1:
new è un figlio sinistro e
lo zio è nero e figlio destro



ricolorazione
di `new.p` e di `new.p.p`

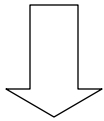
rotazione destra su
`new.p.p`

ora l'albero è rosso-nero

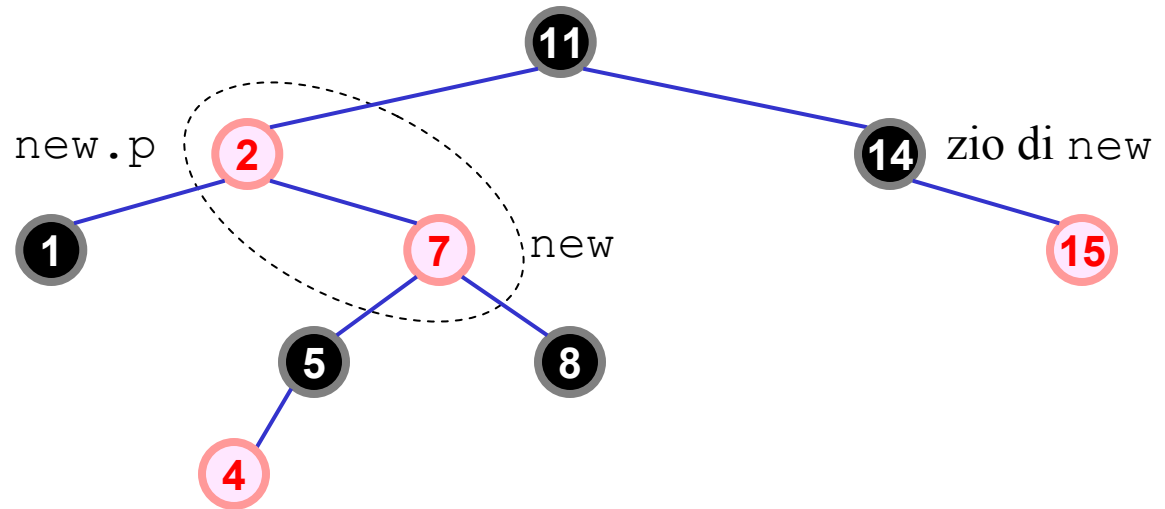


Violazione: caso 2

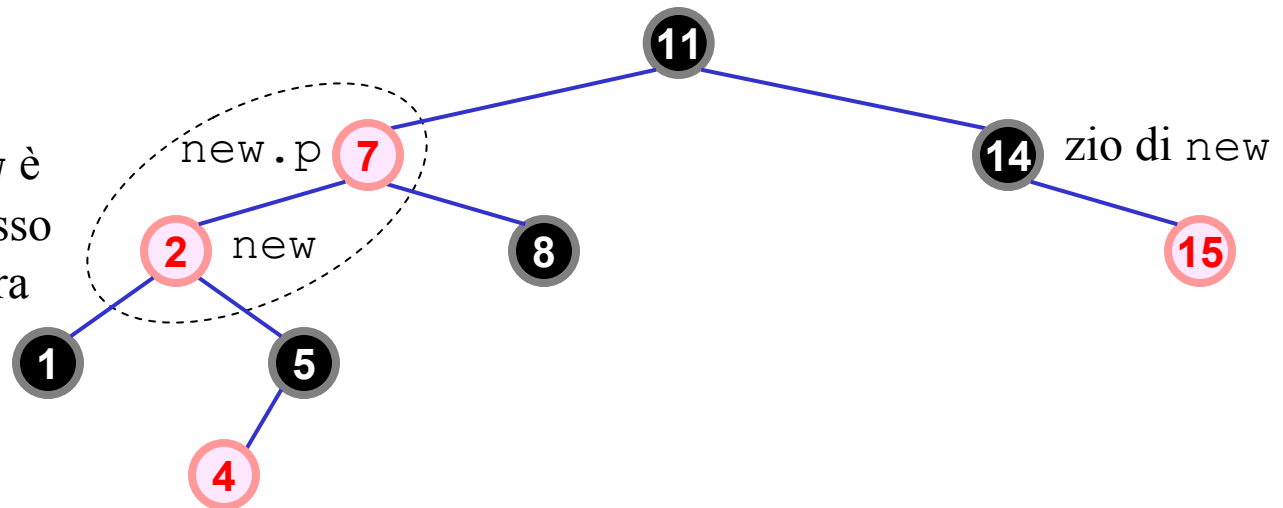
caso 2:
new è un figlio destro e lo
zio è nero e figlio destro



rotazione sinistra su
new.p

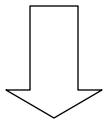


i due nodi violano
ancora la regola 3
(ma questa volta new è
un figlio sinistro e posso
applicare la procedura
del caso 1)

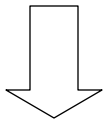


Violazione: caso 3

caso 3:
lo zio di new è rosso

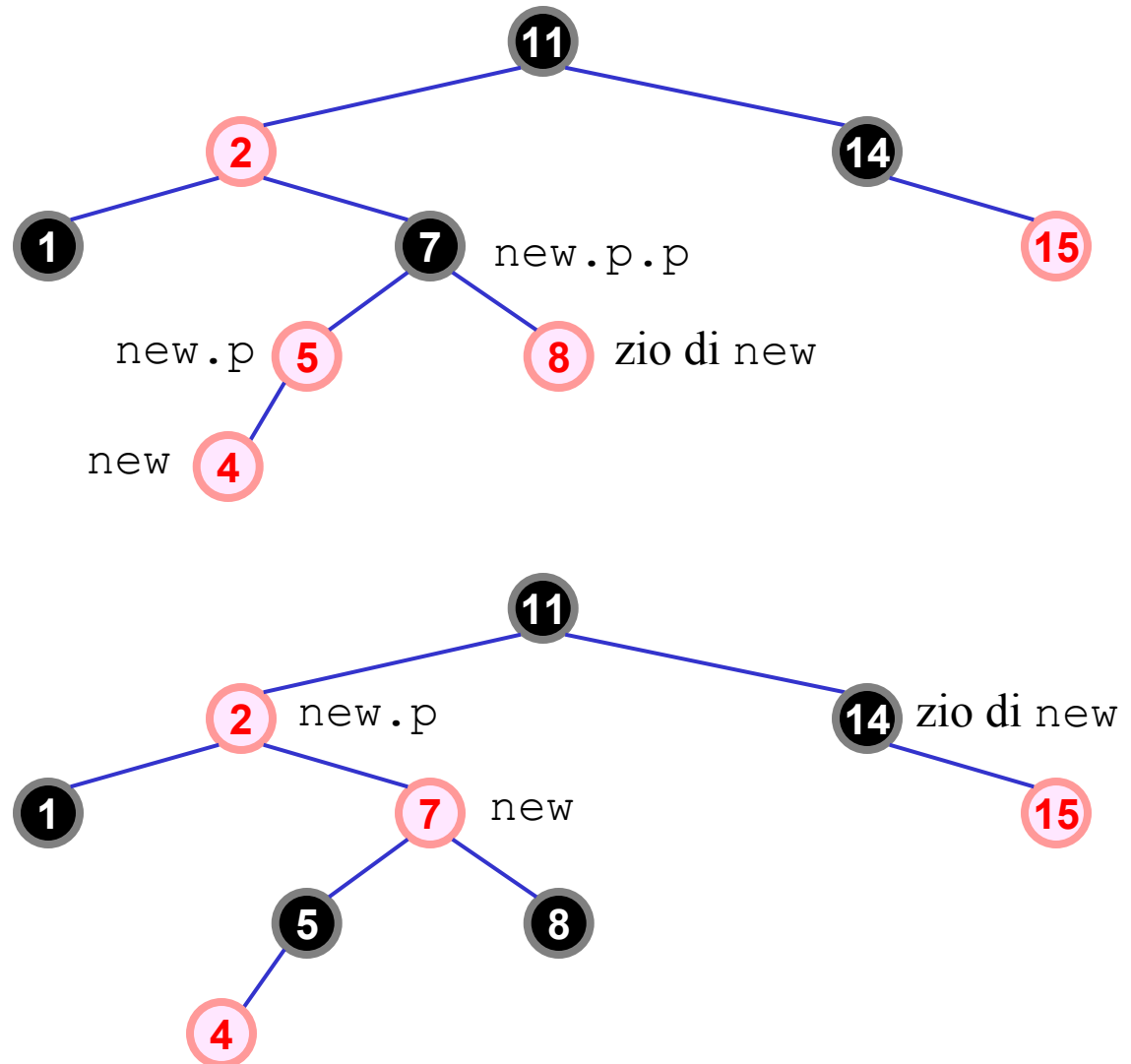


ricolorazione
di new.p, dello zio di
new e di new.p.p



iterazione:
new = new.p.p

ora new e new.p
potrebbero ancora
violare la regola 3
(ma new è più
vicino alla radice)



Violazione: caso 3

- Nel caso 3 `new` è più vicino alla radice ma potrebbe violare la regola 3 con `new.p`
- Occorre rilanciare la procedura con il nuovo `new`
- Il caso peggiore è quando si ha una sequenza di casi 3 fino a che non si risale alla radice
- Quando arriviamo alla radice questa diventa rossa
 - in questo caso è sufficiente ricolorare la radice di nero
 - questo equivale ad incrementare di uno il numero dei nodi in ogni cammino dalla radice al nodo `t.null`

Complessità di RB_INSERT_FIXUP

- Le violazioni nel caso 1 e 2 vengono risolte in tempo $\Theta(1)$
- Poiché l'albero è alto $\Theta(\log n)$, la procedura per risolvere una violazione nel caso 3 può essere rilanciata al massimo $\Theta(\log n)$ volte
- La complessità di RB_INSERT_FIXUP, e dunque di RB_INSERT, è $\Theta(\log n)$

Cancellazioni in un albero rosso-nero

- Analogamente ad `RB_INSERT`, la procedura `RB_DELETE`
 - prima cancella un nodo con la stessa strategia di `TREE_DELETE` degli alberi binari di ricerca
 - poi ripristina le proprietà degli alberi rosso-neri chiamando una opportuna procedura `RB_DELETE_FIXUP`
 - `RB_DELETE_FIXUP` utilizza rotazioni e ricolorazioni

Conclusioni

- Complessivamente gli alberi rosso-neri offrono una realizzazione di alberi binari di ricerca con le seguenti complessità nel caso peggiore
 - inserimento in $\Theta(\log n)$
 - cancellazione in $\Theta(\log n)$
 - ricerca in $\Theta(\log n)$

Esercizi

1. Qual è la complessità dell'algoritmo `TREE_SORT`, che utilizza un albero binario di ricerca per ordinare un array, nel caso in cui l'albero sia un albero rosso-nero?
2. Data una realizzazione del tipo astratto di dato “insieme” tramite un albero rosso-nero con le seguenti funzioni
 - `INSERT(t,k)` in $\Theta(\log n)$
 - `REMOVE(t,k)` in $\Theta(\log n)$
 - `SEARCH(t,k)` in $\Theta(\log n)$realizza la funzione `UNIONE( $t_1, t_2$ )` che calcola l'unione di due insiemi t_1 e t_2 e discute la complessità

Esercizi

3. Mostra come un albero rosso-nero possa essere utilizzato per costruire una coda di priorità
- come si può fare per accedere all'elemento minimo/massimo della coda?
 - qual è il costo delle operazioni di accesso, di cancellazione e di inserimento?

Algoritmi e Strutture di Dati

Complessità dei problemi

m.patrignani

170-complessita-problemi-08 copyright ©2018 maurizio.patrignani@uniroma3.it

Nota di copyright

- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

170-complessita-problemi-08 copyright ©2018 maurizio.patrignani@uniroma3.it

Contenuto

- Definizioni
 - complessità $O(f(n))$, $\Omega(f(n))$ e $\Theta(f(n))$ di un problema
- Problemi e complessità
 - esempi di problemi di complessità ignota
 - lower bound per gli algoritmi di ricerca basati su confronti
 - lower bound per gli algoritmi di ordinamento per confronto

170-complessita-problemi-08

copyright ©2018 maurizio.patrignani@uniroma3.it

Problemi e complessità

- Sappiamo che
 - un algoritmo corretto per un problema computazionale è una “ricetta” per la sua soluzione
 - termina sempre
 - produce un output che, nella definizione del problema, corrisponde all’istanza in input
- Un problema ammette infiniti algoritmi corretti
 - di ogni algoritmo possiamo calcolare la complessità asintotica
- Alcuni problemi ammettono algoritmi più efficienti di altri problemi
 - i problemi hanno una complessità asintotica intrinseca?

170-complessita-problemi-08

copyright ©2018 maurizio.patrignani@uniroma3.it

Analisi della complessità dei problemi

- Obiettivo
 - classificare i problemi in base alla loro difficoltà di soluzione intrinseca
 - determinare la quantità di risorse che comunque è necessario spendere per risolverli
- Strumento
 - associare al problema la complessità dell'algoritmo più efficiente che lo risolve
- Inconveniente
 - dato un problema non è possibile considerare tutti gli infiniti algoritmi che lo risolvono
 - non possiamo determinare direttamente la complessità dell'algoritmo più efficiente

170-complessita-problemi-08

copyright ©2018 maurizio.patrignani@uniroma3.it

Complessità $O(f(n))$ di un problema

- Un problema ha *complessità temporale* $O(f(n))$ se esiste un algoritmo che lo risolve che ha complessità temporale $O(f(n))$
- In forma stenografica:

$$P \in O(f(n)) \Leftrightarrow \exists A \in O(f(n))$$

- $O(f(n))$ sono le risorse *sufficienti* a risolvere il problema

170-complessita-problemi-08

copyright ©2018 maurizio.patrignani@uniroma3.it

Complessità $O(f(n))$ di un problema

- Se un problema ha complessità temporale $O(f(n))$
 - è garantito che il problema possa essere risolto spendendo $O(f(n))$ risorse
 - è possibile che il problema possa essere risolto spendendo meno di $O(f(n))$ risorse
 - potrebbe esistere un algoritmo più efficiente che non conosciamo
 - $f(n)$ è un *limite superiore* (upper bound) alle risorse sufficienti a risolvere il problema
- Per dimostrare che un problema ha complessità $O(f(n))$
 - occorre produrre un algoritmo che lo risolva e che abbia complessità $O(f(n))$

170-complessita-problemi-08

copyright ©2018 maurizio.patignani@uniroma3.it

Esempio: problema $O(f(n))$

```

SOMMA (A) ▷ restituisce la somma degli elementi dell'array A
1. somma = A[0]
2. for i = 1 to A.length-1
3.     somma = somma + A[i]
4. return somma
  
```

- La complessità temporale dell'algoritmo SOMMA è $O(n)$, dove n è il numero degli elementi dell'array A
- Il problema della somma di n interi
 - ha complessità temporale $O(n)$
 - è limitato superiormente da $f(n) = n$
 - “è $O(n)$ ”

170-complessita-problemi-08

copyright ©2018 maurizio.patignani@uniroma3.it

Complessità $\Omega(f(n))$ di un problema

- Un problema ha *complessità temporale* $\Omega(f(n))$ se **ogni** algoritmo che lo risolve ha complessità temporale $\Omega(f(n))$
- In forma stenografica:

$$P \in \Omega(f(n)) \Leftrightarrow \forall A \in \Omega(f(n))$$

- $\Omega(f(n))$ sono le risorse *necessarie* a risolvere il problema

170-complessita-problemi-08

copyright ©2018 maurizio.patrignani@uniroma3.it

Complessità $\Omega(f(n))$ di un problema

- Se un problema ha *complessità temporale* $\Omega(f(n))$
 - non è possibile che il problema possa essere risolto spendendo meno di $\Omega(f(n))$
 - non è detto che il problema sia risolvibile spendendo $O(f(n))$
 - $f(n)$ è un *limite inferiore* (lower bound) alle risorse necessarie per risolvere il problema
- Per dimostrare che un problema ha complessità $\Omega(f(n))$
 - non possiamo considerare tutti gli algoritmi che lo risolvono
 - non esiste un metodo preciso per determinare $\Omega(f(n))$
 - generalmente si ragiona sulla natura delle istanze e delle relative soluzioni

170-complessita-problemi-08

copyright ©2018 maurizio.patrignani@uniroma3.it

Esempio: problema $\Omega(f(n))$

- Consideriamo il problema del calcolo della somma di n interi
- Tutti gli algoritmi che risolvono il problema devono necessariamente prendere in considerazione gli n interi in input
 - altrimenti cambiando un valore di input l'algoritmo darebbe lo stesso output, e questo è assurdo
- Il problema della somma di n interi
 - ha complessità temporale $\Omega(n)$
 - è limitato inferiormente da $f(n) = n$
 - “è $\Omega(n)$ ”

170-complessita-problemi-08 copyright ©2018 maurizio.patignani@uniroma3.it

Complessità $\Theta(f(n))$ di un problema

- Un problema ha *complessità temporale* $\Theta(f(n))$ se ha contemporaneamente complessità temporale $O(f(n))$ e $\Omega(f(n))$
 - non è possibile che il problema possa essere risolto spendendo meno di $O(f(n))$
 - esiste almeno un algoritmo che risolve il problema in $\Theta(f(n))$
- Limite inferiore e limite superiore coincidono
 - $f(n)$ è la complessità intrinseca del problema
- Non sempre è possibile determinare $\Theta(f(n))$
 - di molti problemi la complessità intrinseca è ignota

170-complessita-problemi-08 copyright ©2018 maurizio.patignani@uniroma3.it

Esempio: problema $\Theta(f(n))$

- Per quanto detto sopra il problema della somma di n interi ha complessità $\Theta(n)$
 - l'algoritmo proposto per dimostrare che il problema è $O(n)$ è un algoritmo asintoticamente ottimo
 - possiamo desistere dalla ricerca di algoritmi più efficienti
 - è anche vero che questo algoritmo ha complessità temporale $\Theta(n)$

170-complessita-problemi-08 copyright ©2018 maurizio.patignani@uniroma3.it

Problemi dalla complessità ignota

- Problema del commesso viaggiatore
 - trovare il circuito più breve che tocca n città
- Upper-bound
 - esiste un algoritmo che ha complessità $O(n^2 2^n)$
- Lower-bound
 - siccome occorre leggere l'input, il problema è $\Omega(n)$
 - non è mai stato dimostrato che il problema non possa essere risolto in tempo polinomiale
 - in realtà non è mai stato dimostrato che il problema non possa essere risolto in tempo lineare!

170-complessita-problemi-08 copyright ©2018 maurizio.patignani@uniroma3.it

Problemi NP-completi

- Il problema del commesso viaggiatore appartiene ad una classe di problemi noti come problemi NP-completi
- I problemi NP-completi sono tutti equivalenti
 - se si trovasse un algoritmo polinomiale in grado di risolvere un qualunque problema NP-completo si potrebbero risolvere in tempo polinomiale tutti i problemi NP-completi
- Si ritiene (ma non è stato mai dimostrato) che un algoritmo polinomiale per un problema NP-completo non possa esistere

170-complessita-problemi-08 copyright ©2018 maurizio.patignani@uniroma3.it

Lower bound di problemi comuni

- E' molto difficile dimostrare un lower bound per un problema
- Nel seguito dimostreremo dei lower bound limitati al caso in cui gli algoritmi utilizzati siano basati su confronti
- In particolare dimostreremo
 - lower bound $\Omega(\log n)$ per algoritmi di ricerca basati su confronti
 - lower bound $\Omega(n \log n)$ per algoritmi di ordinamento basati su confronti

170-complessita-problemi-08 copyright ©2018 maurizio.patignani@uniroma3.it

Il problema della ricerca

- Il problema della ricerca può essere descritto come segue
 - è nota una collezione di coppie <chiave, valore>
 - un'istanza del problema è il valore di una chiave
 - la soluzione del problema è il relativo valore
 - oppure l'informazione che una coppia con tale chiave è assente nella collezione

170-complessita-problemi-08 copyright ©2018 maurizio.patrigiani@uniroma3.it

Cosa sappiamo del problema della ricerca

- Dipendentemente dal tipo di struttura dati che adottiamo per la collezione di coppie <chiave, valore> il problema della ricerca ha diverse complessità nel caso peggiore

Liste	$O(n)$
Array non ordinati	$O(n)$
Array ordinati	$O(\log n)$
Alberi rosso-neri	$O(\log n)$

- Esiste un algoritmo più veloce di $O(\log n)$?

170-complessita-problemi-08 copyright ©2018 maurizio.patrigiani@uniroma3.it

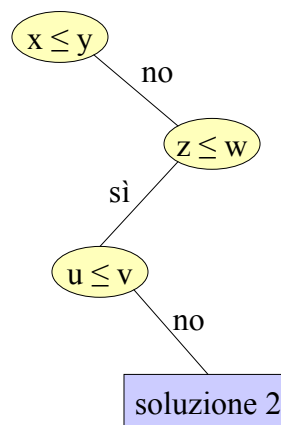
Algoritmi basati su confronti

- Un algoritmo di ricerca è detto “*algoritmo di ricerca basato su confronti*” se il flusso delle operazioni dipende esclusivamente dal confronto tra la chiave cercata ed una chiave della collezione
- Esempio
 - nella ricerca binaria si accede all’elemento intermedio dell’intervallo di ricerca e si ricorre su uno dei due sottointervalli generati in base al confronto della chiave cercata con il valore della chiave dell’elemento intermedio

170-complessita-problemi-08 copyright ©2018 maurizio.patrigiani@uniroma3.it

Esecuzione di una ricerca per confronto

- Immaginiamo di lanciare un algoritmo di ricerca basato su confronti
- L’algoritmo eseguirà un certo numero di confronti per poi produrre un output
 - l’output è un’opportuna cella di memoria
- Uno qualsiasi dei valori della collezione potrebbe essere l’output giusto



170-complessita-problemi-08 copyright ©2018 maurizio.patrigiani@uniroma3.it

Albero di decisione

- Possiamo definire un albero i cui nodi interni sono i vari confronti eseguiti dall'algoritmo e le cui foglie sono le possibili risposte
- Questo albero è un albero binario con n foglie
 - l'altezza dell'albero è $\Omega(\log n)$
 - il numero dei confronti necessari per individuare una foglia è $\Omega(\log n)$
- Ne consegue che nel caso peggiore una ricerca implica $\Omega(\log n)$ confronti

170-complessita-problemi-08 copyright ©2018 maurizio.patrignani@uniroma3.it

Algoritmi di ordinamento

- Con considerazioni analoghe dimostreremo un lower bound sugli algoritmi di ordinamento per confronto
- Gli algoritmi più veloci che conosciamo, come il MERGE_SORT, hanno una complessità temporale $\Theta(n \log n)$
- Dimostreremo che tutti gli algoritmi di ordinamento per confronto hanno una complessità nel caso peggiore $\Omega(n \log n)$

170-complessita-problemi-08 copyright ©2018 maurizio.patrignani@uniroma3.it

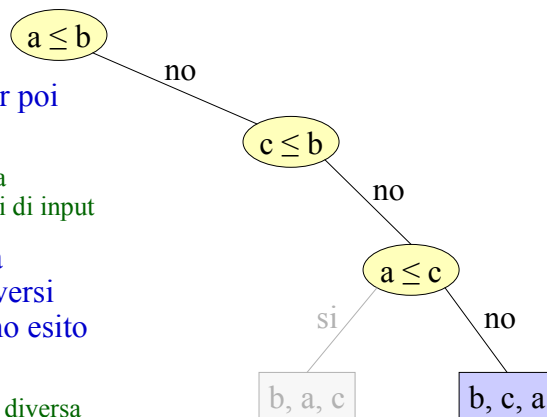
Algoritmi di ordinamento per confronto

- Un algoritmo di ordinamento è detto “*algoritmo di ordinamento per confronto*” se il flusso delle operazioni dipende dal confronto tra due elementi della sequenza
- Esempio
 - nel MERGE_SORT l’operazione MERGE confronta i valori delle due sotto-sequenze ordinate per ottenere un’unica sequenza ordinata

170-complessita-problemi-08 copyright ©2018 maurizio.patrignani@uniroma3.it

Esecuzione di un algoritmo per confronto

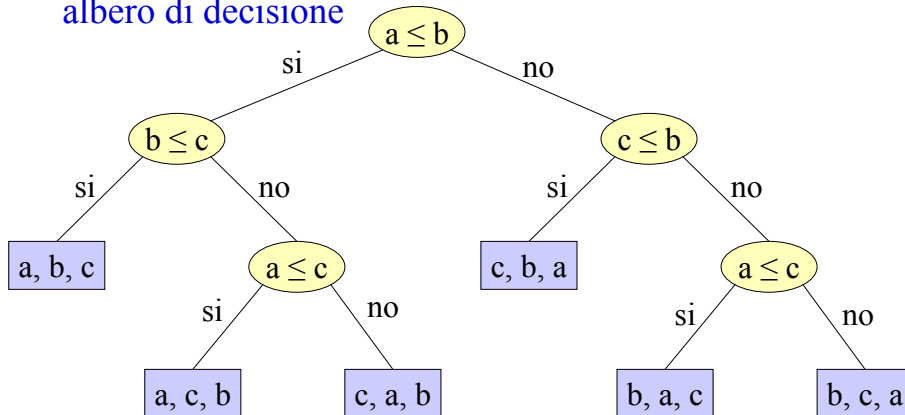
- Immaginiamo di lanciare un algoritmo di ordinamento per confronto con una generica sequenza di input (a,b,c)
- L’algoritmo eseguirà un certo numero di confronti per poi produrre un output
 - l’output è un’opportuna permutazione dei valori di input
- Se lo lanciamo con una sequenza con valori diversi alcuni confronti avranno esito diverso
 - l’output prodotto è una diversa permutazione dei valori di input



170-complessita-problemi-08 copyright ©2018 maurizio.patrignani@uniroma3.it

Albero di decisione

- L'esecuzione di un algoritmo di ordinamento per confronto equivale alla discesa in un immaginario albero di decisione



170-complessita-problemi-08

copyright ©2018 maurizio.patrignani@uniroma3.it

Numero di confronti necessari

- Tutte le permutazioni degli elementi da ordinare devono essere foglie dell'albero di decisione
 - ogni possibile permutazione dei valori di input deve essere raggiungibile
 - se n sono gli elementi da ordinare le possibili permutazioni sono $n!$
- Il numero di confronti eseguiti nel caso peggiore equivale al cammino più lungo tra la radice ed una foglia
 - l'altezza di un albero binario con $n!$ foglie è almeno $\log_2 n!$
 - il problema dell'ordinamento per confronto è $\Omega(\log_2 n!)$

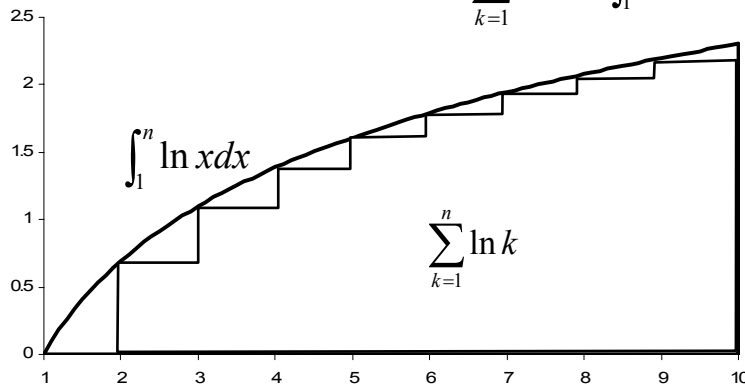
170-complessita-problemi-08

copyright ©2018 maurizio.patrignani@uniroma3.it

Approssimazione di Stirling

- Consideriamo la funzione $\ln n!$
 - nel calcolo asintotico la base del logaritmo è indifferente

$$\ln n! = \ln 1 + \ln 2 + \dots + \ln n = \sum_{k=1}^n \ln k \approx \int_1^n \ln x dx$$



Calcolo di $\int_1^n \ln x dx$

- Integrazione per parti: $\int u \frac{dv}{dx} dx = uv - \int v \frac{du}{dx} dx$
- Nel nostro caso

$$u = \ln x \quad v = x$$

$$\bullet \quad dv/dx = 1; \quad du/dx = 1/x$$

$$\int \ln x \cdot 1 dx = x \ln x - \int x \frac{1}{x} dx = x \ln x - x$$

$$\sum_{k=1}^n \ln k \approx \int_1^n \ln x dx = (x \ln x - x) \Big|_1^n = n \ln n - n + 1$$

Ordinamento per confronto: lower bound

- L'esecuzione di un algoritmo di ordinamento per confronto corrisponde alla discesa in un albero di decisione con $n!$ foglie
 - n è il numero di elementi da ordinare
- Nel caso peggiore il numero di confronti (nodi interni nel cammino radice-foglia) è $\Omega(n \ln n)$
 - MERGE_SORT è un algoritmo di ordinamento per confronto asintoticamente ottimo

170-complessita-problemi-08 copyright ©2018 maurizio.patrignani@uniroma3.it

Domande sulla complessità dei problemi

1. Supponiamo che il problema P abbia complessità $O(n)$. E' possibile che esista un algoritmo A che risolve P che abbia una complessità $\Omega(n^2)$?
2. Supponiamo che un problema P abbia complessità $\Theta(n^2)$. Può esistere un algoritmo A che risolve P e ha complessità $\Omega(n)$?
3. Supponiamo che un problema P abbia complessità $\Theta(n)$. Può esistere un algoritmo A che risolve P e ha complessità $\Theta(n^2)$?

170-complessita-problemi-08 copyright ©2018 maurizio.patrignani@uniroma3.it

Soluzioni

1. $P \in O(n)$. Può esistere $A \in \Omega(n^2)$?

- Sì, se $P \in O(n)$ vuol dire che esiste un (opportuno) algoritmo $A' \in O(n)$. Gli altri algoritmi, tra cui A , che risolvono P possono avere complessità arbitrariamente elevata

2. $P \in \Theta(n^2)$. Può esistere $A \in \Omega(n)$?

- Sì. Non solo, tutti gli algoritmi che risolvono P hanno complessità $\Omega(n^2)$ e dunque anche $\Omega(n)$

3. $P \in \Theta(n)$. Può esistere $A \in \Theta(n^2)$?

- Sì, ciò non contraddice $P \in O(n)$ né $P \in \Omega(n)$.

Algoritmi e Strutture di Dati

Ordinamento in tempo lineare

m.patrignani

Nota di copyright

- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Contenuto

- Ordinamento in tempo lineare
 - Counting sort
 - Bucket sort
 - bucket sort per numeri reali
 - Radix sort

Ordinamento in tempo lineare

- In alcuni casi è possibile ordinare n elementi senza eseguire confronti tra i loro valori
 - per esempio si dispone di informazioni diverse sulla distribuzione dei valori degli elementi
- In questi casi si possono concepire algoritmi con complessità $\Theta(n)$ nel caso peggiore

Ordinamento in tempo lineare

- Esempio banale

- supponiamo di sapere che l'input sia una permutazione dei primi n numeri interi
- potrei scandire l'array di input e quando trovo l'elemento con chiave i lo metto in posizione i dell'output



- questo algoritmo è evidentemente lineare
 - tuttavia i vincoli sull'input sono molto stringenti

Assunzioni sull'input

- Gli algoritmi COUNTING_SORT e BUCKET_SORT che seguono assumono che gli n elementi in input siano interi non negativi minori o uguali a k , con $k \in O(n)$
 - k può essere maggiore, minore o uguale ad n
 - l'input può contenere ripetizioni
 - alcuni interi in $[0 \dots k]$ potrebbero non essere presenti

Strategia del counting sort

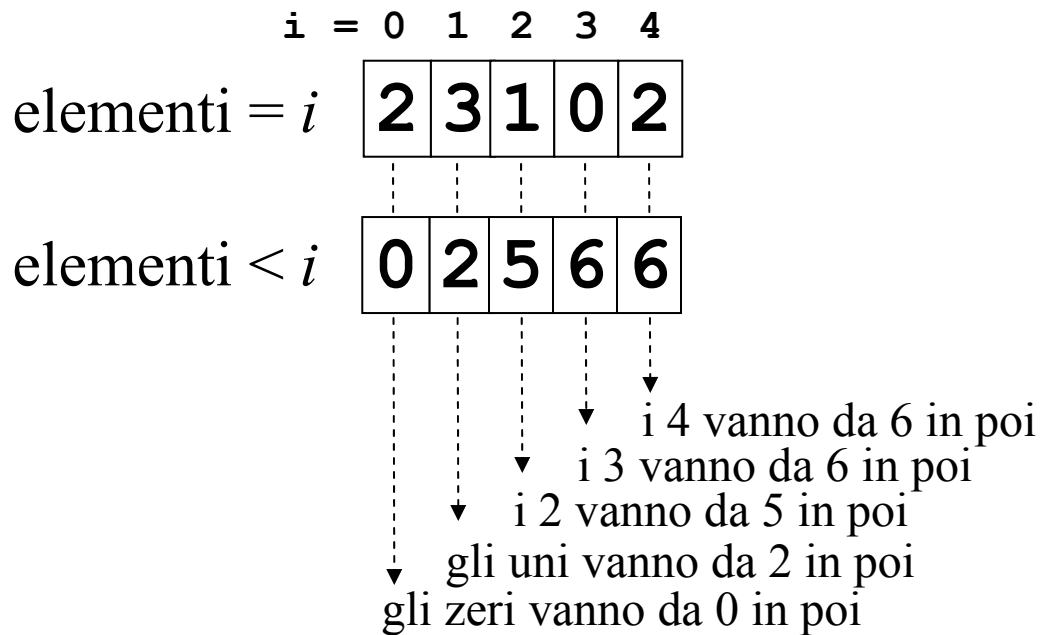
- Creo k contatori, uno per ogni possibile valore di i
 - tempo $O(k) = O(n)$
- Per ogni i conto quanti sono gli elementi in input uguali ad i
 - tempo $O(n)$
- Conto quanti sono gli elementi in input minori ad i
 - tempo $O(n)$ a partire dai valori precedenti
- Scorro l'array di input e posiziono gli elementi nel punto opportuno dell'array di output
 - tempo $O(n)$ a partire dai valori precedenti

Esempio di counting sort

- In questo esempio $n=8$ e $k=4$

input

4	1	4	0	2	0	1	1
---	---	---	---	---	---	---	---



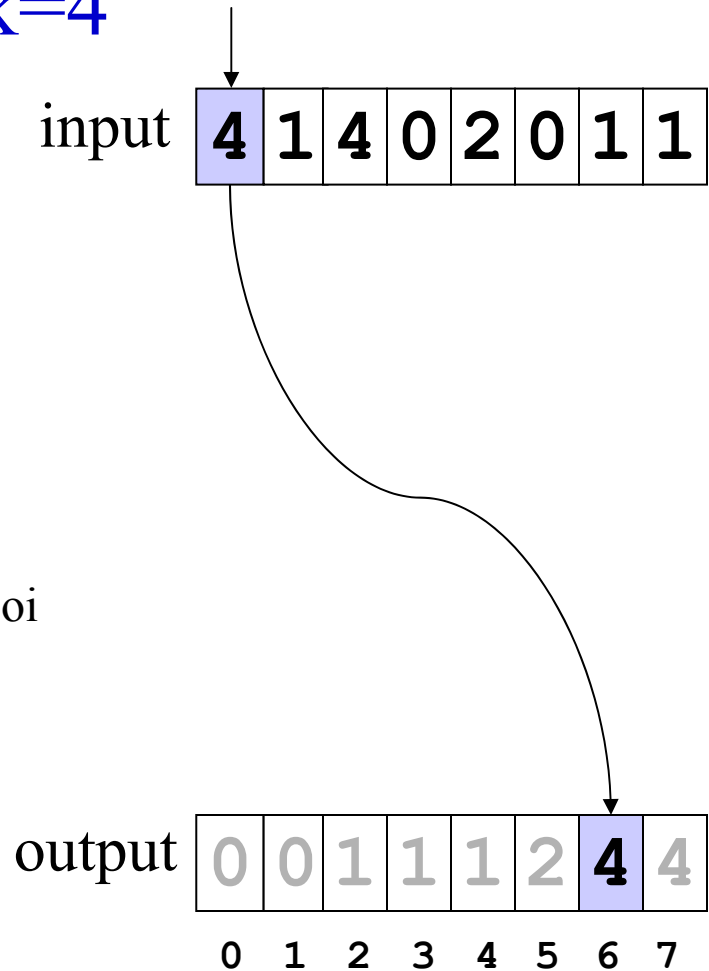
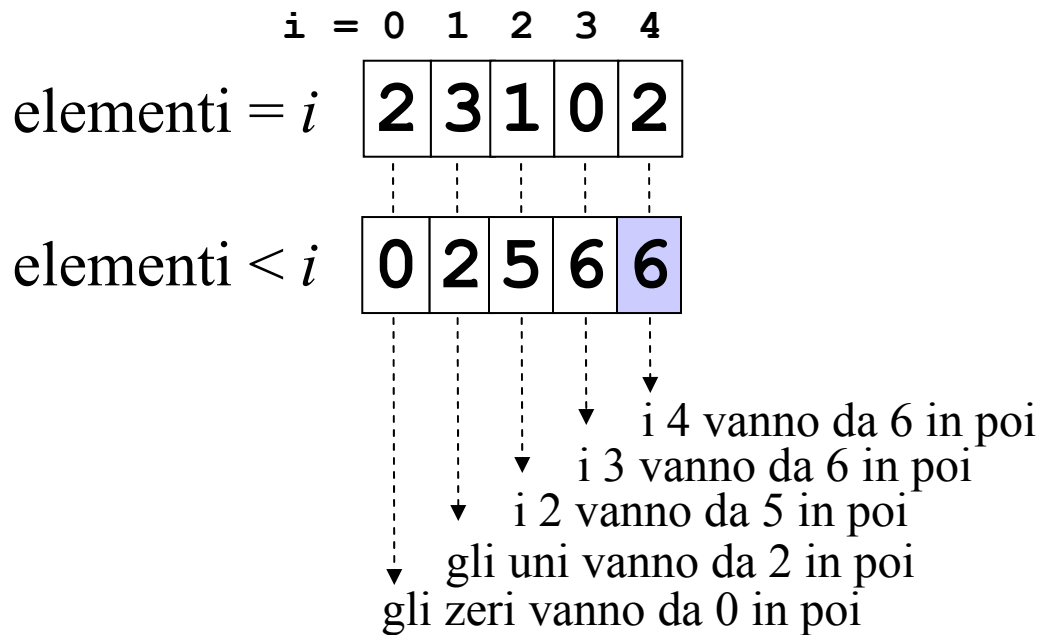
output

0	0	1	1	1	2	4	4
---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7

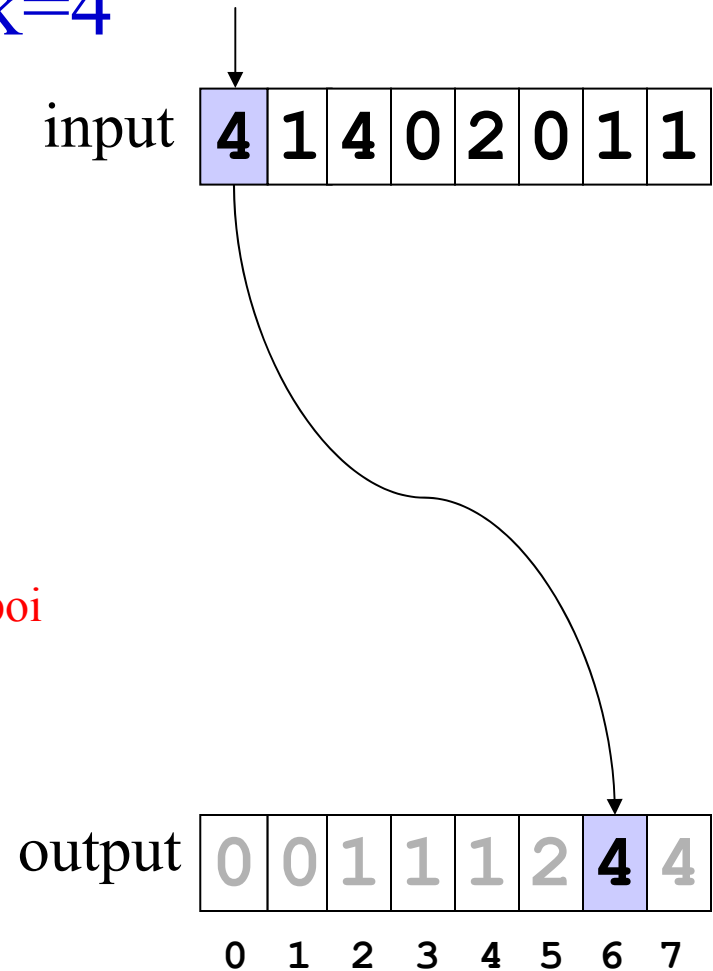
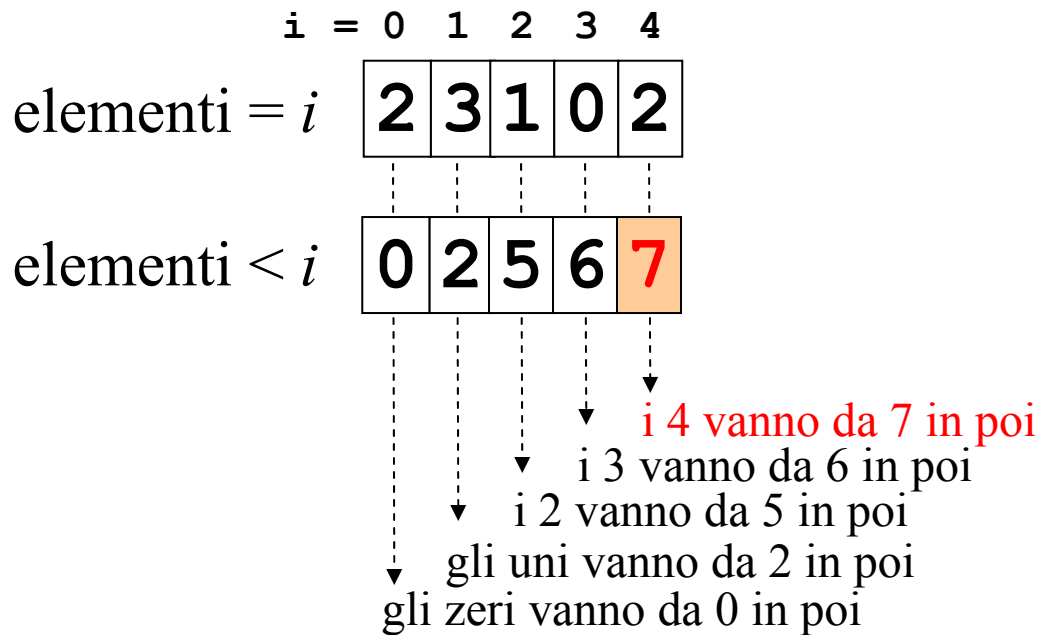
Esempio di counting sort

- In questo esempio $n=8$ e $k=4$



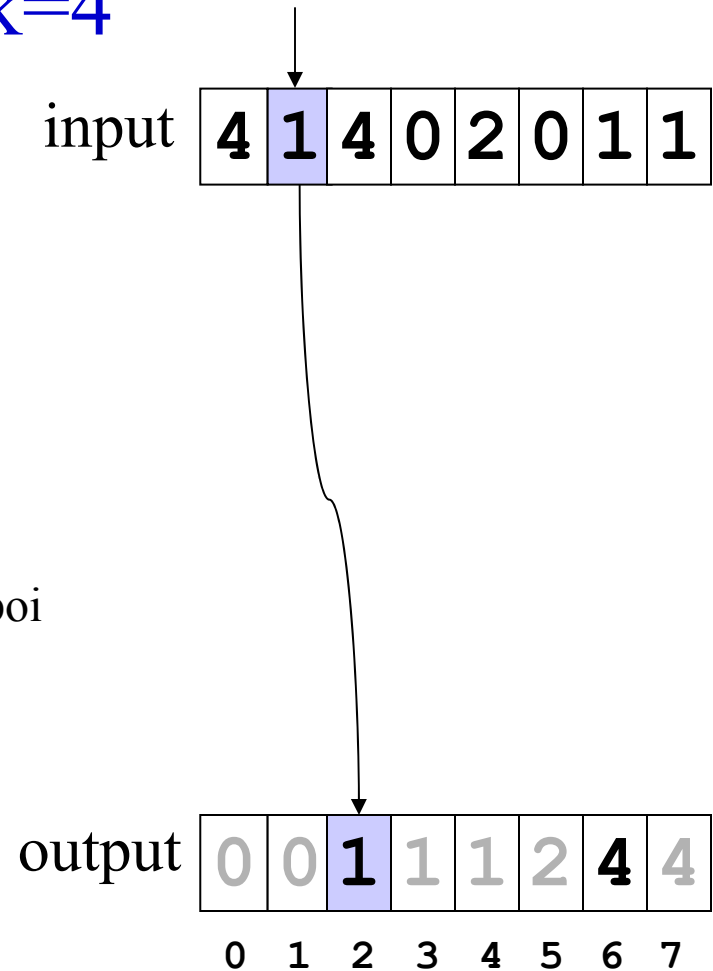
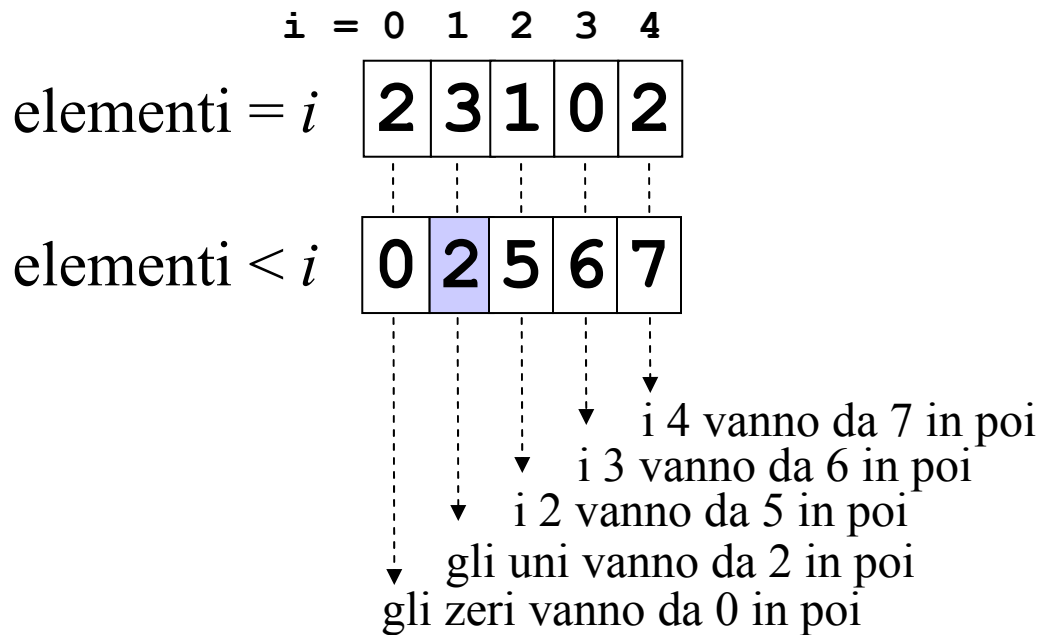
Esempio di counting sort

- In questo esempio $n=8$ e $k=4$



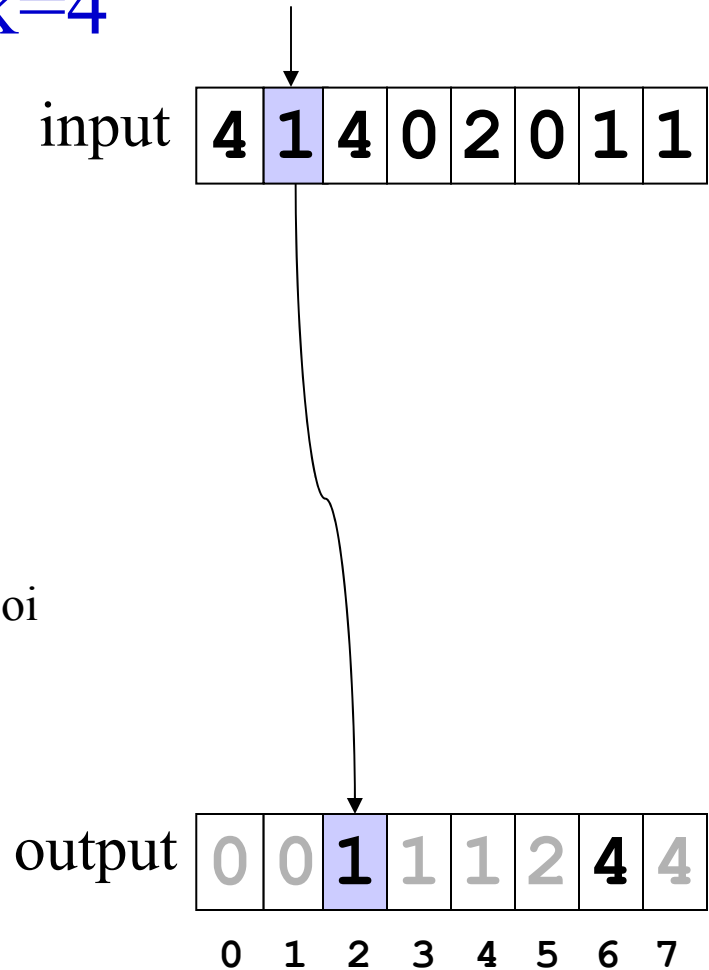
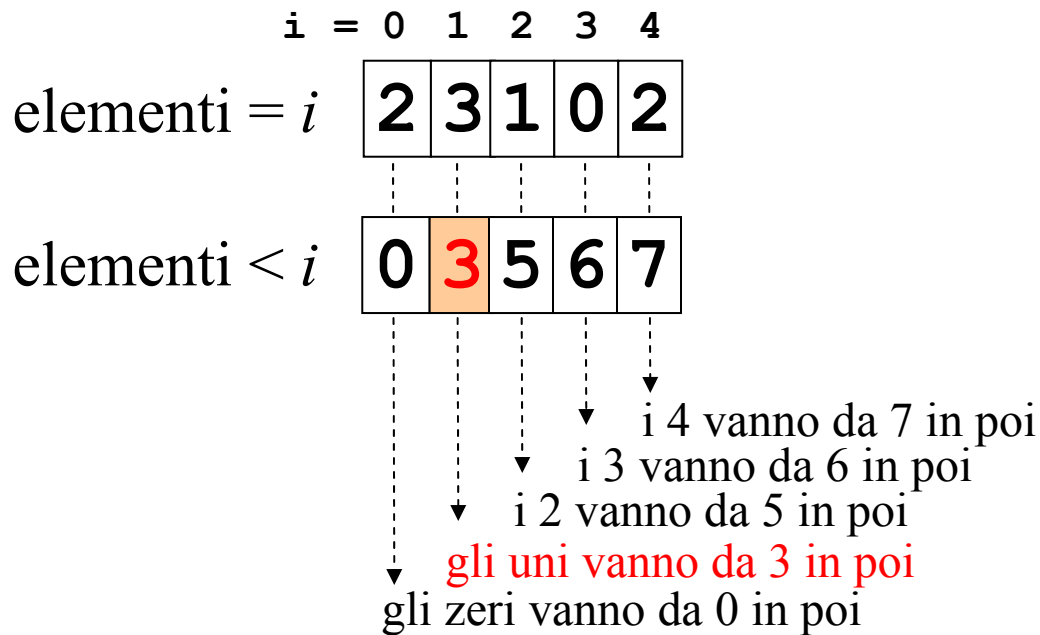
Esempio di counting sort

- In questo esempio $n=8$ e $k=4$



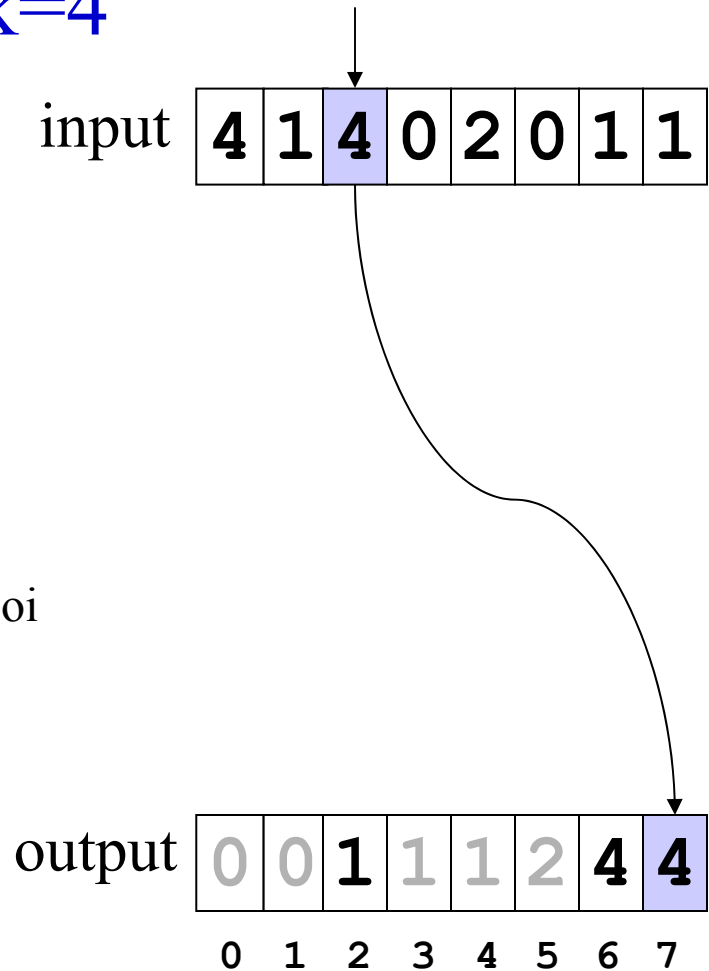
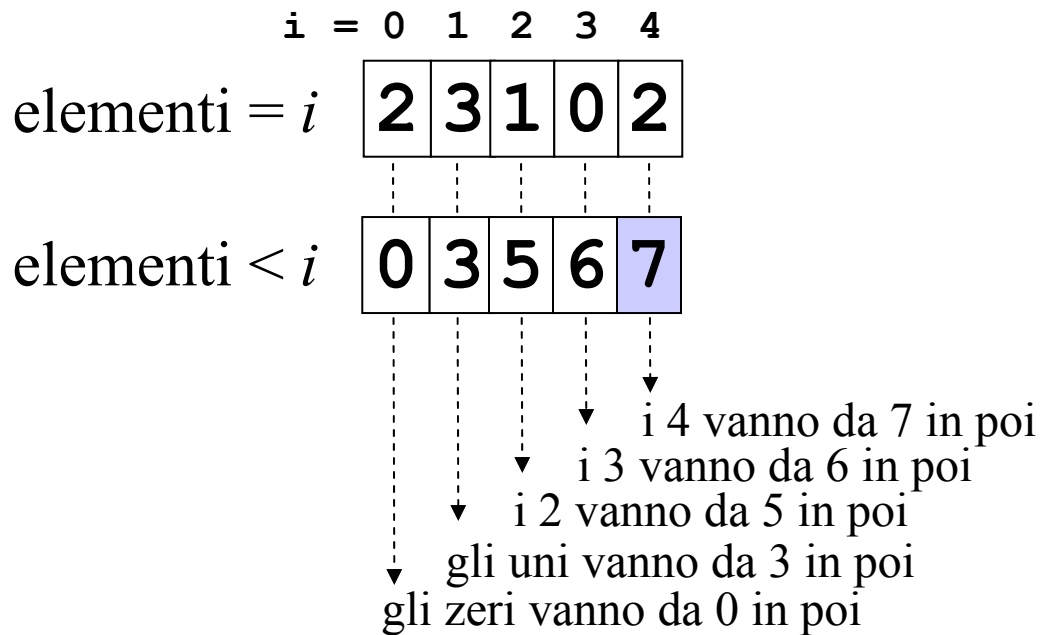
Esempio di counting sort

- In questo esempio $n=8$ e $k=4$



Esempio di counting sort

- In questo esempio $n=8$ e $k=4$



COUNTING_SORT

- Prima versione
 - usa due array di appoggio C e D

```
COUNTING_SORT(A,B,k)    // A=input, B=output, k=valore max
1.  for i = 0 to k      // pongo a zero gli array
2.      C[i] = 0         // numero elementi con valore = i
3.      D[i] = 0         // numero elementi con valore < i
4.  for i = 0 to A.length-1
5.      C[A[i]] = C[A[i]]+1
6.  for i = 1 to k
7.      D[i] = C[i-1] + D[i-1]
8.  for i = 0 to A.length-1
9.      B[D[A[i]]] = A[i]
10.     D[A[i]] = D[A[i]] + 1
```

COUNTING_SORT_2

- Versione ancora più sintetica (ma un po' involuta)
 - usa un solo array di appoggio C

COUNTING-SORT-2 (A, B, k)

```
1. for i = 0 to k          // pongo a zero l'array di appoggio
2.   C[i] = 0
3. for i = 0 to A.length-1
4.   C[A[i]] = C[A[i]]+1    // numero elementi = i
5. for i = 1 to k
6.   C[i] = C[i] + C[i-1]   ▷ numero elementi ≤ i
7. for i = A.length-1 downto 0
8.   C[A[i]] = C[A[i]] - 1
9.   B[C[A[i]]] = A[i]
```

Proprietà del counting sort

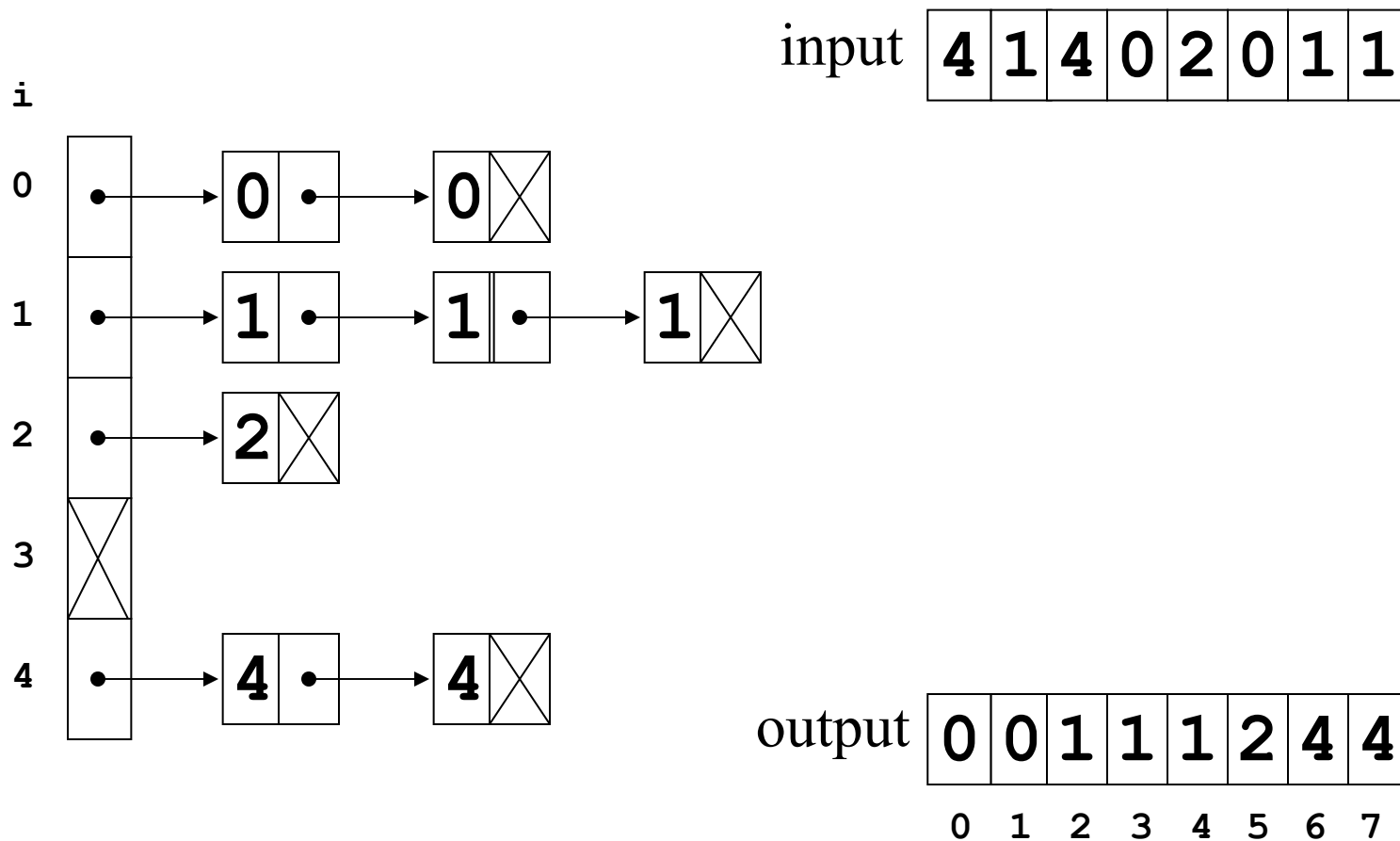
- E' un algoritmo di ordinamento stabile
 - la posizione reciproca di due occorrenze della stessa chiave viene mantenuta nell'output
- Non è un algoritmo di ordinamento in loco
 - l'output è memorizzato su un array diverso da quello di input
 - fa uso di array di supporto di dimensione $O(n)$
- Non è un algoritmo online
 - occorre conoscere tutto l'input per iniziare ad ordinare gli elementi
- Lanciato su un input generico ha complessità $\Theta(n+k)$

Strategia del bucket sort

- Creo una lista (un “bucket”) per ogni i minore o uguale a k
 - tempo $\Theta(k) = \Theta(n)$
- Scorro l’array di input e metto l’elemento corrente nel bucket che corrisponde al suo valore
 - tempo $\Theta(n)$
- Svuoto i bucket ponendo i loro elementi nell’array di output
 - tempo $\Theta(n)$

Esempio di bucket sort

- In questo esempio $n=8$ e $k=4$



BUCKET_SORT

- Algoritmo per ordinare n interi minori o uguali a k , con $k \in \Theta(n)$

```
BUCKET_SORT(A, k)          // A=input, k=valore massimo
1. /* creo un array B di k bucket (liste) */
2. for i = 0 to B.length-1
3.     B[i] = NULL          // inizializzo i bucket vuoti
4. for i = 0 to A.length-1
5.     INSERT(B, A[i])       // inserisco in testa
6. i = A.length-1           // riempio A dalla coda
7. for j = B.length-1 downto 0
8.     while not B[j] == NULL
9.         A[i] = EXTRACT-FIRST(B, j)
10.        i = i - 1
```

BUCKET_SORT – funzioni accessorie

- Dove

```
INSERT(B,i)  // B=array di riferimenti, i=intero
```

```
1. /* creo un elemento della lista temp */
```

```
2. temp.info = i          // creo un elemento con il valore i
```

```
3. temp.next = B[i]
```

```
4. B[i] = temp           // lo aggiungo alla lista i-esima
```

```
EXTRACT_FIRST(B,j)  // B=array di riferimenti, j=intero
```

```
1. output = B[j].info    /* valore da ritornare */
```

```
2. B[j] = B[j].next
```

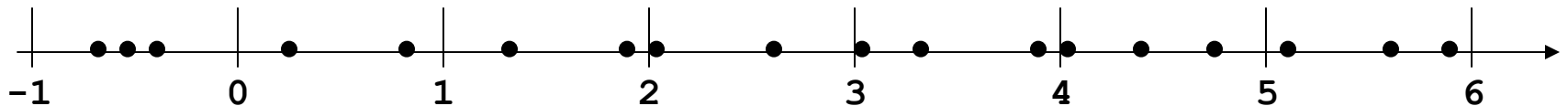
```
3. return output
```

Proprietà del bucket sort

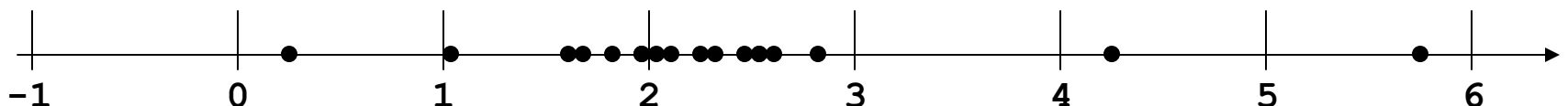
- E' un algoritmo di ordinamento stabile
- Non è un algoritmo di ordinamento in loco
 - fa uso di una struttura di dati per realizzare i bucket
- Non è un algoritmo online
 - occorre conoscere tutto l'input per iniziare ad ordinare gli elementi
- Lanciato su un input generico ha complessità $\Theta(n+k)$

Estensione ai numeri uniformemente distribuiti

- Generalizzazione del COUNTING_SORT e del BUCKET_SORT con complessità lineare nel caso medio
- Si suppone che i numeri in input siano arbitrari (cioè non necessariamente interi) ma uniformemente distribuiti in un determinato intervallo
 - uniformemente distribuiti



- non uniformemente distribuiti



Generalizzazione del BUCKET_SORT

- Strategia
 - suddivido l'intervallo in n parti uguali e creo un bucket per ognuna di esse
 - tempo $O(n)$
 - scorro l'array di input ed inserisco ogni elemento nel bucket corrispondente
 - tempo $O(n)$
 - ordino ogni bucket con un algoritmo di ordinamento ottimale
 - si può dimostrare che il tempo atteso totale è $O(n)$
 - svuoto i bucket ponendo i loro elementi nell'array di output
 - tempo $O(n)$
- La complessità di questo algoritmo è $O(n)$ nel caso peggiore

BUCKET_SORT_REAL

- Algoritmo per ordinare n numeri reali con distribuzione uniforme nell'intervallo aperto $(0,1)$

BUCKET-SORT-REAL (A, k)

```
1. for  $i = 0$  to  $k-1$                 // ho  $k$  bucket
2.      $B[i] = \text{NULL}$                 // inizializzo i bucket vuoti
3. for  $i = A.\text{length}-1$  downto  $0$ 
4.      $\text{bucket} = \lfloor k \cdot A[i] \rfloor$  // uno dei  $k$  bucket
5.     INSERT ( $B, \text{bucket}, A[i]$ )
6.  $i = 0$                             // mi posiziono su  $A[0]$ 
7. for  $j = 0$  to  $k$ 
8.     ANY-OPTIMAL-SORT ( $B[j]$ )
9.     while not  $B[j] == \text{NULL}$ 
10.         $A[i] = \text{EXTRACT-FIRST}(B, j)$ 
11.         $i = i + 1$ 
```

Assunzioni del radix sort

- Il radix sort assume che gli elementi da ordinare siano descritti da una sequenza di d cifre (o più genericamente di d colonne)
 - il numero d delle cifre (o colonne) sia costante
 - i simboli su ogni cifra (o colonna) siano $O(n)$
 - nel caso dei numeri sono anch'essi costanti: “0”, “1”, “2”, ..., “9”
 - per esempio: le targhe automobilistiche hanno una lunghezza costante e un numero di simboli predefinito (cifre e lettere)

Considerazioni preliminari al radix sort

- Supponiamo di voler ordinare un insieme su due valori
 - per esempio un insieme di nomi ordinati per cognome e poi per nome, come in un elenco telefonico

Bianchi	Giulia
Rossi	Andrea
Verdi	Maria
Bianchi	Antonio
Verdi	Antonio
Rossi	Maria

Proviamo ad ordinare due volte

- Prima per cognome e poi per nome

Bianchi	Giulia
Rossi	Andrea
Verdi	Maria
Bianchi	Antonio
Verdi	Antonio
Rossi	Maria

Bianchi	Giulia
Bianchi	Antonio
Rossi	Andrea
Rossi	Maria
Verdi	Maria
Verdi	Antonio

Rossi	Andrea
Bianchi	Antonio
Verdi	Antonio
Bianchi	Giulia
Rossi	Maria
Verdi	Maria

ordino per cognome

ordino per nome

- Il risultato non è quello sperato

Proviamo ad ordinare due volte

- Prima per nome e poi per cognome

ordino per nome

Bianchi	Giulia
Rossi	Andrea
Verdi	Maria
Bianchi	Antonio
Verdi	Antonio
Rossi	Maria

Rossi	Andrea
Bianchi	Antonio
Verdi	Antonio
Bianchi	Giulia
Verdi	Maria
Rossi	Maria

Bianchi	Antonio
Bianchi	Giulia
Rossi	Andrea
Rossi	Maria
Verdi	Antonio
Verdi	Maria

ordino per cognome

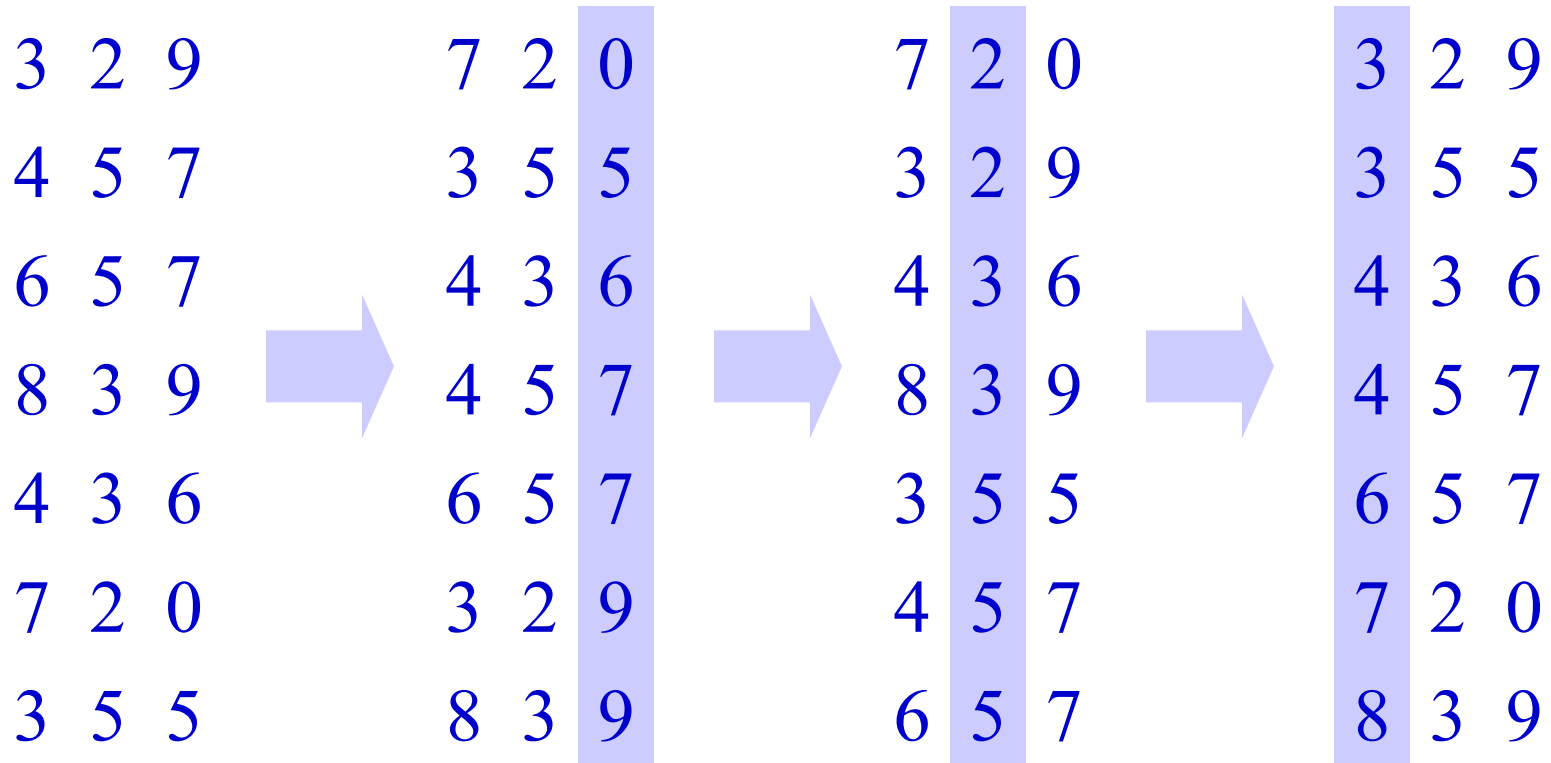
- L'ordinamento trovato è quello giusto

Conclusioni dell'esperimento

- Quando vogliamo ordinare in base a più criteri dobbiamo prima considerare i criteri meno significativi e poi i criteri più significativi
- L'algoritmo di ordinamento utilizzato deve essere stabile
 - altrimenti a determinare l'ordine finale degli elementi sarebbe solo l'ultimo algoritmo applicato

Strategia del radix sort

- Applichiamo lo stesso principio ai numeri



RADIX_SORT

- Supponiamo che l'input sia una matrice con d colonne

```
RADIX_SORT(A)           // A = matrice di input
1. for i = A.length-1 downto 0
2.     ordinamento stabile di A rispetto alla colonna i
```

- Analisi della complessità
 - dobbiamo eseguire d ordinamenti
 - il tempo complessivo è $\Theta(d(n+k))$, dove k è il valore massimo dei valori nelle celle
 - nel caso di numeri decimali
 - il numero delle colonne è uguale al numero delle cifre
 - su ogni cifra il massimo valore $k = 9$
 - la complessità è $\Theta(d \cdot n)$
 - quando d è costante allora la complessità è $\Theta(n)$

Esercizi

1. Qual è la complessità del COUNTING_SORT nel caso in cui il valore massimo k degli n elementi da ordinare è $O(n \log n)$
2. Mostra i passi intermedi del RADIX_SORT per ordinare questa lista di parole:

COW, DOG, SEA, RUG, ROW, MOB, BOX,
TAB, BAR, EAR, TAR, DIG, BIG, TEA, NOW,
FOX

Esercizi

3. Considera il seguente algoritmo di ordinamento su interi

- scorro la sequenza in input
- se rilevo che $k \in O(n)$ lancio l'algoritmo BUCKET_SORT
- altrimenti lancio l'algoritmo MERGE_SORT

L'algoritmo è ben formulato? Può essere implementato?

Esercizi

4. Vogliamo ordinare con il COUNTING_SORT un array con i seguenti elementi: 21, 3, 1024, 52

Quale di queste affermazioni è corretta?

- il tempo di esecuzione è lineare, in quanto il valore massimo k contenuto nell'array è $O(n)$, infatti

$$k=256n = 2^8 \cdot 2^2 = 2^{8+2}=2^{10}=1024$$

- il tempo di esecuzione è quadratico, in quanto il valore massimo k contenuto nell'array è $O(n^2)$, infatti

180-ordinamento-in-tempo-lineare-04 Copyright © 2021 maurizio.petrignani@uniroma3.it

$$k=64 \cdot n^2 = 2^6 \cdot 2^2 \cdot 2^2 = 2^{6+4}=2^{10}=1024$$

Esercizi

5. Considera il seguente algoritmo di ordinamento su interi

- scorro la sequenza in input
- se rilevo che $k \leq n$ lancio l'algoritmo `BUCKET_SORT`
- altrimenti lancio l'algoritmo `MERGE_SORT`

L'algoritmo è ben formulato? Può essere implementato?

Qual è la complessità dell'algoritmo?

Soluzioni

1. La complessità è $\Theta(n \log n)$ nel caso peggiore
3. L'algoritmo non è ben formulato perché non c'è modo di stabilire se una singola cifra k sia $O(n)$
4. Non è corretto fare considerazioni di complessità asintotica su una sola istanza. Per stabilire se il valore massimo k contenuto nell'array è $O(n)$ o meno occorre considerare tutte le istanze su cui si intende lanciare l'algoritmo
5. L'algoritmo è ben formulato. Se lanciato su insiemi di numeri generici la sua complessità è $\Theta(n \log n)$ nel caso peggiore; se lanciato su insiemi di n numeri il cui valore massimo è n la sua complessità è $\Theta(n)$

Algoritmi e Strutture di Dati

Tabelle Hash

m.patrignani

Nota di copyright

- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Sommario

- Richiami sui tipi astratti di dato
 - array associativo
 - insieme
- Tabelle hash
 - collisioni e liste di trabocco
 - uso per la realizzazione di tipi astratti di dato
- Funzioni hash
 - per interi, per stringhe, per oggetti

Array associativi

- Gli array associativi sono tipi astratti di dato costituiti da coppie <chiave, valore>
 - le operazioni che vogliamo fare sono l'inserimento di una nuova coppia, la cancellazione di una specifica chiave e la ricerca del valore corrispondente ad una chiave
- Possono essere realizzati con coppie di array, liste, alberi binari di ricerca, alberi rosso-neri
- Le realizzazioni di array associativi si prestano anche a realizzare insiemi generici
 - basta omettere il valore e tenere solo la chiave
 - le operazioni supportate sono inserisci elemento, cancella elemento e verifica esistenza elemento

Realizzazioni efficienti di array associativi

- La realizzazione più efficiente che conosciamo degli array associativi utilizza alberi rosso-neri
 - garantisce un tempo $\Theta(\log n)$ per la ricerca, l'inserimento e la cancellazione nel caso peggiore e nel caso medio
 - occorre che sulle chiavi siano definite le funzioni MINORE e UGUALE
- Tutti gli algoritmi di ricerca basati su confronti hanno complessità $\Omega(\log n)$ nel caso peggiore
 - la ricerca è l'operazione più frequente e critica

Tabelle hash

- Le tabelle hash realizzano array associativi con le seguenti caratteristiche
 - gli algoritmi di ricerca, inserimento e cancellazione non sono basati su confronti
 - le ricerche, gli inserimenti e le cancellazioni avvengono con un tempo $\Theta(n)$ nel caso peggiore
 - peggiorativo rispetto agli alberi rosso-neri
 - le ricerche, gli inserimenti e le cancellazioni avvengono con un tempo $\Theta(1)$ nel caso medio
 - migliorativo rispetto agli alberi rosso-neri
 - la struttura di dati utilizzata è effettivamente un singolo array

Difficoltà da superare nella realizzazione

- Due difficoltà principali

1. la tipologia delle chiavi

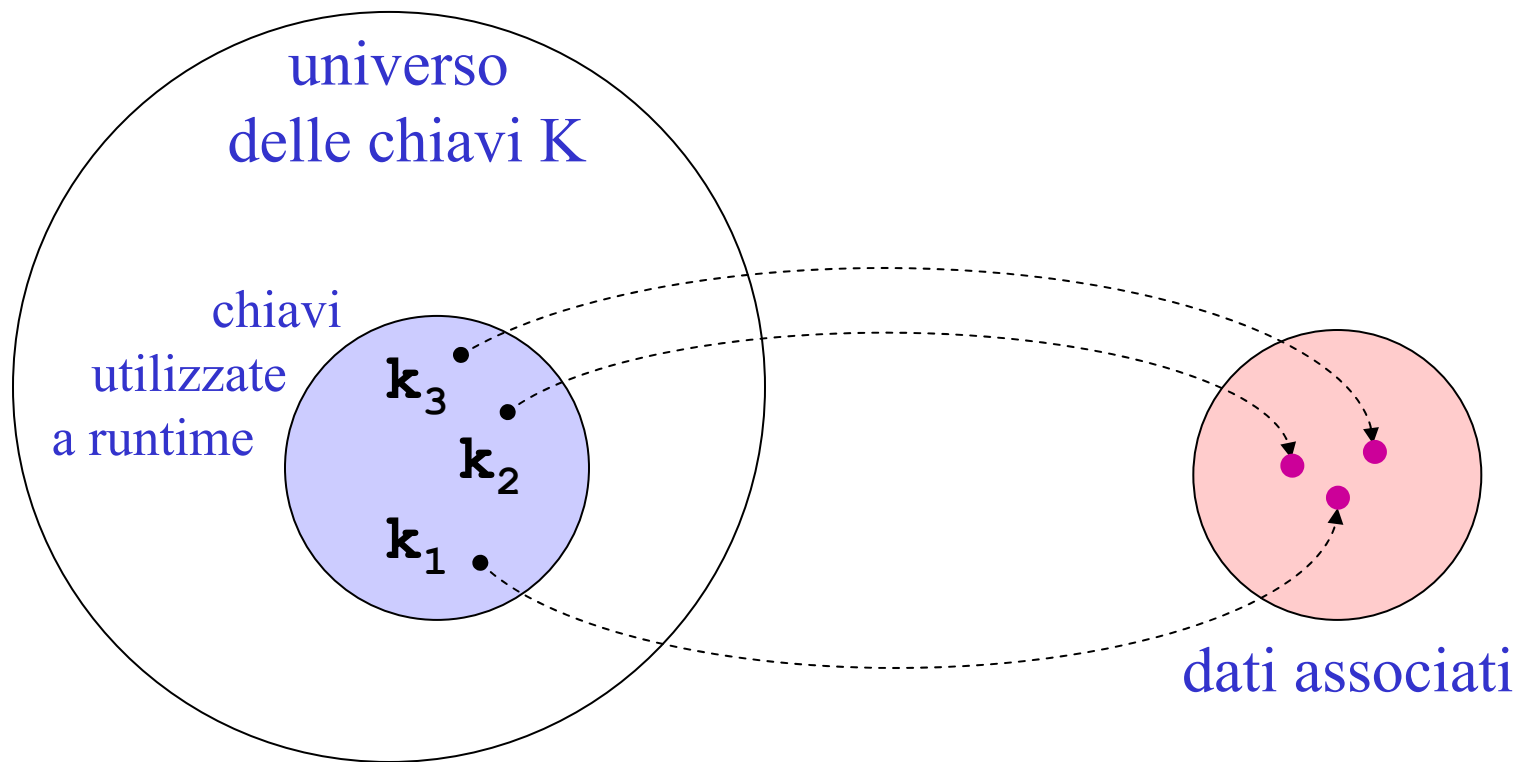
- le chiavi non sono necessariamente degli interi
- non possiamo fidare nelle chiavi per indicizzare direttamente un array

2. la numerosità delle possibili chiavi

- anche se le chiavi fossero degli interi, un array in grado di contenere tutte le chiavi sarebbe troppo grande e troppo sparso
 - per esempio se la chiave fosse un numero di matricola di sei cifre dovrei allocare un array con un milione di posizioni anche se gli studenti del corso che voglio considerare sono solo qualche centinaio

Array associativi: osservazione

- Il numero di chiavi effettivamente utilizzate dal programma in esecuzione è molto minore del numero delle chiavi possibili
 - quest'ultimo è chiamato “universo” delle chiavi K

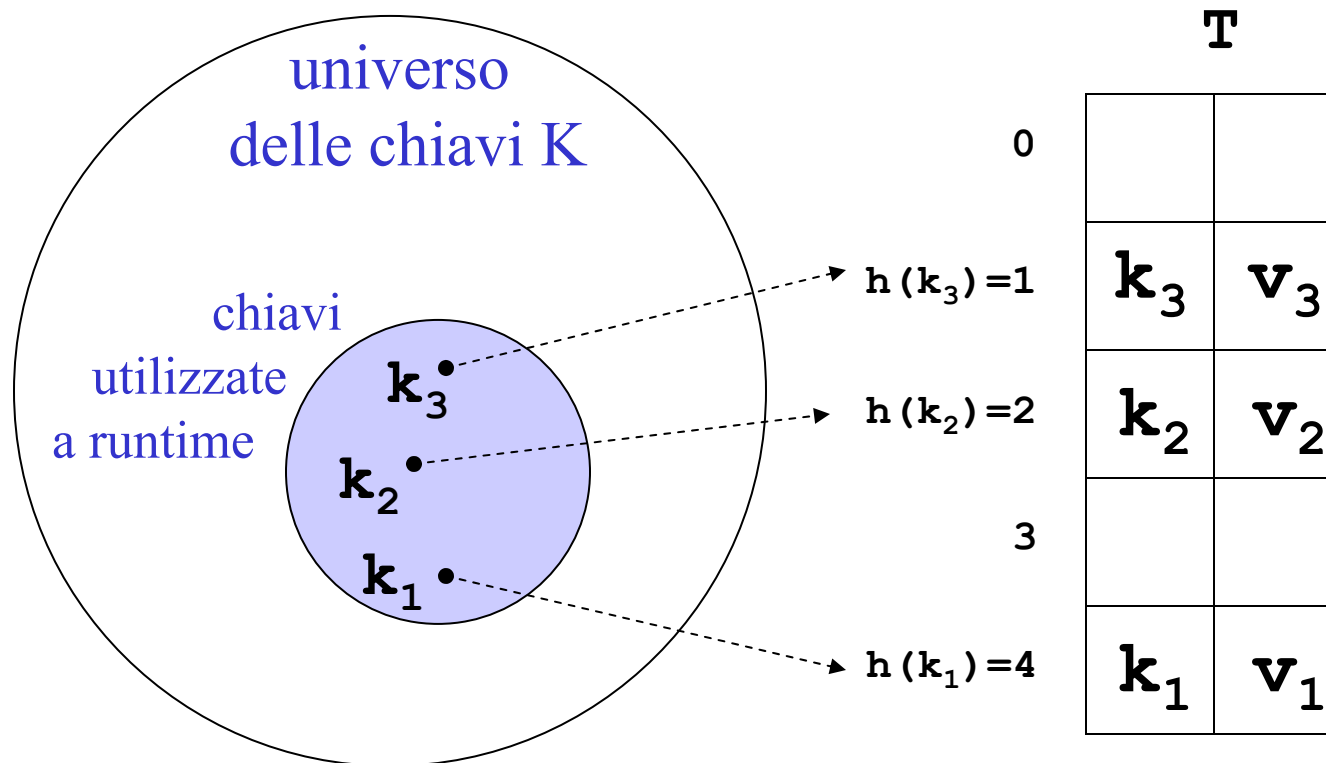


Realizzazione tramite tabelle hash

1. Utilizzo un array $\mathbf{T}$ per memorizzare i dati associati
 - la dimensione m dell'array $\mathbf{T}$
 - è molto minore della dimensione dell'universo K
 - è molto vicina al numero delle chiavi effettivamente utilizzate dal programma in esecuzione
 - l'array $\mathbf{T}$, come tutti gli array, può essere indicizzato solo da un intero
2. Definisco una funzione hash h che trasforma le chiavi di K negli interi nel range $[0 \dots m-1]$

Tabella hash

- L'array **T** indicizzato tramite la funzione hash h è chiamato *tabella hash* (oppure *hashtable*)

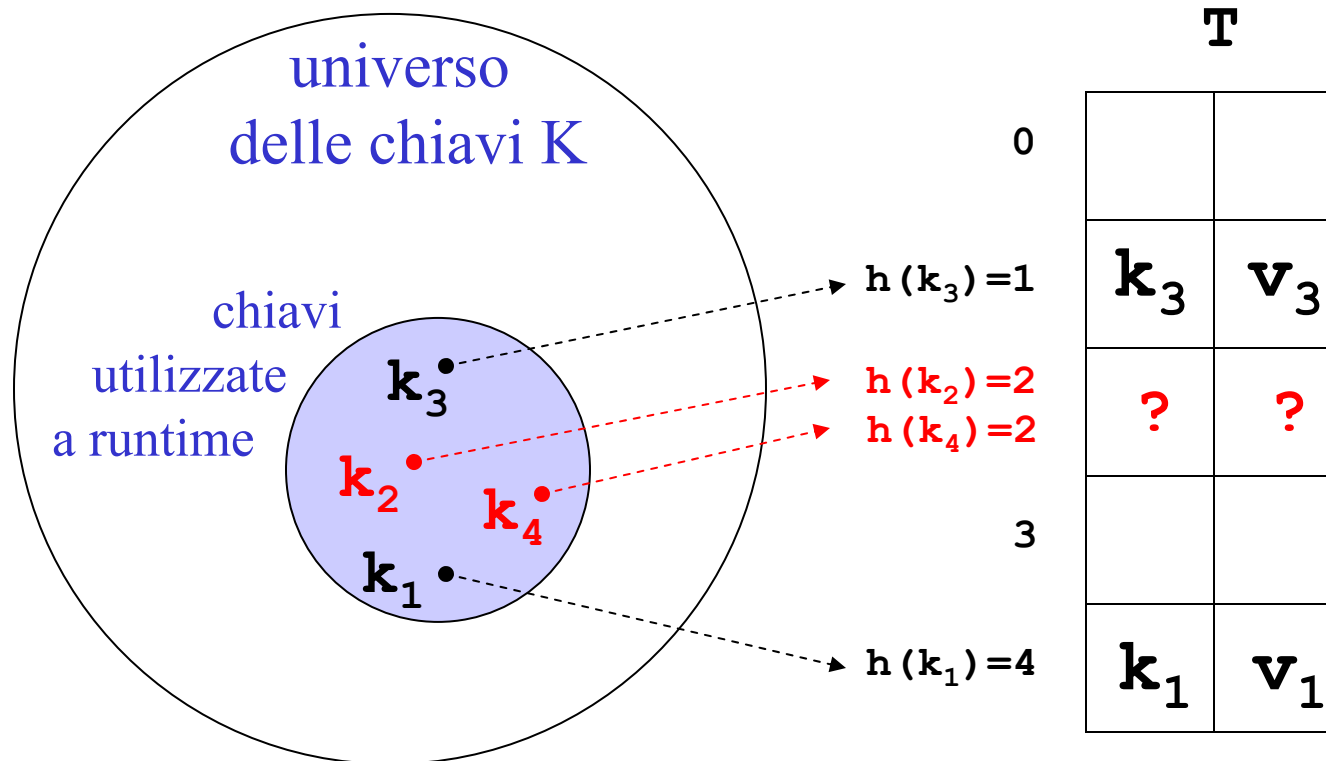


Funzione hash

- La funzione hash h
 - definisce una corrispondenza tra l'universo K delle chiavi e gli indici della tabella hash $\mathbf{T}$ $[0 \dots m-1]$
$$h: K \rightarrow \{0, 1, \dots, m-1\}$$
 - deve essere deterministica
 - altrimenti dopo aver messo i valori nell'array non riesco più a ritrovare la loro posizione
 - si richiede che sia calcolabile in tempo costante
 - per contenere i tempi di calcolo
- L'elemento con chiave $k \in K$ si troverà nella posizione $h(k)$ nella tabella $\mathbf{T}$

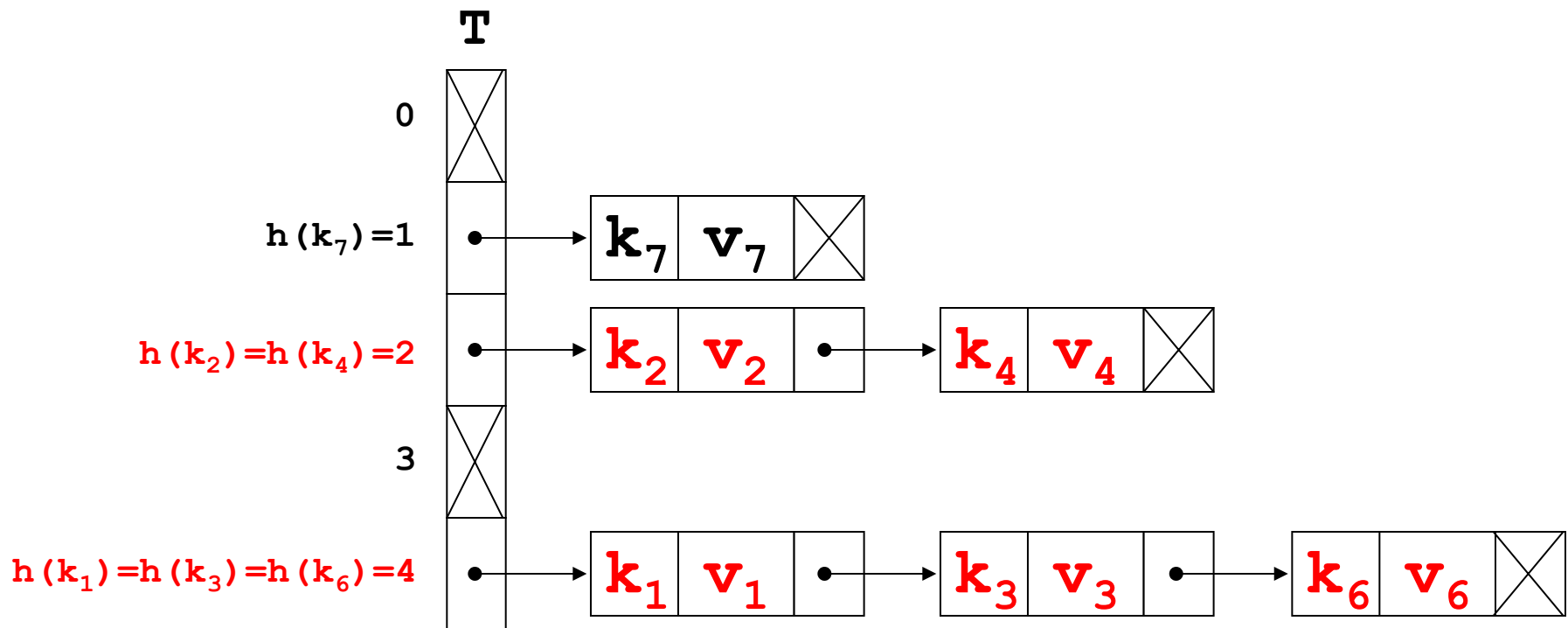
Il problema della collisione

- La funzione h ha un codominio (gli m indici di $\mathbf{T}$) molto più piccolo del dominio (tutti gli elementi di K)
 - è inevitabile che si generino collisioni



Gestione delle collisioni con liste di trabocco

- Ogni posizione di **T** è un riferimento al primo elemento di una lista detta “di trabocco”



Liste di trabocco

- La lista di trabocco è una lista semplicemente concatenata
- Ogni nodo della lista è un oggetto con tre campi
 - `key`: valore della chiave
 - può essere un riferimento ad oggetto
 - `info`: valore associato alla chiave
 - può essere un riferimento ad oggetto
 - `next`: riferimento al prossimo nodo
 - è NULL per l'ultimo nodo della lista

Ricerca di un elemento in base alla chiave

- Per ricercare un elemento devo scorrere la lista di trabocco opportuna

```
GET (T,k) // ritorna il valore associato alla chiave (o NULL)
1.  i = HASH(k)    // devo guardare la lista i-esima
2.  x = T[i]       // iteratore per elementi della lista T[i]
3.  while x != NULL
4.      if EQUAL(k,x.key)
5.          return x.info    // l'ho trovato!
6.      x = x.next
7.  return NULL           // non l'ho trovato
```

Inserimento di una coppia $\langle \text{chiave}, \text{valore} \rangle$

- Anche l'inserimento prevede una ricerca

```
PUT (T,k,v) // inserisce <k,v> (eventualmente sovrascrive)
1. i = HASH(k) // devo cercare nella lista i-esima di T
2. x = T[i] // iteratore per elementi della lista T[i]
3. while x != NULL
4.     if EQUAL(x.key,k)
5.         x.info = v // sovrascrivo il vecchio valore
6.         return // ho finito ed esco
7.     x = x.next
8. y.key = k // y nuovo elemento della lista
9. y.info = v
10. y.next = T[i]
11. T[i] = y // inserimento in testa
```

Cancellazione di un elemento

- L'elemento viene rimosso in base alla chiave

```
DELETE(T,k) // rimuove l'elemento (se esistente)
1. i = HASH(k) // devo cercare nella lista i-esima di T
2. x = T[i] // iteratore per elementi della lista T[i]
3. prev = NULL // punterà all'elemento che precede x
4. while x != NULL
5.     if EQUAL(x.key,k) // l'ho trovato
6.         if prev == NULL // x è il primo della lista
7.             T[i] = x.next
8.         else // non è il primo della lista
9.             prev.next = x.next // lo saltiamo
10.        return // ho finito ed esco
11.    prev = x // non trovato, provo il prossimo
12.    x = x.next
```

Funzione EQUAL e funzione HASH

- Una preconditione perché si possa realizzare un array associativo con hashtable è che siano definite
 - una funzione EQUAL
 - una funzione HASH
- Entrambe le funzioni devono essere definite in base al contesto applicativo
- Per motivi di efficienza si richiede generalmente che entrambe le funzioni siano calcolabili in tempo costante
 - rispetto al numero n degli elementi nell'hashtable

Una funzione EQUAL errata

- Supponi che le chiavi siano stringhe
 - per esempio realizzate tramite array di caratteri
- La funzione EQUAL seguente è errata:

```
EQUAL_WRONG(A,B)    // A e B sono due array di caratteri  
1.  return A == B
```

- in questo modo non vengono confrontati i valori contenuti negli array A e B, ma i loro riferimenti
 - cioè gli indirizzi, che sono necessariamente diversi anche quando le due stringhe sono uguali

Una funzione EQUAL corretta

- La funzione seguente è una funzione corretta per questo contesto applicativo

```
EQUAL(A,B) // A e B sono due array di caratteri
1. if A.length != B.length
2.     return FALSE           // lunghezza diversa
3. for i=0 to A.length-1
4.     if A[i] != B[i]
5.         return FALSE      // almeno un carattere diverso
6. return TRUE
```

- questa volta vengono confrontati tutti i caratteri delle due stringhe
 - la complessità della procedura è ancora $\Theta(1)$ se le stringhe hanno una dimensione massima nota e indipendente da n

HASH: requisiti

- Requisiti funzionali
 - è deterministica
 - data una chiave k , dà sempre lo stesso risultato $\text{HASH}(k)$
- Requisiti prestazionali
 - è calcolabile in tempo costante
 - distribuisce le chiavi utilizzate in esecuzione in maniera pseudocasuale nell'intervallo $[0 \dots m-1]$
 - questo requisito potrà solo essere soddisfatto solo in modo probabilistico
 - le chiavi che saranno utilizzate dall'utente in esecuzione non sono note a priori
 - comunque si scelga la funzione HASH esisterà sempre un insieme di chiavi che corrispondono alla stessa casella di T

HASH: ipotesi di distribuzione uniforme

- E' soddisfatta dalla funzione HASH quando, data una chiave $k \in K$, la probabilità che $\text{HASH}(k)=c$ sia la stessa per ogni casella $c \in [0 \dots m-1]$ di T
 - indipendentemente da quali altre chiavi siano state già inserite in T
- Implicazioni dell'ipotesi di distribuzione uniforme
 - per chiavi simili vengono generati hash diversi
 - spesso le chiavi utilizzate sono molto simili
 - quali che siano le chiavi utilizzate, queste vengono con alta probabilità distribuite uniformemente nell'intervallo $[0 \dots m-1]$

Fattore di carico

- Si definisce *fattore di carico* il rapporto α tra il numero n di elementi memorizzati e il numero m di posizioni disponibili
$$\alpha = \frac{n}{m}$$
- α è il numero medio di elementi memorizzati in ogni lista concatenata
- A seconda del valore di α abbiamo
 - $\alpha < 1$ molte posizioni disponibili rispetto agli elementi memorizzati
 - $\alpha = 1$ il numero di elementi corrisponde al numero delle posizioni disponibili
 - $\alpha > 1$ molti elementi da memorizzare rispetto al numero delle posizioni disponibili

Complessità delle operazioni

- Caso migliore

- l'operazione è eseguita su una lista di trabocco vuota o con un solo elemento
- la complessità dell'operazione è data dalla complessità di HASH oppure di $\text{HASH} + \text{EQUAL}$
 - complessità $\Theta(1)$

- Caso peggiore

- tutte le chiavi utilizzate corrispondono alla stessa posizione
- la complessità coincide con quella che si ha per il calcolo di $\text{HASH}(k)$ + la ricerca in una lista con n posizioni + il calcolo di EQUAL per n volte
 - complessità $\Theta(1) + \Theta(n) + \Theta(n) = \Theta(n)$

Complessità nel caso medio

- Caso medio
 - se la funzione HASH distribuisce le chiavi in modo uniforme nell'intervallo $[0 \dots m-1]$
 - la lunghezza attesa delle liste di trabocco coincide con la lunghezza media α
 - le operazioni hanno complessità $\Theta(\alpha)$
 - se α non supera mai una soglia fissata $\alpha_{\max}$ la complessità di ogni operazione è $\Theta(1)$
 - se la funzione HASH non dà garanzie rispetto alla distribuzione delle chiavi
 - la complessità è la stessa del caso peggiore

Note sulla complessità nel caso medio

- Abbiamo stabilito che nel caso medio la complessità delle operazioni sulle tabelle hash è $\Theta(\alpha)$
 - dove α è per definizione $\alpha = n/m$
- Dunque sembrerebbe che $\alpha \in \Theta(n)$
- Questo vorrebbe dire che, anche nel caso medio, la complessità delle operazioni è $\Theta(\alpha) = \Theta(n)$
 - cioè lineare come nel caso delle liste
 - non si avrebbe nessun vantaggio dall'adozione delle tabelle hash
- In che cosa sono errate le considerazioni qui sopra?

Note sulla complessità nel caso medio

- E' errato considerare $\alpha \in \Theta(n)$ a partire dalla definizione $\alpha = n/m$
- Anche m , infatti, cambia al crescere di n
 - quando il fattore di carico α supera una determinata soglia la dimensione m della tabella viene raddoppiata
 - il fattore di carico α viene dimezzato
 - il costo medio delle operazioni è ancora costante in virtù del fatto che il costo (lineare) del raddoppio della tabella viene assorbito dai precedenti inserimenti eseguiti in tempo costante
 - vedi slides sulla gestione telescopica delle pile e sulla complessità ammortizzata
- Una realizzazione delle tabelle hash che non preveda la gestione telescopica della tabella non può garantire tempi costanti di accesso

Funzioni hash

Funzioni hash

- Considereremo delle funzioni hash per le seguenti tipologie di chiavi
 - funzioni hash per interi
 - metodo della divisione
 - veloce ma raramente adottato
 - metodo della moltiplicazione
 - funzioni hash per stringhe
 - funzioni hash per oggetti arbitrari

Metodo della divisione: MOD_HASH

- Utilizza il resto di una divisione intera

$$h(k) = k \bmod m$$

- In pseudocodice:

```
MOD_HASH(k, m)          // k ed m sono interi  
1. return k mod m
```

- E' un metodo molto veloce
- Se le chiavi sono già degli interi pseudocasuali la funzione MOD_HASH viene utilizzata per riportare le chiavi nell'intervallo $[0 \dots m-1]$

Metodo della divisione: MOD_HASH

- Se le chiavi non sono pseudocasuali
 - la praticità del metodo è compromessa
 - MOD_HASH ha delle forti proprietà di località
 - con altissima probabilità $\text{MOD_HASH}(k+1) = \text{MOD_HASH}(k)+1$
 - se m è una potenza di 2 o di 10, $\text{MOD_HASH}(k)$ produce la parte meno significativa del numero k espresso in quella base
 - se si vuole usare questo metodo, è raccomandabile adottare come m un numero primo lontano da una potenza di due per limitare le collisioni
 - per esempio 701

Metodo della moltiplicazione: osservazione 1

- Supponiamo che le chiavi siano numeri reali pseudocasuali nell'intervallo $(0,1)$
- La funzione

$$h(k) = \lfloor m \cdot k \rfloor$$

è una buona funzione hash

- $h(k)$ è deterministica
- $h(k)$ può essere calcolata in $\Theta(1)$
- $h(k)$ distribuisce uniformemente le chiavi nell'intervallo $[0 \dots m-1]$
 - in quanto le chiavi erano già uniformemente distribuite!

Metodo della moltiplicazione: osservazione 2

- Sia irr un numero irrazionale
 - irr ha infinite cifre dopo la virgola, ma non è periodico
- Date delle chiavi intere qualsiasi, le cifre decimali dopo la virgola del prodotto $k \cdot irr$ si possono assumere uniformemente distribuite nell'intervallo $(0,1)$
 - questo valore coincide con $k \cdot irr - \lfloor k \cdot irr \rfloor$
 - Knuth propone

$$irr = \frac{\sqrt{5}-1}{2} = 0.6180339...$$

- è la parte dopo la virgola della sezione aurea

$$\varphi = \frac{\sqrt{5}+1}{2} = 1.6180339...$$

- è un numero irrazionale

Metodo della moltiplicazione

- Si utilizza come hash la parte intera di un prodotto

$$h(k) = \lfloor m \cdot (k \cdot irr - \lfloor k \cdot irr \rfloor) \rfloor$$

- Dove
 - irr è un numero irrazionale in $(0,1)$
 - per esempio la parte dopo la virgola della sezione aurea φ definita nella slide precedente
 - m può essere scelto arbitrariamente
 - di solito si usa una potenza di due: $m=2^p$, dove p è un intero

Metodo della moltiplicazione: MUL_HASH

- La costante *irr* viene calcolata una volta sola

```
1. a.irr = (SQRT(5)-1)*0.5    // a.irr costante irrazionale
```

- La funzione MUL_HASH riceve in input l'array associativo *a* (per avere *m* ed *irr*) e la chiave *k*

```
MUL_HASH(a, k)                // a array associativo, k intero
1. m = a.T.length              // m è un numero intero
2. prod = k * a.irr            // prod è un numero reale
3. prod = m * (prod - INT(prod)) // reale in (0,m)
4. out = INT(prod)              // intero in [0,m-1]
5. return out
```

– la funzione INT tronca un reale all'intero inferiore

HASH per stringhe: SIMPLE_HASH

```
SIMPLE_HASH(S)          // S è una stringa (array di caratteri)
1. hash = 0
2. for i = 0 to S.length-1
3.     hash = hash + ASCII(S[i])
4. return hash
```

- Ritorna un numero intero che poi deve essere ridotto nell'intervallo $[0 \dots m-1]$ con la funzione MOD_HASH
- Introdotta nella prima edizione del Kernigham-Ritchie
- Veloce ma generalmente considerata poco efficace nel distribuire i valori in modo pseudocausuale
 - permutazioni di caratteri hanno lo stesso hash!

HASH per stringhe: DJB2_HASH

```
DJB2_HASH(S) // S è una stringa (array di caratteri)
```

```
1. hash = 5381
```

```
2. for i = 0 to S.length-1
```

```
3.     hash = hash*33 + ASCII(S[i])
```

```
4. return hash
```

- Introdotta da Daniel J. Bernstein (dove il nome)
- Il numero 5381 è un numero primo
- Il “magic number” 33 non è giustificato teoricamente
 - ma dà ottimi risultati nella pratica
 - corrisponde al prodotto * 32 (traslazione di cinque caselle della rappresentazione) + un incremento di uno
 - può essere realizzato velocemente

Funzioni hash per oggetti

- Supponiamo che
 - l'oggetto abbia come chiave h campi $c_1, c_2, \dots, c_h$
 - siano già definite opportune funzioni hash
 $\text{HASH}_1(c_1), \text{HASH}_2(c_2), \dots, \text{HASH}_h(c_h)$
- Una funzione hash si può ottenere facilmente con la loro somma

$$\text{HASH}(o) = \text{HASH}_1(o.c_1) + \text{HASH}_2(o.c_2) + \dots + \text{HASH}_h(o.c_h)$$

- Il risultato può essere riportato nell'intervallo opportuno tramite MOD_HASH

Insiemi

- Un insieme è una collezione di elementi omogenei
- Esempi di insieme
 - l'insieme degli studenti
 - l'insieme degli oggetti creati da un programma
 - l'insieme delle variabili utilizzate da un programma

Il tipo astratto di dato insieme

- Domini

- il dominio di interesse è l'insieme I degli insiemi
- dominio di supporto: gli elementi E dell'insieme
- dominio di supporto: i booleani $B = \{\text{true}, \text{false}\}$

- Costanti

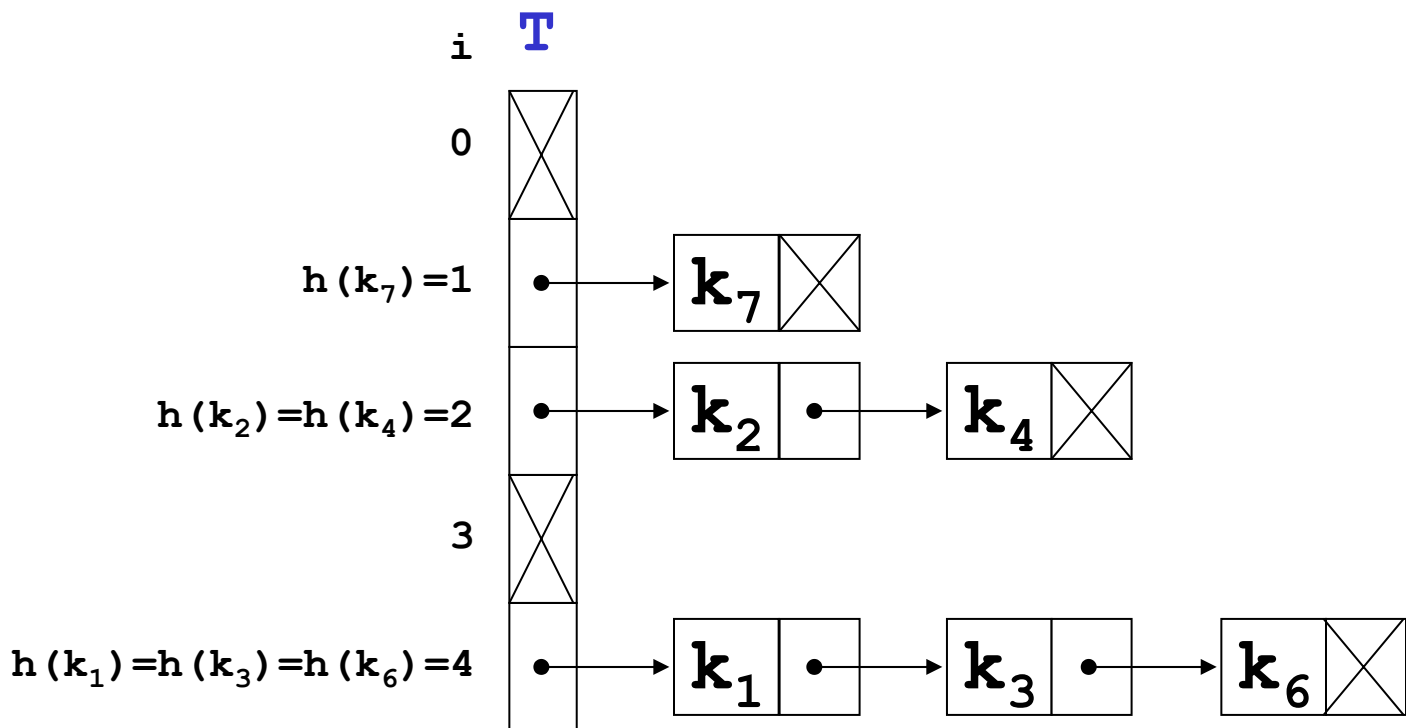
- l'insieme vuoto

- Operazioni

- aggiunge un elemento: $\text{ADD}: I \times E \rightarrow I$
- elimina un elemento dall'insieme: $\text{REMOVE}: I \times E \rightarrow I$
- verifica l'appartenenza: $\text{CONTAINS}: I \times E \rightarrow B$
- ...

Realizzazione di un insieme

- Si può usare una hashtable in cui lo stesso elemento funge da valore e da chiave



Problemi

1. Illustra l'inserimento in una tabella hash di dimensione $m=10$ gestita con liste di trabocco delle chiavi 32, 17, 19, 31, 33, 15, 38, 46, utilizzando la funzione hash MOD-H
2. Scrivi lo pseudocodice delle funzioni $\text{ADD}(I,e)$, $\text{REMOVE}(I,e)$ e $\text{CONTAINS}(I,e)$ dove I è un insieme realizzato tramite una hashtable ed e è un elemento dell'insieme
 - assumi che siano definite opportune funzioni $\text{HASH}(e)$ e $\text{EQUAL}(e_1,e_2)$

Problemi

1. Quali sono le prestazioni delle tabelle hash se al posto di una lista semplicemente concatenata usiamo come trabocco un albero rosso-nero?

Algoritmi e Strutture di Dati

Tabelle hash con indirizzamento aperto

m.patrignani

Nota di copyright

- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Contenuto

- Tabelle hash ad indirizzamento aperto
- Tecniche di scansione
 - scansione lineare
 - scansione quadratica
 - doppio hashing

Tabelle hash con liste di trabocco

- Nelle tabelle hash con liste di trabocco le collisioni vengono gestite utilizzando memoria aggiuntiva

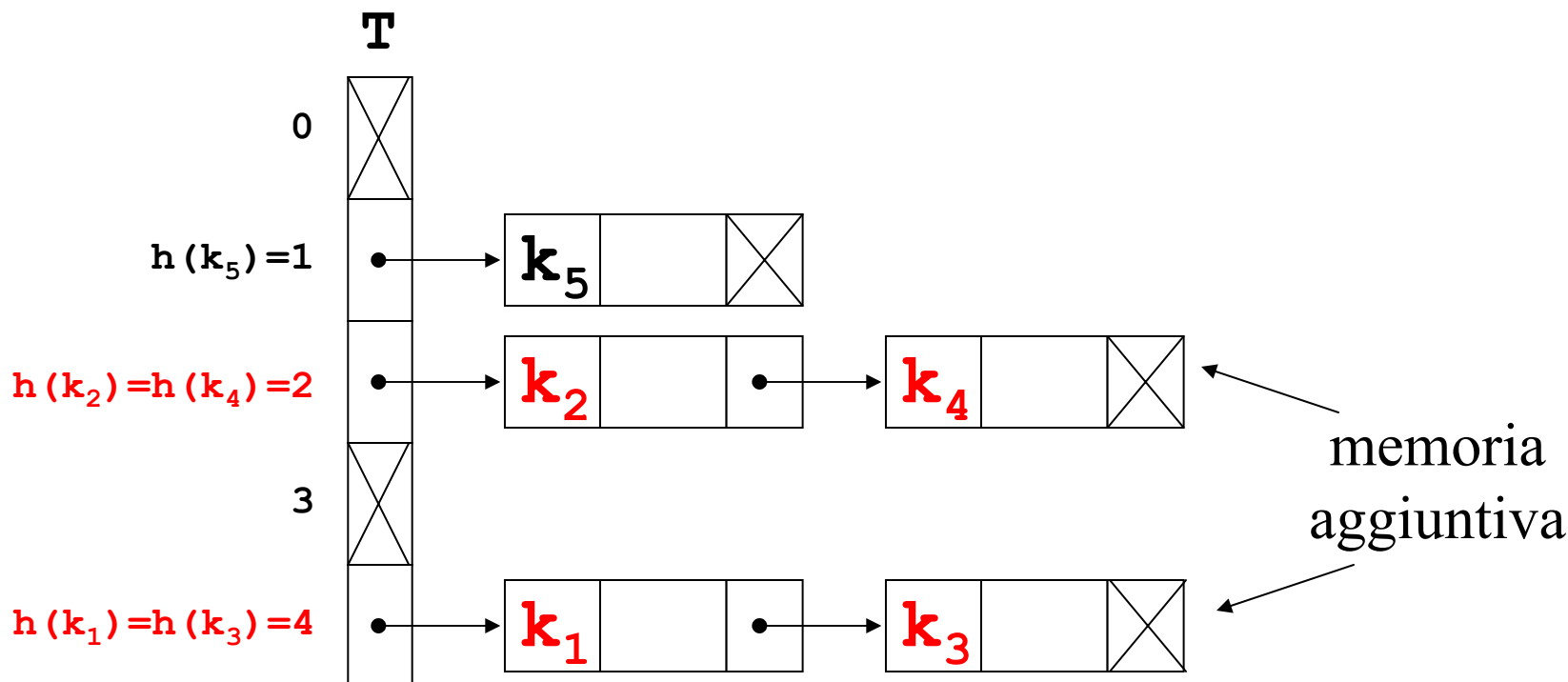


Tabelle hash con indirizzamento aperto

- Nelle tabelle hash con indirizzamento aperto le collisioni vengono gestite senza ricorrere a memoria aggiuntiva

T

0			
$h(k_5) = 1$	k_5	v_5	
$h(k_2) = h(k_4) = 2$	k_2	v_2	← dove memorizzo k_4 ?
3			
$h(k_1) = h(k_3) = 4$	k_1	v_1	← dove memorizzo k_3 ?

Tabelle hash con indirizzamento aperto

- Le collisioni vengono gestite memorizzando gli elementi in altri punti della tabella hash stessa
 - se una posizione è occupata se ne calcola un'altra
- Il fattore di carico α non potrà mai superare 1
 - l'inserimento di $m+1$ valori genera un'errore di overflow o un raddoppio telescopico della tabella
- Si può implementare senza far uso di riferimenti
 - utilizzando meno memoria rispetto alla rappresentazione con le liste concatenate

Collisioni in caso di indirizzamento aperto

- In caso di collisione con la chiave k si ispeziona la tabella alla ricerca di una cella libera per k
- La tabella è usata come un sequenza circolare
 - la cella $T[m-1]$ è seguita dalla cella $T[0]$
- Le celle potrebbero essere scandite banalmente in modo sequenziale

$$h(k), h(k)+1, h(k)+2, \dots$$

tuttavia si preferisce far dipendere anche l'ordine di scansione dal valore di k

Funzione di scansione $h(k,i)$

- Per generalità si definisce una funzione $h(k,i)$ detta *funzione di scansione* che determina la posizione associata a k , dopo che gli i tentativi precedenti hanno generato una collisione
 - si calcola prima $h(k,0)$
 - se la cella $h(k,0)$ è occupata si calcola $h(k,1)$
 - se la cella $h(k,1)$ è occupata si calcola $h(k,2)$
 - ...
- Si richiede che la funzione di scansione $h(k,i)$ abbia tutti valori diversi per $0 \leq i \leq m-1$
 - ne segue che $\langle h(k,0), h(k,1), \dots, h(k,m-1) \rangle$ è una permutazione di $\langle 0, 1, \dots, m-1 \rangle$

Esempio di funzione di scansione

1. Inserisco $\mathbf{k}_1$ in $h(\mathbf{k}_1, 0)$
2. Inserisco $\mathbf{k}_2$ in $h(\mathbf{k}_2, 0)$

T	
0	
1	
$h(\mathbf{k}_2, 0) = 2$	$\mathbf{k}_2 \quad \mathbf{v}_2$
3	
$h(\mathbf{k}_1, 0) = 4$	$\mathbf{k}_1 \quad \mathbf{v}_1$

Esempio di funzione di scansione

1. Inserisco $\mathbf{k}_1$ in $h(\mathbf{k}_1, 0)$
2. Inserisco $\mathbf{k}_2$ in $h(\mathbf{k}_2, 0)$
3. Collisione su $h(\mathbf{k}_3, 0)$

T	
0	
1	
$h(\mathbf{k}_2, 0) = 2$	$\mathbf{k}_2 \quad \mathbf{v}_2$
3	
$h(\mathbf{k}_3, 0) = h(\mathbf{k}_1, 0) = 4$	$\mathbf{k}_1 \quad \mathbf{v}_1$

Esempio di funzione di scansione

1. Inserisco $\mathbf{k}_1$ in $h(\mathbf{k}_1, 0)$
2. Inserisco $\mathbf{k}_2$ in $h(\mathbf{k}_2, 0)$
3. Collisione su $h(\mathbf{k}_3, 0)$
4. Inserisco $\mathbf{k}_3$ in $h(\mathbf{k}_3, 1)$

T											
$h(\mathbf{k}_3, 1) = 0$	<table><tr><td>$\mathbf{k}_3$</td><td>$\mathbf{v}_3$</td></tr><tr><td></td><td></td></tr><tr><td>$\mathbf{k}_2$</td><td>$\mathbf{v}_2$</td></tr><tr><td></td><td></td></tr><tr><td>$\mathbf{k}_1$</td><td>$\mathbf{v}_1$</td></tr></table>	$\mathbf{k}_3$	$\mathbf{v}_3$			$\mathbf{k}_2$	$\mathbf{v}_2$			$\mathbf{k}_1$	$\mathbf{v}_1$
$\mathbf{k}_3$	$\mathbf{v}_3$										
$\mathbf{k}_2$	$\mathbf{v}_2$										
$\mathbf{k}_1$	$\mathbf{v}_1$										
1											
$h(\mathbf{k}_2, 0) = 2$											
3											
$h(\mathbf{k}_1, 0) = 4$											

Esempio di funzione di scansione

1. Inserisco $\mathbf{k}_1$ in $h(\mathbf{k}_1, 0)$
2. Inserisco $\mathbf{k}_2$ in $h(\mathbf{k}_2, 0)$
3. Collisione su $h(\mathbf{k}_3, 0)$
4. Inserisco $\mathbf{k}_3$ in $h(\mathbf{k}_3, 1)$
5. Collisione su $h(\mathbf{k}_4, 0)$

$$h(\mathbf{k}_3, 1) = 0$$

1

$$h(\mathbf{k}_4, 0) = h(\mathbf{k}_2, 0) = 2$$

3

$$h(\mathbf{k}_1, 0) = 4$$

T

$\mathbf{k}_3$	$\mathbf{v}_3$
$\mathbf{k}_2$	$\mathbf{v}_2$
$\mathbf{k}_1$	$\mathbf{v}_1$

Esempio di funzione di scansione

1. Inserisco $\mathbf{k}_1$ in $h(\mathbf{k}_1, 0)$
2. Inserisco $\mathbf{k}_2$ in $h(\mathbf{k}_2, 0)$
3. Collisione su $h(\mathbf{k}_3, 0)$
4. Inserisco $\mathbf{k}_3$ in $h(\mathbf{k}_3, 1)$
5. Collisione su $h(\mathbf{k}_4, 0)$
6. Collisione su $h(\mathbf{k}_4, 1)$

$$h(\mathbf{k}_4, 1) = h(\mathbf{k}_3, 1) = 0$$

1

$$h(\mathbf{k}_4, 0) = h(\mathbf{k}_2, 0) = 2$$

3

$$h(\mathbf{k}_1, 0) = 4$$

T	
$\mathbf{k}_3$	$\mathbf{v}_3$
$\mathbf{k}_2$	$\mathbf{v}_2$
$\mathbf{k}_1$	$\mathbf{v}_1$

Esempio di funzione di scansione

1. Inserisco $\mathbf{k}_1$ in $h(\mathbf{k}_1, 0)$
2. Inserisco $\mathbf{k}_2$ in $h(\mathbf{k}_2, 0)$
3. Collisione su $h(\mathbf{k}_3, 0)$
4. Inserisco $\mathbf{k}_3$ in $h(\mathbf{k}_3, 1)$
5. Collisione su $h(\mathbf{k}_4, 0)$
6. Collisione su $h(\mathbf{k}_4, 1)$
7. Inserisco $\mathbf{k}_4$ in $h(\mathbf{k}_4, 2)$

	T	
$h(\mathbf{k}_3, 1) = 0$	$\mathbf{k}_3$	$\mathbf{v}_3$
1		
$h(\mathbf{k}_2, 0) = 2$	$\mathbf{k}_2$	$\mathbf{v}_2$
$h(\mathbf{k}_4, 2) = 3$	$\mathbf{k}_4$	$\mathbf{v}_4$
$h(\mathbf{k}_1, 0) = 4$	$\mathbf{k}_1$	$\mathbf{v}_1$

Esempio di funzione di scansione

1. Inserisco $\mathbf{k}_1$ in $h(\mathbf{k}_1,0)$
2. Inserisco $\mathbf{k}_2$ in $h(\mathbf{k}_2,0)$
3. Collisione su $h(\mathbf{k}_3,0)$
4. Inserisco $\mathbf{k}_3$ in $h(\mathbf{k}_3,1)$
5. Collisione su $h(\mathbf{k}_4,0)$
6. Collisione su $h(\mathbf{k}_4,1)$
7. Inserisco $\mathbf{k}_4$ in $h(\mathbf{k}_4,2)$
8. Inserisco $\mathbf{k}_5$ in $h(\mathbf{k}_5,0)$

T	
$h(\mathbf{k}_3, 1) = 0$	$\mathbf{k}_3 \quad \mathbf{v}_3$
$h(\mathbf{k}_5, 0) = 1$	$\mathbf{k}_5 \quad \mathbf{v}_5$
$h(\mathbf{k}_2, 0) = 2$	$\mathbf{k}_2 \quad \mathbf{v}_2$
$h(\mathbf{k}_4, 2) = 3$	$\mathbf{k}_4 \quad \mathbf{v}_4$
$h(\mathbf{k}_1, 0) = 4$	$\mathbf{k}_1 \quad \mathbf{v}_1$

Requisiti della funzione di scansione

- Nel seguito vedremo diverse funzioni di scansione
- Tutte le funzioni di scansione hanno la forma
$$h(k,i): U \times \underbrace{\{0, 1, \dots, m-1\}}_{\substack{\text{non ha senso lanciare} \\ h(k,i) \text{ con valori di } i \geq m}} \rightarrow \{0, 1, \dots, m-1\}$$
- Se tutti valori $h(k,0), h(k,1), \dots, h(k,m-1)$ generano una collisione allora si ha un errore di overflow

Inserimento di elementi

- Per semplicità supponiamo che il valore -1 per la chiave indichi una posizione libera

```
HASH_INSERT(T,k,v)    // k è la chiave, v è il valore
1.  i = 0                // primo tentativo
2.  repeat c = h(k,i)    // c = cella candidata
3.      if T[c].key == -1 // se la cella è libera
4.          T[c].key = k
5.          T[c].val = v
6.          return c
7.      else i = i + 1
8.  until i == m
9.  error("overflow sulla tabella hash")
```

Ricerca di elementi

- La sequenza di posizioni esaminata quando la chiave è stata inserita è esattamente la stessa sequenza esaminata quando la chiave viene ricercata
- La ricerca fallisce se una cella candidata viene trovata vuota

```
HASH_SEARCH(T,k)    // ritorna la posizione della chiave
1.  i = 0              // primo tentativo
2.  repeat c = h(k,i)  // c = cella candidata
3.    if T[c].key == k  // trovato l'elemento cercato
4.      return c
5.    i = i + 1
6.  until T[c].key == -1 or i == m
7.  return -1          // elemento non trovato
```

Cancellazione di un elemento

- Per le tabelle hash ad indirizzamento aperto la cancellazione è un'operazione critica
 - la cancellazione di un elemento, infatti, potrebbe interrompere le sequenze di scansione
 - le future ricerche di altri elementi potrebbero fallire
- Le tabelle hash con indirizzamento aperto si usano preferibilmente quando non sono previste cancellazioni

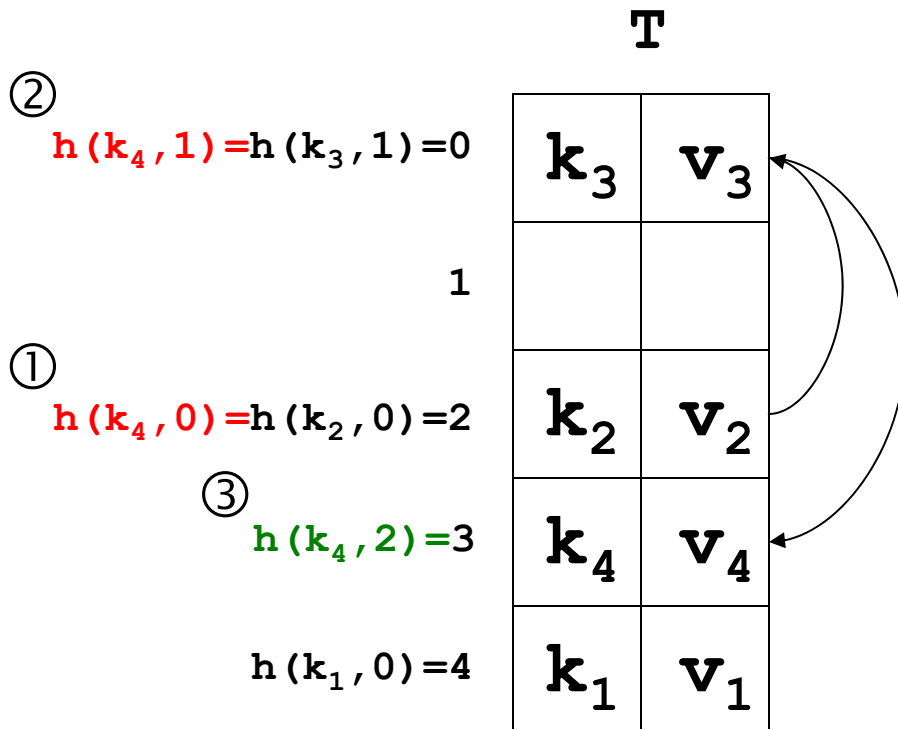
Strategia di cancellazione

- Quando si cancella una chiave la cella viene marcata con DELETED e non con NULL
- La sequenza di scansione durante l'inserimento si interrompe quando trova un NULL oppure un DELETED
 - in questo modo le celle marcate DELETED vengono riutilizzate
 - piccola modifica rispetto allo pseudocodice già visto
- La sequenza di scansione durante la ricerca si interrompe quando trova un NULL, ma continua quando trova un DELETED
 - non occorre modificare lo pseudocodice già visto

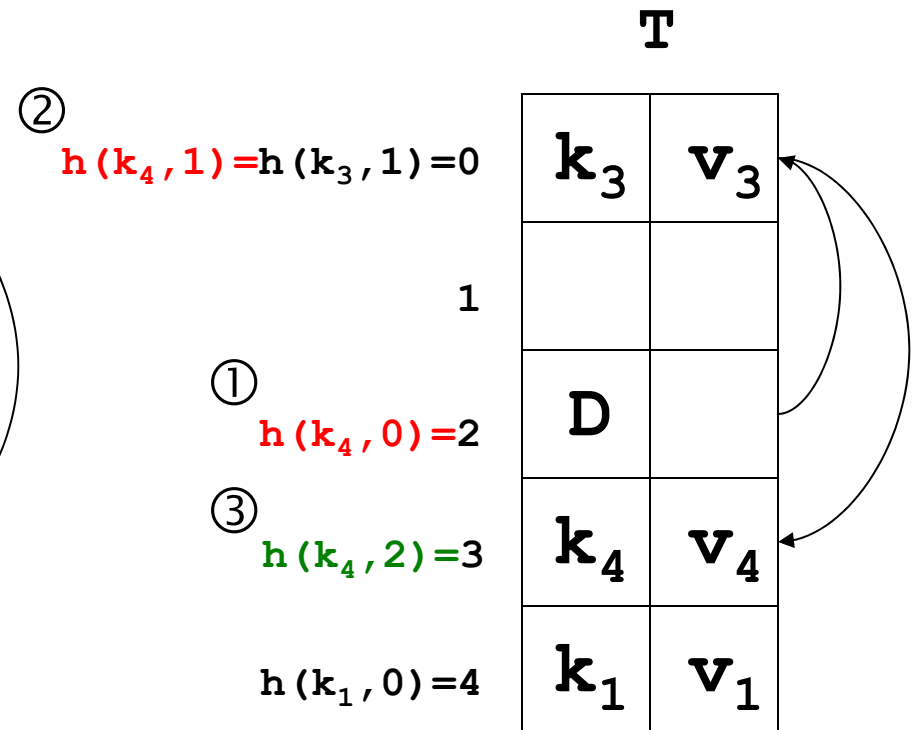
Effetto della cancellazione di elementi

- Ricerca di $\mathbf{k}_4$ prima e dopo la cancellazione di $\mathbf{k}_2$

prima della
cancellazione di $\mathbf{k}_2$



dopo la
cancellazione di $\mathbf{k}_2$



Complessità delle operazioni

- Non dimostreremo il seguente teorema
- Data una tabella hash con indirizzamento aperto e con fattore di carico $\alpha < 1$, il numero atteso di accessi in una ricerca è al massimo

$$\frac{1}{\alpha} \ln \left(\frac{1}{1-\alpha} \right)$$

- si assume hashing uniforme
- si assume che ogni chiave della tabella abbia la stessa probabilità di essere cercata
- se $\alpha=0,5$ ci aspettiamo 1.387 accessi
- se $\alpha=0,9$ ci aspettiamo 2.559 accessi

Scansione lineare

- Viene adottata la funzione di scansione:

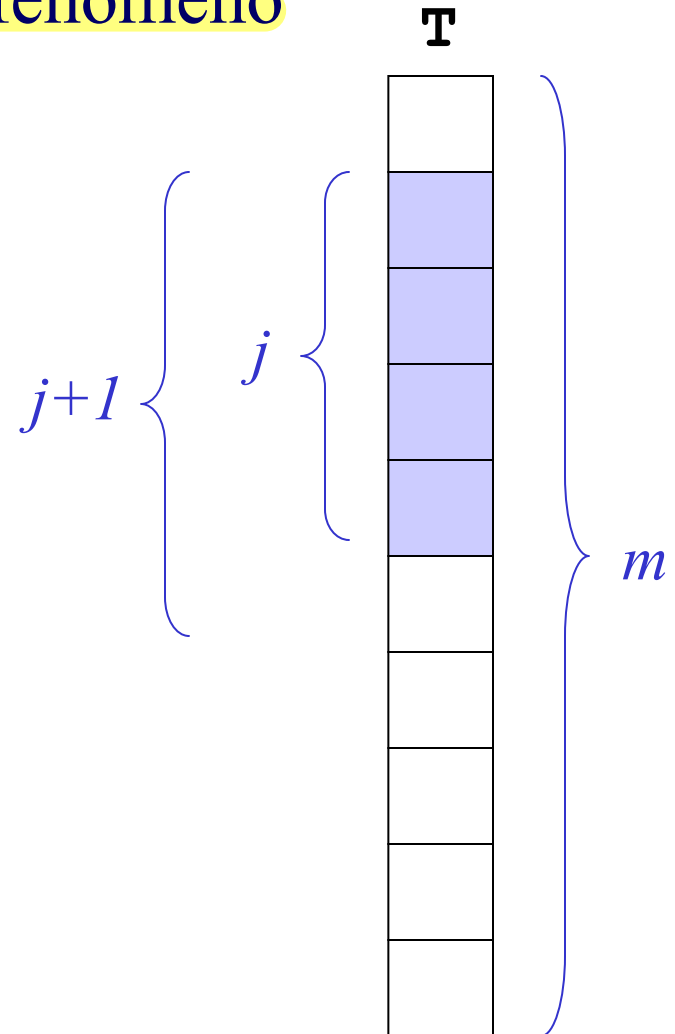
$$h(k,i) = (h(k)+i) \bmod m$$

- Il primo tentativo viene eseguito su $h(k)$, poi su $h(k)+1$, $h(k)+2$, $h(k)+3 \dots$
- E' evidentemente una funzione facile da implementare

Scansione lineare: addensamento primario

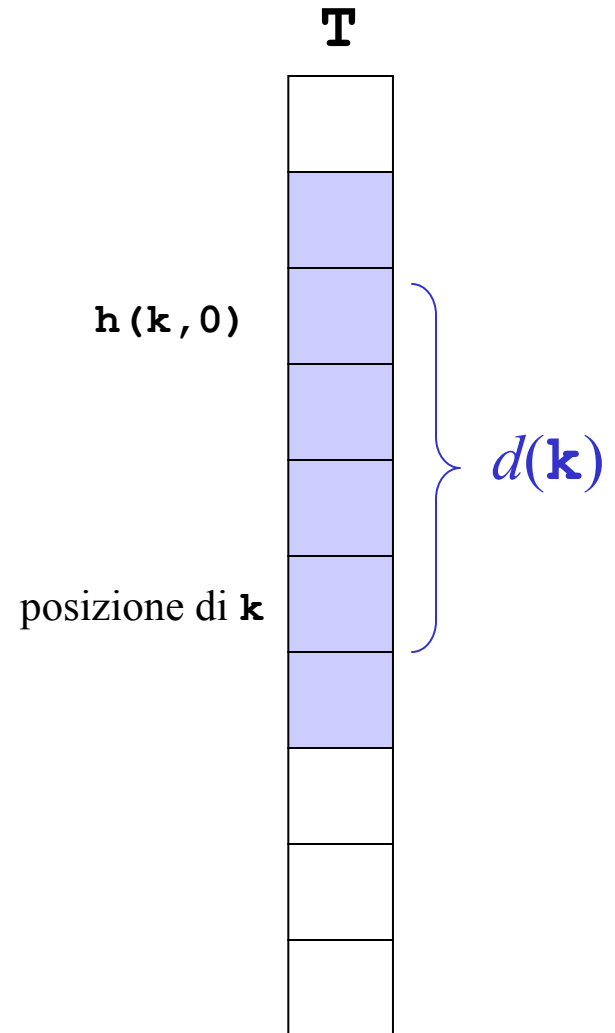
- La scansione lineare presenta il fenomeno dell'*addensamento primario*

- supponiamo che $h(k,0)$ distribuisca uniformemente le chiavi
 - ogni cella ha probabilità $1/m$ di essere la cella selezionata da $h(k,0)$
- una cella vuota preceduta da j celle piene ha probabilità $(j+1)/m$ di essere la prossima cella utilizzata
- le celle precedute da celle vuote hanno probabilità $1/m$ di essere le prossime celle utilizzate



Scansione lineare: addensamento primario

- Si formano lunghe sequenze di celle occupate
- La distanza media $d(k)$ tra $h(k,0)$ e l'effettiva posizione della chiave k aumenta
- Il tempo di ricerca di una chiave k è proporzionale a $d(k)$
 - il tempo medio della ricerca aumenta



Scansione quadratica

- Viene adottata la funzione di scansione:

$$h(k,i) = (h(k) + c_1i + c_2i^2) \bmod m$$

- Il primo tentativo viene eseguito sempre su $h(k)$
- Le costanti ausiliarie c_1 e $c_2 \neq 0$ devono essere scelte opportunamente
 - la scelta di c_1 e c_2 influenza la correttezza di $h(k,i)$

Scansione quadratica: parametri errati

- Proviamo ad applicare la formula

$$h(k,i) = (h(k) + c_1 i + c_2 i^2) \bmod m$$

usando come parametri: $m=12$, $c_1=1$, $c_2=1$

$$h(k,0)=h(k)$$

$$h(k,1)=h(k)+1+1 = h(k)+2$$

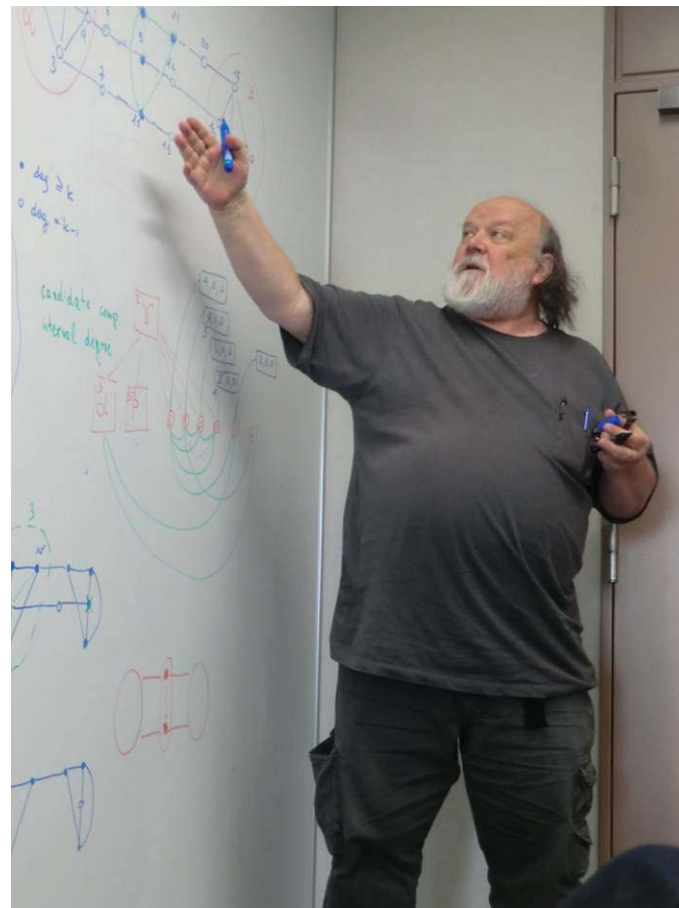
$$h(k,2)=h(k)+2+4 = h(k)+6$$

$$h(k,3)=h(k)+3+9 = h(k)+12 = h(k)$$

- Abbiamo ottenuto $h(k,0)=h(k,3)$
 - $h(k,i)$ non è una funzione di scansione legittima

Scansione quadratica: parametri corretti

- Se $m = 2^p$ una scelta possibile è $c_1 = c_2 = \frac{1}{2}$
 - tale scelta implica infatti che i valori di $h(k,i)$ siano tutti distinti per $i = 0, 1, \dots, m-1$
 - Vladimir Batagelj, “The quadratic hash method when the table size is not a prime number”, ACM, 1975



Vladimir Batagelj nel 2016

Dimostrazione che gli $h(k,i)$ sono distinti

- Assumi che esistano i e j tali che $i, j \in [0, m-1]$ e

$$(i + i^2)/2 = (j + j^2)/2 \pmod{m}$$

- Questo implica

$$i + i^2 = j + j^2 \pmod{2m}$$

- Da cui si ottiene

$$i^2 - j^2 + i - j = 0 \pmod{2m}$$

$$(i - j)(i + j) + (i - j) = 0 \pmod{2m}$$

$$(i - j)(i + j + 1) = 0 \pmod{2m}$$

- Poiché $2m$ è una potenza di 2 e solo uno dei due fattori può essere pari deve essere

$$i - j = 0 \pmod{2m} \text{ oppure}$$

$$i + j + 1 = 0 \pmod{2m}$$

- La seconda ipotesi è impossibile con $i, j \in [0, m-1]$
- La prima ipotesi implica $i = j$

Scansione quadratica: esempio

- Scegliamo $m = 2^3 = 8$ e $c_1 = c_2 = \frac{1}{2}$

$$h(k,0) = h(k)$$

$$h(k,1) = h(k) + \frac{1}{2}1 + \frac{1}{2}1 = h(k) + 0.5 + 0.5 = h(k) + 1 = h(k,0) + 1$$

$$h(k,2) = h(k) + \frac{1}{2}2 + \frac{1}{2}4 = h(k) + 1 + 2 = h(k) + 3 = h(k,1) + 2$$

$$h(k,3) = h(k) + \frac{1}{2}3 + \frac{1}{2}9 = h(k) + 1.5 + 4.5 = h(k) + 6 = h(k,2) + 3$$

$$h(k,4) = h(k) + \frac{1}{2}4 + \frac{1}{2}16 = h(k) + 2 + 8 = h(k) + 10 = h(k,3) + 4$$

$$h(k,5) = h(k) + \frac{1}{2}5 + \frac{1}{2}25 = h(k) + 2.5 + 12.5 = h(k) + 15 = h(k,4) + 5$$

$$h(k,6) = h(k) + \frac{1}{2}6 + \frac{1}{2}36 = h(k) + 3 + 18 = h(k) + 21 = h(k,5) + 6$$

$$h(k,7) = h(k) + \frac{1}{2}7 + \frac{1}{2}49 = h(k) + 3.5 + 24.5 = h(k) + 28 = h(k,6) + 7$$

0	1	4	2	7	6	3	5
---	---	---	---	---	---	---	---

Scansione quadratica: addensamento secondario

- La scansione quadratica è affetta dal fenomeno dell'*addensamento secondario*
- Se per due chiavi k_1 e k_2 si verifica $h(k_1)=h(k_2)$ allora
 - $h(k_1,0)=h(k_2,0)$
 - $h(k_1,1)=h(k_2,1)$
 - $h(k_1,2)=h(k_2,2)$
 - ...

k_1 e k_2 generano la medesima sequenza di scansione
- In pratica si possono generare solo m sequenze diverse
 - differiscono solamente per l'elemento iniziale
- L'addensamento secondario affligge comunque anche la scansione lineare

Doppio hashing

- Viene adottata la funzione di scansione:

$$h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

- Dove $h_1(k)$ e $h_2(k)$ sono due funzioni hash ausiliarie
- Il primo tentativo viene sempre eseguito su $h_1(k)$
- Si deve avere sempre $h_2(k) > 0$ altrimenti $h(k,i)$ non è una funzione di scansione valida
- Il doppio hashing utilizza la chiave k due volte
 - viene usata per determinare la posizione iniziale: $h_1(k)$
 - viene usata per determinare il passo della scansione: $h_2(k)$

Doppio hashing

- Il doppio hashing non soffre di fenomeni di addensamento né primario né secondario
- Ogni possibile chiave k produce una sequenza di scansione distinta
 - sono possibili $O(m^2)$ sequenze di scansione distinte che si distinguono per la cella iniziale e per il passo

Vincoli sulle funzioni $h_1(k)$ e $h_2(k)$

- $h_1(k)$ può essere una funzione hash arbitraria
 - infatti la tabella T è usata come una sequenza circolare e $h_1(k)$ determina solamente il punto di inizio
- $h_2(k)$ deve essere primo con la dimensione m della tabella
 - due numeri sono *primi tra loro* se non hanno un divisore comune maggiore di uno

Dimostrazione della necessità

- Supponiamo infatti che $h_2(k)$ non sia primo rispetto ad m per qualche k
 - cioè che $\text{MCD}(h_2(k), m) = d > 1$
 - per esempio $m=12$ e $h_2(k)=3$ e $\text{MCD}(12, 3) = d = 3$
- La ricerca di k andrebbe ad esaminare solo una porzione ($1/d$) della tabella
 - supponiamo che $h_1(k)=1$, allora si cercherebbe solamente in (1), (4), (7), (10), (13=1), (4), (7), (10), ...

Dimostrazione della sufficienza

- Supponiamo invece che $h_2(k)$ sia primo rispetto ad m per ogni k
 - per esempio $m=12$ e $h_2(k)=5$
- Perché si abbia una collisione per due indici i e j occorre che

$$h_1(k) + i h_2(k) = h_1(k) + j h_2(k) \pmod{m}$$

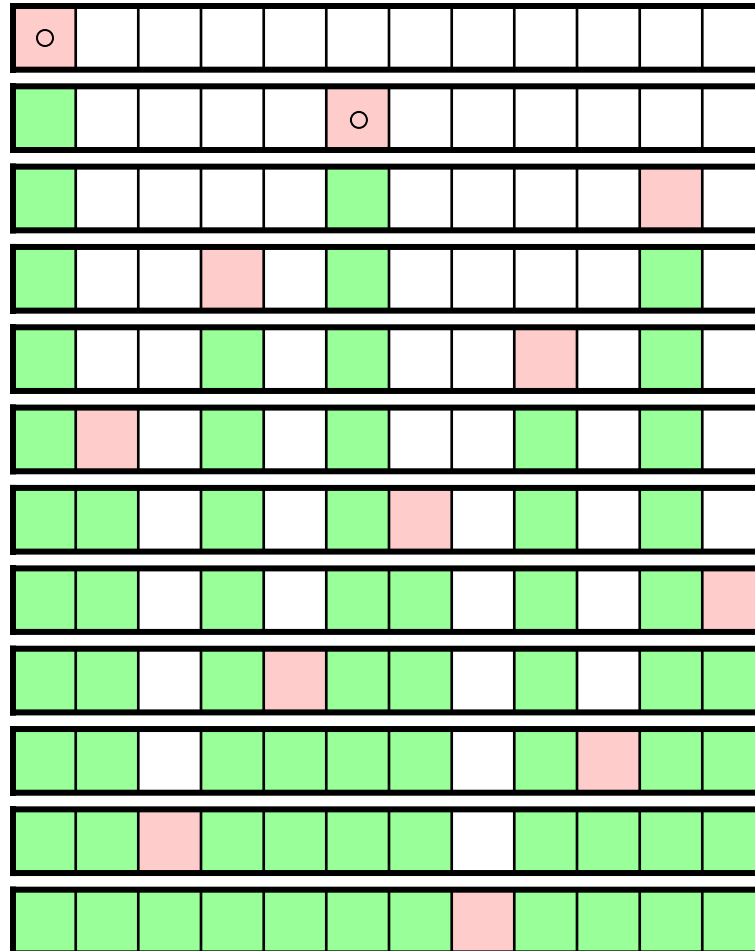
$$i h_2(k) = j h_2(k) \pmod{m}$$

$$(i-j)h_2(k) = 0 \pmod{m}$$

- Per ipotesi m non può essere ottenuto dal prodotto di un intero $(i-j)$ per $h_2(k)$ e $h_2(k) > 0$
- Ne segue che $(i-j) = 0 \pmod{m}$ cioè $i = j \pmod{m}$

Esempio

- Supponiamo $h_1(k)=1$, $m=12$ e $h_2(k)=5$



Scelta delle funzioni $h_1(k)$ e $h_2(k)$

- Se m è una potenza 2^p di 2 è sufficiente che $h_2(k)$ restituisca sempre un numero dispari
 - in questo modo è garantito che $h_2(k)$ sia primo rispetto 2^p
 - per esempio $h_2(k) = (1+2k) \bmod m$
- Se m è un numero primo va bene qualsiasi $h_2(k)$
 - si usano spesso $h_1(k) = k \bmod m$ e $h_2(k) = 1 + (k \bmod m_2)$ dove m_2 è appena più piccolo di m
 - ad esempio $m_2 = m-1$ oppure $m_2 = m-2$

Problemi

1. Simula l'inserimento delle chiavi 10, 22, 31, 4, 15, 28, 17, 88, 59 in una tabela hash ad indirizzamento aperto con 8 celle usando la funzione hash $h(k) = k \bmod m$ nell'ipotesi di:
 - scansione lineare
 - scansione quadratica con $c_1=c_2=1/2$
 - hashing doppio con $h_2(k) = (1+2k) \bmod m$
2. Scrivi lo pseudocodice della procedura HASH_DELETE, HASH_INSERT e HASH_SEARCH che prevedano il valore speciale DELETED

Problemi sul doppio hashing

3. Illustra l'inserimento in una tabella hash con indirizzamento aperto con $m=7$ componenti delle chiavi 34, 12, 15, 26, 2, 32, 40, utilizzando il doppio hashing con le seguenti funzioni:
- $h_1(k) = k \bmod 7$
 - $h_2(k) = 1 + (k \bmod 6)$
 - ricorda che $h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$
4. In quali condizioni si verifica la coincidenza tra l'hashing doppio e la scansione lineare?

Algoritmi e Strutture di Dati

I grafi

rappresentati con matrici e liste di adiacenza

M. Patrignani

Nota di copyright

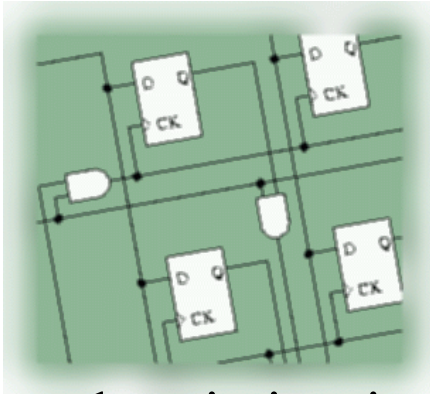
- Queste slides sono protette dalle leggi sul copyright
- Il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- Le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- Ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- Gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- Questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Contenuto

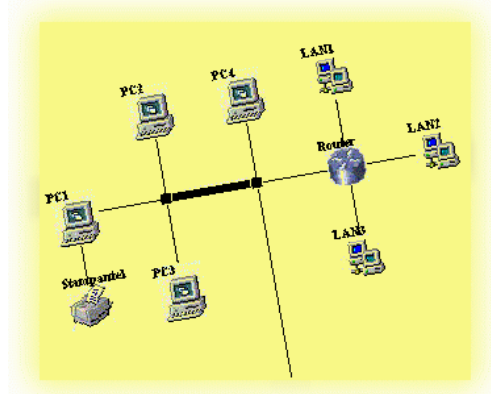
- Definizione di grafi diretti e indiretti
- Rappresentazione di grafi tramite:
 - matrici di adiacenza
 - liste di adiacenza
- Esercizi su grafi

Grafi nelle applicazioni

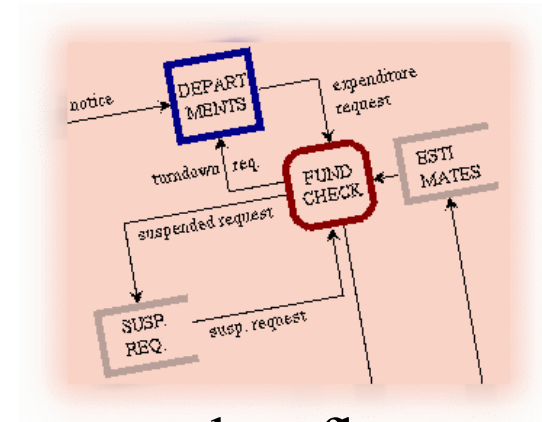
- Molti diagrammi utilizzati in ingegneria sono dei grafi



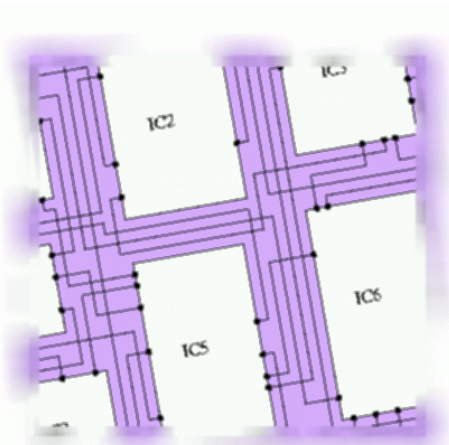
schemi circuitali



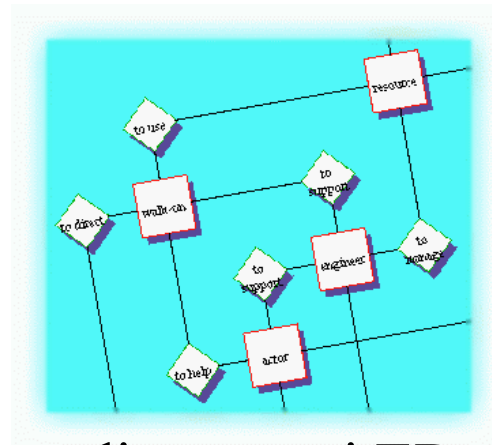
topologie di rete



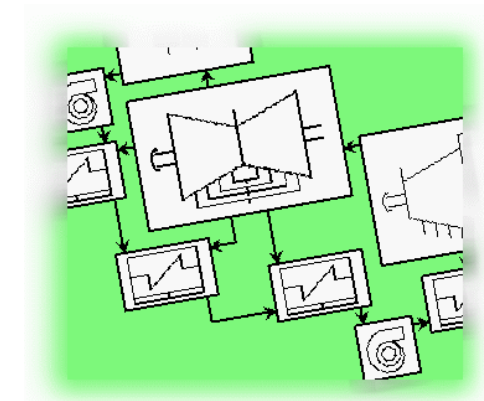
data flow



circuiti integrati



diagrammi ER

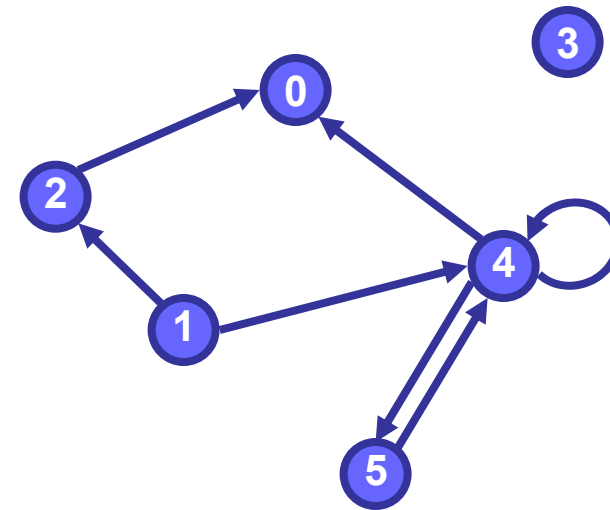


impianti industriali

Grafi diretti

- Un *grafo orientato* (o *diretto*) $G=(V,E)$ è costituito da un insieme di nodi V e un insieme di archi E
 - ogni arco è una coppia ordinata di nodi (u,v)
- Denotiamo con n il numero dei nodi ($n = |V|$) e con m il numero degli archi ($m = |E|$)
 - si ha sempre $m \in O(n^2)$
- Esempio:

$$V = \{0, 1, 2, 3, 4, 5\}$$
$$E = \{(2,0) (1,2) (4,0) (4,4) (4,5) (5,4) (1,4)\}$$

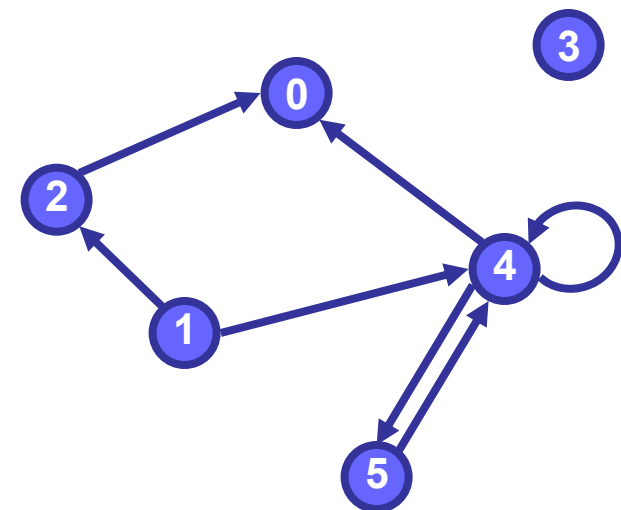


Grafi diretti e relazioni

- La versatilità dei grafi diretti deriva dal fatto che essi corrispondono a relazioni binarie
- Per esempio
 - contatti tra utenti di una rete di telefonia
 - dipendenze tra invocazioni di metodi in un software
 - partecipazioni di aziende nel capitale di altre
 - rapporti di eredità
 - rapporti di precedenza tra attività
 - reti sociali
 - ...

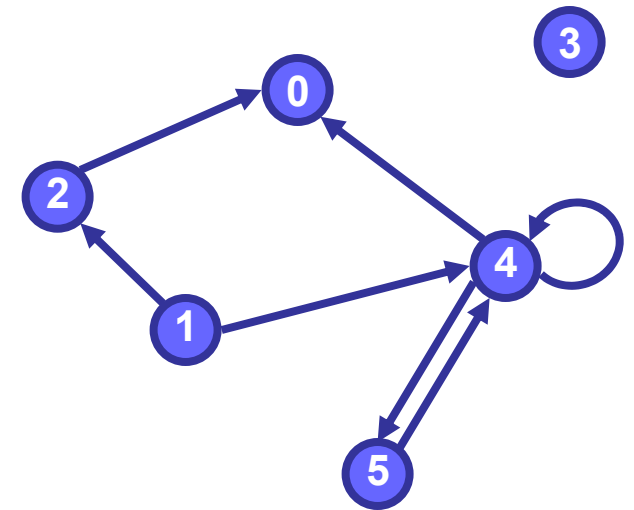
Archi uscenti ed entranti

- Dato un nodo u
 - un suo *arco uscente* è un arco $(u,v) \in E$
 - un suo *arco entrante* è un arco $(v,u) \in E$
 - un *nodo adiacente* è un nodo v per cui esiste $(u,v) \in E$
 - il suo *grado di uscita* è il numero dei suoi archi uscenti
 - il suo *grado di ingresso* è il numero dei suoi archi entranti
- Un nodo u è detto...
 - *sorgente* se non ha archi entranti
 - *pozzo* se non ha archi uscenti



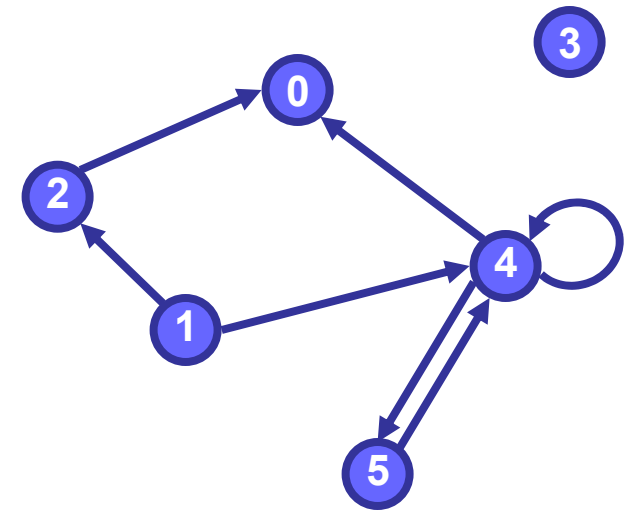
Cammini

- Un *cammino* (è sottinteso che sia diretto) è una sequenza di nodi $u_1, u_2, \dots, u_k$ tali che per $i=1, 2, \dots, k-1$ esistono gli archi (u_i, u_{i+1})
- Il cammino è detto *semplice* se tutti i suoi nodi sono distinti
- Il numero di archi è la *lunghezza* del cammino



Cicli

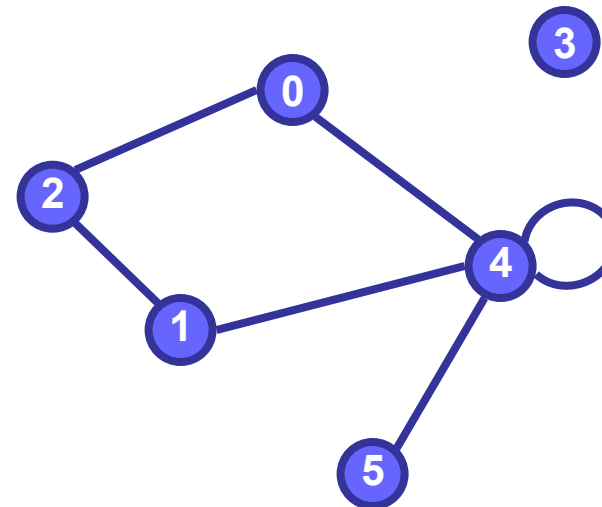
- Un *ciclo* è un cammino (non semplice) in cui il primo e l'ultimo nodo coincidono
- Un ciclo è detto *semplice* se il primo e l'ultimo nodo sono gli unici nodi che coincidono
- Un *cappio* (o *loop*) è un ciclo di un solo arco (e un solo nodo)
- Un *grafo semplice* è un grafo senza cappi
- Un grafo diretto è *aciclico* se non ha cicli (diretti)



Grafi non orientati

- In un grafo *non orientato* (o *non diretto*) $G=(V,E)$ l'insieme degli archi E è un insieme di coppie non ordinate (u,v)
 - (u,v) e (v,u) rappresentano lo stesso arco
- Graficamente si conviene di rappresentare una sola linea tra i nodi u e v

$$V = \{0, 1, 2, 3, 4, 5\}$$
$$E = \{(0,2) (0,4) (1,2) (1,4) (4,4) (4,5)\}$$



Il tipo astratto di dato grafo

■ Domini

- il dominio di interesse è l'insieme G dei grafi (diretti o non diretti)
- dominio di supporto: l'insieme degli iteratori NI per i nodi
- dominio di supporto: l'insieme degli iteratori AI per gli archi
- dominio di supporto: i booleani $B = \{\text{true}, \text{false}\}$

■ Costanti

- il grafo vuoto
- gli iteratori non validi per nodi e archi

■ Operazioni

- trova il primo nodo del grafo:
- trova il prossimo nodo:
- trova il primo arco di un nodo:
- trova il nodo adiacente tramite l'arco:
- trova il prossimo arco:
- determina se due nodi sono adiacenti
- aggiunge un nodo al grafo:
- aggiunge un arco tra due nodi:

FIRST_NODE: $G \rightarrow NI$

NEXT_NODE: $G \times NI \rightarrow NI$

FIRST_EDGE: $G \times NI \rightarrow AI$

ADJ_NODE: $G \times N1 \times AI \rightarrow N1$

NEXT_EDGE: $G \times N1 \times AI \rightarrow AI$

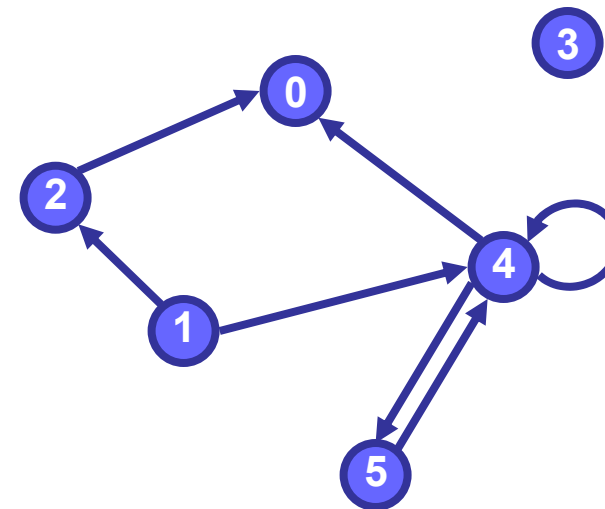
ARE_ADJ: $G \times N1 \times NI \rightarrow B$

ADD_NODE: $G \rightarrow NI$

ADD_EDGE: $G \times N1 \times NI \rightarrow AI$

Realizzazione mediante matrice di adiacenza

- Rappresentazione preferita per grafi densi
 - cioè per i quali il numero degli archi m è prossimo ad n^2
- Consente di sapere rapidamente se c'è un arco tra due nodi
- Usa una matrice (o un array di array) in cui l'elemento in posizione (i,j) segnala se esiste l'arco (i,j)
- Occupa $\Theta(n^2)$ spazio

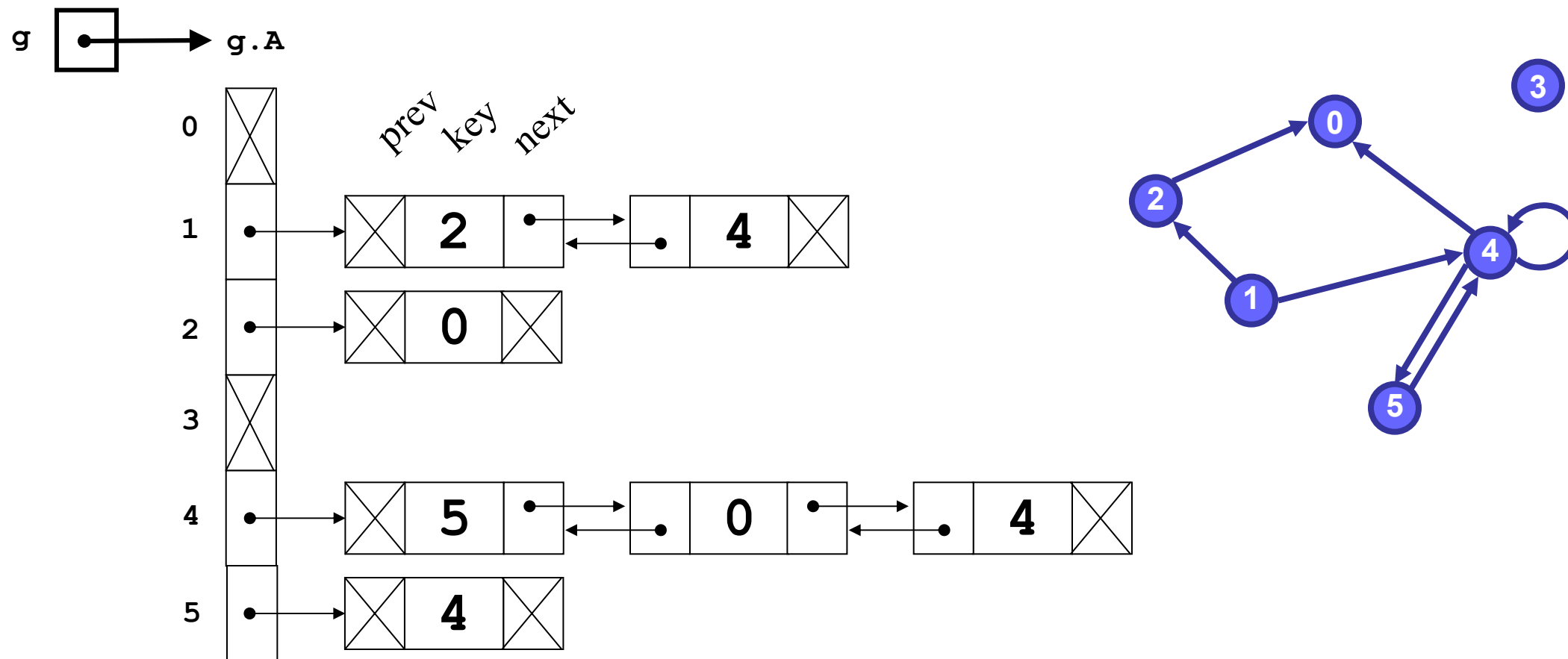


$g \rightarrow g.A$

	0	1	2	3	4	5	
0	0	0	0	0	0	0	0
1	0	0	1	0	1	0	1
2	1	0	0	0	0	0	2
3	0	0	0	0	0	0	3
4	1	0	0	0	1	1	4
5	0	0	0	0	1	0	5

Realizzazione tramite liste di adiacenza

- Si fa uso di un array A di liste doppiamente concatenate
 - generalmente si mette nell'array direttamente il riferimento al primo elemento della lista



Rappresentazione dei grafi: liste di adiacenza

- Questa rappresentazione occupa spazio $O(n) + O(m)$
 - nel caso peggiore, siccome $m = O(n^2)$ utilizza uno spazio $O(n^2)$ come le rappresentazioni con matrici di adiacenza
 - in numerose applicazioni, però, $m \in O(n)$
 - in questo caso le rappresentazioni con liste di adiacenza sono preferibili
- La lista di adiacenza di un nodo può essere lunga $O(n)$
- Percorrendo tutte le liste di adiacenza di tutti i nodi si impiega un tempo $O(n) + O(m)$
 - che diventa $O(n^2)$ se il grafo è denso

Realizzazioni del tipo astratto di dato grafo

■ Matrice di adiacenza

- l'iteratore per un nodo è un intero
 - è valido se è nell'intervallo legittimo per l'array g.A
- l'iteratore per un arco uscente da un nodo è ancora un intero
 - è valido se è nell'intervallo legittimo e se la cella corrispondente dell'array g.A contiene un uno
- si rinuncia a realizzare efficientemente l'operazione di aggiunta di un nodo

■ Liste di adiacenza

- l'iteratore per un nodo è un intero
 - è valido se è nell'intervallo legittimo per l'array g.A
- l'iteratore per un arco è un riferimento ad un elemento di una lista
 - l'iteratore non valido è NULL
- anche in questo caso l'operazione di aggiunta di un nodo richiede una gestione non semplice e non sempre efficiente

Rappresentazione di grafi non orientati

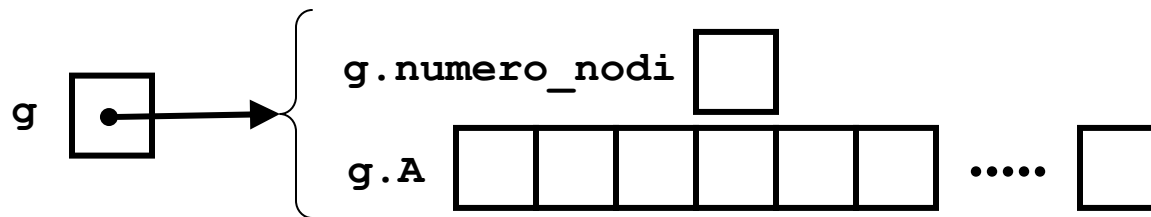
- Per ogni arco non orientato (u,v) vengono rappresentati i due archi orientati (u,v) e (v,u)
- Nel caso di matrice di adiacenza
 - la matrice è simmetrica
- Nel caso di liste di adiacenza
 - se la lista del nodo i contiene il nodo j , allora la lista del nodo j contiene il nodo i

Grafi pesati sugli archi

- Sono grafi in cui ad ogni arco e è associato un peso w_e
- Nella rappresentazione tramite matrici di adiacenza si usano i valori dei pesi al posto degli uni:
 - si assume che non esistano archi con peso zero e si usa lo zero per rappresentare l'assenza dell'arco
 - si usa w_e per rappresentare un arco di peso w_e
- Nella rappresentazione tramite liste di adiacenza
 - ogni elemento della lista ha, oltre all'indice del nodo adiacente, anche un attributo weight con valore w_e

Realizzazioni in linguaggio C

- La rappresentazione dei grafi in linguaggio C segue gli stessi paradigmi della rappresentazione in pseudocodifica
 - tuttavia, poiché gli array in linguaggio C non hanno un campo “length” che ne riveli la lunghezza occorre aggiungere un campo “numero_nodi” ad entrambe le rappresentazioni



Esercizi su matrici di adiacenza

- Dato un grafo g rappresentato tramite una matrice di adiacenza $g.A$ (un array di array)
 1. scrivi una procedura `LISTE( $g$ )` che ne restituisca la sua rappresentazione mediante un array di liste di adiacenza doppiamente concatenate
 2. scrivi una procedura `GRADO_USCITA( $g, u$ )` che calcoli il grado di uscita del nodo con indice u
 3. scrivi una procedura `GRADO_INGRESSO( $g, u$ )` per il calcolo del grado di ingresso del nodo con indice u
 4. scrivi una procedura `GRADO_USCITA_MEDIO( $g$ )` per il calcolo del grado di uscita medio dei nodi del grafo
 5. scrivi una procedura `GRAFO_SEMPlice( $g$ )` che verifica se il grafo è semplice (privo di cappi)
- Discuti la complessità degli algoritmi che hai proposto

Esercizi su liste di adiacenza 1/3

- Dato un grafo diretto g rappresentato tramite un array $g.A$ di liste di adiacenza doppiamente concatenate
 6. scrivi una procedura $\text{MATRICE}(g)$ che ne restituisca la sua rappresentazione mediante una matrice di adiacenza
 7. scrivi una procedura $\text{GRADO_USCITA}(g,u)$ che calcoli il grado di uscita del nodo con indice u
 8. scrivi una procedura $\text{GRADO_INGRESSO}(g,u)$ per il calcolo del grado di ingresso del nodo con indice u
 9. scrivi una procedura $\text{GRADO_USCITA_MEDIO}(g)$ per il calcolo del grado di uscita medio dei nodi del grafo
 10. scrivi una procedura $\text{GRAFO_SEMPLICE}(g)$ che verifica se il grafo è semplice (privo di cappi)
- Discuti la complessità degli algoritmi che hai proposto

Esercizi su liste di adiacenza 2/3

- Dato un grafo diretto g rappresentato tramite un array $g.A$ di liste di adiacenza doppiamente concatenate
 11. scrivi lo pseudocodice della funzione $VERIFICA_ARCO(g,u,v)$ che restituisce **true** se esiste l'arco che va dal nodo identificato dall'indice u al nodo identificato dall'indice v e **false** altrimenti
 12. scrivi lo pseudocodice della funzione $VERIFICA_NON_ORIENTATO(g)$ che restituisce **true** se il grafo presenta un arco (u,v) per ogni arco (v,u) e **false** altrimenti
 - puoi utilizzare la funzione $VERIFICA_ARCO(g,u,v)$
 13. scrivi lo pseudocodice della funzione $VERIFICA_POZZO(g,u)$ che restituisce **true** se il nodo identificato dall'indice u non ha archi uscenti, **false** altrimenti
 14. scrivi lo pseudocodice della funzione $VERIFICA_SORGENTE(g,u)$ che restituisce **true** se il nodo identificato dall'indice u non ha archi entranti, **false** altrimenti
- Discuti la complessità degli algoritmi che hai proposto

Esercizi su liste di adiacenza 3/3

- Dati due grafi $g1$ e $g2$ rappresentati tramite array di liste di adiacenza doppiamente concatenate
 15. scrivi lo pseudocodice della funzione `VERIFICA_UNIONE( $g1, g2$ )` che verifica che tra ogni possibile coppia di nodi ci sia un arco in $g1$ o in $g2$ (o in entrambi)
 - puoi supporre che $g1$ e $g2$ abbiano lo stesso numero di nodi ($g1.A.length = g2.A.length$)
 16. scrivi lo pseudocodice della funzione `VERIFICA_POZZI_E_SORGENTI( $g1, g2$ )` che restituisce **true** se tutti i pozzi di $g1$ sono sorgenti di $g2$ e tutte le sorgenti di $g1$ sono pozzi di $g2$ e restituisce **false** altrimenti
 - puoi supporre che $g1$ e $g2$ abbiano lo stesso numero di nodi
 - puoi utilizzare le funzioni `VERIFICA_POZZO( $g, u$ )` e `VERIFICA_SORGENTE( $g, u$ )`
- Discuti la complessità degli algoritmi che hai proposto

Algoritmi e Strutture di Dati

I grafi

rappresentati con oggetti e riferimenti

m.patrignani

Nota di copyright

- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Contenuto

- Rappresentazione di grafi tramite oggetti e riferimenti
 - costo delle operazioni di inserimento e cancellazione di nodi e archi
 - uso dei puntatori di posizione
- Esercizi su grafi rappresentati tramite oggetti e riferimenti

Rappresentazione con oggetti

- Nelle rappresentazioni che abbiamo visto finora rimuovere o aggiungere un nodo contempla la riallocazione e la copia dell'array g.A
- Se è necessario modificare anche l'insieme dei nodi una rappresentazione più efficiente è quella in cui ogni nodo e ogni arco è un oggetto
- Nell'oggetto nodo e nell'oggetto arco possono essere memorizzati altri valori, come identificatori, pesi, marcatori, etichette, e tutto ciò che è richiesto dalle applicazioni

nodo

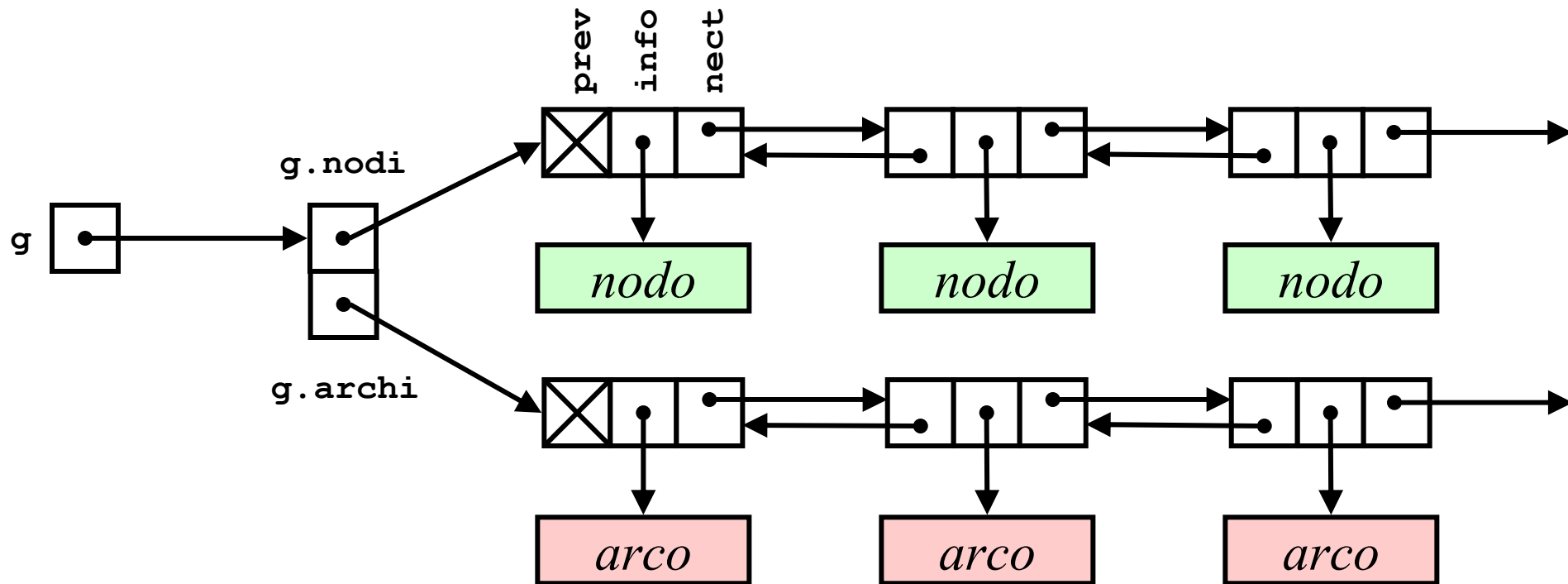
1	a
id	label

arco

1	3	b
id	weight	label

Il grafo

- I nodi e gli archi non sono conservati in array, ma in liste doppiamente concatenate

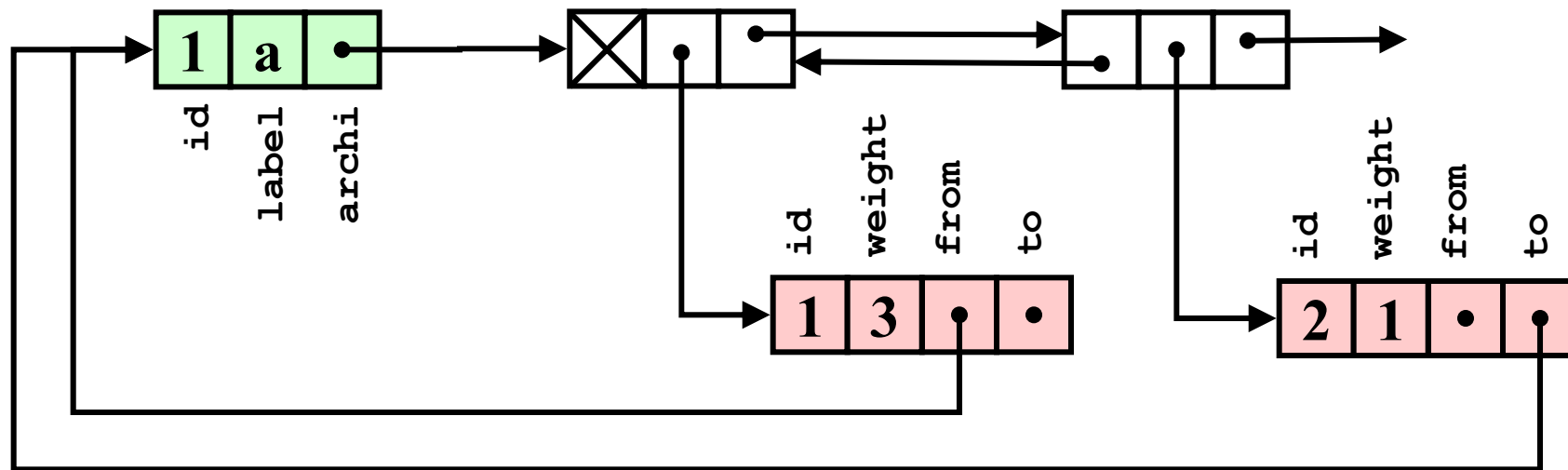


Identificatori dei nodi

- In un grafo rappresentato tramite una matrice di adiacenza o un array di liste di adiacenza un nodo è identificato da un intero
 - cioè l'indice dell'array o della matrice
- Nella rappresentazione con oggetti e riferimenti un identificatore di un nodo è un puntatore al nodo
 - se l'utente ha bisogno di identificatori numerici o di etichette è possibile memorizzarle nei nodi stessi

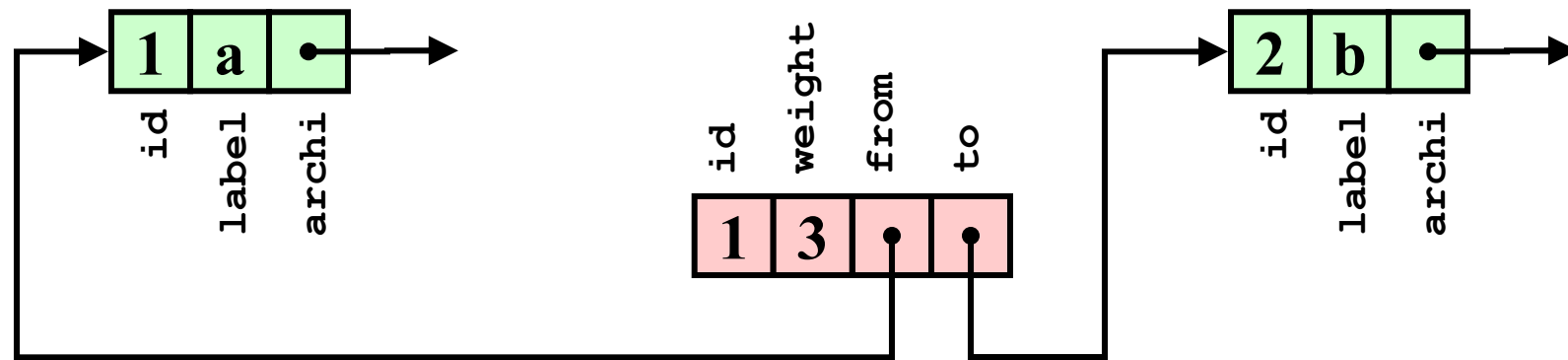
Relazioni tra nodi e archi

- E' necessario che ogni nodo abbia una lista dei suo archi incidenti e che ogni arco abbia dei riferimenti ai suoi due estremi



Grafi diretti e grafi non diretti

- Gli archi diretti e gli archi non diretti vengono rappresentati allo stesso modo



- Nel caso in cui il grafo sia indiretto la differenza tra i nodi **from** e **to** viene ignorata
 - in questo modo un arco (diretto o indiretto) è sempre rappresentato da un singolo oggetto arco

Aggiunta di nodi e archi

- Per aggiungere un nodo occorre
 - creare un nuovo oggetto nodo
 - aggiungere il nodo in testa alla lista dei nodi del grafo
- Per aggiungere un arco occorre
 - creare un nuovo oggetto arco
 - aggiungerlo in testa alle seguenti liste
 - lista degli archi del grafo
 - lista degli archi del nodo sorgente
 - lista degli archi del nodo destinazione
- Entrambe le operazioni possono essere fatte in tempo costante

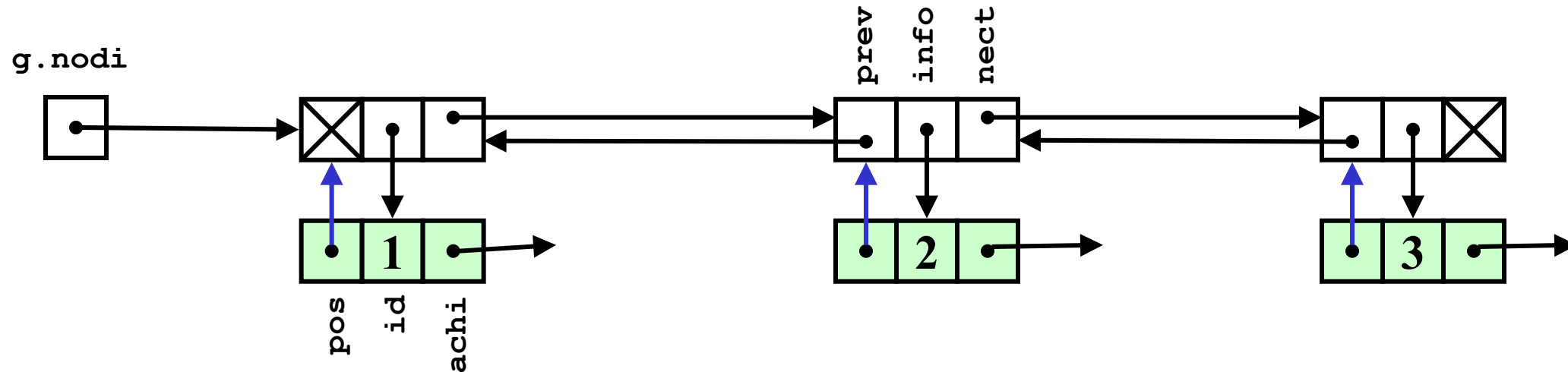
Rimozione di un arco o di un nodo

- Per rimuovere un arco a partire da un suo indentificatore (riferimento) occorre
 - scorrere tutte le liste in cui è menzionato l'arco e rimuoverlo
 - cancellare l'arco
- Per rimuovere un nodo a partire da un suo indentificatore (riferimento) occorre
 - rimuovere tutti i suoi archi incidenti
 - scorrere la lista dei nodi e rimuoverlo dalla lista
 - cancellare il nodo
- Queste operazioni richiederebbero un tempo lineare!

Uso dei puntatori di posizione

- Per rendere le rimozioni più veloci ogni oggetto (nodo o arco) possiede dei campi aggiuntivi
 - ogni nodo ha un riferimento **pos** all'elemento corrispondente della lista dei nodi del grafo
 - ogni arco ha tre riferimenti **pos**, **frompos**, **topos** agli elementi corrispondenti delle tre liste (del grafo, del nodo from e del nodo to) in cui compare
- In questo modo a partire dal nodo o dall'arco è possibile rimuoverlo dal grafo in tempo costante

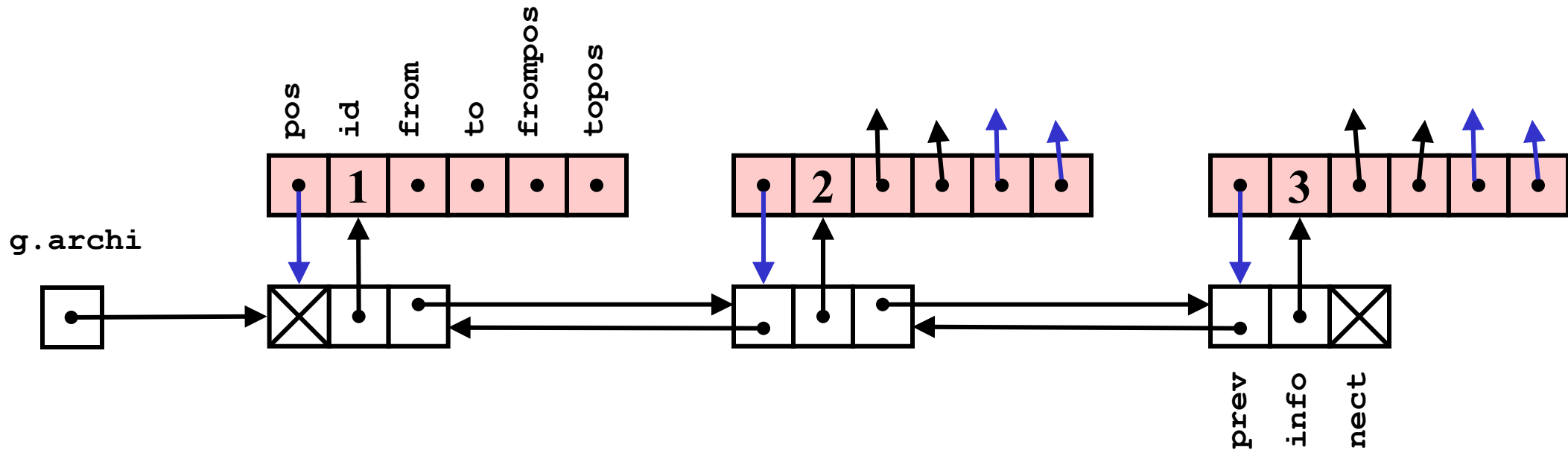
Puntatori di posizione del nodo



- Poiché la lista dei nodi è doppiamente concatenata è possibile rimuovere efficientemente un nodo dalla lista una volta che sia noto l'elemento corrispondente

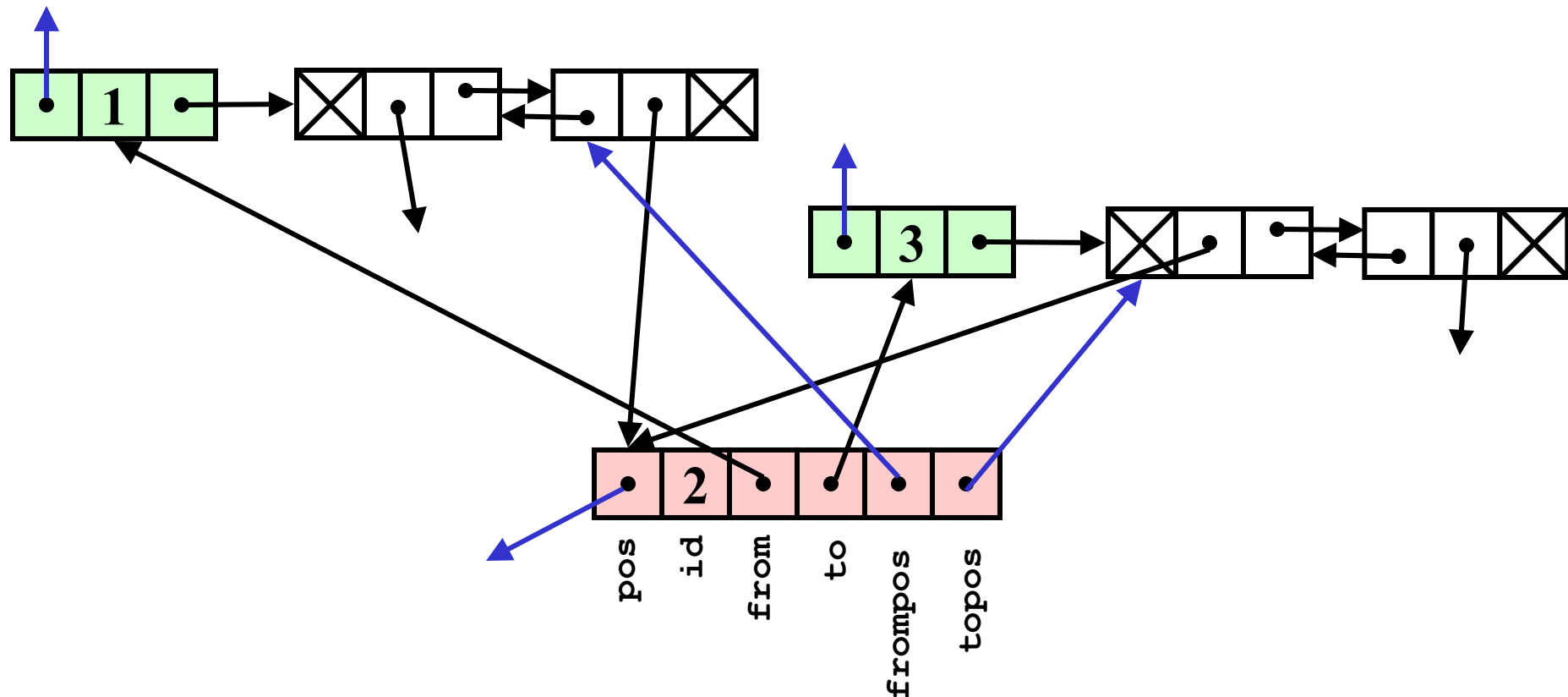
Puntatori di posizione dell'arco

- Ogni arco ha tre puntatori di posizione in quanto compare in tre liste doppiamente concatenate

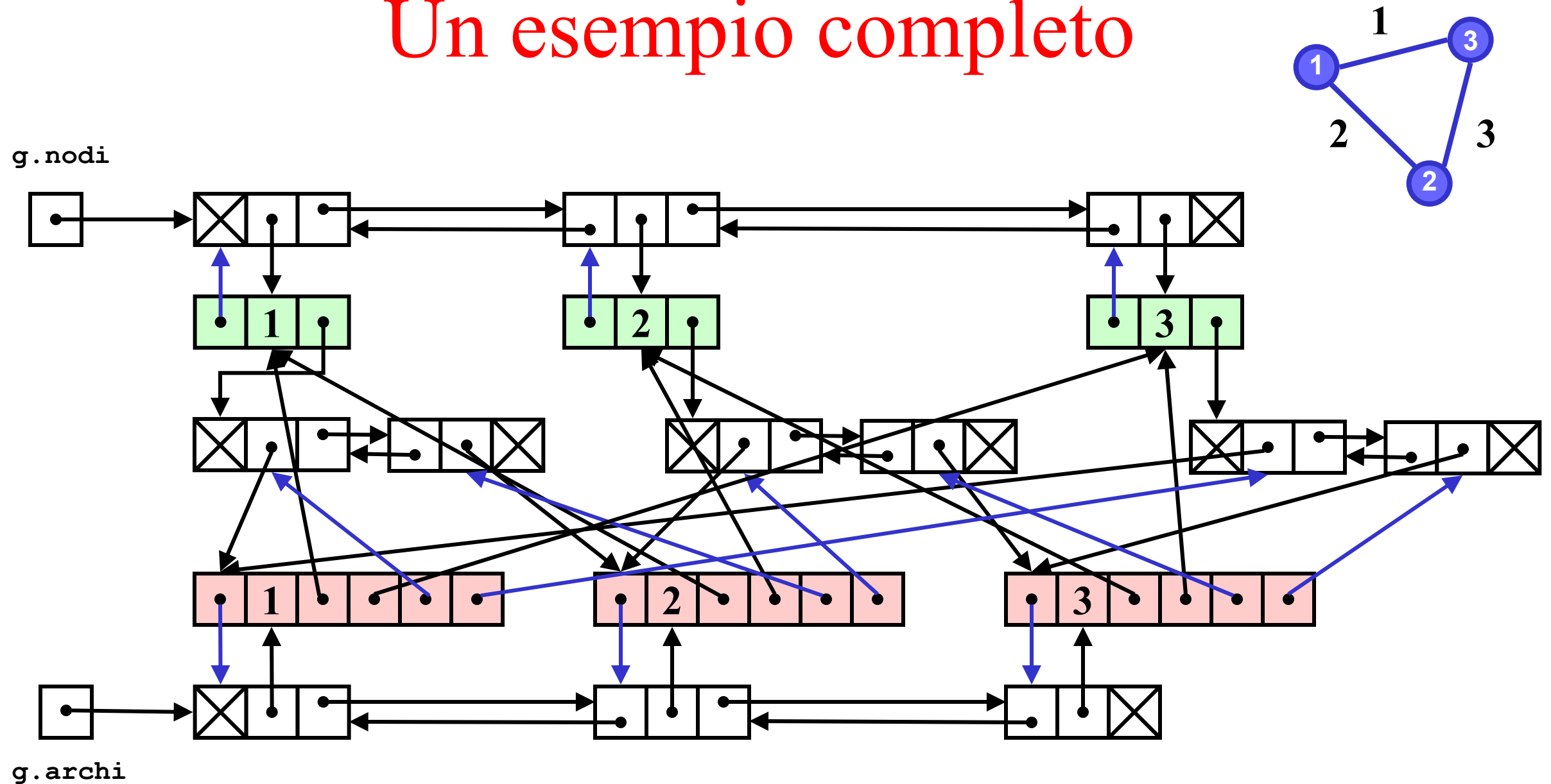


Puntatori di posizione dell'arco

- Tramite i campi **pos**, **frompos** e **topos** l'arco conosce la sua posizione nella lista degli archi del grafo, nodo **from** e del nodo **to**



Un esempio completo



Attributi opzionali del grafo

- Nelle rappresentazioni dei grafi con oggetti sono frequenti attributi opzionali
 - `numero_nodi`
 - per conoscere il numero dei nodi senza scorrere la lista `g.nodi`
 - `numero_archi`
 - per conoscere il numero degli archi senza scorrere la lista `g.archi`
 - `max_id_nodi`
 - per poter scegliere un id di un nuovo nodo senza dover verificare che non sia già utilizzato
 - `max_id_archi`
 - per poter scegliere un id di un nuovo arco senza dover verificare che non sia già utilizzato

Esercizi sulle rappresentazioni con oggetti

- Scrivi lo pseudocodice delle seguenti procedure
 1. AGGIUNGI_NODO(g, u) che inserisce nel grafo g un nuovo nodo con id u e restituisce un riferimento al nodo aggiunto
 2. AGGIUNGI_ARCO($g, from, to, u$) che inserisce nel grafo g un nuovo nodo con id u dal nodo $from$ al nodo to (passati come riferimento) e restituisce un riferimento all'arco aggiunto
 3. RIMUOVI_ARCO(g, e) che cancella dal grafo g l'arco e passato come riferimento
 4. RIMUOVI_NODO(g, n) che cancella dal grafo g il nodo n passato come riferimento

Algoritmi e Strutture di Dati

Visita in ampiezza di un grafo

m.patrignani

Nota di copyright

- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Contenuto

- Visita in ampiezza di un grafo indiretto
- Grafi e connettività
- Esercizi sulle visite in ampiezza

Algoritmi di visita di un grafo

- Lo scopo di questi algoritmi è quello di visitare tutti i nodi raggiungibili a partire da un nodo di partenza
- Caratteristiche:
 - i nodi non sono raggiunti in ordine casuale, ma in un ordine determinato dalla forma del grafo
 - diversi algoritmi su grafi sono modifiche di algoritmi di visita
 - non tutti i nodi vengono raggiunti
 - perché il grafo potrebbe avere più componenti connesse
 - alcuni nodi possono essere raggiunti da più nodi adiacenti
 - occorre marcare i nodi per non rischiare di ciclare ad infinito

Uso dei marcatori

- Gli algoritmi di visita dei grafi fanno tutti uso di marcatori
- Un marcatore è un valore associato ad ogni nodo di un grafo
 - per esempio un booleano (TRUE o FALSE) oppure un intero (generalmente 1 o 0)
- Nel caso più generale si può associare ad un nodo un generico intero che viene spesso chiamato “colore” (color) del nodo

Realizzazioni di marcatori

- Se un nodo è identificato da un intero è sufficiente affiancare (o aggiungere) alla struttura del grafo un array di interi con n posizioni, dove n è il numero dei nodi

```
1. // "color" è un array di interi con n posizioni
2. for i = 0 to color.length-1
3.     color[i] = 0 // inizializzo l'array con zero
4. color[6] = 1 // coloro con 1 il nodo con indice 6
```

- Per verificare se il nodo i è marcato eseguiremo:

```
1. if ( color[i] == 1 ) // nodo i marcato
```

Realizzazioni di marcatori

- Se un nodo è identificato da un riferimento ad un oggetto è sufficiente aggiungere alla struttura del nodo un campo intero “color”

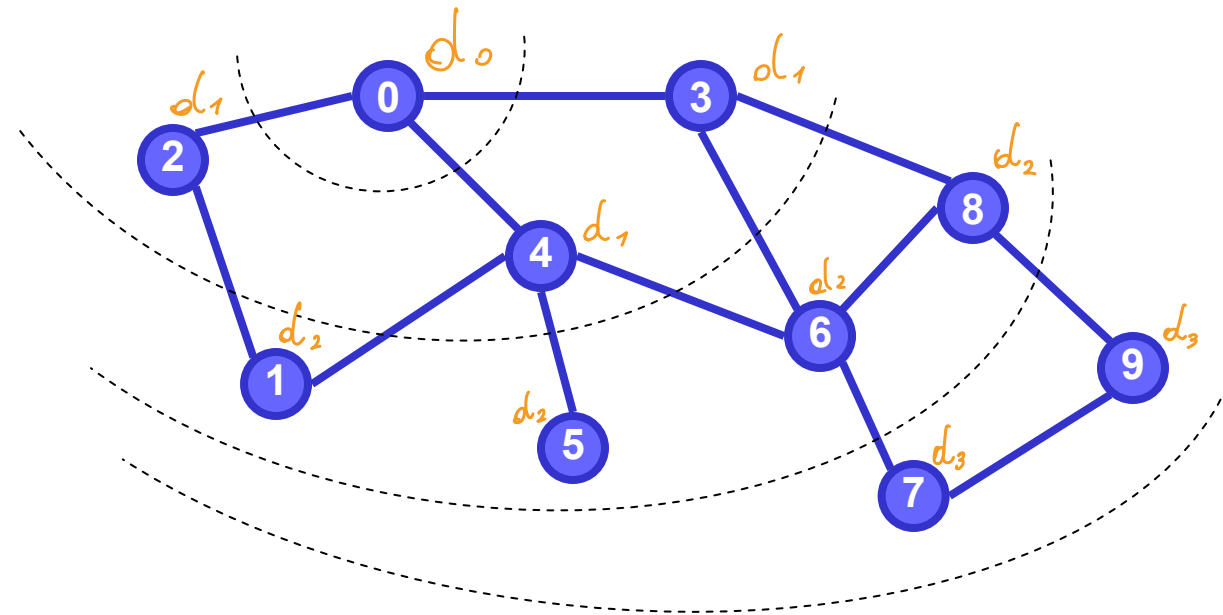
```
1.  x = g.nodi
2.  while x != NULL           // scorro la lista dei nodi
3.      x.info.color = 0      // inizializzo il colore con zero
4.      x = x.next
```

- Per verificare se il nodo n è marcato eseguiremo:

```
1. if ( n.color == 1 )      // nodo n marcato
```

Visita in ampiezza (Breadth-First Search)

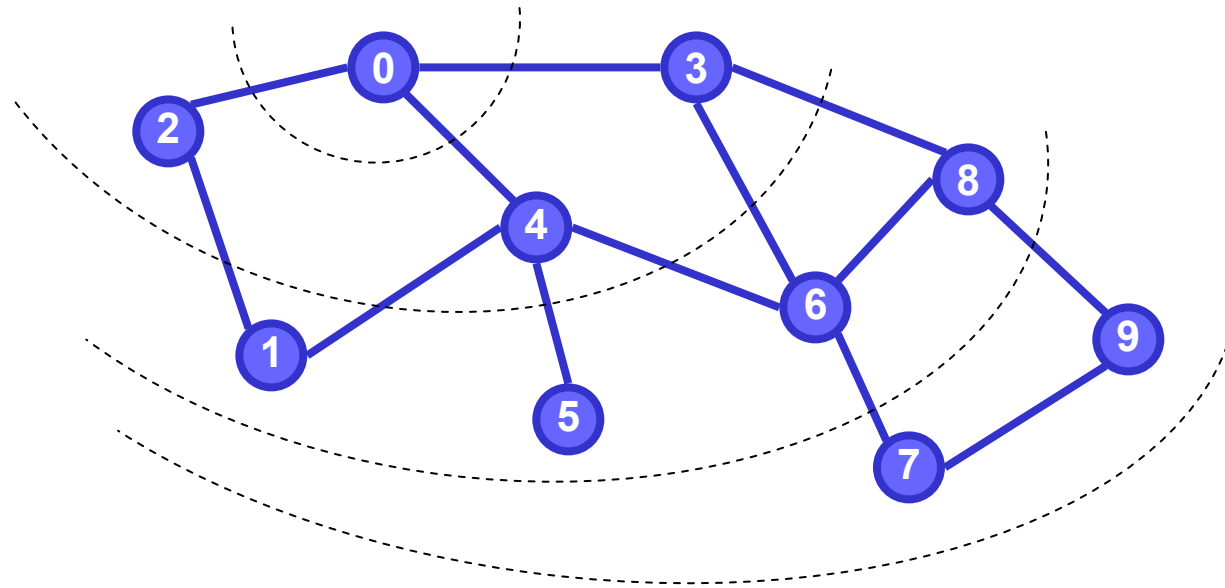
- A partire da un nodo v si visitano i nodi raggiungibili da v nell'ordine imposto dalla loro distanza
 - prima i più vicini, poi i più lontani



Strategia per una visita in ampiezza

- Facciamo uso di una coda sulla quale è possibile eseguire le operazioni ENQUEUE e DEQUEUE
- La coda viene inizializzata inserendoci il nodo di partenza
- I nodi raggiunti vengono marcati e messi in coda
 - per essere sicuri di non metterli in coda due volte li marchiamo appena li mettiamo in coda
- Finché la coda non è vuota
 - estraiamo un nodo dalla coda
 - consideriamo tutti i suoi adiacenti e se sono raggiunti per la prima volta (non sono marcati) li marchiamo e li mettiamo in coda
- L'ordine con cui i nodi sono estratti dalla coda corrisponde ad una visita in ampiezza
 - è lo stesso ordine con cui i nodi sono messi nella coda e marcati

Esempio di visita in ampiezza



esplorati	coda
	0
0	2,4,3
0,2	4,3,1
0,2,4	3,1,5,6
0,2,4,3	1,5,6,8

0,2,4,3,1	5,6,8
0,2,4,3,1,5	6,8
0,2,4,3,1,5,6	8,7
0,2,4,3,1,5,6,8	7,9
0,2,4,3,1,5,6,8,7	9
0,2,4,3,1,5,6,8,7,9	

Procedura BFS (liste di adiacenza)

```
BFS(g,v)          // g.A è un array di liste di adiacenza, v è un indice
1.  for i = 0 to g.A.length-1
2.      color[i] = 0      /* zero = non raggiunto */
3.  q = QUEUE-EMPTY()    /* creo una coda vuota */
4.  color[v] = 1          /* uno = raggiunto e messo in coda */
5.  ENQUEUE(q,v)          /* metto in coda l'indice v */
6.  while not QUEUE-VOID(q) /* finché la coda q non è vuota */
7.      u = DEQUEUE(q)    /* estraggo un indice dalla coda */
8.      x = g.A[u]         /* mi preparo ad esporare gli adiacenti di u */
9.      while x != NULL /* finché c'è un nodo adiacente */
10.         k = x.info      /* k è l'indice del nodo adiacente a u */
11.         if (color[k] == 0) /* se k non è stato già raggiunto */
12.             color[k] = 1      /* raggiunto e messo in coda */
13.             ENQUEUE(q,k)
14.         x = x.next
```

CICLO PER SMARCARE I NODI

Complessità della visita BFS

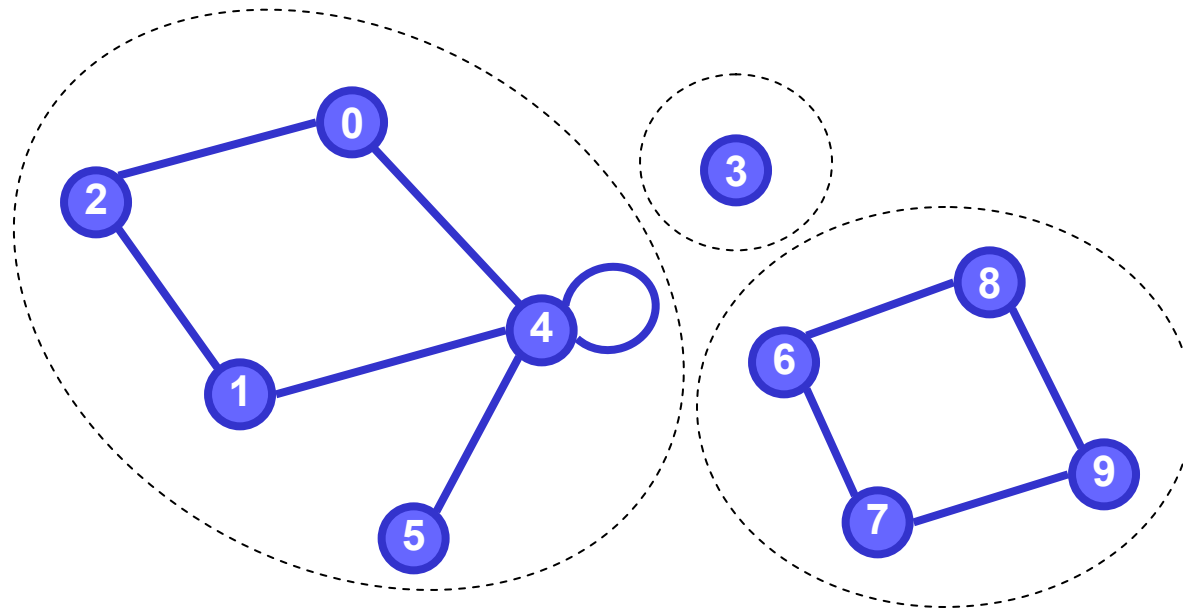
- In una visita in ampiezza
 - ogni nodo è inserito ed estratto dalla coda una sola volta
 - ogni arco (adiacenza) è considerata sia dal nodo di partenza che dal nodo di arrivo
- Dunque la complessità nel caso peggiore è $\Theta(n+m)$

Esercizi sulle visite in ampiezza

1. Scrivi lo pseudocodice della procedura $\text{BFS}(g,v)$ nel caso in cui il grafo non diretto g sia rappresentato tramite una matrice di adiacenza
2. Scrivi lo pseudocodice della procedura $\text{BFS}(g,v)$ nel caso in cui il grafo non diretto g sia rappresentato tramite oggetti e riferimenti

Grafi non orientati e connettività

- Dato un grafo non orientato $G=(V,E)$
 - un nodo v è *raggiungibile* da un nodo u se esiste un cammino da u a v
 - se per ogni coppia di nodi u e v di V esiste un cammino da u a v il grafo è detto *connesso*
 - la proprietà raggiungibilità tra nodi di un grafo non orientato è una proprietà di equivalenza le cui classi di equivalenza sono chiamate *componenti connesse*



Esercizi sulle visite in ampiezza

3. Scrivi lo pseudocodice della procedura `IS_CONNECTED(g)` che restituisce `TRUE` se il grafo è connesso
- possibile strategia:
 - eseguo una visita a partire da un nodo qualunque
 - se alcuni nodi rimangono non marcati il grafo non è connesso

Visite di un grafo non connesso

- Per eseguire una BFS di un grafo non necessariamente connesso è sufficiente lanciare diverse visite con lo stesso array color
 - per esempio nel caso di grafo rappresentato con liste/matrice di adiacenza il codice potrebbe essere il seguente

```
BFS_non_connesso(g)          /* g è rappresentato tramite liste di adiacenza */  
1.  for i = 0 to g.A.length-1  
2.      color[i] = 0          /* zero = non raggiunto */  
3.  for i = 0 to g.A.length-1  
4.      if color[i] == 0      /* se i non ancora visitato... */  
5.          BFS(g,i,color)    /* ...comincia una BFS da qui */
```

Calcolo delle componenti connesse

4. Scrivi lo pseudocodice della procedura

COMPONENTI_CONNESSE(g)

- input: un grafo non diretto g
- output: il numero delle componenti connesse del grafo g
- possibile strategia:
 - pongo il contatore delle componenti connesse a zero
 - finché c'è un nodo non visitato
 - incremento il contatore delle componenti connesse
 - eseguo una visita a partire dal nodo non visitato marcando tutti i nodi visitati

Esercizi sulle visite in ampiezza

5. Supponi di disporre di un'implementazione di una tabella hash dai nodi agli interi

- `new_table()`
 - ritorna una nuova tabella hash vuota
- `add_pair( $h, n, i$ )`
 - aggiunge alla tabella h una coppia formata da un nodo n ed un intero i
- `get_value( $h, n$ )`
 - ritorna il valore associato al nodo n

Scrivi lo pseudocodice della procedura `BFS_order( $g, v$ )` che restituisce in output una tabella hash `order` dove `get_value(order,  $n$ )` è il numero d'ordine con cui il nodo n è stato visitato

- nel caso in cui il grafo sia rappresentato come una matrice o un array di liste di adiacenza al posto della tabella hash si potrebbe ritornare un semplice array

Esercizi sulle visite in ampiezza

6. Scrivi lo pseudocodice della procedura `DISTANZE(g,v)` che restituisce una tabella hash delle distanze di tutti i nodi dal nodo v
- i nodi non raggiunti devono avere distanza -1
 - possibile strategia
 - eseguo un visita in ampiezza a partire da v
 - quando un nodo viene marcato, la sua distanza da v è pari alla distanza del nodo da cui è raggiunto più uno
 - nel caso in cui il grafo si rappresentato tramite un array di liste di adiacenza o una matrice di adiacenza la funzione potrebbe restituire un semplice array di interi

Esercizi sulle visite in ampiezza

7. Scrivi lo pseudocodice della funzione $\text{CAMMINO_MINIMO}(g, u, v)$ che prende in input un grafo g e gli identificatori di due nodi u e v e produce in output la lista dei nodi del cammino più corto da u a v
- possibile strategia
 - eseguo una visita in ampiezza a partire da v e memorizzo per ogni nodo u il parent di u , cioè il nodo dal quale è stato raggiunto
 - la catena di parent che conduce da u a v è il cammino minimo cercato

Algoritmi e Strutture di Dati

Visita in profondità di un grafo

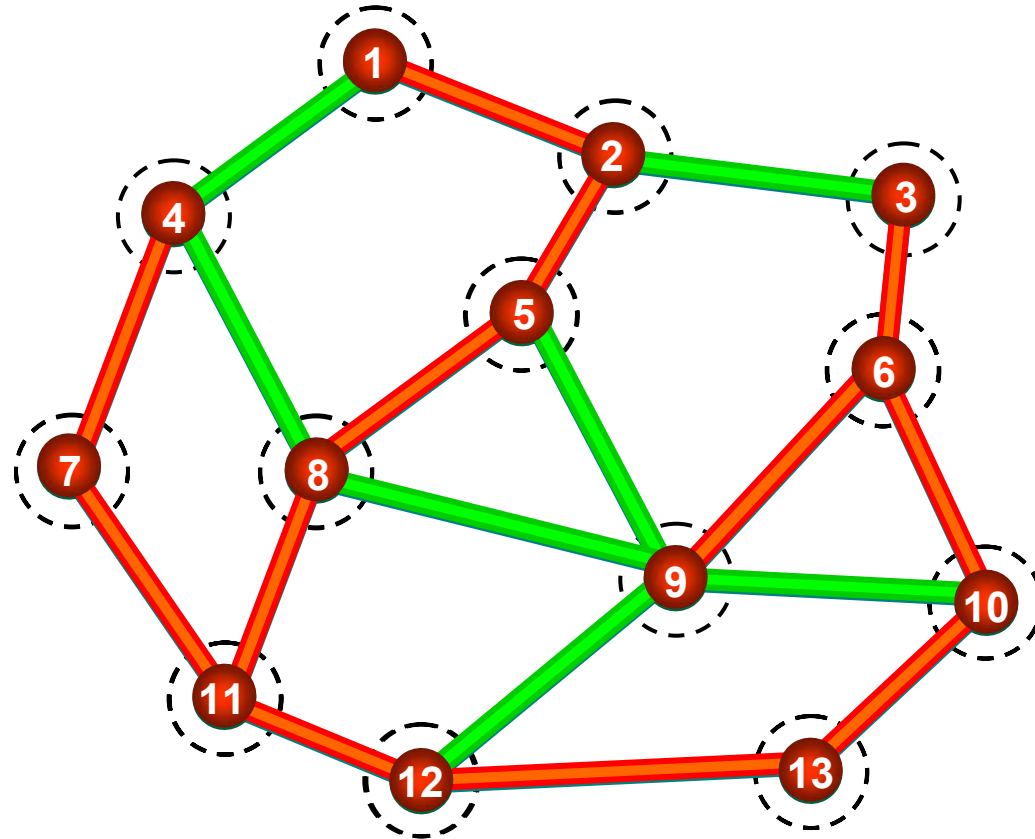
m.patrignani

Nota di copyright

- queste slides sono protette dalle leggi sul copyright
- il titolo ed il copyright relativi alle slides (inclusi, ma non limitatamente, immagini, foto, animazioni, video, audio, musica e testo) sono di proprietà degli autori indicati sulla prima pagina
- le slides possono essere riprodotte ed utilizzate liberamente, non a fini di lucro, da università e scuole pubbliche e da istituti pubblici di ricerca
- ogni altro uso o riproduzione è vietata, se non esplicitamente autorizzata per iscritto, a priori, da parte degli autori
- gli autori non si assumono nessuna responsabilità per il contenuto delle slides, che sono comunque soggette a cambiamento
- questa nota di copyright non deve essere mai rimossa e deve essere riportata anche in casi di uso parziale

Visita in profondità (Depth-First Search)

- A partire da un nodo v si percorre il grafo fino a che si trovano nodi non visitati
- Quando tutti i vicini sono visitati si torna indietro verso v per verificare che non ci siano nodi adiacenti non visitati



Procedura DFS (liste di adiacenza)

- DFS di un grafo rappresentato con liste di adiacenza

```
DFS(g,v)          /* g è un grafo rappresentato tramite liste di adiacenza */  
1.  for i = 0 to g.A.length-1  
2.      color[i] = 0          /* zero = non raggiunto */  
3.  DFS_visit(g,v,color)     /* comincia una DFS da v */
```

```
DFS_visit(g,i,color)    /* g è un grafo, i è un indice, color un array */  
1.  color[i] = 1          /* uno = raggiunto */  
2.  x = g.A[i]  
3.  while x != NULL  
4.      v = x.key          /* v è l'indice di un nodo adiacente ad i */  
5.      if color[v] == 0          /* se v non ancora visitato... */  
6.          DFS_visit(g,v,color)    /* ...continua la visita da v */  
7.      x = x.next          /* passa al prossimo adiacente di i */
```

Visite di un grafo non connesso

- Come nel caso della BFS, per eseguire una DFS di un grafo non necessariamente connesso è sufficiente lanciare diverse visite con lo stesso array color
 - per esempio nel caso di grafo rappresentato con liste/matrice di adiacenza il codice potrebbe essere il seguente

```
DFS_non_connesso(g)          /* g è rappresentato tramite liste di adiacenza */
1.  for i = 0 to g.A.length-1
2.      color[i] = 0           /* zero = non raggiunto */
3.  for i = 0 to g.A.length-1
4.      if color[i] == 0       /* se i non ancora visitato... */
5.          DFS_visit(g,i,color) /* ...comincia una DFS da qui */
```

Complessità della visita DFS

- In una visita in profondità
 - su ogni nodo viene lanciata **DFS_visit** una sola volta
 - ogni arco (adiacenza) è considerata sia dal nodo di partenza che dal nodo di arrivo
- Dunque la complessità è $\Theta(n+m)$

Esercizi sulle visite di grafi

1. Scrivi lo pseudocodice della funzione ALBERO-RICOPRENTE(g, u)

- input: un grafo non orientato g rappresentato da un array $g.A$ di liste di adiacenza e un nodo u
- output: un array $parent$ dove $parent[v]$ è l'indice del nodo da cui è stato raggiunto v in una DFS a partire dal nodo u
 - il $parent[u]$ è convenzionalmente posto uguale a -1

2. Variante:

- produci in output un albero t (connesso e di grado arbitrario) con le seguenti caratteristiche
 - i nodi di t hanno gli stessi indici dei nodi del grafo
 - c'è un arco in t diretto da un nodo x ad un nodo y solo se nella DFS il nodo y è stato raggiunto da x

Esercizi sulle visite di grafi

3. Scrivi lo pseudocodice della procedura $\text{DFS}(g, v)$ nel caso in cui il grafo sia rappresentato da una matrice di adiacenza $g.A$
4. Scrivi lo pseudocodice della procedura $\text{DFS}(g, v)$ che restituisce in output un array `ordine` dove `ordine[v]` è il numero d'ordine con cui il nodo v è stato visitato nella visita in profondità

Esercizi sulle visite di grafi

5. Scrivi la funzione in linguaggio C

```
int COMPONENTI_UGUALI (grafo* g)
```

che prende in input un grafo g rappresentato tramite oggetti e riferimenti e verifica se tutte le componenti connesse hanno lo stesso numero di nodi

6. Scrivi la funzione in linguaggio C

```
int COMPONENTI_BILANCIATE (grafo* g)
```

che prende in input un grafo g rappresentato tramite oggetti e riferimenti e verifica se tutte le componenti connesse hanno tanti nodi quanti archi

1. Alberi in generale

Un albero radicato è un insieme di nodi su cui è definita una relazione binaria “x è figlio di y”. Il numero di figli viene detto grado di un nodo. Una sequenza di nodi tali che uno è il genitore del successivo è detta cammino, il numero degli archi di un cammino è la sua lunghezza. La profondità di un nodo è la lunghezza del cammino dal nodo alla radice. La profondità del nodo più profondo è l'altezza dell'albero. Un albero ordinato è un albero per il quale l'ordine dei figli di ogni nodo è significativo. Un albero binario è un albero ordinato in cui i nodi hanno al massimo grado due. Un albero binario è completo se ogni livello presenta tutti i nodi possibili. Un albero binario ha 2^h foglie, ha $2^h - 1$ nodi interni e $2^{h+1} - 1$ nodi. Per quanto riguarda le visite, sono tre le possibili: visita in preordine: dopo aver processato un nodo si procede a processare i suoi figli, visita in postordine: un nodo può essere processato solo quando i suoi figli sono stati processati, visita simmetrica: si processa prima il figlio sinistro, poi il nodo stesso, poi il figlio destro.

$$h = \log_2 (\text{num Foglie})$$

2. Alberi rosso-neri: obiettivo che si vuole raggiungere e come sono fatti

Gli alberi rosso-neri sono una struttura di dati pensata per mantenere gli alberi binari di ricerca bilanciati. Per semplicità vorremmo che non ci siano nodi con un solo figlio destro o un solo figlio sinistro. Questo viene realizzato tramite un nodo Sentinella detto t.null e viene sostituito a ogni valore NULL di un figlio destro o sinistro di un nodo. Un albero rosso-nero è un albero binario di ricerca nel quale:

1. ogni nodo è rosso o nero
2. la radice t.root e la sentinella t.null sono nere

3. se un nodo è rosso entrambi i suoi figli sono neri
4. tutti i cammini che vanno dalla radice a t.null contengono lo stesso numero di nodi neri

Chiamiamo altezza dell'albero rosso-nero la lunghezza del cammino più lungo tra la radice e t.null (corrisponde all'altezza + 1). Tutti i cammini dalla radice a t.null hanno k nodi neri, il cammino più lungo alterna nodi neri e rossi e ha $2(k-1)$ archi (questo valore corrisponde anche alla profondità massima) e può essere al massimo due volte la lunghezza del cammino più corto. L'altezza di un albero binario appartiene a $\Theta(\log n)$ e di conseguenza anche le operazioni eseguibili su di esso. Le operazioni di inserimento e cancellazione però non garantiscono la conservazione delle proprietà degli alberi rosso-neri. L'operazione che viene utilizzata per ripristinare le proprietà dell'albero rosso-nero è la rotazione ($\Theta(1)$).

- come sono fatti il cammino più corto e il cammino più lungo?

Il cammino più lungo alterna nodi rossi e nodi neri, è composto da $2(k-1)$ archi e può essere al massimo due volte la lunghezza del cammino più corto. ($k = n$, nodi neri)

- supponi che ci siano 4 nodi neri e la sentinella, quanti archi ci sono?

Ci sono quindi 5 nodi neri dato che la sentinella è un nodo nero di conseguenza $2(k-1) = 8$ archi.

- Invece nel cammino più lungo quanti nodi e quanti archi ci sono?

Nel cammino più lungo vi sono $2(k-1)$ archi e $2(k-1)+1$ nodi.

- Conseguenze delle regole di un albero rosso-nero: come giustifichi il fatto che l'altezza è logaritmica?

Considerando che i nodi dell'albero sono almeno quelli del sottoalbero completo e sapendo che la profondità del sotto albero completo $h' = h/2 - 1$ e ricordandosi che un albero completo di altezza x ha $2^{x+1} - 1$ nodi.

$$n \geq 2^{h'+1} - 1 = 2^{\left(\frac{h}{2}-1\right)+1} - 1 = 2^{\frac{h}{2}} - 1$$

$$n+1 \geq 2^{\frac{h}{2}}$$

$$\frac{h}{2} \leq \log(n+1)$$

$$h \leq 2 \log(n+1)$$

Dunque $h \in O(\log n)$.

- Quanto costano inserimento e cancellazione in un albero rosso nero?

Inserimento e Cancellazione in un albero rosso nero costano $\Theta(\log n)$.

- Quando si fanno queste operazioni bisogna fare attenzione?

Quando si fanno queste operazioni bisogna fare molta attenzione perché quest'ultime non garantiscono la conservazione delle proprietà degli alberi rosso-neri.

- Quanto è il massimo che puoi spendere per fixare l'albero rimanendo efficiente?

Il massimo che si può spendere rimanendo efficiente è $\Theta(\log n)$ dato che l'albero è alto come dimostrato precedentemente $\Theta(\log n)$.

- Descrivere l'operazione di fix, in particolare la rotazione

La rotazione è l'operazione di base che viene utilizzata per ripristinare le proprietà dell'albero rosso-nero. Le rotazioni non alterano i colori dei nodi. L'albero rimane un albero binario di ricerca. L'operazione può essere eseguita in tempo $\Theta(1)$.

- Come fai ad usare un albero rosso nero per gestire una coda di priorità? Se ci riesci, che cosa realizzi? Una max heap o una min heap?

Si potrebbe pensare di realizzare un albero rosso-nero per gestire una coda di priorità utilizzando un max-heap e non un min-heap (dato che come primo valore dell'albero ha il valore più piccolo),

infatti bisognerebbe fondere le regole dell'albero rosso-nero al max-heap. La radice sarebbe un nodo con coda di priorità massima.

3. Se volessi fare un algoritmo di ordinamento con un albero rosso nero come potrei fare?
Si potrebbe implementare qualcosa di molto simile al tree sort, utilizzando una visita simmetrica per riordinare l'albero.
4. Se abbiamo un albero binario di ricerca e dobbiamo gestire chiavi uguali come facciamo?

Per gestire chiavi duplicate all'interno di un albero binario di ricerca abbiamo diversi approcci tra cui: Conteggio dei duplicati: ogni nodo dell'albero ha un campo aggiuntivo per tenere traccia del numero di occorrenze della chiave. Quando una chiave duplicata viene inserita, il contatore associato a quella chiave viene incrementato. Lista di occorrenze: Ogni nodo dell'albero mantiene una lista di occorrenze della chiave. Quando una chiave duplicata viene inserita, viene aggiunta all'elenco associato al nodo corrispondente.

5. Qual'è la complessità della ricerca all'interno di un albero binario?
La complessità della ricerca all'interno di un albero binario, logicamente dipende dal tipo di albero binario considerato, nei binari di ricerca nel caso migliore è $O(\log n)$ (dato che l'altezza dell'albero è logaritmica) mentre nel peggiore come tutti gli altri alberi è $O(n)$
6. Cos'è una coda di priorità?
Una coda di priorità (priority queue) è una collezione di elementi, ad ogni elemento è associato un valore di priorità, i valori di priorità definiscono un ordinamento. Solitamente la loro struttura dati più efficiente è l'heap, sostanzialmente è uno speciale array i cui valori sono in rapporto con la loro posizione nell'array, e può essere un max-heap o min-heap.
7. Come implementare un max-heap con un albero

Per implementare un max-heap con un albero, bisogna ragionare su alberi binari quasi completi, ovvero che l'ultimo livello dell'albero potrebbe essere incompleto nella sua parte destra. In un max-heap l'elemento memorizzato nel nodo i ha valore maggiore o uguale degli elementi memorizzati nei suoi figli, infatti la radice contiene il valore più alto dell'array e logicamente il valore dei genitori sarà sempre maggiore del valore del nodo figlio.

- min heap

Il min-heap ha la proprietà simmetrica al max-heap, infatti la radice contiene il valore minore dell'albero/array.

8. Qual'è la complessità asintotica di un inserimento in un heap?

La complessità asintotica di un inserimento in un heap è $\Theta(\log n)$.

9. Perché abbiamo visto prima le code e dopo gli alberi?

Le code sono una struttura di dati meno complessa da capire rispetto agli alberi, difatti la comprensione di una struttura come le code più semplice agevola poi la comprensione di strutture più complicate come gli alberi.

10. Come si fa a dire che un problema abbia una determinata complessità?

Un problema ha una determinata complessità temporale (che può essere $O(f(n))$, $\Omega(f(n))$, $\Theta(f(n))$) se esiste un algoritmo che lo risolve che ha quella determinata complessità temporale, dove ($O(f(n))$, $\Omega(f(n))$, $\Theta(f(n))$) sono le risorse sufficienti a risolvere il problema.

11. Complessità dei problemi:

- cercare un elemento in un array

Per gli array non ordinati la complessità del problema di ricerca nel caso peggiore è $O(n)$, mentre per gli array ordinati la complessità del problema di ricerca nel caso peggiore è $O(\log n)$.

- sommare elementi di un array

La complessità della somma degli elementi in un array è $\Theta(n)$.

- quando sono sufficienti ordine di $f(n)$ risorse per risolvere un problema?

Sono sufficienti ordine di $f(n)$ risorse per risolvere un problema quando l'algoritmo che lo risolve ha complessità temporale $O(f(n))$.

12. Se ho un problema $\Theta(n)$, può esistere un algoritmo che appartiene a $\Theta(n^2)$?

Si può esistere infatti potrebbe semplicemente essere una realizzazione di un algoritmo poco efficiente per risolvere il problema originario.

13. Perché gli algoritmi hanno una determinata complessità?

I fattori che influenzano la complessità degli algoritmi sono vari, tra cui: la struttura dell'algoritmo, il tipo di operazioni eseguite, la struttura dei dati utilizzati, la distribuzione dell'input e la presenza di ottimizzazioni.

14. Selection Sort

Il selection sort utilizza una tecnica greedy (una tecnica che consiste nello scegliere sempre l'alternativa che al momento sembra più appetibile) per ordinare un array. Difatti la strategia del selection sort è quella di selezionare l'elemento più piccolo e posizionarlo al primo posto, poi seleziona l'elemento più piccolo dei rimanenti e lo mette al secondo posto e così via. Il tempo di esecuzione dell'algoritmo è $\Theta(n^2)$. Esso opera in loco, ovvero, non necessita di copiare l'input in strutture di dati diverse da quella utilizzata per l'input. Inoltre è un algoritmo di ordinamento stabile, ovvero non modifica l'ordine degli elementi che hanno lo stesso valore.

- vantaggi di questo algoritmo di ordinamento?

L'utilizzo di questo algoritmo di ordinamento non ha particolari vantaggi in termini di tempo, un vantaggio potrebbe essere quello di essere un algoritmo in grado di lavorare in loco.

- a cosa serve un algoritmo stabile?

Un algoritmo di ordinamento stabile serve a non modificare l'ordine degli elementi che hanno lo stesso valore.

- perchè la complessità è uguale per tutti i casi?

La complessità è uguale per tutti i casi perché con il Selection Sort devo comunque scorrere tutti gli elementi e verificare per ogni elemento se c'è un elemento da scambiare.

15. Insertion Sort

L'Insertion Sort si basa sulla seguente osservazione: la soluzione di un'istanza di dimensione $n-1$ può essere utile per risolvere un'istanza di dimensione n . Infatti l'insertion sort mantiene un sottoinsieme ordinato di elementi (logicamente cominciando da un singolo elemento). A quest'ultimo viene aggiunto un elemento alla volta e i suoi elementi vengono traslati per far posto al nuovo elemento. Quando tutti gli elementi sono inseriti il problema è risolto. Insertion Sort è un algoritmo che opera in loco ed è stabile. Ha complessità $\Theta(n)$ nel caso migliore mentre nel caso medio e peggiore ha complessità $\Theta(n^2)$.

16. Merge Sort

Per poter parlare di Merge Sort bisogna prima introdurre la tecnica "divide et impera": consiste nel suddividere il problema in diversi sottoproblemi. Quest'ultimi sono dello stesso tipo del problema originale ma di dimensioni più piccole, essi vengono risolti in maniera ricorsiva suddividendoli a loro volta, arrivando a dei casi la cui soluzione è banale. Difatti a ciascun passo della ricorsione Divide (l'istanza viene divisa in due o più istanze più piccole), Impera (l'algoritmo viene lanciato sulle istanze più piccole), combina (le soluzioni delle istanze più piccole vengono utilizzare per produrre una soluzione dell'istanza corrente). Il merge sort prende spunto da questa tecnica, infatti divide la sequenza di input in due sottosequenze, le ordina tramite lo stesso merge sort e fonde le due

sottosequenze ordinate, il caso base è un array di un solo elemento che per definizione è ordinato. Il Merge Sort ha complessità $\Theta(n \log n)$

17. Teorema dell'esperto: come è fatta la formula di ricorrenza?

Siano $a, b \geq 1$

Il master theorem considera l'equazione di ricorrenza seguente:

$$T(n) = \begin{cases} \Theta(1) & \text{per } n = 0 \\ a \cdot T(n/b) + O(n^k) & \text{per } n > 0 \end{cases}$$

Il master theorem afferma che tale equazione di ricorrenza ammette le soluzioni seguenti:

1. se $a < b^k$ allora $T(n) = \Theta(n^k)$
2. se $a = b^k$ allora $T(n) = \Theta(n^k \log n)$
3. se $a > b^k$ allora $T(n) = \Theta(n^{\log_b a})$

18. Quick sort

Questo algoritmo opera in loco ma non è stabile, esso viene basato sul paradigma divide et impera. Infatti per ordinare un array, nel quick sort il Divide, divide l'array in due sottoarray non vuoti, in modo che ogni elemento del primo sia minore o uguale $A[q]$ e ogni elemento del secondo sia maggiore di $A[q]$. q è l'indice che viene calcolato dalla procedura di partizionamento e viene detto pivot. Impera: i due sottoarray vengono ordinati ricorsivamente, Combina: i due sottoarray vengono riuniti. La procedura di partizionamento ritorna un valore q che è compreso tra il primo valore dell'array e l'ultimo. Il suo tempo di esecuzione è $\Theta(n)$. Il caso peggiore per Quick Sort è quando partition elegge a pivot il valore massimo o minimo dell'array e ha tempo di esecuzione $\Theta(n^2)$ mentre il caso migliore è quando partition elegge a pivot il valore mediano dell'array e ha tempo di esecuzione $\Theta(n \log n)$

n).

- qual'è lo scopo generale della funzione PARTITION?

Lo scopo generale della funzione PARTITION è quello di ritornare la posizione di un valore detto pivot e posizionare a sinistra di esso tutti i valori minori.

- perchè nel caso peggiore ho un costo $\Theta(n^2)$?

- Si ha

$$T(0) = a$$

$$T(n) = T(n-1) + \Theta(n)$$

- Sappiamo che la soluzione di questa equazione di ricorrenza è

$$T(n) = a + \sum_{k=1}^n g(k)$$

- E dunque

$$T(n) = \sum_{k=1}^n \Theta(k) = \Theta\left(\sum_{k=1}^n k\right) = \Theta(n^2)$$

- vi è un altro modo per poter spiegare la complessità del quick sort?

Si potrebbe considerare una spiegazione simile a quella che utilizziamo nel merge sort se non volessimo utilizzare il teorema dell'esperto. Infatti questi algoritmi di ordinamento dividono sempre di più il nostro sottoarray andando a formare quello che potrebbe sembrare un albero. A ogni livello di profondità vi è sempre lo stesso numero di elementi e la complessità si potrebbe ricondurre all'altezza dell'albero stesso. Di conseguenza avvengono $\log n$ livelli di ricorsione che vanno moltiplicati per il numero di elementi n .

19. bubble sort (non lo abbiamo fatto)

L'algoritmo esegue una serie di iterazioni attraverso l'array, confrontando coppie di elementi adiacenti. Durante ogni iterazione se gli elementi confrontati sono in ordine inverso, vengono scambiati. Questo processo viene ripetuto fino a quando tutti gli elementi sono disposti in ordine. Le iterazioni continuano fino a quando non sono più necessari scambi durante un'intera passata attraverso l'array. La complessità è $\Theta(n^2)$ mentre nel caso migliore è $\Theta(n)$.

20. Che cos'è un algoritmo online?

Un algoritmo online è un algoritmo che non ha bisogno di conoscere l'intero input per iniziare a ordinare gli elementi. Viene detto online proprio perché elaborano i dati in input mentre essi arrivano.

21. Algoritmi in tempo lineare

Per utilizzare un algoritmo in tempo lineare vi sono dei vincoli sull'input molto stringenti, infatti Counting sort e Bucket sort assumono che gli n elementi in input siano interi non negativi minori o uguali a k , con $k \in O(n)$.

22. Counting Sort

La strategia del counting sort si basa sul creare k contatori, uno per ogni possibile valore di i , per ogni i conto quanti sono gli elementi in input uguali ad i , conto quanti sono gli elementi in input minori ad i , scorro l'array di input e posiziono gli elementi nel punto opportuno dell'array di output. È un algoritmo di ordinamento stabile, non è un algoritmo di ordinamento in loco, non è un algoritmo online.

23. Bucket sort

Creo una lista (un "bucket") per ogni i minore o uguale a k , scorro l'array di input e metto l'elemento corrente nel bucket che corrisponde al suo valore, svuoto i bucket ponendo i loro elementi nell'array di output. È un algoritmo di ordinamento stabile, non è un algoritmo di ordinamento in loco, non è un algoritmo online.

24. Radix sort (ordinamento di lista di nomi cognomi e numeri telefonici)

Il radix sort assume che gli elementi da ordinare siano descritti da una sequenza di d cifre (o più generalmente di d colonne). Per poter ordinare in base a più criteri abbiamo bisogno di un algoritmo di ordinamento stabile.

25. Tree sort

Il tree sort è un algoritmo di ordinamento che utilizza un albero binario di ricerca. L'algoritmo esegue una visita

simmetrica, leggendo gli elementi nel modo corretto;
L'algoritmo durante la visita legge gli elementi e li aggiunge a un array. Poichè la visita legge gli elementi in ordine crescente, l'array risulterà ordinato. Albero sbilanciato $O(n^2)$, albero bilanciato $O(n \log n)$.

26. Heap sort

L'heap sort è un algoritmo di ordinamento che utilizza un heap. Gli elementi dell'array vengono considerati come nodi e vengono convertiti in un heap. Questo processo viene chiamato "heapify". Una volta creato l'heap, l'elemento di testa viene estratto e inserito nell'array risultante, questo passaggio viene ripetuto fino a quando l'heap è vuoto. Dopo ogni estrazione è necessario ripristinare la proprietà dell'heap, ciò viene fatto riorganizzando gli elementi rimanenti nell'heap in modo che mantengano la struttura di heap. $O(n \log n)$.

27. negli algoritmi di ordinamento si fa uso di confronti, e la complessità è $\Theta(n^2)$, conosciamo un lower bound per gli algoritmi di ordinamento basati su confronti che è $\Omega(n \log n)$

28. algoritmi di ricerca che fanno uso di confronti, qual'è il limite inferiore alla complessità della ricerca? ci può essere una complessità in un tempo minore di $\log n$?
Il limite inferiore per gli algoritmi di ricerca che fanno uso di confronti è $\Omega(\log n)$, non vi può essere una complessità in un tempo minore e questo è dimostrabile con un albero di decisione. Possiamo definire un albero i cui nodi interni sono i vari confronti eseguiti dall'algoritmo e le cui foglie sono le possibili risposte. Questo albero è un albero binario con n foglie: l'altezza dell'albero è $\Omega(\log n)$, il numero di confronti necessari per individuare una foglia è $\Omega(\log n)$.

29. Tabelle hash

Prima di parlare di tabelle hash è necessario introdurre il concetto di Array associativi:

Gli array associativi sono tipi astratti di dato costituiti da

coppie <chiave, valore>, le operazioni che vogliamo fare sono l'inserimento di una nuova coppia, la cancellazione di una specifica chiave e la ricerca del valore corrispondente ad una chiave. Le tabelle Hash realizzano array associativi con alcune caratteristiche: gli algoritmi di ricerca, inserimento e cancellazione non sono basati su confronti, le ricerche, gli inserimenti e le cancellazioni avvengono con un tempo $\Theta(n)$ nel caso peggiore, le ricerche, gli inserimenti e le cancellazioni avvengono con un tempo $\Theta(1)$ nel caso medio, la struttura di dati utilizzata è effettivamente un singolo array. Vi sono però due difficoltà principali nella realizzazione: la tipologia delle chiavi (le chiavi non sono necessariamente degli interi) e la numerosità delle possibili chiavi. Il numero di chiavi effettivamente utilizzate dal programma in esecuzione è molto minore del numero delle chiavi possibili (universo delle chiavi k). Per realizzare delle tabelle Hash utilizzo un array T per memorizzare i dati associati. La dimensione m dell'array T è molto minore della dimensione dell'universo k ma molto vicina al numero di chiavi effettivamente utilizzate. Definisco poi una funzione hash h che trasforma le chiavi di k negli interi nel range $[0..m-1]$. La funzione hash quindi si occupa di definire una corrispondenza tra l'universo K delle chiavi e gli indici della tabella hash. Con le funzioni hash avendo un codominio (gli m indici di T) molto più piccolo del dominio (tutti gli elementi di k) è inevitabile che si generino delle collisioni. Per questo motivo vengono gestite con liste di trabocco, dove ogni posizione di T è un riferimento al primo elemento di una lista detta "di trabocco". Ogni nodo della lista è un oggetto con tre campi: Key (valore della chiave), info (valore associato alla chiave) e next (riferimento al prossimo nodo).

30. Funzioni hash

-Metodo della divisione: `mod_hash`

È un metodo molto veloce, lavora con chiavi pseudocasuali in modo tale da riportare le chiavi nell'intervallo $[0-m-1]$;

-Metodo della moltiplicazione

Questa funzione hash è deterministica, può essere calcolata in $\Theta(1)$. Sia irr un numero irrazionale, date delle chiavi intere qualsiasi le cifre decimali dopo la virgola del prodotto $k * irr$ si possono assumere uniformemente distribuite nell'intervallo $(0,1)$

31. perchè si chiamano tabelle hash?

Vengono dette tabelle hash perché per permetterne l'utilizzo è necessario utilizzare delle funzioni hash che mettono in relazione l'universo delle chiavi k e gli indici della tabella hash.

32. Come funziona la ricerca nelle tabelle hash?

La ricerca nelle tabelle hash consiste nel cercare l'opportuna coppia chiave valore avviene in un tempo $\Theta(1)$ nel caso medio e $\Theta(n)$ nel caso peggiore.

- perchè non si possono evitare le collisioni? può essere sicuramente inettiva una funzione?

Le collisioni non si possono evitare perché la funzione hash ha un codominio (gli indici m di T) molto più piccolo del dominio (tutti gli elementi di K)

- proprietà della funzione hash?

La funzione Hash ha varie proprietà, tra cui: è deterministica (data una chiave k , dà sempre lo stesso risultato $HASH(k)$), è calcolabile in tempo costante, distribuisce le chiavi utilizzate in esecuzione in maniera pseudocasuale nell'intervallo $[0 \dots m-1]$.

- fattore di carico, quando cresce troppo provvedo ad allargare la tabella hash

Si definisce fattore di carico il rapporto α tra il numero n di elementi memorizzati e il numero m di posizioni disponibili. α è il numero medio di elementi memorizzati in ogni lista concatenata. A seconda del valore di α abbiamo:
 $\alpha < 1$ molte posizioni disponibili rispetto agli elementi

memorizzati

$\alpha=1$ il numero di elementi corrisponde al numero delle posizioni disponibili

$\alpha>1$ molti elementi da memorizzare rispetto al numero delle posizioni disponibili

Quando il fattore di carico α supera una determinata soglia la dimensione m della tabella viene raddoppiata.

- il fattore di carico dopo che faccio il raddoppio della tabella hash cosa fa?

Una volta che raddoppio la tabella hash il fattore di carico si dimezza.

- crescita telescopica di una tabella hash

La crescita telescopica di una tabella hash è molto importante dato che una realizzazione senza la gestione telescopica della tabella non potrebbe garantire tempi costanti di accesso.

- quando sciupa troppo la funzione hash?

La funzione hash si rovina quando la ricerca di un elemento ci mette più tempo degli elementi della tabella stessa, difatti la ricerca deve essere costante.

- se metto troppi elementi dentro la tabella cosa succede?

Se metto troppi elementi all'interno della tabella hash, avrei delle liste di trabocco troppo lunghe e perderei i vantaggi per cui uso le tabelle hash.

- se non si fa la crescita telescopica che mi tiene il fattore di carico, il tempo di ricerca

Senza la crescita telescopica la tabella diventerebbe più densa e aumenterebbero le probabilità di collisioni che porterebbe alla diminuzione di prestazioni della tabella hash.

Aumenterebbe il fattore di carico che complicherebbe la ricerca di posizioni vuote per l'inserimento di nuovi elementi. Le liste concatenate diventerebbero più lunghe e lunghe catene comporterebbero a un aumento nel tempo di ricerca, in quanto è necessario attraversare più elementi per trovare la chiave desiderata.

33. Rapporto tra logaritmi e costanti

Il rapporto tra logaritmi di diversa base può essere espresso

utilizzando il cambiamento di base dei logaritmi. Se hai due logaritmi di una stessa quantità x con basi diverse a e b , puoi esprimere il rapporto come segue: